Java
для опытных разработчиков

ВТОРОЕ ИЗДАНИЕ

Бенджамин Эванс
Джейсон Кларк
Мартин Фербург

Предисловие Хайнца Кабуца



MANNING

The Well-Grounded Java Developer

SECOND EDITION

BENJAMIN J. EVANS, JASON CLARK,
AND MARTIJN VERBURG



MANNING
SHELTER ISLAND

https://t.me/javalib

Java для опытных разработчиков

Второе издание

Бенджамин Эванс, Джейсон Кларк,
Мартин Фербург

https://t.me/javalib

Язык Java — сердце корпоративного ПО. Если программист хорошо знает Java, он без особого труда найдет интересную работу. Поднимите ваши навыки Java на новый уровень вместе с экспертами. Основательно изучите такие мощные инструменты, как модели конкурентного выполнения и модули, и даже раскройте некоторые глубинные тайны Java.

Узнайте, как Java работает на уровне байт-кода. Освойте ценные приемы конкурентного выполнения и оптимизации быстродействия, а еще ключевые методы сборки, тестирования и развертывания. Также рассмотрите альтернативные языки для JVM — Kotlin и Clojure. Изучив материал, вы будете выделяться на фоне других разработчиков!

https://t.me/javalib

Краткое содержание

Предисловие 22

Введение 24

Благодарности 27

Об этой книге 30

Об авторах 34

Иллюстрация на обложке 35

От издательства 36

Часть I
От 8 до 11 и дальше

Глава 1. Современный язык Java 38

Глава 2. Модули Java 64

Глава 3. Java 17 97

Часть II
Что там внутри?

Глава 4. Файлы классов и байт-код 124

Глава 5. Основы конкурентности в Java 166

Глава 6. Конкурентные библиотеки Java 221

Глава 7. Производительность кода на Java 262

https://t.me/javalib

Часть III
Другие языки на JVM

Глава 8. Альтернативные языки на JVM 308

Глава 9. Kotlin 329

Глава 10. Clojure: другой взгляд на программирование 360

Часть IV
Сборка и развертывание

Глава 11. Сборка в Gradle и Maven 408

Глава 12. Выполнение Java в контейнерах 469

Глава 13. Основы тестирования 509

Глава 14. Тестирование за пределами JUnit 540

Часть V
Передовые рубежи Java

Глава 15. Продвинутое функциональное программирование 570

Глава 16. Продвинутое конкурентное программирование 612

Глава 17. Внутреннее устройство современной JVM 649

Глава 18. Будущее Java 690

Приложения

Приложение А. Выбор версии Java 722

Приложение Б. Потоки данных в Java 8 725

https://t.me/javalib

Оглавление

Отзывы о первом издании ... 21

Предисловие ... 22

Введение ... 24

Благодарности ... 27

Об этой книге ... 30

Для кого эта книга ... 30
Как использовать эту книгу ... 31
О коде ... 32
Форум liveBook ... 33
Другие сетевые ресурсы ... 33

Об авторах ... 34

Иллюстрация на обложке ... 35

От издательства ... 36

Часть I
От 8 до 11 и дальше

Глава 1. Современный язык Java ... 38

1.1. Язык и платформа ... 39
1.2. Новый цикл выпуска Java ... 41
1.3. Расширенное выведение типов (синтаксис var) ... 45

https://t.me/javalib

1.4. Изменения языка и платформы ... 49
1.4.1. Синтаксический сахар ... 50
1.4.2. Изменения языка ... 50
1.4.3. Документы JSR и JEP ... 51
1.4.4. Инкубационная и предварительная функциональность ... 52
1.5. Небольшие изменения в Java 11 ... 53
1.5.1. Фабрики коллекций (JEP 213) ... 53
1.5.2. Удаление корпоративных модулей (JEP 320) ... 55
1.5.3. HTTP/2 (Java 11) ... 56
1.5.4. Однофайловое выполнение (JEP 330) ... 61
Итоги ... 63

Глава 2. Модули Java ... 64

2.1. Модули Java: общая картина ... 65
2.1.1. Проект Jigsaw ... 66
2.1.2. Граф модулей ... 69
2.1.3. Защита внутреннего устройства ... 71
2.1.4. Новая семантика управления доступом ... 72
2.2. Основы синтаксиса модулей ... 73
2.2.1. exports и requires ... 75
2.2.2. Транзитивность ... 76
2.3. Загрузка модулей ... 77
2.3.1. Платформенные модули ... 78
2.3.2. Прикладные модули ... 78
2.3.3. Автоматические модули ... 79
2.3.4. Анонимные модули ... 79
2.4. Создание первого модульного приложения ... 80
2.4.1. Параметры командной строки для модулей ... 81
2.4.2. Выполнение модульного приложения ... 83
2.4.3. Модули и рефлексия ... 85
2.5. Проектирование архитектуры для модулей ... 86
2.5.1. Расщепленные пакеты ... 87
2.5.2. Компактные профили Java 8 ... 88
2.5.3. Многоверсионные файлы JAR ... 90
2.6. Не только модули ... 93
Итоги ... 95

https://t.me/javalib

Глава 3. Java 17 97
3.1. Текстовые блоки 97
3.2. Выражения switch 99
3.3. Записи 103
3.3.1. Номинативная типизация 109
3.3.2. Компактные конструкторы записей 110
3.4. Запечатанные типы 113
3.5. Новая форма instanceof 118
3.6. Сопоставление с шаблонами и предварительная функциональность 119
Итоги 122

Часть II
Что там внутри?

Глава 4. Файлы классов и байт-код 124
4.1. Загрузка классов и объекты классов 125
4.1.1. Загрузка и компоновка 126
4.1.2. Объекты Class 129
4.2. Загрузчики классов 130
4.2.1. Нестандартная загрузка классов 132
4.2.2. Модули и загрузка классов 140
4.3. Анализ файлов классов 140
4.3.1. Введение в javap 141
4.3.2. Внутренняя форма сигнатур методов 141
4.3.3. Пул констант 143
4.4. Байт-код 146
4.4.1. Дизассемблирование классов 146
4.4.2. Среда выполнения 149
4.4.3. Введение в коды операций 150
4.4.4. Операции загрузки и сохранения 152
4.4.5. Арифметические операции 153
4.4.6. Операции управления порядком выполнения 154
4.4.7. Операции вызова 155
4.4.8. Платформенные операции 158
4.4.9. Сокращенные формы кодов операций 159

https://t.me/javalib

4.5. Рефлексия ... 159
4.5.1. Введение в рефлексию ... 160
4.5.2. Загрузка классов в сочетании с рефлексией ... 163
4.5.3. Проблемы с рефлексией ... 163
Итоги ... 165

Глава 5. Основы конкурентности в Java ... 166
5.1. Краткий курс конкурентности ... 167
5.1.1. Но я уже знаю о Thread ... 167
5.1.2. Оборудование ... 168
5.1.3. Закон Амдала ... 169
5.1.4. Потоковая модель Java ... 170
5.1.5. Полезный опыт ... 171
5.2. Факторы проектирования ... 172
5.2.1. Безопасность и конкурентная типобезопасность ... 172
5.2.2. Живость ... 173
5.2.3. Производительность ... 174
5.2.4. Повторное использование ... 174
5.2.5. Как и почему возникают конфликты между факторами? ... 174
5.2.6. Источники затрат ... 175
5.3. Конкурентное выполнение с блоковой структурой (до Java 5) ... 176
5.3.1. Синхронизация и блокировки ... 176
5.3.2. Модель состояний потока ... 178
5.3.3. Полностью синхронизированные объекты ... 180
5.3.4. Взаимная блокировка ... 181
5.3.5. Почему synchronized? ... 184
5.3.6. Ключевое слово volatile ... 186
5.3.7. Состояния и методы потока ... 187
5.3.8. Неизменяемость ... 193
5.4. Модель памяти Java (JMM) ... 197
5.5. Конкурентность в байт-коде ... 200
5.5.1. Потерянное обновление ... 201
5.5.2. Синхронизация в байт-коде ... 204
5.5.3. Синхронизированные методы ... 208
5.5.4. Несинхронизированное чтение ... 209

https://t.me/javalib

5.5.5. Снова о взаимной блокировке ... 211
5.5.6. Снова об устранении взаимных блокировок ... 213
5.5.7. Обращения к volatile ... 218
Итоги ... 220

Глава 6. Конкурентные библиотеки Java ... 221
6.1. Структурные элементы современных конкурентных приложений ... 221
6.2. Атомарные классы ... 222
6.3. Классы Lock ... 224
6.3.1. Объекты условий ... 226
6.4. CountDownLatch ... 226
6.5. ConcurrentHashMap ... 228
6.5.1. Упрощенное представление HashMap ... 228
6.5.2. Ограничения Dictionary ... 232
6.5.3. Конкурентная реализация Dictionary ... 232
6.5.4. ConcurrentHashMap ... 235
6.6. CopyOnWriteArrayList ... 238
6.7. Блокирующие очереди ... 241
6.7.1. Интерфейсы BlockingQueue ... 247
6.7.2. WorkUnit ... 249
6.8. Преднамеченные объекты ... 250
6.8.1. CompletableFuture ... 252
6.9. Задачи и выполнение ... 254
6.9.1. Моделирование задач ... 255
6.9.2. Исполнители ... 256
6.9.3. Однопоточный исполнитель ... 257
6.9.4. Пул фиксированного размера ... 258
6.9.5. Пул потоков с кэшированием ... 259
6.9.6. ScheduledThreadPoolExecutor ... 260
Итоги ... 261

Глава 7. Производительность кода на Java ... 262
7.1. Терминология производительности: основные определения ... 265
7.1.1. Задержка ... 265
7.1.2. Пропускная способность ... 265

https://t.me/javalib

7.1.3. Загруженность ... 266
7.1.4. Эффективность ... 266
7.1.5. Емкость ... 267
7.1.6. Масштабируемость ... 267
7.1.7. Деградация ... 267
7.2. Прагматичный подход к анализу производительности ... 268
7.2.1. Понимайте, что измеряете ... 268
7.2.2. Как выполнять измерения ... 269
7.2.3. Установите цели по быстродействию ... 271
7.2.4. Знайте, когда остановиться ... 271
7.2.5. Учитывайте цену повышения быстродействия ... 272
7.2.6. Преждевременная оптимизация опасна ... 272
7.3. Что пошло не так? Почему мы должны этим заниматься? ... 274
7.3.1. Закон Мура ... 274
7.3.2. Иерархия задержек памяти ... 276
7.4. Почему так сложно оптимизировать производительность кода на Java? ... 278
7.4.1. Роль времени в оптимизации производительности ... 279
7.4.2. Промахи кэша ... 280
7.5. Сборка мусора ... 282
7.5.1. Основы ... 282
7.5.2. Пометка и очистка ... 283
7.5.3. Области памяти ... 285
7.5.4. Сборка мусора в молодом поколении ... 286
7.5.5. Полная сборка мусора ... 286
7.5.6. Безопасные точки ... 286
7.5.7. G1: сборщик мусора по умолчанию в Java ... 287
7.5.8. Сборщик мусора Parallel ... 289
7.5.9. Параметры конфигурации сборщика мусора ... 290
7.6. JIT-компиляция с HotSpot ... 292
7.6.1. Зачем нужна динамическая компиляция? ... 293
7.6.2. Введение в HotSpot ... 293
7.6.3. Встраивание методов ... 295
7.6.4. Динамическая компиляция и мономорфные вызовы ... 295

https://t.me/javalib

7.6.5. Чтение журналов компиляции ... 296
7.6.6. Деоптимизация ... 297
7.7. JDK Flight Recorder ... 298
7.7.1. Flight Recorder ... 298
7.7.2. Mission Control ... 300
Итоги ... 306

Часть III
Другие языки на JVM

Глава 8. Альтернативные языки на JVM ... 308
8.1. Классификация языков ... 309
8.1.1. Интерпретируемые и компилируемые языки ... 309
8.1.2. Динамическая и статическая типизация ... 310
8.1.3. Императивные и функциональные языки ... 311
8.1.4. Оригинал и повторная реализация ... 312
8.2. Многоязычное программирование на JVM ... 314
8.2.1. Зачем нужны другие языки? ... 315
8.2.2. Перспективные языки ... 317
8.2.3. Языки, которые мы могли выбрать (но не выбрали) ... 318
8.3. Как выбрать альтернативный язык для проекта ... 320
8.3.1. Является ли область проекта низкорискованной? ... 321
8.3.2. Насколько просто язык взаимодействует с Java? ... 322
8.3.3. Насколько развита инструментальная и тестовая поддержка для языка? ... 323
8.3.4. Легко ли изучить язык? ... 323
8.3.5. Насколько популярен этот язык среди разработчиков? ... 324
8.4. Как JVM поддерживает альтернативные языки ... 324
8.4.1. Производительность ... 325
8.4.2. Среды выполнения для альтернативных языков ... 326
8.4.3. Фикции компилятора ... 326
Итоги ... 328

Глава 9. Kotlin ... 329
9.1. Почему стоит использовать Kotlin ... 330
9.1.1. Установка ... 330

https://t.me/javalib

9.2. Удобство и компактность 330
9.2.1. Начиная с меньшего 331
9.2.2. Переменные 331
9.2.3. Проверка равенства 332
9.2.4. Функции 333
9.2.5. Коллекции 336
9.2.6. Выражения 339
9.3. Новый взгляд на классы и объекты 341
9.3.1. Классы данных 347
9.4. Безопасность 348
9.4.1. Null-безопасность 348
9.4.2. Умное приведение типа 350
9.5. Конкурентность 352
9.6. Совместимость с Java 355
Итоги 359

Глава 10. Clojure: другой взгляд на программирование 360
10.1. Введение в Clojure 361
10.1.1. Программа «Hello World» на Clojure 363
10.1.2. Основы работы с REPL 365
10.1.3. Ошибки 367
10.1.4. Привыкаем к круглым скобкам 370
10.2. Clojure: синтаксис и семантика 371
10.2.1. Краткий курс специальных форм 371
10.2.2. Списки, векторы, отображения и множества 374
10.2.3. Арифметические действия, проверка равенства и другие операции 378
10.2.4. Работа с функциями в Clojure 379
10.2.5. Циклы в Clojure 383
10.2.6. Макросы считывателя и диспетчеризация 384
10.3. Функциональное программирование и замыкания 386
10.4. Знакомство с последовательностями Clojure 388
10.4.1. Последовательности и функции переменной арности 393
10.5. Взаимодействие между Clojure и Java 394
10.5.1. Вызов Java из Clojure 395

https://t.me/javalib

10.5.2. Как устроены вызовы Clojure ... 396
10.5.3. Тип Java для значений Clojure ... 396
10.5.4. Заместители в Clojure ... 397
10.5.5. Исследовательское программирование в REPL ... 398
10.5.6. Использование Clojure из Java ... 399
10.6. Макросы ... 400
Итоги ... 406

Часть IV
Сборка и развертывание

Глава 11. Сборка в Gradle и Maven ... 408
11.1. Почему средства сборки важны для опытного разработчика ... 408
11.1.1. Автоматизация рутинных операций ... 409
11.1.2. Управление зависимостями ... 409
11.1.3. Согласование усилий разработчиков ... 413
11.2. Maven ... 414
11.2.1. Жизненный цикл сборки ... 414
11.2.2. Команды и знакомство с POM ... 415
11.2.3. Сборка ... 417
11.2.4. Управление манифестом ... 418
11.2.5. Добавление нового языка ... 419
11.2.6. Тестирование ... 422
11.2.7. Управление зависимостями ... 425
11.2.8. Анализ покрытия ... 429
11.2.9. За пределами Java 8 ... 430
11.2.10. Многоверсионные файлы JAR в Maven ... 431
11.2.11. Maven и модули ... 435
11.2.12. Разработка плагинов для Maven ... 438
11.3. Gradle ... 441
11.3.1. Установка Gradle ... 442
11.3.2. Задачи ... 443
11.3.3. Сценарии сборки ... 444
11.3.4. Использование плагинов ... 445
11.3.5. Сборка ... 445
11.3.6. Эффективность ... 448

https://t.me/javalib

11.3.7. Зависимости в Gradle ... 449
11.3.8. Добавление кода на Kotlin ... 454
11.3.9. Тестирование ... 455
11.3.10. Автоматизация статического анализа ... 456
11.3.11. За пределами Java 8 ... 457
11.3.12. Gradle и модули ... 458
11.3.13. Настройка Gradle ... 465
Итоги ... 467

Глава 12. Выполнение Java в контейнерах ... 469

12.1. Почему контейнеры важны для опытных разработчиков ... 470
12.1.1. Управляющие операционные системы, виртуальные машины и контейнеры ... 470
12.1.2. Преимущества контейнеров ... 473
12.1.3. Недостатки контейнеров ... 474
12.2. Основы работы с Docker ... 475
12.2.1. Построение образов Docker ... 475
12.2.2. Запуск контейнеров Docker ... 478
12.3. Разработка приложений на Java с Docker ... 480
12.3.1. Выбор базового образа ... 481
12.3.2. Сборка образа в Gradle ... 482
12.3.3. Сборка в Docker ... 483
12.3.4. Порты и узлы ... 486
12.3.5. Локальная разработка с Docker Compose ... 488
12.3.6. Отладка в Docker ... 491
12.3.7. Протоколирование в Docker ... 493
12.4. Kubernetes ... 495
12.5. Наблюдаемость и производительность ... 503
12.5.1. Наблюдаемость ... 503
12.5.2. Производительность в контейнерах ... 506
Итоги ... 507

Глава 13. Основы тестирования ... 509

13.1. Зачем нужно тестирование ... 509
13.2. Как проводить тестирование ... 510
13.3. Разработка через тестирование ... 514

https://t.me/javalib

13.3.1. TDD в двух словах ... 516
13.3.2. Пример TDD с одним сценарием использования ... 516
13.4. Тестовые дублеры ... 522
13.4.1. Пустышка ... 523
13.4.2. Заглушка ... 525
13.4.3. Суррогаты ... 527
13.4.4. Макеты ... 530
13.4.5. Проблемы с макетами ... 531
13.5. От JUnit 4 к 5 ... 533
Итоги ... 539

Глава 14. Тестирование за пределами JUnit ... 540
14.1. Интеграционное тестирование с Testcontainers ... 540
14.1.1. Установка testcontainers ... 541
14.1.2. Пример с Redis ... 541
14.1.3. Получение логов контейнеров ... 545
14.1.4. Пример с Postgres ... 546
14.1.5. Пример сквозного тестирования с Selenium ... 548
14.2. Спецификационное тестирование в Spek и Kotlin ... 550
14.3. Тестирование на основе свойств в Clojure ... 556
14.3.1. clojure.test ... 556
14.3.2. clojure.spec ... 558
14.3.3. test.check ... 561
14.3.4. clojure.spec и test.check ... 567
Итоги ... 568

Часть V
Передовые рубежи Java

Глава 15. Продвинутое функциональное программирование ... 570
15.1. Введение в понятия функционального программирования ... 571
15.1.1. Чистые функции ... 571
15.1.2. Неизменяемость ... 571
15.1.3. Функции высшего порядка ... 572
15.1.4. Рекурсия ... 573
15.1.5. Замыкания ... 574

https://t.me/javalib

15.1.6. Отложенные вычисления 574
15.1.7. Каррирование и частичное применение 575
15.2. Ограничения функционального программирования на Java 576
15.2.1. Чистые функции 576
15.2.2. Изменяемость 577
15.2.3. Функции высшего порядка 579
15.2.4. Рекурсия 580
15.2.5. Замыкания 583
15.2.6. Отложенные вычисления 585
15.2.7. Каррирование и частичное применение 587
15.2.8. Система типов Java и коллекции 588
15.3. Функциональное программирование на Kotlin 590
15.3.1. Чистые функции и функции высшего порядка 590
15.3.2. Замыкания 591
15.3.3. Каррирование и частичное применение 592
15.3.4. Неизменяемость 593
15.3.5. Хвостовая рекурсия 596
15.3.6. Отложенное вычисление 600
15.3.7. Последовательности 601
15.4. Функциональное программирование на Clojure 606
15.4.1. Включения 606
15.4.2. Отложенные последовательности 607
15.4.3. Каррирование в Clojure 610
Итоги 611

Глава 16. Продвинутое конкурентное программирование 612
16.1. Фреймворк Fork/Join 613
16.1.1. Простой пример F/J 614
16.1.2. Параллелизация задач для F/J 617
16.1.3. Алгоритмы перехвата работы 618
16.2. Конкурентность и функциональное программирование 620
16.2.1. Снова о CompletableFuture 620
16.2.2. Параллельные потоки данных 624
16.3. Как работают корутины Kotlin 626
16.3.1. Как работают корутины 626
16.3.2. Области видимости и диспетчеризация корутин 631

https://t.me/javalib

16.4. Конкурентное выполнение в Clojure 633
16.4.1. Перманентные структуры данных 634
16.4.2. Преднамеченные объекты и pcalls 641
16.4.3. Программная транзакционная память 643
16.4.4. Агенты 647
Итоги 648

Глава 17. Внутреннее устройство современной JVM 649
17.1. Знакомство с внутренним устройством JVM: вызов методов 650
17.1.1. Вызов виртуальных методов 650
17.1.2. Вызов методов интерфейсов 653
17.1.3. Вызов «специальных» методов 655
17.1.4. Конечные методы 656
17.2. Внутренняя реализация рефлексии 657
17.3. Дескрипторы методов 661
17.3.1. MethodHandle 662
17.3.2. MethodType 663
17.3.3. Как получать дескрипторы методов 664
17.3.4. Рефлексия, заместители и дескрипторы методов 665
17.4. invokedynamic 668
17.4.1. Реализация лямбда-выражений 670
17.5. Небольшие внутренние изменения 673
17.5.1. Конкатенация строк 673
17.5.2. Компактные строки 675
17.5.3. Общие контексты 677
17.6. Unsafe 680
17.7. Замена Unsafe поддерживаемыми API 684
17.7.1. VarHandle 685
17.7.2. Скрытые классы 687
Итоги 689

Глава 18. Будущее Java 690
18.1. Проект Amber 691
18.2. Проект Panama 693
18.2.1. API внешних функций и памяти 693

https://t.me/javalib

18.3. Проект Loom 700
18.3.1. Виртуальные потоки 704
18.3.2. Строители потоков 705
18.3.3. Программирование с помощью виртуальных потоков 706
18.3.4. Когда проект Loom станет доступным? 708
18.4. Проект Valhalla 709
18.4.1. Изменение модели языка 713
18.4. Последствия внедрения объектов-значений 714
18.4.3. Снова об обобщениях 717
18.5. Java 18 718
Итоги 719

Приложения

Приложение А. Выбор версии Java 722
А.1. Java остается бесплатным 722
А.1.1. Java SE / OpenJDK / Сборки Oracle OpenJDK / Oracle JDK 722
А.2. Для тех, кто остается на Java SE 8 723
А.2.1. Бесплатное использование Java SE 8 723
А.3. Как получить Java SE 11 723
А.3.1. Бесплатное использование Java SE 11 724
А.4. Как получить Java SE 17 (LTS) 724
А.4.1. Бесплатное использование Java SE 17 724
А.5. Платная поддержка 724

Приложение Б. Потоки данных в Java 8 725
Б.1. Обратная совместимость 725
Б.2. Методы по умолчанию 726
Б.3. Потоки данных 727
Б.3.1. Пример 728
Б.4. Ограничения коллекций 731
Б.5. Бесконечные потоки данных 732
Б.6. Работа с примитивами 732
Б.7. Параллельные операции? 733

https://t.me/javalib

ОТЗЫВЫ О ПЕРВОМ ИЗДАНИИ

Вы побываете на передовых рубежах Java-разработки и научитесь говорить на Java 7 и языках следующего поколения.

Пол Бенедикт, Corporate Personnel & Associates

Купите эту книгу, чтобы узнать о новых возможностях Java 7. Обращайтесь к ней, чтобы овладеть Java на экспертном уровне.

Стивен Харрисон, PhD, FirstFuel Software

Настоящий кладезь знаний о платформе JVM.

Рик Вагнер, Red Hat

Как стать опытным Java-разработчиком и как им остаться.

Хайнц Кабуц, The Java Specialists' Newsletter

https://t.me/javalib

Предисловие

В словарях прилагательное «основательный» определяется как «дельный, серьезный, солидный, обстоятельный». Мне нравится это слово. Java-разработчику стоит серьезно и обстоятельно овладевать практическими знаниями, которые необходимы, чтобы называть себя экспертом по Java.

Перед вами второе издание превосходной книги. Первое издание научило нас всему, что нужно было знать о Java 7. Кажется, с тех пор прошла целая вечность. Версия Java 7 относилась к другой эпохе, когда новые возможности добавлялись в язык в лучшем случае раз в три года. Тогда было легко различать версии. Java 5 — обобщения и перечисления. Java 7 — `try` с ресурсами. Java 8 — потоки данных и лямбда-выражения. Эти счастливые времена закончились, когда компания Oracle перешла на шестимесячный цикл. Когда появились записи — в Java 14, 15 или 17? А расширенный `switch`? Или он уже был в Java 11?

Быстрый цикл выпуска хорошо подходит для программистов в динамично развивающихся компаниях. Каждые полгода у них появляются новые игрушки. Они даже могут испытывать предварительную реализацию того, что появится в следующей версии. Обилие новшеств хорошо для программистов, но не для авторов книг. Не успеют высохнуть чернила, как в языке появляются новые возможности, из-за которых часть материала устаревает.

Бен, Джейсон и Мартин отлично поработали над новой книгой о Java. Ее основная идея остается прежней, и я бы сформулировал ее так: «Если вы хотите нанять профессионального программиста на Java, каких знаний и умений от него стоит ожидать? Какими навыками он должен обладать, чтобы его подготовку можно было считать по-настоящему основательной?»

Новое издание книги настолько актуально, насколько это возможно при шестимесячном графике выпуска новых версий языка. В то же время авторы не перегружают нас новым материалом. Суровая реальность такова, что большинство организаций до сих пор не расстались со старыми версиями Java. Даже после выхода Java 18 многие банки, страховые компании и правительственные учреждения все еще работают на Java 8.

https://t.me/javalib

Книга почти на 200 страниц толще предыдущего издания. В некоторых разделах содержится абсолютно новый материал. Это один из тех случаев, когда с выходом нового издания старое не устаревает полностью. Обе книги заслуживают того, чтобы стоять на полке серьезного Java-программиста.

Бенджамин Эванс, Джейсон Кларк и Мартин Фербург — настоящие эксперты в области Java. Они занимают должности сеньор-разработчиков на Java в Red Hat, New Relic и Microsoft. Возьмите на вооружение их коллективную мудрость. Эта книга поможет вам выявить свои слабые места, которые стоит подтянуть. В итоге, приложив достаточно усилий, вы сможете с полным правом называть себя основательным Java-программистом.

Хайнц Кабуц

The Java Specialists' Newsletter

https://t.me/javalib

Введение

Первое издание этой книги началось с комплекта учебных заметок для начинающих разработчиков в валютном отделе банка. Один из нас (Бен), знакомясь с ассортиментом книг в продаже, обнаружил, что на рынке не хватает современных материалов для Java-разработчиков, которые хотят повысить свой профессиональный уровень. Взявшись составлять такие материалы, он обнаружил, что пишет ту самую недостающую книгу, и привлек себе в помощь Мартина.

Это произошло более 10 лет назад: книга писалась, когда Java 7 еще находился в процессе разработки. С тех пор мир сильно изменился, и поэтому книга тоже существенно переработана по сравнению с первым изданием. Изначально мы собирались изложить такие темы:

- Многоязычное программирование.
- Внедрение зависимостей.
- Многопоточное программирование.
- Проверенные методы сборки и непрерывной интеграции.
- Что нового появилось в Java 7.

Однако когда мы начали работать над вторым изданием, оказалось, что в материал нужно вносить изменения, например:

- Немного сократить тему многоязычного программирования.
- Усилить акцент на функциональном программировании.
- Доработать материал о многопоточности.
- Рассмотреть под новым углом тему сборки и развертывания (включая контейнеры).
- Представить новые возможности Java 11 и 17.

Очень важное изменение связано с тем, что в первом издании помимо Java рассматривались три других языка, одним из которых был Scala (а двумя

https://t.me/javalib

другими — Groovy и Clojure; Kotlin тогда еще не существовал). В то время многие разработчики рассматривали Scala в качестве языка, который «как Java, только лучше» — и по сути, в первом издании мы представили именно такое ви́дение.

Однако с тех пор мир не стоял на месте. Java 8 и 11 получили широкое распространение, а те, кому нужен «Java, только лучше», в основном пишут на Kotlin (или просто остаются на Java). За это время Scala стал мощным функционально-ориентированным языком со статической типизацией для JVM. Он отлично подходит для тех, кому нужно именно такое сочетание возможностей, но за это приходится расплачиваться: в частности, со временем среда выполнения становится все сложнее, а язык все меньше похож на Java.

Эту эволюцию иногда описывают фразой «Scala хочет стать Haskell на JVM», хотя эта формулировка не вполне точна. Потому после того, как мы решили исключить Groovy из второго издания, пришлось долго и серьезно размышлять над тем, стоит ли оставить Scala или заменить его на Kotlin.

В конечном итоге мы пришли к выводу, что Scala идет в своем, функционально-ориентированном направлении, а мы хотели бы представить язык, более доступный для Java-разработчиков, которые начинают осваивать другие языки, например Kotlin. Перед нами возникла дилемма. Те части Scala, которые близки Java-разработчикам, очень похожи на Kotlin (в некоторых случаях синтаксис практически идентичен), но философия и направление развития этих двух языков полностью различаются. Мы поняли, что достаточно глубокое рассмотрение Scala — так, чтобы оно не повторяло материал, посвященный Kotlin, — займет слишком много места в книге.

Итак, мы решили из трех дополнительных языков оставить только два (Kotlin и Clojure), зато глубже их описать. Хотя Scala время от времени упоминается в книге, ему не посвящаются целые разделы (и тем более главы).

Язык Clojure сильно отличается от Kotlin и Java. Например, в главе 15 у нас возникли небольшие затруднения, потому что многие концепции, которые добавлялись в другие языки по мере их развития (например, функции высшего порядка и рекурсия), органически поддерживались в Clojure с самого начала. Поэтому разговор об этом языке строится не по тому же шаблону, что о Java и Kotlin. Clojure по своей сути — гораздо более функционально-ориентированный язык, и если бы мы следовали той же структуре, нам пришлось бы слишком много повторяться.

Надеемся, что в этой книге нам удалось заострить внимание на том, что разработка программного обеспечения — социальная деятельность. Мы убеждены, что технические аспекты программирования важны, однако менее очевидные факторы взаимодействия и общения между людьми по крайней мере не уступают им по важности. Объяснить эти грани разработки в книге непросто, но эта тема проходит красной нитью через весь материал.

На протяжении своей карьеры разработчики черпают силы в увлеченности технологией и желании учиться. В этой книге мы постарались акцентировать

https://t.me/javalib

некоторые вопросы, которые подпитывают эту увлеченность. Книга скорее напоминает экскурсию, чем энциклопедическое исследование, но так и было задумано — обеспечить вам отправную точку, с которой вы сами сможете продвигаться в интересующих вас направлениях.

Мы проделаем путь от новых возможностей последних версий Java до передовых методов современной разработки и будущего развития платформы. На этом пути мы поделимся некоторыми наблюдениями, которые оказались полезными для нас самих при работе с Java.

Конкурентное выполнение, быстродействие, байт-код, загрузка классов — вот некоторые из основных механизмов, которые произвели на нас наибольшее впечатление. Также мы поговорим о новых языках для JVM помимо Java по двум причинам:

- Другие языки играют все более важную роль в общей экосистеме Java.
- Понимание разных подходов, присущих разным языкам, повысит вашу квалификацию программиста независимо от того, на каком языке вы пишете.

Прежде всего эта книга нацелена на перспективу и ставит на первый план вас и ваши интересы. Мы верим, что если вы станете основательным Java-разработчиком, это поможет вам оставаться увлеченным и поддерживать свое профессиональное развитие, а также лучше ориентироваться в изменяющемся мире Java и окружающей его экосистеме. Надеемся, что концентрированный практический опыт, который вы держите в руках, окажется вам полезен и интересен, а чтение даст пищу для размышлений и будет увлекательным. По крайней мере, писать эту книгу было увлекательно!

https://t.me/javalib

Благодарности

Мы хотели бы поблагодарить многих людей за их вклад в книгу:

Элишу Хайд (Elesha Hyde) как самого замечательного ведущего редактора; Джонатана Томса (Jonathan Thoms) за превосходное научное редактирование; Алекса Бакли (Alex Buckley) за очень подробный разбор процесса загрузки классов; Хайнца Кабуца (Heinz Kabutz) за отличные предложения и соображения (и даже поправки в формате pull request'ов!) по поводу глав, посвященных конкурентному выполнению, а также за очередное замечательное предисловие; Холли Камминс (Holly Cummins) не только за то, что она вдохновила нас на работу над первым изданием, но и за неизменно обоснованные практические советы; Брюса Дюрлинга (Bruce Durling) за содействие с материалом о Clojure; Дэна Хейдингу (Dan Heidinga) за подробную обратную связь по поводу текущего состояния проекта Valhalla; Петра Ягельского (Piotr Jagielski), Луи Жакоме (Louis Jacomet), Йозефа Бартока (József Bartók) и Тома Тресански (Tom Tresansky) за исправления, касающиеся некоторых нюансов Gradle; и Эндрю Бинстока (Andrew Binstock) за тщательную вычитку нескольких глав и, как обычно, дельные советы.

Также хотим поблагодарить персонал издательства Manning: Михаэлу Батинич (Mihaela Batinić), нашего редактора; Майкла Халлера (Michael Haller), нашего научного редактора; Дейрдре Хайам (Deirdre Hiam), нашего руководителя проекта; Памелу Хант (Pamela Hunt), нашего выпускающего редактора; и Джейсона Эверетта (Jason Everett), нашего корректора. Также мы благодарны всем рецензентам: Адаму Коху (Adam Koch), Алану Ломпо (Alain Lompo), Алексу Гауту (Alex Gout), Андресу Сакко (Andres Sacco), Энди Кеффаласу (Andy Keffalas), Аншуману Пурохиту (Anshuman Purohit), Эшли Итли (Ashley Eatly), Кристиану Тоудалю (Christian Thoudahl), Кристоферу Карделлу (Christopher Kardell), Клаудии Мадертанер (Claudia Maderthaner), Конору Редмонду (Conor Redmond), Ирфану Улле (Dr. Irfan Ullah), Эдду Мелендесу Гонзалесу (Eddú Meléndez Gonzales), Эзре Симелоффу (Ezra Simeloff), Джорджу Томасу (George Thomas), Гильберто Таккари (Gilberto Taccari), Хуго да Силве Поссани (Hugo da Silva Possani), Игорю Карпу (Igor Karp), Джареду Дункану (Jared Duncan), Джавиду Асгарову (Javid Asgarov), Жан-Франсуа Морену (Jean-François Morin),

https://t.me/javalib

Джерому Майеру (Jerome Meyer), Кенту Р. Спиллнеру (Kent R. Spillner), Кимберли Л. Уинстон-Джексон (Kimberly L. Winston-Jackson), Константину Еремину (Konstantin Eremin), Мэтту Деймелу (Matt Deimel), Майклу Халлеру (Michael Haller), Майклу Уоллу (Michael Wall), Михаилу Ковалеву (Mikhail Kovalev), Патрисии Джи (Patricia Gee), Раманану Натараджану (Ramanan Natarajan), Рафаэлю Виллеле (Raphael Villela), Сатеджу Кумару Саху (Satej Kumar Sahu), Серхио Эдгару Мартинесу Пачеко (Sergio Edgar Martínez Pacheco) и Симоне Русо (Simona Ruso) — ваши советы помогли улучшить эту книгу.

От Джейсона

Я благодарен многим людям, которые помогали мне на протяжении многих лет.

Миссис Ниммо — спасибо вам за то, что хвалили мои наивные сочинения на уроках английского в школе. Не будет преувеличением сказать, что благодаря вашему поощрению я выбрал карьеру писателя на всю жизнь.

Маме — спасибо за заразительную любовь к чтению, которую мне так повезло унаследовать.

Папе — спасибо за то, что привил мне любовь к компьютерам. Она не только определила мой профессиональный путь, но и сподвигла на то, чтобы писать книги и делиться этой увлеченностью с другими.

Бену — прежде всего спасибо за дружбу. Было невероятно интересно погружаться в глубины JVM, вдохновляясь твоей потрясающей любознательностью и энтузиазмом. И конечно, спасибо за то, что уговорил меня заняться вторым изданием книги. Работы оказалось больше, чем мы ожидали, но и книга в итоге стала лучше!

Наконец, спасибо моей жене Эмбер и детям, Коралине и Эшеру, за неисчерпаемую любовь и поддержку в таком своеобразном и замечательном деле, как работа над книгой.

От Мартина

Прежде всего хочу поблагодарить Бена и Джейсона за то, что пригласили меня поучаствовать в работе над вторым изданием. Мой вклад был минимален по сравнению с их трудом, но они весьма великодушно настояли, чтобы мое имя тоже значилось на обложке!

Керри, ты обеспечивала надежную поддержку на головокружительных поворотах моей карьеры за последнее десятилетие и только понимающе улыбалась, когда я в очередной раз говорил:«На этот раз всего несколько правок, честное слово!»

Хантер, твое жизнелюбие не дает забыть о радости творчества, из-за которой я когда-то встал на путь программирования. Надеюсь, ты обретешь ту же радость независимо того, какую стезю выберешь сам.

Спасибо и всем хорошим людям из Java Engineering Group в Microsoft, сообществ Eclipse Adoptium, London Java, Java Champions и многих других; в книге

https://t.me/javalib

не хватит страниц, чтобы упомянуть их все. Благодаря этим людям я не теряю вдохновения, каждый день узнаю что-то новое и добавляю по пять новых закладок в список чтения на завтра!

От Бена

Моим родителям Сью и Мартину за их непоколебимую веру в то, что мы найдем свой непроторенный путь и пройдем по нему.

Моей жене Анне за иллюстрации и художественное чутье, а также за неустанную поддержку и понимание в ходе работы над очередной книгой.

Светлой памяти Марианито, который где-то на середине работы над книгой обнаружил, что открытый ноутбук — теплое и уютное место для того, чтобы на нем спать.

Хоселито, который отчасти поборол свои страхи, интересуясь тем, почему я увлеченно провожу столько времени за экраном, на котором так мало интересного по сравнению с экраном в соседней комнате, с космическими кораблями и взрывами.

https://t.me/javalib

Об этой книге

ДЛЯ КОГО ЭТА КНИГА

Задача книги «Java для опытных разработчиков», 2-е изд., — сделать вас Java-разработчиком следующего десятилетия и оживить вашу страсть к языку и платформе. Попутно вы познакомитесь с новыми возможностями Java и освежите представления о важнейших методологиях современной разработки ПО (таких, как разработка через тестирование и контейнерное развертывание), а также заглянете в мир альтернативных языков для JVM.

Вот как Джеймс Айри (James Iry) описал язык Java в замечательной статье «A Brief, Incomplete, and Mostly Wrong History of Programming Languages» («Краткая, неполная и преимущественно неверная история языков программирования»), доступной по адресу http://mng.bz/2rz9:

> 1996 — Джеймс Гослинг изобретает Java. Java — довольно велеречивый объектно-ориентированный язык со сборкой мусора, основанный на классах, со статической типизацией, одиночным наследованием реализации и множественным наследованием интерфейсов. Sun помпезно восхваляет Java за новизну.

И хотя Java упоминается в статье прежде всего ради шутки, в которой C# описывается теми же словами, это не отменяет того, что язык охарактеризован вполне адекватно. В статье немало других перлов, и ее стоит прочитать на досуге.

Отсюда вытекает насущный вопрос: зачем мы все еще обсуждаем язык, которому уже более четверти века? Наверняка он давно стабилизировался, и вряд ли о нем можно сказать что-то новое или интересное.

Если бы это было так, то книга получилась бы очень короткой. Но мы все еще обсуждаем Java именно потому, что одной из самых сильных сторон языка стала его способность развиваться на основе немногих проектировочных решений, которые оказались очень успешными на рынке:

- Автоматическое управление средой выполнения (например, сборка мусора или JIT-компиляция).

https://t.me/javalib

- Простой синтаксис и относительно небольшое количество элементов в базовом языке.
- Консервативный подход к развитию языка.
- Дополнительная функциональность и сложность выделены в библиотеки.
- Широкая, открытая экосистема.

Благодаря этим решениям в мире Java продолжают происходить инновации: простое ядро языка способствует низкому порогу входа в сообщество разработчиков, а широкая экосистема позволяет новичкам легко найти готовые компоненты под свои потребности. В результате платформа и язык Java остаются сильными и энергичными — даже при том, что по исторической тенденции язык не склонен к быстрым переменам. Немало разработчиков остаются приверженцами Java именно потому, что в этом языке устойчивость сочетается с эволюционными изменениями.

КАК ИСПОЛЬЗОВАТЬ ЭТУ КНИГУ

Книга рассчитана прежде всего на то, чтобы ее читали от начала до конца. Однако мы понимаем, что некоторым читателям хочется сразу перейти к определенным темам, поэтому мы постарались предусмотреть и такой стиль чтения.

Мы верим в силу обучения на практике и поэтому рекомендуем вам опробовать примеры кода, которые встречаются по ходу изложения материала. Оставшаяся часть этого раздела посвящена тому, как пользоваться книгой, если вы предпочитаете читать ее по отдельным главам.

Книга разделена на пять частей:

- **Часть I «От 8 к 11 и дальше»** (главы 1–3) состоит из трех глав, в которых рассматриваются новейшие версии Java. По умолчанию в книге используется синтаксис и семантика Java 11, а когда в особых случаях встречается более поздний синтаксис, это оговаривается отдельно.
- **Часть II «Что там внутри?»** (главы 4–7) дает возможность заглянуть «под капот» Java. Пословица гласит, что прежде чем нарушать правила, стоит их хотя бы узнать. В этих главах показано, как сначала овладеть правилами языка программирования Java, а затем нарушать их.
- **Часть III «Другие языки на JVM»** (главы 8–10) посвящена многоязычному программированию на JVM. Главу 8 можно считать обязательной, потому что она готовит почву для дальнейшего материала, рассматривая классификацию и использование альтернативных языков.
- В следующих двух главах рассматривается похожий на Java язык с объектно-ориентированной и функциональной парадигмой (Kotlin) и полноценный функциональный язык (Clojure). Эти главы можно читать независимо друг от друга, хотя, если у вас мало опыта в функциональном программировании, то лучше осваивать их по порядку.

https://t.me/javalib

- **Часть IV «Сборка и развертывание»** (главы 11–14) описывает процессы сборки, развертывания и тестирования в том виде, в каком они применяются в современных проектах. Предполагается, что у читателей есть хотя бы минимальное представление о модульном тестировании, например в той форме, которая реализована в JUnit.
- **Часть V «Передовые рубежи Java»** (главы 15–18) развивает темы, представленные ранее, и углубляется в мир функционального программирования, конкурентного выполнения и внутреннего устройства платформы. И хотя главы можно читать по отдельности, в некоторых разделах мы предполагаем, что вы уже прочли предыдущие главы или другим образом познакомились с теми или иными темами.

Эта книга прежде всего ориентируется на Java-разработчиков, которые хотят обновить свои представления о языке и платформе. Если вам хотелось бы синхронизировать свои знания с современной версией Java, то эта книга для вас.

Если вы стремитесь повысить свою квалификацию и лучше разобраться в таких темах, как функциональное программирование, конкурентность и продвинутое тестирование, книга даст начальные сведения по этим вопросам. Также она предназначена для разработчиков, которые интересуются, что полезного можно почерпнуть из других языков и как расширение кругозора помогает программировать эффективнее.

О КОДЕ

Для начала вам нужно будет загрузить и установить Java 17 (или 11). Просто следуйте инструкциям по загрузке и установке двоичного дистрибутива для вашей ОС. И дистрибутивы, и инструкции можно найти на сайте вашего привычного поставщика Java или на сайте проекта Adoptium, который не привязан к конкретному поставщику и управляется Eclipse Foundation (https:// adoptium.net/).

Java 11 (и 17) работает на Mac, Windows, Linux, а также практически на всех остальных современных ОС и аппаратных платформах.

> **ПРИМЕЧАНИЕ** Если вас интересуют нюансы лицензирования Java и смежные вопросы, вы можете ознакомиться с подробной информацией в приложении A.

В книге много примеров исходного кода как в пронумерованных листингах, так и среди обычного текста. В обеих ситуациях исходный код выделяется моноширинным шрифтом, чтобы отличаться от обычного текста.

Во многих случаях первоначальная версия кода была переформатирована: мы добавили разрывы строк и подправили отступы, чтобы эффективнее использовать место на странице. Кроме того, в листингах обычно опущены комментарии, которые были в исходном коде, если код описывается в тексте. Многие листинги сопровождаются аннотациями, которые подчеркивают важные идеи.

https://t.me/javalib

Полный код примеров из книги доступен для загрузки на сайте Manning по адресу https://www.manning.com/books/the-well-grounded-java-developer-second-edition, а также на GitHub по адресу https://github.com/well-grounded-java/resources.

Впрочем, большинство читателей наверняка захотят опробовать примеры кода в IDE. Java 11/17 и последние версии Kotlin и Clojure хорошо поддерживаются новейшими версиями основных IDE:

- Eclipse IDE
- IntelliJ IDEA Community Edition (или Ultimate Edition)
- Apache NetBeans

ФОРУМ LIVEBOOK

Приобретая книгу, вы получаете бесплатный доступ к закрытому веб-форуму издательства Manning (на английском языке), на котором можно оставлять комментарии о книге, задавать технические вопросы и получать помощь от автора и других пользователей. Чтобы получить доступ к форуму, откройте страницу https://livebook.manning.com/book/the-well-grounded-java-developer-second-edition/discussion. За информацией о форумах Manning и правилах поведения на них обращайтесь по адресу https://livebook.manning.com/discussion.

В рамках своих обязательств перед читателями издательство Manning предоставляет ресурс для содержательного общения читателей и авторов. Эти обязательства не подразумевают конкретную степень участия автора, которое остается добровольным (и неоплачиваемым). Задавайте автору хорошие вопросы, чтобы он не терял интереса к происходящему! Форум и архивы обсуждений доступны на веб-сайте издательства, пока книга продолжает издаваться.

ДРУГИЕ СЕТЕВЫЕ РЕСУРСЫ

https://github.com/well-grounded-java/resources

https://t.me/javalib

Об авторах

Бен Эванс — обладатель статуса Java Champion и главный сеньор-разработчик в Red Hat. Раньше он был ведущим архитектором систем измерительного контроля в New Relic, а также одним из основателей jClarity — стартапа, который разрабатывал средства повышения производительности и был приобретен компанией Microsoft. Бен также работал старшим специалистом по архитектуре в отделе биржевых деривативов в Deutsche Bank и старшим техническим инструктором в Morgan Stanley. Он шесть лет состоял в комитете Java Community Process Executive Committee, участвуя в разработке новых стандартов Java.

Бен — автор шести книг, включая «Optimizing Java»[1] и новые издания «Java in a Nutsell»[2], а также технических статей, которые ежемесячно читают тысячи разработчиков. Он регулярно выступает на конференциях и проводит обучение для организаций по всему миру по таким темам, как платформа Java, системная архитектура, быстродействие и конкурентность.

Джейсон Кларк — главный инженер и архитектор в New Relic, где он работал над множеством проектов: от библиотек измерительного контроля Ruby до платформ координации контейнеров. Ранее он был архитектором в WebMD, где занимался веб-службами на базе .NET.

Джейсон регулярно выступает на конференциях и участвует в проекте с открытым кодом Shoes, который направлен на то, чтобы начинающим программистам было проще и интереснее осваивать программирование графических интерфейсов пользователя.

Мартин Фербург — главный менеджер по разработке ПО в инженерной группе Java компании Microsoft и один из руководителей группы London Java User Group (LJC), где он участвовал в создании AdoptOpenJDK (сейчас Eclipse Adoptium) — ведущего дистрибутива OpenJDK, который не находится под управлением Oracle. Мартин — соавтор первого издания этой книги и член многих комитетов по стандартизации Java (JCP, Jakarta EE и т. д.).

---

[1] Эванс Б., Гоф Дж., Ньюланд К. «Java. Оптимизация программ. Практические методы повышения производительности приложений в JVM».

[2] Эванс Б., Флэнаган Д. «Java. Справочник разработчика».

https://t.me/javalib

Иллюстрация на обложке

Иллюстрация на обложке книги называется «Продавщица букетов» и позаимствована из энциклопедии национальных костюмов Сильвена Марешаля, изданной в XIX веке во Франции.

В те времена по одежде человека было легко определить, где он живет и чем занимается. Издательство Manning приветствует изобретательность и инициативность — качества, присущие индустрии IT, — и в знак этого размещает на обложках изображения, которые отражают разнообразие народов и культур мира, запечатленное на старинных иллюстрациях.

https://t.me/javalib

https://t.me/javalib

Часть I

От 8 до 11 и дальше

Первые три главы помогут подтянуть ваши знания о Java до версии 17. Мы начнем с вводной главы, где рассматриваются некоторые концептуальные изменения в Java 11. Вы узнаете, как экосистема и цикл выпуска Java изменились по сравнению с Java 8. В частности, мы расскажем о следующих новшествах и о том, что они означают для разработчиков:

- зарезервированное имя типа `var`;
- фабрики коллекций;
- новый клиент HTTP с поддержкой HTTP/2;
- однофайловые программы.

После этого мы углубимся в одно из самых значительных изменений Java за многие годы — появление полноценной системы модулей. Мы расскажем, почему это радикальное изменение было необходимо. Оно тщательно проектировалось так, чтобы его можно было внедрять поэтапно. Вы не только познакомитесь с основными принципами этой системы, но и узнаете, как эффективно применять ее в своих приложениях и библиотеках.

Java 17 — продукт нового цикла выпуска — воплотила внушительный набор новых языковых средств, в числе которых:

- текстовые блоки;
- выражения `switch`;
- записи;
- ограниченно расширяемые («запечатанные») типы (sealed types).

К концу части I вы привыкнете органично мыслить и программировать на Java 17 и будете готовы применять новые знания на протяжении оставшейся части книги.

https://t.me/javalib

1 Современный язык Java

В ЭТОЙ ГЛАВЕ

- ✓ Java как платформа и язык
- ✓ Новый цикл выпуска Java
- ✓ Расширенное выведение типов (`var`)
- ✓ Изменения языка и платформы
- ✓ Инкубационная и предварительная функциональность
- ✓ Незначительные изменения языка в Java 11

Добро пожаловать в мир Java, где не перестают происходить увлекательные события. В сентябре 2021 года была выпущена Java 17 — наиболее свежая (на момент написания книги) версия с долгосрочной поддержкой (LTS, Long-Term Support), и самые бесстрашные команды начали переходить на нее[1].

На момент написания книги, если не считать отдельных первопроходцев, приложения Java более или менее поровну распределены между Java 11 (выпущенной в сентябре 2018 года) и намного более старой Java 8 (2014). Java 11 предлагает много интересных возможностей, особенно для команд, которые развертывают свои продукты в облаке, но многие все еще не спешат на нее переходить.

[1] 19 сентября 2023 вышла Java 21 — следующая после 17-й LTS-версия. — *Примеч. ред.*

https://t.me/javalib

В части I книги мы потратим немного времени, чтобы рассказать о новых возможностях, которые появились в Java 11 и 17. Хочется надеяться, что этот материал убедит некоторые команды и руководителей, которым неохота мигрировать с Java 8, что в новых версиях все гораздо лучше.

В этой главе мы сосредоточимся на Java 11, потому что: 1) это LTS-версия с наибольшей долей рынка и 2) массовый переход на Java 17 еще не произошел. Однако в главе 3 мы расскажем о новых возможностях Java 17, чтобы вы были в курсе нынешнего положения дел.

Начнем с дуализма «язык или платформа», который лежит в основе современного Java. Это чрезвычайно важная тема, к которой мы будем неоднократно возвращаться в книге, поэтому необходимо с самого начала понять суть.

1.1. ЯЗЫК И ПЛАТФОРМА

Термин «Java» может относиться к нескольким взаимосвязанным предметам. В частности, он может означать как язык программирования, на котором люди пишут код, так и гораздо более широкое понятие — «платформа Java».

Как ни странно, разные авторы иногда по-разному определяют, что считать языком, а что — платформой. Это приводит к недоразумениям, и люди начинают путаться в том, чем различаются эти две сущности и какая из них отвечает за те или иные программные возможности, которые используются в коде приложения.

Давайте проясним это различие прямо сейчас, потому что на него опираются многие темы из этой книги. Вот наши определения:

- *Язык Java* — объектно-ориентированный язык со статической типизацией, над которым мы слегка поиронизировали в разделе «Об этой книге». Надеемся, что он вам уже хорошо знаком. У исходного кода, написанного на языке Java, есть одна очевидная особенность: он понятен для человека (или по крайней мере должен быть таковым!).
- *Платформа Java* — это программная система, которая предоставляет среду выполнения. Ключевой компонент этой среды — JVM (виртуальная машина Java), которая компонует и выполняет ваш код, а код она получает в форме файлов классов (которые не предназначены для чтения человеком). JVM не интерпретирует исходный код на языке Java напрямую, а требует, чтобы сначала он был преобразован в файлы классов.

Один из важных факторов успеха Java как программной системы — ее стандартизация. Иначе говоря, у Java есть спецификации, которые описывают, как она должна работать. Стандартизация позволяет разным компаниям и проектным группам создавать реализации, которые будут работать одинаково (по крайней мере в теории). Спецификации не дают никаких гарантий относительно того, какую производительность покажут разные реализации на одной и той же задаче, но стандарты могут гарантировать правильность результатов.

https://t.me/javalib

Систему Java регламентируют несколько разных спецификаций, самые важные из которых — JLS (Java Language Specification, спецификация языка Java) и JVM (VMSpec). Это разделение играет важную роль в современном мире Java; более того, в VMSpec больше нет никаких прямых ссылок на JLS. Позже мы еще вернемся к различиям между этими двумя спецификациями.

ПРИМЕЧАНИЕ В наши дни JVM фактически представляет собой независимую от языка среду общего назначения для выполнения программ. Это одна из причин разделять спецификации.

Когда вы сталкиваетесь с этим дуализмом, возникает очевидный вопрос: «А как тогда связаны эти понятия?» Если их разделили, то как тогда они вместе формируют систему Java?

Язык и платформа связаны друг с другом через общее определение формата файлов классов (файлов .class). Вы не пожалеете, если обстоятельно изучите определение файлов классов (соответствующий материал ждет вас в главе 4), — собственно, это один из способов превратиться из хорошего Java-программиста в выдающегося. На рис. 1.1 изображен полный процесс того, как производится и используется код на Java.

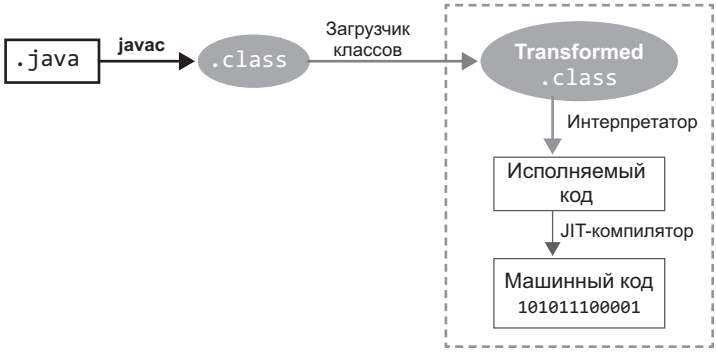


Рис. 1.1. Исходный код на Java преобразуется в файлы .class, а затем обрабатывается во время загрузки перед JIT-компиляцией

Как видно на этой схеме, код на Java начинает свое существование в виде человекочитаемого исходного кода, после чего он с помощью `javac` компилируется в файл .class и загружается в JVM. В процессе загрузки классы часто обрабатываются и изменяются. Многие популярные фреймворки Java преобразуют классы в процессе загрузки, внедряя в них динамическое поведение, например механизмы измерительного контроля или альтернативные контексты поиска загружаемых классов.

ПРИМЕЧАНИЕ Загрузка классов — чрезвычайно важная функция платформы Java, и в главе 4 вы узнаете о ней намного подробнее.

https://t.me/javalib

Java — это компилируемый или интерпретируемый язык? Обычно его представляют как язык, который компилируется в файлы .class перед тем, как запускаться на JVM. Если допытываться настойчивее, многие разработчики также пояснят, что байт-код изначально интерпретируется JVM, но проходит JIT-компиляцию на более поздней стадии. Однако здесь представление многих людей ограничивается довольно расплывчатой концепцией байт-кода как фактически машинного кода для воображаемого или упрощенного процессора.

На самом деле байт-код JVM скорее занимает промежуточное положение между исходным кодом, понятным для человека, и машинным кодом. В терминах теории компиляторов байт-код представляет собой разновидность промежуточного языка (IL, Intermediate Language), а не реальный машинный код. То есть преобразование исходного кода на Java в байт-код не является компиляцией в том смысле, в каком ее понимает программист на C++ или Go, а `javac` — это не компилятор вроде `gcc`, а генератор, который создает файлы классов из исходного кода на Java. Настоящий компилятор в экосистеме Java — это JIT-компилятор, как показано на рис. 1.1.

Иногда систему Java описывают как «динамически компилируемую». Этот термин подчеркивает, что реальная компиляция — это JIT-компиляция во время выполнения, а не создание файла класса на этапе сборки.

> **ПРИМЕЧАНИЕ** Из-за того что в Java есть компилятор исходного кода javac, многие разработчики воспринимают Java как статический, компилируемый язык. Далеко не все в курсе, что на стадии выполнения среда Java в действительности чрезвычайно динамична — просто это не заметно на первый взгляд.

Таким образом, правильный ответ на вопрос, является ли Java компилируемым или интерпретируемым языком, звучит так: «И тем и другим».

Разобравшись с различиями между языком и платформой, поговорим о новом цикле выпуска Java.

1.2. НОВЫЙ ЦИКЛ ВЫПУСКА JAVA

Java не всегда был языком с открытым исходным кодом, но после объявления на конференции JavaOne в 2006 году исходный код самого Java (кроме нескольких фрагментов, которые не принадлежали компании Sun) был опубликован на условиях лицензии GPLv2+CE (https://openjdk.java.net/legal/gplv2+ce.html).

Это случилось во времена Java 6, так что Java 7 стала первой версией, которая распространялась по лицензии OSS (Open Source Software). С тех пор разработка открытого исходного кода платформы Java сосредоточена вокруг проекта OpenJDK (https://openjdk.java.net).

Обсуждение проекта происходит в основном в списках рассылки, каждый из которых посвящен отдельному аспекту всей кодовой базы. Существуют как «постоянные» рассылки (например, `core-libs` — базовые библиотеки), так

https://t.me/javalib

и временные, которые создаются ради конкретных проектов OpenJDK вроде `lambda-dev` (лямбда-функции) и перестают быть активными, когда проект завершается. По большей части эти списки рассылки стали полезными форумами, где можно обсуждать возможную будущую функциональность, что позволяет широкому кругу разработчиков участвовать в создании новых версий Java.

> **ПРИМЕЧАНИЕ** Компания Sun Microsystems была приобретена Oracle незадолго до выпуска Java 7. Таким образом, все выпуски Java от Oracle основаны на кодовой базе с открытым исходным кодом.

Новые версии Java с открытым исходным кодом выпускались по мере готовности функциональности: фактически выпуск определялся одной доминирующей функциональной возможностью, например лямбда-функции в Java 8 или модули в Java 9.

Однако после Java 9 принцип выпусков изменился. Компания Oracle решила, что, начиная с Java 10, язык Java будет выпускаться по строгому графику. Это значит, что OpenJDK теперь опирается на магистральную модель разработки, которая подразумевает следующее:

- Новые возможности разрабатываются в отдельной ветви, а слияние выполняется только после стабилизации кода.
- Выпуски происходят с жесткой периодичностью.
- Запаздывающие возможности не задерживают выпуск, а переносятся на следующий выпуск.
- Текущая основная ветвь разработки всегда должна быть пригодной к выпуску (теоретически).
- Если необходимо экстренное исправление, его можно подготовить и выпустить в любой момент.
- Отдельные проекты OpenJDK используются для экспериментов и исследования перспективных направлений.

Новые версии Java выпускаются через каждые шесть месяцев. Разные поставщики (Oracle, Eclipse Adoptium, Amazon, Azul и т. д.) могут объявить *любой* из этих выпусков версией с долгосрочной поддержкой (LTS, Long Term Support). Однако на практике все поставщики традиционно назначают LTS-версию раз в три года.

> **ПРИМЕЧАНИЕ** В конце 2021 года обсуждалась идея уменьшить периодичность LTS с трех лет до двух. Возможно, следующей LTS-версией станет Java 21 в 2023 году (вместо Java 23 в 2024-м)[1].

Первой LTS-версией стала Java 11, а Java 8 была включена в число LTS-версий задним числом. Компания Oracle предполагала, что сообщество Java будет охотно

[1] Так и вышло. — *Примеч. ред.*

https://t.me/javalib

поспевать за обновлениями и осваивать новые выпуски по мере выхода. Однако на практике сообщество (особенно корпоративные клиенты) не приняло такой подход, предпочитая обновляться с одной LTS-версии на следующую. Конечно, при этом новые возможности Java осваиваются медленнее и нововведения пробуксовывают. Однако такова реальность корпоративных продуктов, и многие все еще рассматривают обновление версии Java как весьма масштабное мероприятие.

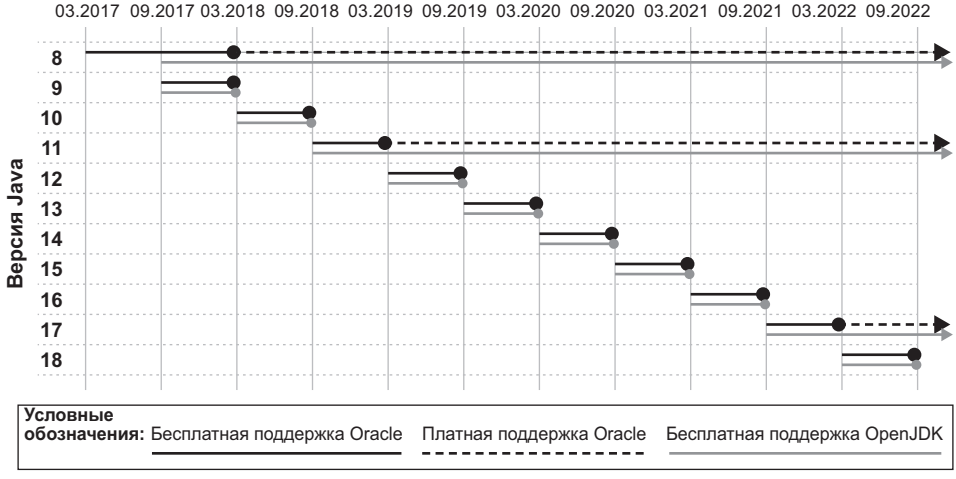


Рис. 1.2. Временна́я шкала недавних и будущих выпусков Java

Это означает, что хотя на графике на рис. 1.2 раз в полгода значится основной выпуск, на практике широко применяются только LTS-версии: Java 17 (сентябрь 2021 года), Java 11 (сентябрь 2018 года) и Java 8, выпущенная до появления модулей (в 2014 году). Java 8 и Java 11 занимают примерно одинаковую долю рынка, хотя Java 11 в последнее время захватила более 50 % рынка и быстро наращивает популярность. Ожидается, что переход на Java 17 произойдет намного быстрее, чем переход с Java 8 на Java 11, потому что самые серьезные препятствия, связанные с системой модулей и ограничениями безопасности, уже были преодолены при предыдущей миграции.

Другое важное изменение в новой модели выпуска заключается в том, что компания Oracle пересмотрела лицензию на распространение своих продуктов. Хотя JDK от Oracle строится из исходного кода OpenJDK, на двоичные файлы не распространяется лицензия OSS. JDK от Oracle — закрытое ПО, и по состоянию на JDK 11 Oracle предоставляет поддержку и обновления для каждой версии только на шесть месяцев. Это означает, что многие люди, которые полагались на бесплатные обновления Oracle, теперь сталкиваются с выбором:

- платить Oracle за поддержку и обновления или
- использовать другой дистрибутив, который предоставляет двоичные файлы на условиях открытой лицензии.

https://t.me/javalib

Среди альтернативных поставщиков JDK — Eclipse Adoptium (ранее AdoptOpenJDK), Alibaba (Dragonwell), Amazon (Corretto), Azul Systems (Zulu), IBM, Microsoft, Red Hat и SAP.

> **ПРИМЕЧАНИЕ** Двое авторов этой книги (Мартин и Бен) стояли у истоков проекта AdoptOpenJDK, который позднее развился в независимый проект сообщества Eclipse Adoptium, цель которого — выпускать высококачественные бесплатные двоичные дистрибутивы Java с открытым кодом. За дополнительной информацией обращайтесь на сайт adoptium.net.

При меняющихся лицензиях и таком количестве поставщиков выбрать правильную версию Java для вас и вашей команды оказывается весьма ответственным решением. К счастью, авторитетные лидеры экосистемы Java написали подробные руководства по этой теме, а в приложении А приведена их сводка.

Хотя модель выпуска Java перешла на выпуск по графику, подавляющее большинство команд разработчиков все еще используют JDK 8 или 11. Эти LTS-выпуски сопровождаются сообществом (в том числе крупными фирмами-разработчиками), и для них все еще регулярно публикуются обновления безопасности и исправления ошибок. Изменения в LTS-версиях намеренно имеют небольшую область действия и относятся к «хозяйственным» обновлениям. Помимо исправлений, касающихся безопасности и мелких ошибок, разрешен лишь минимальный набор изменений: в основном это исправления, которые необходимы, чтобы обеспечить правильную работу LTS-выпусков в течение их предполагаемого срока эксплуатации. В частности, сюда относятся:

- добавление новой японской эры;
- обновления базы данных часовых поясов;
- TLS 1.3;
- добавление Shenandoah — сборщика мусора с низкой задержкой для крупных современных нагрузок.

Еще одно необходимое изменение — обновление сценариев сборки для macOS, чтобы они работали с новейшей версией среды Xcode от Apple, а значит, продолжали функционировать в новых выпусках macOS.

В проектах по сопровождению JDK 8 и 11 (которые иногда называются «проектами обновлений») все еще сохраняется потенциальная возможность обратно портировать новую функциональность, но она минимальна. Например, одно из правил гласит, что портируемая функциональность не должна нарушать семантику программ. Примеры допустимых изменений — поддержка TLS 1.3 или обратное портирование Java Flight Recorder в Java 8u272. После того как мы прояснили различия между языком и платформой и разобрались с новой моделью выпусков, давайте перейдем к техническим возможностям современного Java. О первой возможности, с которой мы познакомимся, разработчики просили почти с самого первого выпуска Java ведь она позволяет сократить объем кода при написании программ.

https://t.me/javalib

1.3. РАСШИРЕННОЕ ВЫВЕДЕНИЕ ТИПОВ (СИНТАКСИС VAR)

Java традиционно имеет репутацию весьма пространного языка. Тем не менее в последних версиях язык эволюционировал так, чтобы шире использовать *выведение типов* (type inference). Оно позволяет компилятору исходного кода определить часть нужной информации автоматически, чтобы вам не приходилось указывать все явно.

> **ПРИМЕЧАНИЕ** Выведение типов позволяет сократить объем шаблонного кода, устранить повторения и сделать код более компактным и удобочитаемым.

Эта тенденция началась в Java 5, где были добавлены обобщенные методы, которые поддерживают крайне ограниченную форму выведения типа для аргументов обобщенных типов. Вместо того чтобы явно указывать нужный тип, как в следующем примере:

```
List<Integer> empty = Collections.<Integer>emptyList();
```

параметр обобщенного типа в правой части можно было опустить:

```
List<Integer> empty = Collections.emptyList();
```

Такая запись вызова обобщенных методов стала настолько привычной, что многие разработчики с трудом вспомнят форму с явными аргументами типов. И это хорошо: значит, выведение типов справляется со своей задачей и избавляет код от лишних шаблонных конструкций, чтобы он стал понятнее.

Следующее важное улучшение выведения типов появилось в Java 7, где изменилась работа с обобщениями. До Java 7 такой код встречался очень часто:

```
Map<Integer, Map<String, String>> usersLists =
                        new HashMap<Integer, Map<String, String>>();
```

Здесь вы крайне развернуто объявляете, что у вас есть какие-то пользователи, которые идентифицируются по `userid` (целое число), и что у каждого пользователя есть специфический для него набор свойств (представленный как отображение строк на строки).

Но на самом деле почти половина этого кода — дублирующиеся символы, которые не несут никакой полезной информации. Начиная с Java 7, можно использовать такую запись:

```
Map<Integer, Map<String, String>> usersLists = new HashMap<>();
```

При этом компилятор сам определяет правильный тип для выражения в правой части — он не ограничивается подстановкой текста, определяющего полный тип.

> **ПРИМЕЧАНИЕ** Так как пара угловых скобок в сокращенном объявлении типа напоминает бубновую масть, эта форма называется «бубновым синтаксисом» (diamond syntax).

https://t.me/javalib

В Java 8 выведение типов расширилось до поддержки лямбда-выражений. Например, здесь алгоритм выведения типов может сделать вывод, что `s` имеет тип `String`:

```
Function<String, Integer> lengthFn=s-> s.length();
```

В современном Java выведение типов сделало еще один шаг вперед: появилось выведение типов локальных переменных (LVTI, Local Variable Type Inference), которое также обозначается `var`. Эта возможность, добавленная в Java 10, позволяет автоматически выводить типы *переменных* вместо типов *значений*, как в следующем примере:

```
var names = new ArrayList<String>();
```

Это достигается благодаря тому, что `var` сделано зарезервированным, «волшебным» именем типа, а не ключевым словом языка. Теоретически `var` по-прежнему можно использовать в качестве имени переменной, метода или пакета.

> **ПРИМЕЧАНИЕ** У правильного использования `var` есть важный побочный эффект: на передний план выводится предметная область вашего кода, а не информация о типе. Но кому много дано, с того много и спросится! Тщательно выбирайте имена переменных, чтобы помочь будущим читателям вашего кода.

С другой стороны, код, в котором ранее `var` использовалось как имя типа, придется перекомпилировать. Впрочем, практически все разработчики Java следуют соглашению о том, что имена типов должны начинаться с буквы в верхнем регистре, так что, по всей видимости, количество экземпляров существующих типов с именем `var` пренебрежимо мало. Это означает, что формально ничто не мешает написать код вроде приведенного в следующем листинге.

Листинг 1.1. Плохой код

```
package var;

public class Var {
  private static Var var = null;

  public static Var var() {
    return var;
  }

  public static void var(Var var) {
    Var.var = var;
  }
}
```

А потом вызвать его таким образом:

```
var var = var();
if (var == null) {
  var(new Var());
}
```

https://t.me/javalib

Тем не менее «ничто не мешает» еще не означает «так стоит делать». Писать такой код, как в листинге 1.1, — верный способ нажить врагов; кроме того, такой код наверняка не пройдет код-ревью!

Синтаксис `var` предназначен для того, чтобы сократить объем кода на Java, и он выглядит привычно для программистов, которые приходят в Java из других языков. Он не вводит в язык динамическую типизацию, и каждой переменной Java по-прежнему соответствует статический тип — просто этот тип не всегда обязательно указывать явно.

Выведение типов в Java действует *локально*, и в случае с `var` алгоритм проверяет только объявление локальной переменной. Это означает, что `var` нельзя использовать для полей, аргументов методов или возвращаемых типов. Компилятор применяет разновидность *анализа ограничений* (constraint solving), чтобы определить, существует ли тип, который удовлетворял бы всем требованиям кода в том виде, в каком он записан.

> **ПРИМЕЧАНИЕ** `var` реализуется исключительно компилятором исходного кода (`javac`) и никак не влияет на выполнение или быстродействие.

Например, в объявлении `lengthFn` в недавнем примере анализатор ограничений способен заключить, что тип параметра `s` метода должен быть совместим с типом `String`, который явно указан как тип параметра `Function`. Конечно, в Java тип `String` — конечный (`final`), поэтому компилятор выводит, что типом `s` является именно `String`.

Чтобы компилятор мог выводить типы, программист должен предоставить достаточно информации, которая позволит решить систему ограничивающих уравнений. Например, такой код:

```
var fn = s -> s.length();
```

не содержит достаточно информации для того, чтобы компилятор мог вывести тип `fn`, так что компилироваться он не будет. Стоит отметить важный частный случай:

```
var n = null;
```

Компилятор не может вывести тип, потому что значение `null` можно присвоить переменной любого ссылочного типа, так что нельзя однозначно заключить, какой тип должна принимать `n`. В таких случаях говорят, что соответствующая система уравнений ограничения типов является *недоопределенной*. Этот математический термин означает, что решаемых уравнений меньше, чем неизвестных.

Можно представить себе такое выведение типов, когда анализируется не только исходное объявление локальных переменных, но и другие участки кода:

```
var n = null;
String.format(n);
```

https://t.me/javalib

Более сложный алгоритм выведения (или человек) смог бы сделать вывод, что `n` в этом примере относится к типу `String`, потому что метод `format()` получает строку в первом аргументе.

На первый взгляд идея выглядит заманчиво, но как это часто бывает в программировании, она связана с компромиссами. Чем сложнее алгоритм, тем больше времени требуется на компиляцию и тем больше шансов на неудачу при выведении. В свою очередь, это означает, что программисту придется развивать более тонкую интуицию, чтобы правильно обращаться с выведением нелокальных типов.

В других языках могут быть другие компромиссы, но в Java ситуация предельно однозначна: для выведения типов используются только объявления. Выведение типов локальных переменных призвано сократить объем шаблонного кода и избавиться от дублирования. Однако выведение стоит применять только там, где оно помогает сделать код яснее, а не как грубый инструмент, который применяется для всех задач без разбора (см. антипаттерн «Золотой молоток» (Golden Hammer)).

Например, LVTI полезно в таких случаях:

- В простых инициализаторах, где правая часть — вызов конструктора или статического фабричного метода.
- Если удаление явного типа позволяет исключить повторяющуюся или избыточную информацию.
- Если имена переменных уже обозначают их типы.
- Если локальная переменная действует и используется в простой и узкой области.

Стюарт Маркс (Stuart Marks), один из ключевых разработчиков языка Java, представляет полный набор практических правил использования LTVI в руководстве по адресу http://mng.bz/RvPK.

Завершая этот раздел, рассмотрим более сложный вариант использования `var` — так называемые *необозначаемые* (nondenotable) типы. Они допустимы в Java, однако их нельзя указывать как тип переменной, а можно только выводить как тип присваиваемого выражения. Рассмотрим простой пример с использованием интерактивной среды `jshell`, которая появилась в Java 9:

```
jshell> var duck = new Object() {
   ...> void quack() {
   ...>     System.out.println("Кря-кря!");
   ...> }
   ...> }
duck ==> $0@5910e440

jshell> duck.quack();
Кря-кря!
```

У переменной `duck` необычный тип: фактически это `Object`, но расширенный методом с именем `quack()`. Хотя объект может крякать как утка, у его типа нет

https://t.me/javalib

имени, поэтому этот тип нельзя использовать ни как параметр метода, ни как тип возвращаемого значения.

С LVTI этот тип может фигурировать как выводимый тип локальной переменной, что позволяет использовать его внутри метода. Конечно, такой тип недоступен за пределами этой узкой локальной области видимости, так что его существование — скорее технический курьез, чем полезная возможность.

Несмотря на ограничения LVTI, оно дает некоторое представление о том, как Java подходит к возможности, которая есть в других языках и обычно называется *структурной типизацией* в языках со статической типизацией и *утиной типизацией* в языках с динамической типизацией (например, Python).

1.4. ИЗМЕНЕНИЯ ЯЗЫКА И ПЛАТФОРМЫ

На наш взгляд, важно понимать не только *что* изменилось в языке, но и *почему*. Многие из тех, кто следит за разработкой новых версий Java, активно интересуются новыми возможностями языка, однако не всегда понимают, сколько работы требуется, чтобы полноценно реализовать соответствующие изменения и подготовить их для массового использования.

Возможно, вы также заметили, что в такой зрелой среде выполнения, как Java, возможности языка нередко приходят из других языков или библиотек, развиваются в популярных фреймворках и только потом добавляются в сам язык или среду выполнения. Мы надеемся пролить свет на эти процессы, а заодно рассеять несколько сопутствующих мифов. Но если эволюция Java вас не особо интересует, переходите прямо к разделу 1.5, где речь пойдет об изменениях в языке.

В развитии языка Java одни изменения требуют больше инженерных усилий, а другие — меньше. На рис. 1.3 мы попытались представить, какие бывают типы изменений и насколько затратно реализовывать каждый из них по сравнению с остальными.

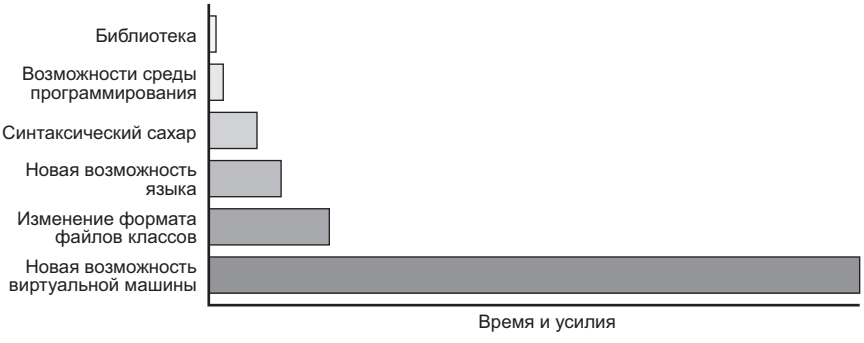


Рис. 1.3. Относительный объем усилий, необходимых для реализации разных типов новой функциональности

https://t.me/javalib

В общем случае лучше выбирать путь, который требует меньше усилий. Это означает, что если новую возможность можно реализовать в виде библиотеки, обычно так и стоит сделать. Но многие решения нелегко, а то и вообще невозможно реализовать в виде библиотек или функций IDE. Ряд возможностей приходится внедрять на более глубоком уровне внутри платформы. Вот как некоторые из недавних нововведений можно классифицировать по нашей шкале сложности:

- *Изменения в библиотеке*: фабричные методы коллекций (Java 9).
- *Синтаксический сахар*: подчеркивания в числах (Java 7).
- *Небольшая новая возможность языка*: `try`-с-ресурсами (Java 7).
- *Изменение формата файлов классов*: аннотации (Java 5).
- *Новая возможность JVM*: гнезда и соседи (nestmates) (Java 11).
- *Крупное обновление языка*: лямбда-выражения (Java 8).

Давайте поближе познакомимся с тем, как вносятся изменения на разных уровнях шкалы сложности.

1.4.1. Синтаксический сахар

Синтаксическим сахаром (syntactic sugar) называются удобства, которые упрощают программистам работу с уже существующей функциональностью. Как правило, на ранней стадии компиляции синтаксический сахар удаляется из внутреннего представления программы на уровне компилятора.

Таким образом, изменения из категории синтаксического сахара легко реализовать, потому что обычно они требуют относительно небольшого объема работы, а изменения вносятся только в компилятор (`javac` в случае Java).

Возможно, к этому моменту у вас появился вопрос: «Какие изменения в спецификации считаются незначительными?» Одним из самых прямолинейных изменений в Java 7 стало добавление единственного слова — `String` — в раздел 14.11 спецификации JLS, что позволило использовать строки в инструкциях `switch`. Трудно придумать изменение, которое было бы еще мельче, однако даже эта поправка затронула ряд других аспектов спецификации. Всякое нововведение ведет к последствиям, которые приходится отслеживать по всей структуре языка.

1.4.2. Изменения языка

Для *любого* изменения необходимо выполнить полный набор действий (или по крайней мере проанализировать, нужно ли выполнять каждое из них):

- Обновить JLS.
- Реализовать прототип в компиляторе исходного кода.
- Добавить соответствующую библиотечную поддержку.

https://t.me/javalib

- Написать тесты и примеры.
- Обновить документацию.

Кроме того, если изменение затрагивает платформу или JVM, необходимы также следующие действия:

- Обновить VMSpec.
- Реализовать изменения в JVM.
- Добавить поддержку изменений в инструменты JVM и файлы классов.
- Проанализировать последствия для рефлексии.
- Проанализировать последствия для сериализации.
- Изучить, как изменение повлияет на низкоуровневые программные компоненты, например JNI (Java Native Interface).

Немалый объем работы — и это после того, как вы проанализировали последствия изменений по всей спецификации языка!

Одна из проблемных областей при внесении изменений — система типов. Дело не в том, что эта система в Java чем-то плоха; просто в языках с богатыми системами статических типов обычно оказывается много потенциальных точек взаимодействия между разными составляющими этих систем. Изменения в них могут быть чреваты неприятными неожиданностями.

1.4.3. Документы JSR и JEP

Для внесения изменений в платформу Java используются два основных механизма, и первый из них — *JSR* (Java Specification Request, «запрос на спецификацию Java»), который подчиняется процедуре *JCP* (Java Community Process, «процесс сообщества Java»). С помощью этого метода определяются стандартные API — как внешние библиотеки, так и основные внутренние API платформы.

Исторически это был единственный способ вносить изменения в платформу Java, и он лучше всего подходил, чтобы закрепить консенсус по поводу уже зрелой технологии. Однако в последние годы стремление реализовывать изменения быстрее (и в меньших масштабах) привело к тому, что появилась упрощенная альтернатива — документы *JEP* (JDK Enhancement Proposal, «предложение по улучшению JDK»). Кумулятивные платформенные документы JSR теперь состоят из JEP, ориентированных на следующую версию Java. Процесс JSR обеспечивает дополнительную защиту интеллектуальной собственности для всей экосистемы.

Обсуждая новые возможности Java, часто бывает удобно ссылаться на будущую или недавно внедренную функциональность по номеру JEP. Полный список всех JEP (в том числе реализованных и отозванных) можно найти по адресу https://openjdk.java.net/jeps/0.

https://t.me/javalib

1.4.4. Инкубационная и предварительная функциональность

В новой модели выпуска Java появились два механизма, которые позволяют опробовать предлагаемые возможности перед тем, как закрепить их в более поздней версии. Цель этих механизмов — обеспечить улучшенную функциональность благодаря обратной связи от гораздо более широкого круга пользователей, сохраняя возможность изменить или аннулировать функциональность до того, как она станет неотъемлемой частью Java.

К *инкубационной функциональности* (incubating features) относятся новые API и их реализация, которая в простейшей форме представляет собой новый API, поставляемый в виде автономного модуля (модули Java подробнее рассматриваются в главе 2). Имя модуля выбирается так, чтобы оно ясно указывало, что это временный API. Когда функциональность окончательно утвердится, имя меняется на постоянное.

> **ПРИМЕЧАНИЕ** Это означает, что если код опирается на нефинальную версию инкубационной функциональности, то в него придется вносить изменения, когда она станет финальной.

Характерный пример инкубационной функциональности — поддержка версии 2 протокола HTTP (обычно обозначаемой HTTP/2). В Java 9 она присутствовала в виде инкубационного модуля `jdk.incubator.http`. Имя модуля в сочетании с пространством имен `jdk.incubator` вместо `java` четко обозначало, что эта возможность еще не утверждена и может измениться. В Java 11 эта функциональность была стандартизирована и перемещена в модуль `java.net.http` в пространстве имен `java`.

> **ПРИМЕЧАНИЕ** Еще один пример инкубационной функциональности встретится в главе 18, когда мы будем обсуждать Foreign Access API — часть проекта OpenJDK с кодовым названием Panama.

Главное преимущество этого подхода — в том, что инкубационную функциональность можно изолировать в одном пространстве имен. Разработчики могут ее быстро опробовать и даже использовать в промышленном коде — при условии, что они готовы внести изменения в код, а также перекомпилировать и перекомпоновать его, когда функциональность будет стандартизирована.

Предварительная функциональность (preview features) — другой механизм, с помощью которого свежие версии Java предоставляют возможности, которые еще не окончательно утверждены. Эта функциональность изолирована в меньшей степени, чем инкубационная, потому что реализуется как часть самого языка на более глубоком уровне. Теоретически для этого может понадобиться поддержка на следующих уровнях:

- компилятор `javac`;
- формат байт-кода;
- файлы классов и загрузка классов.

https://t.me/javalib

Предварительная функциональность доступна, только если передать компилятору и среде выполнения конкретные флаги. Если пытаться использовать предварительную функциональность без флагов, это вызовет ошибку как на стадии компиляции, так и во время выполнения.

В результате работать с предварительной функциональностью значительно сложнее, чем с инкубационной, и поэтому ее невозможно использовать в промышленном коде. Начнем хотя бы с того, что предварительная функциональность представляется соответствующей версией формата файла класса, которая не финализирована и, возможно, не будет поддерживаться ни в одной эксплуатационной версии Java.

Таким образом, предварительная функциональность подходит только для экспериментов, внутреннего тестирования и ознакомления. К сожалению, почти во всех вариантах развертывания кода, предназначенного для реальной эксплуатации, можно использовать только полностью финализированную функциональность.

В Java 11 нет никакой предварительной функциональности (хотя в Java 12 появилась первая предварительная версия *switch-выражений*), так что в этом разделе трудно привести соответствующий пример. Мы подробнее рассмотрим предварительную функциональность в главе 3, когда пойдет речь о Java 17.

1.5. НЕБОЛЬШИЕ ИЗМЕНЕНИЯ В JAVA 11

В выпусках, последовавших за Java 8, появилось довольно много небольших нововведений. Давайте кратко пройдемся по самым важным из них, однако имейте в виду, что здесь мы рассмотрим далеко не все изменения.

1.5.1. Фабрики коллекций (JEP 213)

Часто звучали предложения расширить Java так, чтобы язык поддерживал простой способ объявления *литералов коллекций* — списков, отображений и т. д. Эта идея выглядит привлекательно, потому что во многих других языках такая возможность поддерживается в том или ином виде, и в самом Java всегда были литералы массивов:

```
jshell> int[] numbers = {1, 2, 3};
numbers ==> int[3] { 1, 2, 3 }
```

Но несмотря на всю внешнюю привлекательность, добавить эту возможность на уровне языка оказывается не так-то просто. Например, хотя разработчикам прекрасно знакомы реализации `ArrayList`, `HashMap` и `HashSet`, основополагающий принцип проектирования коллекций Java гласит, что они должны представляться как интерфейсы, а не как классы. Также доступны другие реализации, которые применяются достаточно широко.

А это значит, что если бы появился новый синтаксис, который напрямую связан с конкретными реализациями (какими бы популярными они ни были), это пошло

https://t.me/javalib

бы вразрез с идеологией языка. Вместо этого было решено добавить в соответствующие интерфейсы простые фабричные методы; при этом использовался тот факт, что в Java 8 стало возможно включать статические методы в интерфейсы. В результате получается код следующего вида:

```
Set<String> set = Set.of("a", "b", "c");

var list = List.of("x", "y");
```

Хотя запись оказывается более громоздкой, чем при гипотетической поддержке на уровне языка, накладная сложность с точки зрения реализации оказывается значительно ниже. Новые методы реализуются в виде набора перегруженных версий:

```
List<E> List<E>.<E>of()
List<E> List<E>.<E>of(E e1)
List<E> List<E>.<E>of(E e1, E e2)
List<E> List<E>.<E>of(E e1, E e2, E e3)
List<E> List<E>.<E>of(E e1, E e2, E e3, E e4)
List<E> List<E>.<E>of(E e1, E e2, E e3, E e4, E e5)
List<E> List<E>.<E>of(E e1, E e2, E e3, E e4, E e5, E e6)
List<E> List<E>.<E>of(E e1, E e2, E e3, E e4, E e5, E e6, E e7)
List<E> List<E>.<E>of(E e1, E e2, E e3, E e4, E e5, E e6, E e7, E e8)
List<E> List<E>.<E>of(E e1, E e2, E e3, E e4, E e5, E e6, E e7, E e8, E e9)
List<E> List<E>.<E>of(E e1, E e2, E e3, E e4, E e5, E e6, E e7, E e8, E e9,
  E e10)
List<E> List<E>.<E>of(E... elements)
```

Определены наиболее частые случаи (до 10 элементов), а также форма с переменным числом аргументов (vararg, вариадическая) для маловероятного сценария, когда в коллекции должно быть более 10 элементов.

Для отображений ситуация немного усложняется, потому что у них есть два обобщенных параметра — тип ключа и тип значения. А значит, хотя простые случаи можно записывать в виде:

```
var m1 = Map.of(k1, v1);
var m2 = Map.of(k1, v1, k2, v2);
```

нет простого способа записать для отображения подобную форму с переменным числом аргументов. Вместо такой формы используется другой фабричный метод — `ofEntries()` в сочетании со статическим вспомогательным методом `entry()`:

```
Map.ofEntries(
    entry(k1, v1),
    entry(k2, v2),
    // ...
    entry(kn, vn));
```

И последнее, что стоит учитывать разработчикам, — фабричные методы порождают экземпляры неизменяемых типов:

https://t.me/javalib

```
jshell> var ints = List.of(2, 3, 5, 7);
ints ==> [2, 3, 5, 7]

jshell> ints.getClass();
$2 ==> class java.util.ImmutableCollections$ListN
```

Эти классы — новые реализации интерфейсов коллекций Java, которые являются неизменяемыми. Не путайте их со знакомыми изменяемыми классами, такими как `ArrayList` и `HashMap`. При попытке изменить экземпляры этих типов возбуждается исключение.

1.5.2. Удаление корпоративных модулей (JEP 320)

Долгое время в стандартном выпуске Java (Java SE) был ряд модулей, которые на самом деле относились к корпоративному выпуску (Java EE), в том числе:

- JAXB
- JAX-WS
- CORBA
- JTA

В Java 9 следующие пакеты, реализовавшие эти технологии, были перемещены в неосновные модули и объявлены устаревшими, чтобы их можно было удалить:

- `java.activation` (JAF)
- `java.corba` (CORBA)
- `java.transaction` (JTA)
- `java.xml.bind` (JAXB)
- `java.xml.ws` (JAX-WS, а также некоторые сопутствующие технологии)
- `java.xml.ws.annotation` (Common Annotations)

В ходе дальнейшей оптимизации платформы в Java 11 эти модули были удалены. Также из основного дистрибутива Java SE были исключены следующие три модуля со вспомогательными инструментами и механизмом агрегирования:

- `java.se.ee` (модуль-агрегатор для шести модулей, перечисленных выше)
- `jdk.xml.ws` (инструменты для JAX-WS)
- `jdk.xml.bind` (инструменты для JAXB)

Если в проектах, созданных в Java 11 и выше, приходится пользоваться этими возможностями, то требуется подключить явную внешнюю зависимость. Это означает, что некоторые программы, зависящие от этих API, не испытывают проблем со сборкой в Java 8, но в Java 11 сценарий их сборки приходится модифицировать. Эта проблема подробнее рассматривается в главе 11.

https://t.me/javalib

1.5.3. HTTP/2 (Java 11)

Относительно недавно была опубликована спецификация HTTP/2 — новая версия стандарта HTTP. Сначала мы разберемся, зачем понадобилось обновлять почтенную спецификацию HTTP/1.1, которая появилась еще в 1997 году. Затем вы увидите, как Java 11 предоставляет опытным разработчикам доступ к новым возможностям и быстродействию HTTP/2.

Как и следовало ожидать от технологии 1997 года, HTTP/1.1 начал проявлять признаки устаревания, особенно по части быстродействия в современных веб-приложениях. Вот некоторые из его недостатков:

- блокировка начала очереди;
- ограничение количества подключений к одному сайту;
- затраты ресурсов на управляющие заголовки HTTP.

HTTP/2 — это обновление протокола на *транспортном* уровне, и оно призвано исправить фундаментальные проблемы быстродействия, которые не согласуются с реалиями современных веб-приложений. HTTP/2 улучшает быстродействие на уровне передачи байтов между клиентом и сервером, не изменяя при этом многие знакомые концепции HTTP: запрос/ответ, заголовки, коды состояний, тело ответа, — на семантическом уровне все это остается таким же, как в HTTP/1.1.

Блокировка начала очереди

Обмен данными по протоколу HTTP происходит через сокеты TCP. Хотя HTTP/1.1 по умолчанию повторно использует отдельные сокеты, чтобы не дублировать затраты на инициализацию, тем не менее этот протокол требует, чтобы ответы возвращались в том же порядке, в каком поступали запросы, даже если несколько запросов используют один сокет (это называется *конвейеризацией* (pipelining); рис. 1.4). Это значит, что медленный ответ от сервера блокирует последующие запросы, которые теоретически можно было обработать быстрее. Например, этот эффект наглядно проявляется, когда браузер приостанавливает прорисовку страницы, пока загружает статические ресурсы. Этот же фактор (в каждый момент времени поступает только по одному ответу на каждое подключение) ограничивает взаимодействие JVM со службами на основе HTTP.

Протокол HTTP/2 с самого начала проектировался так, чтобы мультиплексировать запросы по одному и тому же подключению, как показано на рис. 1.5. Протокол поддерживает множественные *потоки данных* между клиентом и сервером. Клиент может даже по отдельности получать заголовки и тело одного и того же ответа.

Это решение радикально изменило представления, на которые десятилетиями опирались разработчики, привыкшие к HTTP/1.1. Например, было принято считать, что если сайт возвращает множество мелких ресурсов, это менее эффективно, чем передавать данные большими порциями. Для JavaScript,

https://t.me/javalib

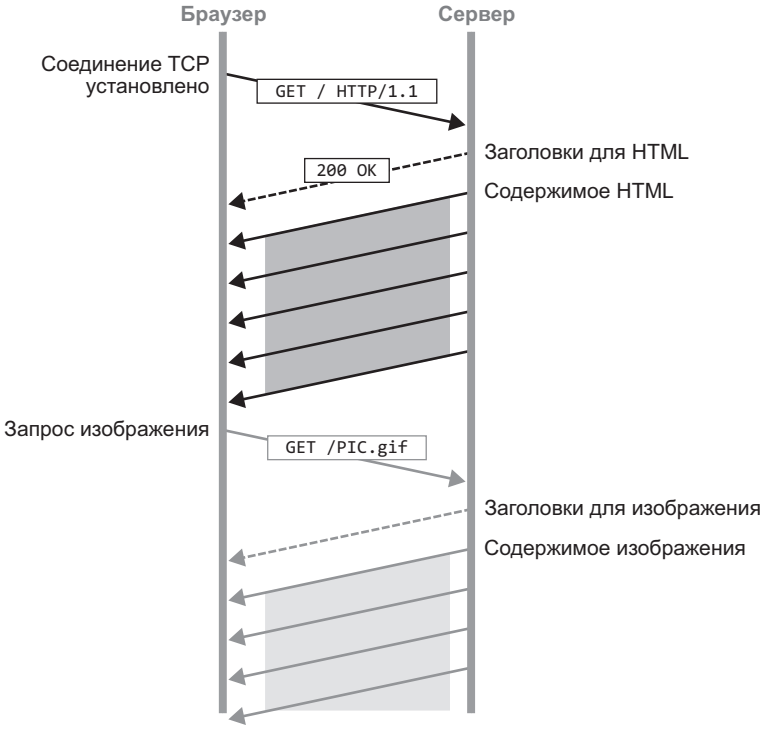


Рис. 1.4. Передача данных по HTTP/1.1

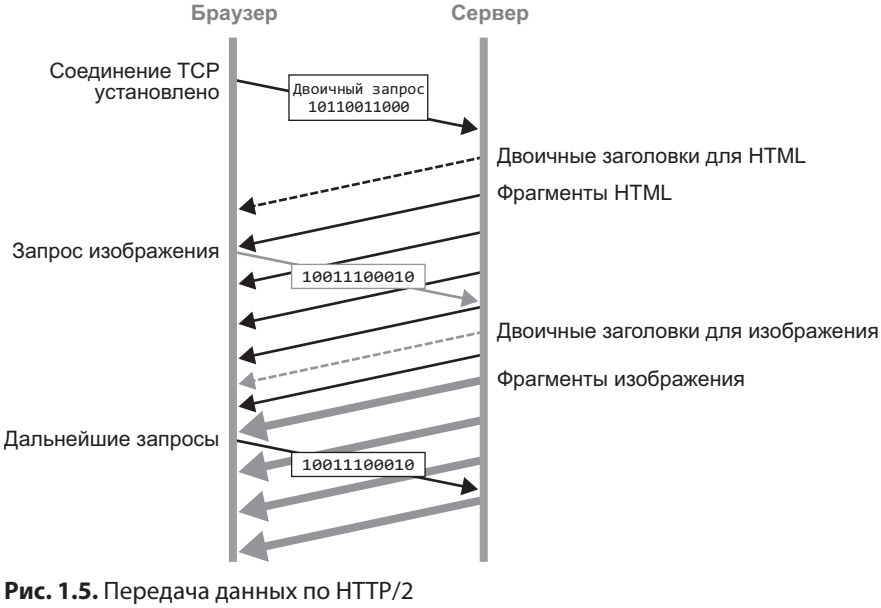


Рис. 1.5. Передача данных по HTTP/2

https://t.me/javalib

CSS и изображений появились стандартные инструменты и приемы, которые объединяют небольшие файлы в пакеты, чтобы передавать их более эффективно. Но в HTTP/2 благодаря мультиплексированию ответов ресурсы не блокируются другими медленными запросами, а меньшие ответы лучше кэшируются, что в целом улучшает опыт работы с приложением.

Ограниченные подключения

Спецификация HTTP/1.1 рекомендует допускать не более двух параллельных подключений к серверу. Это рекомендация, а не жесткое требование, и современные браузеры часто поддерживают 6–8 подключений на домен. Из-за этого ограничения на параллельные загрузки разработчики часто предоставляли сайты с разных доменов или настраивали механизмы слияния загружаемых ресурсов вроде упоминавшихся ранее.

HTTP/2 исправляет эту ситуацию: через каждое подключение можно фактически передавать сколько угодно запросов одновременно. Браузеры открывают только одно подключение к каждому домену, но могут параллельно выполнять множество запросов через это подключение.

В случае HTTP/1.1 в приложениях для JVM можно было создавать пулы подключений, чтобы улучшить параллелизм, однако HTTP/2 органически позволяет вместить больше запросов в одно соединение.

Заголовки HTTP

Отправка *заголовков* вместе с запросами — одна из важнейших возможностей HTTP. Хотя протокол HTTP сам по себе не сохраняет состояния, именно благодаря заголовкам приложения могут поддерживать состояние между запросами — например, информацию о том, что пользователь успешно ввел регистрационные данные.

Хотя тело данных HTTP/1.1 может сжиматься, если клиент и сервер согласуют алгоритм сжатия (обычно gzip), заголовки не сжимаются. По мере того как полнофункциональные веб-приложения инициируют все больше запросов, повторяющиеся заголовки все большего размера могут создать проблемы, особенно для крупных веб-сайтов.

Чтобы решить эту проблему, в HTTP/2 введен новый двоичный формат заголовков. Вам как пользователю протокола не придется глубоко вникать в подробности: это просто встроенный механизм передачи заголовков между клиентом и сервером.

TLS — всему голова

В 1997 году, когда появился HTTP/1.1, интернет был совсем не таким, как сегодня. Коммерция в Сети только делала первые шаги, и при разработке ранних протоколов проблема безопасности не всегда считалась первостепенной. Кроме

https://t.me/javalib

того, вычислительные системы были довольно медленными, отчего шифрование часто обходилось слишком дорого.

Протокол HTTP/2 был официально принят в 2015 году, когда в интернете уже намного серьезнее относились к безопасности. Кроме того, вычислительная нагрузка на повсеместное шифрование веб-запросов через TLS (ранее известный как SSL) существенно снизилась, отчего сошли на нет споры о том, нужно ли шифровать трафик. Как следствие, на практике HTTP/2 работает только с шифрованием TLS. (Теоретически протокол позволяет передавать данные в виде простого текста, но ни одна из основных реализаций не поддерживает эту возможность.)

Это обстоятельство поначалу затрудняло распространение HTTP/2, потому что для этого протокола требовались сертификаты, чей цикл предусматривает истечение срока действия и возобновление. В корпоративных средах это требует больше усилий по управлению сертификатами. В ответ на эту потребность развиваются Let's Encrypt (https://www.letsencrypt.org) и другие решения конфиденциальности.

Другие факторы

Хотя интернет постепенно переходит на HTTP/2, этот протокол распространяется не слишком быстро. Помимо требований к шифрованию, которые влияют даже на локальную разработку, причинами задержки могут быть такие факторы:

- HTTP/2 — чисто двоичный протокол, а с непрозрачным форматом труднее работать.
- Для поддержки HTTP/2 требуется обновлять такие службы уровня HTTP, как распределители нагрузки, брандмауэры и средства отладки.
- Быстродействие улучшается в основном при использовании HTTP в браузерах. Для серверных служб, которые работают по протоколу HTTP, этот выигрыш будет менее заметным.

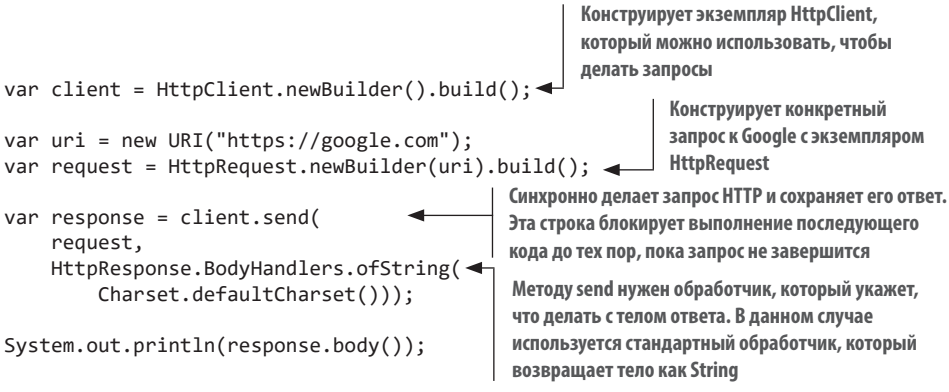
HTTP/2 в Java 11

Когда по прошествии стольких лет появилась новая версия HTTP, это стимулировало авторов JEP 110 ввести совершенно новый API. В JDK это означает замену (но не удаление) `HttpURLConnection`, чтобы предоставить практичный HTTP API «из коробки», потому что раньше многие разработчики решали свои задачи, связанные с HTTP, с помощью внешних библиотек.

Новый API, совместимый с HTTP/2 и веб-сокетами, впервые появился в Java 9 как инкубационная функциональность. В JEP 321 он переместился на свое постоянное место обитания в Java 11 — в `java.net.http`. Новый API поддерживает как HTTP/1.1, так и HTTP/2 и может переключиться на HTTP/1.1, если целевой сервер не поддерживает HTTP/2.

https://t.me/javalib

В новом API взаимодействия начинаются с типов `HttpRequest` и `HttpClient`. Их экземпляры создаются построителями (builders), а конфигурация настраивается перед тем, как отправлять фактический вызов HTTP, как показано в следующем примере:




```
var client = HttpClient.newBuilder().build();

var uri = new URI("https://google.com");
var request = HttpRequest.newBuilder(uri).build();

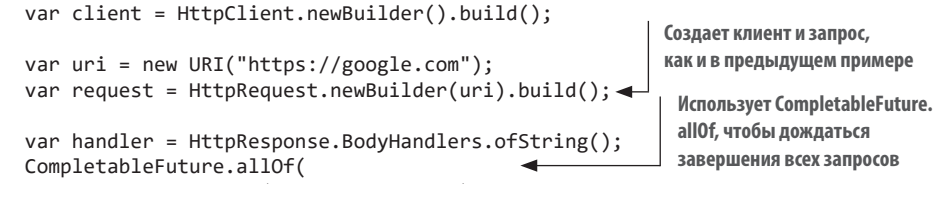
var response = client.send(
    request,
    HttpResponse.BodyHandlers.ofString(
        Charset.defaultCharset()));

System.out.println(response.body());
```

Здесь мы продемонстрировали синхронное использование API. После того как созданы запрос и клиент, мы инициируем вызов HTTP методом `send`. Мы получим объект `response` только после того, как завершится весь вызов HTTP, аналогично тому, как это происходило с более старыми HTTP API из JDK.

Первый параметр — это наш запрос, но на второй параметр стоит взглянуть более пристально. Вместо того чтобы всегда рассчитывать на какой-либо один возвращаемый тип, метод `send` ожидает, что мы предоставим реализацию интерфейса `HttpResponse.BodyHandler<T>`, которая укажет, как поступать с ответом. В классе `HttpResponse.BodyHandlers` есть полезные элементарные обработчики для того, чтобы получить ответ в виде массива байтов, строки или файла. Однако настройка этого поведения сводится к простой реализации `BodyHandler`. Вся эта вспомогательная конструкция опирается на механизмы публикатора и подписчика из `java.util.concurrent.Flow`; такие конструкции обычно называются *реактивными потоками* (reactive streams).

Одно из самых значительных преимуществ HTTP/2 — встроенное мультиплексирование. Однако `send` в синхронном режиме не позволяет в полной мере реализовать эту возможность, так что неудивительно, что `HttpClient` также поддерживает метод `sendAsync`, который возвращает объект `CompletableFuture` — обертку для `HttpResponse` с богатым набором возможностей, которые могут быть знакомы вам по опыту работы с другими компонентами платформы:




```
var client = HttpClient.newBuilder().build();

var uri = new URI("https://google.com");
var request = HttpRequest.newBuilder(uri).build();

var handler = HttpResponse.BodyHandlers.ofString();
CompletableFuture.allOf(
```

https://t.me/javalib

```
    client.sendAsync(request, handler)
        .thenAccept((resp) ->
                        System.out.println(resp.body()),
    client.sendAsync(request, handler)
        .thenAccept((resp) ->
                        System.out.println(resp.body()),
    client.sendAsync(request, handler)
        .thenAccept((resp) ->
                        System.out.println(resp.body())
).join();
```

sendAsync вызывает запрос HTTP, но возвращает преднамеченный объект (future) и не блокирует выполнение кода

Когда преднамеченный объект (future) завершается, мы получаем ответ с помощью thenAccept

Один и тот же клиент можно использовать, чтобы одновременно отправлять много запросов

На этот раз мы снова создаем запрос и клиент, но затем вызов асинхронно повторяется трижды. `CompletableFuture.allOf` объединяет все три преднамеченных объекта (futures), чтобы с помощью одного вызова `join` можно было дождаться, пока они все завершатся.

Здесь мы крайне поверхностно рассмотрели две основные точки входа этого API. В нем есть множество возможностей и настроек: от конфигурации таймаутов и TLS до расширенных асинхронных возможностей, например приема активных передач (push) от серверов HTTP/2 с помощью `HttpResponse.PushPromiseHandler`.

Новый HTTP API в JDK, опирающийся на преднамеченные объекты и реактивные потоки, может служить привлекательной альтернативой большим библиотекам для работы с HTTP, которые прежде доминировали в экосистеме. Модуль `java.net.http`, спроектированный с прицелом на современное асинхронное программирование, поможет языку Java оставаться на передовых позициях, как бы интернет ни развивался дальше.

1.5.4. Однофайловое выполнение (JEP 330)

Чтобы выполнить программу на Java, обычно нужно откомпилировать исходный код в файл класса, а затем запустить процесс виртуальной машины, который действует как исполнительный контейнер и интерпретирует байт-код класса.

Эта схема сильно отличается от языков вроде Python, Ruby и Perl, где исходный код программы интерпретируется напрямую. Такие *языки сценариев* имеют долгую историю в среде Unix, но язык Java традиционно не принадлежал к этой категории.

В Java 11 внедрен JEP 330, который предлагает новый способ выполнения программ. Исходный код может компилироваться в памяти, а затем интерпретатор выполняет его, даже не создавая файл .class на диске, как показано на рис. 1.6.

С точки зрения пользователя эта схема работает примерно так же, как в Python и других языках сценариев.

https://t.me/javalib

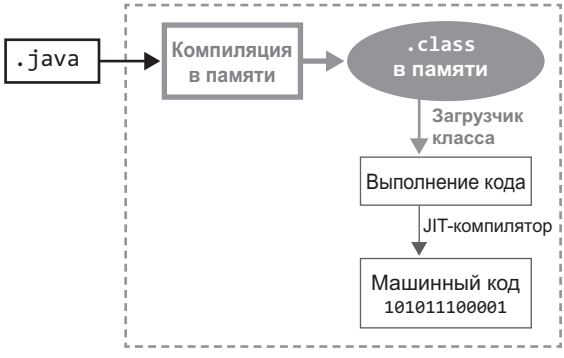


Рис. 1.6. Однофайловое выполнение

У такого режима есть ряд ограничений:

- Весь код должен находиться в одном исходном файле.
- Во время запуска не могут компилироваться дополнительные исходные файлы.
- В исходном файле может быть сколько угодно классов.
- Первый класс должен быть объявлен в исходном файле как точка входа.
- В этом классе должен быть определен метод `main`.

Также в этой схеме используется флаг `--source`, который обозначает режим совместимости исходного кода — фактически это версия языка сценария.

При выполнении нужно соблюдать принципы именования файлов Java, так что имя класса должно соответствовать имени файла. Тем не менее расширение .java использовать *не следует*, потому что это может сбить с толку подсистему запуска.

Такие сценарии Java также могут содержать строку с признаком начала сценария (shebang):

```
#!/usr/bin/java --source 11
public final class HTTP2Check {
    public static void main(String[] args) {
        if (args.length < 1) {
            usage();
        }
        // Реализация сторон вызова HTTP...
    }
}
```

Полный код HTTP2Check содержится в репозитории кода, сопровождающем эту книгу

Первая строка предоставляет необходимые параметры, чтобы файл можно было пометить как исполняемый и вызвать напрямую:

```
$ ./HTTP2Check https://www.google.com
https://www.google.com: HTTP_2
```

https://t.me/javalib

Хотя однофайловое выполнение не позволяет полностью перенести в Java опыт работы с языками сценариев, оно может пригодиться, чтобы писать простые и полезные инструменты в традициях Unix, не задействуя другие языки программирования.

ИТОГИ

- Язык Java и платформа Java — два разных (хотя и тесно связанных) компонента экосистемы Java. Платформа поддерживает многие другие языки помимо Java.
- После Java 8 платформа Java перешла на новый процесс периодических выпусков. Новые версии появляются через каждые шесть месяцев, а выпуски с долгосрочной поддержкой (LTS) — раз в два или три года.
- Текущие LTS-версии — 11 и 17, хотя Java 8 все еще поддерживается.
- Из-за того что Java чрезвычайно заботится об обратной совместимости, вносить изменения в язык не всегда просто. Изменения, ограниченные библиотекой или компилятором, обычно менее затратны, чем изменения, для которых нужно обновлять виртуальную машину.
- В Java 11 появилось много полезных возможностей, ради которых стоит перейти со старой версии:
  - Зарезервированное имя типа `var`, которое упрощает определения переменных.
  - Фабричные методы, которые облегчают создание списков, отображений и других коллекций.
  - Новая реализация `HttpClient` с полноценной поддержкой HTTP/2.
  - Однофайловые программы, которые можно запускать напрямую, не компилируя в файлы классов.

https://t.me/javalib

2 Модули Java

В ЭТОЙ ГЛАВЕ

- ✓ Платформенные модули Java
- ✓ Изменения в семантике управления доступом
- ✓ Написание модульных приложений
- ✓ Универсальные файлы JAR

Как упоминалось в главе 1, график выпуска версий Java до Java 9 включительно определялся готовой функциональностью; новые выпуски часто были тесно связаны с той или иной радикально новой возможностью языка.

Для Java 9 такой возможностью стали платформенные модули Java (также известные как JPMS, Jigsaw или просто «модули»). Это фундаментальное улучшение платформы обсуждалось много лет: изначально его планировали включить в Java 7 еще в 2009–2010 годах.

В этой главе мы объясним, почему возникла необходимость в модулях, рассмотрим новый синтаксис, который выражает концепцию модульности, и покажем, как использовать модули в приложениях. Вы научитесь использовать JDK и сторонние модули в своих сборках, а также упаковывать приложения или библиотеки в модули.

ПРИМЕЧАНИЕ Модули — это новый способ упаковывать и развертывать код, и благодаря им приложения становятся качественнее. Однако если вы только начинаете

https://t.me/javalib

использовать современные возможности Java (появившиеся в версиях 11 или 17), вам не обязательно задействовать модули прямо сейчас, пока вы не чувствуете в них острой необходимости.

Появление модулей существенно повлияло на архитектуру приложений. В частности, в современных проектах модули помогают оптимизировать потребление памяти процесса, затраты на запуск проекта и время разогрева. Благодаря модулям также удается решить так называемую проблему «ада JAR», от которой часто страдают приложения на Java со сложными зависимостями. Давайте познакомимся с модулями поближе.

2.1. МОДУЛИ JAVA: ОБЩАЯ КАРТИНА

С выходом Java 9 *модуль* стал принципиально новой концепцией в языке Java. Модуль как единица развертывания приложений и управления зависимостями имеет семантическое значение для среды выполнения. До появления модулей в Java были характерные проблемы:

- Файлы JAR невидимы для среды выполнения: фактически они представляют собой архивированные каталоги с файлами классов.
- Пакеты — это просто пространства имен, которые группируют классы для управления доступом.
- Зависимости определяются только на уровне класса.
- Управление доступом в сочетании с рефлексией порождает открытую по своей сути систему, где у единиц развертывания нет четких границ, а вся конструкция недостаточно хорошо контролируется.

С модулями многое меняется:

- Определяются зависимости между модулями, чтобы все проблемы с разрешением и компоновкой можно было обнаружить при компиляции или при запуске приложения.
- Модули обеспечивают корректную инкапсуляцию, которая защищает внутренние пакеты и классы от дотошных пользователей, желающих покопаться в них.
- Каждый модуль — это полноценная единица развертывания с метаданными, которые современная среда выполнения Java умеет воспринимать и потреблять и которые отражаются в системе типов Java (например, с помощью рефлексии).

ПРИМЕЧАНИЕ До появления модулей в базовом языке и среде выполнения не было сводных метаданных о зависимостях. Эти метаданные определялись только в системах сборки (таких, как Maven) или в сторонних модульных системах (например, OSGI или JBoss), о которых JVM понятия не имеет.

https://t.me/javalib

Платформенные модули Java воплощают «недостающее звено», которого не хватало в мире Java до версии 8 включительно.

> **ПРИМЕЧАНИЕ** Модули Java часто упаковываются в специальные файлы JAR, но они не привязаны к этому формату, и позже мы представим другие возможные форматы.

Система модулей создавалась для того, чтобы единицы развертывания (модули) как можно меньше зависели друг от друга. Основная идея состояла в том, чтобы модули можно было загружать и компоновать по отдельности, хотя на практике приложения могут зависеть от группы модулей, которые предоставляют смежную функциональность (например, средства безопасности).

2.1.1. Проект Jigsaw

Проект по добавлению функциональности модулей в OpenJDK был известен как «проект Jigsaw». Его задачей было предоставить полноценное решение модульности, которое преследовало такие цели:

- модуляризовать исходный код платформы JDK;
- сократить потребление памяти процессами;
- ускорить запуск приложений;
- сделать модули доступными как для JDK, так и для кода приложений;
- впервые в истории Java обеспечить полноценную строгую инкапсуляцию;
- добавить в язык Java новые, ранее невозможные режимы управления доступом.

В свою очередь, эти цели были обусловлены другими целями, которые сильнее ориентированы на среду выполнения Java и JDK:

- отказаться от одного монолитного файла JAR среды выполнения (rt.jar);
- эффективно инкапсулировать и обезопасить внутреннюю структуру JDK;
- обеспечить возможность вносить серьезные внутренние изменения, в том числе те, которые блокируют несанкционированное использование вне JDK;
- внедрить модули как «суперпакеты».

Эти вторичные цели могут потребовать более подробных объяснений, потому что они в первую очередь касаются внутреннего устройства платформы и ее реализации.

Переход от монолитной среды выполнения Java к модульной

Старый формат JAR фактически представляет собой zip-файл с классами. Он применялся с первых дней существования самóй платформы и совершенно не оптимизирован для классов и приложений Java. Отказ от формата JAR для

https://t.me/javalib

классов платформы может принести немало преимуществ, например значительно ускорить процесс запуска.

Модули предоставляют два новых формата — *JMOD* и *JIMAGE*, — которые используются на разных этапах жизненного цикла программы, а именно во время компиляции/компоновки и во время выполнения соответственно.

Формат JMOD во многом похож на JAR, но отличается, в частности, тем, что в него можно включать низкоуровневый код как часть единого файла, вместо того чтобы прикладывать файл с отдельным совместно используемым объектом, как делалось в Java 8. Для большинства задач разработчиков (в том числе для публикации модулей в Maven) лучше упаковывать собственные модули в модульные файлы JAR, чем в JMOD.

Формат JIMAGE представляет образ среды выполнения Java. До Java 8 существовали только два возможных образа среды выполнения — JDK и JRE, — но это в основном было обусловлено историческими причинами. В Java 8 компания Oracle ввела *серверные JRE* (а также *компактные профили*) как промежуточную ступень на пути к полной модульности. По сути, эти образы отбрасывали часть функциональности (например, GUI-фреймворки), чтобы меньше расходовать память, что было особенно актуально для приложений на стороне сервера.

Модульное приложение содержит достаточно метаданных, чтобы точный набор зависимостей был известен до запуска программы. Это позволяет загружать только то, что действительно необходимо, и такой подход намного эффективнее. Можно пойти еще дальше и определить *специально подготовленный образ среды выполнения*, который будет поставляться вместе с приложением и содержать не полную установку Java общего назначения, а только то, что нужно приложению. Мы продемонстрируем эту возможность в конце главы, когда будем рассматривать инструмент `jlink`.

А пока давайте займемся программой `jimage`, которая выводит подробную информацию об образе среды выполнения Java. Например, для полной среды Java 15 (то есть того, что раньше содержалось в JDK) вывод программы выглядит так:

```
$ jimage info $JAVA_HOME/lib/modules
Major Version: 1
Minor Version: 0
Flags: 0
Resource Count: 32780
Table Length: 32780
Offsets Size: 131120
Redirects Size: 131120
Locations Size: 680101
Strings Size: 746471
Index Size: 1688840
```

Или так:

```
$ jimage list $JAVA_HOME/lib/modules
jimage: /Library/Java/JavaVirtualMachines/java15/Contents/Home/lib/modules
```

https://t.me/javalib

```
Module: java.base
    META-INF/services/java.nio.file.spi.FileSystemProvider
    apple/security/AppleProvider$1.class
    apple/security/AppleProvider$ProviderService.class
    apple/security/AppleProvider.class
    apple/security/KeychainStore$CertKeychainItemPair.class
    apple/security/KeychainStore$KeyEntry.class
    apple/security/KeychainStore$TrustedCertEntry.class
    apple/security/KeychainStore.class
    com/sun/crypto/provider/AESCipher$AES128_CBC_NoPadding.class
    ... и еще очень много строк
```

Отказ от монолитного rt.jar позволяет ускорить запуск и оптимизировать среду только для тех компонентов, которые необходимы приложению. Новые форматы проектировались как непрозрачные для разработчиков, и они зависят от реализации. Теперь нельзя просто распаковать rt.jar и получить библиотеку классов JDK. Впрочем, это лишь один шаг на пути к тому, чтобы закрыть внутреннюю структуру платформы от программистов на Java, что было одной из целей системы модулей.

Инкапсуляция внутреннего устройства

Контракт между платформой Java и ее пользователями всегда задумывался как контракт API: обратная совместимость должна обеспечиваться на уровне интерфейса, а не на уровне внутреннего устройства реализации.

Однако программисты на Java не соблюдали свою часть контракта и со временем все больше эксплуатировали особенности реализации платформы, которые не предназначались для общедоступного использования.

Это было нежелательно, потому что разработчики платформы OpenJDK хотели иметь возможность свободно изменять реализацию JVM и классы платформы и модернизировать их, чтобы предоставлять новую функциональность и улучшать быстродействие, не беспокоясь о том, что работоспособность пользовательских приложений может нарушиться.

Вносить критические изменения во внутреннее устройство платформы мешало одно важное препятствие — подход Java к управлению доступом, существовавший в Java 8. В Java определены только уровни доступа `public`, `private`, `protected` и `package-private`, и эти модификаторы применяются лишь на уровне класса и ниже.

Эти ограничения можно обойти разными способами: например, применяя рефлексию или создавая дополнительные классы в соответствующих пакетах, — так что внутреннее устройство никак не удается полностью защитить «от дурака» (или от эксперта).

Обходные решения, с помощью которых разработчики получали доступ ко внутреннему устройству платформы, исторически нередко бывали оправданными. Однако по мере того, как платформа развивалась, появлялись официальные

https://t.me/javalib

способы обратиться почти ко всей нужной функциональности. Таким образом, легкий доступ к «технической изнанке» мешал развивать платформу, не давая никаких преимуществ, и модульность помогала бороться с этой унаследованной проблемой.

Подведем итог. Проект Jigsaw старался решить сразу несколько проблем, прежде всего уменьшить размер среды выполнения, сократить время запуска и упорядочить зависимости между внутренними пакетами. Все эти проблемы было очень трудно (а то и нереально) решать постепенно. Возможности для подобных фундаментальных улучшений возникают нечасто, особенно в зрелых программных платформах, поэтому команда Jigsaw решила воспользоваться обстоятельствами.

JVM стала модульной

Чтобы убедиться в этом, рассмотрим очень простую программу:

```
public class StackTraceDemo {
    public static void main(String[] args) {
        var i = Integer.parseInt("Ошибка");
    }
}
```

Если скомпилировать и запустить этот код, возбудится исключение времени выполнения:

```
$ java StackTraceDemo
Exception in thread "main" java.lang.NumberFormatException:
  For input string: "Ошибка"
    at java.base/java.lang.NumberFormatException.forInputString(
    NumberFormatException.java:65)
    at java.base/java.lang.Integer.parseInt(Integer.java:652)
    at java.base/java.lang.Integer.parseInt(Integer.java:770)
    at StackTraceDemo.main(StackTraceDemo.java:3)
```

Однако из этого примера ясно видно, что формат трассировки стека несколько изменился по сравнению с Java 8. В частности, кадры стека теперь обозначаются именем модуля (`java.base`), а также именем пакета, именем класса и номером строки. Это демонстрирует, что модульная природа платформы проявляется повсеместно и отражается даже в самой простой программе.

2.1.2. Граф модулей

Краеугольным камнем всей модульности является *граф модулей* — представление того, как одни модули зависят от других. Модули явно выражают свои зависимости с помощью нового синтаксиса, и эти зависимости служат твердыми гарантиями, на которые могут положиться компилятор и среда выполнения. Очень важно, что граф модулей должен быть *направленным ациклическим графом* (НАГ, или DAG — directed acyclic graph), а следовательно, не может содержать циклических зависимостей.

https://t.me/javalib

ПРИМЕЧАНИЕ Важно понимать, что в современных средах Java все приложения работают поверх модульной JRE; не существует никакого «модульного режима» или «унаследованного режима с classpath».

Не каждому разработчику нужно становиться экспертом в системе модулей, но каждый опытный программист на Java получит преимущества, если будет хорошо разбираться в новой подсистеме, которая изменила принципы выполнения всех программ на JVM. На рис. 2.1 изображена элементарная схема системы модулей в том виде, в каком с ней сталкивается большинство разработчиков.

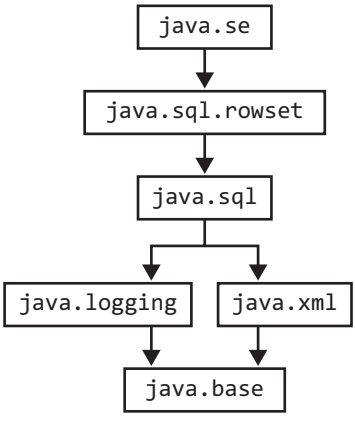


Рис. 2.1. Системные модули JDK (упрощенное представление)

Здесь показано упрощенное представление некоторых основных модулей в JDK. Обратите внимание, что модуль `java.base` всегда входит в набор зависимостей любого другого модуля. На диаграммах графов модулей зависимость от `java.base` часто не изображается явно, чтобы уменьшить визуальный шум.

Ясный и относительно простой набор модулей, который мы видим на рис. 2.1, стоит сравнить с тем, как выглядел JDK в Java 8. До появления модулей высокоуровневой единицей кода в Java был пакет, а в стандартной среде исполнения Java 8 их насчитывалось почти тысяча. Изобразить их все было бы нереально, а зависимости на графе получились бы настолько запутанными, что никто не смог бы в них разобраться.

Преобразовать прежнюю структуру JDK в нынешнюю стройную форму было непросто, и путь к модуляризации JDK был долгим. Версия Java 9 вышла в сентябре 2017 года, но разработка модульной функциональности началась несколькими годами ранее, до выпуска Java 8. Необходимыми подготовительными шагами для внедрения модулей были несколько промежуточных целей, включая следующие:

- модуляризировать композицию исходного кода в JDK (JEP 201);
- модуляризировать структуру образов среды выполнения (JEP 220);
- распутать сложные зависимости реализаций между пакетами JDK.

https://t.me/javalib

Несмотря на то что готовая функциональность модулей не была опубликована до Java 9, бóльшáя часть подготовительной работы вошла в Java 8, что даже позволило включить в этот выпуск функциональность *компактных профилей*, о которых будет рассказано в конце главы.

2.1.3. Защита внутреннего устройства

Одна из главных проблем, которую должны были решить модули, заключалась в том, что пользовательские фреймворки Java были чересчур тщательно приспособлены к особенностям внутренней реализации. Например, следующий фрагмент кода на Java 8 расширяет внутренний класс, чтобы получить доступ к низкоуровневому преобразователю URL в каноническую форму.

Этот код приводится только для демонстрационных целей, чтобы мы могли обсуждать модули и управление доступом на конкретном примере. Вам никогда не следует в собственном коде обращаться к внутренним классам напрямую:

```
import sun.net.URLCanonicalizer;

public class MyURLHandler extends URLCanonicalizer {

    public boolean isSimple(String url) {
        return isSimpleHostName(url);
    }
}
```

Здесь преобразователь URL — это фрагмент кода, который получает адрес в одном из различных форматов, разрешенных стандартом URL, и преобразует его в стандартную (каноническую) форму. Предполагается, что канонический URL может служить единым источником истины относительно адреса контента, к которому можно обращаться по разным URL. При попытке откомпилировать код в Java 8 `javac` предупредит, что вы обращаетесь к внутреннему API:

```
$ javac MyURLHandler.java
MyURLHandler.java:1: warning: URLCanonicalizer is internal proprietary API
  and may be removed in a future release[1]

import sun.net.URLCanonicalizer;
              ^
MyURLHandler.java:3: warning: URLCanonicalizer is internal proprietary API
  and may be removed in a future release

public class MyURLHandler extends URLCanonicalizer {
                                  ^
2 warnings
```

Однако по умолчанию компилятор не запрещает доступ к этому API, и в результате получается пользовательский класс, который тесно сопряжен с внутренней

[1] Предупреждение: `URLCanonicalizer` — внутренний проприетарный API, и он может быть удален в будущих выпусках. — *Примеч. пер.*

https://t.me/javalib

реализацией JDK. Это сопряжение слишком хрупкое: оно разрушится, если вызываемый код будет перемещен или заменен.

Если разработчики будут массово злоупотреблять этой открытостью, это приведет к ситуации, когда вносить изменения во внутреннее устройство платформы станет трудно или невозможно, потому что они нарушат работоспособность ранее развернутых библиотек или приложений.

> **ПРИМЕЧАНИЕ** Класс URLCanonicalizer приходится вызывать из нескольких разных пакетов, а не только из своего, поэтому он должен быть открытым (public) классом, а не пакетно-доступным (package private).

Чтобы решить эту весьма глобальную проблему, пришлось внести однократное изменение в модель управления доступом Java. Оно применимо как к пользовательскому коду, который обращается к JDK, так и к приложениям, вызывающим сторонние библиотеки.

2.1.4. Новая семантика управления доступом

Модули добавили в модель управления доступом Java новую концепцию — *экспортирование* пакетов. В Java 8 и ранее код любого пакета мог вызывать открытые методы любого открытого класса в любом пакете. Иногда это называется «защитой дробовика» (shotgun privacy) по мотивам знаменитой цитаты о другом языке программирования:

> Perl не увлекается принудительной конфиденциальностью. Он предпочитает, чтобы вы не заходили к нему в комнату потому, что вас туда не приглашали, а не потому, что у него есть дробовик (Ларри Уолл).

Однако для Java «защита дробовика» создавала серьезную проблему. Все больше библиотек использовали внутренние API, чтобы обеспечить функциональность, которую было трудно или невозможно предоставить другим способом, и это ставило под угрозу жизнеспособность платформы в долгосрочной перспективе.

В Java 8 было невозможно принудительно управлять доступом в пределах целого пакета. Это означало, что команда JDK не могла определить открытый API и при этом гарантировать, что его клиенты не смогут обойти его или напрямую обратиться к внутренней реализации.

Соглашение о том, что содержимое пакета образует открытый API, только если его имя начинается с `java` или `javax`, а все остальное считается внутренним, — это именно соглашение. Как мы уже убедились, ни одна виртуальная машина или механизм загрузки классов не заставляют его соблюдать.

Однако с модулями ситуация изменилась. Появилось ключевое слово `exports`, которое указывает, какие пакеты считаются открытым API модуля. В модульном JDK пакет `sun.net` не экспортируется, поэтому приведенный выше код преобразователя URL для Java 8 не будет компилироваться. Вот что произойдет, если вы попытаетесь запустить его в Java 11:

https://t.me/javalib

```
$ javac src/ch02/MyURLHandler.java
src/ch02/MyURLHandler.java:3: error: package sun.net is not visible[1]
import sun.net.URLCanonicalizer;
          ^
  (package sun.net is declared in module java.base, which does not export
     it to the unnamed module)[2]
src/ch02/MyURLHandler.java:8: error: cannot find symbol[3]
        return isSimpleHostName(url);
               ^
  symbol: method isSimpleHostName(String)
  location: class MyURLHandler
2 errors
```

Обратите внимание: сама форма сообщения об ошибке явно указывает, что пакет `sun.net` стал невидимым — компилятор даже не распознает символическое имя `isSimpleHostName`. Это стало фундаментальным изменением в том, как устроено управление доступом в Java. Теперь доступны только методы экспортированных пакетов. Открытый метод открытого класса больше не становится автоматически видимым всему остальному коду.

Впрочем, многие разработчики могут не заметить этого изменения. Если вы программируете, добросовестно следуя правилам, то вы никогда не вызываете API внутренних пакетов напрямую. Однако не исключено, что вы используете библиотеку или фреймворк, который так поступает, поэтому будет полезно понять, что же реально изменилось, чтобы не пришлось теряться в догадках.

> **ПРИМЕЧАНИЕ** Полноценная инкапсуляция связана с определенными ограничениями, а до появления модулей система Java была весьма открытой. Так что вполне естественно, что при столкновении с более структурированной системой, которую обеспечивают модули, многим Java-разработчикам кажется, что дополнительная защита их стесняет или раздражает. Давайте рассмотрим синтаксис, который воплощает новую семантику модулей Java.

2.2. ОСНОВЫ СИНТАКСИСА МОДУЛЕЙ

Модуль платформы Java определяется как концептуальная единица — набор пакетов и классов, которые объявляются и загружаются как единое целое. В каждом модуле должен быть объявлен новый файл — *дескриптор модуля*, который называется module-info.java и содержит следующую информацию:

- Имя модуля.
- Зависимости модуля.
- Открытый API (экспортируемые пакеты).

---

[1] Ошибка: пакет `sun.net` не входит в область видимости. — *Примеч. пер.*

[2] Пакет `sun.net` объявлен в модуле `java.base`, который не экспортирует его в анонимный модуль. — *Примеч. пер.*

[3] Ошибка: не удается найти символическое имя. — *Примеч. пер.*

https://t.me/javalib

- Разрешения доступа при рефлексии.
- Предоставляемые службы.
- Потребляемые службы.

Этот файл должен находиться в подходящем месте в иерархии исходного кода. Например, в структуре в стиле Maven модуль с полным именем `wgjd.discovery` размещается непосредственно в каталоге src/main/java и содержит файл module-info.java и корневой каталог пакета:

```
src
  └── main
      └── java
          └── wgjd.discovery
              ├── wgjd
              │   └── discovery
              │       ├── internal
              │       │   ├── AttachOutput.java
              │       │   └── PlainAttachOutput.java
              │       ├── VMIntrospector.java
              │       └── Discovery.java
              └── module-info.java
```

Конечно, эта структура несколько отличается от немодульных проектов Java, где src/main/java часто назначается корневым каталогом пакетов. Тем не менее внутри корневого каталога модуля по-прежнему можно видеть знакомую иерархическую структуру пакетов.

> **ПРИМЕЧАНИЕ** При сборке модульного проекта дескриптор модуля будет скомпилирован в файл класса module-info.class, но этот файл, несмотря на свое имя, сильно отличается от обычных файлов классов, которые мы привыкли видеть на платформе Java.

В этой главе мы рассмотрим основные директивы дескриптора, но не будем глубоко изучать все возможности, которые предоставляют модули. В частности, здесь не обсуждаются аспекты модулей, связанные со службами.

Простой пример дескриптора модуля выглядит так:

```
module wgjd.discovery {
  exports wgjd.discovery;

  requires java.instrument;
  requires jdk.attach;
  requires jdk.internal.jvmstat;
}
```

Дескриптор содержит три новых ключевых слова: `module`, `exports` и `requires`, а его синтаксис должен показаться привычным большинству программистов на Java. Ключевое слово `module` просто открывает область видимости объявления.

https://t.me/javalib

ПРИМЕЧАНИЕ Имя module-info.java похоже на package-info.java, и эти файлы действительно отчасти связаны. Так как среда выполнения не видит пакетов как таковых, понадобилось обходное решение, чтобы подключить метаданные, которые должны применяться ко всему пакету. Таким решением стал файл package-info.java. С модулем может быть связано намного больше метаданных, поэтому было выбрано похожее имя. Новый синтаксис состоит из *ограниченных ключевых слов*, которые описываются в JLS таким образом:

Следующие десять последовательностей символов являются ограниченными ключевыми словами: `open`, `module`, `requires`, `transitive`, `exports`, `opens`, `to`, `uses`, `provides` *и* `with`. *Эти последовательности символов рассматриваются как ключевые слова только там, где они появляются в качестве терминалов в конструкциях* `ModuleDeclaration` *и* `ModuleDirective`.

Проще говоря, это означает, что новые ключевые слова будут считаться таковыми только в дескрипторе для метаданных модуля и не будут ключевыми словами в других местах исходного кода Java. Тем не менее на практике их лучше не использовать в качестве идентификаторов Java, хотя формально это не запрещено. Это похоже на ситуацию с `var`, о которой рассказывалось в главе 1, однако мы предпочтем не страдать педантичностью, так что далее в книге эти конструкции будут называться просто «ключевыми словами».

2.2.1. exports и requires

Ключевое слово `exports` ожидает получить аргумент с именем пакета. В нашем примере

```
exports wgjd.discovery;
```

означает, что модуль `discovery` экспортирует пакет `wgjd.discovery`. Однако никакие другие пакеты в дескрипторе не упоминаются, поэтому `wgjd.discovery.internal` не экспортируется, и обычно к нему нет доступа за пределами модуля `discovery`.

Дескриптор модуля может содержать несколько строк `exports`; более того, это довольно распространенная практика. Для более избирательного экспорта существует синтаксис `exports … to …`, который указывает, что только определенные внешние модули могут обращаться к указанному пакету из данного модуля.

ПРИМЕЧАНИЕ Каждый модуль обычно экспортирует один или несколько пакетов, которые составляют открытый API этого модуля, и код из других модулей может обращаться лишь к этим пакетам, если только не используется переопределение (например, параметр командной строки).

Ключевое слово `requires` объявляет зависимость текущего модуля и всегда должно получать аргумент с именем *модуля*, а не пакета.

Модуль `java.base` содержит самые фундаментальные пакеты и классы среды выполнения Java. Его содержимое можно просмотреть с помощью команды `jmod`:

https://t.me/javalib

```
$ jmod describe $JAVA_HOME/jmods/java.base.jmod
java.base@11.0.3
exports java.io
exports java.lang
exports java.lang.annotation
exports java.lang.invoke
exports java.lang.module
exports java.lang.ref
exports java.lang.reflect
exports java.math
exports java.net
exports java.net.spi
exports java.nio
//  ... и еще очень много строк
```

Эти пакеты используются всеми программами на Java, так что модуль `java.base` всегда является неявной зависимостью каждого модуля, и его не нужно явно объявлять в module-info.java — по аналогии с тем, как `java.lang` неявно импортируется в каждый класс Java.

Вот несколько базовых правил и соглашений для имен модулей:

- Модули находятся в глобальном пространстве имен.
- Имена модулей должны быть уникальными.
- Используйте стандартный формат `com.company.project` там, где это уместно.

С модулями связано такое важное понятие, как *транзитивность*. Давайте познакомимся с ним поближе, потому что транзитивность встречается не только в контексте модулей, но и в более привычных зависимостях библиотек Java (то есть файлах JAR) — об этом пойдет речь в главе 11.

2.2.2. Транзитивность

Транзитивность — общий термин программирования, который относится не только к языку Java. Так называется ситуация, когда компоненту для правильной работы необходимы другие компоненты, а им, в свою очередь, могут потребоваться третьи. В нашем коде эти компоненты «следующего порядка» могут даже не упоминаться, но они все равно должны быть доступны, иначе приложение не будет работать.

Чтобы разобраться, в чем тут дело и почему это важно, рассмотрим два модуля, `A` и `B`, и допустим, что модулю `A` необходим доступ к `B`. Возможны два случая:

- `A` не экспортирует никакие методы, в которых явно упоминаются типы из `B`.
- В API модуля `A` входят типы из `B`.

Если `A` экспортирует методы, которые возвращают типы, определенные в `B`, это приведет к тому, что `A` нельзя будет использовать, если клиенты `A` (модули, которые зависят от `A`) тоже не потребуют `B`. Для клиентов `A` это создаст избыточную нагрузку.

https://t.me/javalib

Чтобы решить эту проблему, система модулей предоставляет простой синтаксис: `requires transitive`. Если модуль `A` зависит от другого модуля транзитивно, то любой код, который зависит от `A`, тоже неявно получит его транзитивные зависимости.

Хотя в реальной практике без `requires transitive` иногда нельзя обойтись, в общем случае при написании модулей лучше всего свести использование транзитивности к минимуму. Мы еще вернемся к транзитивным зависимостям, когда будем обсуждать инструменты сборки в главе 11.

2.3. ЗАГРУЗКА МОДУЛЕЙ

Если вы впервые встретились с темой загрузки классов Java, когда она кратко упоминалась в главе 1, и другого опыта у вас пока нет, не беспокойтесь. Самое важное, что необходимо знать прямо сейчас, — что существуют четыре типа модулей, которые при загрузке ведут себя немного по-разному:

- платформенные (platform) модули;
- прикладные (application) модули;
- автоматические (automatic) модули;
- анонимные (unnamed) модули.

С другой стороны, если вы уже знакомы с загрузкой классов, то имейте в виду, что появление модулей изменило некоторые особенности этой загрузки.

Современные JVM используют загрузчики классов с поддержкой модулей, и классы JRE загружаются совсем не так, как в Java 8. Одним из ключевых понятий является *путь к модулям* (module path) — последовательность путей к модулям (или каталогам, содержащим модули). Это похоже на традиционный для Java путь к классам (classpath), однако не совсем то же самое.

> **ПРИМЕЧАНИЕ** Мы рассмотрим загрузку классов в главе 4, где представим современный подход как для начинающих, так и для опытных читателей.

Фундаментальные принципы модульного подхода к загрузке классов таковы:

- Разрешение модулей выполняется на основании пути к модулям, а не традиционного пути к классам (classpath).
- При запуске JVM обрабатывает граф модулей, который должен быть ациклическим.
- Один модуль находится в корне графа, и с него начинается выполнение. Он содержит класс с методом `main`, который станет точкой входа.

Зависимости, которые уже были модуляризованы, называются *прикладными модулями* и размещаются в пути к модулям. Немодуляризованные зависимости размещаются в традиционном classpath и внедряются в систему модулей через механизм миграции.

https://t.me/javalib

При разрешении модулей используется обход в глубину, а поскольку граф ациклический, алгоритм разрешения завершится, притом за линейное время. Рассмотрим все четыре разновидности модулей подробнее.

2.3.1. Платформенные модули

К этой категории относятся модули самого́ модульного JDK. В Java 8 они входили бы в состав монолитной среды выполнения (rt.jar) или, возможно, были бы вспомогательными файлами JAR, такими как tools.jar. С помощью флага `--list-modules` можно получить список всех доступных платформенных модулей:

```
$ java --list-modules
java.base@11.0.6
java.compiler@11.0.6
...
java.xml@11.0.6
java.xml.crypto@11.0.6
jdk.accessibility@11.0.6
...
jdk.unsupported@11.0.6
...
```

Этот код выводит полный список модулей, а не сокращенный набор, показанный на рис. 2.1.

> **ПРИМЕЧАНИЕ** Точный список модулей и их имен зависит от используемой версии Java. Например, в реализации GraalVM от Oracle могут быть некоторые дополнительные модули: com.oracle.graal.graal_enterprise, org.graalvm.js.scriptengine, org.graalvm.sdk и т. д.

Платформенные модули интенсивно используют механизм *избирательного экспортирования*, когда некоторые пакеты экспортируются только в заданный список модулей и не становятся общедоступными.

Самый важный модуль в дистрибутиве — `java.base` — всегда является неявной зависимостью любого другого модуля. Он содержит `java.lang`, `java.util`, `java.io` и другие базовые пакеты. Фактически этот модуль соответствует минимальной среде выполнения Java, которая необходима, чтобы запустить приложение.

На другом конце спектра находятся *модули-агрегаторы*, которые не содержат никакого кода, но служат вспомогательным механизмом, который позволяет приложениям транзитивно подключать очень широкий набор зависимостей. Например, модуль `java.se` подключает всю платформу Java SE.

2.3.2. Прикладные модули

Сюда относятся модуляризованные зависимости приложения или само приложение. Модули этого типа также иногда называются *библиотечными модулями*.

https://t.me/javalib

ПРИМЕЧАНИЕ Технически платформенные и прикладные модули ничем не отличаются друг от друга; различия носят скорее философский характер. Однако они загружаются разными загрузчиками классов, как будет показано в главе 4.

Сторонние библиотеки, от которых зависит приложение, становятся прикладными модулями. Например, библиотеки Jackson для работы с JSON были модуляризованы в версии 2.10 и считаются прикладными (то есть библиотечными) модулями.

Прикладные модули обычно зависят как от платформенных модулей, так и от других прикладных модулей. Зависимости этих модулей желательно ограничивать, насколько это возможно, и не включать, например, `java.se` в качестве зависимости.

2.3.3. Автоматические модули

У системы модулей есть одна специально реализованная особенность: из модуля нельзя обращаться к classpath. Может показаться, что это ограничение создает проблемы: что делать, если модуль зависит от кода, который еще не модуляризован?

Чтобы решить проблему, немодульный файл JAR перемещается в путь к модулям (и удаляется из classpath). После этого файл JAR становится *автоматическим модулем*. Система модулей автоматически генерирует для него имя, которое основано на имени файла JAR.

Автоматический модуль экспортирует все содержащиеся в нем пакеты и автоматически становится зависимым от всех остальных модулей из пути к модулям. У автоматических модулей нет полноценной информации о зависимостях модулей, потому что они не объявляют свои зависимости явно и не публикуют свои API. Это означает, что они не являются приоритетными сущностями в системе модулей и не обеспечивают такой же уровень гарантий, как полноценные модули Java.

Имя также можно задать явно, добавив запись `Automatic-Module-Name` в файл `MANIFEST.MF` в JAR. Часто это делается как промежуточный шаг при миграции на модули Java, потому что у разработчиков появляется возможность зарезервировать имя модуля и начать пользоваться некоторыми преимуществами взаимодействия с модульным кодом.

Например, на момент написания этой книги библиотека Apache Commons Lang еще не полностью модуляризована, но она указывает `org.apache.commons.lang3` в качестве имени автоматического модуля. В результате другие модули могут объявлять о своей зависимости от этого автоматического модуля, несмотря на то, что его разработчики не завершили переход к полной модульности.

2.3.4. Анонимные модули

Все классы и файлы JAR из classpath добавляются в один модуль — анонимный, или `UNNAMED`. Это делается для обратной совместимости, но недостаток такого

https://t.me/javalib

подхода заключается в том, что пока часть кода остается в анонимном модуле, вся система модулей работает не настолько эффективно, как могла бы.

В случае полностью немодульных приложений (например, приложений Java 8, которые работают поверх среды выполнения Java 11) содержимое classpath собирается в анонимный модуль, и корневым модулем считается `java.se`.

Модульный код не может зависеть от анонимного модуля, так что на практике модули не могут зависеть ни от чего, что находится в classpath. Чтобы преодолеть это затруднение, часто используются автоматические модули. Формально анонимный модуль зависит от всех модулей, которые есть в JDK и в пути к модулям, потому что он воспроизводит поведение, которое было до модулей.

2.4. СОЗДАНИЕ ПЕРВОГО МОДУЛЬНОГО ПРИЛОЖЕНИЯ

Давайте создадим первый пример модульного приложения. Для этого понадобится построить граф модулей (который, конечно, будет направленным ациклическим графом). В графе должен быть *корневой модуль* — в нашем случае это модуль, который содержит класс точки входа приложения. Граф модулей приложения является транзитивным замыканием всех модульных зависимостей корневого модуля.

Для примера мы адаптируем программу проверки HTTP, созданную в конце главы 1, и преобразуем ее в модульное приложение. Структура файлов будет выглядеть так:

```
.
└── wgjd.sitecheck
    └── wgjd
    |   └── sitecheck
    |       ├── concurrent
    |       |   └── ParallelHTTPChecker.java
    |       ├── internal
    |       |   └── TrustEveryone.java
    |       └── HTTPChecker.java
    |       └── SiteCheck.java
    └── module-info.java
```

Мы выделяем некоторые компоненты (например, провайдер `TrustEveryone`) в собственные классы, вместо того чтобы представлять их в виде статических внутренних классов, как приходилось делать, когда весь код был в одном файле. Также мы создаем отдельные пакеты и не экспортируем их все. Файл модуля очень похож на тот, который уже встречался раньше:

```
module wgjd.sitecheck {
  requires java.net.http;
  exports wgjd.sitecheck;
  exports wgjd.sitecheck.concurrent;
}
```

https://t.me/javalib

Обратите внимание на зависимость от модуля `java.net.http`. Чтобы понять, что произойдет, если этой зависимости не будет, закомментируем ее и попробуем скомпилировать проект при помощи `javac`:

```
$ javac -d out wgjd.sitecheck/module-info.java \
    wgjd.sitecheck/wgjd/sitecheck/*.java \
    wgjd.sitecheck/wgjd/sitecheck/*/*.java
wgjd.sitecheck/wgjd/sitecheck/SiteCheck.java:8: error:
  package java.net.http is not visible
import java.net.http.*;
               ^
  (package java.net.http is declared in module java.net.http, but
    module wgjd.sitecheck does not read it)[1]
wgjd.sitecheck/wgjd/sitecheck/concurrent/ParallelHTTPChecker.java:4:
  error: package java.net.http is not visible
import java.net.http.*;
               ^

// И еще несколько аналогичных ошибок
```

Этот эксперимент показывает, что простые проблемы с модулями решаются очень легко. Система модулей обнаружила, что определенного модуля нет, и пытается помочь, предлагая решение: добавить отсутствующий модуль как зависимость. Если так и поступить, то, как и ожидалось, модуль подключается без каких-либо затруднений. Однако более сложные проблемы могут потребовать изменений на этапе компиляции или ручного вмешательства с помощью параметров командной строки, которые управляют системой модулей.

2.4.1. Параметры командной строки для модулей

При компиляции модуля можно использовать параметры командной строки, которые управляют модульными аспектами компиляции и последующего выполнения. Чаще всего используются такие параметры:

- `list-modules` — выводит список всех модулей;
- `module-path` — задает один или несколько каталогов, содержащих ваши модули;
- `add-reads` — добавляет дополнительную директиву `requires` в процесс разрешения;
- `add-exports` — добавляет дополнительную директиву `exports` в процесс компиляции;
- `add-opens` — включает рефлексивный доступ ко всем типам во время выполнения;
- `add-modules` — добавляет список модулей в набор по умолчанию;

[1] Пакет `java.net.http` объявлен в модуле `java.net.http`, но модуль `wgjd.sitecheck` его не считывает.

https://t.me/javalib

- `illegal-access=permit|warn|deny` — изменяет правило рефлексивного доступа.

Большинство этих настроек уже встречалось раньше — кроме квалификаторов, относящихся к рефлексии, о которой мы подробнее поговорим в разделе 2.4.3.

Рассмотрим один из этих параметров в действии. В этом примере мы продемонстрируем типичную проблему, связанную с упаковкой модулей; многие разработчики сталкиваются с ней, когда начинают пользоваться модулями в своем коде.

Начиная работать с модулями, мы иногда обнаруживаем, что приходится нарушать инкапсуляцию. Например, приложение, портированное из Java 8, может обращаться к внутреннему пакету, который больше не экспортируется.

Для примера возьмем проект с простой структурой, который использует Attach API, чтобы динамически подключаться к другим JVM, работающим на узле, и выводит основную информацию о них. Структура файлов проекта на диске во многом похожа на предыдущий пример:

```
.
└── wgjd.discovery
    └── wgjd
    |   └── discovery
    |       ├── internal
    |       |   └── AttachOutput.java
    |       └── Discovery.java
    |       └── VMIntrospector.java
    └── module-info.java
```

При компиляции проекта выдается серия ошибок:

```
$ javac -d out/wgjd.discovery wgjd.discovery/module-info.java \
  wgjd.discovery/wgjd/discovery/*.java \
  wgjd.discovery/wgjd/discovery/internal/*

wgjd.discovery/wgjd/discovery/VMIntrospector.java:4: error: package
  sun.jvmstat.monitor is not visible[1]
import sun.jvmstat.monitor.MonitorException;
                   ^
  (package sun.jvmstat.monitor is declared in module jdk.internal.jvmstat,
    which does not export it to module wgjd.discovery)[2]
wgjd.discovery/wgjd/discovery/VMIntrospector.java:5: error: package
  sun.jvmstat.monitor is not visible
import sun.jvmstat.monitor.MonitoredHost;
                   ^
  (package sun.jvmstat.monitor is declared in module jdk.internal.jvmstat,
    which does not export it to module wgjd.discovery)
```

[1] Ошибка: пакет sun.jvmstat.monitor не входит в область видимости. — *Примеч. пер.*

[2] Пакет sun.jvmstat.monitor объявлен в модуле module jdk.internal.jvmstat, который не экспортирует его в модуль wgjd.discovery. — *Примеч. пер.*

https://t.me/javalib

Проблемы возникают из-за того, что в проекте есть код, который использует внутренние API:

```
public class VMIntrospector implements Consumer<VirtualMachineDescriptor> {

    @Override
    public void accept(VirtualMachineDescriptor vmd) {
        var isAttachable = false;
        var vmVersion = "";
        try {
            var vmId = new VmIdentifier(vmd.id());
            var monitoredHost = MonitoredHost.getMonitoredHost(vmId);
            var monitoredVm = monitoredHost.getMonitoredVm(vmId, -1);
            try {
                isAttachable = MonitoredVmUtil.isAttachable(monitoredVm);
                vmVersion = MonitoredVmUtil.vmVersion(monitoredVm);
            } finally {
                monitoredHost.detach(monitoredVm);
            }
        } catch (URISyntaxException | MonitorException e) {
            e.printStackTrace();
        }

        System.out.println(
            vmd.id() + '\t' + vmd.displayName() + '\t' + vmVersion +
                '\t' + isAttachable);
    }
}
```

Хотя такие классы, как `VirtualMachineDescriptor`, входят в экспортируемый интерфейс модуля `jdk.attach` (потому что класс находится в экспортированном пакете `com.sun.tools.attach`), тем не менее остаются недоступными другие классы, от которых зависит код (например, `MonitoredVmUtil` из `sun.jvmstat.monitor`). К счастью, инструментарий позволяет ослабить границы модулей и получить доступ к неэкспортируемому пакету.

Для этого следует с помощью параметра `--add-exports` включить принудительный доступ к внутренней структуре модуля `jdk.internal.jvmstat`; при этом мы, безусловно, нарушаем инкапсуляцию. В итоге командная строка компиляции выглядит так:

```
$ javac -d out/wgjd.discovery \
  --add-exports=jdk.internal.jvmstat/sun.jvmstat.monitor=wgjd.discovery \
  wgjd.discovery/module-info.java \
  wgjd.discovery/wgjd/discovery/*.java \
  wgjd.discovery/wgjd/discovery/internal/*
```

Синтаксис `--add-exports` требует предоставить имя модуля и пакета, доступ к которому требуется, а также указать модуль, которому доступ предоставляется.

2.4.2. Выполнение модульного приложения

До того как появились модули, запускать приложения Java можно было только двумя способами:

https://t.me/javalib

```
java -cp classes wgjd.Hello
java -jar my-app.jar
```

Эти способы знакомы программистам на Java, соответственно, как запуск класса и запуск главного класса из файла JAR. В современном Java добавились еще два способа. В разделе 1.5.4 был представлен новый способ запуска однофайловых программ, а сейчас мы рассмотрим четвертый способ: запуск главного класса модуля. Его синтаксис выглядит так:

```
java --module-path mods -m my.module/my.module.Main
```

Однако для компиляции понадобятся дополнительные параметры командной строки. Возьмем фрагмент более раннего примера интроспекции:

```
$ java --module-path out -m wgjd.discovery/wgjd.discovery.Discovery
Exception in thread "main" java.lang.IllegalAccessError:
  class wgjd.discovery.VMIntrospector (in module wgjd.discovery) cannot
    access class sun.jvmstat.monitor.MonitorException (in module
      jdk.internal.jvmstat) because module jdk.internal.jvmstat does not
        export sun.jvmstat.monitor to module wgjd.discovery[1]
    at wgjd.discovery/wgjd.discovery.VMIntrospector.accept(
    VMIntrospector.java:19)
    at wgjd.discovery/wgjd.discovery.Discovery.main(Discovery.java:26)
```

Чтобы предотвратить эту ошибку, также необходимо добавить параметр, который нарушает инкапсуляцию при запуске программы:

```
$ java --module-path out \
  --add-exports=jdk.internal.jvmstat/sun.jvmstat.monitor=wgjd.discovery \
  -m wgjd.discovery/wgjd.discovery.Discovery

Java processes:
PID Display Name VM Version Attachable
53407 wgjd.discovery/wgjd.discovery.Discovery 15-ea+24-1168 true
```

Если система выполнения не может найти корневой модуль, который вы запрашиваете, должно возбудиться следующее исключение:

```
$ java --module-path mods -m wgjd.hello/wgjd.hello.HelloWorld
Error occurred during initialization of boot layer[2]
java.lang.module.FindException: Module wgjd.hello not found[3]
```

Даже это простое сообщение об ошибке показывает, что здесь задействованы новые аспекты JDK, в числе которых:

---

[1] У класса wgjd.discovery.VMIntrospector (в модуле wgjd.discovery) нет доступа к классу sun.jvmstat.monitor.MonitorException (в модуле jdk.internal.jvmstat), потому что модуль jdk.internal.jvmstat не экспортирует sun.jvmstat.monitor в модуль wgjd.discovery. — *Примеч. пер.*

[2] При инициализации загрузочного уровня произошла ошибка. — *Примеч. пер.*

[3] Модуль wgjd.hello не обнаружен. — *Примеч. пер.*

https://t.me/javalib

- пакеты, в частности `java.lang.module`;
- исключения, в частности `FindException`.

Этот пример снова демонстрирует, что система модулей органически задействована в выполнении каждой программы Java, даже если это не очевидно на первый взгляд.

В следующем разделе мы в общих чертах рассмотрим, как модули взаимодействуют с рефлексией. Предполагается, что вы уже знакомы с механизмом рефлексии, а если нет, ничто не мешает вам пока пропустить этот раздел и вернуться к нему после главы 4, где кратко описана загрузка классов и рефлексия.

2.4.3. Модули и рефлексия

В Java 8 разработчики могут использовать рефлексию для доступа практически к любым аспектам среды выполнения. Можно даже обойти проверки контроля доступа и, например, вызвать частные методы других классов при помощи так называемого трюка `setAccessible()`.

Как мы уже видели, модули меняют правила управления доступом. Это относится и к рефлексии: изначально предполагалось, что по умолчанию через рефлексию будут доступны только экспортируемые пакеты.

Однако создатели системы модулей осознали, что иногда разработчики хотят предоставить рефлексивный (но не прямой) доступ к некоторым пакетам. Для этого требуется явное разрешение, и с помощью ключевого слова `opens` можно предоставить чисто рефлексивный доступ к внутреннему пакету. Разработчики также могут настроить более избирательный доступ, используя синтаксис `opens … to …`, чтобы открыть к тому или иному набору пакетов рефлексивный доступ только из конкретных модулей, а не отовсюду.

После чтения предыдущего материала может показаться, будто подобные трюки с рефлексией теперь запрещены. Истинное положение дел чуть сложнее, и его проще всего объяснить, представив параметр командной строки `--illegal-access`. Он принимает три возможных значения: `permit` (разрешить), `warn` (предупреждать) и `deny` (запретить) — и управляет строгостью проверок при рефлексии.

Система модулей изначально создавалась с тем намерением, чтобы со временем вся экосистема Java постепенно двигалась к полноценной инкапсуляции, в том числе для рефлексии, так что планировалось, что с какого-то момента этот параметр будет по умолчанию принимать значение `deny`, а в конечном итоге он будет удален. Разумеется, такие изменения не могут происходить в одночасье: если параметр рефлексии внезапно перейдет в состояние `deny`, то огромные участки экосистемы Java перестанут работать, и *никто не захочет обновляться*.

Однако к моменту выхода Java 17 прошло уже четыре года после выпуска Java 9, когда соответствующее предупреждение впервые начало появляться. Безусловно, времени прошло достаточно, а предупреждение было вполне недвусмысленным.

https://t.me/javalib

Соответственно, в Java 16 было решено изменить значение `--illegal-access` по умолчанию на `deny`, а в Java 17 полностью убрать этот параметр.

> **ПРИМЕЧАНИЕ** Это изменение в семантике рефлексивной инкапсуляции — одна из причин, по которым прямая миграция приложения с Java 8 на Java 17 может вызвать больше проблем, чем миграция в два этапа (сначала с 8-й на 11-ю, а потом с 11-го на 17-ю).

При этом отдельные пакеты все еще можно открывать, если использовать параметр командной строки `--add-opens` или добавить в манифест JAR атрибут `Add-Opens`. Такая практика может понадобиться для некоторых библиотек или фреймворков, которые исторически использовали рефлексию и еще не были полностью модуляризованы. Однако глобальное разрешение доступа методом «грубой силы» в Java 17 больше не работает.

Еще одна полезная возможность, которая упрощает этот переход, — *открытые модули*. Это простое объявление используется для того, чтобы разрешить полностью открытый рефлексивный доступ: все пакеты модуля открываются для рефлексии, но не для доступа на стадии компиляции. Так обеспечивается простая совместимость с существующим кодом и фреймворками, но за счет ослабленной формы инкапсуляции.

Поэтому лучше избегать открытых модулей или использовать их только как переходную форму при миграции на модульную сборку. В главе 17 мы рассмотрим особый случай использования `Unsafe` — отличный пример, который демонстрирует характерные проблемы с рефлексией в модульном мире.

2.5. ПРОЕКТИРОВАНИЕ АРХИТЕКТУРЫ ДЛЯ МОДУЛЕЙ

Модули — это принципиально новый способ упаковывать и развертывать код. Команды разработчиков должны овладевать новыми приемами, чтобы извлечь максимум пользы из новой функциональности и ее архитектурных преимуществ. К счастью, этим не обязательно заниматься прямо сейчас только для того, чтобы начать пользоваться современными возможностями Java. Традиционные методы, основанные на classpath и файлах JAR, продолжат работать до того момента, когда команды будут полностью готовы принять модули.

Вот что Марк Рейнолд (главный архитектор Java в Oracle) говорит о том, насколько необходимо переделывать приложения на модульный лад:

> *Нет необходимости переключаться на модули.*
>
> *Никогда не было необходимости переключаться на модули.*
>
> *Java 9 и последующие версии поддерживают традиционные файлы JAR с традиционным classpath, используя для этого концепцию анонимного модуля,*

https://t.me/javalib

и скорее всего, будут поддерживать все это до момента тепловой смерти Вселенной.

Переходить ли на использование модулей — решайте сами.

Если вы сопровождаете большой унаследованный проект, который особо не меняется, — вероятно, игра не стоит свеч.

— *Марк Рейнолд,* https://stackoverflow.com/a/62959016

В идеальном мире модули использовались бы по умолчанию для всех вновь создаваемых приложений, но на практике этого сложно добиться, так что в качестве альтернативы при миграции можно действовать по такой схеме:

1. Выполните обновление до Java 11 (только classpath).
2. Задайте имя автоматического модуля.
3. Создайте *монолитный модуль*, который содержит весь код.
4. Выделяйте отдельные модули по мере необходимости.

Как правило, на шаге 3 раскрывается слишком много кода реализации. Это означает, что довольно часто на шаге 4 часть работы будет заключаться в том, чтобы создавать дополнительные пакеты для размещения внутреннего кода и кода реализации и перемещать в них код посредством рефакторинга.

Если вы до сих пор используете Java 8 и еще не готовы мигрировать на модульную сборку, то подготовить код к миграции можно так:

- Включите имя автоматического модуля в MANIFEST.MF.
- Исключите расщепленные пакеты из артефактов развертывания.
- Используйте `jdeps` и компактные профили, чтобы снизить затраты памяти на лишние зависимости.

Что касается первого шага, то явное назначение имени автоматического модуля (о чем упоминалось ранее в этой главе) упростит переход. Все версии Java, которые не поддерживают модули, будут игнорировать имя автоматического модуля, но оно позволит вам зарезервировать стабильное имя для вашей библиотеки и перенести часть кода из анонимного модуля. Еще одно преимущество заключается в том, что потребители вашей библиотеки будут готовы к переходу на модули, потому что вы уже анонсировали имя, под которым модуль будет фигурировать. Давайте внимательнее присмотримся к двум другим рекомендациям.

2.5.1. Расщепленные пакеты

Типичная проблема, с которой сталкиваются разработчики в начале работы с модулями, — *расщепление пакетов*, когда в двух и более разных файлах JAR содержатся классы, принадлежащие одному пакету. В немодульных приложениях расщепление пакетов не создает проблем, потому что ни файлы JAR, ни пакеты не имеют особого значения для среды выполнения. Однако

https://t.me/javalib

в модульном мире пакет должен принадлежать только одному модулю и не может расщепляться.

Если вы модуляризируете существующее приложение и в его зависимостях есть расщепленные пакеты, эту проблему придется решать — обходных путей не существует. Над кодом, который вы контролируете, придется провести дополнительную работу, хотя и не слишком сложную. Например, можно использовать специальный артефакт (часто с суффиксом `-all`), который система сборки генерирует наряду с немодульными версиями; он содержит один файл JAR со всеми частями расщепленного пакета.

Для внешних зависимостей решение может оказаться сложнее. Возможно, вам придется перепаковать сторонний открытый код в файл JAR, который можно будет использовать как автоматический модуль.

2.5.2. Компактные профили Java 8

Компактные профили появились в Java 8. Это среды выполнения уменьшенного размера, которые должны реализовывать обе спецификации — JVM и JLS. Они были включены в Java 8 как полезный промежуточный шаг на пути к модульности, которая появилась в Java 9.

В компактный профиль должны входить все классы и пакеты, которые явно указаны в JLS. Профили представляют собой списки пакетов, каждый из которых обычно идентичен одноименному пакету полной платформы Java SE. Существуют единичные исключения, но они указываются явно.

Одна из основных задач профилей на практике — служить основой для серверных приложений или других сред, где нежелательно развертывать избыточную функциональность. Например, исторически многие дефекты безопасности Java были связаны с функциональностью GUI, особенно в Swing и AWT. Если в приложениях, где не нужны эти возможности, не развертывать пакеты, которые их реализуют, можно немного повысить уровень безопасности, особенно для серверных приложений.

> **ПРИМЕЧАНИЕ** Одно время компания Oracle поставляла урезанную версию JRE («Серверную JRE»), которая была очень похожа на Compact 1.

Compact 1 — наименьший набор пакетов, с которыми имеет смысл развертывать приложения. Он содержит 50 пакетов, от хорошо знакомых

- `java.io`
- `java.lang`
- `java.math`
- `java.net`
- `java.text`
- `java.util`

https://t.me/javalib

до других, которые могут показаться неожиданными, однако предоставляют критически важные классы для современных приложений:

- `java.util.concurrent.atomic`
- `java.util.function`
- `javax.crypto.interfaces`
- `javax.net.ssl`
- `javax.security.auth.x500`

Набор Compact 2 значительно больше: в частности, он содержит пакеты, которые необходимы для работы с XML, SQL, RMI и безопасностью. Compact 3 еще больше: по сути, он включает всю JRE, кроме оконной подсистемы и компонентов GUI — подобно модулю `java.se`.

> **ПРИМЕЧАНИЕ** Все профили включают транзитивное замыкание типов, на которые ссылается Object, и все типы, упомянутые в спецификации языка.

Профиль Compact 1 ближе всего к минимальной среде выполнения, так что в некоторых отношениях он похож на прототипную форму модуля `java.base`. В идеале, если можно сделать так, чтобы ваше приложение или библиотека запускались с единственной зависимостью от Compact 1, так и следует поступить.

Чтобы помочь определить, можно ли запускать приложение с Compact 1 или другим профилем, JDK предоставляет `jdeps` — программу статического анализа, которая поставляется с Java 8 и 11 и отслеживает зависимости пакетов или классов. Ее можно использовать многими разными способами: например, чтобы узнать, с каким профилем должно выполняться приложение, или чтобы обнаружить код, который обращается с вызовами к недокументированным внутренним API из JDK (таким, как классы `sun.misc`), или даже чтобы отследить транзитивные зависимости. `jdeps` может помочь мигрировать с Java 8 на 11; она работает как с файлами JAR, так и с модулями. В простейшей форме `jdeps` принимает класс или пакет и выводит краткий список пакетов, которые являются его зависимостями. Например, для `discovery` результат может выглядеть так:

```
$ jdeps Discovery.class
Discovery.class -> java.base
Discovery.class -> jdk.attach
Discovery.class -> not found
    wgjd.discovery            -> com.sun.tools.attach       jdk.attach
    wgjd.discovery            -> java.io                    java.base
    wgjd.discovery            -> java.lang                  java.base
    wgjd.discovery            -> java.util                  java.base
    wgjd.discovery            -> wgjd.discovery.internal    not found
```

Параметр `-P` показывает, какой профиль нужен, чтобы запускать класс (или пакет); впрочем, конечно, он работает только в среде выполнения Java 8.

https://t.me/javalib

А теперь перейдем к другому приему миграции, которым должен владеть каждый опытный Java-разработчик, — использованию *многоверсионных файлов JAR*.

2.5.3. Многоверсионные файлы JAR

Эта новая функциональность позволяет создавать файлы JAR с библиотеками и компонентами, которые могут работать и в Java 8, и в современных модульных JVM. Например, ПО может зависеть от библиотечных классов, которые доступны только в поздних версиях, но при этом оно будет запускаться и в более ранних версиях благодаря резервным механизмам и заглушкам.

Чтобы создать многоверсионный файл JAR, добавьте в файл манифеста JAR такую запись:

```
Multi-Release: true
```

Эту запись понимают только JVM версии 9 и выше, так что если файл JAR используется на виртуальной машине Java 8 (или более ранней версии), его многоверсионность будет проигнорирована.

Классы, ориентированные на версии после Java 8, называются *вариационным кодом* и хранятся в специальном каталоге в `META-INF` внутри файла JAR:

```
META-INF/versions/<номер_версии>
```

Этот механизм использует переопределение на уровне класса. Java версии 9 и выше ищет в каталоге versions класс, имя которого в точности совпадает с именем класса в корневом каталоге проекта. Если такой класс найден, то вместо класса из корневого каталога используется переопределенная версия.

> **ПРИМЕЧАНИЕ** Файлы классов Java помечаются номером версии компилятора Java, который их создал (номером версии файла класса), и код, созданный в более поздней версии Java, не будет работать на старой JVM.

Каталог META-INF/versions игнорируется в Java 8 и более ранних версиях, благодаря чему удается элегантно обойти ту проблему, что часть кода в многоверсионном файле JAR помечена слишком высокой версией файла класса и не может выполняться в Java 8.

Однако это означает, что у всех переопределенных вариантов класса должен быть точно такой же API, как у класса в корневом каталоге проекта, потому что оба класса будут компоноваться абсолютно одинаково.

Пример: создание многоверсионного JAR-файла

Для примера рассмотрим такую задачу: получить идентификатор процесса текущей JVM. К сожалению, до версии Java 9 это делалось несколько неуклюже и требовало низкоуровневых ухищрений из пакета `java.lang.management`.

https://t.me/javalib

В Java 11 есть API, который позволяет получить PID из Process API. Поэтому мы создадим многоверсионный файл JAR, который будет использовать простой API, если тот доступен, а если нет — откатываться к решению на основе JMX.

Главный класс выглядит так:

```
public class Main {
    public static void main(String[] args) {
        System.out.println(GetPID.getPid());
    }
}
```

Обратите внимание: мы выделили функциональность, реализация которой зависит от версии, в отдельный класс `GetPID`. Версия соответствующего кода для Java 8 получается довольно громоздкой:

```
public class GetPID {
  public static long getPid() {
    System.out.println("Версия для Java 8…");
    // ManagementFactory.getRuntimeMXBean().getName() возвращает имя,
    // представляющее текущую JVM. В JVM от Sun и Oracle это имя
    // имеет формат <pid>@<имя_узла>.

    final var jvmName = ManagementFactory.getRuntimeMXBean().getName();
    final var index = jvmName.indexOf('@');
    if (index < 1) {
        return 0;
    }

    try {
        return Long.parseLong(jvmName.substring(0, index));
    } catch (NumberFormatException e) {
        return 0;
    }
  }
}
```

Включаем эту строку, чтобы убедиться, что выполняется версия для Java 8

Здесь нам приходится разбирать строку, полученную от метода JMX, — и даже в этом случае не гарантируется, что решение будет переносимо между реализациями JVM. С другой стороны, Java 9 и более поздние версии предоставляют простой стандартный метод в API, как показано в следующем фрагменте:

```
public class GetPID {
    public static long getPid() {
        // Используем ProcessHandle API, появившийся в Java 9 ...
        var ph = ProcessHandle.current();
        return ph.pid();
    }
}
```

Класс `ProcessHandle` находится в пакете `java.lang`, поэтому нам даже не понадобится инструкция `import`.

https://t.me/javalib

Теперь нужно организовать многоверсионный файл JAR так, чтобы в нем был код для Java 11, который будет использоваться вместо резервного варианта, если версия JVM достаточно высока. Например, структура кода может выглядеть так:

```
.
└── src
    ├── main
    |   └── java
    |       └── wgjd2ed
    |           └── Main.java
    |           └── GetPID.java
    └── versions
        └── 11
            └── java
                └── wgjd2ed
                    └── GetPID.java
```

Основную часть кодовой базы нужно скомпилировать в Java 8, а код, рассчитанный на более поздние версии, компилируется отдельно в другой версии Java, после чего вручную упаковывается в многоверсионный файл JAR (с помощью программы командной строки `jar`).

> **ПРИМЕЧАНИЕ** Эта структура кода соответствует соглашениям, которые приняты в системах сборки Maven и Gradle; они будут описаны в главе 11.

Откомпилируем код с помощью командной строки: используем `javac` от текущей версии JDK, но будем генерировать вывод для Java 8 при помощи флага `--release`:

```
$ javac --release 8 -d out src/main/java/wgjd2ed/*.java
```

Затем соберем код для Java 11 в отдельном выходном каталоге для вариационного кода `out-11`:

```
$ javac --release 11 -d out-11 versions/11/java/wgjd2ed/GetPID.java
```

Еще нам понадобится файл `MANIFEST.MF`, но его можно автоматически сгенерировать утилитой `jar` (из Java 11):

```
$ jar --create --release 11 \
    --file pid.jar --main-class=wgjd2ed.Main
    -C out/ . \
    -C out-11/ .
```

В результате получится многоверсионный файл JAR, который также будет исполняемым (`Main` является классом точки входа). Если запустить этот файл в Java 11, получится такой результат:

```
$ java -version
openjdk version "11.0.3" 2019-04-16
OpenJDK Runtime Environment AdoptOpenJDK (build 11.0.3+7)
OpenJDK 64-Bit Server VM AdoptOpenJDK (build 11.0.3+7, mixed mode)
```

https://t.me/javalib

```
$ java -jar pid.jar
13855
```

А вот вывод для Java 8:

```
$ java -version
openjdk version "1.8.0_212"
OpenJDK Runtime Environment (AdoptOpenJDK)(build 1.8.0_212-b03)
OpenJDK 64-Bit Server VM (AdoptOpenJDK)(build 25.212-b03, mixed mode)

$ java -jar pid.jar
Версия для Java 8…
13860
```

Обратите внимание на дополнительное сообщение, которое мы добавили в версию для Java 8, чтобы различать два случая и убедиться, что здесь действительно выполняются два разных класса. Когда многоверсионные файлы JAR используются в реальной эксплуатации, код обычно должен либо порождать одинаковый результат в обоих случаях (если предусмотрена совместимость с Java 8), либо выдавать сбой корректно и предсказуемо — если он выполняется на JVM, которая не поддерживает эту возможность.

Важный архитектурный шаблон, которому мы рекомендуем следовать, предписывает изолировать код, который предназначен для конкретной версии JDK, в пакете или группе пакетов (в зависимости от объема функциональности).

Вот несколько ключевых рекомендаций по структуре проекта:

- Основная кодовая база должна быть такой, чтобы ее можно было собрать под Java 8.
- Код для Java 11 нужно собирать под Java 11.
- Код для Java 11 должен находиться в отдельном корневом каталоге, который изолирован от основной сборки.
- Конечным результатом должен быть единый файл JAR.
- Следите за тем, чтобы конфигурация сборки оставалась простой, насколько это возможно.
- Также подумайте о том, нельзя ли сделать многоверсионный файл JAR модульным.

Последний пункт особенно важен, причем он продолжает оставаться актуальным для более сложных проектов, где неизбежно потребуются более мощные средства сборки вместо простых `javac` и `jar`.

2.6. НЕ ТОЛЬКО МОДУЛИ

В завершение этой главы мы немного выйдем за рамки темы модулей. Вспомните, что задача модулей заключалась в том, чтобы ввести в язык Java недостающую

https://t.me/javalib

абстракцию: идею единиц развертывания с гарантиями зависимостей, на которые могут полагаться компилятор исходного кода и среда выполнения.

Информация о модульных зависимостях, которой можно доверять, — это важная концепция, которая постепенно завоевывает современный мир развертываемых программных продуктов. В Java наблюдается медленный, но верный переход на модули по мере того, как инструментарий и экосистема начинают полноценно их поддерживать, а их практические преимущества становятся более очевидными.

Напоследок рассмотрим новую функциональность, которая добавилась в платформу вместе с модулями, — *JLink*. Она позволяет упаковывать сокращенную среду выполнения Java вместе с приложением. Это создает следующие преимущества для приложений:

- Приложение вместе с JVM упаковывается в один автономный каталог.
- Сокращается потребление памяти и общий объем загрузки приложения и JRE.
- Снижаются затраты на поддержку, потому что не нужно отлаживать взаимодействие между приложением Java и JVM, установленной на узле.

Автономные каталоги, которые производит `jlink`, легко упаковываются в артефакты развертывания (такие, как .rpm или .deb в Linux, .dmg на Mac, .msi в Windows), благодаря чему современные приложения Java проще устанавливать.

В каком-то отношении технология компактных профилей Java 8 предоставляет раннюю версию JLink, однако версия, появившаяся с модулями, намного полезнее и функциональнее. Для примера мы еще раз обратимся к проекту `discovery`, который уже встречался в этой главе. В нем используется простой файл module-info.java:

```
module wgjd.discovery {
  exports wgjd.discovery;

  requires java.instrument;
  requires java.logging;
  requires jdk.attach;
  requires jdk.internal.jvmstat;
}
```

Чтобы создать комплект (bundle) JLink, можно воспользоваться следующей командой:

```
$ jlink --module-path $JAVA_HOME/jmods/:out --output bundle/ --add-modules
  wgjd.discovery
```

В нашем простом примере получается комплект JLink, который можно распространять в формате TAR или упаковать в пакет Linux (такой, как .deb или .rpm). Можно пойти еще дальше и использовать *статическую компиляцию*, чтобы

https://t.me/javalib

преобразовать такой комплект в низкоуровневый исполняемый файл, но подробное обсуждение этой возможности выходит за рамки книги.

Считаем нужным предупредить: JLink — превосходная технология, но у нее есть ряд важных ограничений, о которых необходимо помнить:

- Она работает только для приложений с полностью модуляризованными зависимостями.
- Она не работает с немодульным кодом.
- Даже автоматических модулей недостаточно.

Это объясняется тем, что для полной уверенности в том, что все необходимые части JRE включены в комплект, JLink полагается на чисто декларативную информацию в графе модулей и, как следствие, требует module-info.class для каждой зависимости. Без этой информации, вероятнее всего, сокращенную JRE не удастся создать безопасным способом.

К сожалению, в реальном мире многие библиотеки, от которых зависят приложения, еще не полностью перешли на модули, а это резко сокращает пользу от JLink. Чтобы решить эту проблему, создатели инструментария разработали плагины, которые переупаковывают немодуляризованные библиотеки, синтезируя из них «настоящие» модули. Мы рассмотрим эти инструменты в главе 11. Однако чтобы их применять, требуется система сборки, и поэтому реальные примеры использования JLink откладываются до тех пор, пока мы не займемся средствами сборки в главе 11.

ИТОГИ

- Модули — новое явление в языке Java. Модули группируют пакеты и предоставляют метаданные о целой программной единице, ее зависимостях и открытых интерфейсах. Компилятор и среда выполнения обеспечивают соблюдение этих ограничений.
- Модули *не являются* конструкцией развертывания (как, например, новый формат файла). Модуляризованные библиотеки и приложения по-прежнему можно распространять в виде файлов JAR и загружать стандартными средствами сборки.
- Переход на модули требует изменить принципы разработки приложений Java:
  - Новый синтаксис файла module-info.java управляет доступом к классам и методам через систему модулей.
  - Механизм загрузки классов знает об ограничениях, определяемых модулями, и справляется с загрузкой немодульного кода.
  - Сборка с модулями требует новых параметров командной строки и изменений в стандартной структуре проектов Java.

https://t.me/javalib

- В обмен на дополнительные усилия разработчиков модули предоставляют ряд преимуществ:
  - Возможности более избирательного управления приводят к тому, что модули позволяют эффективнее формировать архитектуру приложений, которые нужно развертывать сейчас и сопровождать в будущем.
  - Модули играют ключевую роль в сокращении затрат памяти, особенно в контейнерах.
  - Модули прокладывают путь для других новых возможностей, например статической компиляции.
- Перейти на использование модулей не всегда просто, особенно если речь идет об унаследованных монолитных приложениях. Даже через несколько лет после того, как вышла первая модуляризованная среда выполнения, миграция на систему модулей происходит неравномерно и неполноценно.
- Такие средства, как многоверсионные файлы JAR и компактные профили, помогают подготовить существующие проекты Java 8 для интеграции с экосистемой модулей, даже если немедленная миграция невозможна.

https://t.me/javalib

3 Java 17

В ЭТОЙ ГЛАВЕ

- ✓ Текстовые блоки
- ✓ Выражения `switch`
- ✓ Записи
- ✓ Запечатанные типы

Здесь перечислены основные новые возможности, которые были добавлены в язык и платформу Java после выхода Java 11 и вплоть до Java 17 включительно.

> **ПРИМЕЧАНИЕ** Чтобы понимать, как изменилась методология выпусков Java после Java 8, полезно ознакомиться с материалом из главы 1 или с приложением A.

Кроме крупных обновлений языка, которые заметны пользователям, в Java 17 внедрено много внутренних улучшений (особенно по части быстродействия). Однако в этой главе мы сосредоточимся на главных новшествах, которые, как нам кажется, способны изменить ваш подход к тому, как писать код на Java.

3.1. ТЕКСТОВЫЕ БЛОКИ

С самой первой версии Java 1.0 разработчики жаловались на строки в Java. По сравнению с другими языками программирования, такими как Groovy, Scala или Kotlin, строки в Java иногда выглядели немного примитивно.

https://t.me/javalib

Исторически в Java поддерживался только один тип строк — хорошо знакомые вам строки в двойных кавычках, где некоторые символы (прежде всего " и \) нужно было экранировать, чтобы безопасно использовать. Все это приводило к тому, что программистам слишком часто приходилось конструировать громоздкие экранированные строки даже в самых стандартных ситуациях.

Проект текстовых блоков прошел несколько итераций, начиная с предварительной функциональности (эта тема кратко рассматривалась в главе 1), а в Java 17 они вошли в официальный набор возможностей. Задача текстовых блоков — расширить понятие строк в синтаксисе Java за счет строковых литералов, которые охватывают несколько физических строк. Это позволяет обойтись без многих экранированных последовательностей, которые традиционно казались программистам на Java лишним бременем.

> **ПРИМЕЧАНИЕ** В отличие от других языков программирования, текстовые блоки Java пока не поддерживают *интерполяцию*, хотя эта возможность сейчас активно рассматривается для включения в будущую версию.

Текстовые блоки не только избавляют программистов от хлопот с нескончаемым экранированием символов, но и специально разработаны так, чтобы с их помощью можно было создавать удобочитаемые строки кода, которые написаны на других языках, но вставляются в программу на Java. Признайтесь, вам наверняка порой приходится включать в программу фрагменты кода на SQL или JSON (а то и XML)?

До выхода Java 17 этот процесс бывал довольно неприятным, и разработчики часто поддавались соблазну использовать внешние библиотеки шаблонов со всеми их дополнительными сложностями. После того как появились текстовые блоки, во многих ситуациях эти библиотеки больше не нужны.

Чтобы продемонстрировать, как работать с текстовыми блоками, рассмотрим запрос SQL. В этой главе мы используем примеры из области валютного рынка. Допустим, ордера клиента хранятся в базе данных SQL, к которой мы обращаемся с помощью запросов, подобных этому:

```
String query = """
        SELECT "ORDER_ID", "QUANTITY", "CURRENCY_PAIR" FROM "ORDERS"
        WHERE "CLIENT_ID" = ?
        ORDER BY "DATE_TIME", "STATUS" LIMIT 100;
        """;
```

Обратите внимание на два обстоятельства. Во-первых, текстовый блок начинается и заканчивается последовательностью """, которая не допускалась в Java до версии 15. Во-вторых, в начале каждой строки блока может быть отступ, причем пробельные символы будут игнорироваться на этапе компиляции.

Если вывести переменную `query`, мы получим именно ту строку, которую сконструировали:

https://t.me/javalib

```
SELECT "ORDER_ID", "QUANTITY", "CURRENCY_PAIR" FROM "ORDERS"
WHERE "CLIENT_ID" = ?
ORDER BY "DATE_TIME", "STATUS" LIMIT 100;
```

Это происходит потому, что текстовый блок — это константное выражение (типа `String`), как и строковый литерал. Разница в том, что текстовый блок обрабатывается компилятором `javac` перед тем, как константа записывается в файл класса:

1. Символы перевода строки преобразуются в LF (`\u000A`), как принято в Unix.
2. Дополнительные пробельные символы вокруг блока удаляются: это позволяет создавать удобные отступы в исходном коде Java, как в нашем примере.
3. Все экранированные последовательности в блоке интерпретируются.

Эти операции выполняются в указанном порядке не просто так, а именно: поскольку экранированные последовательности интерпретируются в последнюю очередь, это означает, что в блоках допускаются литеральные экранированные последовательности (например, `\n`), которые не будут изменены или удалены на предшествующих шагах.

> **ПРИМЕЧАНИЕ** Во время выполнения строковые константы, полученные из литералов, ничем не отличаются от констант из текстовых блоков. В файле класса не сохраняется первоначальный источник константы.

За дополнительной информацией о текстовых блоках обращайтесь к JEP 378 (https://openjdk.java.net/jeps/378). А мы перейдем к следующему нововведению — выражениям `switch`.

3.2. ВЫРАЖЕНИЯ SWITCH

С самых первых версий в Java поддерживались *инструкции* `switch`. Синтаксис Java был во многом вдохновлен опытом C и C++, и `switch` не стал исключением:

```
switch(month) {
  case 1:
    System.out.println("Январь");
    break;
  case 2:
    System.out.println("Февраль");
    break;
  // ... и т. д.
}
```

В частности, инструкция `switch` в Java унаследовала такую особенность: если `case` не завершается инструкцией `break`, то выполнение продолжается в следующей секции `case`. Это позволяет группировать случаи, которые надо обрабатывать одинаково, например:

https://t.me/javalib

```
switch(month) {
  case 12:
  case 1:
  case 2:
    System.out.println("Зима, холода");
    break;
  case 3:
  case 4:
  case 5:
     System.out.println("Весна красна!");
     break;
  // ... и т. д.
}
```

Однако у удобства в подобных ситуациях есть неприятная обратная сторона. Пропущенный `break` — типичная оплошность как начинающих, так и опытных программистов, и часто она порождает ошибки. Если в нашем примере пропустить первую инструкцию `break`, то код ошибочно выведет сообщения и про зиму, и про весну.

Инструкции `switch` также получаются слишком громоздкими, если пытаться сохранить значение, чтобы использовать его впоследствии. Например, если бы мы хотели не вывести сообщение, а сохранить его на будущее, нам пришлось бы создать переменную за пределами `switch`, присвоить ей нужное значение в каждой ветви и, возможно, после `switch` проверить, что значение действительно присвоилось; это выглядит примерно так:

```
String message = null;
switch(month) {
  case 12:
  case 1:
  case 2:
    message = "Зима, холода";
    break;
  case 3:
  case 4:
  case 5:
    message = "Весна красна!";
    break;
  // ... и т. д.
}
```

Как и в случае с пропущенной командой `break`, теперь нужно следить за тем, чтобы в каждой секции `case` правильно присваивалось сообщение; иначе пользователи закидают вас баг-репортами. Конечно, это не идеальное решение.

Выражения `switch`, появившиеся в Java 14 (JEP 361), предоставляют альтернативы, которые преодолевают эти недостатки, а также открывают новые перспективы для дальнейшего развития языка. В частности, они сокращают разрыв с функционально-ориентированными языками (например, Haskell, Scala или Kotlin). Первая версия выражений `switch` более компактна:

https://t.me/javalib

```
String message = switch(month) {
  case 12:
  case 1:
  case 2:
    yield "Зима, холода";
  case 3:
  case 4:
  case 5:
    yield "Весна красна!";
    // ... и т. д.
}
```

Эта форма отличается от обычного `switch` тем, что мы больше не присваиваем значение переменной в отдельных ветвях. Вместо этого в нужных секциях `case` используется новое ключевое слово `yield`, которое передает желаемое значение для присваивания строковой переменной `message`, а выражение в целом *генерирует значение* — из одной или другой ветви `case` (и каждой ветви `case` должна соответствовать инструкция `yield`).

После такого примера название новой возможности — *выражения* `switch` вместо *инструкций* `switch` — становится более понятным. В языках программирования *инструкцией* (statement) называется фрагмент кода, который выполняется ради своего побочного эффекта. А *выражением* (expression) называется код, который выполняется, чтобы породить значение. До Java 14 конструкция `switch` существовала только в виде инструкции с побочным эффектом, но теперь она может производить значение, если использовать ее как выражение.

Выражения `switch` допускают еще более компактный синтаксис — не исключено, что он станет особенно популярным:

```
String message = switch(month) {
  case 1, 2, 12   -> "Зима, холода";
  case 3, 4, 5    -> "Весна красна!";
  case 6, 7, 8    -> "Лето — это маленькая жизнь";
  case 9, 10, 11 -> "Осень, прозрачное утро";
  default         -> {
    throw new IllegalArgumentException("Упс, это не месяц");
  }
}
```

Конструкция `->` обозначает, что мы находимся в выражении `switch`, и позволяет не включать команду `yield` явно. Секция `default` демонстрирует, как можно поступать там, где нет одного значения: использовать блок, заключенный в `{}`. Если значение выражения `switch` задействовано в программе (как в нашем случае, где оно присваивается переменной `message`), то многострочные секции `case` должны содержать либо `yield`, либо `throw`.

Однако новый формат не только удобнее и короче — он решает реальные проблемы. Во-первых, напрямую поддерживаются множественные совпадения: для этого после `case` следует список, разделенный запятыми. При этом отпадает необходимость следить за рискованными сквозными переходами в `switch`, как

https://t.me/javalib

раньше. В новом синтаксисе выражений `switch` не бывает сквозных переходов, а значит, нет и связанных с ними распространенных ошибок.

Это еще не все факторы, повышающие надежность кода. В традиционных инструкциях `switch` программисты часто делали и другую ошибку: пропускали секцию `case`, которую нужно было обработать. А если убрать строку `default` из предыдущего примера, вы получите сообщение об ошибке компиляции:

```
error: the switch expression does not cover all possible input values[1]
    String message = switch(month) {
                     ^
```

В отличие от инструкций `switch`, выражения `switch` должны покрывать все возможные значения входного типа, иначе код даже не будет компилироваться. Это отличная гарантия того, что вы ничего не упустили. Выражения `switch` также хорошо сочетаются с перечислениями Java; чтобы убедиться в этом, достаточно переписать `switch` с помощью типобезопасных констант вместо целых чисел:

```
String message = switch(month) {
    case JANUARY, FEBRUARY, DECEMBER  -> "Зима, холода";
    case MARCH, APRIL, MAY            -> "Весна красна!";
    case JUNE, JULY, AUGUST           -> "Лето — это маленькая жизнь";
    case SEPTEMBER, OCTOBER, NOVEMBER -> "Осень, прозрачное утро";
};
```

Эта новая возможность очень полезна, потому что она упрощает использование `switch` в одном из самых распространенных случаев: когда `switch` применяется подобно функции и возвращает выходное значение на основании входного. Согласно правилам выражений `switch`, для каждого возможного входного значения они должны заведомо генерировать выходное значение.

> **ПРИМЕЧАНИЕ** Если в выражении switch присутствуют все возможные константы перечисляемого типа, это означает, что все варианты учтены и включать секцию default не обязательно: компилятор может воспользоваться тем, что список констант уже исчерпывающий.

Но если выражение `switch`, например, получает `int`, то секция `default` необходима, потому что иначе пришлось бы перебирать примерно четыре миллиарда возможных значений.

Выражения `switch` — это также важный шаг на пути к поиску по шаблону — новой крупной возможности, которая может появиться в будущей версии Java[2]. Мы поговорим о ней далее в этой главе и позднее в книге, а пока перейдем к следующей новой функциональности — записям.

---

[1] Ошибка: выражение `switch` не покрывает все возможные входные значения. — *Примеч. пер.*

[2] Pattern matching для switch (а заодно и Record Patterns) появились как полноценные новые фичи в Java 21. — *Примеч. ред.*

https://t.me/javalib

3.3. ЗАПИСИ

Записи (records) — это новая разновидность классов Java, разработанная для следующих целей:

- Предоставить инструменты первого класса для моделирования структур, которые содержат только данные.
- Ликвидировать потенциальный пробел в системе типов Java.
- Предоставить синтаксис уровня языка для распространенного паттерна программирования.
- Сократить объем шаблонного кода классов.

Порядок этих пунктов важен: собственно, записи в большей степени относятся к семантике языка, чем к сокращению кода и синтаксису (хотя многие разработчики склонны сосредоточиваться именно на втором аспекте). Для начала разберемся, что же представляет собой запись Java.

Записи были созданы для того, чтобы выразить в языке Java концепцию класса, который содержит «поля, только поля и ничего, кроме полей». Если объявить класс таким образом, то компилятор вам поможет: он создаст все методы автоматически и будет задействовать все поля в таких методах, как `hashCode()`.

> **ПРИМЕЧАНИЕ** Можно сказать, что семантика «запись — это прозрачное хранилище для полей» определяет синтаксис: «методы доступа и другой шаблонный код автоматически выводятся из определения записи».

Чтобы посмотреть, как записи могут пригодиться в повседневном программировании, вспомните одну из самых частых жалоб по поводу Java: чтобы класс был полезным, приходится писать довольно много кода. Достаточно часто требуется создавать методы:

- `toString()`;
- `hashCode()` и `equals()`;
- методы чтения;
- открытый конструктор;

и т. д.

Для простых классов предметной области эти методы обычно получаются однообразными и утомительными. Их легко можно было бы генерировать автоматически (и многие IDE предоставляют такую возможность), но до появления записей язык не умел делать этого напрямую. Это раздражает еще сильнее, когда вы читаете чужой код. Например, может показаться, что автор задействует сгенерированные IDE версии `hashCode()` и `equals()`, в которых используются все поля класса, но как убедиться в этом, не проверяя каждую строку реализации? А что, если поле добавляется в процессе рефакторинга, а методы при этом не пересоздаются?

https://t.me/javalib

Записи решают подобные проблемы. Если тип объявляется как запись, это весьма серьезное заявление, и компилятор со средой выполнения должны относиться к нему соответственно. Давайте рассмотрим записи в действии.

Чтобы в полной мере объяснить эту функциональность, понадобится достаточно богатая предметная область, поэтому мы продолжим пример с валютными операциями. Не беспокойтесь, если вы не разбираетесь в этой теме: по ходу дела мы объясним все необходимые понятия. Далее в книге мы будем возвращаться к сфере финансов, так что это место станет хорошей отправной точкой.

Посмотрим, как с помощью записей и некоторых других возможностей усовершенствовать модель предметной области и получить более четкий, компактный и простой код. Возьмем ордер, который мы размещаем на валютных торгах. В простейшем случае он может состоять из таких данных:

- Количество покупаемых или продаваемых единиц (в миллионах денежных единиц).
- Сторона сделки: продаете вы или покупаете.
- Валюты, которые участвуют в сделке (*валютная пара*).
- Время размещения ордера.
- Срок, в течение которого ордер остается действительным («*время жизни*», или TTL — time-to-live).

Таким образом, если у меня есть миллион фунтов, я хочу продать их за доллары США в течение следующей секунды и за каждый фунт хочу получить 1,25 доллара, то я объявляю, что «покупаю GBP/USD по ставке 1,25 доллара, ордер размещен сейчас и актуален в течение 1 секунды». В Java для этого можно было бы объявить такой класс предметной области (суффикс `classic` указывает, что пока мы используем традиционный класс; более эффективные решения последуют позже):

```
public final class FXOrderClassic {
    private final int units;            // количество единиц валюты
    private final CurrencyPair pair;    // валютная пара
    private final Side side;            // сторона сделки
    private final double price;         // цена
    private final LocalDateTime sentAt; // время размещения
    private final int ttl;              // TTL

    public FXOrderClassic(int units, CurrencyPair pair, Side side,
                          double price, LocalDateTime sentAt, int ttl) {
        this.units = units;
        this.pair = pair; // CurrencyPair — простое перечисление
        this.side = side; // Side — простое перечисление
        this.price = price;
        this.sentAt = sentAt;
        this.ttl = ttl;
    }
```

https://t.me/javalib

```
public int units() {
    return units;
}

public CurrencyPair pair() {
    return pair;
}

public Side side() {
    return side;
}

public double price() {
    return price;
}

public LocalDateTime sentAt() {
    return sentAt;
}

public int ttl() {
    return ttl;
}

@Override
public boolean equals(Object o) {
    if (this == o) return true;
    if (o == null || getClass() != o.getClass()) return false;

    FXOrderClassic that = (FXOrderClassic) o;

    if (units != that.units) return false;
    if (Double.compare(that.price, price) != 0) return false;
    if (ttl != that.ttl) return false;
    if (pair != that.pair) return false;
    if (side != that.side) return false;
    return sentAt != null ? sentAt.equals(that.sentAt) :
                            that.sentAt == null;
}

@Override
public int hashCode() {
    int result;
    long temp;
    result = units;
    result = 31 * result + (pair != null ? pair.hashCode() : 0);
    result = 31 * result + (side != null ? side.hashCode() : 0);
    temp = Double.doubleToLongBits(price);
    result = 31 * result + (int) (temp ^ (temp >>> 32));
    result = 31 * result + (sentAt != null ? sentAt.hashCode() : 0);
    result = 31 * result + ttl;
    return result;
}

@Override
```

https://t.me/javalib

```
    public String toString() {
        return "FXOrderClassic{" +
               "units=" + units +
               ", pair=" + pair +
               ", side=" + side +
               ", price=" + price +
               ", sentAt=" + sentAt +
               ", ttl=" + ttl +
               '}';
    }
}
```

Код получается довольно длинным, зато ордер можно создать в таком виде:

```
var order = new FXOrderClassic(1, CurrencyPair.GBPUSD, Side.Bid,
                               1.25, LocalDateTime.now(), 1000);
```

Но весь ли этот код действительно необходим, чтобы объявить класс? В прежних версиях Java многие разработчики, вероятно, просто объявят поля, а потом поручат IDE автоматически сгенерировать все методы. Давайте посмотрим, как записи улучшают ситуацию.

> **ПРИМЕЧАНИЕ** В Java нет возможности описывать совокупности данных иначе, чем путем определения классов, поэтому очевидно, что любой тип, который содержит «только поля», будет классом.

Новое понятие — *класс записи* (или обычно просто *запись*). Это неизменяемое (в обычном для Java смысле «все поля — конечные») прозрачное хранилище для фиксированного набора значений, которые называются *компонентами* записи. Для каждого компонента создается конечное поле, в котором хранится предоставленное значение, и метод доступа, с помощью которого можно получить это значение. Имена поля и метода доступа совпадают с именем компонента.

Список полей формирует *описание состояния* для записи. В обычном классе поле `x`, аргумент конструктора `x` и метод доступа `x()` не обязательно должны быть связаны между собой, но в записи они *по определению* относятся к одной и той же сущности — сама запись *является* своим состоянием.

Чтобы можно было создавать новые экземпляры классов записей, также генерируется конструктор (он называется *каноническим конструктором*), чей список параметров в точности соответствует объявленному описанию состояния. В языке Java также есть сокращенный синтаксис объявления записей, при котором от программиста требуется лишь объявить имена и типы компонентов, которые образуют запись:

```
public record FXOrder(int units,
                      CurrencyPair pair,
                      Side side,
                      double price,
                      LocalDateTime sentAt,
                      int ttl) {}
```

https://t.me/javalib

Такое объявление записи не только уменьшает объем кода, но и делает сильное семантическое заявление. Тип `FXOrder` представляет собой всего лишь переданное состояние, и любой его экземпляр — не более чем прозрачная совокупность значений полей.

Если теперь проанализировать класс при помощи утилиты `javap` (которую мы подробнее рассмотрим в главе 4), можно увидеть, что компилятор автоматически сгенерировал шаблонный код:

```
$ javap FXOrder.class
Compiled from "FXOrder.java"
public final class FXOrder extends java.lang.Record {
  public FXOrder(int, CurrencyPair, Side,
                 double, java.time.LocalDateTime, int);

  public java.lang.String toString();
  public final int hashCode();
  public final boolean equals(java.lang.Object);
  public int units();
  public CurrencyPair pair();
  public Side side();
  public double price();
  public java.time.LocalDateTime sentAt();
  public int ttl();
}
```

Примечательно, что это очень похоже на набор методов, которые нам пришлось бы написать, если бы мы реализовывали код на базе класса. Собственно, конструктор и методы доступа ведут себя точно так же, как раньше. Однако реализация таких методов, как `toString()` и `equals()`, может насторожить некоторых разработчиков:

```
public java.lang.String toString();
    Code:
        0: aload_0
        1: invokedynamic #51, 0 // InvokeDynamic #0:toString:
                                // (LFXOrder;)Ljava/lang/String;
        6: areturn
```

Получается, что метод `toString()` (как и методы `equals()` и `hashCode()`) реализуется через механизм на базе `invokedynamic`. Это мощный прием, который мы рассмотрим позднее (в главах 4 и 16).

Также в этом фрагменте встречается новый класс `java.lang.Record`, который служит надтипом для всех классов записей. Это абстрактный класс, который объявляет методы `equals()`, `hashCode()` и `toString()` абстрактными. Класс `java.lang.Record` нельзя расширять напрямую, в чем легко убедиться, если попробовать откомпилировать код следующего вида:

```
public final class FXOrderClassic extends Record {
    private final int units;
    private final CurrencyPair pair;
```

https://t.me/javalib

```
    private final Side side;
    private final double price;
    private final LocalDateTime sentAt;
    private final int ttl;

    // ... Оставшая часть класса опущена
}
```

Компилятор отказывается это делать:

```
$ javac FXOrderClassic.java
FXOrderClassic.java:3: error: records cannot directly extend Record[1]
public final class FXOrderClassic extends Record {
                   ^
1 error
```

Создать запись можно только одним способом: явно объявить ее и поручить `javac` создать файл класса. При этом также гарантируется, что все классы записей будут создаваться как конечные.

У некоторых других базовых возможностей Java тоже появляются особенности в применении к записям. Во-первых, записи должны соблюдать специальный контракт в отношении метода `equals()`: если запись `R` содержит компоненты `c1`, `c2`, ... `cn` и экземпляр записи копируется следующим образом:

```
R copy = new R(r.c1(), r.c2(), ..., r.cn());
```

то `r.equals(copy)` должно возвращать `true`. Имейте в виду, что этот инвариант дополняет обычный контракт, касающийся `equals()` и `hashCode()`, а не заменяет его.

А сейчас перейдем к аспектам записей, которые в большей степени относятся к уровню проектирования. Для этого будет полезно вспомнить, как работают перечисления в Java. Перечисление — это особая форма класса, которая реализует паттерн проектирования «Ограниченное количество типобезопасных экземпляров» с минимальными синтаксическими излишествами: компилятор генерирует большой объем кода за вас.

Аналогичным образом запись Java — это особая форма класса, которая реализует паттерн «Носитель данных» (Data Carrier), он же «Хранилище полей» (Just Holds Fields) с минимальным синтаксисом. Компилятор автоматически генерирует весь шаблонный код, который нам потребуется. Но хотя простая концепция класса — носителя данных, который содержит только поля, интуитивно понятна, давайте все-таки разберемся, что она означает на более глубоком уровне.

Когда записи только начали обсуждаться, рассматривалось много разных вариантов их проектирования, в том числе:

- шаблонная редуцированная форма POJO;
- Java Beans 2.0;

[1] Ошибка: записи не могут напрямую расширять `Record`. — *Примеч. пер.*

https://t.me/javalib

- именованные кортежи;
- типы-произведения (разновидность алгебраического типа данных).

Брайан Гетц подробнее проанализировал эти возможности в своем проектировочном обзоре (http://mng.bz/M5j8). Каждый вариант порождает дополнительные вопросы, которые связаны с предполагаемым проектировочным решением записей, например такие:

- Сможет ли Hibernate проксировать их?
- Полностью ли они совместимы с классическими Java Beans?
- Позволяют ли они отвязывать имена, чтобы записи одинаковой «формы» можно было преобразовывать друг в друга?
- Будут ли они поддерживать сопоставление с шаблонами и деструктуризацию?

Функциональность записей вполне могла опираться на любой из четырех перечисленных вариантов — у каждого есть свои достоинства и недостатки. Тем не менее в итоге было решено, что записи будут *именованными кортежами*. Отчасти на решение повлияла одна из ключевых идей системы типов Java — *номинативная типизация* (nominal typing). Давайте поближе познакомимся с этим понятием.

3.3.1. Номинативная типизация

Номинативный подход к статической типизации заключается в том, что у каждой единицы хранимой информации в Java (переменные, поля) есть определенный тип, а у каждого типа есть имя, которое должно быть (по крайней мере в какой-то степени) осмысленным для человека.

Даже в случае анонимных классов у типов все равно есть имена — просто они назначаются компилятором и не являются действительными именами типов в языке Java (хотя остаются допустимыми для JVM). Например, в `jshell` можно увидеть следующее:

```
jshell> var o = new Object() {
    ...> public void bar() { System.out.println("bar!"); }
    ...> }
o ==> $0@37f8bb67

jshell> var o2 = new Object() {
    ...> public void bar() { System.out.println("bar!"); }
    ...> }
o2 ==> $1@31cefde0

jshell> o = o2;
| Error:
| incompatible types: $1 cannot be converted to $0[1]
| o=o2;
|   ^^
```

[1] Ошибка: несовместимые типы: `$1` невозможно преобразовать в `$0`. — *Примеч. пер.*

https://t.me/javalib

Обратите внимание: хотя анонимные классы объявлены абсолютно одинаково, тем не менее компилятор все равно генерирует два разных анонимных класса, `$0` и `$1`, и запрещает присваивание, потому что в системе типов Java переменные относятся к разным типам.

> **ПРИМЕЧАНИЕ** В некоторых языках, отличных от Java, общая форма класса (то есть содержащиеся в нем поля и методы) может использоваться как тип (вместо явного указания имени типа). Это называется *структурной типизацией*.

Если бы записи нарушили традиции Java и стали поддерживать структурную типизацию, это стало бы очень серьезным изменением. В результате выбор варианта «Записи — номинативные кортежи» означает, что записи с большой вероятностью будут лучше работать там, где в других языках подошли бы кортежи. Сюда входят такие сценарии использования, как составные ключи отображений или методы, которые возвращают несколько значений. Например, составной ключ отображения может выглядеть так:

```
record OrderPartition(CurrencyPair pair, Side side) {}
```

И наоборот, записи не всегда хороши как замена существующего кода, который использует Java Beans. Тому есть много причин: прежде всего Java Beans изменяемы, а записи — нет, а их методы доступа используют разные соглашения. У записей имена методов доступа совпадают с именами полей (это возможно, потому что те и другие имена принадлежат к разным пространствам имен в Java), а у Beans они снабжаются суффиксами `get` и `set`.

Записи оказываются более гибкими, чем форма с простым однострочным объявлением, потому что они являются полноценными классами, а именно: в дополнение к автоматически сгенерированным компонентам разработчик может определять дополнительные методы, конструкторы и статические поля. Впрочем, этими возможностями следует пользоваться осторожно. Не забывайте: записи создавались для того, чтобы можно было группировать взаимосвязанные поля в едином неизменяемом элементе данных.

Например, в качестве дополнительного метода для записи можно создать статический фабричный метод, который моделирует значения по умолчанию для некоторых параметров записи. Еще один пример — класс `Person` (с неизменяемой датой рождения), в котором определен метод `currentAge()`.

Хорошее практическое правило: чем больше вам хочется добавлять дополнительные методы к простому хранилищу данных (или заставлять его реализовывать несколько интерфейсов), тем вероятнее, что вместо записи лучше использовать полноценный класс.

3.3.2. Компактные конструкторы записей

Одно из важных исключений, когда запись не обязательно переделывать в полноценный класс, — использование *компактных конструкторов*, которые так описываются в спецификации языка:

https://t.me/javalib

> Формальные параметры компактного конструктора класса записи объявляются неявно. Они задаются производным списком формальных параметров класса записи.
>
> Объявление компактного конструктора вводится с расчетом на то, чтобы в теле канонического конструктора мог понадобиться только код проверки и/или нормализации; остальной код инициализации предоставляется компилятором.
>
> — *Спецификация языка Java*

Например, можно проверять ордера, чтобы убедиться, что никто не пытается покупать или продавать отрицательные денежные суммы или задавать недопустимый срок жизни:

```
public record FXOrder(int units, CurrencyPair pair, Side side,
                      double price, LocalDateTime sentAt, int ttl) {
    public FXOrder {
        if (units < 1) {
            throw new IllegalArgumentException(
                "FXOrder units must be positive");
                // Количество единиц FXOrder должно быть положительным
        }
        if (ttl < 0) {
            throw new IllegalArgumentException(
                "FXOrder TTL must be positive, or 0 for market orders");
                // Срок жизни FXOrder должен быть положительным
                // или 0 для рыночных ордеров
        }
        if (price <= 0.0) {
            throw new IllegalArgumentException(
            "FXOrder price must be positive");
             // Цена FXOrder должна быть положительной
        }
    }
}
```

Одно из преимуществ записей Java перед анонимными кортежами из других языков заключается в том, что при создании записей можно выполнять код в теле конструктора записи. Это позволяет выполнить проверку и возбудить исключение, если передается некорректное состояние. В чисто структурных кортежах это было бы невозможно.

В теле записи также могут быть уместны статические фабричные методы — скажем, чтобы преодолеть ту проблему, что в Java не бывает значений параметров по умолчанию. В нашем примере можно добавить такую статическую фабрику, чтобы было удобнее создавать ордера с параметрами по умолчанию:

```
public static FXOrder of(CurrencyPair pair, Side side, double price) {
    var now = LocalDateTime.now();
    return new FXOrder(1, pair, side, price, now, 1000);
}
```

https://t.me/javalib

Конечно, этот метод также можно было объявить как альтернативный конструктор. Вам стоит решать, какой подход будет лучше в каждом конкретном случае.

Также с помощью альтернативных конструкторов можно создавать записи, которые используются в качестве составных ключей отображения, как в этом примере:

```
record OrderPartition(CurrencyPair pair, Side side) {
    public OrderPartition(FXOrder order) {
        this(order.pair(), order.side());
    }
}
```

В результате тип `OrderPartition` можно легко использовать как ключ отображения. Например, можно построить биржевой стакан, который будет использоваться системой сопоставления ордеров:

```
public final class MatchingEngine {
    private final Map<OrderPartition, RankedOrderBook> orderBooks =
                                                          new TreeMap<>();
    public void addOrder(final FXOrder o) {
        orderBooks.get(new OrderPartition(o)).addAndRank(o);
        checkForCrosses(o.pair());
    }

    public void checkForCrosses(final CurrencyPair pair) {
        // Соответствуют ли какие-либо ордера на покупку ордерам на продажу?
    }

    // ...
}
```

Затем, когда поступает новый ордер, метод `addOrder()` извлекает его нужную часть (состоящую из кортежа с валютной парой и стороной покупки/продажи) и на ее основе добавляет новый ордер в стакан с соответствующим диапазоном цен. Новый ордер может соответствовать тем, которые уже есть в биржевых стаканах (это называется сведе́нием ордеров); такая проверка выполняется в методе `checkForCrosses()`.

Иногда вместо компактного конструктора имеет смысл использовать полный, явно определенный канонический конструктор. Это может понадобиться, если в конструкторе будет выполняться реальная работа, — впрочем, существует не так много потенциальных сценариев, в которых это уместно для простых классов хранилищ данных. Тем не менее иногда это необходимо, например, чтобы создавать резервные копии входных параметров. Из этих соображений компилятор позволяет явно определить канонический конструктор, но прежде чем идти по этому пути, стоит дважды подумать.

Записи создавались как простые хранилища данных — разновидность кортежей, которая просто и логично вписывается в устоявшуюся систему типов

https://t.me/javalib

Java. Они помогают создавать более понятные и компактные классы предметной области во многих приложениях. Кроме того, благодаря записям разработчикам не приходится снова и снова вручную реализовывать одни и те же стандартные паттерны. Наконец, записи позволяют избавиться от таких библиотек, как Lombok.

Многие разработчики уже сообщают о том, что записи оказались для них очень полезны. Кроме того, записи хорошо сочетаются с другой новой возможностью, которая тоже появилась в Java 17, — запечатанными типами.

3.4. ЗАПЕЧАТАННЫЕ ТИПЫ

Перечисления в Java хорошо знакомы всем разработчикам. Они позволяют моделировать конечный набор альтернатив, которые представляют все возможные значения типа, — по сути, это типобезопасные константы.

Продолжая наш пример с валютными операциями, с помощью перечисления `OrderType` обозначим разные типы ордеров:

```
enum OrderType {
    MARKET,
    LIMIT
}
```

Это перечисление представляет два возможных типа ордеров: *рыночный* ордер, который исполняется по лучшей текущей цене, и *лимитный* ордер, который исполняется, только если заданная цена доступна. В платформенной реализации перечислений компилятор Java автоматически генерирует особую форму типа класса.

> **ПРИМЕЧАНИЕ** На самом деле среда выполнения обращается с библиотечным типом `java.lang.Enum` (который напрямую расширяют все классы перечислений) немного не так, как с другими классами, но здесь мы не будем углубляться в подробности.

Давайте декомпилируем это перечисление и посмотрим, что сгенерировал компилятор:

```
$ javap -c -p OrderType.class
final class OrderType extends java.lang.Enum<OrderType> {
  public static final OrderType MARKET;

  public static final OrderType LIMIT;

  ...
  // Частный конструктор
}
```

В файле класса все возможные значения перечисления определены как открытые статические конечные переменные, а конструктор является частным, поэтому нельзя создавать дополнительные экземпляры.

https://t.me/javalib

По сути, перечисление можно считать обобщением паттерна «Одиночка» (Singleton), разве что вместо одного экземпляра класса существует конечное количество таких экземпляров. Этот паттерн чрезвычайно полезен, в первую очередь потому, что он обеспечивает концепцию полноты: если у вас есть объект `OrderType`, отличный от `null`, можно быть полностью уверенными в том, что это либо экземпляр `MARKET`, либо экземпляр `LIMIT`.

Но если мы хотим моделировать в Java 11 много разных ордеров, придется выбирать из двух зол. Во-первых, можно завести один реализующий класс (или запись) `FXOrder` с полем состояния, в котором хранится фактический тип. Такой паттерн работает, потому что поле состояния относится к перечисляемому типу, и по его значению можно определить, какой тип на самом деле подразумевается для этого конкретного объекта. Очевидно, такое решение не оптимально, потому что программисту приходится отслеживать данные, которыми, по-хорошему, должна заниматься система типов. Также можно объявить абстрактный базовый класс `BaseOrder` и сделать его подклассами конкретные типы `MarketOrder` и `LimitOrder`.

Дело здесь в том, что язык Java всегда проектировался как открытый и расширяемый по умолчанию. Подклассы могут компилироваться спустя годы (и даже десятилетия) после того, как были скомпилированы соответствующие классы. В Java 11 поддерживались только две конструкции наследования: открытое наследование (по умолчанию) и запрет наследования (`final`).

Классы могут объявлять пакетно-доступный (package-private) конструктор, что фактически означает «может расширяться только другими членами того же пакета». Однако среда выполнения никак не мешает пользователям создавать новые классы в пакетах, которые не являются частью платформы, так что эта защита в лучшем случае получается неполной.

Если определить класс `BaseOrder`, то ничто не помешает третьей стороне создать класс `EvilOrder`, наследующий `BaseOrder`. Что еще хуже, это расширение может произойти через годы (или десятилетия) после того, как тип `BaseOrder` был откомпилирован, а это крайне нежелательно.

Таким образом, до сих пор разработчики были стеснены этим фактором, и если они хотели подстраховаться на будущее, им приходилось заводить поле, в котором будет храниться фактический тип `BaseOrder`. В Java 17 состояние дел изменилось и появился новый способ более точного управления наследованием: *запечатанные типы* (sealed types).

> **ПРИМЕЧАНИЕ** Эта возможность в той или иной форме присутствует в других языках программирования. Она стала особенно популярной в последние годы, хотя сама идея существует уже давно.

В Java запечатанные типы работают так: тип можно расширять, но только известным списком подтипов и никакими другими типами. Рассмотрим новый синтаксис для простого примера класса `Pet`, который моделирует домашнее животное (вскоре мы вернемся к примерам с валютными операциями):

https://t.me/javalib

```
public abstract sealed class Pet {
    private final String name;

    protected Pet(String name) {
        this.name = name;
    }

    public String name() {
        return name;
    }

    public static final class Cat extends Pet {
        public Cat(String name) {
            super(name);
        }

        void meow() {
            System.out.println(name() +" мяукает");
        }
    }

    public static final class Dog extends Pet {
        public Dog(String name) {
            super(name);
        }

        void bark() {
            System.out.println(name() +" лает");
        }
    }
}
```

Класс `Pet` снабжен модификатором `sealed` — это не ключевое слово, и его можно было использовать в Java до выхода этой версии. Без уточнений `sealed` означает, что класс можно расширять только внутри текущей единицы компиляции, а следовательно, подклассы должны быть вложены в текущий класс. Класс `Pet` также объявлен абстрактным, потому что нам не нужны дополнительные экземпляры `Pet` помимо объектов `Pet.Cat` и `Pet.Dog`. Таким образом у нас появился новый способ реализовать ранее описанный паттерн объектно-ориентированного моделирования без упоминавшихся выше недостатков.

Интерфейсы тоже можно запечатывать, и не исключено, что форма с интерфейсами будет применяться на практике чаще, чем с классами. Давайте посмотрим, что произойдет, если мы захотим добавить запечатывание, чтобы проще моделировать различные типы валютных ордеров:

```
public sealed interface FXOrder permits MarketOrder, LimitOrder {
    int units();
    CurrencyPair pair();
    Side side();
    LocalDateTime sentAt();
}

public record MarketOrder(int units,
```

https://t.me/javalib

```
                          CurrencyPair pair,
                          Side side,
                          LocalDateTime sentAt,
                          boolean allOrNothing) implements FXOrder {
    // Конструкторы и фабрики опущены
}

public record LimitOrder(int units,
                         CurrencyPair pair,
                         Side side,
                         LocalDateTime sentAt,
                         double price,
                         int ttl) implements FXOrder {

    // Конструкторы и фабрики опущены
}
```

Здесь заслуживает внимания ряд моментов. Во-первых, `FXOrder` теперь — запечатанный интерфейс. Во-вторых, в коде встречается второе новое ключевое слово `permits`, которое позволяет разработчику перечислить все допустимые реализации этого запечатанного интерфейса, а наши реализации — это записи.

> **ПРИМЕЧАНИЕ** Если используется permits, реализующие классы не обязательно размещать в том же файле: они могут быть отдельными единицами компиляции.

Наконец, стоит отметить приятный побочный эффект: поскольку `MarketOrder` и `LimitOrder` являются полноценными классами, у каждого из них может быть поведение, специфическое для его типа. Например, для рыночного ордера не нужно специально указывать цену: он просто использует лучшую цену, доступную в настоящий момент. С другой стороны, для лимитного ордера нужно задать желаемую цену, а также время, в течение которого мы пытаемся ее добиться (TTL). Эта задача не решалась бы так легко, если бы мы хранили «реальный тип» объекта в отдельном поле, потому что пришлось бы включить все методы всех подтипов в базовый тип или использовать уродливые понижающие приведения.

Теперь, когда мы пишем код с этими типами, мы точно знаем, что любой экземпляр `FXOrder`, который нам встречается, должен быть экземпляром либо `MarketOrder`, либо `LimitOrder`. Более того, компилятор тоже может использовать эту информацию. Библиотечный код теперь может безопасно предполагать, что это все возможные варианты, и клиентский код не нарушит это предположение.

Объектно-ориентированная модель Java представляет две наиболее фундаментальные концепции отношений между типами, а именно «Тип „*X* является частным случаем *Y*“» и «Тип „*X* содержит *Y*“». Запечатанные типы добавляют концепцию, которая раньше не моделировалась в Java: «Тип „X является либо Y, либо Z“». Такие типы также можно рассматривать как:

- Промежуточное звено между конечными и открытыми классами.
- Паттерн перечисления, который применяется к типам вместо экземпляров.

https://t.me/javalib

В терминах теории объектно-ориентированного программирования запечатанные типы представляют новую разновидность формальных отношений, потому что набор возможных типов для `o` является *объединением* `Y` и `Z`. Соответственно, во многих языках это называется *типом-объединением* (union type) или *типом-суммой* (sum type), но это совсем не то же самое, что `union` в языке C.

Например, на Scala аналогичную идею можно реализовать с помощью `case`-классов и собственной версии ключевого слова `sealed`, а с подходом Kotlin к этой концепции мы познакомимся позднее.

Если рассматривать примеры за пределами JVM, то в языке Rust тоже есть концепция *несвязных объединений* (disjoint union), но они обозначаются ключевым словом `enum`, что может сильно сбить с толку программистов на Java. В мире функционального программирования некоторые языки (например, Haskell) предоставляют так называемые *алгебраические типы данных*, которые содержат типы-объединения как специальный случай. Собственно, запечатанные типы в сочетании с записями реализуют подобную функциональность в Java 17.

На первый взгляд может показаться, что запечатанные типы — совершенно новое понятие в Java, однако их глубокое сходство с `enum` должно помочь освоиться многим программистам. Нечто похожее на эти типы уже встречалось в Java, а именно тип параметра исключения в секции `catch` для множественных исключений.

Из спецификации языка Java (JLS 11, раздел 14.20):

> Объявленный тип параметра исключения, который обозначает свой тип как объединение альтернатив D1 | D2 | ... | Dn, представляет собой lub[1](D1, D2, ..., Dn).

Тем не менее в секции `catch` для множественных исключений истинный тип объединения нельзя записать как тип локальной переменной: не предусмотрено соответствующего обозначения, и нельзя создать переменную такого типа.

И еще одно последнее замечание о запечатанных типах Java: у них должен быть базовый класс, который расширяют все допустимые типы (или общий интерфейс, который должны реализовывать все допустимые типы). Невозможно объявить тип, который «является `String` или `Integer`», потому что типы `String` и `Integer` не наследуют никакому общему предку помимо `Object`.

> **ПРИМЕЧАНИЕ** Некоторые языки позволяют конструировать обобщенные типы объединений, но в Java такая возможность не предусмотрена.

Перейдем к другой возможности языка, которая появилась в Java 17, — новой форме ключевого слова `instanceof`.

---

[1] lub = Least Upper Bound, то есть «наименьший верхний предел». — *Примеч. пер.*

https://t.me/javalib

3.5. НОВАЯ ФОРМА INSTANCEOF

Хотя оператор `instanceof` был частью языка еще с Java 1.0, иногда он пользуется дурной репутацией среди разработчиков. В простейшем виде он выполняет элементарную проверку: `x instanceof Y` возвращает `true`, если значение `x` можно присвоить переменной типа `Y`, и `false` в противном случае (при этом `null instanceof Y` возвращает `false` для любого `Y`).

Этот механизм критиковали за то, что он подрывает основы объектно-ориентированного проектирования, не обеспечивая точность в типах объектов и, возможно, в выборе типов параметров. Однако на практике разработчикам иногда приходится иметь дело с объектами, типы которых неизвестны полностью во время компиляции. Например, представьте себе объект, который получен посредством рефлексии, так что о нем ничего (или почти ничего) не известно.

В такой ситуации уместно воспользоваться `instanceof` и проверить, что тип совпадает с ожидаемым, а затем выполнить понижающее приведение. Проверка `instanceof` служит защитой, которая гарантирует, что приведение не возбудит исключение `ClassCastException` во время выполнения. Код выглядит примерно так:

```
Object o = // ...
if (o instanceof String) {
    String s = (String)o;
    System.out.println(s.length());
} else {
    System.out.println("Это не строка");
}
```

С точки зрения разработчика, новая функциональность `instanceof`, которая появилась в Java 17, очень проста — она всего лишь позволяет избежать приведения, как в следующем примере:

```
if (o instanceof String s) {
    System.out.println(s.length());   <-- Находится в области видимости этой ветви
} else {
    System.out.println("Это не строка");   <-- Не находится в области видимости ветви else
}
                                  <-- Не находится в области видимости после того,
                                      как инструкция if завершилась
// ... Последующий код
```

Хотя на первый взгляд это может показаться несущественным, но есть определенный смысл в том, как эта функциональность названа в JEP. Документ JEP 394 назывался «Pattern Matching for instanceof» («Сопоставление с шаблонами для instanceof»), и в нем вводится новое понятие — *шаблон*.

> **ПРИМЕЧАНИЕ** Очень важно понимать, что термин «сопоставление с шаблонами» в этом контексте — это не то же самое, что поиск по шаблону, который используется в обработке текста и регулярных выражениях.

https://t.me/javalib

В этом контексте шаблоном называется комбинация двух составляющих:

1. Предикат (условие), которое будет применено к значению.
2. Набор локальных переменных, называемых *шаблонными переменными* (pattern variables), которые должны быть извлечены из значения.

Здесь принципиально то, что шаблонные переменные извлекаются, только если предикат был успешно применен к значению.

В Java 17 оператор `instanceof` был расширен, чтобы принимать как тип, так и *шаблон типа*, где шаблон состоит из предиката, задающего тип, и одной шаблонной переменной.

ПРИМЕЧАНИЕ Шаблоны типов подробнее рассматриваются в следующем разделе.

Возможно, обновленный вариант `instanceof` не кажется чем-то выдающимся, но это пока первый случай применения шаблонов в языке Java — как вы увидите, на подходе и другие!

После обзора новых возможностей языка Java 17 давайте заглянем в будущее и вернемся к теме предварительной функциональности.

3.6. СОПОСТАВЛЕНИЕ С ШАБЛОНАМИ И ПРЕДВАРИТЕЛЬНАЯ ФУНКЦИОНАЛЬНОСТЬ

В главе 1 мы представили понятие предварительной функциональности, но не смогли привести хороший пример, потому что в Java 11 такой функциональности не было. Теперь, когда речь идет о Java 17, можно продолжить эту тему.

Все новые средства языка, о которых говорилось в этой главе, — в том числе выражения `switch`, записи и запечатанные типы, прошли один и тот же жизненный цикл. Сначала они появились как предварительная функциональность, а затем преодолели один или несколько циклов открытых испытаний, прежде чем воплотиться в окончательном виде. Например, запечатанные классы были предварительно введены в Java 15, потом еще раз в Java 16 и только после этого стали официальной функциональностью в Java 17 LTS.

В этом разделе мы рассмотрим предварительную функциональность, которая расширяет сопоставление с шаблонами с `instanceof` до `switch`. В Java 17 эта возможность включена как первая версия предварительной функциональности (за дополнительной информацией о предварительной функциональности обращайтесь к главе 1). По дороге к окончательному выпуску синтаксис может измениться, и даже сама функциональность может быть исключена (хотя для сопоставления с шаблонами это крайне маловероятно).

Давайте посмотрим, как можно использовать сопоставление с шаблоном, чтобы улучшить простой код, предназначенный для работы с объектами неизвестного

https://t.me/javalib

типа. С помощью новой формы `instanceof` можно написать безопасный код такого вида:

```
Object o = // ...

if (o instanceof String s) {
    System.out.println("Строка длины:"+ s.length());
} else if (o instanceof Integer i) {
    System.out.println("Целое число:"+ i);
} else {
    System.out.println("Это не строка и не целое число");
}
```

Однако такой код быстро становится громоздким и однообразным. Вместо этого можно включить в выражения `switch` шаблоны типов, а также уже знакомые простые логические выражения `instanceof`. В синтаксисе текущей предварительной функциональности (Java 17) код можно упростить так:

```
var msg = switch (o) {
    case String s -> "Строка длины:"+ s.length();
    case Integer i -> "Целое число:"+ i;
    case null, default -> "Это не строка и не целое число";
};
System.out.println(msg);
```

null теперь допускается как метка case, чтобы предотвратить возможные исключения NullPointerException

Если вы захотите поэкспериментировать с таким кодом, позвольте объяснить, как проводить сборку и запуск с предварительной функциональностью. Если попытаться откомпилировать код с предварительной функциональностью, как в приведенном примере, компилятор выдаст ошибку:

```
$ javac ch3/Java17Examples.java
ch3/Java17Examples.java:68: error: patterns in switch statements are a
  preview feature and are disabled by default.[1]

            case String s -> "String of length:"+ s.length();
                 ^
  (use --enable-preview to enable patterns in switch statements)[2]
1 error
```

Компилятор подсказывает, что нам, возможно, стоит включить предварительную функциональность, поэтому сделаем вторую попытку с установленным флагом:

```
$ javac --enable-preview -source 17 ch3/Java17Examples.java
Note: ch3/Java17Examples.java uses preview features of Java SE 17.[3]
```

[1] Ошибка: шаблоны в выражениях switch относятся к предварительной функциональности и отключены по умолчанию. — *Примеч. пер.*

[2] Используйте --enable-preview, чтобы включить шаблоны в выражениях switch. — *Примеч. пер.*

[3] Примечание: ch3/Java17Examples.java использует предварительную функциональность из Java SE 17. — *Примеч. пер.*

https://t.me/javalib

```
Note: Recompile with -Xlint:preview for details.
```

На стадии выполнения все происходит почти так же:

```
$ java ch3.Java17Examples
Error: LinkageError occurred while loading main class ch3.Java17Examples
    java.lang.UnsupportedClassVersionError: Preview features are not enabled
  for ch16/Java17Examples (class file version 61.65535). Try running with
  '--enable-preview'
```

Если и здесь установить флаг предварительной функциональности, то код наконец заработает:

```
$ java --enable-preview ch13.Java17Examples
```

Необходимость постоянно включать предварительную функциональность раздражает, но так было задумано для того, чтобы код, который использует незавершенную функциональность, не проник в эксплуатационную версию и не создал там проблем. Также стоит обратить внимание на сообщение о версии файла класса, которое появилось, когда мы попытались запустить класс с предварительной функциональностью без флага `--enable-preview`. Если явно откомпилировать код с поддержкой предварительной функциональности, то полученный файл класса будет нестандартным, и такой код почти никогда не должен выполняться в эксплуатационных версиях продукта.

Предварительная версия сопоставления с шаблонами в Java 17 также умеет интегрироваться с запечатанными типами, а именно: шаблоны могут воспользоваться тем фактом, что запечатанные типы обеспечивают полноту возможных типов. Например, при обработке ответов на ордера можно использовать следующий базовый тип:

```
public sealed interface FXOrderResponse
        permits FXAccepted, FXFill, FXReject, FXCancelled {
    LocalDateTime timestamp();
    long orderId();
}
```

Если объединить это с выражением `switch` и шаблонами типов, мы получим код такого вида:

```
FXOrderResponse resp = // ... Ответ от рынка
var msg = switch (resp) {
    case FXAccepted a -> a.orderId() + " Принят";
    case FXFill f -> f.orderId() + " Выполнен "+ f.units();
    case FXReject r -> r.orderId() + " Отклонен: "+ r.reason();
    case FXCancelled c -> c.orderId() + " Отменен";
    case null -> "Ордер пуст";
};
System.out.println(msg);
```

Обратите внимание, что: 1) мы явно включаем секцию для `null`, чтобы гарантировать `null`-безопасность кода (и предотвратить `NullPointerException`), и 2) секция `default` не нужна. Последнее объясняется тем, что компилятор может

https://t.me/javalib

проанализировать все допустимые подтипы `FXOrderResponse` и заключить, что шаблон покрывает все возможные варианты, так что секция `default` не будет выполняться ни в какой ситуации. Если бы обрабатываемый набор не был полон и не охватывал бы какие-то случаи, то секция `default` была бы необходима.

В первую предварительную версию также входят *защитные шаблоны*: эта возможность позволяет декорировать шаблон логическим защитным условием, чтобы общий шаблон считался совпадающим, только если и шаблон и защитное условие оба истинны. Например, допустим, что вы хотите просмотреть подробности только по большим выполненным ордерам. В этом случае секцию `fill` в предыдущем примере можно изменить так:

```
case FXFill f && f.units() < 100 -> f.orderId() + " Небольшой ордер";
case FXFill f -> f.orderId() + " Выполнено "+ f.units();
```

Обратите внимание: сначала проверяется более частный случай (небольшие ордера менее чем на 100 единиц), и только при неудаче проверяется следующий случай — без защитного шаблона. Переменная шаблона также уже находится в области видимости для любых защитных условий. Мы вернемся к сопоставлению с шаблонами в главе 18, когда пойдет речь о будущем развитии Java, и рассмотрим некоторые возможности, которые не успели реализоваться к версии Java 17.

ИТОГИ

- В Java 17 появился ряд новых возможностей, которые уже приносят пользу разработчикам, применяющим их в своем коде:
  - Текстовые блоки для строковых литералов, охватывающих несколько физических строк.
  - Выражения `switch` для более современных конструкций выбора.
  - Записи как прозрачные носители данных.
  - Запечатанные типы — важное новое понятие объектно-ориентированного моделирования.
  - Сопоставление с шаблонами: хотя эта возможность еще не полностью реализована в Java 17, она наглядно демонстрирует направление развития языка в будущих версиях.

https://t.me/javalib

Часть II

Что там внутри?

Эта часть книги полностью посвящена тому, как *фактически* работает JVM. Мы начнем с загрузки классов. Многие разработчики на Java недостаточно хорошо понимают, как работает виртуальная машина Java, как она загружает, компонует и проверяет классы. Это приводит к неприятностям и потере времени, когда из-за конфликта загрузчика классов выполняется «неправильная» версия какого-нибудь класса.

Умение разбираться во внутреннем устройстве файлов классов Java и содержащемся в них байт-коде — очень полезный навык отладки. Вы узнаете, как при помощи `javap` ориентироваться в байт-коде и понимать его смысл.

Затем речь пойдет о революции многоядерных процессоров, которая происходит в области оборудования. Основательный Java-разработчик должен разбираться в возможностях конкурентного выполнения Java. Главы 5 и 6 научат вас извлекать максимум пользы из потенциала современных процессоров. Сначала мы рассмотрим теорию конкурентного выполнения и структурные элементы, которые применялись в этой области в Java с 2006 года (Java 5). Вы узнаете о модели памяти Java и о том, как в ней реализована обработка потоков и конкурентное выполнение.

Когда у вас появится теоретическая подготовка, мы рассмотрим функциональность пакета `java.util.concurrent`, который предоставляет современные структуры для практической разработки конкурентного кода на Java.

Оптимизацию быстродействия часто рассматривают как искусство, а не как науку. Выявление и исправление дефектов быстродействия часто требует невероятных затрат времени и усилий. В главе 7 — последней главе этой части — вы научитесь измерять, а не предполагать и узнаете, что «оптимизация по народным рецептам» — пагубная практика. Вы освоите научный подход, который позволит быстро выявлять истинные причины проблем с быстродействием.

В частности, мы сосредоточимся на сборке мусора (GC, Garbage Collection) и JIT-компиляторе (Just-In-Time) — двух основных компонентах JVM, от которых зависит быстродействие. Среди прочего вы научитесь читать журналы сборщика мусора и освоите бесплатную программу Java VisualVM (`jvisualvm`), которая анализирует использование памяти.

К концу части II ваш кругозор выйдет далеко за рамки исходного кода, который редактируется в IDE. Вы узнаете, как работают внутренние реализации Java и JVM и сможете использовать весь потенциал, который предлагает, пожалуй, самая мощная в мире виртуальная машина общего назначения.

https://t.me/javalib

4 Файлы классов и байт-код

В ЭТОЙ ГЛАВЕ

- ✓ Загрузка классов
- ✓ Рефлексия
- ✓ Анатомия файлов классов
- ✓ Байт-код JVM и почему он важен

Один из самых верных способов стать опытным Java-разработчиком — глубже разобраться в том, как работает платформа. Для этого, в частности, будет полезно изучить такие базовые механизмы, как загрузка классов и устройство байт-кода JVM.

Представьте себе ситуацию, с которой может столкнуться любой квалифицированный Java-разработчик. Допустим, в приложении широко используются инструменты внедрения зависимостей (dependency injection, DI), такие как Spring. При запуске возникают проблемы, и приложение аварийно завершается с невразумительным сообщением об ошибке. Если дело не сводится к простой ошибке конфигурации, то возможно, чтобы диагностировать проблему, придется разобраться, как реализован фреймворк для внедрения зависимостей. А для этого нужно понимать, как работает загрузка классов.

Или предположим, что вы работали с вендором, который закрыл свой бизнес. У вас осталась последняя сборка откомпилированного кода, неполная документация и нет никаких исходников. Как исследовать откомпилированный код и понять, что он содержит?

https://t.me/javalib

Любое сколько-нибудь нетривиальное приложение может аварийно завершиться с исключением `ClassNotFoundException` или `NoClassDefFoundError`, но многие разработчики не знают, что означают эти исключения, чем они различаются и даже почему они вообще возникают.

В этой главе мы сосредоточимся на аспектах платформы, которые лежат в основе таких проблем. Мы также рассмотрим некоторые продвинутые возможности, но они предназначены для читателей, которые предпочитают разбираться во всем основательно. Если вы торопитесь, их можно пропустить.

Начнем с обзора загрузки классов — процесса, в ходе которого JVM находит и активирует новый тип, чтобы использовать его в действующей программе. Центральное место в этом материале занимают объекты `Class`, которые представляют типы в JVM. Затем мы посмотрим, как эти сущности встраиваются в один из важнейших механизмов языка, называемый *рефлексией* (reflection).

Далее мы поговорим об инструментах, которые позволяют изучать и анализировать файлы классов. Нашим основным инструментом станет утилита `javap`, которая поставляется в составе JDK. Разобравшись с анатомией файла класса, мы обратимся к байт-коду. Вы познакомитесь с основными семействами кодов операций JVM и узнаете, как функционирует среда выполнения на низком уровне.

Итак, давайте займемся *загрузкой классов* (class loading) — процедурой, с помощью которой новые классы включаются в действующий процесс JVM. В этом разделе мы сначала рассмотрим основы «классической» загрузки классов, которая работала в Java 8 и ранее. Позже в этой главе поговорим о том, как появление модульной JVM внесло некоторые (впрочем, незначительные) изменения в загрузку классов.

4.1. ЗАГРУЗКА КЛАССОВ И ОБЪЕКТЫ КЛАССОВ

Файл .class определяет тип для JVM вместе со всеми полями, методами, информацией о наследовании, аннотациями и прочими метаданными. Формат файла класса хорошо описан в стандартах, и его должен придерживаться любой язык, программы на котором рассчитаны на запуск на JVM.

> **ПРИМЕЧАНИЕ** Класс — это фундаментальная единица программного кода, которую воспринимает и выполняет платформа Java.

Значительная часть механизма загрузки классов остается скрытой от начинающих разработчиков. Разработчик предоставляет либо исполняемый файл JAR, либо имя главного класса приложения (который должен находиться в classpath), а JVM находит и выполняет класс.

Любые зависимости приложения (например, библиотеки, отличные от JDK) тоже должны входить в classpath, и JVM аналогично находит и загружает их. Однако в спецификации Java не указано, должно ли это происходить при запуске приложения или позже, когда возникнет такая необходимость.

https://t.me/javalib

ПРИМЕЧАНИЕ Система загрузки классов Java предоставляет пользователям относительно простой API, большáя часть сложности которого скрыта намеренно. API, доступный для разработчиков, будет рассмотрен позднее в этой главе.

Начнем с очень простого примера:

```
Class<?> clazz = Class.forName("MyClass");
```

Этот блок кода загружает класс `MyClass` в текущее состояние выполнения. С точки зрения JVM для этого необходимо выполнить ряд действий. Сначала нужно найти файл класса, который соответствует имени `MyClass`, а затем выполнить разрешение содержащегося в нем класса. Эти шаги выполняются в низкоуровневом коде — в HotSpot соответствующий «родной» метод называется `JVM_DefineClass()`.

В общих чертах процедура состоит в том, что низкоуровневый код строит внутреннее представление JVM, называемое *klass-объектом*[1] (который не является объектом Java — эта тема рассматривается в главе 17). Затем при условии, что klass-объект удалось извлечь из файла класса, JVM конструирует совместимый с Java аналог klass-объекта, который передается обратно коду Java как объект `Class`.

После этого объект `Class`, представляющий тип, становится доступным для действующей системы, и можно создавать его новые экземпляры. В предыдущем примере в переменной `clazz` в итоге сохраняется объект `Class`, соответствующий типу `MyClass`. В ней не может храниться klass-объект, потому что это внутренний объект JVM, а не объект Java.

ПРИМЕЧАНИЕ Аналогичный процесс используется для главного класса приложения, всех его зависимостей и всех остальных классов, которые могут потребоваться после запуска программы.

В этом разделе мы подробнее рассмотрим эту процедуру с точки зрения JVM и познакомимся с загрузчиками классов — объектами, которые управляют всем процессом.

4.1.1. Загрузка и компоновка

JVM, в частности, можно рассматривать как контейнер выполнения. С этой точки зрения задача JVM — потреблять файлы классов и выполнять байт-код, который в них содержится. Для этого JVM должна принять содержимое файла класса в виде потока байтов, преобразовать его в форму, пригодную для использования, и добавить в состояние выполнения. По сути, именно этот процесс будет описан в этом разделе.

Эту довольно сложную процедуру можно по-разному разделить на составляющие, но мы будем рассматривать этапы *загрузки* и *компоновки*.

[1] Это не опечатка — именно klass, а не class. — *Примеч. пер.*

https://t.me/javalib

ПРИМЕЧАНИЕ Говоря о загрузке и компоновке, мы будем упоминать некоторые подробности, специфические для кода HotSpot, однако от других реализаций тоже стоит ожидать чего-то похожего.

Первый шаг — получить поток байтов, который образует файл класса. Эта операция начинается с массива байтов, который часто считывается из файловой системы (хотя возможны и другие варианты).

Получив поток данных, его нужно разобрать и убедиться, что он содержит действительную структуру файла класса (иногда это называется *проверкой формата*). Если все в порядке, создается потенциальный klass-объект, или *klass-кандидат*. На этой фазе, пока klass-кандидат заполняется содержимым, происходят некоторые базовые проверки. (Например: может ли загружаемый класс обратиться к своему объявленному надклассу? не пытается ли он переопределить конечные методы?)

Однако по окончании процесса загрузки структура данных, соответствующая классу, еще не подходит для того, чтобы другой код ее использовал. В частности, еще нет полностью функционального klass-объекта.

Чтобы класс можно было использовать, его нужно скомпоновать, а затем инициализировать. На логическом уровне этот шаг делится на три подфазы: верификацию, подготовку и разрешение. Однако реальные реализации не всегда четко отделяют соответствующие участки кода друг от друга, так что если вы планируете читать исходный код, имейте в виду, что здесь представлено высокоуровневое (концептуальное) описание процесса, которое не обязательно буквально соответствует коду из фактической реализации.

С учетом всего сказанного верификацию можно воспринимать как стадию, на которой проверяется, что класс соответствует требованиям спецификаций Java и не вызовет ошибок времени выполнения или других проблем в действующей системе. Это отношение между фазами компоновки изображено на рис. 4.1.

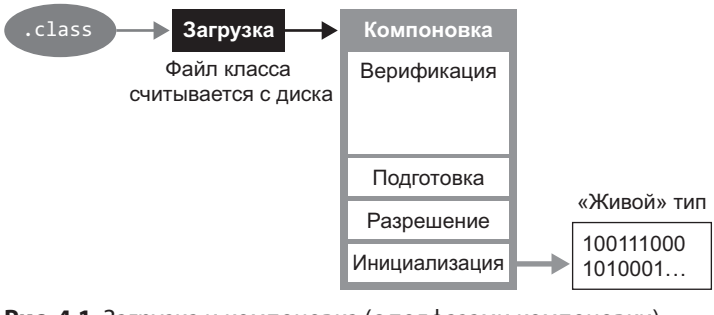


Рис. 4.1. Загрузка и компоновка (с подфазами компоновки)

Рассмотрим все фазы по порядку.

https://t.me/javalib

Верификация

Верификация — довольно сложный процесс, состоящий из нескольких независимых подзадач. Например, JVM необходимо убедиться в том, что множество символических имен, которое содержится в пуле констант (эта тема подробнее обсуждается в разделе 4.3.3), логически непротиворечиво и подчиняется общим правилам для констант.

Другая важная задача (и возможно, самая сложная часть верификации) — проверка байт-кода методов. В частности, нужно убедиться, что байт-код корректно себя ведет и не пытается обойти средства контроля среды JVM.

Перечислим несколько основных проверок:

- Убедиться, что байт-код не пытается выполнять недопустимые или вредоносные операции со стеком.
- Убедиться, что каждой инструкции ветвления (например, из `if` или цикла) соответствует правильная инструкция, к которой происходит переход.
- Проверить, что методы вызываются с правильным количеством параметров правильных статических типов.
- Проверить, что локальным переменным присваиваются только значения подходящих типов.
- Проверить, что для каждого исключения, которое может возбудиться, существует действительный обработчик `catch`.

Эти проверки выполняются по нескольким соображениям, в том числе и ради быстродействия. Они позволяют пропускать некоторые проверки времени выполнения, отчего интерпретируемый код выполняется быстрее. Некоторые из них также упрощают компиляцию байт-кода в машинный код на стадии выполнения (речь идет о JIT-компиляции, которую мы рассмотрим в главе 6).

Подготовка

В процессе подготовки класса выделяется память, а в класс включаются статические переменные, готовые к инициализации. Но пока переменные еще не инициализируются, и байт-код JVM не выполняется.

Разрешение

Разрешение (resolution) — это этап компоновки, на котором JVM проверяет, что надтип компонуемого класса (и все реализуемые им интерфейсы) уже скомпонованы, а если нет, то они компонуются до того, как продолжается компоновка класса. Это может вызвать рекурсивную компоновку для всех новых типов, которые ранее еще не встречались.

> **ПРИМЕЧАНИЕ** Ключевое понятие, которое относится к этой стадии загрузки классов, — *транзитивное замыкание типов*. Компоноваться должны не только типы, которым класс наследует напрямую, но и все типы, которые он использует косвенно.

https://t.me/javalib

После того как все дополнительные типы, которые нужно загрузить, будут обнаружены и разрешены, JVM может инициализировать класс, который изначально взялась загружать.

Инициализация

На этой последней фазе инициализируются все статические переменные и выполняются все статические блоки инициализации. Это важный момент, потому что только теперь JVM наконец-то начинает выполнять код из свежезагруженного класса.

Когда этот шаг завершится, класс полностью загружен и готов к работе. Класс доступен для среды выполнения, и можно создавать его экземпляры. Все дальнейшие операции загрузки классов, в которых задействован этот класс, теперь будут воспринимать его как загруженный и доступный.

4.1.2. Объекты Class

Конечный результат процесса компоновки и загрузки — объект `Class`, который представляет только что загруженный и скомпонованный тип. Теперь он полностью функционален в JVM, хотя по соображениям быстродействия некоторые компоненты объекта `Class` инициализируются только по необходимости.

> **ПРИМЕЧАНИЕ** Объекты Class — обычные объекты Java. Они обитают в куче Java, как и любые другие объекты.

Теперь ваш код может использовать новый тип и создавать новые экземпляры. Кроме того, объект `Class`, соответствующий тому или иному типу, предоставляет ряд полезных методов, например `getSuperclass()`, который возвращает объект `Class`, соответствующий надтипу.

Объекты `Class` можно использовать с Reflection API, чтобы косвенно обращаться к методам, полям, конструкторам и т. д. Объект `Class` содержит ссылки на объекты `Method`, `Field` и другие, которые соответствуют членам класса. Эти объекты тоже можно использовать в Reflection API, чтобы косвенно обращаться к функциональности класса, как будет показано позднее в этой главе. На рис. 4.2 изображена высокоуровневая структура `Class`.

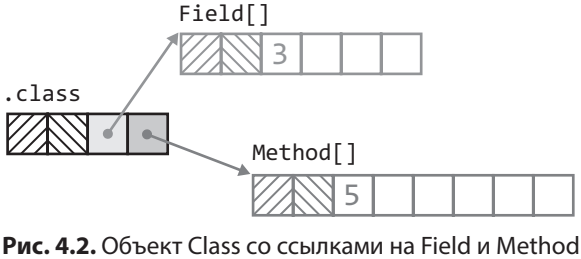


Рис. 4.2. Объект Class со ссылками на Field и Method

https://t.me/javalib

До сих пор мы не говорили о том, какая конкретно часть среды выполнения отвечает за поиск и компоновку потока байтов, который станет свежезагруженным классом. Эту задачу решают загрузчики классов — подклассы абстрактного класса `ClassLoader`, которым будет посвящен следующий раздел.

4.2. ЗАГРУЗЧИКИ КЛАССОВ

По сути, Java — это объектно-ориентированная система с динамической средой выполнения. В частности, это означает, что типы Java остаются «живыми» во время выполнения, а систему типов действующей платформы Java можно изменить, например добавить новые типы. Типы, которые образуют программу Java, можно расширять неизвестными типами во время выполнения (если только они не являются конечными или не относятся к недавно появившимся запечатанным классам). Функциональность загрузки классов открыта для пользователя. Загрузчики классов представляют собой обычные классы Java, которые наследуют `ClassLoader`, — они сами являются типами Java.

> **ПРИМЕЧАНИЕ** В современных средах Java все загрузчики классов являются модульными. Классы всегда загружаются в контексте модуля.

Класс `ClassLoader` содержит ряд низкоуровневых методов, в том числе тех, которые реализуют загрузку и компоновку, отвечающие за разбор файла класса, но пользовательские загрузчики классов не могут переопределить этот аспект загрузки. Невозможно написать загрузчик классов с использованием низкоуровневого кода.

Платформа поставляется со следующими типичными загрузчиками классов, которые выполняют разные задачи при запуске и штатной работе платформы:

- `BootstrapClassLoader` (или *базовый* загрузчик классов) создается на очень ранней стадии запуска JVM, поэтому обычно лучше рассматривать его как часть самой JVM. Как правило, он используется, чтобы загрузить самую минимальную систему — фактически `java.base`.
- `PlatformClassLoader`: после того, как минимальная система инициализирована, этот платформенный загрузчик классов загружает остальные платформенные модули, от которых зависит приложение. Этот загрузчик служит первичным интерфейсом для обращения к любым платформенным классам вне зависимости от того, были ли они загружены им самим или базовым загрузчиком. Он является экземпляром внутреннего класса.
- `AppClassLoader` — загрузчик прикладных классов, наиболее широко используемый загрузчик. Он загружает прикладные классы и выполняет бóльшую часть работы в большинстве современных сред Java. В модульной JVM этот загрузчик является уже не экземпляром `URLClassLoader` (как в Java 8 и более ранних версиях), а экземпляром внутреннего класса.

https://t.me/javalib

Чтобы увидеть новые загрузчики классов в действии, добавим код в начало метода `main` в `SiteCheck` из модуля `wgjd.sitecheck` из главы 2:

```
...
var clThis = SiteCheck.class.getClassLoader();
System.out.println(clThis);
var clObj = Object.class.getClassLoader();
System.out.println(clObj);
var clHttp = HttpClient.class.getClassLoader();
System.out.println(clHttp);
....
```

Перекомпилируем его:

```
$ javac -d out wgjd.sitecheck/module-info.java \
        wgjd.sitecheck/wgjd/sitecheck/*.java \
        wgjd.sitecheck/wgjd/sitecheck/*/*.java
```

а затем запустим следующей командой:

```
$ java -cp out wgjd.sitecheck.SiteCheck http://github.com/well-grounded-java
```

Обратите внимание на то, что мы указали «начальный модуль» вместо того, чтобы явно указывать начальный класс.

Мы получим такой результат:

```
jdk.internal.loader.ClassLoaders$AppClassLoader@277050dc
null
jdk.internal.loader.ClassLoaders$PlatformClassLoader@12bb4df8
http://github.com/well-grounded-java: HTTP_1_1
```

Вместо загрузчика классов для `Object` (находящегося в `java.base`) выводится `null`. Это мера безопасности: базовый загрузчик классов не выполняет верификацию и предоставляет полный незащищенный доступ ко всем классам, которые он загружает. Поэтому нецелесообразно, чтобы представление этого загрузчика было доступно в среде выполнения Java: это открывало бы слишком много возможностей для ошибок или злоупотреблений.

Загрузчики классов также часто используются не только по прямому назначению, но и для того, чтобы загружать ресурсы (файлы, которые не являются классами, например изображения или конфигурационные файлы) из файлов JAR или других каталогов из classpath. Эта возможность часто используется в сочетании с `try`, в результате чего получается код, подобный этому:

```
try (var is = TestMain.class.getResourceAsStream("/resource.csv");
    var br = new BufferedReader(new InputStreamReader(is));) {
    // ...
}
// Обработка исключений опущена
```

Загрузчики классов предоставляют этот механизм в разных формах, возвращая URL или `InputStream`.

https://t.me/javalib

4.2.1. Нестандартная загрузка классов

В более сложных средах часто используются дополнительные, *нестандартные* (custom) *загрузчики классов* — классы, наследующие `java.lang.ClassLoader` (прямо или косвенно). Это возможно, потому что класс загрузчика классов не объявлен конечным и система фактически стимулирует вас к тому, чтобы писать собственные загрузчики, которые подходят для ваших конкретных потребностей.

Нестандартные загрузчики классов представляются как типы Java, так что их тоже должен загружать загрузчик, который обычно называется их *родительским загрузчиком классов*. Не путайте с наследованием классов и родительскими классами: загрузчики классов связываются разновидностью отношений *делегирования*.

На рис. 4.3 изображена иерархия делегирования загрузчиков классов и то, как разные загрузчики связаны друг с другом. В некоторых специальных случаях родителем нестандартного загрузчика классов может быть другой загрузчик, но обычно это загрузчик загружающего класса.

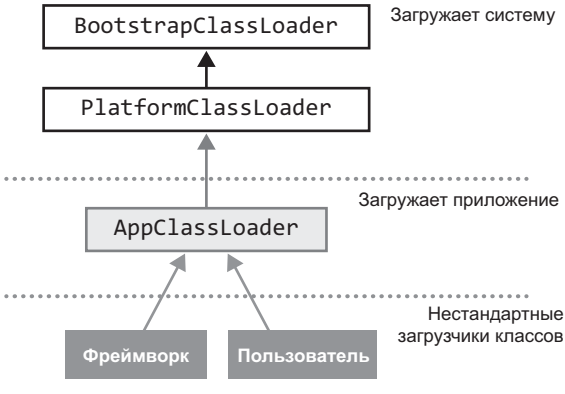


Рис. 4.3. Иерархия загрузчиков классов

Основные инструменты нестандартных загрузчиков классов — методы `loadClass()` и `findClass()`, определенные в `ClassLoader`. `loadClass()` предоставляет главную точку входа, и так выглядит упрощенная форма соответствующего кода:

```
protected Class<?> loadClass(String name, boolean resolve)
        throws ClassNotFoundException
    {
        synchronized (getClassLoadingLock(name)) {
            // Сначала проверить, был ли класс загружен ранее
            Class<?> c = findLoadedClass(name);
            if (c == null) {
                // ...
                try {
                    if (parent != null) {
```

https://t.me/javalib

```
                    c = parent.loadClass(name, false);
                } else {
                    c = findBootstrapClassOrNull(name);
                }
            } catch (ClassNotFoundException e) {
                // ClassNotFoundException порождается, если класс
                // не найден из родительского загрузчика классов,
                // отличного от null
            }
            if (c == null) {
                // Если найти так и не удалось, вызвать findClass
                // для поиска класса.
                // ...
                c = findClass(name);

                // ...
            }
        }
        // ...

        return c;
    }
}
```

По сути, механизм `loadClass()` проверяет, не был ли класс уже загружен, после чего запрашивает свой родительский загрузчик классов. Если эта загрузка класса не удается (обратите внимание на конструкцию `try-catch`, в которую заключен вызов `parent.loadClass(name, false)`), то процесс загрузки делегирует работу методу `findClass()`. Определение `findClass()` в `java.lang.ClassLoader` очень простое — оно всего лишь порождает `ClassNotFoundException`.

А сейчас вернемся к вопросу, поставленному в начале главы, и рассмотрим некоторые типы исключений и ошибок, с которыми можно столкнуться при загрузке классов.

Исключения при загрузке классов

Смысл исключения `ClassNotFoundException` относительно прост: загрузчик классов попытался загрузить указанный класс, но ему это не удалось. Иначе говоря, класс был неизвестен JVM в тот момент, когда была запрошена загрузка, и JVM не смогла его найти.

Далее в списке идет `NoClassDefFoundError`. Обратите внимание, что это *ошибка*, а не исключение. Она означает, что JVM знала о существовании запрошенного класса, но не нашла определение для него в своих внутренних метаданных. Рассмотрим короткий пример:

```
public class ExampleNoClassDef {

    public static class BadInit {
        private static int thisIsFine = 1 / 0;
    }
```

https://t.me/javalib

```
    public static void main(String[] args) {
        try {
            var init = new BadInit();
        } catch (Throwable t) {
            System.out.println(t);
        }
        var init2 = new BadInit();
        System.out.println(init2.thisIsFine);
    }
}
```

При выполнении получится такой результат:

```
$ java ExampleNoClassDef
java.lang.ExceptionInInitializerError
Exception in thread "main" java.lang.NoClassDefFoundError: Could
  not initialize class ExampleNoClassDef$BadInit[1]
    at ExampleNoClassDef.main(ExampleNoClassDef.java:13)
```

Как видите, JVM попробовала загрузить класс `BadInit`, но не смогла. Тем не менее программа перехватила исключение и попыталась продолжить работу. Но когда класс встретился во второй раз, внутренняя таблица метаданных JVM сигнализировала, что такой класс уже встречался, однако соответствующий действительный класс не был загружен.

При неудачной попытке загрузки класса JVM фактически реализует *негативное кэширование*: система не пытается повторно загрузить класс, а выдает ошибку `NoClassDefFoundError`.

Другая распространенная ошибка — `UnsupportedClassVersionError` — выдается при попытке загрузить файл класса, откомпилированный более высокой версией компилятора исходного кода Java, которая не поддерживается средой выполнения. Представьте, что у вас есть класс, откомпилированный в Java 11, который вы пытаетесь запустить в Java 8:

```
$ java ScratchImpl
Error: A JNI error has occurred please check your installation and try again[2]
Exception in thread "main" java.lang.UnsupportedClassVersionError:
  ScratchImpl has been compiled by a more recent version of the Java
    Runtime (class file version 55.0), this version of the Java Runtime
    only recognizes class file versions up to 52.0[3]
    at java.lang.ClassLoader.defineClass1(Native Method)
    at java.lang.ClassLoader.defineClass(ClassLoader.java:763)
```

[1] Исключение в потоке “main” java.lang.NoClassDefFoundError: Не удалось инициализировать класс ExampleNoClassDef$BadInit. — *Примеч. пер.*

[2] Ошибка: Произошла ошибка JNI. Пожалуйста, проверьте свою конфигурацию и попытайтесь снова. — *Примеч. пер.*

[3] Исключение в потоке “main” java.lang.UnsupportedClassVersionError: ScratchImpl был скомпилирован в более новой версии среды выполнения Java (версия файла класса 55.0), а текущая версия среды выполнения Java распознает только версии файла класса до 52.0 включительно. — *Примеч. пер.*

https://t.me/javalib

```
at java.security.SecureClassLoader.defineClass(SecureClassLoader.java:142)
at java.net.URLClassLoader.defineClass(URLClassLoader.java:468)
at java.net.URLClassLoader.access$100(URLClassLoader.java:74)
at java.net.URLClassLoader$1.run(URLClassLoader.java:369)
at java.net.URLClassLoader$1.run(URLClassLoader.java:363)
at java.security.AccessController.doPrivileged(Native Method)
at java.net.URLClassLoader.findClass(URLClassLoader.java:362)
at java.lang.ClassLoader.loadClass(ClassLoader.java:424)
at sun.misc.Launcher$AppClassLoader.loadClass(Launcher.java:349)
at java.lang.ClassLoader.loadClass(ClassLoader.java:357)
at sun.launcher.LauncherHelper.checkAndLoadMain(LauncherHelper.java:495)
```

Байт-код формата Java 11 может содержать возможности, которые не поддерживает более ранняя среда выполнения, поэтому продолжать пытаться его загружать небезопасно. Обратите внимание: поскольку это среда выполнения Java 8, она не поддерживает модульные записи в трассировке стека.

Наконец, стоит еще упомянуть ошибку `LinkageError`: это базовый класс иерархии, которая содержит `NoClassDefFoundError`, `VerifyError` и `UnsatisfiedLinkError`, а также ряд других ошибок.

Первый нестандартный загрузчик классов

Простейшая форма нестандартного загрузчика классов просто наследует `ClassLoader` и переопределяет `findClass()`. Это позволяет повторно использовать логику `loadClass()`, которую мы рассматривали ранее, и сократить сложность загрузчика классов.

Наш первый пример — загрузчик `SadClassLoader`, приведенный в следующем фрагменте кода. Он не делает ничего полезного, а только сообщает о том, что технически участвует в процессе, и желает вам всего наилучшего:

```
public class LoadSomeClasses {

    public static class SadClassloader extends ClassLoader {
        public SadClassloader() {
            super(SadClassloader.class.getClassLoader());
        }

        public Class<?> findClass(String name) throws
            ClassNotFoundException {
              System.out.println("Мне очень жаль, но боюсь,
                что мне не удается найти класс");
              throw new ClassNotFoundException(name);
        }
    }

    public static void main(String[] args) {
        if (args.length > 0) {
            var loader = new SadClassloader();
            for (var name : args) {
                System.out.println(name +" ::");
                try {
```

https://t.me/javalib

```
                var clazz = loader.loadClass(name);
                System.out.println(clazz);
            } catch (ClassNotFoundException x) {
                x.printStackTrace();
            }
        }
    }
}
```

В этом примере мы создали очень простой загрузчик классов и написали код, который с помощью этого загрузчика пытается загружать классы — вне зависимости от того, были ли они уже загружены ранее.

ПРИМЕЧАНИЕ По распространенному соглашению нестандартные загрузчики классов предоставляют конструктор без аргументов, который вызывает конструктор надкласса и передает загрузчик загружающего класса в аргументе (чтобы он стал родительским).

Многие нестандартные загрузчики классов устроены не намного сложнее того, который приведен в этом примере: они просто переопределяют `findClass()`, чтобы предоставить необходимую функциональность, например поиск класса по сети. Мне встречался случай, когда нестандартный загрузчик загружал классы, подключаясь к базе данных через JDBC и обращаясь к зашифрованному двоичному столбцу, чтобы получить нужные байты. Это позволяло удовлетворить требованиям к шифрованию хранимых данных для очень чувствительного кода в сильно регулируемой среде.

Впрочем, наши возможности не ограничиваются простым переопределением `findClass()`. Например, метод `loadClass()` — не конечный, и его тоже можно переопределять; собственно, некоторые нестандартные загрузчики переопределяют его именно для того, чтобы изменить общую логику, представленную ранее.

Наконец, есть еще метод `defineClass()`, определенный для `ClassLoader`. Он играет ключевую роль при загрузке классов, потому что это доступный для пользователя метод, который выполняет процедуру загрузки и компоновки, описанную ранее в этой главе. Он принимает массив байтов и преобразует их в объект класса. Это основной механизм, с помощью которого загружаются новые классы, не входящие в classpath.

Вызов `defineClass()` работает, только если передать ему буфер с байтами, имеющий правильный формат файла класса JVM. В противном случае он ничего не загрузит из-за сбоев на шаге загрузки или верификации.

ПРИМЕЧАНИЕ Этот метод также можно использовать для таких продвинутых возможностей, как загрузка классов, которые генерируются во время выполнения и не представлены в виде исходного кода. На этом принципе основан механизм лямбда-выражений в Java, о котором мы подробнее поговорим в главе 17.

https://t.me/javalib

Метод `defineClass()` объявлен защищенным и конечным и определен для `java.lang.ClassLoader`, так что обращаться к нему могут только подклассы `ClassLoader`. Таким образом, у нестандартных загрузчиков классов всегда есть доступ к базовой функциональности `defineClass()`, но они не могут вмешиваться в процедуру верификации или другую низкоуровневую логику загрузки классов. Последний пункт важен: невозможность изменить алгоритм вертификации — существенный фактор безопасности, ведь плохо написанный нестандартный загрузчик классов не сможет скомпрометировать защиту платформы, которую обеспечивает JVM.

В случае виртуальной машины HotSpot (это самая распространенная на сегодняшний день реализация JVM) `defineClass()` делегирует работу низкоуровневому методу `defineClass1()`, который выполняет ряд базовых проверок, а затем вызывает функцию, написанную на C, с именем `JVM_DefineClassWithSource()`.

Эта функция является точкой входа в JVM и предоставляет доступ к коду C реализации HotSpot. HotSpot использует структуру C `SystemDictionary`, чтобы загрузить новый класс при помощи метода C++ `ClassFileParser::parseClassFile()`. Этот код выполняет бóльшую часть компоновки, в частности, в него входит алгоритм верификации.

Когда загрузка класса завершена, байт-код методов помещается в объекты метаданных HotSpot, которые представляют методы (они называются *объектами methodOops*). После этого они становятся доступными для интерпретатора байт-кода. На концептуальном уровне это похоже на кэширование методов, хотя на самом деле байт-код хранится в объектах methodOops по соображениям быстродействия.

Загрузчик `SadClassloader` вам уже знаком. Рассмотрим пару других примеров нестандартных загрузчиков классов, начиная с того, как загрузку классов можно использовать, чтобы реализовать внедрение зависимостей.

Пример: фреймворк внедрения зависимостей

Стоит особо выделить два принципа, которые имеют первостепенное отношение к внедрению зависимостей (DI):

- У единиц функциональности в системе есть зависимости и сведения о конфигурации, на которые они полагаются, чтобы правильно функционировать.
- У многих объектных систем есть зависимости, которые трудно или неудобно выразить в коде.

Это можно представить себе так: классы содержат поведение и конфигурацию, а также зависимости, внешние по отношению к объектам. Последние обычно называют *коммутацией времени выполнения* (runtime wiring) объектов. В этом примере мы посмотрим, как гипотетический DI-фреймворк мог бы использовать загрузчики классов, чтобы реализовать коммутацию времени выполнения.

https://t.me/javalib

ПРИМЕЧАНИЕ Наш подход напоминает упрощенную версию исходной реализации фреймворка Spring. Однако современные DI-фреймворки устроены намного сложнее. Последующий пример приводится исключительно в демонстрационных целях.

Начнем с того, как будет запускаться приложение с нашим гипотетическим DI-фреймворком:

```
java -cp <CLASSPATH> org.wgjd.DIMain /path/to/config.xml
```

`DIMain` — это класс точки входа для DI-фреймворка. Он читает файл конфигурации, создает систему объектов и компонует («коммутирует») их друг с другом. Обратите внимание, что `DIMain` не является прикладным классом — он происходит из фреймворка и имеет предельно общую природу.

Мы также видим, что в пути `CLASSPATH` для приложения должны быть доступны: 1) файлы JAR DI-фреймворка; 2) классы приложения, которые упоминаются в файле config.xml; и 3) любые другие (не относящиеся к внедрению зависимостей) зависимости, которые есть у приложения.

Рассмотрим пример файла конфигурации:

```
<beans>
 <bean id="dao" class="app.ch04.PaymentsDAO">
  <constructor-arg index="0" value="jdbc:postgresql://db.wgjd.org/payments"/>
  <constructor-arg index="1" value="org.postgresql.Driver"/>
 </bean>

 <bean id="service" class="app.ch04.PaymentService">
  <constructor-arg index="0" ref="dao"/>
 </bean>

</beans>
```

DI-фреймворк использует файл конфигурации, чтобы определить, какие объекты конструировать. В нашем примере необходимы компоненты `dao` и `service`, и для каждого из них фреймворк должен вызывать конструкторы с заданными аргументами.

Классы загружаются за две фазы. На первой фазе (за которую отвечает загрузчик прикладных классов) загружается класс `DIMain` и любые классы фреймворка, к которым он обращается. Затем `DIMain` запускается и получает местонахождение конфигурационного файла в параметре `main()`.

В этот момент фреймворк уже выполняется в JVM, но пользовательские классы, указанные в config.xml, еще не обрабатывались. Собственно, пока `DIMain` не проанализирует конфигурационный файл, фреймворк никак не узнает, какие классы нужно загрузить.

Чтобы использовать конфигурацию приложения, указанную в config.xml, необходима вторая фаза загрузки класса. В нашем примере для этого служит нестандартный загрузчик.

https://t.me/javalib

Сначала нужно проверить файл config.xml на целостность и убедиться, что в нем нет ошибок. Затем, если проверка удалась, нестандартный загрузчик пытается загрузить типы из CLASSPATH. Если хотя бы один из них не загружается, то весь процесс завершается аварийно и происходит ошибка времени выполнения.

Если все хорошо, то DI-фреймворк начинает создавать экземпляры необходимых объектов в правильном порядке (с параметрами их конструкторов). Наконец, если этот этап завершается успешно, то контекст приложения настроен и готов к запуску.

Стоит еще раз повторить, что этот пример — чисто гипотетический и демонстрационный. Теоретически вполне возможно создать простой DI-фреймворк, который работает так, как здесь описано. Однако реальные системы внедрения зависимостей реализуются намного сложнее. Рассмотрим другой пример.

Пример: загрузчик диагностического класса

Рассмотрим загрузчик, который изменяет байт-код классов, добавляя дополнительную диагностическую информацию в процессе загрузки. Когда преобразованный код проходит тестовые сценарии, диагностический код сохраняет данные о том, какие методы и ветви кода фактически тестируются. По этим данным разработчик может определить, насколько полными были модульные тесты класса.

Этот подход лежал в основе EMMA — программы для определения тестового покрытия, которая все еще доступна по адресу http://emma.sourceforge.net/, хотя она заметно устарела и не обновлялась для современных версий Java. Впрочем, часто встречаются и другие фреймворки и утилиты со специализированными загрузчиками классов, которые преобразуют байт-код в процессе загрузки.

> **ПРИМЕЧАНИЕ** Преобразование байт-кода в процессе загрузки также характерно для механизма *агентов Java*, с помощью которого такие программы, как New Relic, анализируют быстродействие, наблюдаемость и другие метрики.

В этом разделе мы лишь кратко затронули пару сценариев того, как нестандартная загрузка классов может применяться на практике. Загрузчики классов и другие сопутствующие технологии активно используются во многих других областях технологического пространства Java. Вот несколько наиболее известных примеров:

- архитектуры плагинов;
- фреймворки (представленные на рынке или частные);
- загрузка файлов классов из нетипичных мест (не из файловых систем или URL);
- Java EE;
- любые ситуации, в которых нужно добавить новый, неизвестный код уже после того, как процесс JVM начал выполняться.

https://t.me/javalib

А теперь поговорим о том, как система модулей влияет на загрузку классов и что при этом меняется в только что описанной классической схеме.

4.2.2. Модули и загрузка классов

Система модулей создавалась так, чтобы работать не на том уровне, на котором происходит загрузка классов (относительно низкоуровневый механизм платформы). Модули относятся к крупномасштабным зависимостям между программными единицами, а загрузка классов действует в малом масштабе. Однако важно понимать, как пересекаются эти два механизма и что изменилось в процессе запуска программы, когда появились модули.

Напомним: чтобы выполнить программу на модульной JVM, среда выполнения должна построить граф модулей и в первую очередь попытаться удовлетворить его требования. Этот процесс называется *разрешением модулей* (module resolution), а его результатом становится транзитивное замыкание корневого модуля и его зависимостей.

Во время этого процесса выполняются дополнительные проверки, чтобы не было модулей с одинаковыми именами, расщепленных пакетов и т. д. Когда есть граф модулей, можно ожидать, что будет меньше проблем с загрузкой классов во время выполнения, потому что недостающие файлы JAR на пути к модулям теперь можно обнаружить еще до полноценного запуска процесса.

В остальном система модулей, как правило, не оказывает серьезного влияния на загрузку классов. Существуют некоторые продвинутые возможности (например, динамическая загрузка модульных реализаций интерфейсов провайдеров служб с помощью рефлексии), но большинству разработчиков вряд ли придется иметь с ними дело.

4.3. АНАЛИЗ ФАЙЛОВ КЛАССОВ

Файлы классов — это двоичные монолиты, и работать с ними напрямую довольно сложно. Однако существует много ситуаций, когда возникает необходимость исследовать файл класса.

Представьте, что в вашем приложении понадобилось сделать определенные методы открытыми, чтобы улучшить качество мониторинга на стадии выполнения (например, средствами JMX). Перекомпиляция и повторное развертывание вроде бы проходят нормально, но когда вы проверяете управляющий API, оказывается, что методов почему-то нет. Повторные пересборки и развертывания не помогают.

Чтобы отладить проблему развертывания, стоит проверить, создал ли `javac` тот файл класса, который должен был создать. А может, вы хотите проанализировать класс, для которого у вас нет исходного кода, когда подозреваете, что в документации ошибка.

В подобных ситуациях приходится пользоваться специальными программами, которые анализируют содержимое файлов классов. К счастью, в стандартную

https://t.me/javalib

поставку Oracle JVM входит утилита `javap`, с помощью которой удобно заглядывать внутрь файлов классов и дизассемблировать их.

Далее мы опишем `javap` и основные ключи, с помощью которых в ней можно управлять анализом тех или иных аспектов файлов классов. Затем пойдет речь о некоторых представлениях имен методов и типов, которые JVM использует на внутреннем уровне. После этого мы рассмотрим пул констант — «сундук со сказками» JVM, знакомство с которым поможет понять, как работает байт-код.

4.3.1. Введение в javap

Утилита `javap` выполняет множество полезных задач — от просмотра списка методов, которые объявляет класс, до вывода байт-кода. Рассмотрим простейшую форму использования `javap` в контексте примера с загрузкой классов, приведенного ранее в этой главе:

```
$ javap LoadSomeClasses.class
Compiled from "LoadSomeClasses.java"
public class LoadSomeClasses {
  public LoadSomeClasses();
  public static void main(java.lang.String[]);
}
```

Внутренний класс был откомпилирован в отдельный класс, который мы тоже рассмотрим:

```
$ javap LoadSomeClasses\$SadClassloader.class
Compiled from "LoadSomeClasses.java"
public class LoadSomeClasses$SadClassloader extends java.lang.ClassLoader {
  public LoadSomeClasses$SadClassloader();
  public java.lang.Class<?> findClass(java.lang.String) throws
    java.lang.ClassNotFoundException;
}
```

По умолчанию `javap` выводит методы с открытым, защищенным и используемым по умолчанию (доступным в границах пакета) уровнем доступа. С ключом `-p` также выводятся частные методы и поля.

4.3.2. Внутренняя форма сигнатур методов

Во внутренней реализации JVM сигнатуры методов представлены не совсем в такой же форме, которая выводится `javap` и понятна человеку. Чем глубже вы погружаетесь в JVM, тем чаще встречаются эти внутренние имена. Если вам не терпится двигаться дальше, этот раздел можно пропустить, но не забывайте о нем: возможно, вам еще придется сюда вернуться из более поздних разделов и глав.

В компактной форме имена типов сокращаются: например, `int` представляется как `I`. Эти компактные формы иногда называются *дескрипторами типов*, и их полный список приведен в табл. 4.1 (в него входит обозначение `void`, которое формально не является типом, но часто встречается в сигнатурах методов).

https://t.me/javalib

Таблица 4.1. Дескрипторы типов

Дескриптор	Тип
B	byte
C	char (16-разрядный символ Юникода)
D	double
F	float
I	int
J	long
L<имя типа>;	Ссылочный тип (например, Ljava/lang/String; для строки)
S	short
V	void
Z	boolean
[	Массив

В некоторых случаях дескриптор типа оказывается длиннее, чем имя типа в исходном коде: например, `Ljava/lang/Object;` длиннее, чем `Object`, но дескрипторы типов всегда записываются в полностью уточненном виде, чтобы их можно было разрешать напрямую.

`javap` предоставляет полезный ключ `-s`, который выводит дескрипторы типов в сигнатурах, чтобы вы не искали их по таблице. Например, можно вызвать `javap` в расширенной форме, чтобы вывести сигнатуры методов, рассматривавшихся ранее:

```
$ javap -s LoadSomeClasses.class
Compiled from "LoadSomeClasses.java"
public class LoadSomeClasses {
  public LoadSomeClasses();
    descriptor: ()V

  public static void main(java.lang.String[]);
    descriptor: ([Ljava/lang/String;)V
}
```

А вот сигнатуры для внутреннего класса:

```
$ javap -s LoadSomeClasses\$SadClassloader.class
Compiled from "LoadSomeClasses.java"
public class LoadSomeClasses$SadClassloader extends java.lang.ClassLoader {
  public LoadSomeClasses$SadClassloader();
    descriptor: ()V

  public java.lang.Class<?> findClass(java.lang.String) throws
```

https://t.me/javalib

```
    java.lang.ClassNotFoundException;
    descriptor: (Ljava/lang/String;)Ljava/lang/Class;
}
```

Как видите, каждый тип в сигнатуре метода представляется дескриптором типа.

В следующем разделе мы рассмотрим другое применение дескрипторов типов. Это очень важная часть файла класса — пул констант.

4.3.3. Пул констант

Пул констант — это область с удобными сокращениями для других (константных) элементов файла класса. Если вы изучали такие языки, как C или Perl, в которых явно используются таблицы символических имен, то пул констант можно рассматривать как нечто похожее в JVM.

В следующем листинге мы продемонстрируем пул констант на очень простом примере, чтобы не углубляться в подробности. Это своего рода «тренажер», который позволяет быстро протестировать средства синтаксиса Java или библиотек, включив небольшой объем кода в `run()`.

Листинг 4.1. Пример класса-«тренажера»

```
package wgjd.ch04;

public class ScratchImpl {
    private static ScratchImpl inst = null;

    private ScratchImpl() {

    }

    private void run() {

    }

    public static void main(String[] args) {
        inst = new ScratchImpl();
        inst.run();
    }
}
```

Чтобы просмотреть информацию в пуле констант, используется команда `javap -v`. Она выводит не только пул констант, но и много дополнительных сведений, однако пока мы ограничимся пулом констант для нашего класса:

```
#1 = Class #2 // wgjd/ch04/ScratchImpl

#2 = Utf8 wgjd/ch04/ScratchImpl

#3 = Class #4 // java/lang/Object

#4 = Utf8 java/lang/Object
```

https://t.me/javalib

```
#5 = Utf8 inst

#6 = Utf8 Lwgjd/ch04/ScratchImpl;

#7 = Utf8 <clinit>

#8 = Utf8 ()V

#9 = Utf8 Code

#10 = Fieldref #1.#11 // wgjd/ch04/ScratchImpl.inst:Lwgjd/ch04/ScratchImpl;

#11 = NameAndType #5:#6 // instance:Lwgjd/ch04/ScratchImpl;

#12 = Utf8 LineNumberTable

#13 = Utf8 LocalVariableTable

#14 = Utf8 <init>

#15 = Methodref #3.#16 // java/lang/Object."<init>":()V

#16 = NameAndType #14:#8 // "<init>":()V

#17 = Utf8 this

#18 = Utf8 run

#19 = Utf8 ([Ljava/lang/String;)V

#20 = Methodref #1.#21 // wgjd/ch04/ScratchImpl.run:()V

#21 = NameAndType #18:#8 // run:()V

#22 = Utf8 args

#23 = Utf8 [Ljava/lang/String;

#24 = Utf8 main

#25 = Methodref #1.#16 // wgjd/ch04/ScratchImpl."<init>":()V

#26 = Methodref #1.#27 // wgjd/ch04/ScratchImpl.run:([Ljava/lang/String;)V

#27 = NameAndType #18:#19 // run:([Ljava/lang/String;)V

#28 = Utf8 SourceFile

#29 = Utf8 ScratchImpl.java
```

Как видите, элементы пула констант типизированы. Они также ссылаются друг на друга: например, элемент типа `Class` ссылается на элемент типа `Utf8`. Элементы `Utf8` обозначают строки, так что элемент `Utf8`, на который ссылается элемент `Class`, — это имя класса.

https://t.me/javalib

В табл. 4.2 перечислены возможные типы элементов пула констант. Эти типы иногда сопровождаются префиксом `CONSTANT_`, например `CONSTANT_Class`. Это помогает подчеркнуть, что они не являются типами Java (в ситуациях, где возможна путаница).

Таблица 4.2. Типы элементов пула констант

Тип	Описание
Class	Константа-класс. Указывает на имя класса (в виде элемента Utf8)
Fieldref	Определяет поле. Указывает на элементы Class и NameAndType этого поля
Methodref	Определяет метод. Указывает на элементы Class и NameAndType этого метода
InterfaceMethodref	Определяет метод интерфейса. Указывает на элементы Class и NameAndType этого метода
String	Строковая константа. Указывает на элемент Utf8, содержащий символы
Integer	Целочисленная константа (4 байта)
Float	Константа с плавающей точкой (4 байта)
Long	Константа с плавающей точкой (8 байт)
Double	Константа с плавающей точкой двойной точности (8 байт)
NameAndType	Описывает пару из имени и типа. Тип указывает на элемент Utf8, содержащий дескриптор типа
Utf8	Поток байтов, представляющих символы в кодировке Utf8
InvokeDynamic MethodHandle MethodType	Константы, связанные с механизмом invokedynamic, — см. главу 17

Пользуясь этой таблицей, можно рассмотреть пример того, как разрешаются константы из пула констант нашего «тренажера». Возьмем элемент `#10` — `FieldRef`. Чтобы разрешить поле, нужно знать его имя, тип и класс, в котором оно находится: `#10` содержит значение `#1.#11`, то есть константу `#11` из класса `#1`. Нетрудно убедиться в том, что `#1` — это действительно константа типа `Class`, а `#11` — константа типа `NameAndType`. `#1` обозначает сам класс Java `ScratchImpl`, а `#11` ссылается на `#5:#6` — переменную с именем `inst` типа `ScratchImpl`. Таким образом, в итоге `#10` обозначает статическую переменную `inst` в само́м классе `ScratchImpl` (как вы, вероятно, уже поняли по приведенному выше выводу).

В фазе верификации процесса загрузки классов, в частности, проверяется целостность статической информации в файле класса. Приведенный пример показывает, какую проверку целостности осуществляет среда выполнения, когда загружает новый класс.

https://t.me/javalib

В этом разделе мы рассмотрели простейшую анатомию файла класса. Давайте перейдем к следующей теме, которая погрузит нас в мир байт-кода. Чем лучше вы будете разбираться в том, как исходный код преобразуется в байт-код, тем лучше будете понимать, как ваша программа будет выполняться. В свою очередь, это даст более полное представление о функциональности платформы, когда мы доберемся до главы 6 и последующих глав.

4.4. БАЙТ-КОД

До сих пор байт-код в каком-то смысле оставался в тени нашего обсуждения. Начнем с краткой сводки того, что вы уже о нем знаете:

- Байт-код — это промежуточное представление программы на полдороге между исходным кодом, понятным человеку, и машинным кодом.
- Байт-код генерируется `javac` из файлов с исходным кодом на Java.
- Во время компиляции некоторые высокоуровневые конструкции языка исключаются и не присутствуют в байт-коде. Например, циклические конструкции Java (`for`, `while` и т. д.) заменяются в байт-коде на инструкции перехода.
- Каждый код операции представляется одним байтом (отсюда и название «байт-код»).
- Байт-код — это абстрактное представление, а не «машинный язык для воображаемого процессора».
- Байт-код может компилироваться в машинный код, обычно в режиме JIT (Just-In-Time).

При объяснении байт-кода возникает классическая проблема «курица или яйцо». Чтобы в полной мере понять, что происходит, необходимо ориентироваться как в байт-коде, так и в среде, где он выполняется. Получается порочный круг, и чтобы преодолеть его, мы начнем с относительно простого практического примера. Даже если вы с первой попытки не поймете всего, что происходит в этом примере, вы сможете вернуться к нему после того, как узнаете больше о байт-коде в следующих разделах.

После примера мы немного поговорим о том, как устроена среда выполнения, а затем займемся классификацией кодов операций JVM, включая байт-коды арифметических операций, вызовов, сокращенных форм и т. д. В заключение мы рассмотрим другой пример, посвященный конкатенации строк. Но прежде всего давайте посмотрим, как анализировать байт-код из файла .class.

4.4.1. Дизассемблирование классов

Чтобы дизассемблировать классы, `javap` запускается с ключом `-c`. В нашем примере будет использоваться уже знакомый класс `ScratchImpl`. В первую очередь мы проанализируем байт-код, который относится к методам. Также мы применим ключ `-p`, чтобы увидеть байт-код из частных методов.

https://t.me/javalib

Будем анализировать вывод `javap` по частям: каждый фрагмент вывода содержит большой объем информации, и все сразу будет нелегко усвоить. Сначала идет заголовок. Здесь нет ничего особенно неожиданного или интересного:

```
$ javap -c -p wgjd/ch04/ScratchImpl.class

Compiled from "ScratchImpl.java"

public class wgjd.ch04.ScratchImpl extends java.lang.Object {
  private static wgjd.ch04.ScratchImpl inst;
```

Затем следует статический блок. Здесь размещается инициализация переменных, так что следующий код представляет, как `inst` инициализируется значением `null`. Внимательный читатель догадается, что `putstatic` — это байт-код, который сохраняет значение в статическом поле:

```
static {};

Code:
  0: aconst_null
  1: putstatic #10 // Field inst:Lwgjd/ch04/ScratchImpl;
  4: return
```

Числа в этом коде обозначают смещение в потоке байт-кода от начала метода. Таким образом, байт 1 содержит код операции `putstatic`, а байты 2 и 3 представляют 16-разрядный индекс в пуле констант. В данном случае 16-разрядный индекс — это значение 10; иными словами, значение (в данном случае `null`) будет сохранено в поле, которое помечено элементом пула констант `#10`. Байт 4 от начала потока байт-кода содержит код операции `return` — это конец блока кода.

Затем идет конструктор:

```
private wgjd.ch04.ScratchImpl();

Code:
  0: aload_0
  1: invokespecial #15 // Method java/lang/Object."<init>":()V
  4: return
```

Не забывайте, что в Java конструктор `void` всегда неявно вызывает конструктор надкласса. Здесь этот факт отражен в байт-коде — это инструкция `invokespecial`. В общем случае любой вызов метода преобразуется в одну из пяти существующих в JVM инструкций `invoke`, о которых мы поговорим в разделе 4.4.7.

При вызове конструктора необходимо указать *цель*, которую предоставляет инструкция `aload_0`. Она загружает ссылку (*адрес*) и использует сокращенную форму (которая будет представлена в разделе 4.4.9), чтобы загрузить нулевую локальную переменную, которой является `this`, то есть текущий объект.

В методе `run()` фактически нет кода, потому что это всего лишь «тренажер» для тестирования кода. Метод немедленно возвращает управление вызывающей стороне и не возвращает значения (что правильно, потому что метод возвращает `void`):

https://t.me/javalib

```
private void run();

Code:
  0: return
```

В методе `main` инициализируется `inst` и создаются некоторые объекты. Здесь используются очень распространенные схемы байт-кода, которые вы вскоре научитесь распознавать:

```
public static void main(java.lang.String[]);

Code:
  0: new #1 // class wgjd/ch04/ScratchImpl
  3: dup
  4: invokespecial #21 // Method "<init>":()V
```

Такая последовательность из трех инструкций байт-кода — `new`, `dup` и `invokespecial` для метода с именем `<init>` — всегда представляет создание нового экземпляра.

Код операции `new` выделяет память для нового экземпляра и размещает ссылку на него на вершине стека. Код операции `dup` дублирует ссылку, которая находится на вершине стека (так что теперь в стеке хранятся две копии). Чтобы завершить создание объекта, необходимо вызвать тело конструктора. Метод `<init>` содержит код тела конструктора, и этот блок кода вызывается операцией `invokespecial`.

Когда вызываются методы, ссылка на объект-получатель (если она есть) извлекается из стека вместе с любыми аргументами метода. Вот почему сначала необходимо выполнить `dup`: без него единственная ссылка на только что созданный объект была бы извлечена инструкцией `invoke` и после этого объект стал недоступным.

Рассмотрим оставшийся байт-код метода `main`:

```
 7: putstatic #10 // Field inst:Lwgjd/ch04/ScratchImpl;
10: getstatic #10 // Field inst:Lwgjd/ch04/ScratchImpl;
13: invokevirtual #22 // Method run:()V
16: return
```

Инструкция 7 сохраняет адрес единственного созданного экземпляра. Инструкция 10 помещает его на вершину стека, чтобы инструкция 13 могла вызвать его метод. За это отвечает код операции `invokevirtual`, который занимается «стандартной» диспетчеризацией методов экземпляров в Java.

> **ПРИМЕЧАНИЕ** В общем случае байт-код, который порождает javac, является простым представлением и не отличается высокой оптимизацией. Общая стратегия заключается в том, что бóльшая часть оптимизации ложится на плечи JIT-компиляторов, поэтому им желательно предоставить относительно простой и незамысловатый исходный материал. Выражение «байт-код должен быть тупым» хорошо описывает, как создатели реализации JVM относятся к байт-коду, который производится из исходных языков.

https://t.me/javalib

Код операции `invokevirtual`, в частности, проверяет, нет ли переопределений метода в иерархии наследования объекта. Вам это может показаться странным, потому что частные методы не могут переопределяться. Можно предположить, что для частных методов компилятор исходного кода может породить операцию `invokespecial` вместо `invokevirtual`. Собственно, раньше так и происходило, и только в последних версиях Java ситуация изменилась. За подробностями обращайтесь к главе 17.

А теперь займемся средой выполнения, которая необходима байт-коду. После этого мы представим таблицы, с помощью которых будем описывать основные категории инструкций байт-кода: загрузка/сохранение, арифметические действия, управление выполнением, вызов методов и платформенные операции. Затем мы рассмотрим возможные сокращенные формы кодов операций, прежде чем перейти к следующему примеру.

4.4.2. Среда выполнения

Чтобы ориентироваться в байт-коде, крайне важно понимать принципы работы стековой машины, которую использует JVM. Одно из самых очевидных отличий JVM от физического процессора (такого, как x64 или ARM) заключается в том, что у JVM нет регистров процессора; вместо них для всех вычислений и операций используется стек, который называется *стеком вычислений*. (В спецификации JVM официально принят термин «*стек операндов*»; мы будем использовать эти два термина как синонимы.)

Стек вычислений локален по отношению к методу, и при вызове метода создается новый стек вычислений. Конечно, у JVM также есть *стек вызовов* для каждого потока Java, в котором сохраняется информация о том, какие методы были вызваны, и который образует основу трассировки стека в Java. Важно четко различать стек вызовов, который существует на уровне потока, и стек вычислений — свой для каждого метода.

На рис. 4.4 показано, как с помощью стека вычислений можно выполнить операцию сложения двух целочисленных констант. Под каждым шагом приводится эквивалентный байт-код Java: этот код еще встретится позже в этой главе, так что не беспокойтесь, если сейчас что-то покажется непонятным.

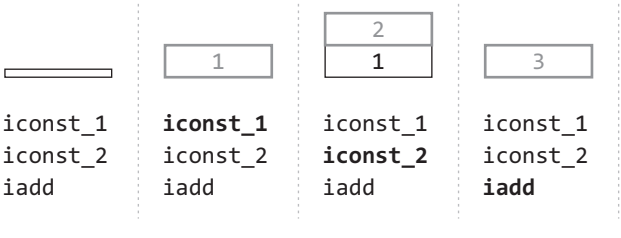


Рис. 4.4. Использование стека для арифметических операций

https://t.me/javalib

Как упоминалось ранее в этой главе, когда класс компонуется в среде выполнения, его байт-код проверяется, и значительная часть этой проверки сводится к анализу типов в стеке.

> **ПРИМЕЧАНИЕ** Манипуляции со значениями в стеке работают только в том случае, если у этих значений правильные типы. Если вы, например, попытаетесь занести в стек ссылку на объект, а потом выполнить с ней арифметические операции, как с int, может произойти что-нибудь нехорошее.

На фазе проверки при загрузке классов выполняются обширные проверки, чтобы убедиться, что методы загруженных классов не пытаются злоупотреблять операциями со стеком. По итогам этих проверок система отвергает некорректно сформированные (или намеренно вредоносные) классы, чтобы они не создали проблем.

Когда метод выполняется, ему нужна область памяти, которая будет использоваться как стек вычислений. Кроме того, каждому выполняемому потоку нужен стек вызовов, в котором сохраняется информация о том, какие методы еще не вернули управление (содержимое этого стека выводится при трассировке). В некоторых случаях эти два стека взаимодействуют друг с другом. Рассмотрим следующий фрагмент кода:

```
var numPets = 3 + petRecords.getNumberOfPets("Вася");
```

Чтобы вычислить результат, JVM помещает 3 в стек операндов. Затем она должна вызвать метод, который вычисляет, сколько домашних питомцев есть у Васи. Для этого объект-получатель (тот, для которого вызывается метод, — в данном случае `petRecords`) помещается в стек выполнения, а за ним следуют аргументы вызова.

Затем метод `getNumberOfPets()` вызывается при помощи одного из кодов операций `invoke`, вследствие чего управление передается вызванному методу, и этот метод заносится в стек вызовов. Однако поскольку JVM вошла в новый метод, она начинает использовать новый стек операндов, так что значения, которые уже находятся в стеке операндов вызывающей стороны, не повлияют на результаты, вычисленные в вызванном методе.

Когда `getNumberOfPets()` завершается, возвращаемое значение помещается в стек операндов вызывающей стороны, и при этом `getNumberOfPets()` удаляется из стека вызовов. Затем операция сложения берет два значения и суммирует их.

Перейдем к анализу байт-кода. Это большая тема, полная особых случаев, поэтому мы приведем скорее сводку основных возможностей, чем подробное описание.

4.4.3. Введение в коды операций

Байт-код JVM состоит из последовательности *кодов операций* (opcodes), при этом за каждой инструкцией могут следовать аргументы. Каждый код операции рассчитан на то, что стек находится в определенном состоянии, которое нужно

https://t.me/javalib

для этого кода. Он преобразует стек, удаляя из него аргументы и помещая в него результаты.

Каждый код операции обозначается однобайтовым значением, так что всего может существовать не более 255 возможных кодов операций, причем в настоящее время используется лишь около 200. Их слишком много, чтобы перечислять в книге, хотя полный список можно найти по адресу http://mng.bz/aJaX. К счастью, большинство кодов операций относится к какому-либо из базовых семейств, каждое из которых объединяет коды со схожей функциональностью. Мы обсудим все семейства по очереди, чтобы вы получили общее представление о них. Некоторые операции невозможно однозначно отнести к тому или иному семейству, но они обычно встречаются реже.

ПРИМЕЧАНИЕ JVM не является чисто объектно-ориентированной средой выполнения. Она поддерживает примитивные типы, что проявляется в некоторых семействах кодов операций: некоторым базовым типам кодов (таким, как store и add) приходится существовать в нескольких различающихся вариациях в зависимости от того, с каким примитивным типом они работают.

Таблицы кодов операций состоят из четырех столбцов:

- *Название* — общее название для типа кода операции. Во многих случаях коды операций с похожими названиями решают сходные задачи.
- *Аргументы* — аргументы, принимаемые кодами операций. Если аргументы начинаются с `i`, это беззнаковые байты, которые образуют индекс для поиска в пуле констант или в таблице локальных переменных.

  ПРИМЕЧАНИЕ Чтобы получить более длинные индексы, байты соединяются друг с другом, так что i1, i2 означает «создать из этих двух байтов 16-разрядный индекс», что делается с помощью поразрядного сдвига и сложения: ((i1 << 8) + i2).

- Если аргумент заключен в квадратные скобки, это значит, что он используется не во всех формах кода операции.
- *Структура стека* — состояние стека до и после выполнения операции. Элементы в квадратных скобках означают, что они используются не во всех формах кода операции или что эти элементы необязательны (как для кодов операций вызова).
- *Описание* — что делает операция.

Для примера рассмотрим строку из табл. 4.3 для кода операции `getfield`. Эта операция считывает значение из поля объекта.

Название	Аргументы	Структура стека	Описание
getfield	i1, i2	[obj] -> [val]	Получает поле, индекс которого в пуле констант задан объектом на вершине стека

https://t.me/javalib

В первом столбце приводится название кода операции — `getfield`. Следующий столбец сообщает, что за кодом операции в потоке байт-кодов следуют два аргумента. Эти аргументы соединяются, образуя 16-разрядный индекс, а затем он ищется в пуле констант, чтобы получить доступ к полю, которое соответствует этому индексу (помните, что индексы в пуле констант всегда являются 16-разрядными). Третий столбец («Структура стека») показывает, что ссылка на объект заменяется значением поля.

В ходе операции удаляются экземпляры объектов: это полезно для того, чтобы байт-код оставался компактным и чтобы вам не приходилось заниматься однообразной зачисткой и следить за тем, когда нужно удалять отработанные экземпляры.

4.4.4. Операции загрузки и сохранения

Коды операций из семейства загрузки и сохранения загружают значения в стек или извлекают их оттуда. В табл. 4.3 перечислены основные операции этого семейства.

Таблица 4.3. Коды операций загрузки и сохранения

Название	Аргументы	Структура стека	Описание
load	(i1)	[] -> [val]	Загружает значение (примитивное или ссылочное) из локальной переменной в стек. Имеет сокращенные формы и разновидности для конкретных типов
ldc	i1	[] -> [val]	Загружает константу из пула в стек. Имеет разновидности для конкретных типов и широкие формы
store	(i1)	[val] -> []	Сохраняет значение (примитивное или ссылочное) в локальной переменной, удаляя его из стека. Имеет сокращенные формы и разновидности для конкретных типов
dup		[val] -> [val, val]	Дублирует значение на вершине стека. Имеет несколько разных форм
getfield	i1, i2	[obj] -> [val]	Получает поле, индекс которого в пуле констант задан объектом на вершине стека
putfield	i1, i2	[obj, val] -> []	Помещает значение в поле объекта с заданным индексом в пуле констант
getstatic	i1, i2	[] -> [val]	Получает значение статического поля с заданным индексом в пуле констант
putstatic	i1, i2	[val] -> []	Помещает значение в статическое поле с заданным индексом в пуле констант

https://t.me/javalib

Как упоминалось ранее, инструкции загрузки и сохранения существуют в разных формах. Например, код операции `dload` загружает значение типа `double` в стек из локальной переменной, а код операции `astore` извлекает ссылку на объект из стека в локальную переменную.

Рассмотрим небольшой пример использования `getfield` и `putfield`. Для следующего простого класса

```
public class Scratch {
    private int i;

    public Scratch() {
      i = 0;
    }

    public int getI() {
      return i;
    }

    public void setI(int i) {
      this.i = i;
    }
}
```

методы чтения и записи декомпилируются в таком виде:

```
public int getI();
    Code:
       0: aload_0
       1: getfield #7     // Field i:I
       4: ireturn

 public void setI(int);
    Code:
       0: aload_0
       1: iload_1
       2: putfield #7     // Field i:I
       5: return
```

Этот пример показывает, как временные переменные сохраняются в стеке перед тем, как будут переданы в кучу.

4.4.5. Арифметические операции

Эти коды операций выполняют арифметические операции со стеком. Они получают аргументы с вершины стека и производят с ними необходимые вычисления. Типы аргументов (которые всегда являются примитивными типами) всегда должны точно совпадать, хотя платформа предоставляет богатый набор кодов операций для приведения одного примитивного типа к другому. В табл. 4.4 перечислены базовые арифметические операции.

У операций приведения очень короткие имена: например, `i2d` для приведения `int` к `double`. В частности, слово `cast` в именах не встречается, поэтому в таблице оно заключено в круглые скобки.

https://t.me/javalib

Таблица 4.4. Коды арифметических операций

Название	Аргументы	Структура стека	Описание	Примечание
add		[val1, val2] -> [res]	Сложение	Выполняет указанную операцию для двух значений, которые находятся на вершине стека. Оба значения должны относиться к одному примитивному типу. Помещает результат в стек. Имеет сокращенные формы и разновидности для конкретных типов
sub			Вычитание	
div			Деление	
mul			Умножение	
(cast)		[value] -> [res]	Приводит значение одного примитивного типа к другому. Имеет формы, соответствующие каждому возможному приведению	

4.4.6. Операции управления порядком выполнения

Как упоминалось ранее, в байт-коде JVM нет управляющих конструкций языков высокого уровня. Вместо этого порядок выполнения управляется немногочисленными примитивами, которые перечислены в табл. 4.5.

Таблица 4.5. Коды операций управления порядком выполнения

Название	Аргументы	Структура стека	Описание
if	b1, b2	[val1, val2] -> [] или [val1] -> []	Если условие выполняется, переходит по заданному смещению ветви
goto	b1, b2	[] -> []	Безусловный переход по заданному смещению. Имеет широкую форму
tableswitch	{варьируется}	[index] -> []	Используется для реализации switch
lookupswitch	{варьируется}	[key]	Используется для реализации switch

Как и байты индекса, которые используются для поиска констант, аргументы `b1`, `b2` объединяются, чтобы получить позицию в байт-коде в пределах метода, по которой передается управление. Аргументы не могут передавать управление за пределы метода — это условие проверяется во время загрузки класса, и если оно нарушается, то класс не пройдет верификацию.

Семейство кодов операций `if` больше, чем можно ожидать: оно включает более 15 инструкций для различных операндов (например, сравнение чисел, равенство ссылок и т. д.).

https://t.me/javalib

ПРИМЕЧАНИЕ В семейство кодов операций `if` также входят две устаревшие инструкции, `jsr` и `ret`, которые `javac` больше не генерирует и которые недопустимы в современных версиях Java.

Широкая форма инструкции `goto` (`goto_w`) принимает 4 байта аргументов и строит смещение, которое может превышать 64 Кбайт. Такая необходимость возникает нечасто, потому что она актуальна только для очень больших методов (а у них есть и другие проблемы: например, огромный размер не позволяет выполнить JIT-компиляцию). Аналогично, существует инструкция `ldc_w`, которую можно использовать для адресации очень больших пулов констант.

4.4.7. Операции вызова

Семейство операций вызова состоит из четырех кодов операций, которые обеспечивают обычный вызов методов, а также особой операции `invokedynamic`, которая была добавлена в Java 7 и будет подробнее рассмотрена в главе 17. Пять кодов операций вызова перечислены в табл. 4.6.

Таблица 4.6. Коды операций вызова

Название	Аргументы	Структура стека	Описание
invokestatic	i1, i2	[(val1, …)] -> []	Вызывает статический метод
invokevirtual	i1, i2	[obj, (val1, …)] -> []	Вызывает «обычный» метод экземпляра
invokeinterface	i1, i2, count, 0	[obj, (val1, …)] -> []	Вызывает метод интерфейса
invokespecial	i1, i2	[obj, (val1, …)] -> []	Вызывает «специальный» метод экземпляра, например конструктор
invokedynamic	i1, i2, 0, 0	[val1, …] -> []	Динамический вызов; см. главу 17

Различия между этими кодами операций проще всего понять на расширенном примере:

```
long time = System.currentTimeMillis();

// Эта явная типизация применяется намеренно, читайте дальше
HashMap<String, String> hm = new HashMap<>();
hm.put("now", "bar");

Map<String, String>m=hm;
m.put("foo", "baz");
```

А теперь просмотрим сгенерированный байт-код с помощью команды `javap -c`:

```
Code:
     0: invokestatic  #2 // Method java/lang/System.currentTimeMillis:()J
     3: lstore_1
```

https://t.me/javalib

```
 4: new           #3 // class java/util/HashMap
 7: dup
 8: invokespecial #4 // Method java/util/HashMap."<init>":()V
11: astore_3
12: aload_3
13: ldc           #5 // String now
15: ldc           #6 // String bar
17: invokevirtual #7 // Method java/util/HashMap.put:(
                     //Ljava/lang/Object;Ljava/lang/Object;)
                     //Ljava/lang/Object;
20: pop
21: aload_3
22: astore        4
24: aload         4
26: ldc           #8 // String foo
28: ldc           #9 // String baz
30: invokeinterface #10, 3 // InterfaceMethod java/util/Map.put:(
                           //Ljava/lang/Object;Ljava/lang/Object;)
                           //Ljava/lang/Object;
35: pop
```

Как упоминалось ранее, вызовы методов Java преобразуются в один из нескольких возможных байт-кодов `invoke*`. Присмотримся повнимательнее:

```
 0: invokestatic #2 // Method java/lang/System.currentTimeMillis:()J
 3: lstore_1
```

Статический вызов `System.currentTimeMillis()` преобразуется в операцию `invokestatic`, которая находится в позиции 0 байт-кода. Метод не получает параметров, так что перед тем, как методу передается управление, в стек выполнения не нужно ничего загружать.

Затем в потоке байтов следуют два байта, `00` и `02`. Они объединяются в 16-разрядное число, которое используется как смещение в пуле констант. Декомпилятор любезно добавляет комментарий, где написано, какому методу соответствует смещение `#2`. В данном случае, как и ожидалось, это метод `System.currentTimeMillis()`.

Когда управление возвращается, результат вызова помещается в стек, а по смещению 3 располагается одиночный код операции без аргументов `lstore_1`, который сохраняет это возвращаемое значение в локальную переменную `1`.

Конечно, человеку видно, что переменная `time` не используется нигде в программе. Однако одна из целей проектирования `javac` заключалась в том, чтобы байт-код по возможности точно представлял содержимое исходного кода Java вне зависимости от того, осмысленный он или нет. Поэтому возвращаемое значение `System.currentTimeMillis()` сохраняется, хотя и не используется нигде в последующей программе.

Этот пример демонстрирует «тупой байт-код» в действии: помните, что с точки зрения платформы формат файла класса является входным форматом для «настоящего» компилятора — JIT-компилятора:

https://t.me/javalib

```
 4: new           #3 // class java/util/HashMap
 7: dup
 8: invokespecial #4 // Method java/util/HashMap."<init>":()V
11: astore_3
12: aload_3
13: ldc           #5 // String now
15: ldc           #6 // String bar
17: invokevirtual #7 // Method java/util/HashMap.put:(
                     //Ljava/lang/Object;Ljava/lang/Object;)
                     //Ljava/lang/Object;
20: pop
```

Байт-коды с 4-го по 10-й создают новый экземпляр `HashMap`, после чего инструкция 11 сохраняет его копию в локальную переменную. Затем инструкции с 12-й по 16-ю заполняют стек объектом `HashMap` и аргументами для вызова `put()`. Фактический вызов метода `put()` выполняется инструкциями с 17-й по 19-ю.

На этот раз используется код операции `invokevirtual`, потому что локальная переменная была объявлена со статическим типом `HashMap` — типом класса. Вскоре вы увидите, что произойдет, если локальная переменная объявляется с типом `Map`.

Вызов метода экземпляра отличается от вызова статического метода, потому что у статического вызова нет экземпляра, для которого вызывается этот метод (иногда называемого *объектом-получателем*).

> **ПРИМЕЧАНИЕ** В байт-коде вызов метода экземпляра необходимо подготовить: поместить получатель и все аргументы вызова в стек вычислений, а затем выполнить инструкцию invoke.

В данном случае значение, которое возвращает `put()`, не используется, так что инструкция 20 отбрасывает его, как показано ниже:

```
21: aload_3
22: astore        4
24: aload         4
26: ldc           #8 // String foo
28: ldc           #9 // String baz
30: invokeinterface #10, 3 //InterfaceMethod java/util/Map.put:(
                           //Ljava/lang/Object;Ljava/lang/Object;)
                           //Ljava/lang/Object;
35: pop
```

Последовательность байтов с 21-го по 25-й на первый взгляд кажется немного странной. Экземпляр `HashMap`, который был создан в позиции 4 и сохранен в локальную переменную 3 инструкцией 11, теперь загружается обратно в стек, а копия ссылки сохраняется в локальную переменную 4. Процесс удаляет ее из стека, поэтому ее нужно загрузить заново (из переменной 4) перед тем, как использовать. Эта перетасовка происходит из-за того, что в исходном коде Java мы создали дополнительную локальную переменную (типа `Map` вместо `HashMap`), хотя она всегда ссылается на тот же объект, что и исходная переменная. Это еще

https://t.me/javalib

один пример того, как байт-код старается по возможности точнее следовать за исходным кодом.

После этих манипуляций со стеком и переменной значения, которые нужно поместить в отображение, загружаются в инструкциях с 26-й по 29-ю. Получатель и аргументы хранятся в стеке, и в инструкции 30 происходит вызов `put()`. На этот раз используется код операции `invokeinterface`, хотя на самом деле вызывается тот же самый метод. Это происходит потому, что локальная переменная Java относится к типу `Map` — типу интерфейса. И снова возвращаемое значение `put()` игнорируется: его отбрасывает операция `pop` в позиции 35.

Полезно не только знать о том, в какие операции преобразуются вызовы тех или иных методов Java, но и понимать некоторые нюансы кодов операций вызова. Прежде всего, у `invokeinterface` есть дополнительные параметры. Они присутствуют по историческим причинам и ради обратной совместимости и в наше время не используются. А два дополнительных нуля у `invokedynamic` предназначены для совместимости с будущими версиями.

Другой важный нюанс — различия между обычным и специальным вызовом метода экземпляра. Обычный вызов является *виртуальным*: конкретный метод, который будет вызван, выбирается во время выполнения по стандартным правилам переопределения методов в Java.

Однако существует пара особых случаев, включая вызовы метода надкласса. В этих случаях правила переопределения не должны срабатывать, так что нужен другой код операции вызова. Вот почему среди кодов операций должен быть код, вызывающий методы без механизма переопределения, — это `invokespecial`, который точно указывает, какой метод надо вызвать.

4.4.8. Платформенные операции

В семейство кодов платформенных операций входит код операции `new`, который создает новые экземпляры объектов, а также коды операций, относящиеся к потокам, такие как `monitorenter` и `monitorexit`. Это семейство описано в табл. 4.7.

Таблица 4.7. Коды платформенных операций

Название	Аргументы	Структура стека	Описание
new	i1, i2	[] -> [obj]	Выделяет память для нового объекта, тип которого задается константой по заданному индексу
monitorenter		[obj] -> []	Устанавливает блокировку объекта. См. главу 5
monitorexit		[obj] -> []	Снимает блокировку объекта. См. главу 5

Платформенные операции управляют некоторыми аспектами жизненного цикла объекта, в частности созданием новых объектов и их блокировкой. Важно

https://t.me/javalib

заметить, что код `new` только выделяет память. Для того чтобы действительно сконструировать объект, нужно также выполнить код, который содержится в конструкторе.

На уровне байт-кода конструктор преобразуется в метод со специальным именем `<init>`. Этот метод нельзя вызвать из кода Java, но можно из байт-кода. Так складывается характерная схема байт-кода, которая соответствует созданию объекта: код операции `new`, за которым следует `dup`, а затем `invokespecial`, который вызывает метод `<init>`, как было показано ранее.

Байт-коды `monitorenter` и `monitorexit` соответствуют началу и концу синхронизированного блока.

4.4.9. Сокращенные формы кодов операций

У многих кодов операций есть сокращенные формы, которые позволяют сэкономить несколько байтов в том или ином месте. Общая закономерность состоит в том, что обращения к одним локальным переменным происходят намного чаще, чем к другим, поэтому уместен специальный код операции, который означает «выполнить такую-то операцию непосредственно с локальной переменной», что позволяет не передавать эту переменную в аргументе. Так в семействе загрузки и сохранения появились такие коды операций, как `aload_0` и `dstore_2`, которые на 1 байт короче эквивалентных последовательностей `aload 00` и `dstore 02`.

> **ПРИМЕЧАНИЕ** Казалось бы, один байт — незначительная экономия, но в масштабах класса она накапливается. Изначально Java использовался в апплетах, которые часто загружались через телефонные модемы со скоростью 28,8 Кбит/с. С такой скоростью соединения было важно экономить байты при любой возможности.

Чтобы стать по-настоящему опытным разработчиком Java, вам стоит запускать `javap` для некоторых своих классов и учиться распознавать типичные закономерности в байт-коде. А пока, вооружившись этим кратким введением в байт-код, перейдем к следующей теме — рефлексии.

4.5. РЕФЛЕКСИЯ

Рефлексия (reflection) — один из ключевых приемов, которыми должен владеть опытный Java-разработчик. Это чрезвычайно мощная функциональность, но на первых порах у многих программистов с ней возникают затруднения, потому что рефлексия кажется чужеродной на фоне того, как большинство Java-разработчиков представляют себе работу с кодом.

Рефлексией называется возможность отслеживать и модифицировать состояние и поведение объектов во время выполнения. В зависимости от контекста рефлексию можно рассматривать как сущность разной природы:

- API языка программирования.
- Стиль или технология программирования.

https://t.me/javalib

- Механизм среды выполнения, на основе которого работает эта технология.
- Свойство системы типов языка.

Рефлексия в объектно-ориентированной системе фактически означает, что среда программирования может представлять типы и методы программы в виде объектов. Это возможно только в тех языках, среда выполнения которых поддерживает такую функциональность, и это фундаментально динамический аспект языка.

Программируя в рефлексивном стиле, можно манипулировать объектами, вообще не задействуя их статические типы. На первый взгляд это кажется шагом назад, однако если вы можете обращаться к объектам, не зная их статических типов, это значит, что можно создавать библиотеки, фреймворки и инструменты, которые способны работать с *любым* типом, в том числе с типами, которые даже не существовали на момент написания кода.

Когда язык Java еще был совсем молодым, рефлексия стала одним из ключевых технологических нововведений, которые он принес в массовое программирование. Хотя в других языках (прежде всего в Smalltalk) рефлексия появилась намного раньше, на момент выпуска Java она еще не была стандартной возможностью многих языков.

4.5.1. Введение в рефлексию

Абстрактное описание рефлексии часто выглядит невразумительным или сложным для понимания. Давайте опробуем несколько простых примеров в JShell, чтобы получить более конкретное представление о том, что же такое рефлексия:

```
jshell> Object o = new Object();
o ==> java.lang.Object@a67c67e

jshell> Class<?> clz = o.getClass();
clz ==> class java.lang.Object
```

Это всего лишь первый проблеск рефлексии: мы получили объект класса для типа `Object`. Собственно, фактическим типом `clz` является `Class<Object>`, но когда мы получаем объект класса при загрузке класса или вызове `getClass()`, с ним приходится работать, используя в обобщениях неизвестный тип `?`:

```
jshell> Class<Object> clz = Object.class;
clz ==> class java.lang.Object

jshell> Class<Object> clz = o.getClass();
| Error:
| incompatible types: java.lang.Class<capture#1 of ? extends
 java.lang.Object> cannot be converted to java.lang.Class<java.lang.Object>[1]
| Class<Object> clz = o.getClass();
|                     ^----------^
```

[1] Ошибка: несовместимые типы: java.lang.Class<capture#1 of ? extends java.lang.Object> нельзя преобразовать в java.lang.Class<java.lang.Object>. — *Примеч. пер.*

https://t.me/javalib

Дело в том, что рефлексия — это динамический механизм, который работает во время выполнения, а компилятору исходного кода неизвестен истинный тип `Class<Object>`. Этот фактор вносит неустранимую дополнительную сложность в работу с рефлексией, потому что система типов Java здесь практически ничем не может помочь. С другой стороны, в этой динамической природе и состоит суть рефлексии: если на стадии компиляции неизвестно, к какому типу что-то относится, и с этим приходится работать на самом общем уровне, то эту гибкость можно использовать, чтобы создавать открытые расширяемые системы.

> **ПРИМЕЧАНИЕ** Рефлексия порождает открытую по своей сути систему, а такой подход, как было показано в главе 2, может конфликтовать со стремлением к более инкапсулированным системам, которые модули Java пытаются воплотить на платформе.

Функциональность многих известных фреймворков и средств разработчика в значительной мере опирается на рефлексию: например, ее задействуют отладчики или браузеры кода. Рефлексия широко применяется в архитектуре плагинов, интерактивных средах и REPL. Собственно, саму среду JShell нельзя было бы встроить в язык без подсистемы рефлексии. Воспользуемся этой функциональностью и продемонстрируем с помощью JShell некоторые ключевые возможности рефлексии:

```
jshell> class Pet {
   ...> public void feed() {
   ...>   System.out.println("Покорми кота");
   ...> }
   ...> }
| created class Pet

jshell> var clz = Pet.class;
clz ==> class Pet
```

Теперь у нас есть объект, который представляет тип класса `Pet`, и с этим объектом можно выполнять различные действия, например создать новый экземпляр:

```
jshell> Object o = clz.newInstance();
o ==> Pet@66480dd7
```

Здесь проблема в том, что `newInstance()` возвращает `Object`, а это не самый полезный тип. Конечно, мы могли бы привести `o` к типу `Pet`, но для этого надо заранее знать, с какими типами мы работаем, а это противоречит динамической природе рефлексии. Попробуем поступить иначе:

```
jshell> import java.lang.reflect.Method;

jshell> Method m = clz.getMethod("feed", new Class[0]);
m ==> public void Pet.feed()
```

https://t.me/javalib

Теперь у нас есть объект, который представляет метод `feed()`, но делает это в виде абстрактных метаданных — он не связан ни с каким конкретным экземпляром.

Самое естественное, что можно сделать с объектом, который представляет метод, — вызвать его. В классе `java.lang.reflect.Method` определен метод `invoke()`, который фактически вызывает метод, представляемый объектом `Method`.

> **ПРИМЕЧАНИЕ** В примерах JShell мы опускаем значительную часть кода обработки исключений. Если вы используете рефлексию в обычном коде на Java, вам придется так или иначе обработать все возможные типы исключений.

Чтобы вызов завершился успешно, надо предоставить правильное количество аргументов правильных типов. В списке аргументов должен быть *объект-получатель*, для которого метод вызывается рефлексивно (предполагается, что метод является методом экземпляра). В нашем простом примере это выглядит так:

```
jshell> Object ret = m.invoke(o);
Покорми кота
ret ==> null   <-- Вызов возвращает null, потому что метод feed() объявлен с void
```

Помимо объектов `Method`, рефлексия также предоставляет объекты для других фундаментальных компонентов системы типов и языка Java — полей, аннотаций, конструкторов и т. д. Эти классы находятся в пакете `java.lang.reflect`, а некоторые из них (например, `Constructor`) являются обобщенными типами.

Подсистему рефлексии также пришлось обновить для работы с модулями. Подобно тому как с классами и методами можно работать рефлексивно, должен существовать рефлексивный API для работы с модулями. Ключевое место в нем вполне ожидаемо занимает класс `java.lang.Module`, и доступ к нему можно получить непосредственно от объекта `Class`:

```
var module = String.class.getModule();
var descriptor = module.getDescriptor();
```

Дескриптор модуля имеет тип `ModuleDescriptor` и предоставляет доступ к метаданным модуля только для чтения — по сути, они аналогичны содержимому `module-info.class`.

Новый рефлексивный API также поддерживает динамическую функциональность, включая поиск модулей. Для этого используются такие интерфейсы, как `ModuleFinder`, хотя подробное описание работы с системой модулей с помощью рефлексии выходит за рамки книги. Если вы заинтересовались этой темой, обратитесь к главе 12 книги Николая Парлога (Nicolai Parlog) «The Java Module System»[1] (Manning, 2019), http://mng.bz/gwGG.

[1] Парлог Н. «Система модулей Java». СПб., издательство «Питер».

https://t.me/javalib

4.5.2. Загрузка классов в сочетании с рефлексией

В следующем примере загрузка классов сочетается с рефлексией. Нам не понадобится полноценный загрузчик классов, который соблюдает обычные протоколы `findClass()` и `loadClass()`. Вместо этого мы просто наследуем `ClassLoader`, чтобы получить доступ к защищенному методу `defineClass()`.

Метод `main` получает список имен файлов, и если файл является классом Java, то с помощью рефлексии `main` последовательно перебирает все его методы и проверяет, являются ли они низкоуровневыми (native):

```
public class NativeMethodChecker {

    public static class EasyLoader extends ClassLoader {
        public EasyLoader() {
            super(EasyLoader.class.getClassLoader());
        }

        public Class<?> loadFromDisk(String fName) throws IOException {
            var b = Files.readAllBytes(Path.of(fName));
            return defineClass(null, b, 0, b.length);
        }
    }

    public static void main(String[] args) {
        if (args.length > 0) {
            var loader = new EasyLoader();
            for (var file : args) {
                System.out.println(file +" ::");
                try {
                    var clazz = loader.loadFromDisk(file);
                    for (var m : clazz.getMethods()) {
                        if (Modifier.isNative(m.getModifiers())) {
                            System.out.println(m.getName());
                        }
                    }
                } catch (IOException | ClassFormatError x) {
                    System.out.println("Не файл класса");
                }
            }
        }
    }
}
```

Такие интересные примеры помогают исследовать динамическую природу платформы Java, а также изучить, как работает Reflection API. Однако опытному Java-разработчику следует знать об ограничениях и проблемах, которые могут возникнуть с рефлексией.

4.5.3. Проблемы с рефлексией

Reflection API был частью платформы Java с версии 1.1 (1996), и за прошедшие 25 лет были выявлены некоторые его недочеты и слабости, например:

https://t.me/javalib

- Это очень старый API, в котором повсеместно используются типы массивов (он появился до коллекций Java).
- Не так уж просто определить, какую из перегруженных версий метода нужно вызвать.
- Чтобы рефлексивно обращаться к методам, в API есть два разных метода — `getMethod()` и `getDeclaredMethod()`.
- API предоставляет метод `setAccessible()`, с помощью которого можно обойти управление доступом.
- Для рефлексивных вызовов сложно обрабатывать исключения: проверяемые исключения повышаются до исключений времени выполнения.
- Для рефлексивных вызовов, которые передают или возвращают примитивы, необходима упаковка и распаковка (boxing/unboxing).
- Примитивным типам необходимы объекты классов, используемые как обертки, например `int.class`, который в действительности имеет тип `Class<Integer>`.
- Методы, объявленные с `void`, требуют задействовать тип `java.lang.Void`.

Кроме неудобств API, рефлексия в Java никогда не отличалась высоким быстродействием по разным причинам, включая плохую сочетаемость с JIT-компилятором JVM.

> **ПРИМЕЧАНИЕ** Решить проблему быстродействия рефлексивных вызовов было одной из основных причин для добавления API дескрипторов методов, который будет рассмотрен в главе 17.

С рефлексией связана еще одна проблема, которая, пожалуй, имеет скорее философский характер (или может рассматриваться как антипаттерн): многие разработчики осваивают рефлексию как один из первых действительно продвинутых механизмов на пути своего профессионального роста. В результате рефлексией начинают злоупотреблять по принципу «золотого молотка»: системы, которые реализуют на ее основе, обладают избыточной гибкостью или предоставляют внутренний мини-фреймворк, который на самом деле не нужен (иногда это называется антипаттерном «Внутренний фреймворк» (Inner Framework)). Такие системы часто предоставляют богатые возможности настройки, но это достигается за счет того, что модель предметной области кодируется в конфигурации, а не в типах предметной области.

Рефлексия — замечательный механизм, который должен освоить каждый опытный Java-разработчик. Но, безусловно, этот механизм подходит не для каждой ситуации, и большинство профессиональных разработчиков используют его лишь в редких случаях.

https://t.me/javalib

ИТОГИ

- Формат файла класса и загрузка классов занимают центральное место в работе JVM. Они чрезвычайно важны для каждого языка, который должен выполняться на виртуальной машине Java.
- Различные фазы загрузки класса улучшают как безопасность, так и быстродействие кода во время выполнения.
- Коды операций байт-кода JVM делятся на семейства, объединенные сходной функциональностью.
- Дизассемблирование файлов классов с помощью `javap` помогает понять, как работает код на низком уровне.
- Рефлексия — важная и исключительно мощная функциональность.

https://t.me/javalib

5 Основы конкурентности в Java

В ЭТОЙ ГЛАВЕ

- ✓ Теория конкурентного выполнения
- ✓ Блоковая структура конкурентности
- ✓ Синхронизация
- ✓ Модель памяти Java (JMM)
- ✓ Поддержка конкурентности в байт-коде

В Java существуют два почти не связанных друг с другом API конкурентного выполнения: старый API, который обычно называется блоковым, синхронизационным и даже «классическим», и новый API, который в основном известен под именем своего пакета в Java — `java.util.concurrent`.

В книге мы рассмотрим оба интерфейса. В этой главе начнем с первого API, а речь о `java.util.concurrent` пойдет в следующей главе. Ближе к концу книги (а именно в главе 16) мы вернемся к теме конкурентности, когда будем обсуждать более сложные вопросы, конкурентность в других языках JVM помимо Java, и взаимодействие между конкурентностью и функциональным программированием.

Итак, начнем с классического подхода к конкурентному выполнению, который был единственным API до выхода Java 5. Как нетрудно догадаться по альтернативному названию — «конкурентность на основе синхронизации», — это API на уровне языка, который встроен в платформу и имеет дело с ключевыми словами `synchronized` и `volatile`.

https://t.me/javalib

Это низкоуровневый API, с которым не всегда просто работать, но его определенно стоит освоить. Он закладывает надежный фундамент для следующих глав этой книги, где рассматриваются другие виды и различные аспекты конкурентности.

На самом деле трудно предметно говорить о других формах конкурентного выполнения без хотя бы минимальных знаний низкоуровневого API и концепций, которые будут представлены в этой главе. Освещая соответствующие темы, мы также приведем достаточно теории, помогающей понять другие подходы к конкурентности, которые будут рассмотрены позднее в книге, в том числе и при описании конкурентного выполнения в языках, отличных от Java.

Чтобы понять, как устроена конкурентность в Java, стоит начать с небольшого теоретического введения. После этого мы обсудим, как «факторы проектирования» влияют на архитектуру и реализацию систем. Мы поговорим о двух самых важных из этих факторов — *безопасности* и *живости*, а также упомянем ряд других.

Важный (и самый длинный в этой главе) раздел посвящен подробностям блоковой конкурентности и анализу низкоуровневого потокового API. В конце главы мы обсудим модель памяти Java (JMM), после чего воспользуемся знаниями байт-кода из главы 4, чтобы понять реальный источник некоторых типичных сложностей в конкурентном программировании на Java.

5.1. КРАТКИЙ КУРС КОНКУРЕНТНОСТИ

Наше путешествие в мир конкурентности начнется с базовой теории, но сначала необходимо сделать небольшое предупреждение.

5.1.1. Но я уже знаю о Thread

Это одно из самых распространенных (и возможно, самых опасных) заблуждений разработчиков: считать, что иметь представление о `Thread`, `Runnable` и базовых примитивах механизма конкурентности Java на уровне языка достаточно для того, чтобы грамотно разрабатывать конкурентный код. На самом деле тема конкурентности весьма обширна, а качественное многопоточное программирование — сложное дело, с которым возникают проблемы даже у лучших разработчиков с многолетним опытом.

Также правда и то, что в области конкурентного выполнения в настоящее время ведутся масштабные, активные исследования: они продолжаются по крайней мере 5–10 последних лет с неослабевающей активностью. Плоды этих исследований с большой вероятностью повлияют на Java и другие языки, с которыми вы будете встречаться на протяжении своей карьеры.

В первом издании книги мы утверждали следующее: «Если бы нам предложили назвать фундаментальную область вычислительной техники, которая должна радикально измениться в практическом отношении в ближайшие несколько лет,

https://t.me/javalib

то мы назвали бы именно конкурентную обработку» . С тех пор мы убедились не только в том, что это утверждение подтвердилось на практике, но и в том, что его можно с полной уверенностью распространить на будущее: в следующие пять лет мы будем наблюдать повышенное внимание к различным подходам к конкурентности, которые стали естественной частью современного программирования.

Итак, вместо того чтобы пытаться подробно описать все аспекты конкурентного программирования, в этой главе мы опишем платформенный механизм, на который оно опирается, и объясним, почему конкурентность в Java работает именно так, а не иначе. Кроме того, мы приведем достаточно общей теории конкурентного выполнения, в результате чего вы овладеете необходимыми понятиями, чтобы ориентироваться в этой теме, а также поймете, почему важно хорошо разбираться в конкурентности и с какими трудностями можно столкнуться в этой области. Но сначала обсудим то, что каждый опытный Java-разработчик должен знать об оборудовании, а также рассмотрим одно из самых важных теоретических ограничений конкурентности.

5.1.2. Оборудование

Начнем с некоторых базовых фактов о конкурентности и многопоточности:

- Конкурентное программирование, по сути, направлено на повышение быстродействия.
- Фактически нет никаких причин реализовывать конкурентный алгоритм, если в целевой системе ваш код с последовательным алгоритмом выполняется достаточно быстро.
- Современные компьютерные системы оснащаются многоядерными процессорами. Даже в нынешних мобильных телефонах обычно бывает 2–4 ядра.
- Все программы на Java являются многопоточными, даже если в них всего один поток приложения.

Последний пункт объясняется тем, что сама JVM представляет собой многопоточный двоичный блок, который может использовать несколько ядер (например, для JIT-компиляции или для сборки мусора). Кроме того, в стандартную библиотеку также входят API, которые используют конкурентное выполнение под управлением исполнительной среды, чтобы реализовать многопоточные алгоритмы в некоторых задачах.

> **ПРИМЕЧАНИЕ** Вполне возможно, что приложение на Java начнет работать быстрее просто потому, что вы обновите JVM, на которой оно запускается: это будет следствием того, что в новой среде выполнения улучшено быстродействие.

Оборудование подробнее обсуждается в главе 7, но эти простые факты настолько фундаментальны и важны для конкурентного программирования, что мы решили изложить их немедленно.

https://t.me/javalib

Теперь познакомимся с законом Амдала, названным в честь ученого из IBM Джина Амдала, которого иногда называют «отцом мейнфреймов».

5.1.3. Закон Амдала

Закон Амдала — это простая приближенная модель, которая позволяет анализировать эффективность распределения работы между несколькими вычислительными единицами. В модели Амдала вычислительные единицы имеют абстрактную природу, так что их можно считать программными потоками, хотя они с тем же успехом могут быть процессами или любыми другими сущностями, которые способны выполнять работу.

> **ПРИМЕЧАНИЕ** Никакие предусловия или следствия закона Амдала не зависят от подробностей того, как выполняется работа, что конкретно представляют собой рабочие единицы или как реализованы вычислительные системы.

Основная предпосылка заключается в том, что есть некоторая задача, которую можно разбить на несколько меньших частей для обработки. Это позволяет использовать несколько рабочих единиц и тем самым сократить время, которое необходимо, чтобы завершить работу.

Таким образом, если у нас есть N процессоров (или потоков, выполняющих работу), можно было бы наивно ожидать, что затраченное время составит T1/N (где T1 — время выполнения задачи на одном процессоре). В такой модели всю работу можно было бы завершить сколь угодно быстро, если просто добавлять вычислительные единицы, то есть увеличивать N.

Однако разбиение работы не обходится бесплатно! Деление задачи на части и их повторное объединение сопряжено с затратами (если повезет, небольшими). Предположим, эти затраты (которые иногда называются *последовательной частью* вычислений) составляют несколько процентов, и их можно представить числом s ($0 < s < 1$). Например, типичное значение s может составлять 0,05 (или 5 %, если вы предпочитаете такой формат). Это означает, что суммарное время на выполнение задачи всегда будет не менее s*T1, сколько бы вычислительных единиц вы ни добавили.

Здесь предполагается, что s не зависит от N, но на практике деление работы, которое выражается с помощью s, может усложняться и требовать *больше* времени с ростом N. Очень трудно представить себе архитектуру системы, в которой s *уменьшалось бы* с ростом N. Таким образом, простое допущение, что s постоянно, обычно интерпретируется как оценка *лучшего случая*.

В итоге закон Амдала проще всего описать так: если s лежит в диапазоне от 0 до 1, то максимальный прирост скорости, которого можно достичь, составляет 1/s раз. Результат немного угнетает: получается, что даже если накладные затраты не превышают 2 %, то хоть при тысячах процессоров, которые работают на полной мощности, скорость вырастет не более чем в 50 раз.

https://t.me/javalib

У закона Амдала также есть чуть более сложная формулировка, которую можно представить так:

```
T(N) = s + (1/N) * (T1 – s)
```

Эта зависимость наглядно показана на рис. 5.1. Обратите внимание: ось x проградуирована в логарифмическом масштабе, потому что сходимость к 1/s было бы трудно продемонстрировать на линейной шкале.

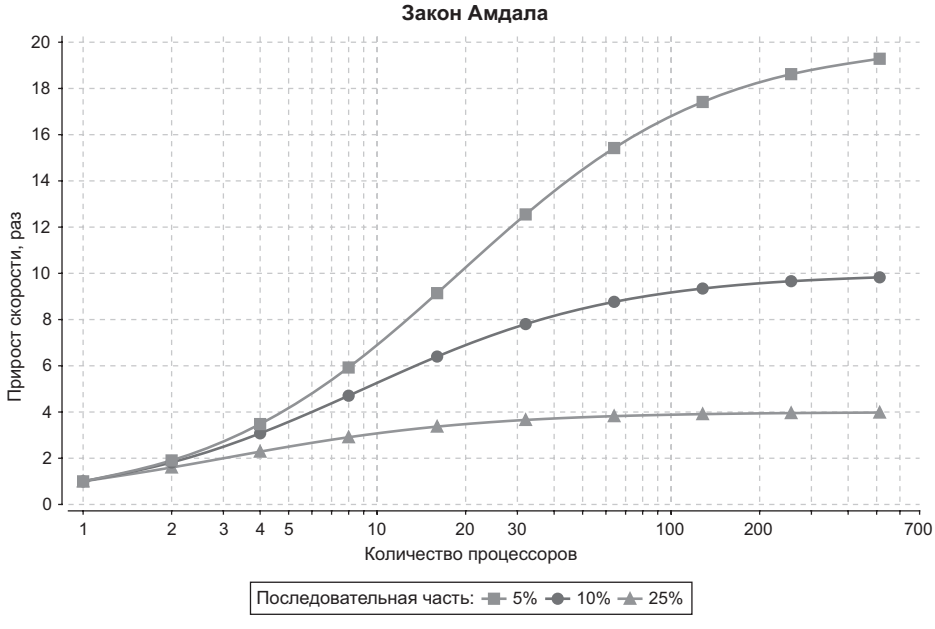


Рис. 5.1. Закон Амдала

Итак , после того как мы затронули тему оборудования и представили первую, очень простую модель конкурентного выполнения, перейдем к специфике потоковой модели Java.

5.1.4. Потоковая модель Java

Потоковая модель Java основана на двух фундаментальных принципах:

- Общее, видимое по умолчанию изменяемое состояние.
- Планирование потоков с вытеснением, осуществляемое операционной системой.

Рассмотрим важнейшие особенности этой модели:

- Объекты могут легко совместно использоваться всеми потоками процесса.
- Любые потоки, у которых есть ссылка на какой-либо объект, могут изменять его.

https://t.me/javalib

- Планировщик потоков (операционная система) может назначать потоки ядрам и выгружать потоки с ядер в любой момент времени (более или менее).
- Должна существовать возможность выгрузить метод во время его выполнения (иначе метод с бесконечным циклом сможет захватить все процессорное время).
- Однако при этом возникает риск переключения потоков в неподходящий момент, отчего метод останется «недовыполненным», а те или иные объекты окажутся в противоречивом состоянии.
- Доступ к объектам может *блокироваться* для защиты данных.

Последний пункт крайне важен: в противном случае возник бы огромный риск того, что в одних потоках не будут правильно отражаться изменения, которые были внесены в других потоках. Чтобы блокировать объекты, в базовом языке Java предусмотрено ключевое слово `synchronized`.

> **ПРИМЕЧАНИЕ** Формально Java предоставляет для всех своих объектов *мониторы*, которые сочетают блокировку (то есть взаимное исключение) с возможностью ожидания, пока некоторое условие не станет истинным.

Конкурентность Java, основанная на потоках и блокировке, функционирует на очень низком уровне, и работать с ней часто бывает трудно. Чтобы преодолеть эту проблему, в Java 5 появился набор конкурентных библиотек, известный под названием `java.util.concurrent` — по имени пакета, в котором находятся новые классы. Это набор инструментов для написания конкурентного кода, с которым, по мнению многих программистов, проще работать, чем с конкурентными примитивами на основе классической блоковой структуры. Мы рассмотрим `java.util.concurrent` в следующей главе, а сейчас сосредоточимся на API уровня языка.

5.1.5. Полезный опыт

Java стал первым массовым языком программирования со встроенной поддержкой многопоточности. На тот момент это было огромным шагом вперед, но сейчас, через 15 лет, мы узнали много нового о том, как писать конкурентный код.

Как выяснилось, некоторые тогдашние проектировочные решения Java оказались слишком сложными для большинства программистов. Об этом можно только пожалеть, потому что аппаратное обеспечение активно развивается в сторону многоядерных процессоров, а раскрыть потенциал таких процессоров можно только в конкурентном коде. В этой главе мы обсудим некоторые трудности работы с конкурентным кодом. Тема современных процессоров, которые органически требуют конкурентного программирования, подробнее рассматривается в главе 7, где идет речь о производительности.

По мере того как разработчики накапливают опыт работы с конкурентным кодом, они начинают сталкиваться с одними и теми же аспектами, которые играют важную роль для их систем. Мы будем называть эти аспекты *факторами*

https://t.me/javalib

проектирования. Речь идет о высокоуровневых концепциях, которые проявляются на практике (и нередко становятся причиной конфликтов) в проектировании конкурентных объектно-ориентированных систем. В следующих двух разделах мы рассмотрим важнейшие из этих факторов.

5.2. ФАКТОРЫ ПРОЕКТИРОВАНИЯ

Самые важные факторы проектирования, перечисленные ниже, сформулировал Даг Ли, занимаясь одним из своих главных проектов — созданием `java.util.concurrent`:

- безопасность (safety), также называемая конкурентной типобезопасностью (concurrent type safety);
- живость (liveliness);
- производительность (performance);
- повторное использование (reusability).

Рассмотрим каждый из этих факторов по отдельности.

5.2.1. Безопасность и конкурентная типобезопасность

Безопасность означает, что экземпляры объектов сохраняют логическую целостность независимо от того, какие еще операции могут выполняться в это же время. Если система объектов обладает таким свойством, она называется *конкурентно типобезопасной* (или просто *безопасной*).

Как нетрудно догадаться по названию, конкурентность можно рассматривать как расширение обычных понятий моделирования объектов и безопасности типов. В неконкурентном коде необходимо следить за тем, чтобы независимо от того, какие открытые методы вызываются для объекта, в конце вызова метода он оставался в четко определенном и непротиворечивом состоянии. Обычно для этого все состояние объекта объявляется частным, а для внешнего доступа предоставляется открытый API с методами, которые изменяют состояние объекта только так, чтобы это имело смысл с точки зрения предметной области.

Конкурентная типобезопасность — такое же базовое понятие, как обычная типобезопасность для объекта, но она применяется в гораздо более сложной обстановке, когда разные потоки потенциально могут обращаться одновременно к одним и тем же объектам на разных ядрах процессора. Например, возьмем такой простой класс:

```
public class StringStack {
    private String[] values = new String[16];
    private int current = 0;

    public boolean push(String s) {
        // Обработка исключений опущена
```

https://t.me/javalib

```
        if (current < values.length) {
            values[current] = s;
            current = current + 1;
        }
        return false;
    }

    public String pop() {
        if (current < 1) {
            return null;
        }
        current = current - 1;
        return values[current];
    }
}
```

Если использовать этот класс в однопоточном клиентском коде, проблем не возникает. Однако планирование потоков с вытеснением может создать неприятности. Например, переключение контекста между потоками может произойти в следующей точке кода:

```
public boolean push(String s) {
        if (current < values.length) {
            values[current] = s;
            // .... здесь переключается контекст
            current = current + 1;
        }
        return false;
    }
```

Объект остается в наполовину измененном, противоречивом состоянии

Если затем обратиться к объекту из другого потока, окажется, что одна часть состояния (`values`) обновлена, а другая (`current`) — нет. Материал этой главы в основном посвящен тому, как исследовать и решать такие проблемы.

В общем случае одна из стратегий безопасности заключается в том, чтобы никогда не возвращать управление из нечастного (non-private) метода в противоречивом состоянии и никогда не вызывать какой-либо нечастный метод (и конечно, метод любого другого объекта) из противоречивого состояния. Если объединить этот принцип с механизмами защиты объекта, который находится в некорректном состоянии (к таким механизмам относится синхронизационная блокировка и критические секции), то безопасность системы будет гарантирована.

5.2.2. Живость

Система называется живой, если любая попытка совершить в ней какое-либо действие когда-нибудь завершается удачей или неудачей. Неживая система фактически застревает в одном состоянии: она и не продолжает работу, и не завершает ее.

В этом определении очень важны слова «когда-нибудь»: есть разница между временным отсутствием прогресса (что само по себе не так уж страшно, хотя

https://t.me/javalib

и не идеально) и постоянным сбоем. Временные неудачи могут вызываться множеством причин:

- блокировка или ожидание блокировки;
- ожидание ввода (например, сетевого ввода/вывода);
- ресурс временно недоступен;
- не хватает процессорного времени, чтобы выполнять поток.

Постоянные сбои тоже могут объясняться разными причинами, и вот несколько типичных примеров:

- взаимная блокировка;
- неустранимая проблема с ресурсом (например, сетевая файловая система вышла из строя);
- пропущенный сигнал.

Позже в этой главе мы обсудим блокировку и некоторые другие проблемы, хотя, возможно, они уже в какой-то степени вам знакомы.

5.2.3. Производительность

Производительность системы можно измерить разными способами. В главе 7 мы поговорим об анализе производительности и приемах оптимизации, а также представим некоторые метрики, о которых стоит знать. Пока можно считать, что производительность показывает, сколько работы может выполнить система при заданном количестве ресурсов.

5.2.4. Повторное использование

Возможность повторного использования становится четвертым фактором проектирования, потому что остальные факторы ее не учитывают. Иногда весьма желательно, чтобы конкурентная система была спроектирована с расчетом на удобство повторного использования, хотя это не всегда легко реализовать. Один из подходов заключается в том, чтобы с помощью повторно используемых инструментов (таких, как `java.util.concurrent`) создавать код приложения, который сам по себе не рассчитан на повторное использование.

5.2.5. Как и почему возникают конфликты между факторами?

Факторы проектирования часто конфликтуют друг с другом, и это главная причина, по которой так сложно построить хорошую конкурентную систему:

- Безопасность противоречит живости: безопасность направлена на то, чтобы не происходило ничего плохого, тогда как живость требует продвигаться вперед.

https://t.me/javalib

- Системы, пригодные для повторного использования, обычно предоставляют доступ к своему внутреннему устройству, что может создать проблемы с безопасностью.
- Система с наивно реализованной безопасностью может быть не очень производительной, потому что ради безопасности она обычно интенсивно применяет блокировки.

В конечном счете стоит стремиться к балансу. Код должен быть достаточно гибким, чтобы приносить пользу в широком диапазоне задач; достаточно закрытым, чтобы поддерживать безопасность; но при этом живым и производительным в разумных пределах. Эта цель кажется излишне амбициозной, но к счастью, есть практические рекомендации, которые помогут ее достичь. Вот несколько стандартных советов (начиная с самых полезных):

1. По возможности ограничьте внешние коммуникации каждой подсистемы. Сокрытие данных — мощный инструмент, который помогает усилить безопасность.
2. Сделайте внутреннюю структуру каждой подсистемы по возможности детерминированной. Например, поддерживайте в каждой подсистеме статические знания о потоках и объектах, даже если подсистемы взаимодействуют друг с другом конкурентным, недетерминированным образом.
3. Установите политику, которой должны придерживаться клиентские приложения. Это мощный инструмент, но он требует, чтобы пользовательские приложения следовали правилам, и трудно будет отлаживать код в ситуации, когда приложение с некорректным поведением их нарушает.
4. Документируйте нужное поведение. Это самая слабая из всех альтернатив, но иногда она бывает необходимой, если код предстоит развертывать в самом общем контексте.

Разработчику следует знать обо всех возможных механизмах безопасности и применять наиболее сильный из доступных вариантов, но при этом сознавать, что в некоторых ситуациях доступны только более слабые механизмы.

5.2.6. Источники затрат

Источником непроизводительных затрат ресурсов могут служить многие аспекты конкурентных систем, например:

- мониторы (то есть блокировки и переменные для условий);
- количество переключений контекста;
- количество потоков;
- планирование;
- локальность памяти;
- проектирование алгоритмов.

https://t.me/javalib

Стоит держать в голове этот чек-лист и при разработке параллельного кода проверять, что вы приняли во внимание все пункты.

В частности, последний пункт — проектирование алгоритмов — дает разработчикам отличную возможность проявить себя, потому что алгоритмическое мастерство повысит вашу квалификацию независимо от того, на каком языке вы программируете.

Есть две классические книги, которые мы рекомендуем как превосходные источники дальнейшей информации об алгоритмах (как однопоточных, так и конкурентных): Кормен и др. (Cormen at al.) «Introduction to Algorithms»[1] (MIT, 2009) и Скиена (Skiena) «The Algorithm Design Manual (3rd ed.)»[2] (Springer-Verlag, 2020).

Многие источники затрат ресурсов будут упоминаться в этой и последующих главах (особенно в главе 7, которая посвящена производительности), а пока обратимся к следующей теме — обзору «классической» конкурентности Java, а также разберемся, почему с ней бывает так сложно программировать.

5.3. КОНКУРЕНТНОЕ ВЫПОЛНЕНИЕ С БЛОКОВОЙ СТРУКТУРОЙ (ДО JAVA 5)

Говоря о конкурентном выполнении Java, мы в основном будем обсуждать технологии, альтернативные по отношению к механизму конкурентности на уровне языка, — она же конкурентность с блоковой структурой, она же *встроенная конкурентность*. Но чтобы извлечь максимум пользы из обсуждения альтернатив, важно четко понимать, чем хорош и чем плох классический подход к конкурентности.

Поэтому в оставшейся части этой главы мы рассмотрим исходный, достаточно низкоуровневый подход к реализации многопоточного программирования с использованием ключевых слов Java: `synchronized`, `volatile` и т. д. Обсуждение пойдет в контексте факторов проектирования и с учетом того, что вы узнаете позднее.

После этого мы кратко обсудим жизненный цикл потока, а затем продемонстрируем стандартные приемы (и ловушки) конкурентного кода: полную синхронизацию объектов, взаимные блокировки, ключевое слово `volatile` и неизменяемость. Начнем с общего обзора синхронизации.

5.3.1. Синхронизация и блокировки

Как вам, вероятно, уже известно, ключевое слово `synchronized` можно применять к блоку или к методу. Оно означает, что перед входом в блок или метод поток

[1] Кормен Т. и др. «Алгоритмы: построение и анализ».

[2] Скиена С. «Алгоритмы. Руководство по разработке. 3-е изд.».

https://t.me/javalib

должен установить соответствующую блокировку. Например, представьте метод, который снимает деньги с банковского счета:

```
public synchronized boolean withdraw(int amount) {
    // Проверить, что amount > 0, если нет - возбудить исключение
    if (balance >= amount) {
        balance = balance - amount;
        return true;
    }

    return false;
}
```

В любой момент времени только один поток может пытаться снять средства с этого счета

Метод должен установить блокировку на экземпляре объекта (а для статических синхронизированных методов — на объекте `Class`). В случае блока программист должен указать, какой объект нужно заблокировать.

В любой момент времени только один поток может обращаться к синхронизированным блокам или методам объекта; если к ним попытаются обратиться другие потоки, JVM их приостановит. Это происходит вне зависимости от того, пытается ли другой поток войти в тот же или в другой синхронизированный блок того же объекта. В теории конкурентного выполнения такие конструкции иногда называются *критическими секциями*, хотя этот термин чаще используется в C++, чем в Java.

> **ПРИМЕЧАНИЕ** А вы когда-нибудь задавались вопросом, почему для критических секций в Java используется ключевое слово `synchronized`? Почему не «critical» или «locked»? Что именно здесь синхронизируется? Мы вернемся к этому вопросу в разделе 5.3.5, но если вы не знаете ответа или никогда над ним не задумывались, поразмыслите пару минут, прежде чем двигаться дальше.

Рассмотрим несколько основных фактов, которые относятся к синхронизации и блокировкам в Java. Хочется верить, что многие из них (или все) вам уже хорошо известны:

- Блокировать можно только объекты, но не примитивы.
- Блокировка массива объектов не блокирует отдельные объекты.
- Синхронизированный метод можно рассматривать как эквивалент блока `synchronized (this) { … }`, который охватывает весь метод (но учтите, что в байт-коде эти конструкции представляются по-разному).
- Метод, объявленный как `static synchronized`, устанавливает блокировку для объекта `Class`, потому что не существует экземпляра, который можно было бы блокировать.
- Если вам необходимо блокировать объект `Class`, хорошо подумайте, стоит ли делать это явно или с вызовом `getClass()`, потому что в подклассе эти решения будут проявляться по-разному.
- Синхронизация во внутреннем классе не зависит от внешнего класса (чтобы понять, почему это так, вспомните, как реализуются внутренние классы).

https://t.me/javalib

- `synchronized` не входит в сигнатуру метода и поэтому не может присутствовать в объявлении метода интерфейса.
- Несинхронизированные методы не обращают внимания на состояние блокировок и могут продолжать работу в то же время, когда выполняются синхронизированные методы.
- Блокировки Java реентерабельны: если поток удерживает блокировку и встречает точку синхронизации для той же блокировки (например, синхронизированный метод, который вызывает другой синхронизированный метод для того же объекта), он сможет продолжить выполнение.

ПРИМЕЧАНИЕ Нереентерабельные схемы блокировки существуют в других языках, и их можно синтезировать в Java (если вам нужны все пугающие подробности, обращайтесь к документации Javadoc для `ReentrantLock` в `java.util.concurrent.locks`). Тем не менее работать с ними довольно тяжело и лучше их избегать — или очень хорошо понимать, что вы делаете.

На этом обзор синхронизации в Java завершен. Давайте теперь рассмотрим состояния, через которые проходит поток на протяжении своего жизненного цикла.

5.3.2. Модель состояний потока

На рис. 5.2 изображена модель состояний для потока Java. Она показывает, между какими состояниями переходит поток Java на протяжении своего жизненного цикла.

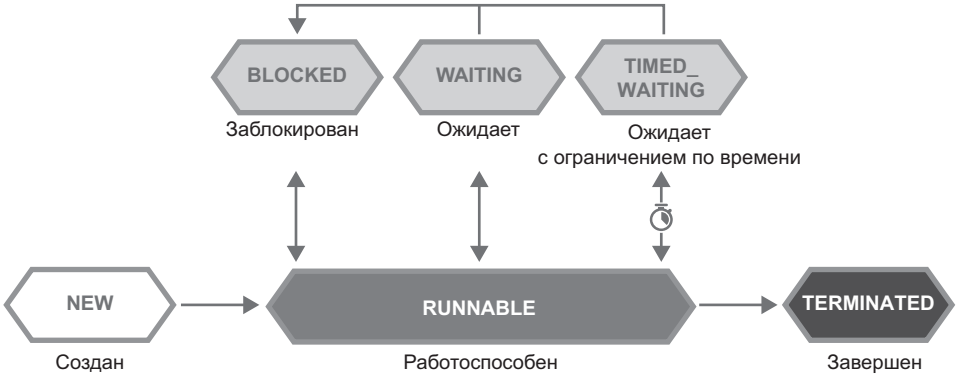


Рис. 5.2. Модель состояний потока Java

В Java есть перечисление `Thread.State`, элементы которого соответствуют состояниям из этой модели. Оно становится прослойкой над представлением состояний потока с точки зрения операционной системы.

Объект потока Java изначально создается в состоянии `NEW`. На этот момент поток ОС еще не существует (и возможно, никогда не будет существовать). Чтобы

https://t.me/javalib

создать поток как ветвь выполнения, необходимо вызвать `Thread.start()` — тем самым вы заставляете ОС создать настоящий поток.

> **ПРИМЕЧАНИЕ** В каждой ОС потоки устроены по-своему, и подробности реализации могут различаться. Впрочем, в современных операционных системах потоки и планирование реализованы примерно похожим образом, хотя так было не всегда (например, в Solaris или Windows XP).

Планировщик помещает новый поток в очередь выполнения и в какой-то будущий момент находит ядро, на котором поток будет выполняться (причем если машина сильно загружена, может потребоваться какое-то время ожидания). С того момента, как потоку будет выделено время, он станет выполняться, пока не израсходует это время, а затем переместится обратно в очередь выполнения, ожидая следующего временно́го интервала от процессора. Так работает принудительное планирование потоков, которое упоминалось в разделе 5.1.1.

В процессе планирования — когда поток назначается ядру, выполняется и возвращается в очередь выполнения — объект Java `Thread` находится в работоспособном состоянии (`RUNNABLE`). Остальные состояния из рис. 5.2 означают, что в настоящий момент поток не может использовать ядро. На уровне потока перейти в такой режим можно двумя способами:

1. Программный код вызывает `Thread.sleep()`, указывая тем самым, что поток должен выждать фиксированное время, прежде чем продолжить работу.
2. Поток понимает, что он должен дождаться, пока выполнится некоторое внешнее условие, и вызывает `Object.wait()`.

В обоих случаях ОС немедленно снимает поток с ядра. Тем не менее дальнейшие события развиваются по-разному.

В первом случае поток запрашивает приостановку на определенный промежуток времени. Поток Java переходит в состояние `TIMED_WAITING`, а операционная система запускает таймер. По истечении срока приостановленный поток активизируется: он снова готов к выполнению и возвращается в очередь выполнения.

Во втором случае задействуется условный аспект мониторов Java, которые действуют на уровне объекта. Поток переходит в состояние `WAITING` и ожидает неопределенно долго. Обычно он не активизируется, пока операционная система не сигнализирует, что условие было выполнено, — как правило, для этого из другого потока вызывается `Object.notify()` для текущего объекта.

Кроме этих двух состояний, которые контролируются на уровне потоков, поток может перейти в заблокированное состояние `BLOCKED`, потому что он ожидает ввода/вывода или получения блокировки, которую удерживает другой поток. Наконец, если поток ОС, соответствующий объекту Java `Thread`, прекратил выполнение, то этот объект перейдет в состояние `TERMINATED`.

А теперь поговорим об одном известном решении проблемы синхронизации: полностью синхронизированных объектах.

https://t.me/javalib

5.3.3. Полностью синхронизированные объекты

Ранее в этой главе мы представили концепцию конкурентной типобезопасности и упомянули, что ее можно добиться с помощью определенной стратегии. Она обычно называется стратегией *полностью синхронизированных объектов*, и сейчас мы рассмотрим ее подробнее. Если выполняются все перечисленные ниже правила, то класс гарантированно будет потоково-безопасным, а также живым.

Класс называется полностью синхронизированным, если он удовлетворяет всем условиям:

- В каждом конструкторе все поля всегда инициализируются логически непротиворечивым (целостным) состоянием.
- В классе нет открытых полей.
- Экземпляры объектов гарантированно остаются непротиворечивыми после возврата из любого нечастного метода (при этом предполагается, что при вызове метода состояние тоже было непротиворечивым).
- Все методы доказуемо завершаются за ограниченное время.
- Все методы синхронизированы.
- Ни один метод не вызывает методы другого экземпляра, когда находится в противоречивом состоянии.
- Ни один метод не вызывает никакие нечастные методы текущего экземпляра, когда находится в противоречивом состоянии.

В листинге 5.1 приведен пример такого класса из внутренней части банковской системы. Класс `FSOAccount` моделирует счет. Префикс `FSO` ясно обозначает, что в реализации используются полностью синхронизированные объекты (Fully Synchronized Objects).

Система позволяет вносить и снимать средства, а также запрашивать текущий баланс. При этом возможен классический конфликт между операциями чтения и записи, так что синхронизация используется, чтобы предотвратить противоречивость.

На первый взгляд все великолепно: класс и безопасный и живой одновременно. Проблема возникает с быстродействием. Если что-то безопасно и живо, это еще не значит, что оно обязательно будет быстрым. Чтобы координировать все обращения к балансу (как для чтения, так и для записи), приходится использовать `synchronized`, и блокировка может существенно замедлить выполнение. Это главная проблема такого подхода к синхронизации.

Листинг 5.1. Полностью синхронизированный класс

```
public class FSOAccount {
    private double balance;  ◄——  Нет открытых полей
```

https://t.me/javalib

```
    public FSOAccount(double openingBalance) {
        // Проверяет, что openingBalance > 0,
        // если нет — возбуждает исключение
        balance = openingBalance;
    }

    public synchronized boolean withdraw(int amount) {
        // Проверяет, что amount > 0,
        // если нет — возбуждает исключение
        if (balance >= amount) {
            balance = balance - amount;
            return true;
        }

        return false;
    }

    public synchronized void deposit(int amount) {
        // Проверяет, что amount > 0,
        // если нет — возбуждает исключение
        balance = balance + amount;
    }

    public synchronized double getBalance() {
        return balance;
    }
}
```

Все поля инициализируются в конструкторе

Все методы синхронизированы

Кроме проблем с быстродействием, код из листинга 5.1 весьма ненадежен. К балансу нельзя обратиться вне синхронизированного метода, но убедиться в этом можно, только если вручную проверить код, а в данном случае это удается только потому, что код невелик.

В реальных крупных системах ручная проверка обычно невозможна из-за крупного объема кода. Если использовать описанный подход в большой кодовой базе, то в код слишком легко проникают ошибки, и это еще одна причина, по которой сообщество Java стало искать более надежные решения.

5.3.4. Взаимная блокировка

Еще одна классическая проблема конкурентного выполнения (и не только его реализации в Java) — *взаимные блокировки* (deadlocks). Взгляните на листинг 5.2 — слегка расширенную версию предыдущего примера. Помимо моделирования баланса, здесь появился метод `transferTo()` для перевода денег с одного счета на другой.

ПРИМЕЧАНИЕ Это весьма наивный набросок многопоточной системы транзакций. Единственная задача этого листинга — продемонстрировать взаимную блокировку; не ориентируйтесь на него, когда пишете реальный код.

https://t.me/javalib

Вот как выглядит метод для перевода денег между двумя объектами FSOAccount:

Листинг 5.2. Пример взаимной блокировки

```
public synchronized boolean transferTo(FSOAccount other, int amount) {
    // Проверяет, что amount > 0, если нет — возбуждает исключение
    // Имитируем некоторые другие проверки, которые должны здесь быть
    try {
        Thread.sleep(10);
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
    }

    if (balance >= amount) {
        balance = balance - amount;
        other.deposit(amount);
        return true;
    }

    return false;
}
```

А теперь добавим немного конкурентности в основной класс:

```
public class FSOMain {
    private static final int MAX_TRANSFERS = 1_000;

    public static void main(String[] args) throws InterruptedException {
        FSOAccount a = new FSOAccount(10_000);
        FSOAccount b = new FSOAccount(10_000);
        Thread tA = new Thread(() -> {
            for (int i = 0; i < MAX_TRANSFERS; i = i + 1) {
                boolean ok = a.transferTo(b, 1);
                if (!ok) {
                    System.out.println("Поток A отказал на "+  i);
                }
            }
        });
        Thread tB = new Thread(() -> {
            for(inti=0;i<MAX_TRANSFERS;i=i+1){
                boolean ok = b.transferTo(a, 1);
                if (!ok) {
                    System.out.println("Поток B отказал на "+ i);
                }
            }
        });
        tA.start();
        tB.start();
        tA.join();
        tB.join();
        System.out.println("Конец: "+ a.getBalance()+":"+ b.getBalance());
    }
}
```

https://t.me/javalib

На первый взгляд код выглядит вполне разумно. В нем две транзакции, которые выполняются разными потоками. Вроде бы ничего подозрительного не происходит: просто потоки переводят деньги между двумя счетами, причем все методы синхронизированы.

Обратите внимание на небольшую паузу (sleep) в методе `transferTo()`. Она введена для того, чтобы планировщик позволил обоим потокам запуститься: тогда мы сможем пронаблюдать взаимную блокировку.

> **ПРИМЕЧАНИЕ** Пауза включена только для демонстрации, а вовсе не потому, что вам тоже нужно делать что-то подобное, разрабатывая код для банковских операций. Она всего лишь имитирует код, который был бы здесь в реальной системе, — задержку, возникающую оттого, что код обращается к базе данных или проверяет авторизацию.

Если вы запустите этот код, то скорее всего, увидите взаимную блокировку: оба потока некоторое время выполняются, но в итоге застревают. Дело в том, что каждый поток требует, чтобы другой поток освободил блокировку, которую он удерживает: без этого метод перевода денег не может продолжить работу. Ситуация наглядно изображена на рис. 5.3.

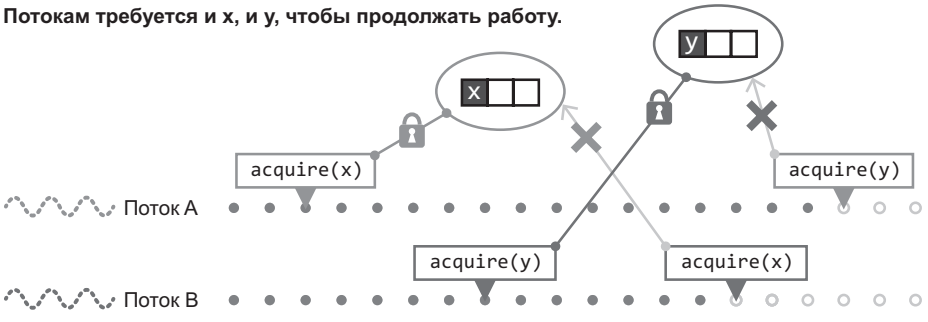


Рис. 5.3. Взаимная блокировка потоков

Другой способ пронаблюдать эту взаимную блокировку изображен на рис. 5.4, где показано представление Thread Dump в программе Mission Control из JDK. (Мы подробнее поговорим об этой программе в главе 7 и тогда же расскажем, как получить это полезное представление.)

Два потока созданы под именами `Thread-0` и `Thread-1`, и мы видим, что поток `Thread-0` заблокировал одну ссылку и находится в состоянии `BLOCKED`, ожидая возможности заблокировать другую. В соответствующем дампе для `Thread-1` выводится аналогичная конфигурация блокировок, отсюда и взаимная блокировка.

> **ПРИМЕЧАНИЕ** С точки зрения принципа полностью синхронизированных объектов взаимная блокировка возникает из-за того, что нарушено правило ограничений по времени. Когда в коде вызывается `other.deposit()`, мы не можем контролировать, как долго будет выполняться код, потому что модель памяти Java не дает гарантий относительно того, когда будет освобожден заблокированный монитор.

https://t.me/javalib

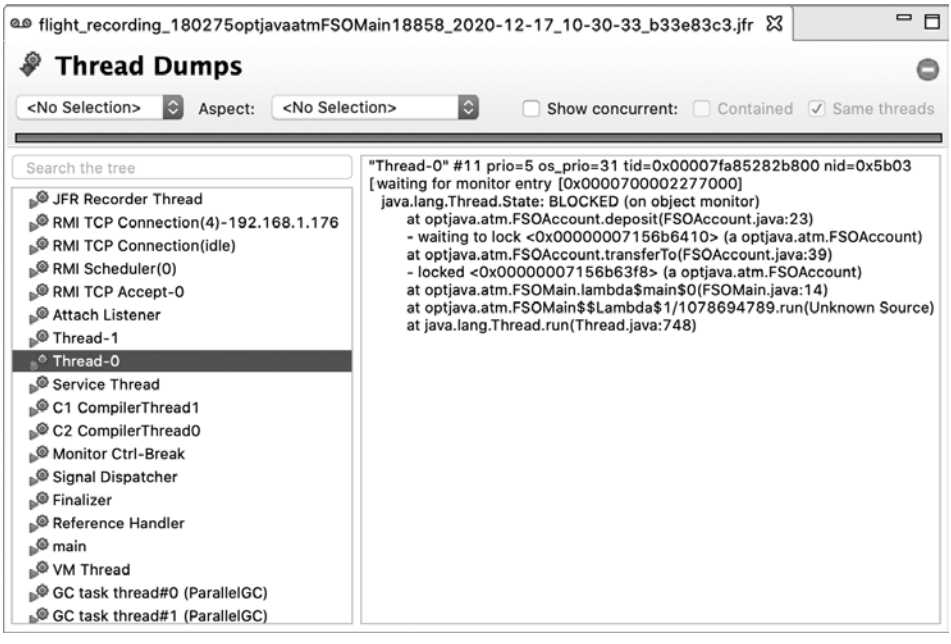


Рис. 5.4. Взаимная блокировка потоков

Одно из возможных решений проблемы взаимной блокировки — сделать так, чтобы каждый поток всегда устанавливал блокировки в одном и том же порядке. В нашем примере первый поток устанавливает их в порядке A, B, а второй — в порядке B, A. Если бы оба потока запрашивали установку блокировок в порядке A, B, то взаимной блокировки удалось бы избежать, потому что второй поток вообще не смог бы выполняться, пока первый не завершится и не освободит свои блокировки. Позднее в этой главе мы покажем, как простым способом добиться, чтобы все блокировки устанавливались в одном порядке, а также как проверить, что этот порядок действительно соблюдается.

Теперь вернемся к вопросу, заданному ранее: почему в Java для обозначения критической секции используется ключевое слово `synchronized`? Это приведет нас к обсуждению ключевого слова `volatile`.

5.3.5. Почему synchronized?

Простейшая концептуальная модель конкурентного программирования выглядит как разделение процессорного времени: потоки загружаются и выгружаются на одном ядре. Эта классическая модель изображена в левой части рис. 5.5.

Однако уже много лет эта схема не соответствует тому, как на самом деле работает аппаратное обеспечение. Двадцать лет назад программист мог работать годами и ни разу не столкнуться с системой, оснащенной более чем одним, максимум двумя ядрами. С тех пор ситуация изменилась.

https://t.me/javalib

В наши дни даже мобильные телефоны комплектуются процессорами с несколькими ядрами, так что ментальную модель тоже пора менять. Теперь стоит представлять себе несколько потоков, которые выполняются на разных ядрах в один и тот же физический момент времени (и, возможно, манипулируют общими данными). Эта ситуация изображена в правой части рис. 5.5. Из соображений эффективности у каждого из одновременно выполняемых потоков может быть собственная кэшированная копия данных, с которыми он работает.

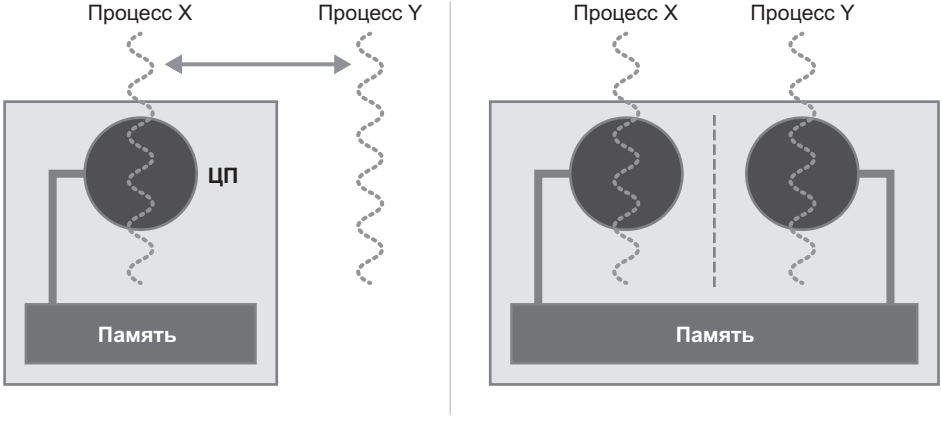


Рис. 5.5. Одноядерная (слева) и многоядерная (справа) модель конкурентного выполнения и потоков

> **ПРИМЕЧАНИЕ** Мы все-таки продолжим представлять теоретические модели выполнения, в которых у нашего гипотетического компьютера только одно вычислительное ядро. Это делается исключительно для того, чтобы продемонстрировать, что недетерминированные проблемы конкурентного выполнения, которые мы обсуждаем, обусловлены самой природой конкурентности и не связаны с какими-либо конкретными свойствами оборудования.

С учетом сказанного вернемся к вопросу о том, из каких соображений выбиралось ключевое слово для обозначения блокируемой секции или метода.

Ранее мы задавали вопрос: что же конкретно синхронизируется в коде из листинга 5.1? Ответ таков: *между разными потоками синхронизируется представление блокируемого объекта в памяти*. Иными словами: после того, как синхронизированный метод (или блок) завершается, любые изменения, внесенные в блокируемый объект, актуализируются в основной памяти до снятия блокировки, как показано на рис. 5.6.

Кроме того, после того как при входе в синхронизированный блок устанавливается блокировка, любые изменения в блокируемом объекте читаются *из* основной памяти, так что поток с блокировкой синхронизируется с представлением объекта в основной памяти, прежде чем код в заблокированной секции начнет выполняться.

https://t.me/javalib

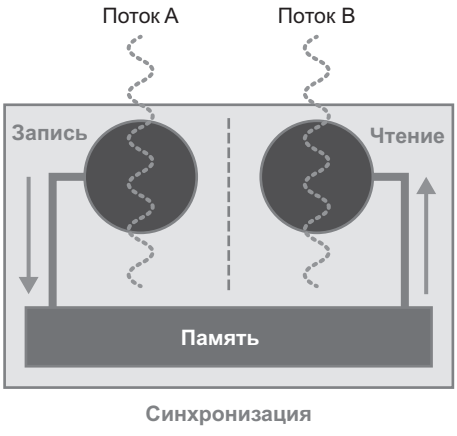


Рис. 5.6. Изменения в объекте распространяются между потоками через основную память

5.3.6. Ключевое слово volatile

Ключевое слово `volatile` («нестабильный») существует в Java с незапамятных времен (Java 1.0) и используется для несложных конкурентных манипуляций с полями объектов, включая примитивы. Нестабильные поля подчиняются следующим правилам:

- Значение, видимое потоку, всегда заново читается из основной памяти перед использованием.
- Любое значение, записанное потоком, всегда выгружается в основную память до того, как завершится инструкция байт-кода.

Иногда это поведение описывается как «крошечный синхронизированный блок» вокруг отдельной операции, но такое описание только сбивает с толку, потому что `volatile` не использует никакой блокировки. Суть синхронизации заключается в том, чтобы применять взаимоисключающую блокировку для объекта и тем самым гарантировать, что только один поток сможет выполнять синхронизированный метод для этого объекта. Синхронизированные методы могут осуществлять много операций чтения и записи с объектом, но все они будут выполняться как единое целое (с точки зрения других потоков), потому что результаты выполнения метода с объектом не будут видны до тех пор, пока мы не выйдем из метода и объект не выгрузится обратно в основную память.

Принципиальная особенность ключевого слова `volatile` состоит в том, что оно позволяет выполнить только *одну* операцию с областью памяти, и результаты этой операции немедленно выгружаются в память. Это может быть одна операция чтения *или* одна операция записи, но не более того. Мы уже видели эти две разновидности операций на рис. 5.6.

Переменную стоит объявлять нестабильной, только если запись в переменную не зависит от ее текущего состояния (состояния чтения). Это следствие того, что `volatile` гарантирует выполнение только одной операции.

https://t.me/javalib

Например, операторы ++ и -- небезопасны для использования с volatile, потому что они эквивалентны v = v + 1 и v = v – 1. Инкремент — это классический пример *обновления, зависящего от состояния*.

Для ситуаций, в которых важно текущее состояние, всегда следует задействовать блокировку, чтобы гарантировать полную безопасность. Таким образом, volatile позволяет писать упрощенный код в некоторых случаях, но за это приходится расплачиваться лишними обновлениями памяти при каждом обращении. Также обратите внимание на то, что поскольку механизм volatile не использует блокировку, для нестабильных полей взаимная блокировка невозможна; она характерна только для синхронизации. Позднее в этой главе мы поговорим о нестабильности подробнее и представим другие применения volatile.

5.3.7. Состояния и методы потока

Объект Java java.lang.Thread находится в куче и содержит метаданные потока операционной системы, который либо существует, либо существовал ранее, либо может потенциально существовать в будущем.

Java определяет для объекта потока следующие состояния, которые соответствуют состояниям потока ОС в основных операционных системах. Они тесно связаны с моделью состояний, изображенной на рис. 5.2:

- NEW — объект Thread был создан, а фактический поток ОС — еще нет.
- RUNNABLE — поток доступен для выполнения, ОС отвечает за его планирование.
- BLOCKED — поток не выполняется; он должен установить блокировку или находится в системном вызове.
- WAITING — поток не выполняется; он вызвал Object.wait() или Thread.join().
- TIMED_WAITING — поток не выполняется; он вызвал Thread.sleep().
- TERMINATED — поток не выполняется; он завершил выполнение.

Все потоки начинают с состояния NEW и завершаются в состоянии TERMINATED независимо от того, произошел ли нормальный выход из метода run() или было возбуждено исключение.

> **ПРИМЕЧАНИЕ** Если поток находится в состоянии RUNNABLE, модель состояний потоков Java не различает, выполняется ли он физически в этот момент времени или ожидает (в очереди выполнения).

Фактическое создание потока осуществляется методом start(), а он передает управление низкоуровневому коду; уже этот код непосредственно выполняет системные вызовы (например, clone() в Linux), которые создают поток и запускают код в его методе run().

Стандартный API потоков в Java делится на три группы методов. Вместо того чтобы воспроизводить множество шаблонных Javadoc-описаний каждого метода,

https://t.me/javalib

мы просто перечислим их, а за дополнительными подробностями вы сможете обратиться к документации API.

Первую группу составляют методы для чтения метаданных, которые относятся к потоку:

- `getId()`
- `getName()`
- `getState()`
- `getPriority()`
- `isAlive()`
- `isDaemon()`
- `isInterrupted()`

Некоторые из этих метаданных (например, идентификатор потока, полученный от `getId()`) остаются фиксированными на протяжении всего времени существования потока. Другие данные (например, состояние потока и состояние прерывания) естественным образом изменяются в процессе выполнения, а третьи (например, имя потока и статус демона) могут задаваться программистом. Для этого используются методы второй группы:

- `setDaemon()`
- `setName()`
- `setPriority()`
- `setUncaughtExceptionHandler()`

Вам обычно стоит настроить все необходимые свойства потока до того, как запускать его.

Наконец, следующий набор управляющих методов служит для того, чтобы запускать новые потоки и взаимодействовать с другими выполняемыми потоками:

- `start()`
- `interrupt()`
- `join()`

Обратите внимание, что `Thread.sleep()` не входит в этот список, потому что это статический метод, который влияет только на текущий поток.

> **ПРИМЕЧАНИЕ** Некоторые методы потоков с тайм-аутами (например, `Thread.join()` с параметром времени ожидания) могут привести к тому, что поток перейдет в состояние `TIMED_WAITING` вместо `WAITING`.

Следующий пример показывает, как использовать методы потока в типичном жизненном цикле простого многопоточного приложения:

https://t.me/javalib

```
Runnable r = () -> {
    var start = System.currentTimeMillis();
    try {
        Thread.sleep(1000);
    } catch (InterruptedException e) {
        e.printStackTrace();
    }
    var thisThread = Thread.currentThread();
    System.out.println(thisThread.getName() +
        " был приостановлен на "+ (System.currentTimeMillis() - start));
};

var t = new Thread(r);
t.setName("Worker");
t.start();
Thread.sleep(100);
t.join();
System.out.println("Выход из приложения");
```

Создаются метаданные потока

Операционная система создает реальный поток

Главный поток приостанавливается и ожидает, пока рабочий поток завершится, после чего главный поток продолжит работу

Здесь нет ничего сложного: главный поток создает рабочий поток, запускает его, а затем ожидает не менее 100 мс, чтобы дать планировщику возможность отработать. После этого вызывается метод `join()`, который приостанавливает главный поток до того момента, когда произойдет выход из рабочего потока. Тем временем рабочий поток выходит из `sleep()`, активизируется и выводит сообщение.

> **ПРИМЕЧАНИЕ** Скорее всего, время, потраченное на `sleep()`, не будет равно в точности 1000 мс. Планировщик операционной системы не детерминирован: он может разве что гарантировать, что операционная система постарается обеспечить, чтобы поток был поставлен на паузу на запрошенный промежуток времени, если он не будет активизирован. Однако в многопоточном программировании часто возникают непредвиденные обстоятельства, как мы увидим в следующем разделе.

Прерывание потоков

При работе с потоками достаточно часто возникает необходимость безопасно прервать работу, которую выполняет поток, и объект `Thread` предоставляет соответствующие методы. Тем не менее они могут вести себя не так, как вы ожидали поначалу. Давайте запустим следующий код, который создает поток, нагружает его работой, а затем пытается прервать его:

```
var t = new Thread(() -> { while (true); });
t.start();

t.interrupt();
t.join();
```

Создает и запускает новый поток, который будет выполняться бесконечно

Заставляет поток прервать себя (то есть прекратить выполняться)

Главный поток ожидает, пока запущенный поток завершится

Если вы попытаетесь запустить этот код, то вас, возможно, удивит, что вызов `join()` блокируется навсегда. Дело в том, что прерывание потока требует

https://t.me/javalib

содействия с его стороны: методы, которые вызываются в потоке, должны явно проверять состояние прерывания и реагировать на него, а наш наивный цикл `while` не выполняет таких проверок. Чтобы исправить ситуацию, можно включить проверку в цикл:

```
var t = new Thread(() -> { while (!Thread.interrupted()); });
t.start();

t.interrupt();
t.join();
```

Проверяет состояние прерывания текущего потока вместо безусловного продолжения по true

Теперь цикл будет прерываться по запросу, а вызов `join()` перестанет блокироваться навсегда.

Обычная практика состоит в том, чтобы методы JDK, в которых происходит блокирование — будь то ввод/вывод, ожидание блокировки или другие факторы, — проверяли состояние прерывания потока. По соглашению эти методы возбуждают проверяемое исключение `InterruptedException`. В частности, это объясняет, почему `Thread.sleep()` требует включать `InterruptedException` в сигнатуру метода или обрабатывать его.

Изменим наш пример из предыдущего раздела, чтобы понять, как ведет себя `Thread.sleep()` при прерывании:

```
Runnable r = () -> {
    var start = System.currentTimeMillis();
    try {
        Thread.sleep(1000);
    } catch (InterruptedException e) {
        e.printStackTrace();
    }
    var thisThread = Thread.currentThread();
    System.out.println(thisThread.getName() +
        " был приостановлен на "+ (System.currentTimeMillis() - start));
    if (thisThread.isInterrupted()) {
        System.out.println("Поток "+ thisThread.getName() +" прерван");
    }
};

var t = new Thread(r);
t.setName("Worker");
t.start();
Thread.sleep(100);
t.interrupt();
t.join();
System.out.println("Выход из приложения");
```

Объект Runnable должен обработать проверяемое исключение InterruptedException. Когда происходит прерывание, он выводит трассировку стека, и выполнение продолжается с этой точки

Создает рабочий поток

Главный поток прерывает рабочий поток и активизирует его

Результат выполнения этого кода выглядит примерно так:

```
java.lang.InterruptedException: sleep interrupted
    at java.base/java.lang.Thread.sleep(Native Method)
    at examples.LifecycleWithInterrupt.lambda$main$0
     (LifecycleWithInterrupt.java:9)
```

https://t.me/javalib

```
    at java.base/java.lang.Thread.run(Thread.java:832)
Worker был приостановлен на 101
Выход из приложения
```

Внимательно присмотревшись, вы заметите, что сообщение «Поток Worker прерван» не выводится. Это отражает важный факт, относящийся к обработке прерываний: проверка состояния прерывания потока на самом деле сбрасывает это состояние. Код, который возбудил стандартное исключение `InterruptedException`, сбросил состояние прерывания, потому что оно считается «обработанным» после того, как исключение возбуждено.

> **ПРИМЕЧАНИЕ** Проверить состояние прерывания можно двумя методами: статическим методом `Thread.interrupted()`, который проверяет текущий поток, и методом уровня экземпляра `isInterrupted()` объекта потока. Статическая версия сбрасывает состояние после проверки; именно ее принято использовать перед тем, как выдать `InterruptedException`. С другой стороны, метод экземпляра не изменяет состояние.

Если вы хотите сохранить информацию о том, что поток был прерван, это придется сделать самостоятельно. В нашем простом примере, где состояние понадобится позднее в коде потока, может подойти примерно такое решение:

```
Runnable r = () -> {
    var start = System.currentTimeMillis();
    var wasInterrupted = false;
    try {
        Thread.sleep(1000);
    } catch (InterruptedException e) {
        wasInterrupted = true;
        e.printStackTrace();
    }
    var thisThread = Thread.currentThread();
    System.out.println(thisThread.getName() +
        " был приостановлен на "+ (System.currentTimeMillis() - start));
    if (wasInterrupted) {
        System.out.println("Поток "+ thisThread.getName() +" прерван");
    }
};
var t = new Thread(r);
t.setName("Worker");
t.start();
Thread.sleep(100);
t.interrupt();
t.join();
System.out.println("Выход из приложения");
```

Настраивает состояние, чтобы регистрировать возможное прерывание

Регистрирует прерывание

В более сложных ситуациях может понадобиться повторно выдать `InterruptedException` для вызывающей стороны, возбудить нестандартное исключение, выполнить особую логику или даже восстановить состояние прерывания в соответствующем потоке. Все это осуществимо и выражается в коде в зависимости от ваших конкретных потребностей.

https://t.me/javalib

Работа с исключениями и потоками

Другая проблема многопоточного программирования — как обрабатывать исключения, которые могут возбуждаться из потока. Например, допустим, что у вас выполняется объект `Runnable` неизвестного происхождения. Если он возбудит исключение и прекратит существование, то другой код может не узнать об этом. К счастью, API потоков позволяет добавлять в поток обработчики неперехваченных исключений перед тем, как запускать его, как в следующем примере:

```
var badThread = new Thread(() -> {
    throw new UnsupportedOperationException(); });

// Назначить имя перед запуском потока
badThread.setName("An Exceptional Thread");

// Назначить обработчик
badThread.setUncaughtExceptionHandler((t, e) -> {
    System.err.printf("Поток %d '%s' возбудил исключение " +
                      "%s на строке %d из %s",
            t.getId(),
            t.getName(),
            e.toString(),
            e.getStackTrace()[0].getLineNumber(),
            e.getStackTrace()[0].getFileName()); });

badThread.start();
```

Обработчик является экземпляром `UncaughtExceptionHandler` — функционального интерфейса, который определяется так:

```
public interface UncaughtExceptionHandler {
    void uncaughtException(Thread t, Throwable e);
}
```

Этот метод предоставляет простой обратный вызов, который позволяет коду управления потоком делать что-то в зависимости от наблюдаемого исключения: например, пул потоков может перезапустить поток, который завершился таким образом, чтобы поддерживать размер пула.

> **ПРИМЕЧАНИЕ** JVM игнорирует любые исключения, которые возбуждает `uncaughtException()`.

Прежде чем двигаться дальше, стоит упомянуть другие управляющие методы `Thread`, которые сейчас считаются устаревшими и не должны использоваться при разработке приложений.

Устаревшие методы потоков

Java стал первым массовым языком со встроенной поддержкой многопоточного программирования. Однако у роли первопроходца была и обратная сторона: многие проблемы, присущие конкурентному программированию, впервые обнаружили именно программисты, работавшие на Java.

https://t.me/javalib

Как ни прискорбно, некоторые методы исходного API потоков были попросту небезопасны и непригодны для использования, прежде всего `Thread.stop()`. Этот метод в принципе нельзя использовать безопасно: он уничтожает другой поток без предупреждения и не дает ему возможности позаботиться о том, чтобы с заблокированными объектами можно было нормально работать.

Метод `stop()` был признан нежелательным вскоре после того, как его стали активно использовать в раннюю эпоху Java, так как, чтобы остановить другой поток, требовалось внедрить исключение в его выполнение. Однако невозможно точно знать, *в какой точке* выполнения это произойдет. Возможно, поток будет уничтожен посреди блока `finally` и программа останется в некорректном состоянии, потому что разработчик рассчитывал, что этот блок будет полностью выполнен.

Этот механизм работал так, что в уничтожаемом потоке возбуждалось непроверяемое исключение `ThreadDeath`. Код не может защититься от такого исключения при помощи блока `try` (это не более эффективно, чем защищаться от `OutOfMemoryError`), так что исключение немедленно начинает разбирать стек уничтоженного потока, а со всех мониторов снимается блокировка. В результате потенциально поврежденные объекты немедленно становятся видимыми для других потоков, так что метод `stop()` попросту небезопасен.

Кроме хорошо известных неприятностей со `stop()`, серьезные проблемы возникали и с другими методами. Например, `suspend()` не освобождает мониторы, так что если поток пытается обратиться к синхронизированному коду, который заблокирован приостановленным потоком, то обращающийся поток будет заблокирован навсегда, если только приостановленный поток снова не активизируется. Это создает серьезную угрозу для живости кода, так что методы `suspend()` и `resume()` точно не стоит использовать. Метод `destroy()` так и не был реализован, но если бы это произошло, то и он страдал бы от тех же проблем.

> **ПРИМЕЧАНИЕ** Эти опасные методы потоков были объявлены устаревшими еще в Java 1.2 — более 20 лет назад, а недавно были помечены для удаления (что станет критическим изменением — просто чтобы вы представляли, насколько серьезно разработчики платформы к этому относятся).

Чтобы надежно управлять одними потоками из других, можно применять решение на основе паттерна «Нестабильное завершение», который будет представлен позднее в этой главе. А сейчас мы рассмотрим один из самых полезных приемов обработки данных, которые должны безопасно находиться в общем доступе при программировании в конкурентном стиле.

5.3.8. Неизменяемость

Использование неизменяемых объектов может принести неоценимую пользу. Такие объекты либо не имеют состояния, либо состоят только из конечных полей (которые, соответственно, должны заполняться в конструкторах объектов).

https://t.me/javalib

Они всегда остаются безопасными и живыми, потому что их состояние нельзя изменить, а значит, они не могут перейти в противоречивое состояние.

С другой стороны, все значения, которые необходимы, чтобы инициализировать конкретный объект, должны передаваться конструктору. Это может привести к громоздким вызовам конструктора со множеством параметров, поэтому многие программисты вместо этого используют *фабричные методы*. Новые объекты создаются не конструктором, а статическим методом класса. Конструкторы обычно объявляются частными или защищенными, поэтому единственным способом создания экземпляров становятся статические фабричные методы. Для примера рассмотрим простой класс вклада, который мог бы использоваться в банковской системе:

```
public final class Deposit {
    private final double amount;
    private final LocalDate date;
    private final Account payee;

    private Deposit(double amount, LocalDate date, Account payee) {
        this.amount = amount;
        this.date = date;
        this.payee = payee;
    }

    public static Deposit of(double amount, LocalDate date, Account payee) {
        return new Deposit(amount, date, payee);
    }

    public static Deposit of(double amount, Account payee) {
        return new Deposit(amount, LocalDate.now(), payee);
    }
```

Здесь объявляются поля класса, частный конструктор и два фабричных метода, один из которых — вспомогательный метод для создания вкладов за текущий день. Затем идут методы доступа для полей:

```
    public double amount() {
        return amount;
    }

    public LocalDate date() {
        return date;
    }

    public Account payee() {
        return payee;
    }
```

Обратите внимание, что в нашем примере имена методов чтения и записи совпадают с именами соответствующих полей. Существует альтернативный стиль именования, в котором методы чтения снабжаются префиксом `get`, а методы записи (для любых неконечных полей) — префиксом `set`.

https://t.me/javalib

Очевидно, неизменяемые объекты не могут изменяться, но что произойдет, если попытаться их изменить? Например, очень часто бывает так, что взнос на депозит или какую-то еще транзакцию нельзя выполнить в конкретный день, и тогда она переносится на следующий день. В таком случае можно создать для типа метод экземпляра, который возвращает почти такой же объект, но с некоторыми измененными полями, как в следующем примере:

```
public Deposit roll() {
    // Зарегистрировать событие аудита для переноса даты
    return new Deposit(amount, date.plusDays(1), payee);
}

public Deposit amend(double newAmount) {
    // Зарегистрировать событие аудита для корректировки суммы
    return new Deposit(newAmount, date, payee);
}
```

Одна из потенциальных проблем неизменяемых объектов заключается в том, что их фабричным методам может передаваться множество параметров. Это не всегда удобно, особенно если состояние приходится собирать из нескольких источников перед тем, как создавать новый неизменяемый объект.

Чтобы решить эту проблему, можно воспользоваться паттерном «Строитель». В нем совмещаются две конструкции: статический внутренний класс, который реализует обобщенный интерфейс строителя, и частный конструктор для самого́ неизменяемого класса.

Статический внутренний класс служит строителем для неизменяемого класса, и новые экземпляры неизменяемого типа можно получить только с помощью этого статического класса. В одной из популярных реализаций класс-строитель `Builder` содержит точно такие же поля, как неизменяемый класс, но позволяет их модифицировать. Следующий листинг показывает, как можно использовать эту структуру, чтобы моделировать более сложное представление вклада.

Листинг 5.3. Неизменяемые объекты и строители

```
public static class DepositBuilder implements Builder<Deposit> {
    private double amount;
    private LocalDate date;
    private Account payee;

    public DepositBuilder amount(double amount) {
        this.amount = amount;
        return this;
    }

    public DepositBuilder date(LocalDate date) {
        this.date = date;
        return this;
    }
```

https://t.me/javalib

```
    public DepositBuilder payee(Account payee) {
        this.payee = payee;
        return this;
    }

    @Override
    public Deposit build() {
        return new Deposit(amount, date, payee);
    }
}
```

Строитель представляет собой обобщенный интерфейс верхнего уровня, который обычно определяется так:

```
public interface Builder<T> {
    T build();
}
```

Следует обратить внимание на пару моментов. Во-первых, это так называемый SAM-тип (сокращение от «Single Abstract Method», то есть «единичный абстрактный метод»), и формально он может быть целевым типом для лямбда-выражений. Однако задача строителя — создавать неизменяемые экземпляры; вся его суть заключается в том, чтобы конструировать состояние, а не представлять функцию или обратный вызов. Это значит, что хотя строитель *можно* использовать как функциональный интерфейс, на практике это абсолютно нецелесообразно.

По этой причине мы *не* помечаем интерфейс аннотацией `@FunctionalInterface`: это очередной хороший пример принципа «можно — еще не значит нужно».

Во-вторых, также заметим, что строитель не является потоково-безопасным. Его архитектура неявно подразумевает, что пользователь знает, что строителей не следует совместно использовать между потоками. Корректное использование API строителей таково: один поток использует строитель, чтобы накопить все необходимое состояние, затем создать неизменяемый объект, а уже этот объект *могут* совместно использовать другие потоки.

> **ПРИМЕЧАНИЕ** Если вы чувствуете потребность в том, чтобы строитель совместно использовался разными потоками, — сделайте шаг назад, пересмотрите свое решение и подумайте, не нуждается ли ваша предметная область в рефакторинге.

Неизменяемость — очень распространенный паттерн (не только в Java, но и в других языках, особенно функциональных), и он широко применяется на практике.

И последнее замечание по поводу неизменяемых объектов: ключевое слово `final` применяется только к объекту, на который непосредственно указывает ссылка. Как видно из рис. 5.7, ссылку на основной объект невозможно обновить так, чтобы она указывала на объект 3, однако внутри основного объекта ссылку на объект 1 можно переназначить на объект 2. Это можно сформулировать

https://t.me/javalib

иначе: конечная ссылка может указывать на объект, который содержит неконечные поля. Иногда это называется *поверхностной неизменяемостью* (shallow immutability).

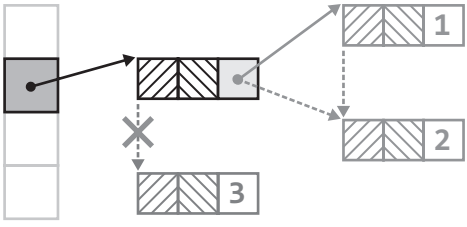


Рис. 5.7. Неизменяемые значения и ссылки

Рассмотрим эту же ситуацию под другим углом: ничто не мешает вам использовать запись:

```
final var numbers = new LinkedList<Integer>();
```

В этой инструкции ссылка `numbers` и целочисленные объекты, которые содержатся в списке, неизменяемы. Однако сам объект списка остается изменяемым, потому что в список можно добавлять целочисленные объекты, удалять и заменять их.

Неизменяемость — очень мощный механизм, и вам стоит применять его всюду, где от него есть польза. Однако иногда эффективная разработка только с неизменяемыми объектами попросту невозможна, потому что каждое изменение в состоянии объекта требует создавать новый объект. Таким образом, иногда мы просто вынуждены иметь дело с изменяемыми объектами.

В следующем разделе мы обсудим некоторые подробности модели памяти Java (JMM, Java Memory Model), которые часто страдают от превратных толкований. Многие Java-программисты имеют представление о JMM и программируют в рамках этого представления, хотя никогда формально не изучали эту модель. Если это относится к вам, то дальнейший материал углубит ваше понимание и поможет структурировать знания. JMM — достаточно сложная тема, и ее можно пропустить, если вам не терпится перейти к следующей главе.

5.4. МОДЕЛЬ ПАМЯТИ JAVA (JMM)

Модель JMM вводится в разделе 17.4 спецификации языка Java (JLS). Она является формальной частью спецификации и описывает JMM в терминах операций синхронизации и некоторых довольно отвлеченных математических концепций, например *частичного порядка* операций.

Такое описание идеально подходит теоретикам языка и создателям реализации спецификации Java (разработчикам компилятора и JVM), но оно не очень

https://t.me/javalib

практично для разработчиков приложений, которым важно глубоко разбираться в том, как выполняется их многопоточный код.

Здесь мы не будем воспроизводить формальные подробности, а перечислим важнейшие правила, опираясь на два основных понятия: отношения синхронизации и предшествования между блоками кода.

- *Предшествование*: один блок кода полностью завершается, прежде чем другой может начаться.
- *Синхронизация*: операция синхронизирует свое представление объекта с основной памятью, прежде чем продолжить работу.

Если вы изучали теорию объектно-ориентированного программирования, то вам, вероятно, встречались отношения наследования («*содержит*», «*has-A*») и композиции («*является*», «*is-A*»), с помощью которых описываются структурные элементы объектно-ориентированных решений. Некоторым разработчикам удобно представлять себе предшествование и синхронизацию как такие же базовые концептуальные структурные элементы, но не для ООП, а для конкурентного выполнения в Java. Однако важно подчеркнуть, что эти два набора понятий напрямую не связаны друг с другом. На рис. 5.8 изображен пример нестабильной записи, которая синхронизируется с последующим обращением для чтения (для `println()`).

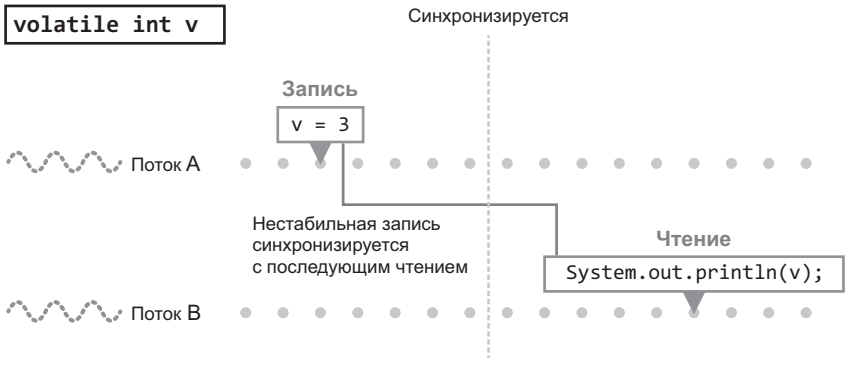


Рис. 5.8. Пример отношения синхронизации

JMM работает по таким правилам:

- Операция освобождения блокировки для монитора синхронизируется с последующими операциями блокировки.
- Запись в нестабильную переменную синхронизируется с последующими операциями чтения из этой переменной.
- Если действие *A* синхронизируется с действием *B*, то *A* предшествует *B*.
- Если *A* следует перед *B* в порядке выполнения программы внутри потока, то *A* предшествует *B*.

https://t.me/javalib

Первые два правила в целом утверждают, что блокировки сначала освобождаются, а потом устанавливаются. Иначе говоря, блокировки, которые поток удерживает при записи, освобождаются до того, как другие операции (включая чтение) могут установить блокировки на те же ресурсы. Например, правила гарантируют, что если один поток запишет значение в нестабильную переменную, то любой поток, который позднее читает эту переменную, увидит именно то значение, которое было записано (при условии, что не было других операций записи).

Перечислим некоторые дополнительные правила, которые скорее отвечают за то, чтобы поведение JMM выглядело разумным:

- Завершение конструктора предшествует началу выполнения финализатора этого объекта. (Объект должен быть полностью сконструирован, прежде чем его можно будет финализировать.)
- Действие, запускающее поток, синхронизируется с первым действием нового потока.
- `Thread.join()` синхронизируется с последним действием (и всеми остальными действиями) в присоединяемом потоке.
- Если *X* предшествует *Y*, а *Y* предшествует *Z*, то *X* предшествует *Z* (транзитивность).

Эти простые правила полностью определяют платформенное представление того, как работает память и синхронизация. Рисунок 5.9 иллюстрирует правило транзитивности.

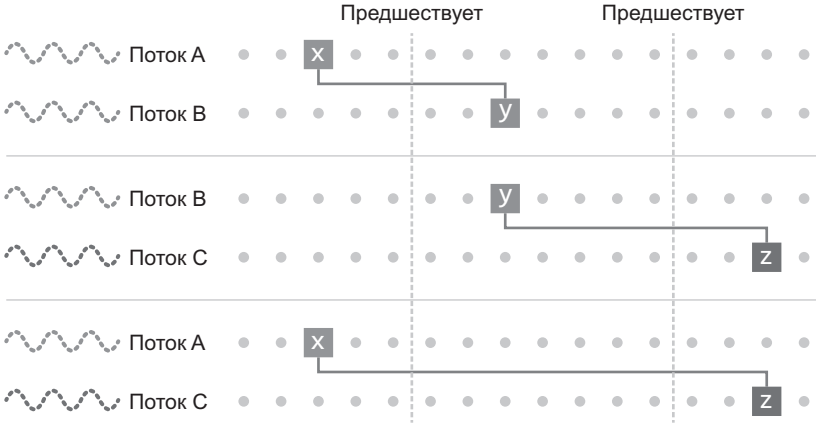


Рис. 5.9. Транзитивность отношения предшествования

ПРИМЕЧАНИЕ Эти правила описывают минимальные гарантии, которые предоставляет JMM. Реальные JVM могут работать намного лучше, чем предполагают эти гарантии. Это легко становится ловушкой для разработчиков, потому что ложное чувство безопасности, которое вселяет та или иная JVM, может обернуться индивидуальной технической особенностью, скрывающей ошибку конкурентного выполнения.

https://t.me/javalib

По этим минимальным гарантиям легко увидеть, почему неизменяемость считается такой важной концепцией в параллельном программировании на Java. Если объект не может изменяться, то нет никаких проблем с тем, чтобы изменения были видны всем потокам.

5.5. КОНКУРЕНТНОСТЬ В БАЙТ-КОДЕ

Попробуем взглянуть на конкурентное выполнение через призму классического примера — банковского счета. Допустим, счет клиента выглядит так, как показано ниже, а операции снятия и внесения средств осуществляются вызовом методов. Мы представили как синхронизированные, так и несинхронизированные реализации важнейших методов:

```
public class Account {
    private double balance;

    public Account(int openingBalance) {
        balance = openingBalance;
    }

    public boolean rawWithdraw(int amount) {
        // Проверить, что amount > 0, если нет - возбудить исключение
        if (balance >= amount) {
            balance = balance - amount;
            return true;
        }
        return false;
    }

    public void rawDeposit(int amount) {
        // Проверить, что amount > 0, если нет - возбудить исключение
        balance = balance + amount;
    }

    public double getRawBalance() {
        return balance;
    }

    public boolean safeWithdraw(final int amount) {
        // Проверить, что amount > 0, если нет - возбудить исключение
        synchronized (this) {
            if (balance >= amount) {
                balance = balance - amount;
                return true;
            }
        }
        return false;
    }

    public void safeDeposit(final int amount) {
        // Проверить, что amount > 0, если нет - возбудить исключение
        synchronized (this) {
```

https://t.me/javalib

```
            balance = balance + amount;
        }
    }

    public double getSafeBalance() {
        synchronized (this) {
            return balance;
        }
    }
}
```

Этот набор методов позволит нам исследовать распространенные проблемы, связанные с конкурентностью в Java.

> **ПРИМЕЧАНИЕ** Мы умышленно использовали блоковую форму синхронизации вместо модификатора метода `synchronized` — позднее в этой главе мы объясним, почему так сделано.

Также допустим, что класс содержит вспомогательные методы с двумя аргументами, которые выглядят примерно так:

```
    public boolean withdraw(int amount, boolean safe) {
        if (safe) {
            return safeWithdraw(amount);
        } else {
        return rawWithdraw(amount);
        }
    }
```

Начнем с одной из фундаментальных проблем, встречающихся в многопоточных системах, которая требует ввести определенный механизм защиты.

5.5.1. Потерянное обновление

Чтобы продемонстрировать распространенную проблему (или антипаттерн), которая известна под названием «Потерянное обновление» (Lost Update), рассмотрим байт-код метода `rawDeposit()`:

```
public void rawDeposit(int);
    Code:
        0: aload_0
        1: aload_0
        2: getfield #2 // Field balance:D   <-- Прибавляет внесенную сумму
        5: iload_1
        6: i2d
        7: dadd   <-- Считывает баланс из объекта
        8: putfield #2 // Field balance:D   <-- Записывает обновленный баланс в объект
       11: return
```

Пусть есть два программных потока, *A* и *B*. С их помощью мы проиллюстрируем, что происходит, если пытаться одновременно выполнить две операции внесения средств на один счет. Пометив инструкции префиксом потока, мы видим, как те

https://t.me/javalib

или иные инструкции байт-кода выполняются в разных потоках, но при этом влияют на один и тот же объект.

> **ПРИМЕЧАНИЕ** Не забывайте, что у некоторых инструкций есть параметры, которые следуют за ними в потоке байт-кода, поэтому в нумерации инструкций возникают пропуски.

Потерянное обновление — это ситуация, в которой из-за недетерминированного планирования потоков приложения в байт-коде появляется примерно такая последовательность операций чтения и записи:

```
A0: aload_0
    A1: aload_0
    A2: getfield #2   // Field balance:D  <-- Поток A считывает значение из balance
    A5: iload_1
    A6: i2d
    A7: dadd

// .... Переключение контекста A →  B

        B0: aload_0
        B1: aload_0
        B2: getfield #2 // Field balance:D  <-- Поток B считывает из balance то же значение, что ранее считал A
        B5: iload_1
        B6: i2d
        B7: dadd
        B8: putfield #2 // Field balance:D  <-- Поток B записывает новое значение в balance
       B11: return

// .... Переключение контекста B → A

    A8: putfield #2   // Field balance:D  <-- Поток A перезаписывает balance. Значение, которое ранее записал поток B, теряется
   A11: return
```

Каждый поток вычисляет обновленное значение `balance` с помощью стека вычислений. Код операции `dadd` — точка, в которой обновленное значение `balance` помещается в стек, однако не забывайте, что у каждого вызова метода есть свой, частный стек вычислений. Таким образом, в точке `B7` в этой последовательности существуют *две* копии обновленного значения: в стеках вычислений потока *A* и потока *B*. Затем в точках `B8` и `A8` выполняются две операции `putfield`, но `A8` перезаписывает значение, которое было записано в `B8`. Возникает ситуация, в которой обе операции внесения средств вроде бы завершились успешно, однако в итоге баланс счета отражает только одну из них.

Система регистрирует, что на баланс внесены средства, но деньги исчезают со счета, потому что поле `balance` считывается дважды (с помощью `getfield`), а затем записывается и перезаписывается (двумя операциями `putfield`). Для примера рассмотрим такой код:

```
Account acc = new Account(0);
Thread tA = new Thread(() -> acc.rawDeposit(70));
```

https://t.me/javalib

```
Thread tB = new Thread(() -> acc.rawDeposit(50));
tA.start();
tB.start();
tA.join();
tB.join();

System.out.println(acc.getRawBalance());
```

Итоговый баланс может оказаться как 50, так и 70, но при этом оба потока «успешно» зачислили деньги на счет. В коде переводилась сумма 120, однако ее часть потерялась — вот классический пример некорректного многопоточного кода.

Обратите внимание, что в этом примере рассматривается очень простой код, в котором, возможно, не проявится весь диапазон последствий недетерминированности. Не стоит обольщаться: если включить такой код в большую программу, его дефекты наверняка заявят о себе. Если вы будете надеяться, что ваш код нормально сработает, потому что он очень простой, или если попытаетесь перехитрить модель конкурентного выполнения, это обязательно кончится плохо.

> **ПРИМЕЧАНИЕ** В репозитории исходного кода есть пример (`AtmLoop`), который демонстрирует этот эффект, но в нем используется класс, который мы еще не рассматривали (`AtomicInteger`), поэтому сейчас мы не будем разбирать этот пример. Однако если мы вас еще не убедили, изучите его самостоятельно.

В общем случае схемы обращений вида

```
A: getfield
B: getfield
B: putfield
A: putfield
```

или

```
A: getfield
B: getfield
A: putfield
B: putfield
```

создадут проблемы для наших объектов банковских счетов.

Вспомните, что в операционной системе потоки фактически планируются недетерминированным образом, поэтому такое чередование всегда возможно, а объекты Java существуют в куче, так что потоки манипулируют общими, изменяемыми данными.

Нам нужен механизм, который каким-то образом предотвратит эти проблемы и гарантирует, что операции всегда будут выполняться в такой последовательности:

https://t.me/javalib

```
...
A: getfield
A: putfield
...
B: getfield
B: putfield
...
```

Этим механизмом является синхронизация, и она станет нашей следующей темой.

5.5.2. Синхронизация в байт-коде

В главе 4 мы познакомились с байт-кодом Java и кратко упомянули коды операций `monitorenter` и `monitorexit`. Синхронизированный блок преобразуется в эти коды (а о синхронизированных методах мы поговорим немного позднее). Рассмотрим их в действии на примере, который уже встречался раньше (мы повторим код Java, чтобы он был у вас перед глазами):

```
public boolean safeWithdraw(final int amount) {
    // Проверить, что amount > 0, если нет - возбудить исключение
    synchronized (this) {
        if (balance >= amount) {
            balance = balance - amount;
            return true;
        }
    }
    return false;
}
```

Этот фрагмент преобразуется в 40 байт байт-кода JVM:

```
public boolean safeWithdraw(int);
    Code:
         0: aload_0
         1: dup
         2: astore_2
         3: monitorenter   <-- Начало синхронизированного блока
         4: aload_0
         5: getfield       #2 // Field balance:D
         8: iload_1
         9: i2d
        10: dcmpl
        11: iflt           29   <-- Инструкция if, которая проверяет баланс
        14: aload_0
        15: aload_0
        16: getfield       #2 // Field balance:D
        19: iload_1
        20: i2d
        21: dsub
        22: putfield       #2 // Field balance:D   <-- Записывает новое значение в поле баланса
        25: iconst_1
        26: aload_2
```

https://t.me/javalib



Внимательный читатель обратит внимание на пару странностей в этом коде. Прежде всего, присмотритесь к пути выполнения кода. Если проверка баланса завершается успехом, то байт-коды 0–28 выполняются без переходов. Если она завершается неудачей, то выполняются байт-коды 0–11, после чего происходит переход к 29–31 и переход к 39–40.

На первый взгляд байт-коды 34–38 не будут выполняться ни при каких обстоятельствах. Эта кажущаяся странность в действительности связана с обработкой исключений: некоторые инструкции байт-кода (в том числе `monitorenter`) могут возбуждать исключения, поэтому в коде должен быть путь, где они обрабатываются.

Вторая загадка — возвращаемый тип метода. В коде Java он объявлен как `boolean`, однако мы видим, что для возврата используются инструкции `ireturn` — целочисленная разновидность кода возврата. Дело в том, что соответствующих разновидностей инструкции для `byte`, `short`, `char` или `boolean` не существует. В процессе компиляции эти типы заменяются на `int`. Это одна из форм *нивелирования («стирания») типов* (type erasure), и эту особенность системы типов Java очень часто понимают неправильно (особенно применительно к обобщениям и параметрам-типам).

В целом предыдущая последовательность байт-кода сложнее, чем в несинхронизированном случае, но в ней все-таки можно разобраться: блокируемый объект помещается в стек выполнения, после чего выполняется операция `monitorenter`, которая устанавливает блокировку. Допустим, блокировка установилась успешно.

Если теперь любой другой поток попытается выполнить `monitorenter` для того же объекта, этот другой поток блокируется, и его инструкция `monitorenter` не завершится, пока поток, который удерживает блокировку на объекте, не выполнит `monitorexit` и не освободит ее. Так решается проблема «Потерянного обновления»: инструкции `monitor` обеспечивают следующий порядок:

```
...
A: monitorenter
A: getfield
```

https://t.me/javalib

```
A: putfield
A: monitorexit
...
B: monitorenter
B: getfield
B: putfield
B: monitorexit
...
```

Тем самым создается отношение предшествования между синхронизированными блоками: конец одного синхронизированного блока предшествует началу любого другого синхронизированного блока для того же объекта, и JMM гарантирует это.

Также следует заметить, что компилятор исходного кода Java заботится о том, чтобы каждый путь в коде метода, содержащего `monitorenter`, приводил к выполнению `monitorexit` до того, как метод завершится. Кроме того, во время загрузки класса верификатор файлов классов отклонит любой класс, который попытается обойти это правило.

Теперь становится понятно, на чем основано утверждение о том, что синхронизация в Java является кооперативным механизмом. Посмотрим, что происходит, когда поток A вызывает `safeWithdraw()`, а поток B вызывает `rawDeposit()`:

```
    public boolean safeWithdraw(final int amount) {
        // Проверить, что amount > 0, если нет - возбудить исключение
        synchronized (this) {
            if (balance >= amount) {
                balance = balance - amount;
                return true;
            }
        }
        return false;
    }
```

Мы снова повторили код Java, чтобы его было удобно сопоставлять с байт-кодом:

```
public boolean safeWithdraw(int);
    Code:
       0: aload_0
       1: dup
       2: astore_2
       3: monitorenter
       4: aload_0
       5: getfield        #2 // Field balance:D
       8: iload_1
       9: i2d
      10: dcmpl
      11: iflt            29
      14: aload_0
      15: aload_0
```

https://t.me/javalib

```
      16: getfield        #2 // Field balance:D
      19: iload_1
      20: i2d
      21: dsub
      22: putfield        #2 // Field balance:D
      25: iconst_1
      26: aload_2
      27: monitorexit
      28: ireturn
```

Код зачисления средств очень прост: всего одно чтение поля, арифметическая операция и запись в то же поле:

```
public void rawDeposit(int amount) {
    // Проверить, что amount > 0, если нет - возбудить исключение
    balance = balance + amount;
}
```

Байт-код выглядит сложнее, но на самом деле он тоже прост:

```
public void rawDeposit(int);
    Code:
       0: aload_0
       1: aload_0
       2: getfield        #2  // Field balance:D
       5: iload_1
       6: i2d
       7: dadd
       8: putfield        #2  // Field balance:D
      11: return
```

ПРИМЕЧАНИЕ Код `rawDeposit()` не содержит инструкций `monitor`, а без `monitorenter` блокировка проверяться не будет.

Такой порядок выполнения потоков A и B вполне возможен, как показывает следующий фрагмент:

```
      // ...
       A3: monitorenter
      // ...

      A14: aload_0
      A15: aload_0
      A16: getfield       #2  // Field balance:D

      // ... Переключение контекста A → B
      B0: aload_0
      B1: aload_0
      B2: getfield        #2  // Field balance:D
      B5: iload_1
      B6: i2d
      B7: dadd
```

https://t.me/javalib

```
B8: putfield        #2  // Field balance:D
// ... Переключение контекста B → A

B11: return
A19: iload_1
A20: i2d
A21: dsub
A22: putfield       #2  // Field balance:D
A25: iconst_1
A26: aload_2
A27: monitorexit
A28: ireturn
```

Запись в balance (через несинхронизированный метод)

Вторая запись в balance (через синхронизированный метод)

Это снова хорошо знакомое нам «Потерянное обновление», но теперь оно происходит, когда один из методов использует синхронизацию, а другой — нет. Внесенная сумма теряется — может, это и хорошо для банка, но только не для клиента. Неизбежный вывод: *все* методы должны правильно пользоваться синхронизацией, чтобы обеспечить безопасность, которую она дает.

5.5.3. Синхронизированные методы

До сих пор мы говорили о синхронизированных блоках, но как насчет синхронизированных *методов*? Можно предположить, что компилятор вставляет синтезированные байт-коды `monitor`, но на самом деле это не так. Чтобы убедиться в этом, достаточно привести безопасные методы к следующему виду:

```
public synchronized boolean safeWithdraw(final int amount) {
    // Проверить, что amount > 0, если нет - возбудить исключение
    if (balance >= amount) {
        balance = balance - amount;
        return true;
    }
    return false;

}

// ... и так далее
```

В последовательности байт-кода нет модификатора `synchronized`, однако он присутствует во флагах метода в виде `ACC_SYNCHRONIZED`. Чтобы убедиться в этом, можно перекомпилировать метод и заметить, что инструкции `monitor` исчезли из байт-кода:

```
public synchronized boolean safeWithdraw(int);
Code:
   0: aload_0
   1: getfield      #2   // Field balance:D
   4: iload_1
   5: i2d
   6: dcmpl
   7: iflt 23
```

https://t.me/javalib

```
  10: aload_0
  // ... нет инструкций monitor
```

Когда интерпретатор выполняет инструкцию `invoke`, он на одном из начальных этапов проверяет, является ли метод синхронизированным. Если да, то интерпретатор идет по другому пути в коде: сначала он пытается установить соответствующую блокировку. Если у метода нет флага `ACC_SYNCHRONIZED`, то такая проверка не выполняется.

А это значит, что, как и следовало ожидать, несинхронизированный метод может выполняться одновременно с синхронизированным, потому что только один из них проверяет блокировку.

5.5.4. Несинхронизированное чтение

Когда начинающие программисты осваивают конкурентное выполнение в Java, они часто придерживаются ошибочного принципа: «Синхронизировать нужно только методы, которые записывают данные, а чтение безопасно». Сейчас вы убедитесь, что это определенно не так.

Ложное представление о том, что операции чтения безопасны, иногда возникает из-за того, что анализируемые примеры кода слишком упрощены. Что произойдет, если добавить в наш пример небольшую комиссию банкомата — допустим, 1 % от снимаемой суммы?

```
private final double atmFeePercent = 0.01;

public boolean safeWithdraw(final int amount, final boolean withFee) {
    // Проверить, что amount > 0, если нет - возбудить исключение
    synchronized (this) {
        if (balance >= amount) {
            balance = balance - amount;
            if (withFee) {
                balance = balance - amount * atmFeePercent;
            }
            return true;
        }
    }
    return false;
}
```

Байт-код этого метода несколько усложняется:

```
public boolean safeWithdraw(int, boolean);
    Code:
       0: aload_0
       1: dup
       2: astore_3
       3: monitorenter
       4: aload_0
       5: getfield      #2   // Field balance:D
       8: iload_1
       9: i2d
```

https://t.me/javalib

```
10: dcmpl
11: iflt          49   <-- Сравнивает баланс с запрошенной суммой
14: aload_0
15: aload_0
16: getfield      #2   // Field balance:D
19: iload_1
20: i2d
21: dsub
22: putfield      #2   // Field balance:D   <-- Баланс счета обновлен
25: iload_2
26: ifeq 45
29: aload_0
30: aload_0
31: getfield      #2   // Field balance:D
34: iload_1
35: i2d
36: aload_0
37: getfield      #5   // Field atmFeePercent:D
40: dmul
41: dsub
42: putfield      #2   // Field balance:D   <-- Применяется комиссия, после чего
45: iconst_1                                   баланс обновляется снова
46: aload_3
47: monitorexit
48: ireturn
49: aload_3
50: monitorexit
51: goto          61
54: astore        4
56: aload_3
57: monitorexit
58: aload         4
60: athrow
61: iconst_0
62: ireturn
```

Теперь в коде встречаются две инструкции `putfield`, потому что `safeWithdraw()` принимает параметр `boolean`, который определяет, нужно ли снимать комиссию. Сам факт того, что в коде выполняются два раздельных обновления, создает почву для ошибки конкурентного выполнения.

Код чтения баланса очень прост:

```
public double getRawBalance();
    Code:
       0: aload_0
       1: getfield        #2   // Field balance:D
       4: dreturn
```

Однако он может чередоваться с кодом для снятия средств с комиссией:

```
A14: aload_0
A15: aload_0
A16: getfield      #2   // Field balance:D
A19: iload_1
A20: i2d
```

https://t.me/javalib

```
A21: dsub
A22: putfield       #2   // Field balance:D
A25: iload_2
A26: ifeq 45
A29: aload_0
A30: aload_0
A31: getfield       #2   // Field balance:D

// ... Переключение контекста A → B

 B0: aload_0
 B1: getfield        #2   // Field balance:D
 B4: dreturn

// ... Переключение контекста B → A

A34: iload_1
A35: i2d
A36: aload_0
A37: getfield       #5   // Field atmFeePercent:D
A40: dmul
A41: dsub
A42: putfield       #2   // Field balance:D
```

Записывается баланс, из которого вычтена снятая сумма (но без комиссии)

Баланс считывается, пока полная сумма списания еще находится в обработке

С несинхронизированным чтением появляется риск *неповторяемого чтения*, когда будет считано значение, которое не соответствует фактическому состоянию системы. Если вы знакомы с базами данных SQL, это напоминает чтение, которое происходит посреди незавершенной транзакции.

ПРИМЕЧАНИЕ Возможно, у вас появится соблазн оптимизировать программу, исходя из знания байт-кода. Тем не менее есть немало причин, из-за которых не стоит поддаваться этому искушению. Только представьте, что произойдет, когда ваш код будет готов и его станут сопровождать другие разработчики, которые не понимают ни контекста, ни последствий вроде бы безобидных изменений в коде.

Таким образом, операции чтения не стоит считать более безопасными, чем операции записи. Даже если всего один путь в коде не обеспечит правильной синхронизации, весь код в целом не будет потоково-безопасным и поэтому не сможет правильно работать в многопоточной среде. А теперь посмотрим, как в байт-коде проявляется взаимная блокировка.

5.5.5. Снова о взаимной блокировке

Допустим, мы хотим добавить в код банковской системы возможность переводить средства между счетами. Исходная версия кода может выглядеть так:

```
public boolean naiveSafeTransferTo(Account other, int amount) {
    // Проверить, что amount > 0, если нет — возбудить исключение
    synchronized (this) {
        if (balance >= amount) {
            balance = balance — amount;
```

https://t.me/javalib

```
            synchronized (other) {
                other.rawDeposit(amount);
            }
            return true;
        }
    }
    return false;
}
```

Из этого получается довольно длинный байт-код, поэтому мы сократили его, опустив уже знакомую проверку того, что баланс позволяет снять средства, а также синтезированные блоки обработки исключений.

> **ПРИМЕЧАНИЕ** Теперь существуют два объекта счета (`this` и `other`), каждый из которых защищен блокировкой. Для безопасности необходимо координировать обращения к обеим блокировкам.

Теперь в коде *две* пары инструкций `monitor`, при этом каждая пара относится к блокировке своего объекта:

```
public boolean naiveSafeTransferTo(Account, int);
    Code:
       0: aload_0
       1: dup
       2: astore_3
       3: monitorenter  <-- Устанавливает блокировку на объекте this

       // Обычный байт-код проверки баланса опущен

      14: aload_0
      15: aload_0
      16: getfield      #2   // Field balance:D
      19: iload_2
      20: i2d
      21: dsub
      22: putfield      #2   // Field balance:D
      25: aload_1
      26: dup
      27: astore        4
      29: monitorenter  <-- Устанавливает блокировку на объекте other
      30: aload_1
      31: iload_2
      32: invokevirtual #6 // Method rawDeposit:(I)V
      35: aload         4
      37: monitorexit   <-- Снимает блокировку с объекта other
      38: goto          49

      // Код обработки исключений опущен

      49: iconst_1
      50: aload_3
      51: monitorexit   <-- Снимает блокировку с объекта this
      52: ireturn

      // Код обработки исключений опущен
```

https://t.me/javalib

Представьте себе два потока, A и B, которые пытаются переводить деньги между теми же самыми двумя счетами. Предположим, что транзакции в потоках помечаются именем счета-отправителя, так что поток A пытается передать деньги от объекта A объекту B, и наоборот:

```
 A0: aload_0
 A1: dup
 A2: astore_3
 A3: monitorenter     Поток A устанавливает блокировку на объект счета A

 // Обычный байт-код проверки баланса опущен

B0: aload_0
B1: dup
B2: astore_3
B3: monitorenter     Поток B устанавливает блокировку на объект счета B

 // Обычный байт-код проверки баланса опущен

B14: aload_0
B15: aload_0
B16: getfield          #2   // Field balance:D
B19: iload_2
B20: i2d
B21: dsub
B22: putfield          #2   // Field balance:D
B25: aload_1
B26: dup
B27: astore            4    Поток B пытается установить блокировку на объект A.
B29: ...                    Это не удается, поток B блокируется
A14: aload_0
A15: aload_0
A16: getfield          #2   // Field balance:D
A19: iload_2
A20: i2d
A21: dsub
A22: putfield          #2   // Field balance:D
A25: aload_1
A26: dup
A27: astore            4    Поток A пытается установить блокировку на объект B.
A29: ...                    Это не удается, поток A блокируется
```

После того как эта последовательность выполняется, ни один поток не сможет продвинуться вперед. Что еще хуже, только поток A может снять блокировку с объекта A, и только поток B может снять блокировку с объекта B — в результате оба этих потока навсегда блокируются механизмом синхронизации, и эти вызовы методов никогда не завершатся. Рассматривая антипаттерн взаимной блокировки на уровне байт-кода, мы можем четко увидеть, вследствие чего он на самом деле возникает.

5.5.6. Снова об устранении взаимных блокировок

Чтобы решить эту проблему, как упоминалось ранее, нужно позаботиться о том, чтобы каждый поток всегда устанавливал блокировки в одном и том же порядке.

https://t.me/javalib

Для этого можно задать порядок для потоков: например, ввести уникальный номер счета и реализовать правило: «устанавливать блокировки, начиная с наименьшего номера счета».

> **ПРИМЕЧАНИЕ** С объектами, у которых нет числовых идентификаторов, придется поступать иначе, но общий принцип использования однозначного *полного порядка* продолжает действовать.

С этим способом связано больше сложности, и чтобы его полноценно реализовать, необходимо гарантировать, что идентификаторы счетов не используются повторно. Для этого можно ввести поле `static int`, в котором хранится следующий создаваемый идентификатор счета, и обновлять его только в синхронизированном методе:

```
private static int nextAccountId = 1;

private final int accountId;

private static synchronized int getAndIncrementNextAccountId() {
    int result = nextAccountId;
    nextAccountId = nextAccountId + 1;
    return result;
}

public Account(int openingBalance) {
    balance = openingBalance;
    atmFeePercent = 0.01;
    accountId = getAndIncrementNextAccountId();
}

public int getAccountId() {
    return accountId;
}
```

Синхронизировать метод `getAccountId()` не нужно, потому что поле объявлено конечным и не может измениться:

```
public boolean safeTransferTo(final Account other, final int amount) {
    // Проверить, что amount > 0, если нет - возбудить исключение
    if (accountId == other.getAccountId()) {
        // Перевод со счета на этот же счет невозможен
        return false;
    }

    if (accountId < other.getAccountId()) {
        synchronized (this) {
            if (balance >= amount) {
                balance = balance - amount;
                synchronized (other) {
                    other.rawDeposit(amount);
                }
                return true;
```

https://t.me/javalib

```
            }
        }
        return false;
    } else {
        synchronized (other) {
            synchronized (this) {
                if (balance >= amount) {
                    balance = balance - amount;
                    other.rawDeposit(amount);
                    return true;
                }
            }
        }
        return false;
    }
}
```

Конечно, итоговый код на Java получается несколько асимметричным.

ПРИМЕЧАНИЕ Мы не удерживаем блокировки дольше, чем это необходимо, и тем самым ясно показываем, в каких частях кода блокировки действительно нужны.

Из приведенного выше кода получается очень длинный байт-код, но давайте разобьем его на части. Начнем с упорядочения идентификаторов счетов:

```
// Проверки баланса и совпадения счетов опущены
13: aload_0
14: getfield          #8   // Field accountId:I
17: aload_1
18: invokevirtual     #10  // Method getAccountId:()I
21: if_icmpge         91
```

Если A < B (в данном случае это так), происходит переход к инструкции 24, как показано ниже (а в противном случае — переход к 91):

```
24: aload_0
25: dup
26: astore_3
27: monitorenter  <-- Начало synchronized (this) {
28: aload_0
29: getfield          #3    // Field balance:D
32: iload_2
33: i2d
34: dcmpl
35: iflt              77  <-- Если средств недостаточно, перейти к 77 (см. далее)
```

Рассмотрим ветвь, где на счете-отправителе достаточно средств, так что управление передается байт-коду 38, который в коде на Java соответствует началу инструкции `balance = balance - amount;`:

```
38: aload_0
39: aload_0
```

https://t.me/javalib

```
40: getfield         #3   // Field balance:D
43: iload_2
44: i2d
45: dsub
46: putfield         #3   // Field balance:D
49: aload_1
50: dup
51: astore           4
53: monitorenter  <-----  Начало synchronized (other) {
54: aload_1
55: iload_2
56: invokevirtual    #9   // Method rawDeposit:(I)V
59: aload            4
61: monitorexit   <-----  Конец synchronized (other) {
62: goto             73
// Код обработки исключений опущен
73: iconst_1
74: aload_3
75: monitorexit   <-----  Конец synchronized (this) {
76: ireturn
```

Для полноты картины также рассмотрим путь, по которому идет выполнение, если на счете-отправителе не хватает средств. Фактически мы просто снимаем блокировку с монитора на `this` и возвращаем `this`:

```
77: aload_3
78: monitorexit   <-----  Конец synchronized (this) {
79: goto             89
// Код обработки исключений опущен
89: iconst_0
90: ireturn
```

Имейте в виду, что некоторые инструкции (такие, как `invoke` и `monitor`) могут возбуждать исключения, и мы, как обычно, опускаем обработчики этих исключений в байт-коде. Оставшаяся часть метода выглядит так:

```
91: aload_1
// ...
// Очень похоже, но для ветви other
```

Посмотрим, что произойдет с двумя потоками (не забывая, что идентификатор счета A < B).

Здесь возникает еще одна сложность: локальные переменные (которые используются в таких инструкциях, как `aload_0`) различаются между двумя потоками. Чтобы обозначить это различие, мы слегка изменили байт-код и пометили локальные переменные именами потоков, так что для ясности будет использоваться запись `aload_A0` и `aload_A1`:

```
A24: aload_A0
A25: dup
A26: astore_A3
```

https://t.me/javalib

```
    A27: monitorenter  <-- Поток A устанавливает блокировку на объект A

// Проверка баланса опущена

    A38: aload_A0
    A39: aload_A0
    A40: getfield         #3   // Field balance:D

// .... Переключение контекста A → B
    B91: aload_B1
    B92: dup
    B93: astore_B3
    B94: monitorenter  <-- Поток B пытается установить блокировку
                           на объект A; поток B блокируется

// .... Переключение контекста B → A

    A43: iload_A2
    A44: i2d
    A45: dsub
    A46: putfield         #3   // Field balance:D
    A49: aload_A1
    A50: dup
    A51: astore           A4
    A53: monitorenter  <-- Поток A устанавливает блокировку на объект B
    A54: aload_A1
    A55: iload_A2
    A56: invokevirtual    #9   // Method rawDeposit:(I)V
    A59: aload            A4
    A61: monitorexit   <-- Поток A устанавливает блокировку на объект B
    A62: goto             73

// Код обработки исключений опущен

    A73: iconst_A1
    A74: aload_A3
    A75: monitorexit   <-- Поток A снимает блокировку с объекта B

// .... Переключение контекста A → B
    B95: aload_B0
    B96: dup
    B97: astore B4
    B99: monitorenter
    // ...
   B132: ireturn

// .... Переключение контекста B → A

    A76: ireturn
```

Несомненно, листинг получился сложным. Ключевой момент состоит в том, что `A0 == B1`, так что блокировка этих двух объектов всегда приводит к блокирующему вызову во втором потоке. Инвариант `A < B` гарантирует, что поток B будет направлен по альтернативной ветви.

https://t.me/javalib

5.5.7. Обращения к volatile

Как `volatile` выглядит в байт-коде? Мы найдем ответ на этот вопрос с помощью важного паттерна — «Нестабильное завершение» (Volatile Shutdown).

Этот паттерн помогает решить проблему коммуникации между потоками, которую мы кратко затрагивали ранее, когда встречались с опасным и устаревшим методом `stop()`. Рассмотрим простой класс, который выполняет некоторую работу. В простейшем случае будем считать, что работа поступает дискретными единицами, с четко определенным статусом «завершено» для каждой единицы:

```
public class TaskManager implements Runnable {
    private volatile boolean shutdown = false;

    public void shutdown() {
        shutdown = true;
    }

    @Override
    public void run() {
        while (!shutdown) {
            // Выполнить работу (обработать одну единицу)
        }
    }
}
```

Надеемся, что вам понятен смысл этого паттерна. Все время, пока флаг `shutdown` содержит `false`, продолжается обработка единиц работы. Если флаг когда-нибудь переключится в состояние `true`, то `TaskManager`, обработав текущую единицу работы, выйдет из цикла `while`, а поток корректно завершится.

Модель памяти Java проявляет более тонкий момент: любая запись в нестабильную переменную предшествует всем последующим операциям чтения из этой переменной. Как только другой поток вызовет `shutdown()` для объекта `TaskManager`, флаг переключится в `true`, и эффект этого изменения гарантированно будет видимым при следующем чтении флага — до того, как принимается в работу следующая единица.

Паттерн «Нестабильное завершение» порождает байт-код следующего вида:

```
public class TaskManager implements java.lang.Runnable {
  private volatile boolean shutdown;

  public TaskManager();
    Code:
       0: aload_0
       1: invokespecial    #1   // Method java/lang/Object."<init>":()V
       4: aload_0
       5: iconst_0
       6: putfield         #2   // Field shutdown:Z
```

https://t.me/javalib

```
       9: return

  public void shutdown();
    Code:
       0: aload_0
       1: iconst_1
       2: putfield      #2   // Field shutdown:Z
       5: return

  public void run();
    Code:
       0: aload_0
       1: getfield      #2   // Field shutdown:Z
       4: ifne          10
       7: goto          0
      10: return
}
```

Внимательно присмотревшись, вы увидите, что нестабильная природа переменной `shutdown` не отражается нигде, кроме определения поля. На эту нестабильность нет никаких намеков в кодах операций, а для обращения к переменной используются стандартные коды операций `getfield` и `putfield`.

> **ПРИМЕЧАНИЕ** `volatile` — аппаратный режим доступа, для которого генерируется инструкция ЦП, заставляющая игнорировать аппаратный кэш и вместо этого читать или записывать данные прямо в основной памяти.

Единственное различие проявляется в поведении `putfield` и `getfield`: реализация интерпретатора байт-кода обеспечивает разные пути в коде для нестабильных и стандартных полей.

Собственно, к *любому* участку физической памяти можно обратиться нестабильным образом, но, как мы увидим позднее, это не единственный возможный режим доступа. `volatile` — всего лишь распространенный случай семантики доступа, который Джеймс Гослинг и другие создатели Java решили реализовать в ядре языка; для нее было заведено ключевое слово, которое можно применять к полям.

Конкурентное выполнение — одна из важнейших особенностей платформы Java, и хорошим разработчикам необходимо уверенно разбираться в нем. Мы рассмотрели основы конкурентности Java и факторы проектирования, которые действуют в многопоточных системах. Также мы обсудили модель памяти Java и низкоуровневые подробности того, как платформа реализует конкурентность.

В этой главе мы не пытались рассказать все, что вам когда-либо понадобится знать о конкурентности. Однако этого материала достаточно, чтобы вы могли перейти к практике, понять, о каких темах нужно больше узнать, и не совершать опасных ошибок при написании конкурентного кода. Но если вы собираетесь

https://t.me/javalib

стать действительно первоклассным разработчиком многопоточного кода, вам понадобится узнать больше, чем мы можем рассказать здесь. Написано немало превосходных книг, которые посвящены исключительно конкурентному выполнению в Java, и одна из лучших — «Java Concurrency in Practice»[1] Брайана Гетца (Brian Goetz) и др.

ИТОГИ

- Потоки Java представляют собой низкоуровневую абстракцию.
- Многопоточность присутствует даже в байт-коде Java.
- Модель памяти Java — чрезвычайно гибкая, но она предоставляет минимум гарантий.
- Синхронизация — это кооперативный механизм: чтобы обеспечить безопасность, в синхронизации должны участвовать все потоки.
- Никогда не используйте `Thread.stop()` или `Thread.suspend()`.

[1] Брайан Гетц и др. «Java Concurrency на практике». СПб., издательство «Питер».

https://t.me/javalib

6 Конкурентные библиотеки Java

В ЭТОЙ ГЛАВЕ

- ✓ Атомарные классы
- ✓ Классы блокировок
- ✓ Конкурентные структуры данных
- ✓ Объект `BlockingQueue`
- ✓ Преднамеченные объекты: `Future` и `CompletableFuture`
- ✓ Исполнители

В этой главе речь пойдет о том, что каждый опытный разработчик должен знать о пакете `java.util.concurrent` и о том, какой инструментарий средств конкурентного выполнения он предоставляет. Хочется верить, что к концу этой главы вы будете готовы использовать эти библиотеки и приемы конкурентного программирования в своем коде.

6.1. СТРУКТУРНЫЕ ЭЛЕМЕНТЫ СОВРЕМЕННЫХ КОНКУРЕНТНЫХ ПРИЛОЖЕНИЙ

Как было показано в предыдущей главе, язык Java поддерживал конкурентное выполнение с самых первых версий. Тем не менее с выходом Java 5 (который случился более 15 лет назад) конкурентность в Java обрела новые очертания.

https://t.me/javalib

Основную роль в этом сыграл пакет `java.util.concurrent`, который содержал богатый новый инструментарий для работы с многопоточным кодом.

> **ПРИМЕЧАНИЕ** В последующих версиях Java этот инструментарий был усовершенствован, но классы и пакеты, появившиеся в Java 5, продолжают работать все так же и по-прежнему приносят огромную пользу разработчикам.

Если вы (до сих пор!) используете многопоточный код, который основан исключительно на старых решениях (до Java 5), стоит подумать о его рефакторинге и переходе на `java.util.concurrent`. По нашему опыту, если грамотно переносить код на новые API, его качество повышается: код становится понятнее и надежнее, что почти всегда оправдывает усилия, затраченные на миграцию.

Глава начинается с краткого обзора ключевых классов в `java.util.concurrent` и сопутствующих пакетах (таких, как `atomic` и `locks`). Мы покажем, как пользоваться этими классами, и рассмотрим примеры ситуаций, в которых их уместно применять.

Также вам стоит ознакомиться с документацией Javadoc этих пакетов и постараться получить представление о пакетах в целом. Большинство разработчиков сходятся во мнении, что более высокий уровень абстракции, который предоставляют эти инструменты, заметно упрощает конкурентное программирование.

6.2. АТОМАРНЫЕ КЛАССЫ

Пакет `java.util.concurrent.atomic` содержит несколько классов с именами, которые начинаются с `Atomic`, например `AtomicBoolean`, `AtomicInteger`, `AtomicLong` и `AtomicReference`. Это некоторые из простейших примеров *примитивов конкурентного программирования*: с помощью таких классов можно разрабатывать эффективные и безопасные конкурентные приложения.

> **ПРЕДУПРЕЖДЕНИЕ** Атомарные классы не наследуют классам с похожими именами, так что `AtomicBoolean` нельзя использовать вместо `Boolean`, а `AtomicInteger` не является частным случаем `Integer` (хотя он расширяет `Number`).

Назначение атомарных классов — обеспечивать потоково-безопасные изменяемые переменные. Каждый из четырех классов предоставляет доступ к отдельной переменной соответствующего типа.

> **ПРИМЕЧАНИЕ** Атомарные классы реализованы с расчетом на возможности современных процессоров, так что они могут работать без блокировок, если оборудование и ОС поддерживают такой режим, — а это так практически во всех современных операционных системах.

Предоставляемый доступ работает без блокировок почти на любом современном оборудовании, так что атомарные классы ведут себя подобно нестабильным полям. Однако они обернуты в API классов, который по своим возможностям

https://t.me/javalib

превосходит `volatile`. Этот API включает атомарные (то есть такие, которые либо целиком выполняются, либо целиком не выполняются) методы для соответствующих операций, включая обновления, зависящие от состояния (которые нельзя осуществить с нестабильными переменными, если не использовать блокировку). Как следствие, с помощью атомарных классов разработчики могут легко избегать ситуаций гонки с совместно используемыми данными.

> **ПРИМЕЧАНИЕ** Если вас интересует, как реализованы атомарные классы, подробности будут рассмотрены в главе 17, когда речь пойдет о внутренней реализации и классе `sun.misc.Unsafe`.

В типичном сценарии атомарные классы можно использовать, чтобы реализовать генератор последовательных чисел, подобный механизму в базах данных SQL. Для этого служат такие методы, как атомарный `getAndIncrement()` для класса `AtomicInteger` или `AtomicLong`. Посмотрим, как можно переписать класс `Account` из примера в главе 5 с помощью атомарных классов:

```
private static AtomicInteger nextAccountId = new AtomicInteger(1);

private final int accountId;
private double balance;

public Account(int openingBalance) {
    balance = openingBalance;
    accountId = nextAccountId.getAndIncrement();
}
```

При создании каждого объекта вызывается `getAndIncrement()` для статического экземпляра `AtomicInteger`. Этот метод возвращает целочисленное значение и атомарно увеличивает изменяемую переменную. Атомарность гарантирует, что у двух объектов не будет одного и того же значения `accountId`, — а это именно то, что нам нужно (как и при последовательной нумерации записей в базах данных).

> **ПРИМЕЧАНИЕ** Можно добавить к атомарному полю квалификатор `final`, но это не нужно, потому что поле объявлено статическим, а класс не предоставляет никаких возможностей изменить это поле.

В качестве другого примера мы перепишем код нестабильного завершения, используя `AtomicBoolean`:

```
public class TaskManager implements Runnable {
    private final AtomicBoolean shutdown = new AtomicBoolean(false);

    public void shutdown() {
        shutdown.set(true);
    }

    @Override
    public void run() {
        while (!shutdown.get()) {
```

https://t.me/javalib

```
            // Выполнить работу (обработать одну единицу)
        }
    }
}
```

Существует также класс `AtomicReference`, который тоже позволяет вносить атомарные изменения, но в объекты. В общем случае некоторое модифицированное (возможно, неизменяемое) состояние строится оптимистически, а затем его можно подставить с помощью операции *сравнения с обменом* (CAS, Compare-and-Swap) для `AtomicReference`.

А теперь посмотрим, как `java.util.concurrent` с помощью интерфейса `Lock` моделирует основу классического подхода к синхронизации.

6.3. КЛАССЫ LOCK

Синхронизация с блоковой структурой опирается на простое представление о том, что такое блокировка. У такого решения есть ряд недостатков:

- Существует только один тип блокировки.
- Он применяется в равной степени ко всем синхронизированным операциям с блокируемым объектом.
- Блокировка устанавливается в начале синхронизированного блока или метода.
- Блокировка освобождается в конце блока или метода.
- Либо блокировка устанавливается, либо поток блокируется на неопределенный срок — другие варианты невозможны.

Если бы мы решили переработать поддержку блокировок, можно было бы внести такие улучшения:

- Добавить разные типы блокировок (например, блокировки для чтения или для записи).
- Не ограничивать блокировку пределами блоков (чтобы блокировку можно было установить в одном методе, а снять в другом).
- Если поток не может установить блокировку (например, если ее удерживает другой поток), позволить потоку отказаться от попыток, продолжить их или заняться чем-то другим — `tryLock()`.
- Позволить потоку отказаться от попыток установить блокировку по истечении определенного времени.

Реализовать все эти возможности позволяет интерфейс `Lock` из `java.util.concurrent.locks`. Он поставляется со следующими реализациями:

- `ReentrantLock` — фактически эквивалент классической блокировки, которая используется в синхронизированных блоках Java, но с большей гибкостью.

https://t.me/javalib

- `ReentrantReadWriteLock` — может обеспечить лучшее быстродействие в случаях, когда выполняется много операций чтения и мало — записи.

ПРИМЕЧАНИЕ Существуют и другие реализации (как в JDK, так и написанные сторонними разработчиками), но две перечисленные, безусловно, самые популярные.

Средствами интерфейса `Lock` можно полностью воспроизвести любую функциональность, которую обеспечивает конкурентность с блоковой структурой. Например, в листинге 6.1 с помощью `ReentrantLock` переписан пример предотвращения взаимной блокировки из главы 5.

Нам понадобилось добавить объект блокировки как поле класса, потому что мы больше не можем полагаться на внутреннюю блокировку объекта. Также нужно по-прежнему соблюдать правило, согласно которому блокировки всегда устанавливаются в одном и том же порядке. В нашем примере используется простое правило: сначала устанавливается блокировка объекта с наименьшим идентификатором счета.

Листинг 6.1. Пример взаимной блокировки, переписанный с использованием ReentrantLock

```
private final Lock lock = new ReentrantLock();

public boolean transferTo(SafeAccount other, int amount) {
    // По-прежнему нужно проверить, что amount > 0,
    // если нет — возбудить исключение
    // ...

    if (accountId == other.getAccountId()) {
        // Перевод со счета на этот же счет невозможен
        return false;
    }

    var firstLock = accountId < other.getAccountId() ?
            lock : other.lock;
    var secondLock = firstLock == lock ? other.lock : lock;

    firstLock.lock();   <-- У объекта firstLock — меньший идентификатор счета
    try {
        secondLock.lock();   <-- У объекта secondLock — больший идентификатор счета
        try {
            if (balance >= amount) {
                balance = balance - amount;
                other.deposit(amount);
                return true;
            }
            return false;
        } finally {
            secondLock.unlock();
        }
    } finally {
        firstLock.unlock();
    }
}
```

https://t.me/javalib

Здесь используется схема, которая станет отличным дополнением к вашему инструментарию: исходный вызов `lock()` сопровождается блоком `try … finally`, и блокировка снимается в `finally`.

> **ПРИМЕЧАНИЕ** Реализация блокировок, как и больша́я часть `java.util.concurrent`, основана на классе `AbstractQueuedSynchronizer`.

Эта схема прекрасно работает, если вы воспроизводите ситуацию, похожую на ту, в которой вы использовали бы конкурентность с блоковой структурой. Однако если вам нужно передавать объекты `Lock` (например, возвращать их из методов), такая схема не подойдет.

6.3.1. Объекты условий

В API, который предоставляет `java.util.concurrent`, входят и так называемые *объекты условий*. Они играют в API ту же роль, что и `wait()` и `notify()` в исходном API, но обладают большей гибкостью. Они предоставляют потокам возможность неопределенно долго ожидать некоторого условия и активизироваться, когда оно станет истинным.

Однако в отличие от исходного API (где у монитора объекта бывает только одно условие активизации), интерфейс `Lock` позволяет создать сколько угодно объектов условий. Это дает возможность разделить обязанности: например, у блокировки может быть несколько раздельных групп методов, которые используют разные условия.

Объект условия (который реализует интерфейс `Condition`) создается вызовом метода `newCondition()` для объекта блокировки (который реализует интерфейс `Lock`). Кроме объектов условий, API предоставляет *защелки* (latches) и *барьеры* (barriers) — конкурентные примитивы, которые могут быть полезны в определенных обстоятельствах.

6.4. COUNTDOWNLATCH

`CountDownLatch` — простой примитив конкурентного выполнения, который обеспечивает *консенсусный барьер*: он позволяет нескольким потокам достичь точки координации и ожидать, пока барьер будет снят. Для этого при конструировании нового экземпляра `CountDownLatch` предоставляется целочисленное значение (счетчик). После этого защелкой управляют два метода: `countDown()` и `await()`. Первый метод уменьшает счетчик на 1, а второй блокирует вызывающий поток, пока счетчик не достигнет 0 (если счетчик уже равен 0 или менее, метод ничего не делает). В следующем листинге каждый объект `Runnable` использует защелку, чтобы сигнализировать, что он завершил свою работу.

Листинг 6.2. Использование защелок для передачи сигналов между потоками

```
public static class Counter implements Runnable {
    private final CountDownLatch latch;
```

https://t.me/javalib

```
    private final int value;
    private final AtomicInteger count;

    public Counter(CountDownLatch l, int v, AtomicInteger c) {
        this.latch = l;
        this.value = v;
        this.count = c;
    }

    @Override
    public void run() {
        try {
            Thread.sleep(100);
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
        count.addAndGet(value);  <-- Атомарно обновляет значение value
        latch.countDown();  <-- Уменьшает защелку на единицу
    }
}
```

Обратите внимание: метод `countDown()` — неблокирующий, поэтому как только значение защелки уменьшается на единицу, происходит выход из потока, в котором выполнялся код `Counter`.

Нам также понадобится управляющий код, который выглядит примерно так (обработка исключений опущена):

```
var latch = new CountDownLatch(5);
var count = new AtomicInteger();
for (int i = 0; i < 5; i = i + 1) {
    var r = new Counter(latch, i, count);
    new Thread(r).start();
}

latch.await();
System.out.println("Итого: "+ count.get());
```

В этом коде защелка инициализируется величиной кворума (на рис. 6.1 она равна 2). Затем создается и инициализируется такое же количество потоков, чтобы можно было начать обработку. Главный поток ожидает (awaits) защелку и блокируется, пока она не освободится. Каждый рабочий поток делает паузу, а после ее окончания вызывает `countDown()`. Главный поток не продолжает выполнение до тех пор, пока оба рабочих потока не завершат свою работу. Эти процессы изображены на рис. 6.1.

Другим примером, когда уместно использовать `CountDownLatch`, может служить приложение, которое должно предварительно заполнить несколько кэшей начальными данными, чтобы подготовить сервер к получению входных запросов. Эта задача легко решается при помощи совместно используемой защелки, ссылка на которую хранится в каждом потоке, заполняющем кэш.

https://t.me/javalib

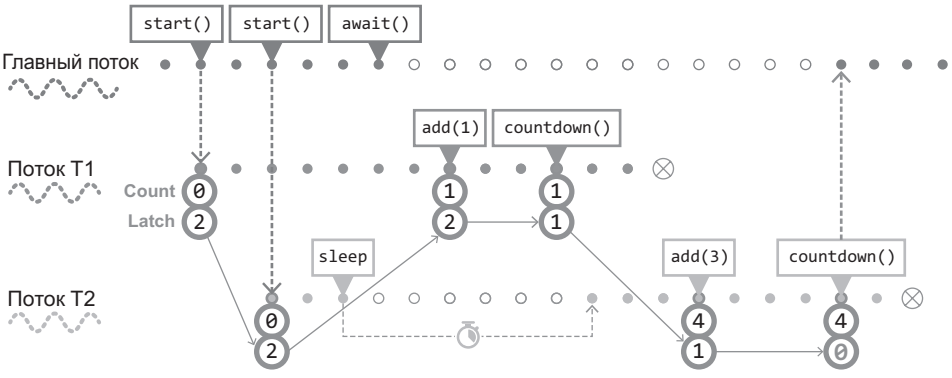


Рис. 6.1. Использование CountDownLatch

Когда заполнение того или иного кэша закончено, заполняющий его объект `Runnable` уменьшает счетчик защелки и завершается. Когда будут заполнены все кэши, главный поток (который ожидал, пока защелка откроется) может продолжить работу: пометить сервер как действующий и начать обрабатывать запросы.

В следующем разделе рассматривается один из самых полезных классов для разработчика многопоточных приложений: `ConcurrentHashMap`.

6.5. CONCURRENTHASHMAP

Класс `ConcurrentHashMap` — это конкурентная версия стандартного класса `HashMap`. Вообще говоря, такие структуры данных, как отображения, очень полезны (и распространены) в разработке конкурентных приложений. Это объясняется (по крайней мере отчасти) тем, как устроена используемая структура данных. Чтобы понять, почему так, стоит повнимательнее присмотреться к классическому варианту `HashMap`.

6.5.1. Упрощенное представление HashMap

Как видно из рис. 6.2, классическая реализация в Java использует *хеш-функцию*, чтобы определить, в какой *ячейке* (bucket) должна храниться пара «ключ — значение». Отсюда происходит «Hash» в имени класса.

Пара «ключ — значение» на самом деле хранится в связанном списке (называемом *хеш-цепочкой*), который начинается с ячейки, соответствующей индексу, который получен хешированием ключа.

В проекте GitHub, который сопровождает эту книгу, представлена упрощенная реализация `Map<String, String>` — класс `Dictionary`. Этот класс на самом деле основан на реализации `HashMap`, которая поставляется как часть Java 7.

https://t.me/javalib

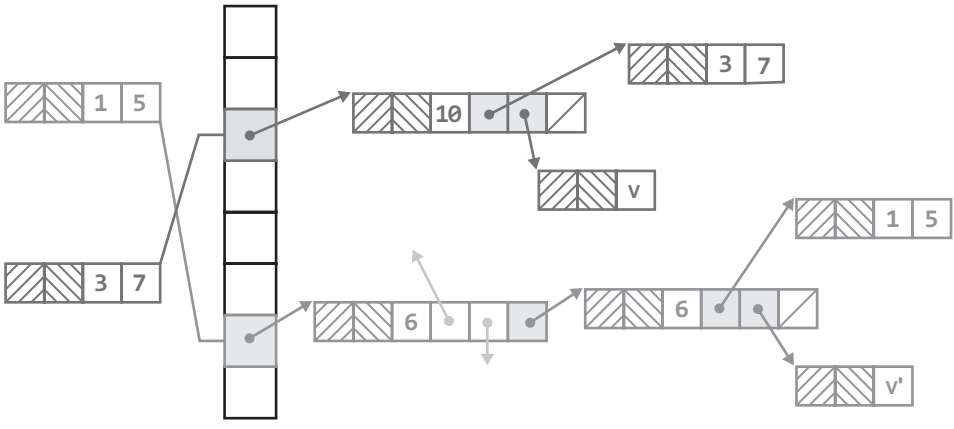


Рис. 6.2. Классическое представление HashMap

ПРИМЕЧАНИЕ В современных версиях Java реализация `HashMap` устроена гораздо сложнее, так что в этом материале мы рассматриваем упрощенную версию, в которой концепции проектирования выражены более наглядно.

Базовый класс `Dictionary` состоит всего из двух полей: основной структуры данных и поля `size`, в котором размер отображения кэшируется по соображениям быстродействия:

```
public class Dictionary implements Map<String, String> {
    private Node[] table = new Node[8];
    private int size;

    @Override
    public int size() {
        return size;
    }

    @Override
    public boolean isEmpty() {
        return size == 0;
    }
```

Здесь используется вспомогательный класс `Node`, который представляет пару «ключ — значение» и реализует интерфейс `Map.Entry`:

```
static class Node implements Map.Entry<String,String> {
    final int hash;
    final String key;
    String value;
    Node next;

    Node(int hash, String key, String value, Node next) {
        this.hash = hash;
```

https://t.me/javalib

```
        this.key = key;
        this.value = value;
        this.next = next;
    }

    public final String getKey() { return key; }
    public final String getValue() { return value; }
    public final String toString() { return key + "=" + value; }

    public final int hashCode() {
        return Objects.hashCode(key) ^ Objects.hashCode(value);
    }

    public final String setValue(String newValue) {
        String oldValue = value;
        value = newValue;
        return oldValue;
    }

    public final boolean equals(Object o) {
        if (o == this)
            return true;
        if (o instanceof Node) {
            Node e = (Node)o;
            if (Objects.equals(key, e.getKey()) &&
                    Objects.equals(value, e.getValue()))
                return true;
        }
        return false;
    }
}
```

Чтобы найти значение в отображении, используется метод `get()`, который зависит от вспомогательных методов `hash()` и `indexFor()`:

```
    @Override
    public String get(Object key) {
        if (key == null)
            return null;
        int hash = hash(key);
        for (Node e = table[indexFor(hash, table.length)];
             e != null;
             e = e.next) {
            Object k = e.key;
            if (e.hash == hash && (k == key || key.equals(k)))
                return e.value;
        }
        return null;
    }

    static final int hash(Object key) {
        int h = key.hashCode();
        return h ^ (h >>> 16);
    }
```

Поразрядная операция, которая проверяет, что значение хеша положительно

https://t.me/javalib

```
static int indexFor(int h, int length) {
    return h & (length - 1);
}
```

Поразрядная операция, которая проверяет, что индекс не превышает размер таблицы

Метод `get()` сначала разбирается с неприятным случаем `null`. Затем мы используем хеш-код объекта ключа, чтобы сконструировать индекс в массиве `table`. Неявно предполагается, что размер `table` является степенью 2, так что `indexFor()`, по сути, вычисляет остаток от целочисленного деления, чтобы убедиться, что возвращаемое значение является действительным индексом в `table`.

ПРИМЕЧАНИЕ Это классический пример ситуации, в которой живой человек поймет, что исключение (в данном случае `ArrayIndexOutOfBoundsException`) никогда не произойдет, но компилятор об этом не догадается.

Теперь, после того как у нас есть индекс в таблице, на его основе мы выбираем соответствующую хеш-цепочку для операции поиска. Поиск начинается с начала списка и следует далее по цепочке. На каждом шаге проверяется, был ли найден объект ключа:

```
if (e.hash == hash && ((k = e.key) == key || key.equals(k)))
    return e.value;
```

Если он найден, возвращается соответствующее значение. Чтобы этот алгоритм работал, ключи и значения хранятся в парах (на самом деле в экземплярах `Node`).

Метод `put()` отчасти похож на предыдущий код:

```
@Override
public String put(String key, String value) {
    if (key == null)
        return null;
    int hash = hash(key.hashCode());
    int i = indexFor(hash, table.length);
    for (Node e = table[i]; e != null; e = e.next) {
        Object k = e.key;
        if (e.hash == hash && (k == key || key.equals(k))) {
            String oldValue = e.value;
            e.value = value;
            return oldValue;
        }
    }

    Node e = table[i];
    table[i] = new Node(hash, key, value, e);

    return null;
}
```

Эту версию хешированной структуры данных нельзя признать полностью пригодной для реальной эксплуатации, но она достаточно наглядно демонстрирует,

https://t.me/javalib

как устроен HashMap, чтобы потом вам было легче понять его конкурентный аналог.

6.5.2. Ограничения Dictionary

Прежде чем переходить к конкурентному случаю, следует упомянуть, что наша упрощенная реализация `Dictionary` не поддерживает некоторые методы `Map`, а именно: методы `putAll()`, `keySet()`, `values()` или `entrySet()` (которые должны быть определены, потому что класс реализует `Map`) просто выдают `UnsupportedOperationException()`.

Мы не поддерживаем эти методы исключительно из-за сложности. Как вы увидите на протяжении книги, интерфейсы коллекций Java обширны и богаты функциональностью. Это хорошо для конечных пользователей, потому что расширяет их возможности, однако реализующей стороне приходится обеспечивать намного больше методов.

В частности, такие методы, как `keySet()`, требуют, чтобы реализация `Map` предоставляла экземпляры `Set`, а для этого часто приходится писать целую реализацию интерфейса `Set` как внутреннего класса. Для наших примеров это создает чересчур много лишних сложностей, поэтому в нашей упрощенной реализации эти методы не поддерживаются.

> **ПРИМЕЧАНИЕ** Позже в книге вы убедитесь, что монолитное, сложное, императивное устройство интерфейсов коллекций создает различные проблемы, когда мы начинаем углубляться в тонкости функционального программирования более подробно.

Простой класс `Dictionary` хорошо работает в пределах своих ограничений. Однако он не защищен от двух возможных проблем:

- Нужно изменять размер таблицы по мере того, как растет количество хранимых элементов.
- Необходима защита от ключей, которые реализуют патологическую форму `hashCode()`.

Первый пункт становится серьезным ограничением. Главное назначение хешированной структуры данных — сократить ожидаемую сложность операций с O(N) до O(log N), например, для выборки значений. Если с ростом количества элементов в отображении размер таблицы не будет изменяться, этот выигрыш по сложности теряется. Реальная реализация должна уметь изменять размер таблицы по мере роста отображения.

6.5.3. Конкурентная реализация Dictionary

Очевидно, в текущей версии класс `Dictionary` не является потоково-безопасным. Представьте себе два потока, один из которых пытается удалить ключ, а другой пытается обновить значение, связанное с этим ключом. В зависимости от порядка операций вполне возможно, что и удаление, и обновление

https://t.me/javalib

отчитаются об успешном выполнении, хотя на самом деле успешно завершится только одна из этих операций. Чтобы решить эту проблему, можно предложить два довольно очевидных (хотя и наивных) способа сделать `Dictionary` конкурентным. (Аналогичные способы можно применить и к реализации `Map` общего вида в Java.)

Первый способ — полная синхронизация, которая упоминалась в главе 5. Результат предсказать нетрудно: в большинстве реальных систем этот подход непригоден из-за плохого быстродействия. Тем не менее стоит посмотреть, как могла бы выглядеть его возможная реализация.

Обеспечить простую потоковую безопасность в данном случае можно двумя способами. Во-первых, можно создать копию класса `Dictionary` (назовем ее `ThreadSafeDictionary`), а затем сделать все ее методы синхронизированными. Этот прием работает, но он требует механически дублировать много кода.

Также можно использовать синхронизированную обертку, чтобы *делегировать* работу нижележащему объекту, в котором фактически размещается словарь. Вот как это могло бы выглядеть:

```
public final class SynchronizedDictionary extends Dictionary {
    private final Dictionary d;

    private SynchronizedDictionary(Dictionary delegate) {
        d = delegate;
    }

    public static SynchronizedDictionary of(Dictionary delegate) {
        return new SynchronizedDictionary(delegate);
    }

    @Override
    public synchronized int size() {
        return d.size();
    }

    @Override
    public synchronized boolean isEmpty() {
        return d.isEmpty();
    }

    // ... Другие методы

}
```

У этого примера немало недостатков, самый важный из которых заключается в том, что объект `d` уже существует и он не синхронизирован. Это прямая дорога к неприятностям: другой код может изменить `d` за пределами синхронизированного блока или метода, и мы окажемся в точно такой же ситуации, которая обсуждалась в предыдущей главе. Для конкурентных структур данных такое решение не подходит.

https://t.me/javalib

Стоит заметить, что JDK, по сути, предоставляет именно такую реализацию — метод `synchronizedMap()` из класса `Collections`. Как нетрудно предположить, он страдает от тех же проблем и не особо полезен на практике.

Второй подход основан на неизменяемости. Как мы уже говорили и повторим еще раз, коллекции Java — это большие и сложные интерфейсы. В частности, это проявляется в том, что в коллекции встроено допущение об изменяемости. Это не какое-то отдельное свойство, которое реализации могут выражать или не выражать по своему желанию: все реализации `Map` и `List` должны реализовывать изменяющие методы.

Из-за этого ограничения может показаться, что в коде Java невозможно смоделировать структуру данных, которая была бы одновременно неизменяемой и соответствовала API коллекций Java: ведь если класс соответствует API, то он также должен предоставить реализацию изменяющего метода. Тем не менее существует одна довольно неприятная лазейка. Реализация интерфейса всегда может возбудить исключение `UnsupportedOperationException`, если в ней не реализован тот или иной метод. Конечно, с точки зрения проектирования языка это ужасно. Контракт интерфейса должен быть именно контрактом.

К сожалению, этот механизм и соглашение возникли до Java 8 (и до того, как появились методы по умолчанию); это была попытка закодировать различия между «обязательными» и «необязательными» методами в то время, когда в языке Java таких различий на самом деле не было.

Это плохой механизм и плохая практика (особенно потому, что `UnsupportedOperationException` — исключение времени выполнения), но их можно было бы использовать приблизительно так:

```
public final class ImmutableDictionary extends Dictionary {
    private final Dictionary d;

    private ImmutableDictionary(Dictionary delegate) {
        d = delegate;
    }

    public static ImmutableDictionary of(Dictionary delegate) {
        return new ImmutableDictionary(delegate);
    }

    @Override
    public int size() {
        return d.size();
    }

    @Override
    public String get(Object key) {
        return d.get(key);
    }

    @Override
    public String put(String key, String value) {
```

https://t.me/javalib

```
        throw new UnsupportedOperationException();
    }

    // Другие изменяющие методы тоже выдают UnsupportedOperationException

}
```

Есть основания утверждать, что этот код нарушает принципы объектно-ориентированного программирования: пользователь ожидает, что он работает с корректной реализацией `Map<String, String>`, однако если попытаться изменить экземпляр, порождается непроверяемое исключение. Это можно с полным основанием рассматривать как угрозу безопасности.

> **ПРИМЕЧАНИЕ** По сути, это компромисс, без которого не удается обойтись методу `Map.of()`: необходимо полностью реализовать интерфейс, поэтому приходится прибегать к исключениям в вызовах изменяющих методов.

Впрочем, это не единственная проблема такого подхода. Он страдает от того же базового недостатка, что и синхронизированный вариант: изменяемый объект по-прежнему существует, на него можно получить ссылку (и изменить), а это противоречит главной цели, которой мы пытаемся достичь. Давайте подведем черту под этими попытками и попробуем найти что-нибудь получше.

6.5.4. ConcurrentHashMap

После того как мы продемонстрировали реализацию простого отображения и обсудили некоторые способы сделать ее конкурентной, пора познакомиться с `ConcurrentHashMap`. Этот класс исключительно прост в использовании, и в большинстве случаев его можно непосредственно подставлять вместо `HashMap`.

Ключевая особенность `ConcurrentHashMap` состоит в том, что несколько потоков могут безопасно обновлять ее одновременно. Чтобы понять, зачем нужна такая возможность, посмотрим, что произойдет, если два потока будут одновременно добавлять элементы в `HashMap` (обработка исключений опущена):

```
var map = new HashMap<String, String>();
var SIZE = 10_000;

Runnable r1 = () -> {
    for (int i = 0; i < SIZE; i = i + 1) {
        map.put("t1" + i, "0");
    }
    System.out.println("Поток 1 завершается");
};
Runnable r2 = () -> {
    for (int i = 0; i < SIZE; i = i + 1) {
        map.put("t2" + i, "0");
    }
    System.out.println("Поток 2 завершается");
};
Thread t1 = new Thread(r1);
```

https://t.me/javalib

```
Thread t2 = new Thread(r2);
t1.start();
t2.start();

t1.join();
t2.join();
System.out.println("Итого: "+ map.size());
```

Запустив этот код, вы увидите новое проявление уже знакомого вам антипаттерна «Потерянное обновление»: выводимое значение будет меньше `2 * SIZE`. Однако на самом деле проблемы конкурентного доступа к отображению намного серьезнее.

Самое опасное поведение `HashMap` в конкурентной модификации не всегда проявляется при малых размерах. Но если увеличить значение `SIZE`, оно в какой-то момент даст о себе знать.

Если увеличить `SIZE` до `1_000_000` или около того, вы, скорее всего, увидите, как один из потоков, которые обновляют `map`, не сможет завершиться. Да, вы правильно поняли: один из потоков может застрять (и застрянет) в бесконечном цикле. Поэтому `HashMap` совершенно небезопасна для использования в многопоточных приложениях (то же относится к классу `Dictionary` из нашего примера).

С другой стороны, если заменить `HashMap` на `ConcurrentHashMap`, мы увидим, что конкурентная версия работает правильно: никаких бесконечных циклов, никаких «потерянных обновлений». У нее есть и другая приятная особенность: что бы вы ни делали, операции с отображением никогда не выдадут `ConcurrentModificationException`.

Давайте в общих чертах рассмотрим, как это достигается. Как выясняется, рис. 6.2, на котором изображена реализация `Dictionary`, также подсказывает способ полезного многопоточного обобщения `Map`, которое оказывается намного лучше двух предыдущих попыток. Оно основано на следующем наблюдении: вместо того чтобы при внесении изменения блокировать целую структуру, достаточно заблокировать только ту хеш-цепочку (ячейку), которая изменяется или считывается.

На рис. 6.3 показано, как работает эта схема. Реализация переместила блокировку на отдельные хеш-цепочки. Этот прием называется *чередованием блокировок* (lock striping), и он позволяет нескольким потокам обращаться к отображению — при условии, что они манипулируют разными цепочками.

Конечно, если двум потокам нужно работать с одной цепочкой, то их выполнение по-прежнему останется взаимоисключающим, но в общем случае такой подход обеспечивает более высокую производительность, чем синхронизация всего отображения.

> **ПРИМЕЧАНИЕ** Вспомните, что если количество элементов отображения растет, то изменяется и размер таблицы ячеек, а это значит, что по мере того, как в `ConcurrentHashMap` добавляется все больше элементов, эта структура продолжает эффективно обслуживать все больше потоков.

https://t.me/javalib

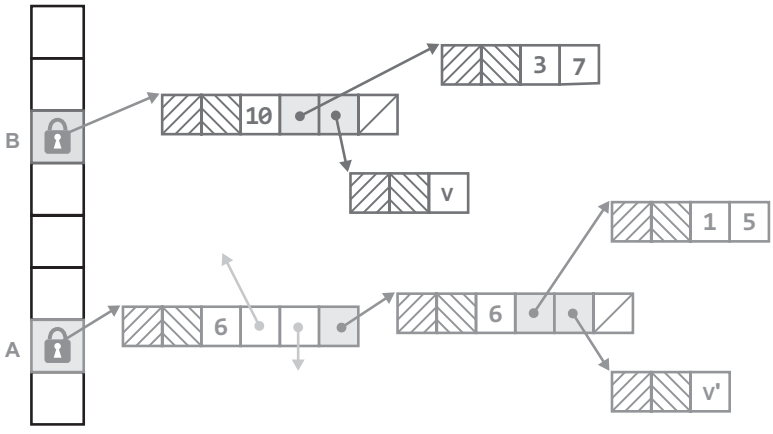


Рис. 6.3. Чередование блокировок

Мы познакомились с поведением `ConcurrentHashMap`, хотя существуют некоторые низкоуровневые подробности, о которых большинству разработчиков не нужно беспокоиться. Кроме того, в Java 8 реализация `ConcurrentHashMap` значительно изменилась, и теперь она устроена гораздо сложнее, чем здесь описано.

Работать с `ConcurrentHashMap` может быть даже *слишком* просто. Достаточно часто, если вы пишете многопоточную программу, где нужен совместный доступ к данным, просто используйте отображение с реализацией `ConcurrentHashMap`. Собственно, если есть хотя бы минимальная вероятность того, что отображение будет изменяться сразу несколькими потоками, всегда лучше применять конкурентную реализацию. Конечно, она потребляет существенно больше ресурсов, чем простая `HashMap`, и работает медленнее из-за синхронизации некоторых операций. Однако, как мы увидим в главе 7, эти неудобства — ничто по сравнению с опасностью гонки, которая приводит к «потерянным обновлениям» или бесконечным циклам.

Наконец, заметим, что `ConcurrentHashMap` фактически реализует интерфейс `ConcurrentMap`, который расширяет `Map`. Изначально в нем было несколько новых методов для потоково-безопасных изменений:

- `putIfAbsent()` — добавляет в `HashMap` пару «ключ — значение», если этого ключа еще нет.
- `remove()` — безопасно удаляет пару «ключ — значение», если ключ есть.
- `replace()` — реализация предоставляет две формы этого метода для безопасной замены в `HashMap`.

Однако в Java 8 некоторые из этих методов были адаптированы для интерфейса `Map` как методы по умолчанию, например:

```
default V putIfAbsent(K key, V value) {
    V v = get(key);
    if (v == null) {
```

https://t.me/javalib

```
            v = put(key, value);
        }

        return v;
    }
```

В недавних версиях Java разрыв между `ConcurrentHashMap` и `Map` несколько сократился, но не забывайте, что при этом `HashMap` остается потоково-небезопасным. Чтобы разные потоки могли безопасно использовать общие данные, применяйте `ConcurrentHashMap`.

В целом `ConcurrentHashMap` — это один из самых полезных классов `java.util.concurrent`. Он безопаснее для многопоточного выполнения и быстрее, чем синхронизация, и у него нет серьезных недостатков при штатном использовании. Для `List` соответствующим аналогом является класс `CopyOnWriteArrayList`, который мы рассмотрим в следующем разделе.

6.6. COPYONWRITEARRAYLIST

Конечно, две неудачные конкурентные реализации из предыдущего раздела можно применить и к `List`. Полностью синхронизированные и неизменяемые списки (но с изменяющими методами, которые выдают исключения времени выполнения) закодировать ничуть не сложнее, чем аналогичные отображения, но и работают они ничуть не лучше.

Возможно ли более эффективное решение? К сожалению, линейная природа списков здесь не поможет. Даже в случае связанного списка, который попытаются изменять несколько потоков, возможны конфликты, например, в задачах, где много операций присоединения (append).

Одна из существующих альтернатив — класс `CopyOnWriteArrayList`. Как следует из названия, это замена для стандартного класса `ArrayList`, в который ради потоковой безопасности добавлена *семантика копирования при записи*. Это означает, что любые операции, которые изменяют список, создают новую копию массива, на котором этот список базируется (рис. 6.4). Кроме того, при этом любым создаваемым итераторам не придется беспокоиться о каких-либо непредвиденных изменениях.

Итераторы гарантированно не будут выдавать исключение `ConcurrentModificationException` и не будут отражать добавления, удаления или изменения в списке, которые произошли после создания итератора. При этом элементы списка (как обычно в Java) все еще могут изменяться: не изменяется только сам список.

Как правило, эта реализация слишком затратна для массового использования, но может пригодиться, если в списке происходит намного больше операций обхода, чем операций изменений, а программист желает избежать сложностей с синхронизацией, но при этом исключить возможность того, что потоки будут мешать друг другу.

https://t.me/javalib

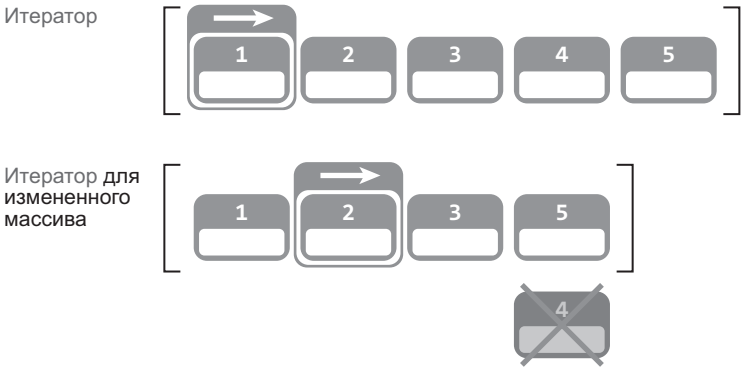


Рис. 6.4. Массив с копированием при записи

Давайте вкратце рассмотрим, как это реализовано. Ключевой метод — `iterator()`, который всегда возвращает новый объект `COWIterator`:

```
public Iterator<E> iterator() {
    return new COWIterator<E>(getArray(), 0);
}
```

Другие важнейшие методы — также `add()`, `remove()` и прочие изменяющие методы. Изменяющие методы всегда заменяют массив новой копией — клонированной и измененной. Защиту массива нужно осуществлять в синхронизируемом блоке, поэтому у класса `CopyOnWriteArrayList` есть внутренняя блокировка, которая используется как монитор (обратите внимание на комментарий):

```
/**
  * Блокировка, защищающая все изменяющие методы. (Мы отдаем предпочтение
  * встроенным мониторам перед ReentrantLock, когда возможны оба варианта.)
  */
final transient Object lock = new Object();

private transient volatile Object[] array;
```

В результате такие операции, как `add()`, могут защищаться следующим образом:

```
public boolean add(E e) {
    synchronized (lock) {
        Object[] es = getArray();
        int len = es.length;
        es = Arrays.copyOf(es, len + 1);
        es[len] = e;
        setArray(es);
        return true;
    }
}
```

В результате в общем случае `CopyOnWriteArrayList` оказывается менее эффективным, чем `ArrayList`, по нескольким причинам:

https://t.me/javalib

- Изменяющие операции синхронизируются.
- Нестабильное хранилище (то есть массив).
- `ArrayList` выделяет память, только когда необходимо изменить размер базового массива, а `CopyOnWriteArrayList` выделяет память и копирует данные при каждом изменении.

При создании итератора сохраняется ссылка на массив в том состоянии, в котором он существует на этот момент. Дальнейшие модификации списка приводят к созданию новой копии, так что итератор будет указывать на старую версию массива:

```
static final class COWIterator<E> implements ListIterator<E> {
    /** Снимок массива */
    private final Object[] snapshot;
    /** Индекс элемента, возвращаемого следующим вызовом next */
    private int cursor;

    COWIterator(Object[] es, int initialCursor) {
        cursor = initialCursor;
        snapshot = es;
    }
    // ...
}
```

Обратите внимание: `COWIterator` реализует `ListIterator`, и поэтому в соответствии с контрактом интерфейса он должен поддерживать методы, изменяющие список, однако для простоты все изменяющие методы выдают исключение `UnsupportedOperationException`.

Подход `CopyOnWriteArrayList` к совместно используемым данным может оказаться полезным, когда важнее быстро получить непротиворечивый снимок данных (который иногда может различаться для разных читающих потоков), чем добиться идеальной синхронизации. Это характерно для ситуаций, где данные не являются критически важными, а благодаря копированию при записи удается избежать потерь быстродействия, которые присущи синхронизации.

Рассмотрим пример копирования при записи в действии.

Листинг 6.3. Пример копирования при записи

```
var ls = new CopyOnWriteArrayList(List.of(1, 2, 3));
var it = ls.iterator();
ls.add(4);
var modifiedIt = ls.iterator();
while (it.hasNext()) {
    System.out.println("Исходный элемент: "+ it.next());
}
while (modifiedIt.hasNext()) {
    System.out.println("Измененный элемент: "+ modifiedIt.next());
}
```

https://t.me/javalib

Этот код написан специально для того, чтобы продемонстрировать, как ведет себя `Iterator` в семантике копирования при записи. Результат выглядит примерно так:

```
Original: 1
Original: 2
Original: 3
Modified: 1
Modified: 2
Modified: 3
Modified: 4
```

В общем случае использовать класс `CopyOnWriteArrayList` стоит чуть более обдуманно, чем `ConcurrentHashMap`, который фактически можно просто подставлять вместо `HashMap`. Это связано с быстродействием: копирование при записи означает, что если список изменяется, то весь массив должен копироваться. Если изменения в списке выполняются достаточно часто по сравнению с обращениями для чтения, этот механизм может и не обеспечить высокого быстродействия.

В целом `CopyOnWriteArrayList` использует другие компромиссные решения, чем `synchronizedList()`. Последний синхронизирует все операции, так что операции чтения из разных потоков могут блокировать друг друга, а к структурам данных копирования при записи это не относится. С другой стороны, `CopyOnWriteArrayList` копирует базовый массив при каждом изменении, тогда как синхронизированная версия делает это, только если массив полон (так же, как `ArrayList`). Однако, как мы неоднократно подчеркнем в главе 7, очень сложно рассуждать о коде, опираясь на теоретические различия. Единственный надежный способ достичь хорошей производительности — тестировать, повторно тестировать и измерять результаты.

Позднее, в главе 15, нам встретится понятие перманентных структур данных — это другой подход к конкурентной работе с данными. Перманентные структуры широко используются в языке программирования Clojure, и коллекция `CopyOnWriteArrayList` (как и `CopyOnWriteArraySet`) дает пример их реализации.

Следующий важный структурный элемент конкурентного кода из `java.util.concurrent` — коллекция `Queue`. С ее помощью можно передавать рабочие элементы между потоками, и она лежит в основе многих гибких и надежных многопоточных архитектур.

6.7. БЛОКИРУЮЩИЕ ОЧЕРЕДИ

Очередь — замечательная абстракция для конкурентного программирования. Это простой и надежный способ распределять вычислительные ресурсы по рабочим единицам (или назначать рабочие единицы вычислительным ресурсам — выбирайте, какая интерпретация вам больше нравится).

Некоторые паттерны в многопоточном программировании Java существенно зависят от потоково-безопасных реализаций `Queue`, поэтому важно хорошо понимать, как работает эта структура. Базовый интерфейс `Queue` определяется

https://t.me/javalib

в `java.util`, потому что очередь может играть важную роль даже в однопоточном программировании, но мы сосредоточимся на многопоточных применениях.

Мы уделим основное внимание одному распространенному сценарию, где с помощью очереди передаются рабочие единицы между потоками. Этот паттерн часто идеально подходит для простейшего конкурентного расширения `Queue` — `BlockingQueue`.

У очереди `BlockingQueue` есть два дополнительных свойства:

- Если очередь заполнена, а поток пытается поместить в нее элемент вызовом `put()`, то поток ожидает, пока в очереди появится свободное место.
- Если очередь пуста, а поток пытается извлечь из нее элемент вызовом `take()`, то поток блокируется.

Эти два свойства очень полезны, потому что если один поток (или пул потоков) заметно обгоняет другой, то более быстрому потоку придется ожидать; так обеспечивается регулировка системы в целом. Схема работы `BlockingQueue` изображена на рис. 6.5.

Java поставляется с двумя основными реализациями интерфейса `BlockingQueue`: `LinkedBlockingQueue` и `ArrayBlockingQueue`. Их свойства несколько различаются: например, реализация с массивом очень эффективна, если известна точная граница длины очереди, тогда как реализация со связанным списком может чуть быстрее работать в некоторых других обстоятельствах.

Однако настоящие различия между реализациями заключаются в подразумеваемой семантике. Хотя реализацию со связанным списком можно создать с ограничением длины, обычно она создается без него, в результате чего получается объект с длиной очереди `Integer.MAX_VALUE`. Фактически это равносильно бесконечности: реальное приложение никогда не справилось бы с двумя миллиардами элементов в одной из своих очередей.

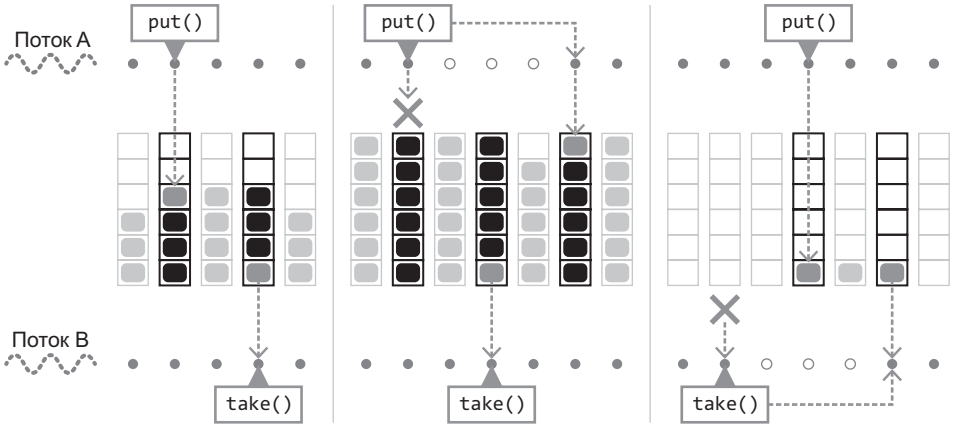


Рис. 6.5. BlockingQueue

https://t.me/javalib

Таким образом, хотя теоретически метод `put()` с `LinkedBlockingQueue` может блокироваться, на практике этого никогда не происходит. А значит, потоки, которые записывают данные в очередь, могут эффективно работать практически с неограниченной скоростью.

С другой стороны, у `ArrayBlockingQueue` длина очереди фиксирована размером внутреннего массива. Если потоки-производители помещают объекты в очередь быстрее, чем получатели их обрабатывают, то в какой-то момент очередь будет заполнена, дальнейшие попытки вызова `put()` будут блокироваться, и производителям придется замедлить темп, с которым они генерируют задачи.

Эту особенность `ArrayBlockingQueue` можно рассматривать как разновидность *противодавления* (back pressure) — важного понятия в разработке конкурентных и распределенных систем.

Чтобы рассмотреть `BlockingQueue` в действии, мы модифицируем пример с банковскими счетами так, чтобы он использовал очереди и потоки. В этом примере мы постараемся избавиться от необходимости блокировать объекты обоих счетов. Базовая структура приложения изображена на рис. 6.6.

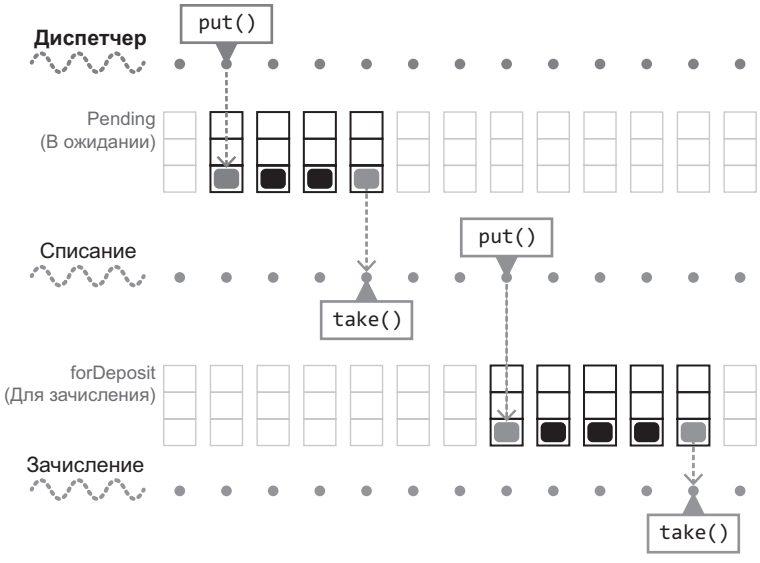


Рис. 6.6. Работа с банковскими счетами с использованием очередей

Начнем с класса `AccountManager` (диспетчер счета) с полями, представленными в листинге 6.4.

Листинг 6.4. Класс AccountManager

```
public class AccountManager {
    private ConcurrentHashMap<Integer, Account> accounts =
        new ConcurrentHashMap<>();
```

https://t.me/javalib

```
    private volatile boolean shutdown = false;

    private BlockingQueue<TransferTask> pending =
        new LinkedBlockingQueue<>();
    private BlockingQueue<TransferTask> forDeposit =
        new LinkedBlockingQueue<>();
    private BlockingQueue<TransferTask> failed =
        new LinkedBlockingQueue<>();

    private Thread withdrawals;
    private Thread deposits;
```

Блокирующие очереди содержат объекты `TransferTask` — простые носители данных, которые описывают задачи перечисления средств:

```
public class TransferTask {
    private final Account sender;
    private final Account receiver;
    private final int amount;

    public TransferTask(Account sender, Account receiver, int amount) {
        this.sender = sender;
        this.receiver = receiver;
        this.amount = amount;
    }

    public Account sender() {
        return sender;
    }

    public int amount() {
        return amount;
    }

    public Account receiver() {
        return receiver;
    }

    // Другие методы опущены
}
```

Никакой дополнительной семантики нет; этот класс — просто примитивный *носитель данных*.

> **ПРИМЕЧАНИЕ** Тип `TransferTask` очень прост, и в Java 17 его можно было бы записать в виде типа записи (о котором шла речь в главе 3).

Методы класса `AccountManager` позволяют создавать счета и формировать задачи перечисления средств:

```
public Account createAccount(int balance) {
    var out = new Account(balance);
    accounts.put(out.getAccountId(), out);
```

https://t.me/javalib

```
    return out;
}

public void submit(TransferTask transfer) {
    if (shutdown) {
        return false;
    }
    return pending.add(transfer);
}
```

Реальную работу `AccountManager` выполняют два потока, которые распределяют задачи между очередями. Сначала рассмотрим операцию списания средств:

```
public void init() {
    Runnable withdraw = () -> {
        boolean interrupted = false;
        while (!interrupted || !pending.isEmpty()) {
            try {
                var task = pending.take();
                var sender = task.sender();
                if (sender.withdraw(task.amount())) {
                    forDeposit.add(task);
                } else {
                    failed.add(task);
                }
            } catch (InterruptedException e) {
                interrupted = true;
            }
        }
        deposits.interrupt();
    };
```

Операция зачисления определяется похожим образом, после чего диспетчер счетов инициализируется задачами:

```
Runnable deposit = () -> {
        boolean interrupted = false;
        while (!interrupted || !forDeposit.isEmpty()) {
            try {
                var task = forDeposit.take();
                var receiver = task.receiver();
                receiver.deposit(task.amount());
            } catch (InterruptedException e) {
                interrupted = true;
            }
        }
    };

    init(withdraw, deposit);
}
```

Перегруженный метод `init`, доступный в границах пакета, запускает фоновые потоки. Этот код выделен в отдельный метод, чтобы его было проще тестировать:

https://t.me/javalib

```
void init(Runnable withdraw, Runnable deposit) {
    withdrawals = new Thread(withdraw);
    deposits = new Thread(deposit);
    withdrawals.start();
    deposits.start();
}
```

Теперь понадобится код, который приводит всю систему в действие:

```
var manager = new AccountManager();
manager.init();
var acc1 = manager.createAccount(1000);
var acc2 = manager.createAccount(20_000);

var transfer = new TransferTask(acc1, acc2, 100);
manager.submit(transfer);  <-- Делает паузу, чтобы перевод успел завершиться
Thread.sleep(5000);  <-- Формирует перевод со счета acc1 на счет acc2
System.out.println(acc1);
System.out.println(acc2);
manager.shutdown();
manager.await();
```

Результат выглядит примерно так:

```
Account{accountId=1, balance=900.0,
    lock=java.util.concurrent.locks.ReentrantLock@58372a00[Unlocked]}
Account{accountId=2, balance=20100.0,
    lock=java.util.concurrent.locks.ReentrantLock@4dd8dc3[Unlocked]}
```

Однако код в таком виде, несмотря на вызовы `shutdown()` и `await()`, не будет выполняться полностью корректно, потому что в нем используются блокирующие вызовы. Чтобы понять, что происходит, взгляните на рис. 6.7.

Когда в основном коде вызывается `shutdown()`, нестабильный логический флаг `shutdown` переключается в `true`, чтобы при каждом последующем чтении он был виден со значением `true`. К сожалению, оба потока списания и зачисления блокируются в вызовах `take()`, потому что очереди пусты. Если в очередь будет каким-то образом помещен объект, то поток списания его обработает, а затем поместит в очередь `forDeposit` (предполагается, что деньги списались успешно). В этот момент поток списания выходит из цикла `while`, после чего штатно завершается.

Поток зачисления теперь видит объект в очереди `forDeposit`, активизируется, берет его, обрабатывает, а затем выходит из своего цикла `while` и также завершается штатно. Однако это штатное завершение зависит от того, остаются ли задачи в очереди. В граничном случае, если очередь пуста, потоки застрянут в блокирующем вызове `take()` навсегда. Чтобы преодолеть эту проблему, давайте рассмотрим весь диапазон методов, который предоставляют реализации блокирующих очередей.

https://t.me/javalib

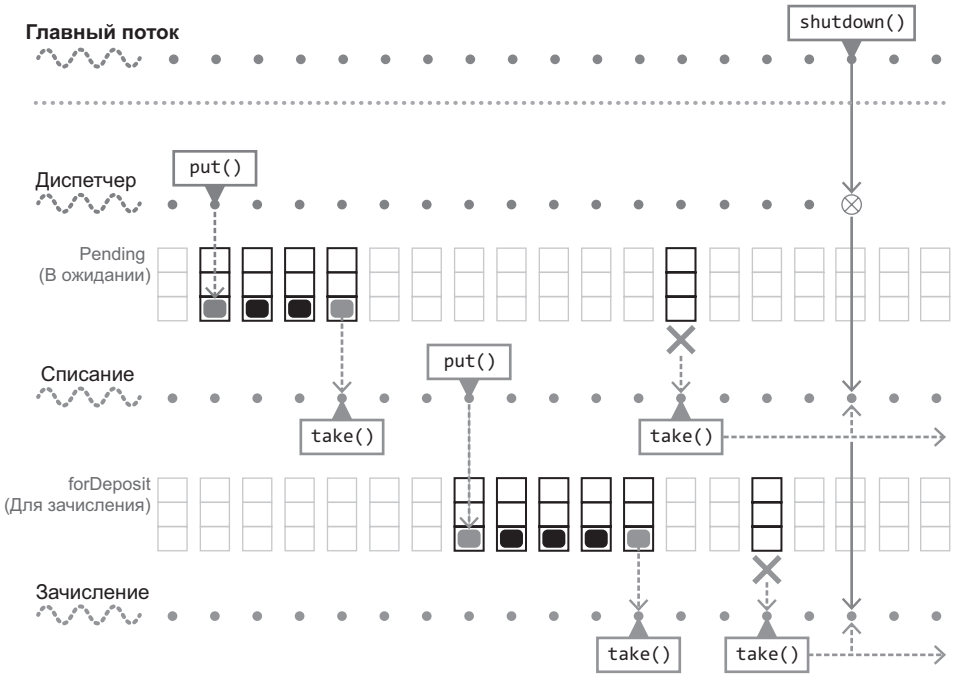


Рис. 6.7. Некорректная последовательность завершения

6.7.1. Интерфейсы BlockingQueue

На самом деле существуют три разные стратегии взаимодействия с интерфейсом `BlockingQueue`. Чтобы понять, чем они различаются, рассмотрим возможные варианты поведения API в таком сценарии: поток пытается вставить элемент в очередь с ограниченной емкостью, которая в настоящий момент не может этот элемент принять (то есть очередь заполнена).

На логическом уровне возможны три варианта. Вызов вставки может:

- блокироваться, пока не освободится место в очереди;
- вернуть значение, которое сигнализирует о неудаче (возможно, логическое `false`);
- возбудить исключение.

Конечно, эти же три варианта возможны и в обратной ситуации (попытка извлечь элемент из пустой очереди). Первая из перечисленных возможностей реализуется уже знакомыми вам методами `take()` и `put()`.

> **ПРИМЕЧАНИЕ** Второй и третий варианты предоставляются интерфейсом `Queue`, который является надынтерфейсом для `BlockingQueue`.

https://t.me/javalib

Второй вариант предоставляет неблокирующий API, который возвращает специальные значения и воплощается в методах `offer()` и `poll()`. Если вставку в очередь не удается завершить, то `offer()` аварийно завершается и возвращает `false`. Программист должен проанализировать результат и принять соответствующие меры.

Похожим образом `poll()` немедленно возвращает `null`, если попытка извлечь элемент из очереди не удалась. Выглядит немного странным, что у класса с именем `BlockingQueue` есть неблокирующий API, но он полезен (а кроме того, он обязательно должен быть, потому что `BlockingQueue` наследует `Queue`).

На самом деле `BlockingQueue` также предоставляет дополнительные перегруженные версии неблокирующих методов. Эти методы позволяют запустить `poll()` или `offer()` с тайм-аутом, чтобы поток, который столкнулся с проблемами, мог отказаться от попыток взаимодействовать с очередью и заняться чем-то другим.

Класс `AccountManager` из листинга 6.4 можно изменить, чтобы в нем использовался неблокирующий API с тайм-аутом:

```
Runnable withdraw = () -> {
    LOOP:
    while (!shutdown) {
        try {
            var task = pending.poll(5,
                                    TimeUnit.SECONDS);
            if (task == null) {
                continue LOOP;
            }
            var sender = task.sender();
            if (sender.withdraw(task.amount())) {
                forDeposit.put(task);
            } else {
                failed.put(task);
            }
        } catch (InterruptedException e) {
            // Зарегистрировать критическую ошибку
            // и перейти к следующему элементу
        }
    }
    // Перевести ожидающие элементы очереди в состояние неудачи
    // или зарегистрировать в журнале
};
```

Явно используется метка цикла, чтобы было видно, какой именно цикл продолжается

Если тайм-аут истекает, poll() возвращает null

Код потока для внесения средств нужно изменить аналогично.

Таким образом решается проблема завершения, описанная ранее в этом разделе, потому что теперь потоки не могут навсегда блокироваться в методах извлечения элементов. Если до истечения тайм-аута объект не появится, то `poll()` все равно вернет управление и предоставит значение `null`. Затем цикл продолжается, но гарантии видимости нестабильного флага теперь обеспечивают выполнение условия цикла `while`, так что происходит выход из цикла, а поток завершается

https://t.me/javalib

корректно. Это означает, что в общем случае после того, как вызван метод `shutdown()`, `AccountManager` завершится за ограниченное время, а это именно то, чего мы добивались.

Завершая обсуждение API `BlockingQueue`, стоит рассмотреть третий подход, упоминавшийся ранее: методы, которые возбуждают исключения, если операция с очередью не может завершиться немедленно. Эти методы — `add()` и `remove()`; откровенно говоря, они создают проблемы по нескольким причинам, не в последнюю очередь из-за того, что исключения, которые они возбуждают при неудаче (`IllegalStateException` и `NoSuchElementException` соответственно), — это исключения времени выполнения, и их не нужно обрабатывать явно.

Впрочем, корни проблем API с возбуждением исключений лежат глубже. Общий принцип Java гласит, что исключения должны использоваться, чтобы обрабатывать исключительные ситуации, то есть те, которые программа не считает частью нормального выполнения. Однако ситуация пустой очереди абсолютно нормальна. Таким образом, возбуждать исключение в этой ситуации — значит нарушать правило, которое иногда выражается как «Не используйте исключения, чтобы управлять последовательностью выполнения».

В общем случае исключения обходятся довольно дорого, потому что при создании экземпляра исключения конструируется трассировка стека, а при возбуждении исключения она разбирается. Считается хорошей практикой не создавать исключение, если только вы не собираетесь его немедленно возбудить. Поэтому мы не рекомендуем использовать форму API `BlockingQueue` с исключениями.

6.7.2. WorkUnit

Все интерфейсы `Queue` являются обобщениями: `Queue<E>`, `BlockingQueue<E>` и т. д. И хотя это может показаться странным, иногда стоит воспользоваться этим обстоятельством и ввести искусственный класс-контейнер, чтобы упаковывать единицы работы.

Например, если ваш класс с именем `MyAwesomeClass` представляет единицы работы, которые вы собираетесь обрабатывать в многопоточном приложении, это не лучший вариант:

```
BlockingQueue<MyAwesomeClass>
```

Вместо этого больше подойдет такая конструкция:

```
BlockingQueue<WorkUnit<MyAwesomeClass>>
```

где `WorkUnit` (или `QueueObject`, или как вы захотите назвать свой класс-контейнер) — класс-обертка, который может выглядеть примерно так:

```
public class WorkUnit<T> {
    private final T workUnit;
```

https://t.me/javalib

```
    public T getWork() {
        return workUnit;
    }

    public WorkUnit(T workUnit) {
        this.workUnit = workUnit;
    }

    // ... Другие методы опущены
}
```

Этот дополнительный уровень косвенности имеет смысл ввести, чтобы обеспечить место для дополнительных метаданных без ущерба для концептуальной целостности внутреннего типа (в данном примере — `MyAwesomeClass`). На рис. 6.8 показано, как работает эта внешняя обертка.

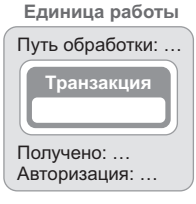


Рис. 6.8. Использование единицы работы в качестве обертки с метаданными

Эта возможность оказывается неожиданно полезной. Дополнительные метаданные пригождаются во многих сценариях, в том числе:

- Тестирование (например, чтобы вывести историю изменений объекта).
- Индикаторы производительности (например, время выполнения задачи или качество обслуживания).
- Информация о системе на стадии выполнения (например, путь маршрутизации этого экземпляра `MyAwesomeClass`).

Добавить этот дополнительный уровень задним числом будет намного сложнее. Если впоследствии вы обнаружите, что в каких-то обстоятельствах нужны дополнительные метаданные, то вместо простых изменений в классе `WorkUnit` вам придется заняться серьезным рефакторингом.

Перейдем к обсуждению *преднамеченных объектов* (futures), которые представляют заглушки для выполняющихся задач (обычно в другом потоке).

6.8. ПРЕДНАМЕЧЕННЫЕ ОБЪЕКТЫ

Интерфейс `Future` в `java.util.concurrent` — это простое представление асинхронной задачи: тип, содержащий результат задачи, которая еще не обязательно

https://t.me/javalib

была завершена, но, возможно, завершится в какой-то момент в будущем. Основные методы `Future`:

- `get()` — получает результат. Если результат еще недоступен, выполнение блокируется, пока он не будет доступен.
- `isDone()` — позволяет вызывающей стороне определить, завершились ли вычисления. Выполняется без блокирования.
- `cancel()` — позволяет отменить вычисления, не завершая их.

Также существует версия `get()`, которая принимает тайм-аут, чтобы избежать блокирования на неограниченное время, по аналогии с представленными ранее методами `BlockingQueue` с тайм-аутом. В листинге 6.5 показано, как с помощью `Future` можно найти *n*-е простое число.

Листинг 6.5. Поиск простых чисел с помощью Future

```
Future<Long> fut = getNthPrime(1_000_000_000);
try {
    long result = fut.get(1, TimeUnit.MINUTES);
    System.out.println("Нашел: " + result);
} catch (TimeoutException tox) {
    // Тайм-аут: задачу лучше отменить
    System.err.println("Время ожидания истекло, задача отменена");
    fut.cancel(true);
} catch (InterruptedException e) {
    fut.cancel(true);
    throw e;
} catch (ExecutionException e) {
    fut.cancel(true);
    e.getCause().printStackTrace();
}
```

В этом фрагменте следует представить, что `getNthPrime()` вернет объект `Future`, который выполняется в некотором фоновом потоке (или даже нескольких потоках) — возможно, в одном из фреймворков исполнителей, которые мы рассмотрим позднее в этой главе.

Поток, который выполняет фрагмент кода, входит в «`get` с тайм-аутом» и блокируется в ожидании ответа до 60 секунд. Если ответ не поступит, поток повторяет попытку и входит в другое ожидание с блокированием. Даже на современном оборудовании эти вычисления могут выполняться достаточно долго, так что в конце концов вам может понадобиться метод `cancel()` (хотя код в своем нынешнем виде не предоставляет никакого механизма отмены запроса).

Во втором примере рассмотрим неблокирующий ввод/вывод. На рис. 6.9 показан объект `Future` в контексте, который позволяет использовать фоновый поток для ввода/вывода.

https://t.me/javalib

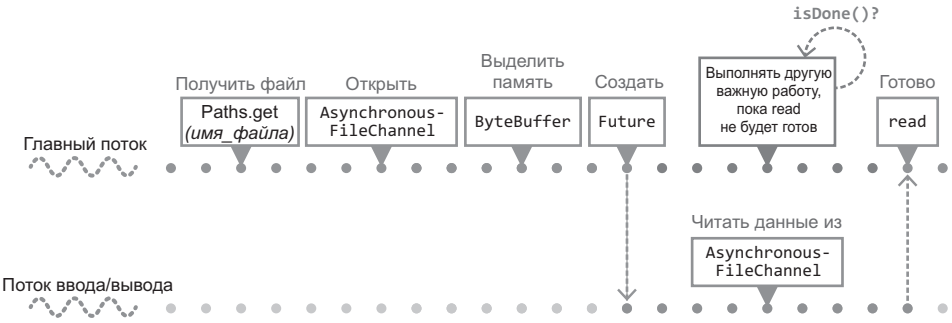


Рис. 6.9. Использование преднамеченного объекта в Java

Этот API существует довольно долго (он появился в Java 7), и с его помощью можно выполнять неблокирующие конкурентные операции наподобие этих:

```
try {
    Path file = Paths.get("/Users/karianna/foobar.txt");

    var channel = AsynchronousFileChannel.open(file);

    var buffer = ByteBuffer.allocate(1_000_000);
    Future<Integer> result = channel.read(buffer, 0);

    BusinessProcess.doSomethingElse();

    var bytesRead = result.get();
    System.out.println("Прочитано байт [" + bytesRead + "]");
} catch (IOException | ExecutionException | InterruptedException e) {
    e.printStackTrace();
}
```

Открывает файл в асинхронном режиме

Запрашивает чтение до миллиона байт

Занимается чем-то другим

Получает результат, когда он будет готов

Такая структура позволяет главному потоку заниматься чем-то иным (`doSomethingElse()`), пока операция ввода/вывода продолжается в другом потоке, которым управляет среда выполнения Java. Это полезный механизм, но он требует поддержки со стороны библиотеки, которая предоставляет эту функциональность. Эта поддержка может быть ограниченной; кроме того, что делать, если мы хотим создавать свои собственные асинхронные рабочие процессы?

6.8.1. CompletableFuture

Тип Java `Future` определяется как интерфейс, а не как конкретный класс. Любой API, который собирается работать в стиле `Future`, должен предоставить его конкретную реализацию.

Не каждый разработчик легко справится с таким кодом, и его отсутствие — очевидный пробел в инструментарии, поэтому, начиная с Java 8, в JDK был включен новый механизм: конкретная реализация `Future`, которая расширяет

https://t.me/javalib

возможности разработчиков и в чем-то похожа на преднамеченные объекты из других языков (таких, как Kotlin и Scala).

Класс называется `CompletableFuture` — это конкретный тип, который реализует интерфейс `Future`, предоставляет дополнительную функциональность и спроектирован как простой структурный элемент для создания асинхронных приложений. Основная идея заключается в том, что можно создавать экземпляры типа `CompletableFuture<T>` (он обобщается по типу возвращаемого значения), и созданный объект представляет преднамеченный объект в *незавершенном* («неисполненном») состоянии.

Позднее любой поток, у которого есть ссылка на `CompletableFuture`, может вызвать для него `complete()` и предоставить значение, тем самым он завершает («исполняет») преднамеченный объект. Завершенное значение немедленно становится видимым для всех потоков, которые заблокированы по вызову `get()`. После завершения все последующие вызовы `complete()` игнорируются.

С `CompletableFuture` невозможна ситуация, при которой разные потоки видят разные значения: преднамеченный объект либо завершен, либо нет. Если он завершен, то в нем хранится значение, которое предоставил первый поток, вызвавший `complete()`.

Разумеется, неизменяемость тут ни при чем: состояние `CompletableFuture` меняется со временем. Однако оно меняется *только один раз* — из незавершенного в завершенное. Нет риска того, что разные потоки будут видеть противоречивые состояния.

> **ПРИМЕЧАНИЕ** Объекты Java `CompletableFuture` отчасти напоминают обещания (promises) из других языков (таких, как JavaScript).

Для примера напишем реализацию метода `getNthPrime()`, который мы упоминали ранее:

```
public static Future<Long> getNthPrime(int n) {
    var numF = new CompletableFuture<Long>();

    new Thread( () -> {
        long num = NumberService.findPrime(n);
        numF.complete(num);
    } ).start();

    return numF;
}
```

Создает завершаемый преднамеченный объект в незавершенном состоянии

Создает и запускает новый поток, который завершит преднамеченный объект

Непосредственное вычисление простого числа

Метод `getNthPrime()` создает «пустой» экземпляр `CompletableFuture` и возвращает этот объект-контейнер вызывающей стороне. Чтобы все это заработало, какой-то код должен вызвать `getNthPrime()`, — например, код из листинга 6.5.

`CompletableFuture` можно рассматривать по аналогии с системами «клиент — сервер». Интерфейс `Future` предоставляет только метод проверки

https://t.me/javalib

`isDone()` и блокирующий `get()`: эта сторона играет роль клиента. Экземпляр `CompletableFuture` играет роль сервера: он позволяет полностью контролировать выполнение и завершение кода, который исполняет преднамеченный объект и предоставляет значение.

В нашем примере `getNthPrime()` обрабатывает вызов числовой службы в отдельном потоке. Когда этот вызов возвращает управление, преднамеченный объект завершается явным образом.

Того же эффекта можно добиться другим, чуть более компактным способом — методом `CompletableFuture.supplyAsync()`, которому передается объект `Callable<T>`, представляющий выполняемую задачу. Этот вызов использует пул потоков уровня приложения, а пулом управляет конкурентная библиотека:

```
public static Future<Long> getNthPrime(int n) {
    return CompletableFuture.supplyAsync(
        () -> NumberService.findPrime(n));
}
```

На этом мы завершаем краткий обзор конкурентных структур данных. Это одни из основных структурных элементов, которые закладывают основу для разработки серьезных многопоточных приложений.

> **ПРИМЕЧАНИЕ** Мы еще вернемся к `CompletableFuture` позднее в книге, а именно в главах, в которых обсуждаются продвинутые методы конкурентного программирования, а также взаимодействие с функциональным программированием.

В следующем разделе рассматриваются *исполнители* и пулы потоков, которые предоставляют более удобные средства управления выполнением, чем низкоуровневый API на базе `Thread`.

6.9. ЗАДАЧИ И ВЫПОЛНЕНИЕ

Класс `java.lang.Thread` существует еще с Java 1.0. Одним из исходных аргументов в пользу языка Java была поддержка многопоточности, встроенная на уровне языка. Это достаточно мощный класс, и он выражает конкурентность в форме, близкой к механизму ОС. Тем не менее по своей сути это низкоуровневый API для управления конкурентным выполнением.

Из-за низкоуровневой природы этого класса многим программистам трудно правильно или эффективно с ним работать. Другие языки, которые появились после Java, учились на опыте работы с потоками в Java и расширяли этот опыт, чтобы предоставить альтернативные подходы. В свою очередь, некоторые из этих подходов повлияли на архитектуру `java.util.concurrent` и последующие нововведения в области конкурентности Java.

В этом разделе наша непосредственная цель — конструировать задачи (или единицы работы), которые можно выполнять, не создавая новый поток для

https://t.me/javalib

каждой задачи. В конечном итоге это означает, что задачи надо моделировать в виде кода, который можно просто вызвать (вместо того, чтобы напрямую представлять его как поток).

Тогда эти задачи можно будет планировать с помощью общего ресурса — пула потоков, который выполняет задачу, пока не завершит ее, а затем переходит к следующей задаче. Давайте посмотрим, как моделировать такие задачи.

6.9.1. Моделирование задач

Этот раздел посвящен двум разным способам моделирования задач — интерфейсу `Callable` и классу `FutureTask`. Мы могли бы еще рассмотреть вариант с `Runnable`, но он не настолько полезен, потому что метод `run()` не возвращает значения, а значит, может выполнять работу только в форме побочных эффектов.

Есть еще одна важная, но не такая очевидная особенность моделирования задач: если мы предполагаем, что емкость потоков конечна, то задачи определенно должны завершаться за ограниченное время.

Если возможен бесконечный цикл, то некоторые задачи могут «украсть» поток-исполнитель из пула, сокращая тем самым общую емкость всех будущих задач. Со временем это может привести к тому, что ресурсы пула потоков истощатся и дальнейшая работа станет невозможной. А значит, нужно внимательно следить за тем, чтобы все создаваемые задачи завершались за конечное время.

Интерфейс Callable

Интерфейс `Callable` представляет очень распространенную абстракцию — фрагмент кода, который можно вызвать, и он вернет результат. Выглядит просто, но на самом деле это нетривиальная и мощная концепция, которая приводит к чрезвычайно полезным шаблонам разработки.

Типичный вариант использования `Callable` — лямбда-выражение (или анонимная реализация). Вот код, в последней строке которого переменной `s` присваивается значение `out.toString()`:

```
var out = getSampleObject();
Callable<String> cb = () -> out.toString();

String s = cb.call();
```

Представляйте себе `Callable` как отложенный вызов одного метода `call()`, предоставляемый лямбда-выражением.

Класс FutureTask

Класс `FutureTask` — единственная часто используемая реализация интерфейса `Future`, которая также реализует `Runnable`. Как вы увидите, это означает, что `FutureTask` можно передавать исполнителям. API класса `FutureTask`, по сути, совмещает в себе API `Future` и `Runnable`: `get()`, `cancel()`, `isDone()`, `isCancelled()`

https://t.me/javalib

и `run()`, хотя последний метод, который выполняет фактическую работу, должен вызываться исполнителем, а не напрямую клиентским кодом.

Для `FutureTask` предоставляются два вспомогательных конструктора: один принимает `Callable`, а другой — `Runnable` (который использует `Executors.callable()`, чтобы преобразовать его в `Callable`). Это позволяет гибко организовывать задачи: написать задание в виде `Callable`, а затем упаковать его в экземпляр `FutureTask`, который можно планировать (а при необходимости отменять) в исполнителе благодаря тому, что `FutureTask` обладает свойствами `Runnable`.

Этот класс предоставляет простую модель состояний для задач и средства управления задачами через эту модель. Возможные переходы между состояниями изображены на рис. 6.10.

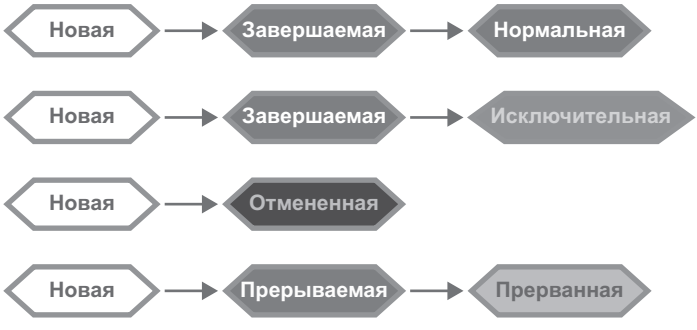


Рис. 6.10. Модель состояний для задач

Этого достаточно для широкого диапазона стандартных возможностей исполнения. Рассмотрим стандартные исполнители, которые предоставляет JDK.

6.9.2. Исполнители

В JDK есть несколько стандартных интефейсов, с помощью которых описываются пулы потоков. Первый интерфейс — `Executor` — очень прост и определяется так:

```
public interface Executor {

    /**
     * Исполняет заданную команду в некоторый момент в будущем. Команда
     * может выполняться в новом потоке, в потоке из пула или в вызывающем
     * потоке на усмотрение реализации {@code Executor}.
     *
     * @param command Runnable-задача
     * @throws RejectedExecutionException если задача не может быть
     * принята для исполнения
     * @throws NullPointerException если command представляет собой null
     */
    void execute(Runnable command);
}
```

https://t.me/javalib

Следует заметить, что хотя этот интерфейс содержит только один абстрактный метод (то есть является так называемым SAM-типом), он *не* помечается аннотацией `@FunctionalInterface`. В принципе, его можно использовать как целевой тип для лямбда-выражения, но официально он не предназначен для применения в функциональном программировании.

Впрочем, `Executor` используется не так часто: намного более распространен интерфейс `ExecutorService`, который расширяет `Executor` и добавляет `submit()`, а также еще несколько методов жизненного цикла (таких, как `shutdown()`).

Чтобы разработчикам было удобнее создавать экземпляры и работать со стандартными пулами потоков, JDK предоставляет класс `Executors` — коллекцию статических вспомогательных методов (в основном фабрик). Вот четыре наиболее часто используемых метода:

```
newSingleThreadExecutor()
newFixedThreadPool(int nThreads)
newCachedThreadPool()
newScheduledThreadPool(int corePoolSize)
```

Сейчас мы рассмотрим их по порядку, а позднее в книге также будут представлены другие, более сложные возможности.

6.9.3. Однопоточный исполнитель

Простейший из исполнителей — однопоточный. По сути, это инкапсулированная комбинация одного потока и очереди задач (которая является блокирующей).

Клиентский код помещает исполняемую задачу в очередь вызовом `submit()`. Один поток исполнения последовательно получает задачи из очереди и выполняет их до завершения, прежде чем переходить к следующей задаче.

> **ПРИМЕЧАНИЕ** Исполнители не реализуются в виде отдельных типов, а определяются разными вариантами параметров при конструировании нижележащего пула потоков.

Любые задачи, отправленные в то время, пока исполняющий поток занят, помещаются в очередь, пока поток не освободится. Из того, что этот исполнитель обеспечивается единственным потоком, следует, что если код будет нарушать ранее упомянутое требование завершения за конечное время, то ни одно из последующих заданий даже не будет запущено.

> **ПРИМЕЧАНИЕ** Эта форма исполнителя часто удобна при тестировании, потому что ее можно сделать более детерминированной по сравнению с другими формами.

Продемонстрируем очень простой пример однопоточного исполнителя:

```
var pool = Executors.newSingleThreadExecutor();
Runnable hello = () -> System.out.println("Hello world");
pool.submit(hello);
```

https://t.me/javalib

Вызов `submit()` помещает исполняемую задачу в очередь заданий исполнителя. Отправка задания является неблокирующей операцией (если только очередь заданий не заполнена).

Тем не менее стоит действовать осторожно: например, если главный поток немедленно завершается, то отправленное задание может не успеть попасть в пул потоков и поэтому не запустится. Вместо прямолинейного выхода имеет смысл сначала вызвать метод `shutdown()`.

Подробности можно найти в классе `ThreadPoolExecutor`, но по сути, этот метод инициирует *упорядоченное завершение*, при котором ранее принятые задачи выполняются, но новые задачи не принимаются. Фактически это решает проблемы из листинга 6.4, позволяя опустошать очереди незавершенных операций.

> **ПРИМЕЧАНИЕ** Задачи, выполняющиеся в бесконечном цикле, плохо сочетаются с запросами на упорядоченное завершение — такой пул потоков никогда не завершится.

Конечно, если бы однопоточного исполнителя было достаточно на все случаи жизни, вам не пришлось бы углубляться в тонкости параллельного программирования и присущие ему проблемы. А значит, необходимо рассмотреть альтернативные варианты с несколькими исполняющими потоками.

6.9.4. Пул фиксированного размера

Пул фиксированного размера, который можно получить с помощью одной из разновидностей `Executors.newFixedT hreadPool()`, — это фактически многопоточное обобщение однопоточного исполнителя. Пул создается с таким количеством потоков, которое указал пользователь в момент создания.

Эти потоки повторно используются, чтобы выполнять много задач, одну за другой. Такая архитектура избавляет пользователей от затрат на создание потоков. Как и в однопоточном случае, если все потоки задействованы, новые задачи сохраняются в блокирующей очереди до того, как какой-либо поток освободится.

Эта разновидность пула потоков особенно эффективна, если поток задач стабилен и известен заранее, а размер всех задач (в смысле продолжительности вычислений) примерно одинаков. Проще всего создавать такие пулы соответствующим фабричным методом:

```
var pool = Executors.newFixedThreadPool(2);
```

Эта инструкция создает пул с двумя исполняющими потоками, которые поочередно принимают задачи из очереди по недетерминированному принципу. Даже если поступающие задачи строго упорядочены по времени, нет гарантий относительно того, какому потоку будет назначена каждая конкретная задача.

Как следствие, в ситуации на рис. 6.11 нельзя рассчитывать на то, что задачи в исходящей очереди будут точно упорядочены по времени, даже если они были упорядочены во входящей очереди.

https://t.me/javalib

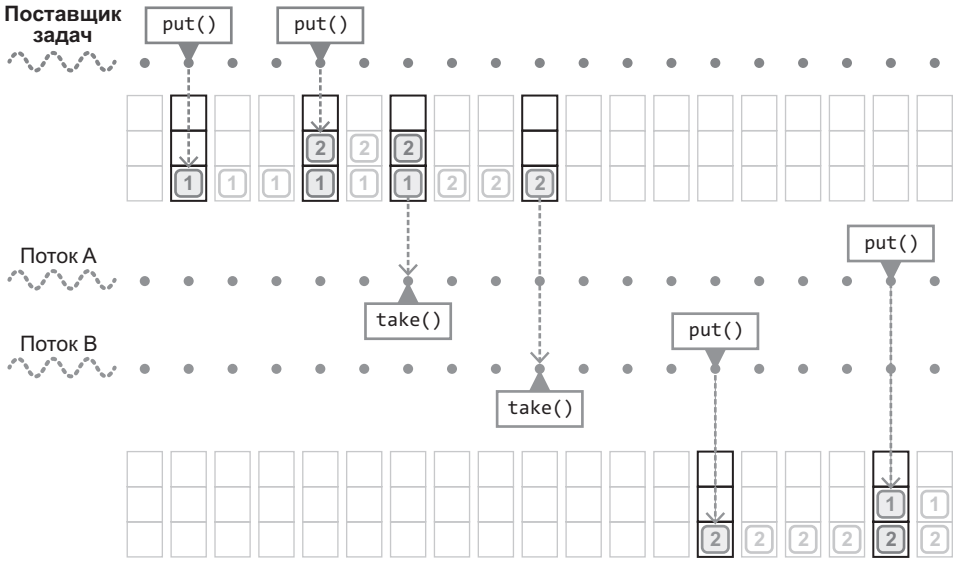


Рис. 6.11. Пул потоков и две очереди

У пулов фиксированного размера есть свои сценарии использования, но есть и ограничения. Во-первых, если входящие в них исполняющие потоки будут уничтожены, они не заменяются. Если существует вероятность того, что отправленные задания возбудят исключение времени выполнения, это может привести к нехватке пулов.

Рассмотрим альтернативу, которая позволяет избежать этой опасности (хотя страдает от других недостатков).

6.9.5. Пул потоков с кэшированием

Пул потоков фиксированного размера часто используется, когда распределение рабочей нагрузки известно и относительно стабильно. Однако если входящая работа неравномерна или имеет выбросы, то пул с фиксированным количеством потоков с большой вероятностью будет неэффективным.

`CachedThreadPool` не ограничивает количество потоков. Он повторно использует потоки, если они доступны, однако по мере надобности создает новые потоки для обработки входных задач:

```
var pool = Executors.newCachedThreadPool();
```

Потоки хранятся в кэше простоя в течение 60 секунд, и если в конце этого периода они все еще там присутствуют, они удаляются из кэша и уничтожаются.

Конечно, очень важно, чтобы задачи реально завершались. В противном случае пул будет со временем создавать все больше и больше потоков, потреблять все

https://t.me/javalib

больше и больше машинных ресурсов и рано или поздно аварийно завершится или зависнет.

Что выбрать: пул фиксированного размера или пул с кэшированием, — зависит от того, что эффективнее для результата, которого мы хотим достичь: повторно использовать потоки или создавать и уничтожать их. С небольшими асинхронными задачами `CachedThreadPool` должен работать быстрее, чем пул фиксированного размера. Но как обычно, если вы предполагаете, что разница эффективности будет существенна, нужно добросовестно протестировать быстродействие.

6.9.6. ScheduledThreadPoolExecutor

Последний пример исполнителя, который мы здесь рассмотрим, несколько отличается от других. Это класс `ScheduledThreadPoolExecutor`, иногда обозначаемый STPE:

```
ScheduledExecutorService pool = Executors.newScheduledThreadPool(4);
```

Обратите внимание: здесь явно указан возвращаемый тип `ScheduledExecutorService`, в то время как другие фабричные методы возвращают `ExecutorService`.

> **ПРИМЕЧАНИЕ** `ScheduledThreadPoolExecutor` — невероятно мощная разновидность исполнителя, которую можно использовать в широком диапазоне обстоятельств.

Планирование расширяет обычную функциональность исполнителя и дополняет ее новыми возможностями: методом `schedule()`, который запускает одноразовую задачу после заданной задержки, и двумя методами для планирования периодических (то есть повторяющихся) задач — `scheduleAtFixedRate()` и `scheduleWithFixedDelay()`.

Эти два метода ведут себя немного по-разному: `scheduleAtFixedRate()` активирует новые копии задачи с фиксированной периодичностью (и делает это вне зависимости от того, завершены ли предыдущие копии), тогда как `scheduleWithFixedDelay()` активирует новую копию задачи только после того, как предыдущий экземпляр завершился и истекло заданное время задержки.

Помимо `ScheduledThreadPoolExecutor`, все остальные пулы, которые упоминались выше, создаются выбором тех или иных параметров для достаточно общего класса `ThreadPoolExecutor`. Например, взгляните на следующее определение `Executors.newFixedThreadPool()`:

```
public static ExecutorService newFixedThreadPool(int nThreads) {
    return new ThreadPoolExecutor(nThreads, nThreads,
                                  0L, TimeUnit.MILLISECONDS,
                                  new LinkedBlockingQueue<Runnable>());
}
```

https://t.me/javalib

Конечно, вспомогательные методы именно для этого и нужны: они предоставляют удобный способ обращаться к некоторым стандартным настройкам пула потоков без необходимости погружаться во все сложности `ThreadPoolExecutor`. За пределами JDK есть много других примеров исполнителей и соответствующих пулов потоков, таких как класс `org.apache.catalina.Executor` из веб-сервера Tomcat.

ИТОГИ

- Для всего нового многопоточного кода стоит в первую очередь использовать классы `java.util.concurrent`, которые предоставляют удобные средства:
  - Атомарные целые числа.
  - Конкурентные структуры данных, в первую очередь `ConcurrentHashMap`.
  - Блокирующие очереди и защелки.
  - Пулы потоков и исполнители.
- С помощью этих классов можно реализовывать безопасные приемы конкурентного программирования, например:
  - Избегать проблем недостаточной гибкости синхронизированных блокировок.
  - Использовать блокирующие очереди для назначения задач.
  - Использовать защелки, чтобы согласовывать работу групп потоков.
  - Разбивать задачи на рабочие единицы.
  - Управлять заданиями, включая безопасное завершение.

https://t.me/javalib

7 Производительность кода на Java

В ЭТОЙ ГЛАВЕ

- ✓ Почему производительность важна
- ✓ Сборщик мусора G1
- ✓ JIT-компиляция
- ✓ JFR (JDK Flight Recorder)

Низкая производительность губит ПО: она отпугивает клиентов и плохо сказывается на репутации вашего приложения. Если только вы не контролируете весь рынок, то клиенты проголосуют ногами и уйдут к конкурентам. Чтобы низкая производительность не повредила вашему проекту, вы должны понимать, как ее анализировать и как эффективно использовать результаты анализа.

Анализ и оптимизация производительности — необъятная область, и в ней слишком часто концентрируются не на тех вещах. Поэтому мы сразу начнем с главного секрета настройки производительности. А звучит он так: *всегда измеряйте. Нельзя правильно оптимизировать без измерений.*

Объясним, почему это так. Пытаясь предположить, какие части системы особенно замедляют ее работу, человеческий мозг почти всегда ошибается. Так бывает со всеми. С вами, со мной, с Джеймсом Гослингом — у всех нас есть подсознательные предпочтения, и мы склонны видеть закономерности там, где их может и не быть. Собственно, правильный ответ на вопрос: «Какую

https://t.me/javalib

часть моего кода на Java нужно оптимизировать?» очень часто звучит так: «Никакую».

Представьте себе типичное (хотя и довольно консервативное) веб-приложение интернет-магазина, которое обслуживает аудиторию зарегистрированных клиентов. Оно использует базу данных SQL, веб-серверы со службами Java, а также довольно стандартную сетевую конфигурацию, которая связывает все это воедино. Часто бывает, что настоящее узкое место не относится к Java: например, это может быть база данных, файловая система или сеть. Однако без измерений Java-разработчик никогда об этом не узнает. Вместо того чтобы найти и исправить реальную проблему, он может попусту тратить время на микрооптимизацию участков кода, которые на самом деле мало на что влияют.

Вот несколько примеров фундаментальных вопросов, на которые вы должны уметь отвечать:

- Если начнется распродажа и количество клиентов внезапно увеличится в 10 раз, хватит ли у системы памяти?
- Какое среднее время реакции наблюдают пользователи вашего приложения?
- Какие результаты приложение показывает по сравнению с конкурентами?

Обратите внимание: все эти вопросы относятся к тем аспектам системы, которые непосредственно актуальны для клиентов — пользователей системы. В частности, здесь не идет речь о таких вопросах, как:

- Работают ли лямбда-выражения и потоки быстрее, чем циклы `for`?
- Работают ли обычные (виртуальные) методы быстрее, чем методы интерфейсов?
- Какова самая быстрая реализация `hashcode()`?

Неопытные специалисты по оптимизации часто совершают ошибку, предполагая, что производительность, которую видят пользователи, существенно зависит от факторов микропроизводительности, которые упоминаются во второй группе вопросов (или, по крайней мере, тесно связана с этими факторами).

Это чисто упрощенческое допущение не оправдывается на практике. На самом деле сложность современных программных систем приводит к тому, что общая производительность оказывается синергетическим свойством системы и всех ее уровней. Отдельные микроэффекты почти невозможно изолировать, а микрохронометраж приносит большинству прикладных программистов весьма ограниченную пользу.

Чтобы действительно оптимизировать производительность, необходимо прекратить «играть в угадайку» относительно того, что замедляет работу системы, причем «замедляет» в данном случае означает «влияет на впечатления пользователей». Нужно не гадать, а знать, а точно узнать можно только одним способом — измерить.

https://t.me/javalib

Также важно понимать, чем *не* является оптимизация производительности. А именно она не является:

- Подборкой трюков и полезных советов.
- Секретным рецептом.
- Магическим порошком, которым нужно посыпать проект на финальной стадии.

Будьте особенно осторожны с «трюками и полезными советами». Истина в том, что JVM — очень сложная и тонко оптимизированная среда, и без правильного контекста большинство таких советов оказываются бесполезными (а могут даже нанести вред). Кроме того, они быстро устаревают, потому что новые версии JVM все умнее подходят к оптимизации кода.

Анализ производительности — это на самом деле своего рода экспериментальная наука. Ваш код можно рассматривать как некий научный эксперимент, который получает входные данные и производит выходные данные — метрики производительности, которые показывают, насколько эффективно система выполняет порученную ей работу. Задача специалиста по производительности — изучить эти выходные данные и попытаться найти закономерности. Оптимизация должна стать направлением прикладной статистики, а не опираться на стариковские истории и фольклор.

Эта глава поможет вам войти в курс дела и представит введение в практику оптимизации производительности Java. Впрочем, это большая тема, а в книге хватит места только для того, чтобы изложить базовую теорию и расставить некоторые ориентиры. Мы постараемся ответить на фундаментальные вопросы:

- Почему производительность важна?
- Почему производительность сложно анализировать?
- Какие аспекты JVM могут усложнять оптимизацию?
- С каких позиций стоит рассматривать оптимизацию производительности и на какие практические подходы лучше ориентироваться?
- Какие факторы чаще всего замедляют работу программ?

Мы также кратко познакомим вас с двумя подсистемами JVM, которые играют наиболее важную роль в вопросах производительности:

- Подсистема сборки мусора.
- JIT-компилятор.

Этого должно быть достаточно для того, чтобы вы получили представление о предмете и начали применять новые (и, откровенно говоря, весьма насыщенные теорией) знания к реальным задачам, с которыми вы сталкиваетесь в своем коде.

https://t.me/javalib

Начнем с фундаментальных терминов, которые позволяют выразить и четко определить ваши проблемы и цели в отношении производительности.

7.1. ТЕРМИНОЛОГИЯ ПРОИЗВОДИТЕЛЬНОСТИ: ОСНОВНЫЕ ОПРЕДЕЛЕНИЯ

Чтобы вы получили максимальную пользу от этой главы, необходимо формализовать некоторые понятия производительности, которые вам, возможно, уже знакомы. Для начала дадим определения важнейших терминов из лексикона специалиста по производительности:

- Задержка (latency).
- Пропускная способность (throughput).
- Загруженность (utilization).
- Эффективность (efficiency).
- Емкость (capacity).
- Масштабируемость (scalability).
- Деградация (degradation).

Даг Ли рассматривает некоторые из этих терминов в применении к многопоточному коду, но мы здесь оперируем гораздо более широким контекстом. Говоря о производительности, мы можем иметь в виду что угодно — от одного многопоточного процесса до целого кластера служб, размещенных в облаке.

7.1.1. Задержка

Задержка (latency) — это время сквозной обработки одной единицы работы для заданной нагрузки. В простых случаях задержку оценивают только для «нормальной» рабочей нагрузки, но достаточно часто полезной метрикой производительности становится график, на котором задержка представлена как функция от растущей нагрузки.

График на рис. 7.1 показывает, как метрика производительности (например, задержка) внезапно, нелинейно снижается с ростом рабочей нагрузки. Обычно такая форма называется «коленом производительности» (или «клюшкой»).

7.1.2. Пропускная способность

Пропускная способность (throughput) — это количество рабочих единиц, которое система может обработать за некоторый период времени при заданных ресурсах. Часто приводится такой показатель, как количество операций в секунду на некоторой эталонной платформе, например на сервере конкретной марки с заданным оборудованием, ОС и программным стеком.

https://t.me/javalib

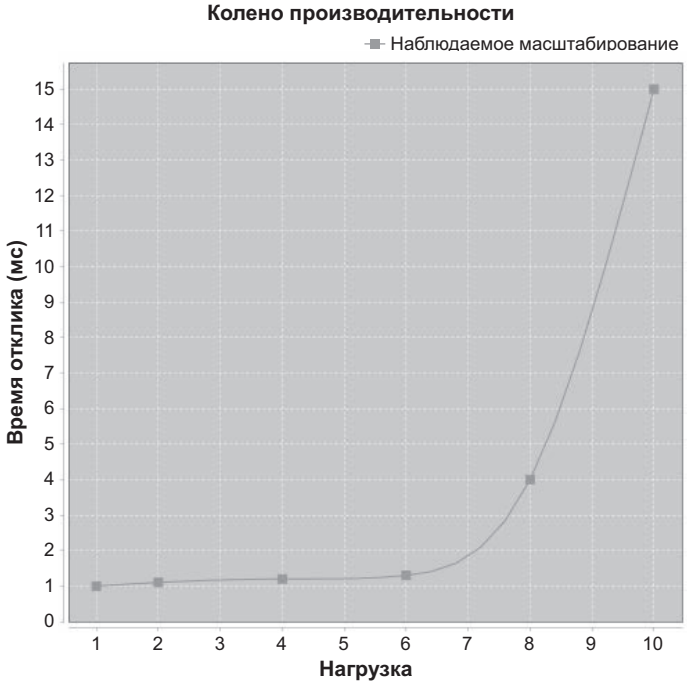


Рис. 7.1. Колено производительности

7.1.3. Загруженность

Загруженность (utilization) — это доля доступных ресурсов, которая занята обслуживанием единиц работы, а не служебными операциями (или простым бездействием). Обычно говорят, например, что сервер загружен на 10 %. Это показывает, какая часть ресурсов процессора расходуется на выполнение единиц работы в ходе штатной эксплуатации. Следует учитывать, что различия между уровнями загруженности разных ресурсов (скажем, процессора и памяти) могут быть очень значительными.

7.1.4. Эффективность

Эффективность (efficiency) системы — это пропускная способность, деленная на объем используемых ресурсов. Система менее эффективна, если она требует бóльших ресурсов, чтобы обеспечить ту же пропускную способность.

Для примера рассмотрим сравнение двух кластерных решений. Если при одинаковой пропускной способности решение A требует вдвое больше серверов, чем решение B, то его эффективность вдвое ниже, чем у B.

Обратите внимание, что ресурсы также можно оценивать по стоимости: если решение A обходится вдвое дороже, чем решение B (или требует вдвое больше персонала для управления средой эксплуатации), то его эффективность вдвое ниже.

https://t.me/javalib

7.1.5. Емкость

Емкость (capacity) — это количество единиц работы (например, транзакций), которые могут находиться в обработке в системе в любой момент времени. Иначе говоря, это объем одновременной обработки, который доступен при заданной задержке или пропускной способности.

7.1.6. Масштабируемость

Когда в систему добавляются новые ресурсы, ее пропускная способность (и задержка) изменяется. Характер этого изменения называется *масштабированием* (scalability) системы.

Если при удвоении количества доступных серверов в пуле пропускная способность решения А удваивается, то масштабирование получается идеально линейным. В большинстве ситуаций добиться идеального линейного масштабирования невероятно трудно — вспомните закон Амдала.

Также следует заметить, что масштабируемость системы зависит от многих факторов и не является постоянной. Иногда система масштабируется почти линейно до некоторой точки, а потом ее производительность начинает резко снижаться. Это другая разновидность «колена производительности».

7.1.7. Деградация

Если при неизменном количестве ресурсов добавлять новые единицы работы (или новые клиенты для сетевых систем), то наблюдаемая задержка или пропускная способность обычно меняется. Такие изменения отражают *деградацию* (снижение эффективности) системы под дополнительной нагрузкой.

При обычных обстоятельствах деградация отрицательна. Иначе говоря, добавление единиц работы отрицательно сказывается на производительности (например, увеличивает задержку обработки).

Впрочем, в определенных обстоятельствах деградация может быть положительной. Например, если из-за дополнительной нагрузки те или иные части системы превышают некоторый порог и переключаются в режим высокой производительности, это может привести к тому, что система начнет работать эффективнее, а время обработки сократится, несмотря на то что фактический объем работы увеличился. JVM — чрезвычайно динамичная система выполнения, и некоторые ее компоненты могут обусловливать эффекты такого рода.

Перечисленные метрики — наиболее часто используемые индикаторы производительности. Иногда встречаются и другие индикаторы, но это базовые статистические показатели системы, с помощью которых можно регулировать ее производительность. В следующем разделе мы представим подход, который основан на самом пристальном внимании к этим числам и настолько объективен, насколько это возможно.

https://t.me/javalib

7.2. ПРАГМАТИЧНЫЙ ПОДХОД К АНАЛИЗУ ПРОИЗВОДИТЕЛЬНОСТИ

Сталкиваясь с задачей анализа производительности, многие разработчики не до конца представляют себе, чего они хотят добиться в результате этого анализа. Очень часто в начале работы у разработчиков и руководства есть только смутное ощущение того, что «код должен работать быстрее».

Однако все совсем наоборот. Чтобы действительно эффективно оптимизировать производительность, необходимо продумать некоторые ключевые аспекты еще до того, как приступать к какой бы то ни было технической работе. Вы должны знать:

- Какие объективно наблюдаемые аспекты кода вы оцениваете.
- Как измерить эти наблюдаемые аспекты.
- Какие цели поставлены для наблюдаемых аспектов.
- Как определить, что оптимизация производительности завершена.
- Какие максимальные издержки (в отношении затрат времени разработчиков и дополнительной сложности кода) допускает оптимизация быстродействия.
- Чем нельзя жертвовать при оптимизации.

Самое важное, о чем неоднократно говорится в этой главе, — необходимы *объективные измерения*. Если вы не измеряете хотя бы один наблюдаемый аспект — это не анализ производительности.

Также часто бывает, что вы начинаете проводить измерения в своем коде и обнаруживаете, что время теряется совсем не там, где вы думали. Многие проблемы производительности могут объясняться нехваткой индекса в базе данных или конкуренцией за блокировки в файловой системе. Размышляя над оптимизацией кода, всегда следует помнить о том, что проблемы могут быть вообще не связаны с кодом. Чтобы определить, в чем конкретно они заключаются, прежде всего необходимо понимать, что вы измеряете.

7.2.1. Понимайте, что измеряете

Чтобы оптимизировать производительность, всегда нужно что-то измерять. Если вы не измеряете наблюдаемый аспект своего кода — это не оптимизация производительности. Если вы просто сидите и таращитесь в свой код, надеясь, что на вас вдруг снизойдет более быстрое решение, — это тоже не оптимизация производительности.

> **ПОДСКАЗКА** Хорошему специалисту по производительности необходимо понимать такие термины, как среднее, медиана, мода, дисперсия, процентиль, стандартное отклонение, размер выборки и нормальное распределение. Если эти понятия вам незнакомы, стоит поискать информацию в интернете и при необходимости обратиться к дополнительной литературе по прикладной статистике. Начните, например,

https://t.me/javalib

с главы 5 книги Леонарда Апельцина (Leonard Apeltsin) «Data Science Bookcamp»[1] (Manning, 2021, http://mng.bz/e7Oq).

Анализируя производительность, важно точно знать, какие из наблюдаемых аспектов, описанных в предыдущем разделе, представляют для вас интерес. Всегда связывайте измерения, цели и выводы с одной или несколькими базовыми метриками, упомянутыми выше. Вот некоторые типичные наблюдаемые аспекты, которые могут стать хорошими целями для оптимизации производительности:

- Среднее время выполнения метода `handleRequest()` (после разогрева).
- 90-й процентиль сквозной задержки системы с десятью одновременными клиентами.
- Деградация времени отклика с ростом количества одновременных пользователей от 1 до 1000.

Это все величины, которые можно объективно измерить — и, возможно, оптимизировать. Чтобы результаты изменений были точными и полезными, необходимы базовые знания статистики.

Понимать, что вы измеряете, и убеждаться, что ваши измерения точны, — хороший первый шаг. Однако нечеткие или изменчивые цели часто мешают получить полезный результат, в том числе в оптимизации производительности. Стоит оперировать целями производительности, которые относятся к категории, известной как SMART (от «Specific, Measurable, Agreed, Relevant, Time-boxed», то есть «конкретные, измеримые, согласованные, актуальные и ограниченные во времени»).

7.2.2. Как выполнять измерения

На самом деле есть только два способа точно определить, сколько времени займет выполнение метода или другого фрагмента кода на Java:

- Измерить время напрямую, встроив код измерения в исходный класс.
- Преобразовать класс, который требует измерений, во время его загрузки.

Эти два подхода называются *ручным* и *автоматическим измерительным анализом* соответственно. Все популярные методы измерения производительности опираются на один или оба этих подхода.

ПРИМЕЧАНИЕ Также существует инструментарий JVMTI (JVM Tool Interface), с помощью которого можно создавать очень изощренные средства контроля производительности, но у него есть свои недостатки: прежде всего JVMTI требует использовать низкоуровневый код, что усугубляет сложность и ухудшает безопасность написанных на нем средств.

[1] Леонард Апельцин. «Data Science в действии». СПб., издательство «Питер».

https://t.me/javalib

Прямые измерения

Прямые измерения — самый понятный механизм, но он требует вмешиваться в нормальную работу кода. В простейшей форме это выглядит так:

```
long t0 = System.currentTimeMillis();
methodToBeMeasured();
long t1 = System.currentTimeMillis();

long elapsed = t1 - t0;
System.out.println("methodToBeMeasured выполнялся "+ elapsed +" мс");
```

Этот код выводит информацию о том, сколько времени заняло выполнение `methodToBeMeasured()`, с точностью до миллисекунды. Неудобно здесь то, что этот код приходится добавлять в кодовую базу, а с ростом количества измерений станет трудно ориентироваться в потоке данных.

Есть и другие проблемы: например, что произойдет, если `methodToBeMeasured()` выполняется быстрее миллисекунды? Как мы увидим позднее в этой главе, также приходится учитывать возможные эффекты «холодного старта»: JIT-компиляция может привести к тому, что после нескольких первых запусков метода он будет выполняться быстрее.

Существуют и менее очевидные нюансы: `currentTimeMillis()` требует вызова низкоуровневого метода и системного вызова, чтобы обратиться к системным часам. Эти вызовы не только сами занимают время, но и могут вытеснить код из конвейеров выполнения, а это приведет к дополнительному ухудшению быстродействия, которого не было бы без кода измерения.

Автоматический измерительный анализ во время загрузки класса

В главах 1 и 4 мы обсуждали, как из классов формируется исполняемая программа. Один из ключевых этапов, о котором часто забывают, — преобразование байт-кода при загрузке. Это чрезвычайно мощная возможность, которая лежит в основе многих современных средств платформы Java.

Один из примеров такого рода — автоматический измерительный анализ методов. При таком подходе `methodToBeMeasured()` загружается специальным загрузчиком классов, который добавляет в начало и в конец метода байт-код, регистрирующий моменты времени входа и выхода из метода. Как правило, эти данные обычно записываются в общую структуру, к которой обращаются другие потоки, — обычно чтобы либо записать вывод в журнал, либо связаться с сетевым сервером, который обрабатывает «сырые» данные.

Этот механизм лежит в основе многих коммерческих средств мониторинга производительности Java (таких, как New Relic), но активно сопровождаемые средства с открытым исходным кодом в этой нише встречаются нечасто. Возможно, ситуация изменится с развитием библиотек и стандартов OpenTelemetry OSS, а также соответствующего подпроекта измерительного анализа Java.

https://t.me/javalib

ПРИМЕЧАНИЕ Как обсуждалось ранее, методы Java сначала интерпретируются, а затем переключаются в компилируемый режим. Чтобы получить настоящие показатели производительности, нужно отбросить время, проведенное в интерпретируемом режиме, потому что оно может сильно исказить результаты. Позднее мы подробнее обсудим, как узнать, когда метод переключается в компилируемый режим.

Используя один или оба этих способа, вы получите числовые данные, которые показывают, сколько времени занимает выполнение конкретного метода. Следующий вопрос: как должны выглядеть эти показатели после настройки?

7.2.3. Установите цели по быстродействию

Ничто так не помогает сосредоточиться, как четко поставленная цель, поэтому ясно выразить конечную цель оптимизации не менее важно, чем знать, что именно измерять. В большинстве случаев ориентир стоит сформулировать просто и точно, например:

- Сократить 90-й процентиль сквозной задержки на 20 % при десяти одновременно работающих пользователях.
- Сократить среднюю задержку `handleRequest()` на 40 %.

В более сложных случаях цель может состоять из нескольких взаимосвязанных критериев производительности. Имейте в виду, что чем меньше связаны друг с другом показатели, которые вы измеряете и оптимизируете, тем сложнее может оказаться ваша задача. Оптимизация по одному показателю может отрицательно отразиться на другом.

Иногда необходимо провести предварительный анализ, например, выявить самые важные методы до того, как задавать цели (допустим, заставить их выполняться быстрее). Не забывайте, что после предварительных исследований почти всегда лучше остановиться и сформулировать цели, прежде чем пытаться их достичь. Слишком часто разработчики увлекаются анализом и не останавливаются, чтобы прояснить свои цели.

7.2.4. Знайте, когда остановиться

Теоретически несложно определить, когда пора завершать оптимизацию: она закончена, когда вы достигли своих целей. Однако на практике нас часто затягивает в водоворот оптимизации. Если она идет хорошо, то искушение продолжать и добиться еще лучших результатов может оказаться слишком сильным. И наоборот, если оптимизация продвигается с трудом, многие разработчики продолжают перебирать все больше разных стратегий, надеясь достичь целевых показателей.

Чтобы знать, когда остановиться, нужно понимать не только свои цели, но и их ценность. Часто бывает достаточно достичь 90 % цели по производительности, после чего время специалистов можно с большей пользой потратить на другие задачи.

Другой важный фактор — сколько усилий уходит на то, чтобы оптимизировать редко используемые пути в коде. Оптимизировать код, на который приходится

https://t.me/javalib

1 % или менее от времени выполнения программы, — это почти всегда напрасный труд, хотя на удивление много разработчиков этим занимаются.

Вот очень простые рекомендации о том, как узнать, что надо оптимизировать. Возможно, вам понадобится адаптировать их для своих конкретных целей, но они вполне эффективны в широком диапазоне ситуаций:

- Оптимизируйте то, что важно, а не то, что проще оптимизировать.
- Начните с самых важных методов — обычно это методы, которые чаще всего вызываются.
- Если что-то удается оптимизировать без особых усилий, этим можно заняться, однако учитывайте, насколько часто вызывается соответствующий код.

В конце проведите еще один цикл измерений. Если вы не достигли целей по производительности, критически оцените ситуацию. Посмотрите, насколько вы приблизились к целевым показателям и насколько внесенные улучшения повлияли на общую производительность.

7.2.5. Учитывайте цену повышения быстродействия

Оптимизация производительности никогда не обходится даром. Цена бывает разной:

- Время, затраченное на анализ и разработку улучшения. (Стоит помнить, что время разработчиков — это почти всегда самые значительные издержки в любом программном проекте.)
- Дополнительная техническая сложность, которая появилась в результате оптимизации. (Существуют улучшения производительности, которые упрощают код, но они встречаются не так часто.)
- Дополнительные потоки, которые вводятся в систему для вспомогательных задач, чтобы основные потоки выполнялись быстрее. Эти новые потоки могут привести к непредвиденным эффектам в системе при более высокой нагрузке.

Какой бы ни была цена, обращайте на нее внимание и постарайтесь ее оценить, прежде чем завершать цикл оптимизации.

Часто полезно представлять себе максимальные затраты, на которые вы готовы ради повышения производительности. Например, это могут быть ограничения по времени, которое тратится на оптимизацию, или предельное количество дополнительных классов или строк кода. Допустим, разработчик может решить, что на оптимизацию можно потратить не более недели или что оптимизированные классы не должны увеличиться более чем на 100 % (то есть в два раза относительно исходного размера).

7.2.6. Преждевременная оптимизация опасна

Одна из самых знаменитых цитат про оптимизацию принадлежит Дональду Кнуту (статья *Structured Programming with go to Statements*, «Структурное

https://t.me/javalib

программирование с инструкциями go to» из журнала «Computing Surveys», вып. 6, № 4 (декабрь 1974)):

> *Программисты тратят огромное количество времени на размышления или беспокойство о скорости некритичных частей своих программ, и эти попытки повысить эффективность на самом деле оказывают огромное негативное влияние... Преждевременная оптимизация — корень всех зол.*

Вокруг этого утверждения кипело немало споров, и часто помнят только вторую его часть. Это прискорбно по нескольким причинам:

- В первой части цитаты Кнут неявно напоминает о том, что необходимы измерения, без которых мы не узнаем, какие части программы являются критическими.
- Следует еще раз напомнить, что причина задержки может быть вовсе не в коде, а где-то в другом месте системы.
- В полной цитате хорошо видно, что Кнут говорит об оптимизации как о сознательной, скоординированной деятельности.
- Сокращенную форму цитаты часто используют как удобную отговорку для неудачных решений по поводу архитектуры или исполнения.

Некоторые принципы оптимизации, и особенно перечисленные ниже, относятся к факторам хорошего стиля программирования:

- Не создавайте объект, который вам не нужен.
- Удаляйте отладочные сообщения, которые вы не используете.

В следующем фрагменте кода мы проверяем, делает ли объект `log` что-нибудь полезное с отладочным сообщением. Если подсистема ведения журнала не настроена для отладочного вывода, то этот код не будет создавать сообщение, что сэкономит затраты на вызов `currentTimeMillis()` и на конструирование объекта `StringBuilder`, в котором хранится сообщение:

```
if (log.isDebugEnabled()) {
    log.debug("Бесполезное сообщение от : "+ System.currentTimeMillis());
}
```

Но если отладочное сообщение действительно бесполезно, можно полностью удалить этот код и тем самым сэкономить несколько тактов процессора (затраты на проверку). Эти затраты мизерны, и они потеряются в шуме остального профиля производительности, но если код действительно не нужен, его лучше удалить.

Оптимизировать производительность легче, если изначально писать качественный, эффективный код. Для этого стоит хорошо знать платформу и разбираться в ее внутреннем поведении (например, учитывать неявное создание объектов при конкатенации двух строк), а также анализировать аспекты производительности.

Итак, у нас есть базовая терминология, с помощью которой можно сформулировать проблемы с производительностью и цели оптимизации, а также описать

https://t.me/javalib

подход к решению проблем. Но мы все еще не разобрались, почему этой проблемой должны заниматься разработчики и откуда происходит эта необходимость. Чтобы разобраться, придется кратко погрузиться в аппаратную часть.

7.3. ЧТО ПОШЛО НЕ ТАК? ПОЧЕМУ МЫ ДОЛЖНЫ ЭТИМ ЗАНИМАТЬСЯ?

В безмятежную пору, где-то до середины 2000-х, нам казалось, что производительность перестала быть серьезной проблемой. Тактовые частоты стремились в заоблачную высь, и создавалось впечатление, что разработчику стоит лишь подождать несколько месяцев, пока скорости процессоров не повысятся настолько, чтобы даже плохо написанный код выполнялся быстро.

Когда же все пошло не по плану? Почему тактовые частоты не растут в прежнем темпе? И что озадачивает еще сильнее, почему компьютер с чипом на 3 ГГц не кажется более быстрым, чем компьютер с чипом на 2 ГГц? Откуда взялась эта тенденция — перекладывать на разработчиков ответственность за производительность?

В этом разделе мы поговорим о факторах, которые определяют эту тенденцию, и о том, почему даже самому высокоуровневому разработчику приходится хотя бы немного учитывать аппаратную часть. Вы подготовитесь к материалу оставшейся части главы и познакомитесь с понятиями, которые вам понадобятся, чтобы разобраться в JIT-компиляции и некоторых продвинутых примерах.

Вероятно, вам уже доводилось слышать о «законе Мура». Многие разработчики знают, что он как-то связан с темпом роста вычислительной мощности компьютеров, но обычно смутно представляют себе подробности. Давайте разберемся, в чем же заключается этот закон и каковы последствия того, что он может перестать работать в ближайшем будущем.

7.3.1. Закон Мура

Закон Мура назван в честь Гордона Мура, одного из основателей Intel. Самая распространенная формулировка закона выглядит так: «*Максимальное количество транзисторов на микросхеме, производство которой экономически оправданно, удваивается приблизительно каждые два года*».

Закон, который на самом деле представляет собой эмпирическое наблюдение о тенденциях в производстве компьютерных процессоров, основан на статье Мура 1965 года, в которой он давал прогноз на 10 лет, то есть до 1975 года. То, что закон продолжал действовать гораздо дольше, — воистину поразительно.

На рис. 7.2 представлены данные о реальных микропроцессорах из разных семейств (прежде всего Intel x86) с 1980 года до новейшего (на момент написания этой книги в 2021 году) Apple Silicon. (Данные взяты из Википедии и незначительно отредактированы для ясности.) График показывает взаимосвязь между количеством транзисторов на микросхемах и датой их выпуска.

https://t.me/javalib

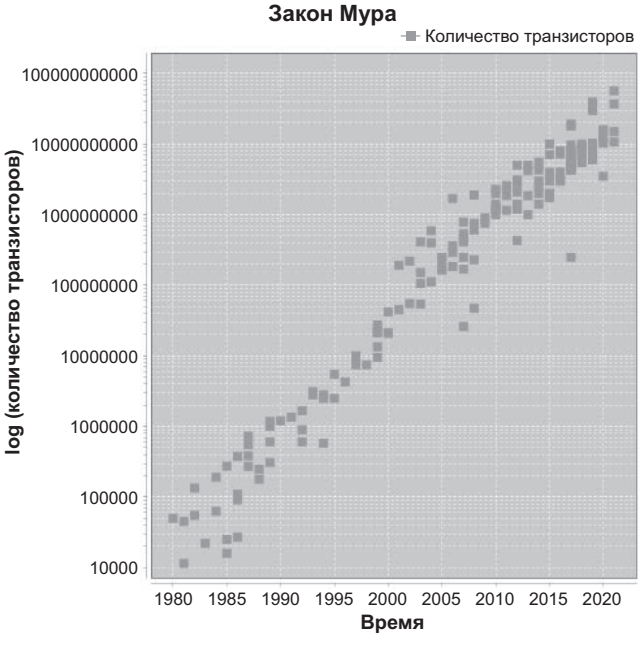


Рис. 7.2. Логарифмически-линейный график зависимости количества транзисторов на микросхемах от времени

График является логарифмически-линейным, то есть каждое деление оси *y* обозначает 10-кратное увеличение. Как видите, данные выстроились практически на прямой линии, а через каждые 6–7 лет эта линия пересекает очередной вертикальный уровень. В этом проявляется закон Мура, потому что увеличение в 10 раз за 6–7 лет — это примерно то же самое, что удвоение за каждые два года.

Еще раз обратите внимание, что ось *y* на графике — логарифмическая. Например, массовая микросхема Intel, произведенная в 2005 году, содержала около 100 миллионов транзисторов, и это приблизительно *в 100 раз* больше, чем содержала микросхема, произведенная в 1990 году.

Очень важно, что в законе Мура речь идет именно о количестве транзисторов. Это основное положение, которое объясняет, почему одного лишь этого закона недостаточно, чтобы разработчики продолжали решать свои проблемы за счет производителей оборудования (см. Герб Саттер (Herb Sutter), *The Free Lunch Is Over: A Fundamental Turn Toward Concurrency in Software*, «Бесплатный сыр закончился. Фундаментальный поворот к конкурентности в ПО», «Dr. Dobb's Journal», № 30 (2005), с. 202–210).

Закон Мура был хорошим ориентиром в прошлом, но он касается количества транзисторов, а это количество не может служить адекватной метрикой производительности, которой разработчикам стоит ожидать от своего кода. Как мы вскоре увидим, в реальности дела обстоят сложнее.

https://t.me/javalib

ПРИМЕЧАНИЕ Количество транзисторов — не то же самое, что тактовая частота. И даже все еще распространенное мнение о том, что чем выше тактовая частота, тем лучше производительность, — это чрезмерное упрощение.

Истина в том, что реальная производительность зависит от множества факторов, и все они важны. Но если бы нам пришлось выбрать один фактор, то мы бы описали его так: насколько быстро программа может найти данные, которые нужны для ближайших выполняемых инструкций? Этот вопрос настолько важен для производительности, что его стоит рассмотреть подробнее.

7.3.2. Иерархия задержек памяти

Чтобы компьютерный процессор работал, ему нужны данные. Если нет данных, которые можно обрабатывать, то тактовая частота процессора становится несущественной: процессору приходится просто ожидать, выполняя пустую операцию (NOP) и, по сути, простаивая, пока не поступят данные.

Это означает, что если мы говорим о задержке, то необходимо рассматривать два самых фундаментальных вопроса:

1. Где находится ближайшая копия данных, которые нужны ядру процессора, чтобы продолжать работу?
2. Насколько быстро данные можно перенести туда, где ядро может их использовать?

Перечислим основные места размещения данных (в так называемой *архитектуре фон Неймана*, самой распространенной из используемых):

- *Регистры* — области памяти, которые находятся непосредственно на процессоре и готовы к немедленному использованию. Инструкции процессора обращаются к этой памяти напрямую.
- *Оперативная память* — обычно DRAM (динамическая память с произвольным доступом). Время доступа для этой памяти составляет около 50 нс (но позднее мы подробнее расскажем, как кэши процессора помогают сократить эту задержку).
- *Твердотельные диски*, или *SSD*, — обращение к этим дискам занимает не более 0,1 мс, но они все еще остаются дороже, чем традиционные жесткие диски.
- *Жесткий диск* — обращение к диску и загрузка запрошенных данных в основную память занимает около 5 мс.

Закон Мура описывал экспоненциальный рост количества транзисторов, однако и возможности памяти развивались по похожей схеме: скорость доступа к памяти тоже экспоненциально росла. Но эти две экспоненты не были одинаковыми. Скорость памяти увеличивалась медленнее, чем на процессорах добавлялись транзисторы, и в результате могло получиться так, что ядра процессоров начали бы простаивать из-за того, что данные для обработки поступают слишком медленно.

https://t.me/javalib

Чтобы решить эту проблему, между регистрами и основной памятью были добавлены кэши — небольшие блоки более быстрой памяти (статической оперативной памяти SRAM вместо DRAM). SRAM стоит намного дороже DRAM и требует больше транзисторов, поэтому она не используется в качестве основной памяти компьютеров.

Кэши обозначаются L1 и L2 (на некоторых машинах бывает еще L3). Чем меньше цифра, тем ближе кэш физически находится к ядру и тем быстрее он работает. Мы подробнее поговорим про кэши в разделе 7.6 (посвященном JIT-компиляции), где будет приведен пример, который показывает, насколько существенно кэш L1 влияет на выполнение кода. На рис. 7.3 показана относительная скорость доступа к кэшам и другим областям памяти.



Рис. 7.3. Относительное время доступа (в тактах) для регистров, кэшей процессора и основной памяти

Наряду с добавлением кэшей в 1990-х и начале 2000-х широко применялась другая практика: чтобы преодолеть задержку памяти, процессоры оснащались все более сложной функциональностью. Изощренные аппаратные механизмы, такие как параллелизм на уровне команд (ILP) и многопоточность на уровне микросхемы (CMT), использовались для того, чтобы процессор продолжал своевременно получать данные даже в условиях расширяющегося разрыва между возможностями процессора и задержкой памяти.

Эти механизмы начали потреблять значительную долю транзисторной базы процессора, но практически не влияли на реальную производительность. В результате стало преобладать мнение, согласно которому будущее проектирования процессоров связано с многоядерными микросхемами. Все современные процессоры фактически являются многоядерными, и это одно из вторичных следствий из закона Мура: возрастающее количество ядер позволяет эффективнее использовать доступные транзисторы.

Тема производительности в обозримой перспективе тесно связана с конкурентным выполнением: один из основных способов повышения общей производительности системы заключается в том, чтобы использовать больше ядер. В этом случае, даже если одно ядро ожидает данные, остальные тем временем могут продолжать работать (но не забывайте о влиянии закона Амдала, который рассматривался в главе 5). Эта связь настолько важна, что мы повторим еще раз:

- Практически все современные процессоры являются многоядерными.
- Производительность и конкурентность тесно связаны друг с другом.

https://t.me/javalib

Мы лишь в самых общих чертах описали принципы компьютерных архитектур, которые имеют отношение к программированию на Java. Любознательному читателю, который захочет узнать больше, стоит обратиться к специализированной литературе, например Хеннесси и др. (Hennessy et al.) «Computer Architecture: A Quantitative Approach, 6th edition» (Morgan Kaufmann, 2017).[1]

Аппаратные факторы касаются не только программирования на Java, однако управляемость JVM создает дополнительные сложности. Рассмотрим их в следующем разделе.

7.4. ПОЧЕМУ ТАК СЛОЖНО ОПТИМИЗИРОВАТЬ ПРОИЗВОДИТЕЛЬНОСТЬ КОДА НА JAVA?

Оптимизировать производительность JVM (а на самом деле и любой другой управляемой среды выполнения) принципиально сложнее, чем производительность неуправляемого кода. Вся суть управляемой системы заключается в том, чтобы среда выполнения могла в определенной степени контролировать окружение, в результате чего разработчику не приходится возиться с каждой мелочью. При этом общая производительность труда программиста значительно повышается, но с другой стороны, программист частично теряет контроль над происходящим.

Такое смещение акцентов мешает рассматривать систему в целом, потому что управляемая исполнительная среда оказывается «черным ящиком» для разработчика. Альтернативный подход состоит в том, чтобы отказаться от всех преимуществ управляемой исполнительной среды, как, например, поступают программисты на C/C++, которым приходится практически все делать самостоятельно. В этом случае ОС обеспечивает только простейшие возможности например рудиментарное планирование потоков, работа с которым всегда отнимает значительно больше времени, чем отдельная оптимизация производительности.

Вот некоторые важные аспекты платформы Java, которые усложняют оптимизацию:

- Планирование потоков.
- Сборка мусора (GC).
- JIT-компиляция.

Эти аспекты могут взаимодействовать друг с другом неочевидными способами. Например, подсистема компиляции использует таймеры, чтобы принять решение, какие методы следует компилировать. А какие методы станут кандидатами для компиляции, это может зависеть от таких факторов, как планирование и сборка мусора. Набор компилируемых методов может изменяться при каждом запуске.

[1] Хеннеси Дж. и др. «Компьютерная архитектура. Количественный подход» (на русском языке выходило 5-е издание).

https://t.me/javalib

Как вы уже видели в этом разделе, точные измерения — важнейший фактор, от которого зависят решения при анализе производительности. Таким образом, если вы хотите грамотно оптимизировать производительность, полезно понимать подробности и ограничения того, как платформа Java управляет временем.

7.4.1. Роль времени в оптимизации производительности

Чтобы оптимизировать производительность, необходимо знать, как интерпретировать данные, собранные в процессе выполнения кода. А для этого, в свою очередь, важно понимать ограничения, которые присущи любым измерениям времени на платформе.

Погрешность

Временны́е величины обычно регистрируются с точностью до ближайшей единицы некоторой шкалы. Цена деления этой шкалы соответствует *погрешности* измерений. Например, время часто измеряется с погрешностью 1 мс. Погрешность мала, если повторные измерения дают малый разброс вокруг одного значения.

Погрешность характеризует случайный шум, который содержится в том или ином измерении. Будем считать, что измерения для конкретного блока кода имеют нормальное распределение. В этом случае типичный способ описания погрешности заключается в том, чтобы указать ширину 95-процентного доверительного интервала.

Точность

Точность измерений (в нашем случае — измерений времени) показывает, насколько полученное значение близко к истинному. В реальности истинное значение обычно неизвестно, так что оценить точность сложнее, чем погрешность.

Точность — это мера систематической ошибки в измерениях. Бывает, что измерения проводятся с высокой точностью и при этом с большой погрешностью (данные состоятельны, но на них накладывается случайный шум среды). Бывают и результаты с малой погрешностью, но неточные (далекие от истинного значения).

Интерпретация измерений

Если интервал указан как 5945 нс с погрешностью до 1 нс, однако получен от таймера с точностью 1 мкс, истинное значение на самом деле лежит в интервале 3945–7945 нс (с вероятностью 95%). Остерегайтесь показателей производительности, которые выглядят слишком точными; всегда проверяйте как точность, так и погрешность измерений.

Гранулярность

Истинная *гранулярность* системы определяется частотой самого быстрого таймера — скорее всего, это таймер прерываний (в диапазоне 10 нс). Иногда эта характеристика называется *различимостью* (distinguishability) и описывается

https://t.me/javalib

как кратчайший интервал между двумя событиями, о которых можно определенно сказать, что они произошли «достаточно близко по времени, но не одновременно».

Имея дело с разными уровнями ОС, JVM и библиотечного кода, становится практически невозможно различить эти чрезвычайно короткие промежутки времени. В большинстве случаев они недоступны для разработчиков приложений.

Хронометраж в распределенных сетевых конфигурациях

Наш материал об оптимизации производительности в первую очередь касается систем, где вся обработка выполняется на одном узле. Однако имейте в виду, что в распределенных сетевых системах может добавиться целый ряд специфических проблем. Синхронизация и хронометраж в сети — весьма нетривиальная задача, и не только в интернете; эти проблемы проявляются даже в локальных сетях.

Полное обсуждение распределенного хронометража выходит за рамки книги, но вы должны понимать, что в общем случае трудно точно хронометрировать рабочие процессы, которые охватывают несколько вычислительных узлов. Кроме того, даже стандартные протоколы, такие как NTP, могут оказаться недостаточно точными для прецизионных измерений.

Вспомним самое важное о хронометраже в Java:

- Во многих системах работает несколько разных часов.
- Хронометраж с точностью до миллисекунды надежен и безопасен.
- С временны́ми данными высокой точности следует работать осторожно, чтобы избежать смещения.
- Важно помнить о точности и погрешности измерений времени.

Прежде чем переходить к теме сборки мусора, рассмотрим пример, который уже упоминался ранее и демонстрирует, как кэши влияют на производительность кода.

7.4.2. Промахи кэша

Один из главных факторов, которые ухудшают производительность для многих фрагментов кода с высокой пропускной способностью, — количество промахов кэша L1 при выполнении кода приложения.

В листинге 7.1 мы перебираем 2-мегабайтный массив и выводим время, которое тратится на выполнение одного из двух циклов. Первый цикл увеличивает на 1 каждый 16-й элемент массива `int[]`. 64 байта почти всегда укладываются в строку кэша L1 (а тип `int` в Java состоит из 4 байт), так что код по одному разу обращается к каждой строке кэша.

Обратите внимание: чтобы получить точные результаты, необходимо провести разогрев кода, чтобы JVM откомпилировала интересующие вас методы. Мы подробнее рассмотрим разогрев JIT-компилятора позднее в этой главе.

https://t.me/javalib

Листинг 7.1. Анализ промахов кэша

```
public class Caching {
    private final int ARR_SIZE = 2 * 1024 * 1024;
    private final int[] testData = new int[ARR_SIZE];

    private void touchEveryItem() {
        for(int i=0;i< testData.length;i=i+1) {
            testData[i] = testData[i] + 1;   <-- Обращается к каждому элементу
        }
    }

    private void touchEveryLine() {
        for(int i=0;i< testData.length;i=i+16) {
            testData[i] = testData[i] + 1;   <-- Обращается к каждой строке кэша
        }
    }

    private void run() {
        for(int i=0;i< 10_000;i=i+1) {   <-- Разогревает код
            touchEveryLine();
            touchEveryItem();
        }
        System.out.println("Строка   Элемент");
        for(int i=0;i< 100; i =i+1) {
            long t0 = System.nanoTime();
            touchEveryLine();
            long t1 = System.nanoTime();
            touchEveryItem();
            long t2 = System.nanoTime();
            long el1 = t1 - t0;
            long el2 = t2 - t1;
            System.out.println("Строка: "+ el1 +" ns ; Элемент: "+ el2);
        }
    }

    public static void main(String[] args) {
        Caching c = new Caching();
        c.run();
    }
}
```

Вторая функция `touchEveryItem()` увеличивает каждый байт в масиве, так что она выполняет в 16 раз больше работы, чем `touchEveryLine()`. Однако посмотрим на результат на типичном ноутбуке:

```
Строка: 487481 ns ; Элемент: 452421
Строка: 425039 ns ; Элемент: 428397
Строка: 415447 ns ; Элемент: 395332
Строка: 372815 ns ; Элемент: 397519
Строка: 366305 ns ; Элемент: 375376
Строка: 332249 ns ; Элемент: 330512
```

Результаты показывают, что `touchEveryItem()` выполняется не в 16 раз дольше, чем `touchEveryLine()`. Всему виной время передачи данных из основной памяти

https://t.me/javalib

в кэш процессора — оно доминирует в общем профиле производительности. В функциях `touchEveryLine()` и `touchEveryItem()` одинаковое количество операций чтения строк кэша, а время передачи данных заметно превалирует над тактами, которые тратятся непосредственно на модификацию данных.

> **ПРИМЕЧАНИЕ** Этот пример поднимает важный вопрос: необходимо сформировать хотя бы общее понимание (или ментальную модель) того, как процессор на самом деле расходует свое время.

Нашей следующей темой станет подсистема сборки мусора платформы Java. Это один из самых важных компонентов общей картины производительности, и его можно в определенной степени регулировать, что принесет вам неоценимую пользу при анализе производительности.

7.5. СБОРКА МУСОРА

Автоматическое управление памятью — одна из важнейших особенностей платформы Java. До того как появились управляемые платформы (такие, как Java и .NET), разработчикам приходилось тратить значительную долю своего времени на поиск дефектов, которые возникли из-за неаккуратной работы с памятью.

Однако за последние годы средства автоматического управления памятью стали настолько надежными и совершенными, что про них часто забывают: многие Java-разработчики не знают о том, как работают платформенные средства управления памятью, как их можно регулировать и как выполнять оптимизацию в пределах ограничений фреймворка.

И это можно рассматривать как признак успеха Java. Разработчики не разбираются в подробностях того, как устроена память и сборка мусора, потому что обычно им *не нужно* в этом разбираться. В большинстве приложений JVM неплохо справляется с управлением памятью без специальной настройки.

Что же делать в ситуации, где все-таки нужна оптимизация? Для начала необходимо понять, чем на самом деле занимается JVM, когда управляет памятью за вас. В этом разделе рассматривается базовая теория, в том числе:

- Управление памятью в работающих процессах Java.
- Основы механизма пометки и очистки при сборке мусора.
- Сборщик мусора G1, используемый в Java по умолчанию с момента выхода Java 9.

Начнем с основ.

7.5.1. Основы

У стандартного процесса Java есть и стек, и куча. В *стеке* хранятся локальные переменные. Если они содержат примитивы, то примитивное значение напрямую размещается в стеке.

https://t.me/javalib

ПРИМЕЧАНИЕ Примитивы — это последовательности битов, которые интерпретируются в соответствии с типом, так что два байта 00000000 01100001 будут интерпретироваться как a, если тип данных — `char`, или как 97, если тип данных — `short`.

С другой стороны, локальные переменные ссылочных типов указывают на область в *куче,* где фактически будут создаваться объекты. На рис. 7.4 показано, где находится память для переменных различных типов.

Обратите внимание, что память для примитивных полей объекта по-прежнему выделяется по адресам внутри кучи. Когда программа на Java выполняется, новые объекты создаются в куче, а связи между объектами изменяются (при обновлении полей). Со временем в куче кончится место для создания новых объектов. Однако многие объекты, которые создавались ранее, становятся ненужными: например, это временные объекты, которые были созданы в каком-либо методе и не передавались другому методу, не возвращались на сторону вызова и т. д.

Таким образом, память в куче можно освободить, чтобы программа могла продолжить работу. Платформа Java умеет освобождать и повторно использовать память в куче, если приложение больше не задействует эту память, — этот механизм называется *сборкой мусора*.

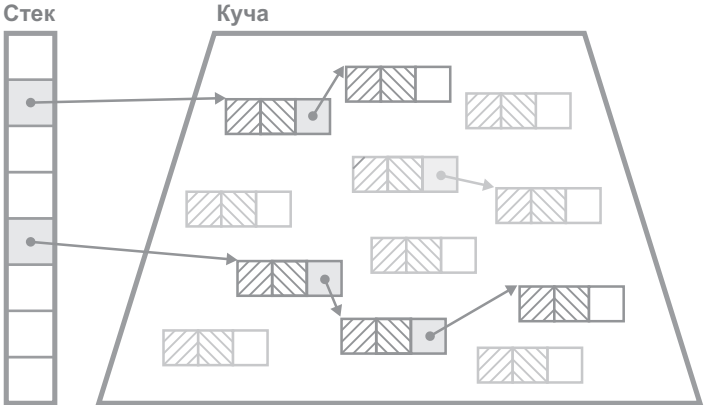


Рис. 7.4. Переменные в стеке и куче

7.5.2. Пометка и очистка

Хорошим примером простого алгоритма сборки мусора является *пометка и очистка* (mark and sweep). Собственно, это был первый из разработанных алгоритмов такого рода (он появился в LISP 1.5, выпущенном в 1965 году).

ПРИМЕЧАНИЕ Существуют и другие механизмы управления памятью, например алгоритм подсчета ссылок, который используется в языках вроде Perl. Он выглядит проще, чем пометка и очистка (по крайней мере на первый взгляд), но в действительности не выполняет сборку мусора (по данным статьи Гая Л. Стила (Guy L. Steele) *Multiprocessing Compactifying Garbage Collection*, «Многопроцессная компактифицирующая сборка мусора», из журнала «Communications of the ACM», вып. 18, № 9 (сентябрь 1975)).

https://t.me/javalib

В простейшей форме алгоритм пометки и очистки приостанавливает все выполняемые потоки и для начала просматривает набор объектов, которые заведомо являются «живыми», — это объекты, на которые есть ссылка в любом кадре стека любого пользовательского потока (вне зависимости от того, хранится ли эта ссылка в локальной переменной, параметре метода, временной переменной или в какой-то более экзотической форме). Затем алгоритм обходит дерево ссылок от живых объектов и помечает любой встреченный объект как живой. Когда обход и пометка завершены, все оставшиеся данные считаются мусором, и сборщик мусора может их уничтожить. Обратите внимание, что очищенная память возвращается в JVM, но не обязательно в ОС.

А как же недетерминированная пауза?

Одна из претензий в адрес Java (и других сред, например .NET) заключается в том, что сборка мусора в форме пометки и очистки неизбежно ведет к паузе типа «Остановите Землю» (*STW*, Stop-the-World). Это состояние, в котором все пользовательские потоки ненадолго приостанавливаются, в результате чего возникают паузы с недетерминированной продолжительностью.

Эту проблему часто преувеличивают. В сегменте серверного ПО существует очень мало приложений, для которых играет роль длительность пауз, вызванных сборкой мусора в современных версиях Java. Например, в Java 11 и выше стандартный сборщик мусора является конкурентным и выполняет бóльшую часть своей работы параллельно с потоками приложений, что сводит продолжительность пауз к минимуму.

> **ПРИМЕЧАНИЕ** Разработчики иногда придумывают хитроумные схемы, с помощью которых можно предотвратить паузы или полностью освободить память. Таких схем почти всегда следует избегать, потому что от них обычно больше вреда, чем пользы.

Платформа Java добавила ряд улучшений к базовому алгоритму пометки и очистки. Одно из простейших улучшений — сборка мусора *с поколениями*. Этот механизм рассматривает кучу не как однородную область памяти, а как несколько областей, которые задействованы в жизненном цикле объекта Java.

Во время сборки мусора объект может перемещаться из одной области в другую в зависимости от того, как долго он существует. В разные моменты его существования ссылки на него могут относиться к разным областям памяти (как показано на рис. 7.5).

Такая архитектура памяти и перемещение объектов обусловлены результатами анализа реальных систем, который показал, что объекты в основном живут либо очень долго, либо совсем недолго. Куча делится на разные области памяти, чтобы платформа могла использовать этот факт и отделять объекты-долгожители от всех остальных.

Учтите, что на рис. 7.5 изображена упрощенная схема кучи, которая поясняет принцип областей поколений объектов. Реальная куча Java устроена сложнее

https://t.me/javalib

и зависит от того, какой сборщик мусора используется, — мы поговорим об этом далее в этой главе.

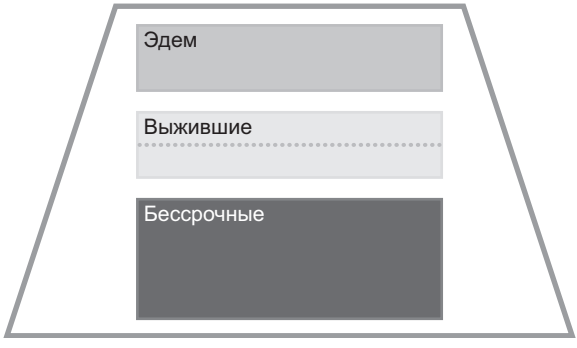


Рис. 7.5. Области памяти

7.5.3. Области памяти

JVM хранит объекты на протяжении их естественного жизненного цикла в следующих областях памяти:

- *Эдем* (Eden) — область кучи, в которой изначально создаются все объекты. Многие объекты проводят всю жизнь только в этой области.
- *Выжившие* (Survivor) — в эти области перемещаются объекты, которые сохранились после цикла сборки мусора (отсюда и название). Изначально они переносятся из Эдема, но также могут перемещаться между областями выживших при последующих сборках мусора.
- *Бессрочные* (Tenured) — область «старого поколения», в которую перемещаются (из областей выживших) объекты, которые считаются «достаточно старыми». Сборка мусора в более «молодых» областях не затрагивает область бессрочных.

Как упоминалось ранее, эти области памяти по-разному ведут себя при сборке мусора. Например, области выживших фактически служат универсальным «приемником-распределителем», который позволяет правильно обрабатывать объекты с коротким сроком жизни, созданные непосредственно перед сборкой мусора.

Что, если бы областей выживших не было? В этом воображаемом случае сборщик мусора помечает совсем недавно созданные объекты (с коротким сроком жизни) как живые и переводит их в область бессрочных. После этого они немедленно прекращают существовать, но продолжают занимать место до следующей сборки мусора в области бессрочных. А эта следующая сборка мусора происходит раньше, чем нужно, из-за некорректного перемещения объектов, которые на самом деле недолговечны. С теоретической точки зрения концепция поколений тоже приводит к идее о том, что возможны два типа сборки мусора: в молодом поколении и полная.

https://t.me/javalib

7.5.4. Сборка мусора в молодом поколении

Сборка мусора в молодом поколении пытается очистить «молодые» области (Эдем и области выживших). Процесс, описанный ниже, относительно прост:

- Все живые молодые объекты, найденные на этапе пометки, перемещаются.
- Достаточно старые объекты (те, которые пережили достаточное число предыдущих сборок мусора) переходят в область бессрочных.
- Все остальные молодые живые объекты перемещаются в пустую область выживших.
- В конце Эдем и все недавно освобожденные области выживших готовы к перезаписи и повторному использованию, потому что в них нет ничего, кроме мусора.

Сборка мусора в молодом поколении запускается, когда Эдем заполнен. Обратите внимание, что стадия пометки обходит весь граф живых объектов. Если молодой объект содержит ссылку на бессрочный, то исходящие ссылки из этого бессрочного объекта все равно просматриваются и помечаются. Если этого не сделать, может возникнуть ситуация, когда бессрочный объект содержит ссылку на объект в Эдеме, но больше ни у кого таких ссылок нет. Если стадия пометки не выполнит полный обход, то этот объект в Эдеме останется невидимым и не будет правильно обработан. На практике, чтобы сократить потенциально высокие затраты на полный обход, используются специальные приемы (например, структуры *card table*).

7.5.5. Полная сборка мусора

Когда сборка мусора в молодом поколении не может перевести объект в область бессрочных (из-за нехватки места), запускается полная сборка мусора. В зависимости от того, какой сборщик используется, она может также перемещать объекты в старом поколении. Эта процедура называется *сжатием* (compacting); она применяется для того, чтобы в старом поколении хватило места для создания большого объекта, если понадобится.

7.5.6. Безопасные точки

Для сборки мусора необходимо, чтобы все потоки приложения приостановились хотя бы на короткое время. Однако их невозможно остановить в первый попавшийся момент, потому что код приложения может изменить содержимое кучи. К счастью, существуют особые моменты времени, когда JVM может быть уверена в том, что куча находится в непротиворечивом состоянии и можно собирать мусор, — они называются *безопасными точками*.

Один из простейших примеров безопасной точки — позиция между инструкциями байт-кода. Интерпретатор JVM выполняет инструкцию байт-кода, после чего переходит к следующей инструкции в потоке. Но перед тем как перейти, поток интерпретатора должен завершить любые изменения в куче (например, последствия `putfield`), чтобы в случае чего поток мог безопасно остановиться

https://t.me/javalib

в этой точке. Когда все потоки приложения достигнут безопасных точек, можно провести сборку мусора.

Это простой пример безопасной точки, но бывают и другие. Подробнее узнать о безопасных точках и о том, как они влияют на некоторые механизмы JIT-компилятора, можно в статье по адресу http://mng.bz/Oo8a. Впрочем, пора заканчивать с теорией, так что рассмотрим некоторые конкретные алгоритмы сборки мусора в JVM.

7.5.7. G1: сборщик мусора по умолчанию в Java

G1 — относительно новый сборщик мусора для платформы Java. Он достиг готовности в Java 8u40 и стал сборщиком мусора по умолчанию в Java 9 (в 2017 году). Изначально он задумывался как сборщик мусора с минимальными паузами, но на практике эволюционировал в сборщик мусора общего назначения (и поэтому стал использоваться по умолчанию).

Этот сборщик поддерживает не только поколения, но и деление на регионы; это означает, что G1 делит кучу Java на регионы одинакового размера (например, по 1, 2 или 4 Мбайт). Поколения по-прежнему существуют, но каждое из них не обязательно занимает непрерывную область памяти. Новая структура регионов в куче представлена на рис. 7.6.

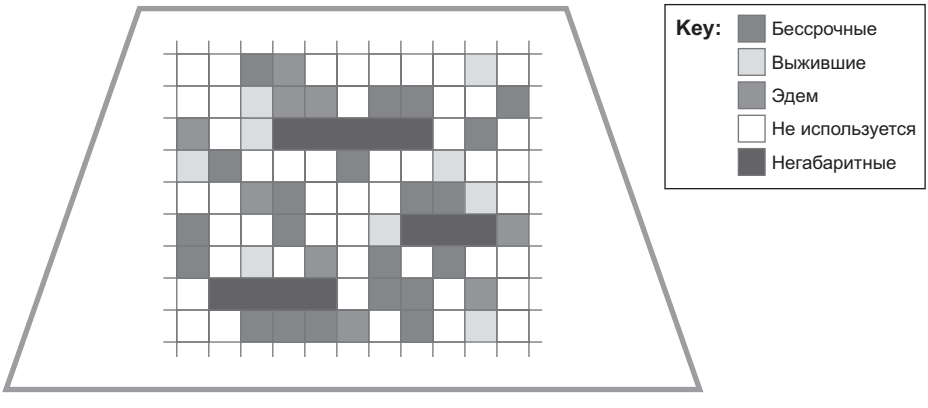


Рис. 7.6. Деление кучи сборщиком мусора G1

Регионы были введены, чтобы обеспечить предсказуемую продолжительность пауз сборки мусора. Прежние сборщики (например, Parallel) страдали от того, что если цикл сборки мусора начался, он должен был отработать до конца независимо от того, сколько времени на это уйдет (по принципу «все или ничего»).

Стратегия сборки мусора G1 обычно не приводит к долгим паузам для куч большого размера. Она проектировалась, чтобы избежать поведения «все или ничего», и для этого в G1 задается целевая продолжительность паузы — промежуток времени, на который программа может приостановиться для сборки

https://t.me/javalib

мусора. G1 делает все возможное (в пределах разумного), чтобы соблюдать эту целевую продолжительность. Во время паузы выжившие объекты перемещаются в другой регион (подобно тому, как объекты из Эдема перемещаются в области выживших), а регион возвращается в список пустых регионов.

Сборка мусора в молодом поколении в G1 работает по принципу «остановите Землю» и выполняется до завершения. Это предотвращает гонку между потоками сборки мусора и выделения памяти (эта гонка может произойти, если собирать мусор в молодом поколении параллельно с потоками приложения).

> **ПРИМЕЧАНИЕ** Механизм поколений опирается на гипотезу о том, что в живых остается лишь небольшая часть объектов, обнаруженных в ходе сборки мусора в молодом поколении. Поэтому время, потраченное на сборку мусора в молодом поколении, будет очень небольшим — намного меньше целевой продолжительности паузы.

Сборка мусора в старых объектах устроена не так, как в молодых, — во-первых, потому, что после того, как объекты попали в старое поколение, они обычно живут продолжительное время. Во-вторых, как правило, для старого поколения выделяется значительно больше места, чем для молодого.

G1 отслеживает объекты, которые перемещаются в старое поколение, и когда будет заполнено достаточно старого пространства (этот порог определяется настройкой `InitiatingHeapOccupancyPercent`, или IHOP, которая по умолчанию равна 45 %), начинается сборка мусора в старом поколении. Она выполняется параллельно (насколько это возможно) с потоками приложения.

Сборка в старом поколении начинается с конкурентной маркировки. Она основана на алгоритме, который впервые описали Дейкстра и Лампорт в 1978 году (см. https://dl.acm.org/doi/10.1145/359642.359655). Когда она завершается, немедленно запускается сборка мусора в молодом поколении. Далее следует гибридная сборка, которая освобождает старые регионы в зависимости от того, сколько в них мусора (а это можно определить по статистике, которая собрана во время конкурентной маркировки). Выжившие объекты из старых регионов переносятся в свежие старые регионы (и сжимаются).

Стратегия G1 также позволяет платформе собирать статистику о том, сколько времени (в среднем) занимает сборка мусора в одном регионе. Так реализуется целевая продолжительность паузы: G1 обрабатывает лишь столько регионов, на сколько у него хватит времени. (Правда, если сборка мусора в последнем регионе займет больше времени, чем предполагалось, эта продолжительность может быть превышена.)

Может оказаться, что за один цикл сборки мусора не удается обработать все старое поколение. В таком случае G1 обрабатывает только часть регионов, а затем завершается и освобождает ядра процессора, которые использовались для сборки мусора. Если в течение значительного периода времени долгоживущие объекты создаются в таких объемах, в каких сборщик успевает их уничтожать, то проблем не возникает.

https://t.me/javalib

А если объекты довольно долго создаются активнее, чем уничтожаются, то в качестве крайней меры сборщик мусора проводит полную сборку по принципу «остановите Землю», очищая и сжимая старое поколение. На практике до этого не доходит, если только приложение не работает в критических условиях.

Стоит упомянуть еще одно обстоятельство: можно создавать объекты, размеры которых превышают один регион. На практике это обычно означает большой массив (часто массив байтов или других примитивов).

> **ПРИМЕЧАНИЕ** Теоретически можно искусственно сконструировать класс с таким количеством полей, что размер одного экземпляра превысит 1 Мбайт, но в реальных системах такого не бывает.

Для таких крупных объектов необходима новая разновидность регионов — *негабаритные регионы*. Сборка мусора обрабатывает их особым образом, потому что для больших массивов требуется выделять непрерывные участки памяти. Если найдется достаточно свободных регионов, смежных друг с другом, можно преобразовать их в один негабаритный регион и выделить в нем память для массива.

Если в памяти не хватает места для массива (даже после сборки мусора в молодом поколении), память считается *фрагментированной*. В этом случае сборщик мусора проводит полную сборку с «остановкой Земли» и сжатием, чтобы попытаться освободить необходимый объем памяти.

G1 завоевал репутацию очень эффективного сборщика мусора в широком диапазоне рабочих нагрузок и типов приложений. Однако для некоторых нагрузок (например, тех, которые требуют высокой пропускной способности или все еще работают под Java 8) может потребоваться другой сборщик мусора, такой как Parallel.

7.5.8. Сборщик мусора Parallel

Сборщик мусора Parallel был стандартным до Java 8, и его все еще можно использовать вместо G1. Имя Parallel может потребовать некоторых разъяснений, потому что когда описывают свойства алгоритмов сборки мусора, используют два разных термина: «конкурентный» и «параллельный». Может показаться, что они означают одно и то же, но на самом деле у них совершенно разный смысл:

- *Конкурентный* — потоки сборщика мусора могут выполняться одновременно с потоками приложения.
- *Параллельный* — алгоритм сборки мусора является многопоточным и может использовать сразу несколько ядер.

Эти термины ни в коем случае не эквивалентны. Лучше рассматривать их как антонимы двух других GC-терминов: «конкурентный» — это противоположность «остановки Земли», а «параллельный» — противоположность однопоточности.

В некоторых сборщиках мусора (включая Parallel) куча не делится на регионы. Вместо этого каждое поколение занимает непрерывную область памяти,

https://t.me/javalib

которая может увеличиваться и уменьшаться по мере надобности. При такой конфигурации в куче есть две области выживших: они иногда обозначаются From и To, и одна из них всегда пуста, если только в настоящий момент не происходит сборка мусора.

> **ПРИМЕЧАНИЕ** В очень старых версиях Java также была область *PermGen* (сокращение от «Permanent Generation» — «Постоянное поколение»). Там выделялась память для внутренних структур JVM, например определений классов и методов. В Java 8 область PermGen была удалена, и если вы найдете ее упоминания в каких-либо ресурсах, то это будут устаревшие ресурсы, которые, скорее всего, утратили актуальность.

Parallel — самый эффективный сборщик мусора из тех, что доступны для массового использования в Java, но у него есть недостаток: нельзя установить целевую продолжительность паузы, так что сборка мусора в старом поколении (по принципу «остановите Землю») должна отработать до конца, сколько бы времени она ни занимала.

Некоторые разработчики иногда интересуются сложностью (запись «*O* большое») алгоритмов сборки мусора. Однако этот вопрос на самом деле бесполезен. Алгоритмы сборки мусора чрезвычайно универсальны, и они должны приемлемо функционировать в целом диапазоне возможных рабочих нагрузок. Если рассматривать только их асимптотическое поведение, такой анализ не принесет особой пользы и уж точно не даст представления об их производительности в общем случае.

Сборка мусора всегда подразумевает те или иные компромиссы. Компромиссы, которые были выбраны для G1, хорошо подходят для большинства рабочих нагрузок (настолько хорошо, что многие разработчики могут просто не обращать на них внимания). Однако для некоторых приложений все-таки важно вникать в эти компромиссы, и разработчикам приходится регулировать подсистему сборки мусора — либо переключать ее алгоритм, либо настраивать параметры сборщика.

7.5.9. Параметры конфигурации сборщика мусора

В JVM есть множество (не менее сотни) полезных параметров, которые позволяют настраивать многие ее характеристики на стадии выполнения. В этом разделе мы рассмотрим некоторые базовые параметры, относящиеся к сборке мусора.

Если параметр начинается с `-X:`, это нестандартный параметр, который не обязательно переносится между разными реализациями JVM (такими, как HotSpot или Eclipse OpenJ9). Если параметр начинается с `-XX:`, это расширенный параметр, который не рекомендуется для повседневного использования. В эту категорию входят многие параметры, относящиеся к производительности.

Некоторые параметры — это фактически логические флаги, перед которыми ставится префикс `+` или `-`, чтобы установить или сбросить их. Для других параметров необходимо задать значение, например `-XX:CompileThreshold=20000` (сколько раз нужно вызвать метод, чтобы JIT-компилятор взял его в обработку).

https://t.me/javalib

В табл. 7.1 перечислены основные параметры сборки мусора и их значения по умолчанию (если они есть).

Таблица 7.1. Основные параметры сборки мусора

Параметр	Эффект	Значение по умолчанию
-Xms<размер в мегабайтах>m	Начальный размер кучи	1/64 физической памяти
-Xmx<размер в мегабайтах>m	Максимальный размер кучи	1/4 физической памяти
-Xmn<размер в мегабайтах>m	Размер молодого поколения в куче	—
-XX:+ `DisableExplicitGC`	Блокирует вызовы `System.gc()`	—

К сожалению, распространенной практикой стало присваивать `-Xms` такое же значение, как `-Xmx`. В результате процесс выполняется с точно заданным размером кучи, который не изменится во время выполнения. На первый взгляд это выглядит разумно и создает иллюзию контроля со стороны разработчика. Тем не менее на практике это антипаттерн. Современные сборщики мусора используют качественные алгоритмы динамического изменения размера, и если искусственно их ограничивать, от этого почти всегда будет больше вреда, чем пользы.

> **ПРИМЕЧАНИЕ** В 2022 году для большинства рабочих нагрузок рекомендуется задавать значение Xmx и не задавать никакого значения Xms (если нет особых причин поступать иначе).

Также стоит обратить внимание на то, как JVM ведет себя в контейнере. Для Java 11 и 17 «физическая память» ограничена предельным объемом, который задан для контейнера, так что максимальный размер кучи не должен превышать этот предельный объем с учетом пространства для кучи, не относящейся к Java, и кучи других процессов помимо JVM. Ранние версии Java 8 не всегда соблюдали пределы контейнера, так что если ваше приложение запускается в контейнере, имеет смысл обновиться до Java 11.

При оптимизации сборщика мусора G1 могут пригодиться две другие настройки, которые приведены в табл. 7.2.

Таблица 7.2. Флаги сборщика мусора G1

Параметр	Эффект
-XX:MaxGCPauseMillis=50	Указывает, что G1 по возможности не должен допускать паузу дольше 50 мс в течение одной сборки мусора
-XX:GCPauseIntervalMillis=200	Указывает, что между сборками мусора должно по возможности проходить не менее 200 мс

Параметры можно комбинировать: например, установить целевую максимальную продолжительность паузы 50 мс с промежутками между паузами не менее

https://t.me/javalib

200 мс. Конечно, систему сборки мусора можно регулировать лишь до некоторого предела. Продолжительности паузы должно хватать, чтобы освободить «мусорную» память. Если запросить паузу в 1 мс раз в 100 лет, JVM не сможет выполнить такой запрос и даже не будет пытаться.

В следующем разделе мы поговорим о JIT-компиляции. Для многих программ она становится основным фактором, от которого зависит производительность кода. Сначала мы рассмотрим основы JIT-компиляции, а в конце раздела покажем, как настроить ее протоколирование, чтобы видеть, к каким методам она применяется.

7.6. JIT-КОМПИЛЯЦИЯ С HOTSPOT

Как упоминалось в главе 1, платформу Java имеет смысл рассматривать как «динамически компилируемую». Во время выполнения некоторые классы приложений и фреймворков проходят дальнейшую компиляцию и преобразуются в машинный код, который может выполняться напрямую.

Этот процесс называется JIT-компиляцией (от «Just-In-Time», то есть «точно вовремя») и обычно выполняется с одним методом за раз. Если вы понимаете, как устроена JIT-компиляция, это часто позволяет выявить важнейшие части крупной кодовой базы.

Вот что прежде всего стоит знать о JIT-компиляции:

- Практически во всех современных JVM есть та или иная разновидность JIT-компилятора.
- JVM, которая выполняет только интерпретацию, работает очень медленно по сравнению с виртуальной машиной с JIT.
- Откомпилированные методы выполняются радикально быстрее, чем интерпретируемый код.
- Имеет смысл в первую очередь компилировать методы, которые чаще всего используются.
- Занимаясь JIT-компиляцией, важно в первую очередь делать то, что требует наименьших усилий.

Последний пункт означает, что анализ всегда следует начинать с откомпилированного кода, потому что если метод все еще находится в интерпретируемом состоянии, то в обычных условиях это означает, что он не выполнялся так часто, как откомпилированный метод. (Бывает, что компиляция метода не удается, но такое случается редко.)

Изначально JVM запускает методы, интерпретируя их байт-код, и подсчитывает количество вызовов каждого метода (и ряд других показателей). Когда это количество достигает порогового значения, а метод удовлетворяет определенным критериям, поток JVM в фоновом режиме компилирует байт-код в машинный код. Если компиляция проходит успешно, то все последующие вызовы метода будут использовать откомпилированную форму — если только не произойдет что-то такое, от чего она станет недействительной или неоптимальной.

https://t.me/javalib

Один и тот же метод, в зависимости от специфики своего кода, в откомпилированном режиме может работать намного быстрее, чем в интерпретируемом. Иногда говорят об «ускорении в сто раз и более», но это крайне приближенная оценка. В результате JIT-компиляции исполняемый код изменяется настолько, что любая оценка в виде одного числа только сбивает с толку. Чтобы улучшить производительность, часто следует в первую очередь понять, какие методы играют важную роль в программе и какие из этих важных методов компилируются.

7.6.1. Зачем нужна динамическая компиляция?

Нас иногда спрашивают: почему платформа Java вообще использует динамическую компиляцию? Почему все не компилируется заранее (как в C++)? Первый и наиболее очевидный ответ таков: если в качестве базовых единиц развертывания используются платформенно-независимые артефакты (файлы .jar и .class), это создает гораздо меньше проблем, чем необходимость возиться с разными откомпилированными двоичными файлами для всех целевых платформ.

Есть и альтернативный, более амбициозный ответ: в языках с динамической компиляцией компилятору доступно больше информации, а именно: в языках с AOT-компиляцией (ahead-of-time, компиляцией перед выполнением) программа не получает никакой информации, касающейся времени выполнения, например сведений о доступности некоторых инструкций или о других аппаратных факторах, а также статистики о том, как выполняется код. Это открывает заманчивую перспективу того, что языки с динамической компиляцией (такие, как Java) могут работать быстрее, чем языки с AOT-компиляцией.

> **ПРИМЕЧАНИЕ** Прямая AOT-компиляция байт-кода Java в машинный код («статический код Java») активно исследуется в сообществе Java, но к сожалению, выходит за рамки этой книги.

Мы продолжим обсуждать механику JIT-компиляции на примере конкретной JVM, которая называется HotSpot. Многие общие положения будут применимы и к другим виртуальным машинам, но подробности могут значительно различаться.

Сначала мы представим разные JIT-компиляторы, которые поставляются с HotSpot, а затем рассмотрим два мощных механизма оптимизации, которые доступны в HotSpot: *встраивание кода* и *мономорфную диспетчеризацию*. В завершение раздела мы покажем, как настроить ведение журнала компиляции методов, чтобы точно видеть, какие методы компилируются. Начнем с общего введения в HotSpot.

7.6.2. Введение в HotSpot

HotSpot — виртуальная машина Java, которую компания Oracle приобрела вместе с Sun Microsystems. (До этого Oracle уже владела JVM JRockit, которую изначально разработали в BEA Systems.) HotSpot лежит в основе OpenJDK и способна работать в двух разных режимах: клиентском и серверном.

https://t.me/javalib

В прежние времена этот режим можно было выбрать, запуская JVM с ключом `-client` или `-server`. В одних сценариях использования предпочтительнее клиентский режим, а в других — серверный.

C1 (клиентский компилятор)

Компилятор C1 был изначально рассчитан на использование в GUI-приложениях. В этой области особенно ценится стабильность работы, поэтому C1 (он же *клиентский компилятор*) склонен использовать более консервативный подход при компиляции. Он не делает неожиданных пауз, чтобы откатить оптимизационные решения, которые оказались неправильными или опирались на ошибочные предположения. У него довольно низкий порог компиляции (метод должен выполниться 1500 раз, прежде чем будет признан пригодным для компиляции) и, как следствие, относительно короткий период разогрева.

C2 (серверный компилятор)

С другой стороны, *серверный компилятор* (C2) делает агрессивные предположения при компиляции. Чтобы обеспечить правильность выполняемого кода, C2 добавляет быструю проверку времени выполнения (обычно называемую *защитным условием*), которая проверяет, что сделанное предположение правильно. Если это оказывается не так, C2 откатывает агрессивную компиляцию и нередко пробует другие способы. Агрессивный подход часто приводит к намного лучшей производительности, чем не склонный к риску клиентский компилятор.

Порог преобразования в машинный код у C2 намного выше, чем у C1. По умолчанию метод не считается пригодным для компиляции C2, пока он не наберет 10 000 вызовов, что подразумевает гораздо более продолжительное время разогрева.

Java в реальном времени

Исторически существовала разновидность Java, которая называлась «Java в реальном времени», и некоторые разработчики интересуются, почему код, которому требуется высокая производительность, просто не использует эту платформу (это отдельная JVM, а не режим HotSpot). Дело в том, что, вопреки распространенному мифу, система реального времени — не самая быстрая.

Программирование реального времени обычно направлено на то, чтобы обеспечить определенные гарантии. С точки зрения статистики система реального времени стремится сократить дисперсию времени, которое тратится на выполнение некоторых операций, и для этого готова до некоторой степени пожертвовать средней задержкой. Общая производительность слегка «проседает» ради более единообразного выполнения. Разработчики, которых интересует более высокая производительность, обычно стремятся снизить среднюю задержку даже ценой увеличенной дисперсии, так что агрессивные приемы оптимизации серверного компилятора, как правило, считаются приемлемыми.

В современных JVM используется как клиентский, так и серверный компилятор: клиентский компилятор работает на ранней стадии, а расширенные механизмы

https://t.me/javalib

оптимизации серверного компилятора применяются после разогрева приложения. Эта двухступенчатая схема называется *многоуровневой компиляцией*. А следующий раздел посвящен встраиванию методов, которое широко используют все JIT-компиляторы.

7.6.3. Встраивание методов

Встраивание (inlining) — один из самых мощных механизмов, которые есть в распоряжении HotSpot. Его суть заключается в том, чтобы не вызывать метод, а сделать его встроенным, то есть внедрить код метода прямо в код вызывающей стороны.

Одно из преимуществ платформы состоит в том, что компилятор может принять решение о встраивании на основании свежей статистики времени выполнения о том, как часто вызывается метод, и о других факторах (например, не приведет ли встраивание к тому, что вызывающий метод станет слишком большим, что может повлиять на кэширование кода). Компилятор HotSpot способен принимать намного более эффективные решения о встраивании, чем статические компиляторы.

А как же методы доступа?

Некоторые разработчики ошибочно предполагают, что HotSpot не умеет встраивать методы доступа (открытые методы чтения, которые обращаются к частному члену класса). Обычно это аргументируют тем, что, поскольку переменная является частной, оптимизация не может избавиться от вызова метода, потому что обращение к переменной за пределами класса запрещено. Тем не менее это заблуждение.

Компилируя методы в машинный код, HotSpot игнорирует механизм управления доступом и заменяет метод доступа прямым обращением к частному полю. Модель безопасности Java при этом не страдает, потому что весь контроль доступа уже был выполнен, когда класс загружался или компоновался.

Встраивание методов происходит полностью автоматически, и почти всегда настройки по умолчанию обеспечивают нужный результат. Существуют параметры, позволяющие управлять размером методов, которые могут встраиваться, и частотой вызова методов, при которой они становятся кандидатами на встраивание.

Эти параметры в основном помогают любознательному программисту лучше понять, как работает встраивание во внутренней реализации. Они редко приносят пользу в реальном коде, и к ним стоит относиться как к крайнему средству оптимизации, потому что они могут непредсказуемым образом повлиять на производительность исполнительной системы.

7.6.4. Динамическая компиляция и мономорфные вызовы

Один из примеров агрессивной оптимизации — *мономорфные вызовы*. Этот механизм оптимизации основан на наблюдении, что в большинстве случаев вызов методов на объекте следующего вида:

```
MyActualClassNotInterface obj = getInstance();

obj.callMyMethod();
```

https://t.me/javalib

будет происходить только для одного типа объекта. Это можно сформулировать иначе: в точке вызова `obj.callMyMethod()` почти никогда не будут встречаться и класс, и его подкласс. В этом случае процедуру поиска метода Java можно заменить прямым вызовом откомпилированного кода, который соответствует методу `callMyMethod()`.

> **ПРИМЕЧАНИЕ** Мономорфная диспетчеризация дает пример профилирования среды выполнения JVM: она позволяет платформе применять средства оптимизации, которые попросту невозможны в языках со статической компиляцией вроде C++.

Нет никаких технических причин, по которым метод `getInstance()` не мог бы в одних случаях возвращать объект типа `MyActualClassNotInterface`, а в других — объект другого подкласса. Чтобы обезопасить программу от такой возможности, `getInstance()` будет считаться кандидатом для мономорфной оптимизации, только если окажется, что абсолютно каждый раз при вызове метода (до достижения порога компиляции) возвращается строго один и тот же тип. Кроме того, в откомпилированный код для будущих вызовов вставляется проверка типа `obj` во время исполнения. Если эта проверка в какой-то момент окажется неудачной, среда выполнения откатит оптимизацию, а программа этого даже не заметит и не совершит никаких ошибок.

Эту довольно агрессивную технику оптимизации применяет только серверный компилятор, а не клиентский.

7.6.5. Чтение журналов компиляции

Рассмотрим пример того, как можно работать с выводом журнальных сообщений JIT-компилятора. В астрономическом каталоге Hipparcos содержится подробная информация о звездах, наблюдаемых с Земли. Наше демонстрационное приложение обрабатывает каталог и строит карты звездного неба, которое можно наблюдать в заданную ночь из заданной точки.

Вот образец вывода, который показывает, какие методы компилируются при запуске приложения. Чтобы получить этот вывод, мы применим флаг JVM `XX:+PrintCompilation` — это один из расширенных параметров, о которых мы упоминали ранее. Если добавить его в командную строку запуска JVM, потоки JIT-компиляции будут выводить сообщения в стандартный журнал. Эти сообщения указывают, какие методы прошли порог компиляции и были преобразованы в машинный код:

```
1 java.lang.String::hashCode (64 bytes)
2 java.math.BigInteger::mulAdd (81 bytes)
3 java.math.BigInteger::multiplyToLen (219 bytes)
4 java.math.BigInteger::addOne (77 bytes)
5 java.math.BigInteger::squareToLen (172 bytes)
6 java.math.BigInteger::primitiveLeftShift (79 bytes)
7 java.math.BigInteger::montReduce (99 bytes)
8 sun.security.provider.SHA::implCompress (491 bytes)
9 java.lang.String::charAt (33 bytes)
```

https://t.me/javalib

```
1% ! sun.nio.cs.SingleByteDecoder::decodeArrayLoop @ 129 (308 bytes)
…
39 sun.misc.FloatingDecimal::doubleValue (1289 bytes)
40 org.camelot.hipparcos.DelimitedLine::getNextString (5 bytes)
41 ! org.camelot.hipparcos.Star::parseStar (301 bytes)
...
2% ! org.camelot.CamelotStarter::populateStarStore @ 25 (106 bytes)
65 s java.lang.StringBuffer::append (8 bytes)
```

Это довольно типичный вывод `PrintCompilation`: он показывает, какие методы были признаны достаточно «горячими» для компиляции. Как и стоило ожидать, раньше всех с большой вероятностью компилируются платформенные методы (такие, как `String::hashCode()`), а со временем также начинают компилироваться прикладные методы, например `org.camelot.hipparcos.Star::parseStar()`, который в нашем примере разбирает запись астрономического каталога.

Строки вывода начинаются с числа, которое указывает, в каком порядке методы компилировались при этом запуске. Учтите, что из-за динамического характера платформы этот порядок может слегка изменяться от запуска к запуску. Вот что означают некоторые другие поля:

- `s` — метод является синхронизированным.
- `!` — у метода есть обработчики исключений.
- `%` — замещение в стеке (OSR, On-Stack Replacement).

OSR означает, что метод был откомпилирован и заменил интерпретируемую версию в выполняемом коде. Обратите внимание, что методы OSR используют собственную схему нумерации, начиная с 1.

Осторожно, зомби!

Если код запускается с помощью серверного компилятора (C2), то при просмотре выходных журналов вы время от времени будете встречать строки с сообщениями о «невозможности входа» и «переходе в зомби». Это означает, что тот или иной метод, который был откомпилирован, теперь стал недействительным — обычно из-за операции загрузки класса.

7.6.6. Деоптимизация

HotSpot может деоптимизировать код, если оптимизация опиралась на предположение, которое оказалось ошибочным. Во многих случаях после этого HotSpot пытается выполнить альтернативную оптимизацию. Таким образом, один и тот же метод может многократно деоптимизироваться и перекомпилироваться.

Со временем количество откомпилированных методов стабилизируется. Код достигает устойчивого откомпилированного состояния и по большей части остается в нем. Точный перечень методов, которые будут компилироваться, может зависеть от конкретной используемой версии JVM и ОС. Ошибкой было бы думать, будто на всех платформах получится одинаковый набор откомпилированных

https://t.me/javalib

методов и что откомпилированный код каждого метода будет иметь приблизительно одинаковый размер на разных платформах. Как практически во всем, что касается производительности, этот эффект стоит измерять, и результаты могут вас удивить. Мы наблюдали случаи, когда даже вполне безобидный метод в результате JIT-компиляции превращался в код, размеры которого в Mac и Linux различались пятикратно.

Измерения нужны всегда. К счастью, современные JVM поставляются с превосходными инструментами, которые упрощают глубокий анализ производительности. Рассмотрим несколько примеров.

7.7. JDK FLIGHT RECORDER

Компания Oracle приобрела утилиты Flight Recorder и Mission Control (обычно обозначаемые сокращениями JFR и JMC) при покупке BEA Systems в 2008 году. Две эти программы работают вместе: JFR представляет собой низкозатратное ядро профилирования, основанное на событиях, с высокопроизводительной внутренней подсистемой записи событий в двоичном формате, а JMC — инструмент с графическим интерфейсом для анализа файлов данных, которые JFR создает на базе телеметрии от одной JVM.

Эти утилиты изначально входили в инструментарий для JRockit JVM от BEA и были перемещены в коммерческую версию Oracle JDK в процессе слияния JRockit с HotSpot. После выпуска JDK 9 компания Oracle изменила модель выпуска Java и объявила, что JFR и JMC теперь распространяются с открытым кодом. JFR была включена в OpenJDK и выпущена в JDK 11 как JEP 328. JMC превратилась в автономный проект с открытым исходным кодом и теперь существует как отдельно загружаемая программа.

> **ПРИМЕЧАНИЕ** В Java 14 в JFR появилась новая возможность: теперь JFR может порождать непрерывный поток событий. В это изменение входит API обратного вызова, чтобы события можно было обрабатывать немедленно, вместо того чтобы разбирать файл постфактум.

К сожалению, поскольку JFR и JMC лишь недавно стали распространяться с открытым кодом, многие Java-разработчики не знают об их выдающихся возможностях. Давайте же с ними познакомимся!

7.7.1. Flight Recorder

JFR впервые стала распространяться с открытым кодом как часть OpenJDK 11, поэтому вам понадобится эта или более поздняя версия Java, чтобы пользоваться этой утилитой. JFR была также обратно портирована в OpenJDK 8, в результате чего она доступна для версий 8u262 и выше.

Записи JFR можно создавать несколькими способами, но мы рассмотрим только два из них: аргументы командной строки при запуске JVM и утилита `jcmd`.

https://t.me/javalib

Сначала посмотрим, какие параметры командной строки позволяют запустить JFR при запуске процесса. Основной параметр выглядит так:

```
-XX:StartFlightRecording:<настройки>
```

Данные могут записываться как в одноразовый файл, так и в непрерывный кольцевой буфер, а многочисленные аргументы командной строки управляют тем, какие именно данные будут регистрироваться.

JFR может регистрировать более сотни разных метрик, большинство из которых практически не влияют на производительность, но некоторые создают определенную дополнительную нагрузку. Управлять конфигурацией всех этих метрик по отдельности было бы непосильной задачей.

Вместо этого, чтобы упростить процесс, JFR использует файлы конфигурации профилирования. Это простые файлы XML, которые содержат конфигурацию каждой метрики и сведения о том, надо ли ее регистрировать. В стандартный комплект JDK входят два базовых файла: default.jfc и profile.jfc.

Уровень регистрации по умолчанию (default.jfc) рассчитан на чрезвычайно низкие затраты, чтобы утилиту можно было использовать практически в любом производственном процессе Java. Конфигурация profile.jfc обеспечивает больше подробной информации, но и потребляет больше ресурсов во время выполнения.

> **ПРИМЕЧАНИЕ** Помимо этих двух файлов, можно создать собственный файл конфигурации, в котором будут только нужные срезы данных. В JMC есть менеджер шаблонов, который упрощает создание таких файлов.

Наряду с файлом настроек JFR при запуске может принимать и другие параметры, включая имя файла для собранных данных и максимальный возраст элементов данных. Например, командная строка JFR может выглядеть так (это все вводится в одну строку):

```
-XX:StartFlightRecording:disk=true,filename=svc/sandbox/service.jfr,
                         maxage=12h,settings=profile
```

> **ПРИМЕЧАНИЕ** Когда программа JFR была частью коммерческой сборки, ее можно было разблокировать параметром `-XX:+UnlockCommercialFeatures`. Однако в Oracle JDK 11+ этот параметр вызывает предупреждение. Так происходит потому, что вся коммерческая функциональность перешла на модель с открытым кодом, а поскольку `-XX:+UnlockCommercialFeatures` никогда не входил в OpenJDK, продолжать пользоваться им не имеет смысла. В сборках OpenJDK этот параметр вызывает ошибку.

Одна из самых замечательных особенностей JFR заключается в том, что ее не обязательно настраивать при запуске процесса. Вместо этого ею можно управлять из командной строки командой `jcmd`:

```
$ jcmd <pid> JFR.start name=Recording1 settings=default
$ jcmd <pid> JFR.dump filename=recording.jfr
$ jcmd <pid> JFR.stop
```

https://t.me/javalib

JFR также предоставляет JMX API для управления регистрацией данных JFR. Тем не менее независимо от того, как вы запустили JFR, конечный результат остается неизменным — один файл на каждый запуск профилирования на JVM. Файл содержит большой объем двоичных данных и не читается человеком, поэтому нам понадобится инструмент, который бы извлекал и визуализировал данные.

7.7.2. Mission Control

JDK Mission Control (JMC) — графическая программа для вывода данных, которые содержатся в выходных файлах JFR. Она запускается командой `jmc`. Когда-то программа входила в загружаемый архив Oracle JDK, но сейчас она доступна отдельно по адресу https://jdk.java.net/jmc/.

Начальный экран Mission Control показан на рис. 7.7. После того как загружен файл, JMC проводит автоматизированный анализ, чтобы выявить в протоколе запуска очевидные проблемы.

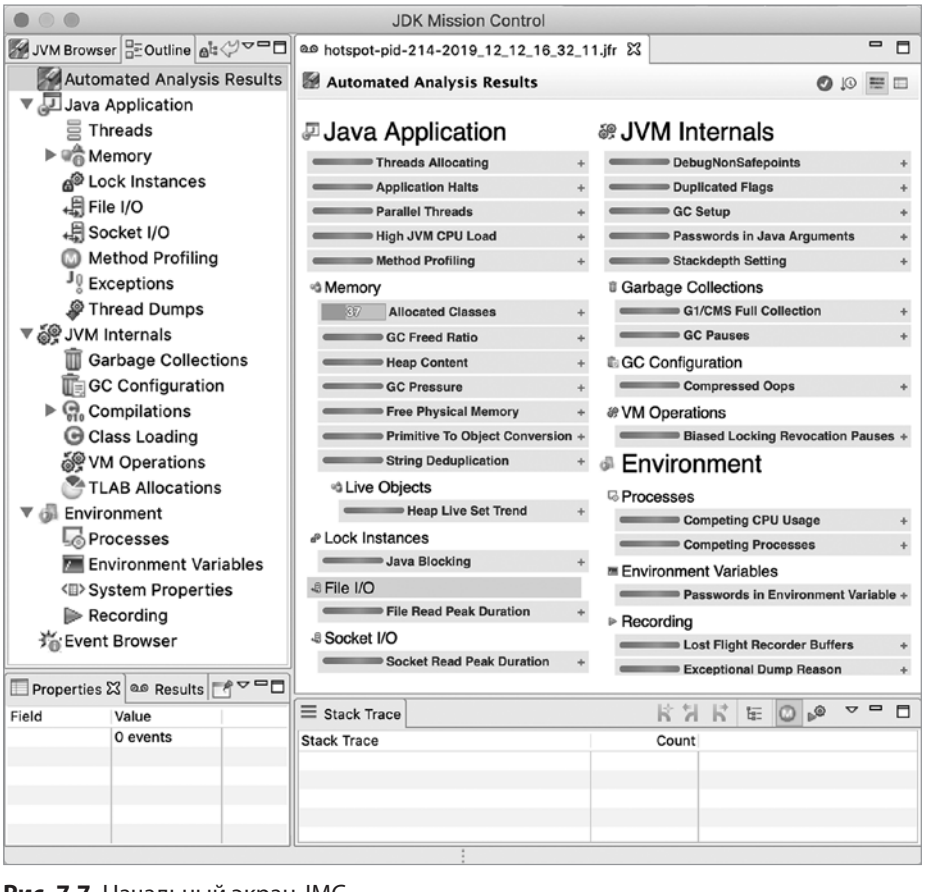


Рис. 7.7. Начальный экран JMC

https://t.me/javalib

ПРИМЕЧАНИЕ Конечно, чтобы выполнять профилирование, в целевом приложении нужно включить Flight Recorder. Можно не только использовать ранее созданный файл, но и динамически подключать JFR к уже запущенному приложению — для этого в верхней левой части окна JMC есть вкладка JVM Browser.

Один из первых экранов JMC — сводный экран телеметрии с верхнеуровневой информационной панелью общей работоспособности JVM (рис. 7.8).

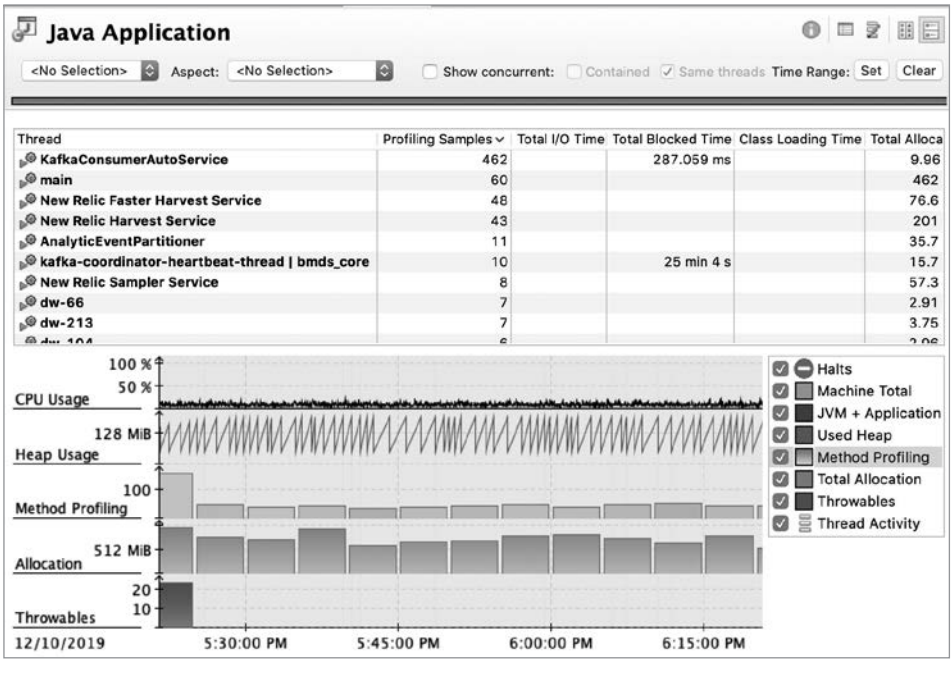


Рис. 7.8. Информационная панель JMC

У каждой основной подсистемы JVM есть специальный экран с инструментами для глубокого анализа. Например, у сборки мусора есть экран со сводкой ее событий на всем протяжении файла JFR. Метрика Longest Pause в нижней части экрана показывает, происходили ли за изучаемый период какие-нибудь аномально долгие события сборки мусора, как показано на рис. 7.9.

В подробной профильной конфигурации также можно просмотреть отдельные события, в которых прикладным потокам выделяются новые буферы (TLAB). Здесь отображается намного более точная картина выделения памяти в процессе (рис. 7.10). Можно легко видеть, каким потокам выделяется больше всего памяти — в нашем случае это поток, который потребляет данные службы Apache Kafka.

Другая крупная подсистема JVM — JIT-компилятор, и JMC позволяет углубленно изучить, как он работает (рис. 7.11).

https://t.me/javalib

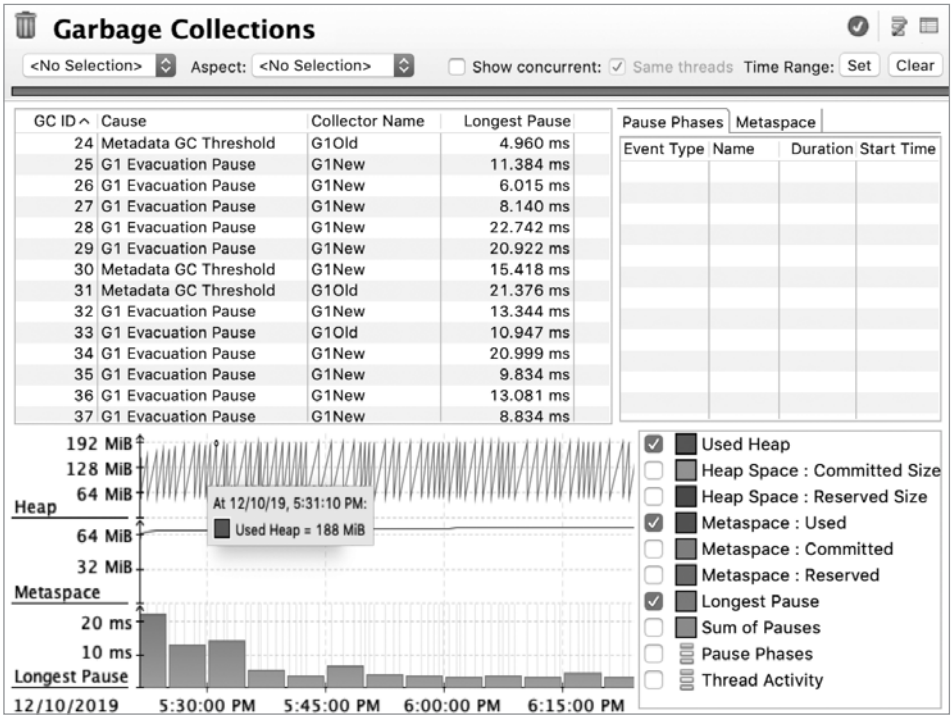


Рис. 7.9. Сборка мусора в JMC

Один из важнейших ресурсов — доступная память в кэше кода JIT-компилятора. Это область памяти, в которой хранятся откомпилированные версии методов. JMC позволяет визуализировать использование кэша кода — пример показан на рис. 7.12.

Если процесс использует много откомпилированных методов, эта область памяти может исчерпаться, что помешает процессу достичь максимальной производительности.

Кроме того, в JMC входит профилировщик методов, который работает примерно так же, как профилировщик из VisualVM или аналогичные средства таких коммерческих инструментов, как JProfiler или YourKit. Типичный результат профилирования методов представлен на рис. 7.13.

Один из самых сложных экранов JMC — представление `VM Operations`. В нем показаны некоторые внутренние операции, которые выполняет JVM, и их продолжительность. Вряд ли это представление понадобится при каждом анализе, но оно может пригодиться, чтобы выявлять некоторые менее распространенные проблемы. Типичная ситуация изображена на рис. 7.14.

https://t.me/javalib

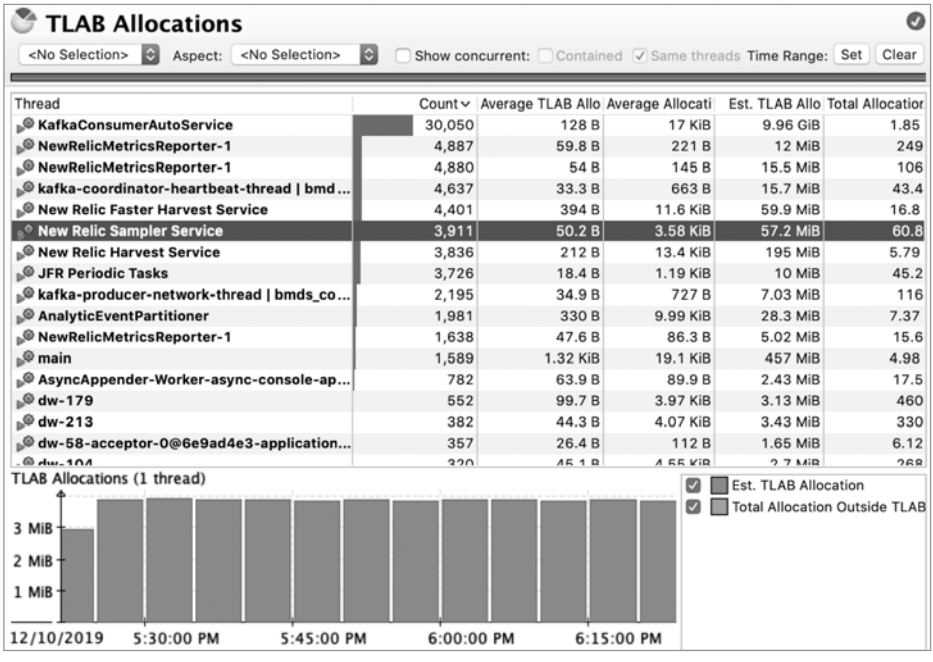


Рис. 7.10. Выделение TLAB в JMC

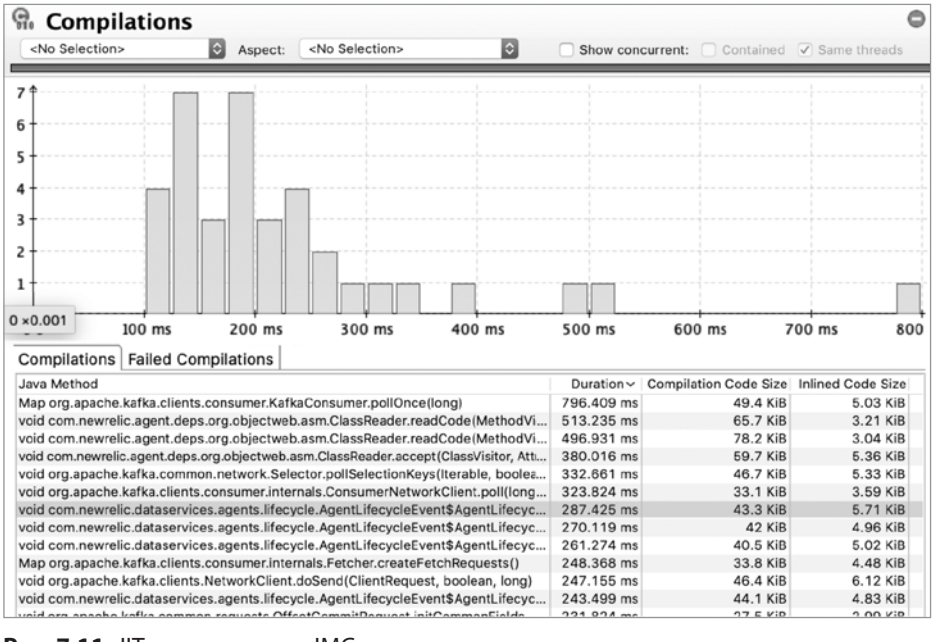


Рис. 7.11. JIT-компиляция в JMC

https://t.me/javalib

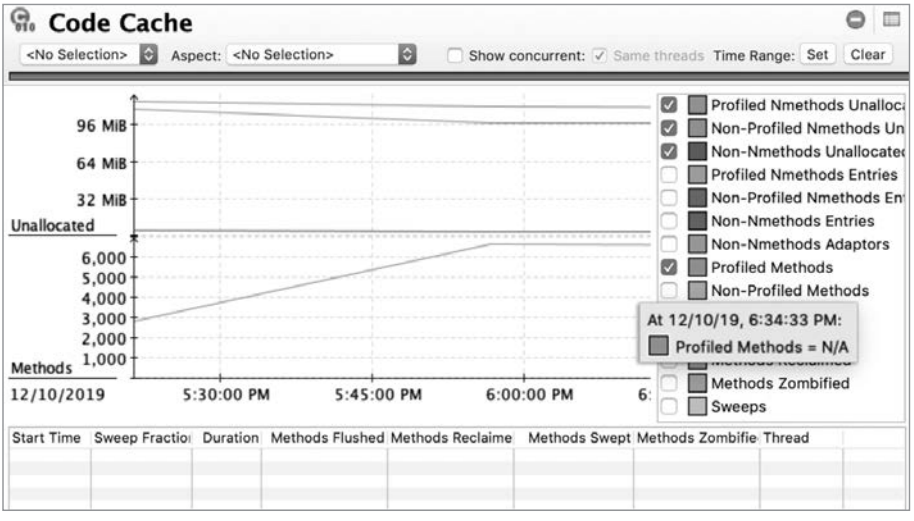


Рис. 7.12. Кэш кода JIT-компилятора в JMC

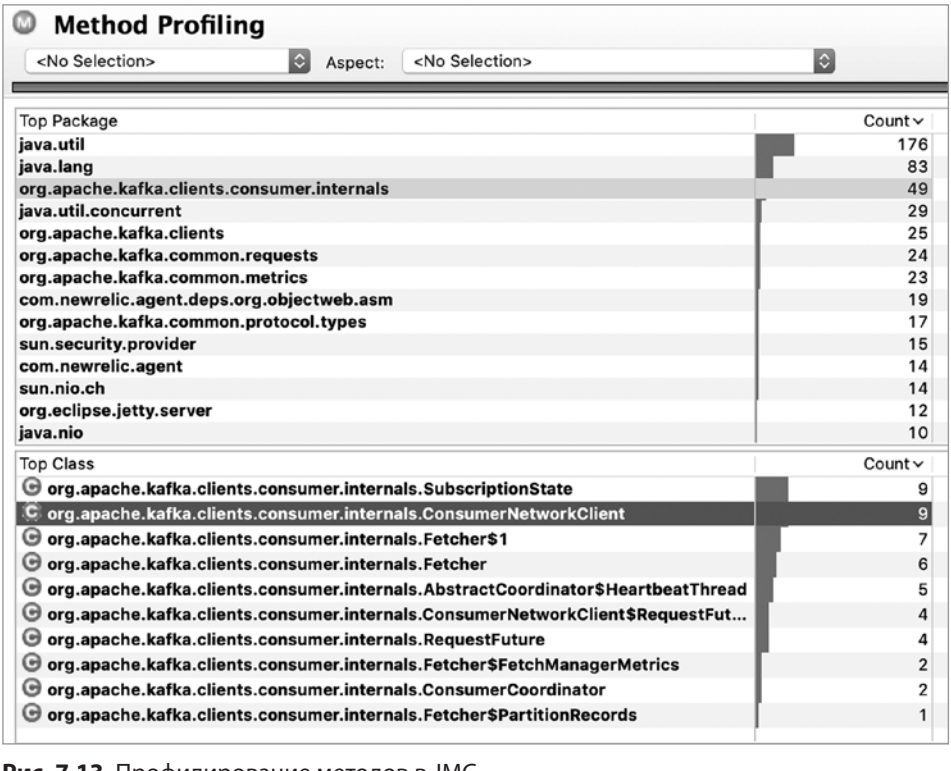


Рис. 7.13. Профилирование методов в JMC

https://t.me/javalib

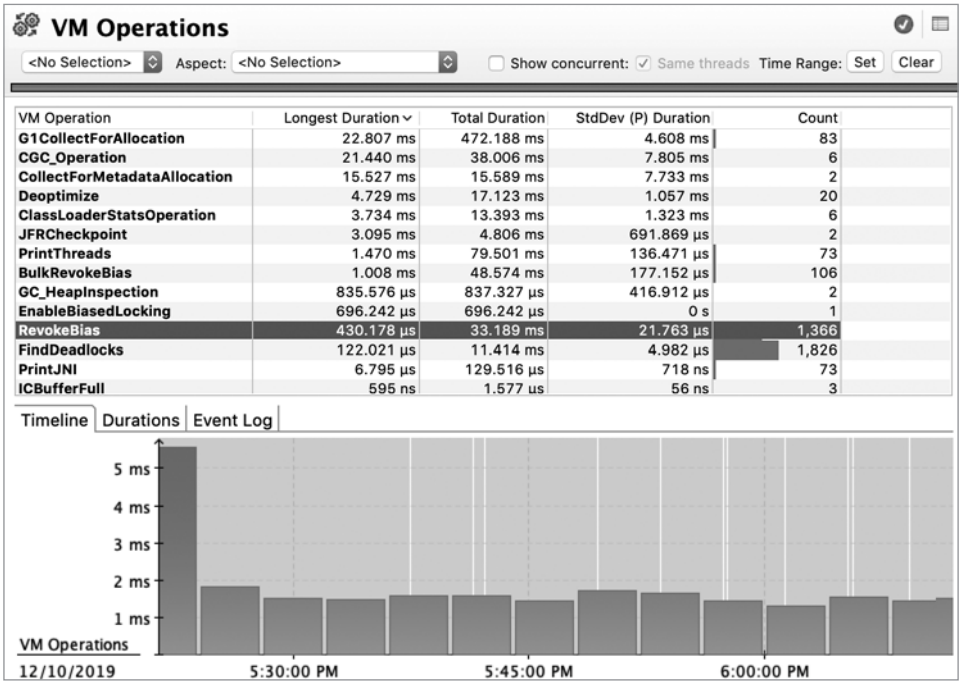


Рис. 7.14. Операции JVM в JMC

JMC умеет диагностировать одну JVM, и эта функциональность превосходна сама по себе. Однако этот сценарий использования не масштабируется, чтобы можно было анализировать целый кластер (или полное приложение). Кроме того, современным системам часто необходимы средства не только для углубленного анализа, но и для оперативного мониторинга, который обеспечивает *наблюдаемость* системы.

Классическая для JFR модель файла журнала (с одним файлом на JVM) не упрощает эту задачу. Она плохо подходит для потока телеметрических данных, которые передаются по сети поставщику SaaS или внутреннему инструменту. Некоторые компании (например, New Relic и DataDog) предоставляют функциональность JFR, но это довольно узкий сегмент.

К счастью, API потоков данных JFR, который появился в Java 14, обеспечивает полезный инструментарий как для сценариев наблюдаемости, так и для глубокого анализа. Однако сообщество в целом не спешит переходить на выпуски Java, которые не имеют статуса LTS. Это означает, что с большой вероятностью только Java 17 (которая будет LTS-версией) станет массовой версией Java, которая поддерживает JFR с потоками данных.

Как мы уже отмечали, оптимизация производительности не сводится к тому, чтобы таращиться в свой код, надеясь на просветление, или применять готовые

https://t.me/javalib

шаблонные решения. Для настоящей оптимизации нужны тщательные измерения, внимание к деталям и терпение. Необходимо прилежно устранять очаги ошибок в тестах, чтобы проявились истинные источники проблем производительности.

В этой главе нам удалось дать только краткое введение в богатую и разнообразную область оптимизации производительности. Эту тему стоит изучить гораздо глубже, и мы рекомендуем любознательному читателю обратиться к специализированной литературе, например: Ben Evans (Бэн Эванс), James Gough (Джеймс Гоф), Chris Newland (Крис Ньюланд) «Optimizing Java» (O'Reilly Media[1].

ИТОГИ

- JVM — невероятно мощная и сложная среда выполнения.
- JVM устроена так, что код, с которым она работает, не всегда легко оптимизировать.
- Чтобы получить верное представление о том, где кроются проблемы, нужно проводить измерения.
- Уделяйте особое внимание подсистеме сборки мусора и JIT-компилятору.
- Средства мониторинга и другие инструменты анализа приносят большую пользу.
- Научитесь читать журналы и другую информацию платформы, потому что инструменты не всегда доступны.

[1] Эванс Б., Гоф Дж., Ньюланд К. «Java: оптимизация программ. Практические методы повышения производительности приложений в JVM».

https://t.me/javalib

Часть III

Другие языки на JVM

Эта часть книги посвящена альтернативным языковым парадигмам на JVM. JVM — замечательная среда выполнения: она не только обеспечивает быстродействие и мощь, но и дает программисту невероятную гибкость. Фактически JVM открывает портал в мир других языков помимо Java и позволяет опробовать различные подходы к программированию.

Если вы программировали только на Java, то, возможно, спросите, что вам даст изучение других языков. Как говорилось в главе 1, быть опытным разработчиком Java — значит постоянно повышать свою квалификацию во всех аспектах языка Java, платформы и экосистемы. В частности, стоит разбираться в темах, которые пока еще лишь виднеются на горизонте, но в недалеком будущем станут неотъемлемой частью профессионального опыта.

> Будущее уже здесь — просто оно еще неравномерно распределено.
>
> *Уильям Гибсон*

Оказывается, что уже сегодня в альтернативных языках JVM развиваются многие новые идеи, которые понадобятся нам в будущем, в частности функциональное программирование. Изучая новый язык JVM, можно заглянуть в другую реальность, которая может оказаться каким-то из ваших будущих проектов. Знакомство с непривычными концепциями также поможет вам увидеть свои нынешние знания в новом свете. Вполне возможно, что при изучении нового языка вы откроете в себе таланты, о которых и не подозревали, и разовьете новые навыки, которые пригодятся на вашем профессиональном пути.

В главе 9 рассматривается Kotlin — относительно молодой язык, который исправляет многие недостатки Java, не внося радикальных изменений в фундаментальные принципы языка. Kotlin нацелен на компактность и безопасность, но при этом также предоставляет возможности, которые раньше скорее относились к сфере динамических языков сценариев.

В последнее время функциональное программирование привлекает все больше внимания как альтернатива типичному объектно-ориентированному подходу, который характерен для Java. Чтобы получить представление об этой параллельной вселенной, мы рассмотрим Clojure — один из функциональных языков JVM, который дальше всего отходит от менталитета Java.

В частях IV и V мы будем часто возвращаться к этим языкам, чтобы показать, как их можно применять в ваших проектах — от сборки и тестирования до хитростей параллелизма и структуры программ. Итак, давайте ненадолго выглянем из уютного мира Java и рассмотрим доступные альтернативы.

https://t.me/javalib

8 Альтернативные языки на JVM

В ЭТОЙ ГЛАВЕ

- ✓ Классификация языков
- ✓ Почему стоит использовать альтернативные языки на JVM
- ✓ Как выбрать альтернативный язык
- ✓ Как JVM работает с альтернативными языками

Если вы использовали Java в серьезных проектах, то, вероятно, заметили, что код иногда получается слишком пространным и неуклюжим. А может быть, вы даже мечтали о том, чтобы вся эта писанина была попроще.

К счастью, как мы продемонстрировали в нескольких последних главах, JVM настолько замечательна, что она становится естественной средой обитания для других языков программирования помимо Java. В этой главе мы покажем, почему и как стоит подключать к вашим проектам другие языки в JVM.

В этой главе мы узнаем, по каким признакам классифицируются языки программирования (например, языки со статической и динамической типизацией), почему имеет смысл обращаться к альтернативным языкам и по каким критериям их выбирать. Также мы обзорно познакомимся с двумя языками (Kotlin и Clojure), которые будут подробнее рассмотрены в следующих главах.

https://t.me/javalib

8.1. КЛАССИФИКАЦИЯ ЯЗЫКОВ

Существует множество систем классификации языков программирования. Этот факт можно выразить иначе: огромное многообразие стилей и подходов к программированию воплощается в разных языках. Часто бывает проще осваивать новые стили, если понимать, как систематизировать различия между языками.

> **ПРИМЕЧАНИЕ** Классификации помогают ориентироваться в разнообразии языков. Одни системы классификации выстроены более четко, чем другие, но ни одна из них не идеальна. Разные люди по-разному представляют себе, как надо классифицировать языки программирования.

В последнее время появилась тенденция включать в языки функциональность из самых разных парадигм. Часто говорят, что тот или иной язык является «менее функциональным» по сравнению с другим языком или что язык «динамически типизированный, но с необязательной статической типизацией по необходимости».

Мы рассмотрим деление языков на «интерпретируемые — компилируемые», «динамические — статические», «императивные — функциональные», а также сравним повторные реализации языков с оригинальными версиями. В общем случае эти классификации скорее помогают ориентироваться в пространстве возможностей, чем служат полными и строгими теоретическими схемами.

Например, можно сказать, что Java — компилируемый во время выполнения императивный язык со статической типизацией, обладающий некоторыми функциональными возможностями. Он приоритизирует безопасность, ясный код, обратную совместимость и производительность, ради чего готов пойти на некоторую избыточность кода и формальности (например, при развертывании).

> **ПРИМЕЧАНИЕ** У разных языков могут быть разные приоритеты: например, в языках с динамической типизацией на первом месте часто бывает скорость развертывания.

Начнем с классификации «интерпретируемые — компилируемые».

8.1.1. Интерпретируемые и компилируемые языки

В *интерпретируемых языках* каждый шаг исходного кода выполняется по отдельности, а в *компилируемых языках* компилятор сначала преобразует весь человекочитаемый исходный код в двоичный машинный код, а уже этот код потом выполняется.

В последнее время граница между этими категориями размывается. В начале 1990-х деление было вполне четким: C/C++, FORTRAN и их соплеменники были компилируемыми языками, а Perl и Python — интерпретируемыми. Но как отмечалось в главе 1, Java сочетает в себе качества и компилируемых, и интерпретируемых языков. Концепция байт-кода еще сильнее запутывает ситуацию: безусловно, это не человекочитаемый, но и не машинный код.

https://t.me/javalib

Для языков на JVM, которые встретятся в этой части книги, мы будем руководствоваться другим критерием: создает ли язык на основе исходного кода файл класса, который выполняет, или нет. Во втором случае используется интерпретатор (вероятно, написанный на Java), который выполняет исходный код построчно. В некоторых языках есть и компилятор, и интерпретатор, а в некоторых других — интерпретатор и JIT-компилятор, который генерирует байт-код JVM.

8.1.2. Динамическая и статическая типизация

В языках с динамической типизацией переменная может в разные моменты выполнения программы содержать значения разных типов. Например, рассмотрим простой фрагмент кода в популярном динамическом языке JavaScript. Хочется надеяться, что пример будет понятен, даже если вы не знаете язык в совершенстве:

```
var answer = 40;
answer = answer + 2;
answer = "Каков ответ? " + answer;
```

Здесь ключевое слово `var` создает новую переменную, которая в динамической системе типов JavaScript может содержать значение любого типа. Сначала этой переменной присваивается `40` — это, безусловно, числовое значение. Затем мы прибавляем к нему `2` и получаем `42`. А потом мы меняем концепцию и сохраняем в `answer` строковое значение. В динамических языках подобные ситуации встречаются довольно часто и не приводят к синтаксическим ошибкам.

Интерпретатор JavaScript также умеет различать два варианта применения оператора `+`. В первый раз мы использовали его для арифметического сложения (прибавили `2` к `40`), но уже в следующей строке интерпретатор по контексту определяет, что разработчик имел в виду конкатенацию строк.

А теперь попробуем этот прием в Java с помощью JShell:

```
jshell> var answer = 40;
answer ==> 40

jshell> answer = answer + 2;
answer ==> 42

jshell> answer = "Каков ответ? " + answer;
|  Error:
|  incompatible types: java.lang.String cannot be converted to int[1]
|  answer = "What is the answer? " + answer;
|           ^----------------------------^
```

Увы и ах! Хотя абсолютно одинаковый исходный код казался допустимым в обоих языках, система статических типов Java не позволила выполнить последнюю строку. Ключевое слово `var` в Java не просто создает переменную `answer`. Как

[1] Ошибка: несовместимые типы: java.lang.String нельзя преобразовать в int. — *Примеч. пер.*

https://t.me/javalib

вы узнали в разделе 1.3, `var` также выводит тип новой переменной из правой части выражения. Нам не понадобилось явно указывать тип переменной `answer`, но система статических типов Java присвоила ей тип, который в дальнейшем никогда не изменится.

> **ПРИМЕЧАНИЕ** Здесь принципиально то, что динамическая типизация определяет тип данных на основании *значения*, которое содержится в переменной (например, число или строка), тогда как статическая типизация определяет тип на основании определения *переменной*.

Статическая типизация хорошо подходит для компилируемых языков, потому что информация о типе относится к переменным, а не к их значениям. Это позволяет выявлять потенциальные нарушения системы типов во время компиляции — еще до того, как код получит возможность выполниться.

Языки с динамической типизацией перекладывают ответственность за информацию о типе на значения, которые содержатся в переменных. Это исключительно гибкий подход, но он допускает несоблюдение типов во время выполнения («Я думал, что там число, а там строка»). Это может привести к лишним ошибкам на стадии выполнения, отлаживать которые сложнее и дороже, чем ошибки на стадии компиляции.

8.1.3. Императивные и функциональные языки

Java — классический пример *императивного языка*. Такие языки моделируют состояние выполнения программы в виде изменяемых данных и задают список инструкций, которые преобразуют это состояние. Таким образом, в императивных языках понятие состояния программы занимает центральное место.

Существуют два основных подтипа императивных языков. *Процедурные языки* — такие, как BASIC и FORTRAN, — рассматривают код и данные по отдельности, используя простую парадигму «код работает с данными». К другому подтипу относятся *объектно-ориентированные* (OO) языки, в которых данные и код (в форме методов) совместно упаковываются в объекты. Состояние программы в объектно-ориентированной системе складывается из состояния всех объектов в программе. В объектно-ориентированных языках метаданные (такие, как информация классов) в той или иной степени формируют дополнительную структуру. Впрочем, различия между этими подтипами не всегда очевидны. Например, C++ официально предназначен для того, чтобы поддерживать как объектно-ориентированное, так и процедурное программирование, а некоторые поздние разновидности BASIC могут похвастаться объектно-ориентированными возможностями.

Функциональные языки исходят из убеждения, что сами вычисления являются наиболее важной концепцией. Как и в процедурных языках, функции работают со значениями, но вместо того чтобы изменять свои входные данные, они действуют по образцу математических функций и возвращают новые значения. Фундаментальную роль в этой модели играет комбинирование нескольких функций новыми и неожиданными способами.

https://t.me/javalib

Как показано на рис. 8.1, функции рассматриваются как «маленькие обрабатывающие машины», которые получают входные значения и выдают новые значения на выходе. У них нет собственного состояния, и нет особого смысла упаковывать в них какое-то внешнее состояние. Объектно-центрическая картина мира иногда вступает в противоречия с естественными принципами функциональных языков.

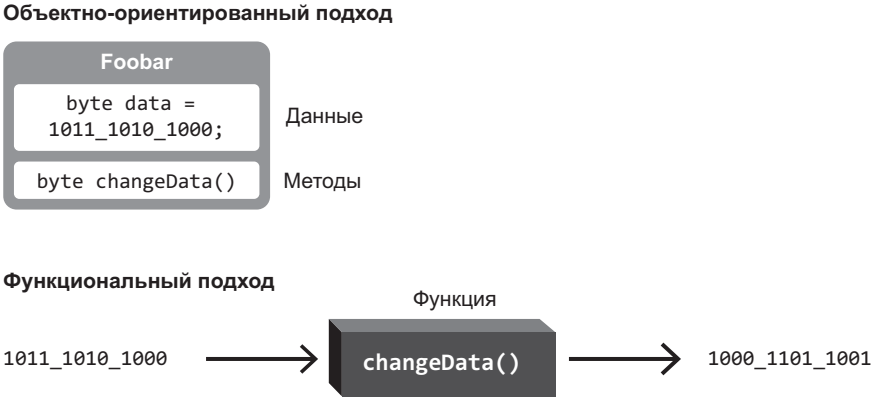


Рис. 8.1. Императивные и функциональные языки

Ключевая особенность функциональных языков — *функции первого класса*, с которыми можно работать как со значениями: присваивать их переменным, передавать другим функциям и даже возвращать одни функции из других.

Это отличный пример того, как языки обрастают функциональностью из разных парадигм, потому что в Java 8 добавился синтаксис лямбда-выражений, который позволяет Java-программистам работать с функциями как со значениями. Однако эта возможность появилась относительно недавно и пока не используется на платформе так широко, как могла бы. На практике продолжают применяться старые средства с похожим поведением, такие как `Runnable` и `Callable`.

В следующих двух главах мы поговорим о разных языках, уделяя основное внимание тому, как в них поддерживаются возможности функционального программирования. На примере Kotlin мы покажем, как даже императивный язык можно спроектировать так, чтобы он элегантно охватывал функциональные принципы. А затем мы рассмотрим Clojure — намного более чистый функциональный язык, который уже вообще не опирается на объектную ориентацию.

8.1.4. Оригинал и повторная реализация

Еще одна важная классификация делит языки JVM на повторные реализации существующих языков и языки, которые разработаны специально для JVM. В общем случае у специально разработанных языков системы типов намного теснее связаны с «родными» типами JVM.

https://t.me/javalib

Перечислим несколько повторных реализаций существующих языков для JVM:

- *JRuby* — реализация языка программирования Ruby для JVM. Ruby — объектно-ориентированный язык с динамической типизацией и некоторыми функциональными возможностями. Обычно он интерпретируется на JVM, но в последних версиях появился JIT-компилятор времени выполнения, который в подходящих условиях генерирует байт-код JVM.
- *Jython* — проект, который запустил в 1997 году Джим Хьюгунин (Jim Hugunin) как средство использования высокопроизводительных библиотек Java из Python. Это повторная реализация Python для JVM, поэтому это динамический, в основном объектно-ориентированный язык. Он генерирует внутренний байт-код Python, а затем преобразовывает его в байт-код JVM. К сожалению, после 2015 года проект почти не развивается, и он поддерживает только Python 2.7, а не текущую версию Python 3.
- Проект *Rhino* был изначально разработан компанией Netscape, а позднее перешел к Mozilla. Это реализация JavaScript для JVM, которая распространялась через JDK.

  В JDK 8 входит новое ядро JavaScript, которое называется *Nashorn* (это каламбур: по-немецки это слово означает «носорог», как и «Rhino» по-английски), но высокий темп изменений в языке JavaScript привел к тому, что это ядро было признано устаревшим в JDK 11 и исключено из JDK 15. И хотя ни одну реализацию JavaScript не планируется распространять в составе будущих JDK, оба продукта можно загрузить отдельно. Rhino от Mozilla доступен по адресу https://github.com/mozilla/rhino, а Nashorn (https://openjdk.java.net/projects/nashorn/) существует как независимый проект OpenJDK, для которого ожидается поддержка в будущих JDK.

  ПРИМЕЧАНИЕ Непросто выяснить, какой альтернативный язык для JVM был самым первым. Например, проект Kawa (реализация Lisp) появился в 1997 году или около того. В последующие годы альтернативные языки развивались так бурно, что их стало практически невозможно отслеживать.

По нашим оценкам, на момент написания книги существует не менее 200 языков для JVM. Не все они активны или широко используются (а некоторые занимают очень узкую нишу), но их внушительное количество говорит о том, что JVM остается весьма активной платформой для развития и реализации языков.

ПРИМЕЧАНИЕ Начиная со спецификации JVM, которая появилась вместе с Java 7, из этой спецификации были исключены все прямые ссылки на язык Java. Сейчас Java официально считается всего лишь одним из многих языков, которые работают на JVM, и у него нет никакого привилегированного статуса.

Ключевая технология, которая позволяет всем этим языкам выполняться на JVM, — это формат файла класса, который рассматривался в главе 4. Любой язык, который умеет производить файлы класса, может считаться компилируемым языком для JVM.

https://t.me/javalib

Перейдем к вопросу о том, что такое многоязычное программирование и как оно вошло в сферу интересов Java-программистов. Мы объясним базовые концепции и расскажем, зачем в вашем проекте может понадобиться альтернативный язык для JVM и как его выбрать.

8.2. МНОГОЯЗЫЧНОЕ ПРОГРАММИРОВАНИЕ НА JVM

Термином «*многоязычное программирование* (polyglot programming) *на JVM*» обозначаются проекты, в которых, помимо основного кода на Java, используются один или несколько других языков для JVM. Многоязычное программирование часто рассматривают как форму разделения ответственности. Как показано на рис. 8.2, потенциально существуют три уровня, на которых могут быть полезны технологии, не относящиеся к Java. Эта диаграмма иногда называется пирамидой многоязычного программирования и отчасти базируется на работе Олы Бини (https://olabini.com/blog/tag/polyglot/).

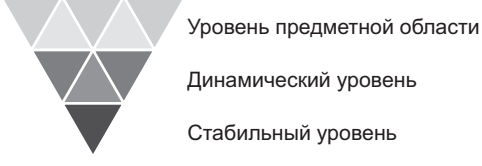


Рис. 8.2. Пирамида многоязычного программирования

Внутри пирамиды зависимости идут в одном направлении: стабильный уровень относительно независим, динамический уровень опирается на стабильный, а уровень предметной области может пользоваться обоими нижележащими уровнями.

В реальных системах не всегда просто разграничить эти уровни: существуют «серые зоны», и не все системы идеально вписываются в эту схему. Тем не менее она помогает выявить ситуации, когда разные части системы обладают разной спецификой и поэтому могут выиграть от использования других языков.

На стабильном уровне находятся базовые API и абстракции для системы. Для этого уровня критичны типобезопасность, тщательное тестирование и производительность.

На динамическом уровне используются абстракции стабильного уровня, чтобы создать рабочую систему. Например, к этой категории может относиться код для представления системы по HTTP или для взаимодействия с другими внутренними подсистемами. На этом уровне может иметь смысл использовать альтернативный язык, если он сокращает время компиляции и обеспечивает гибкость.

Уровень предметной области имеет дело с такими прикладными факторами, как представление, настройка правил и обработки или CI/CD (непрерывная интеграция и непрерывное развертывание). Этот код относится к специфическим

https://t.me/javalib

аспектам предметной области приложения, и для него могут подойти языки, которые создавали бы лишние ограничения на других уровнях.

ПРИМЕЧАНИЕ Принцип многоязычного программирования жизнеспособен, потому что разным участкам кода свойственна разная продолжительность жизни. Подсистема оценки рисков в банковской системе может жить пять и более лет. Веб-страницы на JSP часто существуют по несколько месяцев. Самый недолговечный код стартапа может прожить всего несколько дней. Чем дольше существует код, тем он ближе к стабильному уровню пирамиды (табл. 8.1).

Таблица 8.1. Три уровня пирамиды многоязычного программирования

Название	Описание	Примеры
Уровень предметной области	Язык предметной области. Тесно связан с определенной частью предметной области приложения	Apache Camel, языки DSL, Drools, веб-шаблоны
Динамический уровень	Быстрая, эффективная, гибкая разработка функциональности	Clojure, Groovy, JRuby
Стабильный уровень	Базовая функциональность; стабильный, хорошо протестированный, производительный код	Java, Kotlin, Scala

Как видите, в уровнях проявляются определенные закономерности: языки со статической типизацией скорее тяготеют к задачам стабильного уровня. И наоборот, языки, которые предназначены для более узких задач, обычно более уместны на вершине пирамиды.

Давайте глубже разберемся, почему нельзя считать Java оптимальным языком сразу для всех уровней пирамиды. Сначала мы поговорим о том, для чего вообще стоит рассматривать другие языки, кроме Java, а потом перечислим ряд критериев, которые необходимо учитывать, если вы выбираете альтернативный язык для своего проекта.

8.2.1. Зачем нужны другие языки?

Многие преимущества Java связаны с тем, что это компилируемый язык общего назначения со статической типизацией. Эти качества прекрасно помогают реализовывать функциональность стабильного уровня. Но на верхних уровнях пирамиды они же становятся обузой, и вот почему:

- Перекомпиляция слишком трудоемка.
- Статическая типизация не дает нужной гибкости.
- Развертывание — весьма обременительный процесс.
- Синтаксис Java может оказаться слишком жестким, и он плохо подходит для предметно-ориентированных языков (DSL).

https://t.me/javalib

Перекомпиляция и повторная сборка проекта на Java нередко занимает полторы-две минуты. Это способно серьезно нарушить производственные процессы разработки и плохо подходит для кода, который, возможно, будет находиться в реальной эксплуатации всего несколько недель.

Жесткий синтаксис Java

Язык Java отличается крайне жесткой грамматикой. Фундаментальными компонентами языка служат строго определенные ключевые слова. Невозможно «изобрести новый синтаксис» или создать новую конструкцию, которую можно ошибочно принять за ключевое слово.

Программист может создавать новые классы, а их функциональность состоит в том, чтобы хранить состояние в полях и вызывать методы для классов или объектов. Тем не менее этим все и ограничивается: программист не может создать ничего, что напоминало бы управляющую структуру. Иначе говоря, обращение к полю всегда выглядит так:

```
anObject.someField
AClass.someStaticField
```

А вызов метода всегда выглядит примерно так:

```
anObject.someMethod(params)
AClass.someStaticMethod(params)
```

У методов Java не бывает необязательных параметров (в отличие от некоторых других языков, таких как Kotlin), так что обращения к полям всегда четко отличаются от вызовов методов. Нельзя создать конструкцию, которая выглядит как ключевое слово. Предположим, нам хотелось бы создать управляющую конструкцию `when`:

```
when(value) {
  // Выполняемое действие
}
```

Однако лучшее, что можно сделать, выглядит примерно так:

```
import static when.When.when;

...

when(value, () -> {
  // Выполняемое действие
});
```

Невозможность переопределить синтаксис также проявляется, если пытаться с помощью Java создавать предметно-ориентированные языки. В следующих двух главах вы увидите, как эта проблема решается в альтернативных языках.

В целом самым прагматичным решением будет использовать Java там, где он проявляет свои сильные стороны. Богатый API и поддержка библиотек помогут справиться со всей тяжелой работой на стабильном уровне.

https://t.me/javalib

Впрочем, даже на этом уровне бывают причины, чтобы использовать альтернативные языки:

- Пространность кода Java порой отпугивает разработчиков, и за ней могут скрываться некоторые классы ошибок.
- Хотя со временем Java все лучше поддерживает функциональное программирование, некоторые его паттерны все еще не удается применять без неоправданных затруднений.
- В других языках поддерживаются альтернативные средства конкурентного выполнения, которых нет в Java (корутины в Kotlin, агенты в Clojure).

ПРИМЕЧАНИЕ Даже если вы решили использовать на стабильном уровне другой язык ради его функциональности, не стоит выбрасывать действующий код и переписывать его только ради того, чтобы привести все к одному языку. Можно использовать новый язык только там, где нужна новая функциональность, или для участков с низкой степенью риска — позднее в этой главе мы расскажем, как их выявлять.

Возможно, к этому моменту вы задаете себе вопросы: какие проблемы из области программирования соответствуют разным уровням? Какие языки выбрать? Основательный Java-разработчик знает, что «серебряной пули» не существует, но есть определенные критерии, на которые стоит опираться, сравнивая разные варианты.

8.2.2. Перспективные языки

Для оставшейся части книги мы выбрали два языка, у которых, по нашему мнению, есть огромный потенциал, чтобы стать долгоживущими и влиятельными. Это Kotlin и Clojure — языки для JVM, которые уже завоевали хорошую репутацию среди многоязычных программистов. Почему же эти языки набирают обороты? Рассмотрим их поочередно.

Kotlin

Kotlin — императивный объектно-ориентированный язык со статической типизацией от компании JetBrains (которая создала IntelliJ IDEA). Он стремится снять основные претензии, которые предъявляют к Java, сохраняя знакомую среду разработки. Kotlin — компилируемый язык, который отличается высокой совместимостью — в дополнение к совместимости, которую обеспечивает сам факт выполнения на JVM.

Среди ключевых особенностей Kotlin — компактный синтаксис, `null`-безопасность, чрезвычайно высокая совместимость с кодом Java, а также корутины — механизм конкурентного выполнения, альтернативный по отношению к традиционной потоковой модели Java. Некоторые возможности Kotlin просочились в Java в недавних выпусках, что показывает, насколько ценными они оказались для разработчиков.

https://t.me/javalib

Хотя Kotlin обосновался в качестве приоритетного языка для JVM в нескольких областях, его особые успехи связаны с программированием для мобильных устройств; в 2019 году он был принят как рекомендуемый язык на платформе Android. Kotlin также совместим со сценариями сборки Gradle на таком же уровне, как Groovy. Его начали поддерживать многие другие фреймворки, например Spring. Многочисленные улучшения удобства и безопасности для разработчиков заслуживают внимания независимо от того, где работает JVM. В главе 9 вас ждет введение в Kotlin.

В главе 11 мы будем рассматривать Kotlin как основной язык сценариев для сборок Gradle. Мы также вернемся к нему, чтобы продемонстрировать некоторые уникальные методы функционального программирования в главе 15 и конкурентного программирования (корутин) в главе 16.

Clojure

Clojure, созданный Риком Хики (Rich Hickey), — язык из семейства Lisp. Он унаследовал от своего предка много синтаксических возможностей (и много круглых скобок). Это функциональный язык с динамической типизацией, как это свойственно разновидностям Lisp. Язык Clojure — компилируемый, однако программы на нем обычно распространяются в виде исходного кода по причинам, которые мы рассмотрим позже. Также он добавляет в ядро Lisp немало новых возможностей, особенно в области конкурентности.

Обычно считается, что Lisp предназначен только для экспертов. Clojure несколько проще в изучении, чем другие разновидности Lisp, но при этом он предоставляет разработчикам мощные возможности (а также отлично вписывается в стиль разработки через тестирование). Однако, скорее всего, Clojure останется за пределами массовой разработки, и в основном его будут использовать энтузиасты для специализированных задач. (Например, в некоторых финансовых приложениях его возможности оказываются весьма привлекательными.)

Clojure лучше всего рассматривать как язык динамического уровня, хотя благодаря конкурентности и другим возможностям можно сказать, что он способен выполнять и многие задачи языка стабильного уровня. Введению в Clojure посвящена глава 10.

Мы будем активно обращаться к Clojure, когда начнем рассматривать функциональное программирование за пределами возможностей Java в главе 15. Также в главе 16 этот язык поможет описать модель акторов — мощную альтернативу в области конкурентного программирования.

8.2.3. Языки, которые мы могли выбрать (но не выбрали)

Как упоминалось ранее, существует огромный ассортимент языков, которые мы могли бы рассмотреть. Мы чуть подробнее расскажем о нескольких кандидатах, которые, возможно, заслуживают того, чтобы вы глубже познакомились с ними самостоятельно.

https://t.me/javalib

Groovy

Язык Groovy изобрел Джеймс Стрэкан (James Strachan) в 2003 году. Этот динамический компилируемый язык по синтаксису очень похож на Java, но более гибок. Groovy широко применяется для сценариев и тестирования. В частности, этот язык изначально использовался в системе сборки Gradle, и его применяют для настройки Jenkins — чрезвычайно распространенного инструмента CI/CD. Разработчики на JVM часто изучают его как первый язык, отличный от Java. Groovy хорош как язык динамического уровня, и с его помощью удобно создавать предметно-ориентированные языки.

Мы решили не рассматривать Groovy подробнее, потому что сейчас он теряет популярность и в области прототипирования, и в разработке приложений, а на смену ему приходят активно развивающиеся фреймворки и другие языки.

Scala

Scala — это объектно-ориентированный язык, который также поддерживает многие аспекты функционального программирования. Его история началась в 2003 году, когда Мартин Одерски (Martin Odersky) начал работать над ним в лабораторных условиях после более ранних проектов в области обобщений в Java. Scala, как и Java, является компилируемым языком со статической типизацией, а также обладает развитым механизмом выведения типов, поэтому хорошо вписывается в динамический уровень.

Scala многому научился у Java, и при его проектировании были «исправлены» некоторые типичные недостатки Java. Однако в итоге Scala развил чрезвычайно богатую функциональность и гораздо более совершенную систему типов по сравнению с Java. Программировать на нем — не самое легкое занятие, да и основательно изучить его не так-то просто. Поэтому мы решили сосредоточиться на Kotlin, чтобы наша книга была полезна для разработчиков, которых больше интересуют улучшения уже знакомого языка Java.

GraalVM

Организация Oracle Labs выпустила GraalVM (*https://www.graalvm.org/*), которую создатели описывают как многоязычную виртуальную машину и платформу, частично основанную на кодовой базе Java и JVM. Текущий выпуск позволяет запускать Java и другие языки для JVM (в виде байт-кода), поддерживает JavaScript и бит-код LLVM (промежуточное представление, которое производит популярный компилятор LLVM), а также обеспечивает бета-поддержку Ruby, Python, R и WASM.

В состав платформы входят следующие компоненты:

- Виртуальная машина Java HotSpot.
- Node.js — среда выполнения JavaScript.
- Среда выполнения LLVM для выполнения бит-кода LLVM.

https://t.me/javalib

- Graal — JIT-компилятор, написанный на Java.
- Truffle — набор инструментов и API, который позволяет создавать интерпретаторы языков.
- SubstrateVM — облегченный контейнер выполнения для низкоуровневых образов.

GraalVM позволяет довольно свободно интегрировать языки друг с другом, чтобы компоненты, реализованные на базе разных технологий, можно было объединять и использовать в одном прикладном процессе. Это совершенно иной подход к многоязычному программированию, однако он достаточно близок к нашей теме, поэтому мы решили хотя бы упомянуть о нем.

Языки, не предназначенные для JVM

Эта глава посвящена альтернативным языкам, которые выполняются на JVM. Однако следует признать, что иногда у многоязычных программистов могут появиться причины полностью вывести часть своей системы за пределы JVM.

Вот примеры областей, в которых популярные языки программирования развиваются в первую очередь вне JVM:

- Низкоуровневый системный код (C, Go или Rust).
- Машинное обучение (Python).
- Выполнение в веб-браузере (JavaScript).

Для многих из этих областей существуют также решения на базе JVM, но прежде чем пытаться их применять, стоит учесть, что альтернативы вне JVM — более зрелые, а ваша команда разработчиков может оказаться недостаточно подготовленной для этих решений.

Теперь, когда мы обрисовали возможные варианты, давайте обсудим критерии, на основании которых стоит выбирать альтернативный язык.

8.3. КАК ВЫБРАТЬ АЛЬТЕРНАТИВНЫЙ ЯЗЫК ДЛЯ ПРОЕКТА

Если вы решили в своем проекте поэкспериментировать с языками, отличными от Java, важно определить, какие части проекта естественным образом относятся к тому или иному уровню: стабильному, динамическому и уровню предметной области. В табл. 8.2 перечислены задачи, которые могут оказаться наиболее подходящими для каждого уровня.

Как видите, у альтернативных языков есть широкий диапазон сценариев практического применения. Однако выявить задачу, которую можно решить с помощью альтернативного языка, — это только начало. На следующем шаге нужно

https://t.me/javalib

оценить, уместно ли прибегать к альтернативному языку. Вот несколько полезных критериев, которые следует учитывать при выборе технологических стеков:

- Является ли область проекта низкорискованной?
- Насколько просто этот язык взаимодействует с Java?
- Как этот язык поддерживается на уровне инструментов (например, в IDE)?
- Легко ли изучить этот язык?
- Легко ли нанять программистов с опытом работы на этом языке?

Таблица 8.2. Области проекта, подходящие для разных уровней

Уровень	Описание	Примеры задач
Предметной области	В областях, специфических для предметной области, часто бывает полезна удобочитаемость для экспертов, которые могут не знать Java. В инструментах управления жизненным циклом ПО тоже часто используются языки и конфигурация, специфические для предметной области	Сборка, непрерывная интеграция, непрерывное развертывание. DevOps. Моделирование бизнес-логики
Динамический	На динамических уровнях системы может пригодиться бóльшая гибкость и скорость разработки, доступная в других языках. Это может быть особенно актуально для инструментов, которые предназначены для внутренних задач (тестирование, администрирование)	Быстрая веб-разработка. Прототипирование. Интерактивные административные и пользовательские консоли. Сценарии. Тестирование
Стабильный	Код стабильного уровня выражает основные абстракции системы. Более строгая типобезопасность и тестирование стоят дополнительных усилий со стороны разработчика	Конкурентный код. Контейнеры приложений. Основная бизнес-функциональность

Давайте подробнее рассмотрим каждый из этих критериев.

8.3.1. Является ли область проекта низкорискованной?

Допустим, у вас есть подсистема обработки платежей, которая ежедневно обрабатывает миллионы операций. Это стабильный блок кода на Java, который эксплуатируется уже семь лет, но для него написано не так много тестов, и в коде немало «темных углов». Очевидно, что в контексте перехода на новый язык подсистема обработки платежей является областью с высоким риском, особенно если она успешно работает, тестовое покрытие оставляет желать лучшего и к тому же у вас нет разработчиков, которые понимают ее во всех подробностях.

https://t.me/javalib

Но другой язык может пригодиться не только для того, чтобы переводить на него базовую функциональность системы. Например, в нашей ситуации явно не помешало бы улучшить тесты. В Kotlin есть для этого хорошие инструменты: в том числе фреймворки Spek (https://www.spekframework.org/) и Kotest (https://kotest.io), которые позволяют получить четкие, удобочитаемые спецификации без шаблонного кода, характерного для JUnit. А может быть, вашему ядру правил пошло бы на пользу *тестирование свойств*, при котором пишутся тесты, чтобы проверять условия по сгенерированному вводу; в таком случае вам могла бы помочь библиотека Clojure test.check (https://clojure.org/guides/test_check_beginner).

Или допустим, вы хотите создать веб-консоль, чтобы операторы эксплуатации могли администрировать некоторые некритические статические данные, относящиеся к системе обработки платежей. Предположим, что разработчики уже знают Struts и JSF, но относятся к ним обоим без особого энтузиазма. Это еще одна низкорискованная область, в которой можно опробовать новый язык и технологический стек. Например, очевидным вариантом может стать Spring Boot с Kotlin.

Если вы начнете с ограниченной пилотной версии с низким риском, у вас всегда остается возможность прекратить проект и переключиться на другую технологию без серьезных последствий, если окажется, что новый технологический стек плохо подходит для ваших целей.

8.3.2. Насколько просто язык взаимодействует с Java?

Никому не захочется отказываться от всего кода на Java, который уже написан. Это одна из главных причин, по которым организации неохотно добавляют в свою обойму новые языки программирования. Но альтернативные языки для JVM можно внедрить так, чтобы не отбрасывать существующий код, а, наоборот, максимизировать пользу имеющейся кодовой базы.

Альтернативные языки для JVM способны чисто взаимодействовать с Java, и их, конечно, можно развертывать в существующих средах. Это особенно важно, чтобы не создавать проблем тем, кто отвечает за развертывание, будь то отдел управления производством или специалисты по DevOps из вашей собственной команды. Используя альтернативный язык как часть системы, вы сохраняете опыт работы своей организации, что помогает избавиться от беспокойства и сократить риски, связанные с поддержкой нового решения.

> **ПРИМЕЧАНИЕ** Языки DSL обычно создаются с помощью языков динамического (а в отдельных случаях стабильного) уровня, так что многие из них запускаются на JVM на этих самых языках.

Одни языки взаимодействуют с Java лучше, чем другие. Мы убедились, что все популярные альтернативы JVM (такие, как Kotlin, Clojure, JRuby, Groovy и Scala) хорошо интегрируются с Java (а для некоторых языков интеграция проходит превосходно, практически незаметно). Если вы предпочитаете осторожный подход, стоит сначала провести несколько простых и быстрых экспериментов, чтобы убедиться, что вы понимаете, как работает интеграция.

https://t.me/javalib

Возьмем для примера Kotlin. Пакеты Java можно импортировать прямо в его код знакомой инструкцией `import`. После этого вы можете легко написать небольшой сценарий на Kotlin или даже использовать интерактивную оболочку Kotlin, чтобы «пощупать» объекты модели Java и понять, как работают механизмы взаимодействия. В последующих главах, посвященных Kotlin, мы отдельно рассмотрим его интеграцию с Java.

8.3.3. Насколько развита инструментальная и тестовая поддержка для языка?

Многие разработчики недооценивают время, которое они экономят, если эффективно используют свою среду разработки. Мощные IDE, средства сборки и тестирования помогают быстро производить высококачественные программные продукты. За долгие годы Java-разработчики привыкли к отличной инструментальной поддержке этого языка, поэтому важно понимать, что другие языки не всегда отличаются такой же технологической зрелостью.

Для некоторых языков (таких, как Kotlin) уже давно есть развитые средства IDE, которые помогают компилировать, тестировать и развертывать конечный продукт. В других языках инструменты разработчика еще не достигли аналогичного уровня.

Сопутствующая проблема заключается в том, что когда в альтернативном языке появляется какой-нибудь мощный инструмент для внутреннего использования (например, великолепная система сборки Leiningen для Clojure), он не всегда хорошо подходит для других языков. В результате разработчикам приходится тщательно продумывать, как разделять проект, особенно в отношении развертывания разных, но логически связанных компонентов.

8.3.4. Легко ли изучить язык?

Освоение нового языка всегда требует времени, и это время только возрастает, если парадигма языка отличается от той, к которой привыкла ваша команда разработки. Большинству Java-разработчиков было бы комфортно изучать новый язык, если бы он был объектно-ориентированный с C-подобным синтаксисом (как Kotlin).

Если разработчикам приходится отходить от этой парадигмы, ситуация усложняется. Например, Clojure — самый яркий представитель популярных альтернативных языков — может наделить программистов колоссальной мощью, но ради этого разработчикам придется капитально переобучаться, осваивая функциональные средства Clojure и синтаксис Lisp.

Один из промежуточных вариантов — обратиться к существующим языкам, которые были повторно реализованы для JVM. Например, Ruby и Python — прочно утвердившиеся языки, а разработчикам доступно много материала для самообучения. JVM-версии этих языков могут послужить хорошей отправной точкой, с которой ваша команда начнет работать с доступным альтернативным языком.

https://t.me/javalib

8.3.5. Насколько популярен этот язык среди разработчиков?

Организации вынуждены быть прагматичными: они не всегда могут нанять 2 % лучших специалистов (что бы ни говорилось в их рекламе), а в командах разработки всегда будет естественная текучка кадров. Например, языки Kotlin и Scala уже вышли на достаточно массовый уровень, и у каждого из них сформировался пул разработчиков, из которого можно выбирать кандидатов для найма. Но с таким языком, как Clojure, могут возникнуть трудности. Руководители могут сопротивляться всему нестандартному, потому что не хотят допустить ситуацию, когда кодовая база останется без сопровождения, потому что будет трудно найти соответствующих специалистов.

> **ПРИМЕЧАНИЕ** Стоит сделать еще одно предупреждение по поводу повторных реализаций. Например, многие существующие пакеты и приложения, написанные на Ruby, тестируются только с исходной реализацией на базе C, так что в JVM с ними могут возникнуть проблемы. Принимая решения относительно платформы, закладывайте дополнительное время на тестирование, если вы хотите задействовать весь стек заново реализованного языка.

Эту проблему снова помогут решить заново реализованные языки (JRuby, Jython и т. д.). У немногих разработчиков в резюме упоминается JRuby, но поскольку это всего лишь Ruby для JVM, на самом деле резерв кандидатов для найма достаточно широк: если разработчик знаком с C-версией Ruby, он без труда изучит различия, связанные со спецификой JVM.

Итак, у нас есть набор вопросов, которыми следует задаться при выборе альтернативного языка, а также обзор некоторых доступных вариантов. Теперь стоит глубже разобраться, как JVM поддерживает разные языки. В следующем разделе рассказывается, откуда взялись некоторые проектировочные решения и ограничения в альтернативных языках для JVM.

8.4. КАК JVM ПОДДЕРЖИВАЕТ АЛЬТЕРНАТИВНЫЕ ЯЗЫКИ

Есть два основных механизма, с помощью которых языки выполняются на JVM:

- Существует компилятор исходного кода, который генерирует файлы классов. Именно так работают Kotlin и Clojure.
- Существует интерпретатор языка, реализованный в байт-коде JVM. Этот вариант выбран для JRuby.

В обоих случаях обычно существует среда выполнения, которая предоставляет специальные средства поддержки программ для данного языка. На рис. 8.3 изображен стек среды выполнения для Java и для типичного альтернативного языка.

Эти вспомогательные системы времени выполнения могут различаться по сложности в зависимости от того, какая степень поддержки требуется для того

https://t.me/javalib

или иного альтернативного языка на стадии выполнения. Почти всегда среда выполнения реализуется в виде набора файлов JAR или модулей, которые должны быть в classpath исполняемой программы. Если язык интерпретируется, то интерпретатор активизируется при запуске программы, а затем считывает исходный файл.

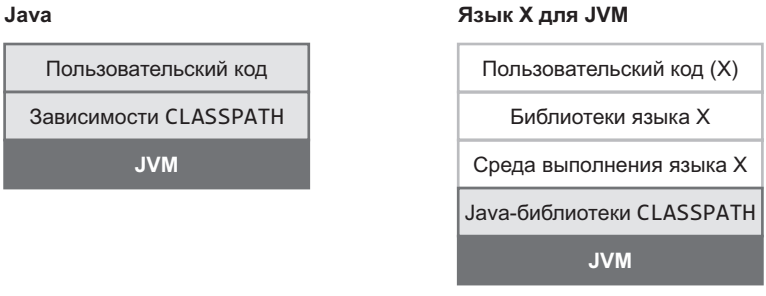


Рис. 8.3. Среда выполнения для Java и альтернативного языка

8.4.1. Производительность

Один из популярных вопросов, которые задают разработчики, — как сравнить производительность разных альтернативных языков? Вопрос выглядит заманчиво, но ответить на него не так просто, да и ответ получится не очень конструктивным.

Как говорилось в главе 7, любой опытный разработчик знает, что производительность оценивается по результатам измерений, а измерения проводятся с конкретными программами, а не с абстрактным понятием языка программирования. Если вы слышите, что «язык X превосходит Y по производительности», — не спешите доверять таким заявлениям, пока не увидите убедительные объективные данные в их пользу.

Тем не менее на самом поверхностном уровне о производительности языков JVM можно судить по реализации языка. Компилируемый язык на стадии выполнения представляет собой тот же байт-код, и он будет JIT-компилироваться точно так же, как Java. Интерпретируемый язык будет существенно отличаться по производительности, потому что JIT-компилируемым кодом будет интерпретатор, а не сама программа.

> **ПРИМЕЧАНИЕ** Некоторые языки (например, JRuby) применяют гибридную стратегию: они используют интерпретатор для сценариев, но также могут динамически компилировать отдельные исходные методы в байт-код JVM, который затем компилируется в машинный код JIT-компиляторами JVM.

В этой книге основное внимание уделяется компилируемым языкам. Для полноты картины мы также упоминаем интерпретируемые языки, такие как Rhino, но не будем подолгу задерживаться на них. Таким образом, мы предполагаем, что

https://t.me/javalib

языки, которые мы рассматриваем, сопоставимы по производительности. Чтобы сделать более точные выводы, нужно подробно проанализировать конкретную программу или рабочую нагрузку.

В оставшейся части этого раздела мы поговорим о том, почему альтернативным языкам (даже компилируемым) нужна поддержка среды выполнения. Затем пойдет речь о *фикциях компилятора* (compiler fictions) — специфических средствах для того или иного языка, которые синтезируются компилятором и могут отсутствовать в низкоуровневом байт-коде.

8.4.2. Среды выполнения для альтернативных языков

Простой способ оценить сложность среды выполнения, которая необходима для конкретного языка, — это посмотреть на размер файлов JAR с ее реализацией. С помощью этой метрики мы видим, что Clojure — относительно легковесная среда, а для JRuby потребуется более основательная поддержка.

Этот критерий не истина в последней инстанции, потому что одни языки упаковывают в свои дистрибутивы гораздо больше стандартных библиотек и дополнительной функциональности, чем другие. Тем не менее размер файлов JAR полезен для хотя бы приблизительной первичной оценки.

В общем случае задача среды выполнения — помочь системе типов и другим аспектам альтернативных языков обеспечить нужную семантику. Альтернативные языки не всегда придерживаются тех же базовых концепций программирования, что Java.

Например, не все языки применяют такую же объектно-ориентированную модель, как Java. В Java все объекты, которые являются экземплярами конкретного класса, содержат абсолютно одинаковый набор методов, который фиксируется на стадии компиляции. А в Ruby на стадии выполнения к отдельному экземпляру класса можно присоединить дополнительные методы, которые были неизвестны на момент определения класса и могут отсутствовать у других экземпляров того же класса.

> **ПРИМЕЧАНИЕ** Фактически байт-код `invokedynamic` был добавлен в JVM, чтобы упростить эффективную реализацию подобных языковых средств.

Чтобы можно было динамически добавлять методы, эту функциональность (которая в JVM по какой-то непонятной причине называется «открытыми классами») должна воспроизводить реализация JRuby. Это было бы невозможно без довольно сложной поддержки со стороны среды выполнения JRuby.

8.4.3. Фикции компилятора

Некоторые языковые средства синтезируются средой программирования и высокоуровневым языком и отсутствуют в используемой реализации JVM. Они называются *фикциями компилятора*.

https://t.me/javalib

ПРИМЕЧАНИЕ Полезно до некоторой степени представлять себе, как реализованы эти средства; в противном случае может оказаться, что ваш код работает слишком медленно или даже порой приводит к аварийному завершению процесса. Иногда среде приходится выполнять значительную работу, чтобы синтезировать ту или иную возможность.

К другим примерам из Java относятся проверяемые исключения и внутренние классы (которые всегда преобразуются в классы верхнего уровня со специально синтезированными методами доступа в случае необходимости, как показано на рис. 8.4). Если вы когда-нибудь заглядывали внутрь файла JAR (при помощи команды `jar tvf`) и видели множество классов с символом `$` в именах, знайте, что это внутренние классы, которые были распакованы и преобразованы в «обычные» классы.

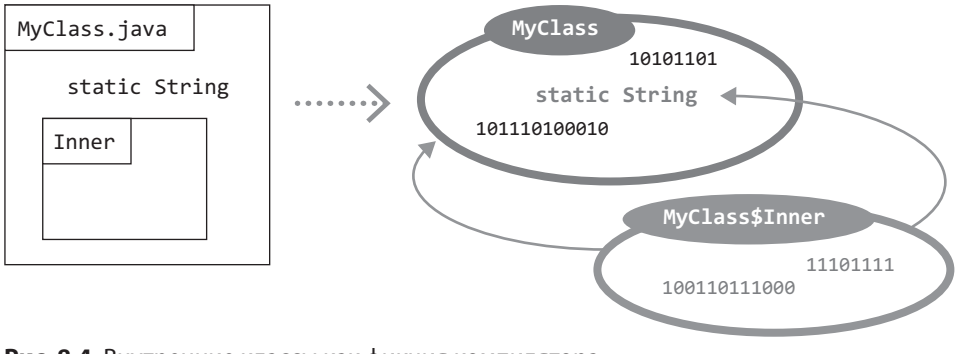


Рис. 8.4. Внутренние классы как фикция компилятора

У альтернативных языков тоже бывают фикции компилятора, которые в отдельных случаях даже входят в основную часть функциональности языка.

В разделе 8.2 мы представили важное понятие *функций первого класса* в функциональном программировании: такие функции должны быть значениями, которые можно хранить в переменных. Все альтернативные языки из части III этой книги поддерживали эту возможность задолго до того, как в Java появились лямбда-выражения. Как решалась эта задача, если в JVM только классы служат минимальной единицей кода и функциональности?

Чтобы устранить это несоответствие между исходным кодом и байт-кодом JVM, можно вспомнить, что объекты — это просто контейнеры данных с методами, которые манипулируют этими данными. Представьте объект, у которого нет состояния и есть только один метод, например простую анонимную реализацию интерфейса Java `Callable`. Нет ничего необычного в том, чтобы поместить такой объект в переменную, передать ее при вызове, а впоследствии вызвать метод `call()` этого объекта:

```
Callable<String> myFn = new Callable<String>() {
    @Override
    public String call() {
```

https://t.me/javalib

```
        return "Результат";
    }
};

System.out.println(myFn.call());
```

ПРИМЕЧАНИЕ Переменная `myFn` в этом примере имеет анонимный тип, поэтому после компиляции она будет отображаться в виде `ИмяВнешнегоКласса$1.class`. Номера классов начинаются с 1 и увеличиваются для каждого анонимного типа, обнаруженного компилятором. Если они создаются динамически и их достаточно много (как иногда бывает в языках вроде JRuby), это может создать слишком высокую нагрузку на память вне кучи, где хранятся определения классов.

Правда, лямбда-выражения Java не используют этот прием с анонимными типами, а вместо этого опираются на общую функциональность JVM `invokedynamic`, которую мы подробно рассмотрим в главе 17. Альтернативные языки тоже отказываются от специализированных реализаций в пользу `invokedynamic`. Это интересный пример того, как фикции компилятора влияют на развитие реалий платформы.

Еще один пример: в следующей главе будут представлены *классы данных* Kotlin — языковая конструкция, которая помогает снизить объем кода и сократить формальности, которые необходимы, если класс объявляется в виде «примитивного набора полей». В Kotlin в его нынешнем виде это фикция компилятора, но в Java 17 появился механизм *записей* (records), которые могут со временем стать альтернативной основой для классов данных в Kotlin.

ИТОГИ

- Альтернативные языки для JVM прошли долгий путь развития, прежде чем предоставить более эффективные решения некоторых задач, чем решения на Java.
- Языки можно классифицировать по-разному (динамические — статические, интерпретируемые — компилируемые, императивные — функциональные), и эти классификации могут помочь выбрать правильный язык для конкретной задачи.
- Многоязычное программирование часто делится на три уровня: стабильный, динамический и уровень предметной области. Java и Kotlin лучше подходят для стабильного уровня разработки, а Clojure больше приспособлен для задач динамического уровня или уровня предметной области.
- Добавляя в проект новый язык, практически никогда не стоит начинать сразу с основной бизнес-функциональности. Для первого развертывания альтернативного языка выберите область с низкой степенью риска.
- В любых командах и проектах есть уникальные особенности, которые влияют на выбор языка. Универсальных правильных решений здесь не существует.

https://t.me/javalib

9 Kotlin

В ЭТОЙ ГЛАВЕ

- ✓ Почему стоит использовать Kotlin
- ✓ Удобство и компактность
- ✓ Безопасность
- ✓ Конкурентное выполнение
- ✓ Совместимость с Java

Язык Kotlin был создан компанией JetBrains (https://jetbrains.com), которая также разработала популярную интегрированную среду IntelliJ IDEA. Kotlin, о выходе которого было объявлено в 2011 году, стремился заполнить пробелы, с которыми его создатели сталкивались в Java, и при этом обойтись без проблем совместимости, которые тогда встречались в других существующих языках для JVM.

На следующий год Kotlin перешел на модель с открытым кодом, а в 2016 году достиг версии 1.0 с гарантированными уровнями поддержки и сопровождения от JetBrains. С тех пор он получил статус рекомендуемого языка для платформы Android и приобрел надежных последователей в кругах программистов под JVM. Организация Kotlin Foundation, созданная в 2018 году, объединяет долгосрочную поддержку языка со стороны JetBrains и Google. Kotlin даже выходит за пределы JVM, поддерживая JavaScript и низкоуровневые внутренние подсистемы.

https://t.me/javalib

9.1. ПОЧЕМУ СТОИТ ИСПОЛЬЗОВАТЬ KOTLIN

Kotlin предоставляет ряд улучшений, которые значительно облегчают работу программистов по сравнению с Java, хотя и обходится без глобальных изменений. Из-за своего внимания к удобству, безопасности и надежной совместимости Kotlin завоевал отличную репутацию для постепенного внедрения в существующих проектах на Java. А поддержка в IntelliJ IDEA позволяет преобразовывать код из Java в Kotlin парой щелчков.

Стоит заметить, что отдельные возможности, которые изначально были доступны только в Kotlin, постепенно проникли в Java. Отличный пример такого рода — сценарии Kotlin: Kotlin может запустить исходный файл (как правило, с расширением kts) напрямую, не заставляя разработчика вводить команды, чтобы его скомпилировать. Если вам кажется, что это похоже на однофайловое выполнение Java 11, о котором мы говорили в главе 1, — вы абсолютно правы!

После этой главы мы будем использовать Kotlin в главе 11 как основной язык сценариев для сборок с Gradle. Также мы вернемся к Kotlin в главе 15, когда будем изучать функциональное программирование для JVM, и в главе 16, где увидим, как встроенный в Kotlin механизм корутин предоставляет привлекательную альтернативу для классической многопоточности Java. Давайте посмотрим, что же может предложить Kotlin.

9.1.1. Установка

Если вы используете IntelliJ IDEA, подключить к ней Kotlin можно с помощью плагина. Как и для всех остальных языков, которые поддерживаются в IDE, установки плагина достаточно, чтобы немедленно начать писать код на Kotlin.

Для тех, кто склонен настраивать все с чистого листа, Kotlin также предоставляет компилятор командной строки `kotlinc` (см. http://mng.bz/YGoa) и интерактивную оболочку `kotlin`.

Чтобы добавить Kotlin в существующий проект, придется обновить сценарии сборки. В главе 11 мы поговорим о них подробнее, а пока вы можете обратиться к превосходной документации Kotlin, чтобы узнать, как начать работу с Maven (http://mng.bz/GEYJ) или Gradle (http://mng.bz/z4vA). После того как Kotlin готов к работе, давайте посмотрим, как использовать его базовые возможности и в чем они улучшают Java.

9.2. УДОБСТВО И КОМПАКТНОСТЬ

Java известен как язык с весьма пространным кодом. И хотя Kotlin внешне похож на Java, он всеми силами старается сделать так, чтобы вам приходилось писать меньше кода.

https://t.me/javalib

9.2.1. Начиная с меньшего

Одним из примеров упрощений в Kotlin стал простой символ — точка с запятой. В Kotlin не обязательно завершать строки этим знаком: вместо них достаточно символа новой строки. Символы `;` не запрещены, а иногда даже обязательны, например, если вы хотите разместить несколько инструкций в одной строке, но чаще всего без них можно обойтись.

Kotlin воспользовался возможностью начать все с нуля, чтобы пересмотреть и другие настройки по умолчанию, которые было бы трудно изменить в Java. Если в Java по умолчанию импортируется только `java.lang`, Kotlin автоматически предоставляет следующие пакеты:

- `java.lang.*`
- `kotlin.*`
- `kotlin.annotation.*`
- `kotlin.collections.*`
- `kotlin.comparisons.*`
- `kotlin.io.*`
- `kotlin.ranges.*`
- `kotlin.sequences.*`
- `kotlin.text.*`
- `kotlin.jvm.*`

Программы, в которых не используются коллекции, текст или ввод/вывод, встречаются так редко, что эти включения практически всегда избавляют вас от лишнего импортирования.

Избыточность кода Java часто проявляется при работе с типами, и как мы видели в главе 1, ключевое слово `var` позволяет не дублировать информацию о типе. Такой стиль автоматического выведения типов (type inference) поддерживался в Kotlin с самого начала, хотя в работе с типами этот язык пошел гораздо дальше.

9.2.2. Переменные

Для создания переменных Kotlin использует то же ключевое слово, которое применяется в новых выпусках Java: `var`. Тип, к которому должна относиться переменная, выводится из выражения в правой части:

```
var i = 1  <-- i относится к типу kotlin.Int
var s = "String"  <-- s относится к типу kotlin.String
```

Однако, в отличие от Java, в Kotlin `var` — не просто сокращение. Если вы хотите явно обозначить тип переменной, нужно по-прежнему использовать `var`, но

https://t.me/javalib

после имени переменной добавляется тип, как показано в следующем фрагменте. Иногда это называется *аннотацией типа* (type hint):

```
var i: Int = 1
var s: String = "Строка"
var n: Int = "Ошибка"
```

Это присваивание приведет к ошибке компиляции: «Несоответствие типов: выведенный тип String, ожидаемый — Int»

В Kotlin у `var` есть «напарник» — ключевое слово `val`. С ним объявляются неизменяемые переменные, значения которых нельзя перезаписать после исходного присваивания. Это можно считать аналогом `final var` в Java — именно так там рекомендуется объявлять любые переменные, которые не должны обновляться. В Kotlin эта более безопасная настройка реализована так же компактно, как изменяемая альтернатива:

```
var i = 1
i = 2
```

Переменным var можно присваивать новые значения

```
val s = "Строка"
s = "Другая строка"
```

Ошибка компиляции: «val нельзя присвоить повторно»

`var` и `val` последовательно применяются в Kotlin для переменных и аргументов. Kotlin предпочитает по умолчанию использовать неизменяемые сущности, и это один из ключевых факторов проектирования, который мы будем постоянно встречать в языке.

Если у вас есть переменные, возникает естественное желание их сравнить. Kotlin предоставляет новые средства проверки равенства, о которых мы сейчас поговорим.

9.2.3. Проверка равенства

Вот одна из стандартных ошибок, которая встречается во многих программах на Java:

```
// Java
String s = retrieveString();
if (s == "Значение") {
    // ...
}
```

Получает откуда-то строку. Обратите внимание, что одинаковые строковые литералы могут интернироваться в один и тот же объект, что создает ложное чувство безопасности при некорректном сравнении

Не достигается, даже если используются одинаковые значения, но с разными ссылками

Оператор `==` в Java сравнивает *ссылки*, а не *значения*, поэтому такое сравнение делает совсем не то, к чему вы привыкли во многих других языках.

Kotlin устраняет это неудобство и рассматривает `==` как проверку равенства значений для распространенных типов (таких, как `String`). Фактически вызовы `==` эквивалентны `null`-безопасным вызовам `equals`. Так Kotlin оптимизирует язык для более распространенных приемов программирования и предотвращает серьезный источник ошибок, свойственный программированию на Java:

https://t.me/javalib

```
// Kotlin
var s: String = retrieveString()
if (s == "Значение") {
    // ...  ◄── Достигается, если значение s — строка «Значение»
}
```

Иногда может иметь смысл сравнивать ссылки, а не значения. Для таких случаев в Kotlin есть оператор `===` и парный оператор `!==`, которые делают то же самое, что `==` и `!=` в Java.

Сравнивать переменные важно, однако не менее важно уметь вызывать другой код или определять собственные подпрограммы.

9.2.4. Функции

Иногда в неформальной речи мы используем термины «функция» и «метод» как синонимы, но на самом деле в Java существуют только методы: нельзя определить повторно используемый блок кода вне контекста класса. Хотя Kotlin поддерживает все объектно-ориентированные удобства Java (об этом в следующем разделе), он также признает, что иногда вам может понадобиться просто функция сама по себе.

В соответствии с принципом компактности Kotlin минимальное определение функции выглядит так:

```
fun doTheThing() {
  println("Готово!")
}
```

Ключевое слово fun определяет функцию в Kotlin. Мы еще встретимся с ним, когда будем определять методы, присоединенные к классам

Это немного выбивается из традиционной идеологии Java, но такой подход легко узнаваем. Помимо ключевого слова `fun`, с которого начинается определение, главное отличие состоит в том, что нет возвращаемого типа. Если функция ничего не возвращает, в Kotlin вместо того, чтобы явно указывать `void`, можно просто ничего не указывать; тогда возвращаемый тип рассматривается как `Unit` (в Kotlin это означает «без возвращаемого значения»).

Если функция должна возвращать значение, об этом необходимо сообщить явно:

```
fun doTheThing(): Boolean {  ◄── Объявляет, что функция возвращает значение типа Boolean
  println("Готово!")
  return true  ◄── Возвращает признак успеха
}
```

Функции не принесут особой пользы, если они не принимают аргументы. Синтаксис передачи аргументов в Kotlin очень похож на объявление переменных:

```
fun doTheThing(value: Int): Boolean {
  println("Готово $value!")
  return true
}
```

Теперь функция принимает один аргумент типа Int

Внутри функции аргументы ведут себя как локально определенные переменные. Здесь используется удобный механизм интерполяции строк в Kotlin

https://t.me/javalib

Впрочем, даже простое определение функции в Kotlin может преподнести сюрприз. Иногда порядок аргументов в функции может быть неясным, особенно если их типы совпадают:

```
fun coordinates(x: Int, y: Int) {
  // ...
}
```

Чтобы вызывать эту функцию, нужно помнить, в каком порядке должны идти аргументы («сначала x, потом y»), иначе может случиться ошибка. В Kotlin эта проблема решается при помощи *именованных аргументов*:

```
fun coordinates(x: Int, y: Int) {
  // ...
}
coordinates(10, 20)
coordinates(y = 20,x = 10)
```

Обычный вызов функции с позиционными аргументами

Несмотря на другой порядок, этот вызов приводит к тому же результату, потому что указаны имена аргументов

> **ПРИМЕЧАНИЕ** Именованные аргументы нельзя использовать при вызове функций Java и при вызове функций Kotlin из Java. Такая возможность не поддерживается, потому что имена аргументов не сохраняются в байт-коде. Кроме того, изменение имен аргументов может рассматриваться как критическое изменение API в коде Kotlin, тогда как в Java проблемы могут возникнуть только при изменении типа, количества или порядка аргументов.

Иногда не обязательно передавать все аргументы, потому что существуют разумные значения по умолчанию. В Java для этого создаются перегруженные версии метода, которые получают разные наборы аргументов. Хотя в Kotlin так тоже можно сделать, лучше определить значения по умолчанию более прямолинейным способом:

```
fun callOtherServices(value: Int, retry: Boolean = false) {
  // ...
}
callOtherServices(10)
```

Аргумент retry в функции содержит значение false, заданное по умолчанию

Вместо того чтобы писать два определения `callOtherServices` — с одним и с двумя аргументами, — можно объединить оба случая в одной функции без шаблонного кода.

Другой типичный случай, для которого в Kotlin есть специальный синтаксис, — однострочные функции вроде приведенной ниже. Такие функции обеспечивают инкапсуляцию и назначают имя для конкретного вычисления или проверки. В духе общего стремления к компактности Kotlin поддерживает альтернативную форму объявления для таких коротких функций:

```
fun checkCondition(n: Int) = n == 42
```

https://t.me/javalib

Этот формат короче не только из-за того, что вместо фигурных скобок используется один знак =, но и благодаря выведению типов. Возвращаемые типы можно не указывать: Kotlin выводит тип выражения автоматически.

Эта возможность намекает на более фундаментальное свойство Kotlin — поддержку функций первого класса. Функции в Kotlin можно передавать в аргументах, сохранять в переменных и свойствах, а также возвращать из других функций. Хотя Kotlin не считается языком функционального программирования, функции первого класса позволяют ему пользоваться многими распространенными паттернами этой парадигмы.

Присвоить функцию переменной можно по-разному в зависимости от происхождения функции. Если функция была объявлена ранее, оператор :: позволяет обратиться к ней по имени:

```
fun checkCondition(n: Int) = n == 42
val check = ::checkCondition
```

В Kotlin также есть синтаксис лямбда-выражений для создания анонимных функций «на лету»:

```
val anotherCheck = { n: Int -> n > 100 }
```

Независимо от того, как было выполнено присваивание, ссылки на функции можно активизировать так, как если бы имя переменной было собственным именем функции. Также существует функция invoke, которая демонстрируется в следующем примере:

```
println(check(42))
println(anotherCheck.invoke(42))
```

Выполняет ранее объявленную функцию checkCondition, которая хранится в переменной check, и выводит true

Выполняет лямбда-функцию, которая проверяет, что значение больше 100, и выводит false

Функции можно не только присваивать локальным переменным, но и передавать другим функциям в аргументах, как любые другие значения. Это одно из основных свойств языков с функциями первого класса.

Как уже отмечалось, Kotlin требует, чтобы у аргументов был объявлен тип. Функции не являются исключением, и тип можно указать с помощью специального синтаксиса:

```
fun callsAnother(funky: (Int) -> Unit) {
  funky(42)
}
```

callsAnother принимает аргумент — функцию, которая принимает Int и ничего не возвращает

callsAnother вызывает функцию, которая была передана в аргументе

```
callsAnother({ n: Int -> println("Получили $n") })
```

Можно вызвать callsAnother, передав ей лямбда-функцию с подходящим типом

https://t.me/javalib

Тип функции состоит из двух частей, разделенных символами ->. Первая часть — список аргументов в круглых скобках, а вторая — возвращаемый тип. Список типов аргументов может быть пустым, но возвращаемый тип обязателен. Если функция, которую вы передаете, ничего не возвращает, ее следует объявить с типом `Unit`.

> **ПРИМЕЧАНИЕ** Если лямбда-выражение получает один аргумент, тип которого удается вывести автоматически, можно использовать идентификатор it. Тогда не требуется указывать ни конкретное имя, ни -> в начале лямбда-выражения.

В аргументах функции должны быть указаны типы, которые они принимают от вызывающих сторон, но Kotlin позволяет сократить код вызова, применяя выведение типов к лямбда-выражениям. Ниже перечислены все способы, которыми можно вызывать ранее упомянутую функцию `callsAnother`, в порядке убывания объема кода:

```
callsAnother({ n: Int -> println("Получили $n") })
callsAnother({ n -> println("Получили $n") })
callsAnother({ println("Получили $it") })
```

Исходный вызов callsAnother с полностью указанными типами лямбда-выражения

Kotlin может вывести тип: n должна относиться к типу Int, потому что этого требует callsAnother

Лямбда-функции, которые принимают один параметр, встречаются так часто, что специально для таких случаев Kotlin поддерживает неявный параметр it

В Kotlin используется еще одна хитрость при передаче лямбда-функций в аргументах. Если последним аргументом вызова функции идет лямбда-функция, она может следовать после круглых скобок. А если лямбда-функция является единственным аргументом функции, можно вообще не использовать круглые скобки! Следующие три вызова идентичны:

```
callsAnother({ println("Получили $it") })
callsAnother() { println("Получили $it") }
callsAnother { println("Получили $it") }
```

Мы подробнее поговорим о функциональном программировании на Kotlin в главе 15, а сейчас посмотрим, как Kotlin применяет все эти функциональные средства, чтобы улучшить коллекции, которые вызывают недовольство у многих программистов на Java.

9.2.5. Коллекции

Коллекции принадлежат к числу структур данных, которые особенно часто используются в программах. Стандартная библиотека Java `Collections`, начиная с ранних версий, обладала значительной мощью и гибкостью. Однако ограничения языка и обратной совместимости часто приводят к тому, что код получается чересчур пространным и формализованным, особенно по сравнению с языками сценариев (такими, как Python) или функциональными языками

https://t.me/javalib

(например, Haskell). В последних выпусках Java ситуация радикально улучшилась (см. обзор фабрик коллекций в главе 1 и материал о потоках данных в приложении Б), но изначальные недочеты проектирования коллекций Java преследуют нас до сих пор.

Естественно, Kotlin учился на этих ошибках и с самого начала предлагал бесшовную интеграцию коллекций. Стандартные функции для создания самых популярных типов коллекций в Java появились только в версии 9, а в Kotlin они были всегда:

```
val readOnlyList = listOf("a", "b", "c")
val mutableList = mutableListOf("a", "b", "c")

val readOnlyMap = mapOf("a" to 1, "b" to 2, "c" to 3)
val mutMap = mutableMapOf("a" to 1, "b" to 2, "c" to 3)

val readOnlySet = setOf(0, 1, 2)
val mutableMap = mutableSetOf(1, 2, 3)
```

Создает списки с выведенными типами kotlin.collections.List<String> и kotlin.collections.MutableList<String>

Создает отображения с выведенными типами kotlin.collections.Map<String, Int> и kotlin.collections.MutableMap<String, Int>. Обратите внимание на встроенный синтаксис определения отображений с ключевым словом to

Создает множества с выведенными типами kotlin.collections.Set<String> и kotlin.collections.MutableSet<String>

Стандартные функции возвращают копии своих коллекций, доступные только для чтения, — отличное решение с точки зрения производительности и целостности. Чтобы получить коллекцию с интерфейсами для изменения, необходимо явно запросить `mutable`-версию. Kotlin снова стремится защитить вас от целых классов ошибок, настраивая варианты по умолчанию так, чтобы вам было удобнее и быстрее оперировать неизменяемыми коллекциями.

Вероятно, вы заметили, что выведенные типы этих коллекций отличаются от своих аналогов в Java: несмотря на похожие имена, они находятся в пакете `kotlin.collections`. Kotlin определяет собственную иерархию интерфейсов коллекций (рис. 9.1), хотя на внутреннем уровне использует реализации из JDK. Это позволяет сформировать более чистые API для интерфейсов `kotlin.collections` и при этом сохранить возможность передавать коллекции коду на Java, потому что эти реализации также поддерживают интерфейсы коллекций `java.util`.

Эти коллекции можно использовать во всех стандартных интерфейсах и паттернах Java. Например, их элементы можно перебирать конструкцией `for … in`:

```
val l = listOf("a", "b", "c")

for (s in l) {
  println(s)
}
```

Тем не менее цикл `for` для перебора коллекции — единственная операция, которая встроена непосредственно в язык. Чтобы было легче писать код для других

https://t.me/javalib

востребованных операций, коллекции Kotlin реализуют богатую функциональность с помощью функций первого класса, о которых рассказывалось в предыдущем разделе. Эти инструменты почти всегда возвращают новую коллекцию, а не изменяют ту, для которой они вызывались. Возможно, вы сталкивались с таким стилем кода для коллекций в Java после того, как появились лямбда-выражения и потоки данных.

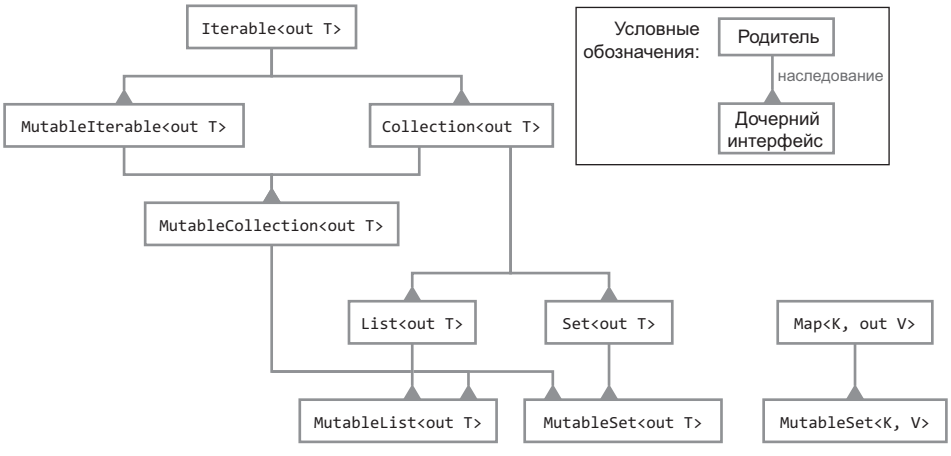


Рис. 9.1. Иерархия коллекций Kotlin

Часто мы берем значение из одной коллекции и преобразуем каждый элемент в другое значение по заданному правилу. Метод `map` делает именно это с помощью передаваемой функции:

```
val l = listOf("a", "b", "c")
val result = l.map { it.toUpperCase() } ◄—— result содержит список с элементами «A», «B», «C»
```

Другая типичная операция заключается в том, чтобы удалить некоторые значения из коллекции перед тем, как продолжать обработку. Метод `filter` принимает лямбда-выражение, которое возвращает `Boolean`. Такие лямбда-выражения называются *предикатами*, а `filter` многократно вызывает предикат, чтобы определить, какие элементы следует возвращать в новой коллекции:

```
val l = listOf("a", "b", "c")
val result = l.filter { it != "b" } ◄—— result содержит список с элементами «a», «c»
```

Если вам требуется только узнать, удовлетворяет ли коллекция некоторым условиям, но ее отдельные элементы вас не интересуют, то функции `all`, `any` и `none` — именно то, что вам нужно. Они избегают копирования данных и возвращают управление как можно раньше (например, для `all()` — после первого `false`):

```
val l = listOf("a", "b", "c")
val all = l.all { it.length == 1 }  ◄——  all == true
```

https://t.me/javalib

```
val any = l.any { it.length == 2 }  ◄—— any == false
val none = l.none { it == "a" }  ◄—— none == false
```

Чтобы построить отображение на основе списка, можно использовать функции `associateWith` и `associateBy`. `associateWith` делает элементы коллекции ключами в итоговом отображении, а `associateBy` — значениями. Если в процессе работы любой из этих функций будут обнаружены дубликаты, используется последнее вычисленное значение, как показывает следующий фрагмент:

```
val l = listOf("!", "-", "--", "---")

val resultWith = l.associateWith { it.length }
val resultBy = l.associateBy { it.length }
```

resultWith содержит mapOf("!" to 1, "-" to 1, "--" to 2, "---" to 3)

resultBy содержит mapOf(1 to "-", 2 to "--", 3 to "---").

В этом разделе мы дали только общее представление о богатом наборе функций Kotlin для работы с интерфейсом Collections. Эти функции можно объединять в цепочки, чтобы создавать выразительные и компактные описания действий с коллекциями. В документации вы найдете превосходные примеры, которые демонстрируют группировку, сортировку, агрегирование, копирование и другие операции. За дополнительной информацией обращайтесь по адресу https://kotlinlang.org/docs/collections-overview.html.

Стремление Kotlin поддерживать тесную связь между фрагментами кода распространяется и на другую фундаментальную функциональность. Предпочтительное использование выражений вместо инструкций — еще один способ сгладить шероховатости в вашем коде.

9.2.6. Выражения

Начиная изучать новый язык программирования, среди первых программных структур вы встречаете `if`. В Java это инструкция, которая управляет последовательностью выполнения программы. В Kotlin `if` служит для той же цели, но это не *инструкция*, которая просто выполняется, а *выражение*, которое возвращает значение:

```
val iffy = if (checkCondition()) {
  "так точно"
} else {
  "никак нет"
}
```

Переменной iffy присваивается значение, которое зависит от выбранной ветви

Если checkCondition() вернет true, то iffy присваивается значение "так точно"

Если checkCondition() вернет false, то iffy присваивается значение "никак нет"

Как и при любом другом присваивании, Kotlin может вывести тип автоматически. В данном случае тип определяется последней строкой каждой ветви.

Выражения `if` настолько мощны, что из Kotlin была исключена конструкция, которую Java унаследовал от C, — тернарный оператор *условие* `? "так точно" :`

https://t.me/javalib

"никак нет". Тернарный оператор уменьшает объем кода, но многие считают, что ради этой компактности приходится жертвовать удобочитаемостью. Хотя версия Kotlin немного длиннее, во многих случаях она лучше читается, а если понадобится добавить новую логику, код можно органически преобразовать в многострочный if:

```
val myTurn = if (condition) "так точно" else "никак нет"
```

В главе 1 упоминалось о том, что в Java появились выражения switch. Это еще одна ситуация, в которой возможности Kotlin предшествовали аналогичным улучшениям в Java. Kotlin не использует традиционный синтаксис switch в стиле C, зато предоставляет мощную альтернативу с ключевым словом when:

```
val w = when (x) {
  1 -> "один"   <-- Если значение x равно 1, то w присваивается «один»
  2 -> "два"    <-- Если значение x равно 2, то w присваивается «два»
  else -> "много"   <-- Если x содержит любое другое значение, то w присваивается «много»
}
```

when поддерживает множество других полезных форм. С ключевым словом in можно проверять, входит ли элемент в коллекцию:

```
val valid = listOf(1, 2, 3)
val invalid = listOf(4, 5, 6)

val w = when (x) {
  in valid -> "valid"
  in invalid -> "invalid"
  else -> "неизвестно"
}
```

Проверяет, входит ли x в каждую коллекцию, — эквивалент вызова valid.contains(x) и invalid.contains(x)

Kotlin также на уровне языка поддерживает числовые интервалы, которые тоже хорошо сочетаются с when и in:

```
val w = when (x) {
  in 1..3 -> "valid"
  in 4..6 -> "invalid"
  else -> "неизвестно"
}
```

Синтаксис .. определяет интервал (включая границы), так что этот код эквивалентен предыдущему примеру со списком

Следует заметить, что условие в левой части when может быть любым допустимым выражением, если только тип соответствует требуемому. Например, можно использовать вызовы функций — это удобный прием, чтобы прояснять сложные условия:

```
fun theBest() = 1
fun okValues() = listOf(2, 3)

val message = when (incoming) {
  theBest() -> "отлично!"
  in okValues() -> "годится!"
  else -> "не годится"
}
```

Значение, которое возвращает theBest, используется напрямую, поэтому функция должна возвращать Int, который можно будет сравнить с Incoming

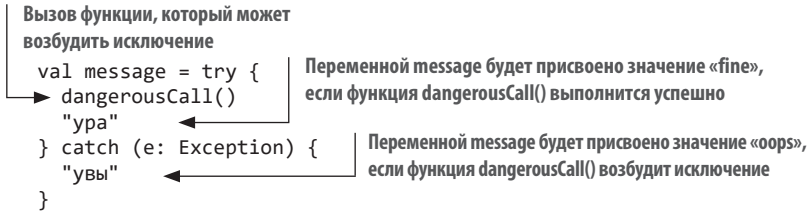
Значение, которое возвращает okValues, используется с in, поэтому оно должно быть коллекцией

https://t.me/javalib

А если вы до сих пор не убедились в сверхъестественной мощи `when`, у него есть еще одно свойство — безопасность. Во всех наших примерах есть секция `else`. Если ее удалить, это приведет к ошибке: компилятор пожалуется, что мы обрабатываем не все случаи:

```
error: 'when' expression must be exhaustive, add necessary 'else' branch[1]
```

Еще одна хитрость, с помощью которой Kotlin заменяет инструкции Java выражениями, — обработка ошибок в `try/catch`:



Так вы избавляетесь от неуклюжей конструкции, когда снаружи от `try/catch` объявляется переменная, которой должны присваиваться правильные значения на всех внутренних путях. Запись на Kotlin не только короче, но и безопаснее, потому что компилятор может гарантировать, что присваивание выполняется корректно.

Хотя и Kotlin, и Java позаимствовали некоторые аспекты функционального программирования, они, по сути, остаются объектно-ориентированными языками. Следующий раздел посвящен определению классов и объектов в Kotlin.

9.3. НОВЫЙ ВЗГЛЯД НА КЛАССЫ И ОБЪЕКТЫ

Классы Kotlin предоставляют функциональность, очень похожую на функциональность Java, и тоже объявляются с ключевым словом `class`. Но как мы уже неоднократно убедились, код на Kotlin выглядит иначе, отдавая приоритет компактности и удобству.

Во-первых, Kotlin не использует ключевое слово `new` при создании экземпляров класса. Вместо этого код больше похож на вызов функции, одноименной с классом:

```
val person = Person()
```

Во-вторых, в Kotlin нет полей в том смысле, в каком они существуют в Java. Вместо них фигурируют *свойства* класса, которые объявляются с помощью уже знакомых нам ключевых слов `val` и `var`. Их можно инициализировать на месте по аналогии с полями Java:

[1] Ошибка: выражение ‘when’ должно быть исчерпывающим, добавьте обязательную ветвь ‘else’. — *Примеч. пер.*

https://t.me/javalib

```
import java.time.LocalDate

class Person {
  val birthdate = LocalDate.of(1996, 1, 23)
  var name = "Ваше имя"
}
```

Свойство birthdate, доступное только для чтения

Изменяемое свойство name

ПРИМЕЧАНИЕ Как было показано в главе 4, на уровне JVM фигурируют поля, так что свойства Kotlin на самом деле преобразуются в обращения к полям в байт-коде. Тем не менее на уровне языка лучше мыслить категориями свойств.

Основной источник шаблонного кода в классах Java — методы чтения и записи полей («геттеры» и «сеттеры»). Чтобы избавиться от этих формальностей, Kotlin по умолчанию автоматически добавляет методы доступа к свойствам. Более того, Kotlin позволяет использовать эти методы так, как если бы вы обращались к полям в Java:

```
  println("Привет, ${person.name}. " +
      "Вы родились ${person.birthdate}")
person.name = "Кто-то еще"
  // person.birthdate = LocalDate.of(2000, 1, 1)
```

Выводит «Привет, Ваше имя. Вы родились 1996-01-23»

Значения val-свойств нельзя менять

У var-свойств есть методы доступа для записи, которые можно использовать с оператором =

Проектируя класс, необходимо серьезно продумать видимость данных, которые представляют его состояние, особенно в контексте инкапсуляции. Создатели Kotlin приняли несколько неоднозначное решение: по умолчанию используется открытый уровень видимости, тогда как в Java по умолчанию принята пакетно-доступная видимость. Свободный доступ ко всем свойствам обычно считается плохим тоном, но создатели Kotlin обнаружили, что в реальном коде уровень `public` назначают намного чаще других, так что если установить его по умолчанию, это поможет писать более компактный код.

Kotlin поддерживает четыре уровня видимости, которые в основном соответствуют своим аналогам в Java:

- `private` (частный) — видимость только в текущем классе или файле (для функций верхнего уровня).
- `protected` (защищенный) — видимость в классе и дочерних классах.
- `internal` (внутренний) — видимость в пределах кода, который компилируется совместно.
- `public` (открытый) — видимость для всех.

Например, если вы хотите сделать поле `birthdate` частным, это можно закодировать так:

https://t.me/javalib

```
class Person {
  private val birthdate = LocalDate.of(1996, 1, 23)
  var name = "Ваше имя"
}
```

Поскольку Kotlin предоставляет в распоряжение программиста только свойства, а не поля, открывается возможность для *делегированных свойств*. Если за свойством следует ключевое слово by, свойство может представить специализированную реализацию своих методов доступа. Эта возможность будет использоваться в некоторых продвинутых методах, которые встретятся в последующих главах.

Вместе со стандартной библиотекой поставляется ряд полезных делегатов. Например, в процессе отладки нередко хочется узнавать, когда изменилось то или иное значение. `Delegates.observable` предоставляет такую возможность:

```
import kotlin.properties.Delegates

class Person {
  var name: String by Delegates.observable("Ваше имя") {
    prop, old, new -> println("Имя изменилось с $old на $new")
  }
}
```

Значение, которое передается при вызове `Delegates.observable`, интерпретируется как исходное значение свойства. Лямбда-выражение, переданное `Delegates.observable`, активизируется после того, как значение свойства меняется. Лямбда-выражение принимает дескриптор самого свойства, а также старое и новое значение. В этом примере мы просто выводим информацию о том, что изменилось.

Как и в Java, в Kotlin можно создавать экземпляры классов с помощью конструкторов, причем существует несколько разных форм конструирования. Первая форма объявляет *первичный конструктор* с именем класса, как показано ниже. Эта форма предоставляет альтернативную точку, в которой можно задавать свойства:

```
class Person(
val birthdate: LocalDate,
var name: String) {
}

val person = Person(LocalDate.of(1996, 1, 23),
"Кто-то еще")
```

val и var в первичном конструкторе создают свойства, так что их не потребуется объявлять позднее

Поскольку значения по умолчанию не заданы, конструктор должен принимать параметры

Если вы хотите снабдить конструктор модификаторами видимости или аннотациями, можно воспользоваться более длинным синтаксисом с ключевым словом `constructor`. Например, так можно скрыть `constructor` от окружающего мира:

```
class Person private constructor(
  val birthdate: LocalDate,
  var name: String) {
}
```

https://t.me/javalib

Чтобы реализовать дополнительную логику во время конструирования объекта, в Kotlin используется ключевое слово `init`:

```
class Person(
  val birthdate: LocalDate,
  var name: String) {

  init {
    if (birthdate.year < 2000) {
      println("Прошлый век")
    }
  }
}
```

init выполняется после того, как свойствам присвоены значения, поэтому здесь к ним можно обращаться

В классе может быть несколько блоков `init`, которые выполняются в том порядке, в каком они определены, как показано ниже. Свойства, определенные в теле класса, доступны для блоков `init` только после определения:

```
class Person(
  val birthdate: LocalDate,
  var name: String) {

  init {
    // println(nameParts)
  }

  val nameParts: List<String> = name.split(" ")

  init {
    println(nameParts)
  }
}
```

Ошибка компиляции: переменная nameParts не определена

Работает, как ожидалось, и выводит список

Если нужны дополнительные конструкторы, их можно определить в теле класса с ключевым словом `constructor`. Они называются *вторичными конструкторами*:

```
class Person(
  val birthdate: LocalDate,
  var name: String) {

  constructor(name: String)
    : this(LocalDate.of(0, 1, 1), name) {
  }
```

Если в классе есть первичный конструктор, то вторичные конструкторы должны вызывать его (напрямую или через другие вторичные конструкторы) с помощью ключевого слова this

> **ПРИМЕЧАНИЕ** Для того чтобы обеспечить значения по умолчанию, в Java часто используются несколько переопределенных конструкторов. В Kotlin для этого предусмотрены значения аргументов по умолчанию.

Хотя класс, состоящий из одних свойств, может быть полезен, чаще всего классы содержат и другую функциональность. В следующем примере, чтобы добавить метод в класс, используется знакомый синтаксис функций, который уже встречался ранее в этой главе:

https://t.me/javalib

```
class Person(
  val birthdate: LocalDate,
  var name: String) {

  fun isBirthday(): Boolean {
    val today = LocalDateTime.now().toLocalDate()
    return today == birthdate
  }
}
```

Как уже упоминалось, функции в Kotlin по умолчанию используют открытый уровень видимости. Если вы хотите скрыть функцию, поставьте перед ней нужный модификатор доступа:

```
class Person(
  val birthdate: LocalDate,
  var name: String) {

  fun isBirthday(): Boolean {
    return today() == birthdate
  }

  private fun today(): LocalDate {
    return LocalDateTime.now().toLocalDate()
  }
}
```

Метод isBirthday доступен в любой точке, где виден класс Person

Метод today доступен только в классе Person

Еще одна ключевая концепция объектно-ориентированного программирования — *наследование*. В Kotlin оно выражается не с помощью ключевого слова `extends`, а через знакомый синтаксис `:`, который встречался ранее в объявлениях типов:

```
class Child(birthdate: LocalDate, name: String)
  : Person(birthdate, name) {
}
```

Параметры конструктора Child. Обратите внимание, что они не помечены ключевыми словами val или var и поэтому не конфликтуют с родительскими свойствами; они доступны в виде локальных переменных, которые можно передать конструктору надкласса

Вызов конструктора надкласса

Для наследования нужно еще одно изменение в классе `Parent`. Чтобы подклассы создавались только тогда, когда вы сознательно этого хотите, по умолчанию классы Kotlin являются *закрытыми*. Чтобы класс мог наследоваться, нужно добавить ключевое слово `open`, как в следующем примере. Этим Kotlin диаметрально отличается от Java, где классы открыты по умолчанию, а с помощью ключевого слова `final` можно указать, что они *не могут* наследоваться:

```
open class Person(
  val birthdate: LocalDate,
  var name: String) {
  //...
}
```

Модификатор видимости open ставится перед ключевым словом class

https://t.me/javalib

Методы в Kotlin также являются закрытыми по умолчанию. Родительский класс должен объявить переопределяемый метод открытым, а в дочернем классе переопределения должны помечаться модификатором `override`:

```
open class Person(
  val birthdate: LocalDate,
  var name: String) {

  open fun isBirthday(): Boolean {
    return today() == birthdate
  }

  private fun today() : LocalDate {
    return LocalDateTime.now().toLocalDate()
  }
}

class Child(birthdate: LocalDate, name: String)
  : Person(birthdate, name) {

  override fun isBirthday(): Boolean {
    val itsToday = super.isBirthday()
    if (itsToday) {
      println("Пусть бегут неуклюже! ")
    }
    return itsToday
  }
}
```

Класс Person объявляет, что функция isBirthday может переопределяться в подклассах

Класс Child должен явно пометить свой метод как переопределяемый

Child может вызвать родительскую реализацию isBirthday при помощи super

Как и Java, Kotlin допускает только один базовый класс, но классы могут наследовать нескольким интерфейсам, как показано в следующем примере. Интерфейсы Kotlin могут содержать реализации функций по умолчанию — как и в Java, начиная с версии 8:

```
interface Greetable {
  fun greet(): String
}

open class Person constructor(
  val birthdate: LocalDate,
  var name: String): Greetable {

  override fun greet(): String {
    return "Привет!"
  }
}
```

Определяет интерфейс с функцией, которая возвращает приветствие

Класс Person объявляет, что он реализует Greetable

Функции интерфейса являются открытыми, поэтому в их реализации нужно указывать override

В типичном стиле Kotlin интерфейсы реализуются в компактной форме, похожей на ту, с помощью которой мы уже расширяли базовые классы: в отличие от Java, вам не придется вспоминать, когда использовать `extends`, а когда `implements`.

https://t.me/javalib

9.3.1. Классы данных

Хотя базовые конструкции Kotlin позволяют строить богатые объектные модели, иногда объекты нужны просто как контейнеры для передачи данных. Для таких случаев Kotlin предоставляет *классы данных*.

> **ПРИМЕЧАНИЕ** В главе 3 уже упоминалась новая функциональность записей в языке Java. Классы данных Kotlin в некоторых отношениях очень похожи на записи Java.

Kotlin уже обеспечивает безупречную интеграцию механизма свойств, но при работе со стандартными классами остаются проблемы равенства: реализации `equals` и `hashCode` по умолчанию используют ссылки на объекты, а не значения свойств.

Но если объявить тип как `data class`, Kotlin создает именно такие функции проверки равенства, какие нужны (если только вы явно не предоставите собственную реализацию):

```
class PlainPoint(val x: Int, val y: Int)

val pl1 = PlainPoint(1, 1)
val pl2 = PlainPoint(1, 1)

println(pl1 == pl2)

data class DataPoint(val x: Int, val y: Int)
val pd1 = DataPoint(1, 1)
val pd2 = DataPoint(1, 1)

println(pd1 == pd2)
```

Стандартная реализация проверки равенства сравнивает ссылки, поэтому здесь выводится false

С реализацией классов данных Kotlin эта строка выводит true

У класса данных должен быть первичный конструктор хотя бы с одним параметром `val` или `var`. Классы данных не могут быть открытыми: во время компиляции Kotlin не сможет правильно сгенерировать функции проверки равенства, если у типа могут быть производные классы. Классы данных также не могут быть внутренними классами. Однако помимо этих (и нескольких еще более экзотических) ограничений, это вполне обычные классы, в которых можно реализовать функции или интерфейсы как вам заблагорассудится.

Наконец, разработчикам с опытом Java может быть интересно, как объявить в классе функцию, которая принадлежит не экземплярам, а классу в целом. Однако создатели Kotlin решили не поддерживать статические функции: функции либо существуют сами по себе, либо являются членами типа.

Тем не менее возможность связать функцию с классом бесспорно удобна, и Kotlin предоставляет похожую функциональность в форме объектов-компаньонов. Их синтаксис объявляет одиночный (singleton) объект, который существует внутри класса. Объявления `object` в Kotlin создают полноценные объекты с типичными свойствами и функциями. Тем самым предотвращаются проблемы (и соответственно, трудности с тестированием) в граничных случаях, от которых

https://t.me/javalib

страдают статические методы в Java, но при этом все равно можно удобно хранить функциональность, связанную с классом.

Типичное применение таких функций — фабричные методы, когда вы хотите объявить конструкторы объектов частными, но при этом разрешить управляемое создание экземпляров при помощи методов с более конкретными именами:

```
class ShyObject private constructor(val name: String) {

  companion object {
    fun create(name: String): ShyObject {
      return ShyObject(name)
    }
  }
}

ShyObject.create("Предмет")
```

ShyObject объявляет свой конструктор частным, чтобы никто за пределами класса не мог им воспользоваться

Фабричный метод внутри объекта-компаньона является частью класса ShyObject, поэтому он может обратиться к частному конструктору

Функции компаньона, объявленного в ShyObject, за пределами класса доступны напрямую по имени класса

Kotlin — это прагматичная альтернатива Java: писать код на нем удобнее, а шаблонного кода становится значительно меньше. Но и это еще не все, как вы увидите в следующем разделе.

9.4. БЕЗОПАСНОСТЬ

Kotlin работает на базе JVM, поэтому ему неизбежно приходится мириться с определенными ограничениями, которые обусловлены архитектурой виртуальной машины. Например, спецификация JVM определяет `null` как значение, которое можно присвоить любой переменной ссылочного типа.

Несмотря на эти проблемы, язык Kotlin старается решить некоторые распространенные проблемы с безопасностью кода, чтобы по возможности свести к минимуму унаследованные хлопоты и неприятности. Ради этого некоторые паттерны кода Java повышаются до языковых средств, отчего код по умолчанию становится безопаснее.

9.4.1. Null-безопасность

Одно из самых распространенных исключений Java — `NullPointerException` — возбуждается, если пытаться обратиться к переменной или полю, где должен храниться объект, но хранится `null`. Тони Хоар, создатель алгоритма быстрой сортировки, назвал `null` своей «ошибкой на миллиард долларов», подразумевая свою причастность к тому, что `null`-ссылки были включены в ALGOL (см. https://qconlondon.com/london-2009/qconlondon.com/london-2009/speaker/Tony+Hoare.html).

За все время, что существует язык Java, было разработано несколько подходов к защите от `null`. Тип `Optional` позволяет всегда иметь конкретный объект и при

https://t.me/javalib

этом предусматривает «отсутствующее» значение без использования `null`. Во многих фреймворках проверки и сериализации поддерживаются аннотации `@NotNull` и `@Nullable`, которые обеспечивают, чтобы значения внезапно не оказались равными `null` в ключевых точках приложения.

Как вы наверняка догадались, Kotlin берет эти стандартные паттерны и встраивает их прямо в язык. Вернемся к одному из предыдущих примеров, где переменным присваиваются значения. Как они поведут себя в сочетании с `null`?

```
val i: Int = null
val s: String = null
```

Если пытаться присвоить null этим типам, код не будет компилироваться

Обе операции присваивания приводят к ошибке компиляции: «`null` не может быть значением необнуляемого типа `Int`». Хотя объявления типов `Int` и `String` похожи на аналогичные типы Java, на самом деле они запрещают значения `null`.

> **ПРИМЕЧАНИЕ** В Kotlin «обнуляемость» стала частью системы типов. Тип String в Kotlin на самом деле не эквивалентен типу `String` в Java, который допускает `null`.

Чтобы переменная Kotlin допускала значение `null`, это нужно явно указать, добавив к типу суффикс `?`:

```
val i: Int? = null
val s: String? = null
```

Если заменить типы на Int? и String?, то Kotlin позволит использовать в них null

> **ПРИМЕЧАНИЕ** По возможности объявляйте свои переменные и аргументы с «необнуляемыми» типами. Будьте уверены, что Kotlin защитит вас от проблем с `NullPointerException`.

Впрочем, не всегда можно обойтись без `null`: например, если вы взаимодействуете с кодом на Java или если ваши классы не были спроектированы с учетом `null`-безопасности. Даже если вам приходится иметь дело с `null`, Kotlin всеми силами постарается предупредить вас о соответствующих рисках:

```
val s: String? = null
println(s.length)
```

Создает переменную с допустимым значением null

Пытается обратиться к свойству этой переменной

Kotlin распознает, что разыменование ссылки в вызове `s.length` потенциально небезопасно, и отказывается компилировать программу с ошибкой: «для `null`-совместимого получателя типа `String?` разрешены только безопасные (`?.`) или проверенные на `null` (`!!.`) вызовы».

Первое, что предлагает Kotlin, — исправить вызов с использованием безопасного оператора `?.`: он проверяет объект, к которому применяется. Если объект содержит `null`, то вместо дальнейшего вызова функции возвращается `null`:

```
val s: String? = null
println(s?.length)
```

?. приводит к тому, что выводится значение null

https://t.me/javalib

Безопасный оператор использует ускоренный возврат управления, поэтому он хорошо работает даже с вложенной цепочкой вызовов. В следующем примере любое звено цепочки может безопасно вернуть `null`, и тогда все выражение просто примет значение `null`:

```
data class Node(val parent: Node?, val value: String)
```

Класс данных, который может содержать необязательный родительский узел

```
val node = getNode()
node.parent?.parent?.parent
```

Получает откуда-либо экземпляр класса Node

Проверяет, есть ли у узла «прадедушка»

Такое решение удобно, но `?.` может маскировать проблемы с данными. Если вы получили `null` при проверке «прадедушки» из предыдущего примера, то без дополнительного анализа невозможно определить, с какого уровня иерархии пришло это значение.

Второе возможное решение для упомянутой ошибки компиляции — использовать оператор `!!` с переменной. Он заставляет Kotlin проверить объект, и если он равен `null`, возбуждает уже знакомое исключение `NullPointerException`:

```
val s: String? = null
println(s!!.length)
```

Возбуждает исключение NullPointerException

Наконец, можно просто проверить переменную на равенство `null`, хотя подобные проверки понадобятся не так часто. Собственно, Kotlin обычно замечает такие проверки и позволяет избежать лишних `?.` или `!!`:

```
val s: String? = null

if (s != null) {
  println(s.length)
}
```

Всегда проверяет на null

Так как мы знаем, что s — не null, эту переменную можно безопасно разыменовать

На самом деле здесь используется более фундаментальная возможность Kotlin, которая называется *умным приведением типа* (smart casting) и заслуживает того, чтобы рассмотреть ее внимательнее.

9.4.2. Умное приведение типа

Хотя хороший стиль объектно-ориентированного проектирования не одобряет прямую проверку типа объектов, иногда без нее не обойтись. Форматы данных на границах системы могут свободнее относиться к типам (например, JSON) и часто вам неподконтрольны. Другой пример — системы плагинов, которые должны динамически проверять функциональность объекта.

Kotlin признает эту необходимость, и его компилятор поддерживает соответствующие стандартные паттерны. Для начала проверять типы объектов можно оператором `if`:

https://t.me/javalib

```
val s: Any = "Привет"
if (s is String) {
  println(s.toUpperCase())
}
```

Any — базовый тип всех объектов, аналог типа Object в Java

Проверяет, содержит ли s экземпляр String

Использует переменную s как String. Если бы компилятор в этой ветви все еще рассматривал ее как тип Any, то нельзя было бы вызвать toUpperCase

Если вам знакома конструкция Java `instanceof`, может показаться, что в этом коде пропущен критический этап: мы проверяем, относится ли `s` к типу `String`, но не выполняем приведение типа перед тем, как работать с переменной как со `String`. К счастью, Kotlin делает это за нас. Внутри блока `if`, где компилятор убедился, что мы имеем дело со `String`, можно использовать `s` как `String` без явного приведения типа. Этот механизм называется *умным приведением типа*.

> **ПРИМЕЧАНИЕ** В Java есть новая функциональность, которая называется *поиском по шаблону* (pattern matching) и постепенно развивается в составе проекта Amber. Часть этой функциональности применяется к `instanceof` и предоставляет некоторые возможности, аналогичные умному преобразованию типа. Поиск по шаблону подробнее рассматривается в главе 18.

Умное преобразование Kotlin также можно использовать в условной конструкции `if`:

```
val s: Any = "Привет"
if (s is String && s.toUpperCase() == "ПРИВЕТ") {
  println("Так и есть")
}
```

Kotlin узнает тип из проверки в левой части &&, поэтому может преобразовывать регистр без приведения типа

На умное приведение типа накладываются определенные ограничения. В частности, оно не работает со свойствами `var` в классе. Это предотвращает ситуацию, когда свойство конкурентно преобразуется к другому совместимому подтипу после того, как выполнена проверка умного приведения типа, но до выполнения следующего блока.

Даже если Kotlin не может выполнить умное приведение типа, его можно осуществить вручную — просто это чуть менее удобно:

```
class Example(var s: Any) {
  fun checkIt() {
    if (s is String) {
      val cast = s as String
      println(cast.toUpperCase())
    }
  }
}
```

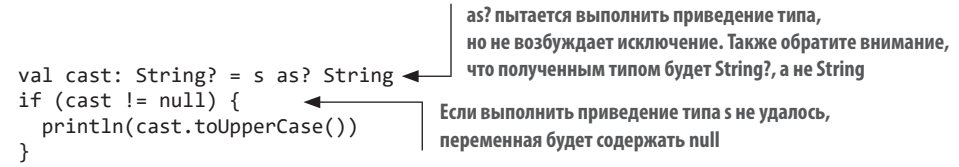
Предполагается, что s определяется так, что для нее недоступно умное приведение типа

as приводит к нужному типу

Когда мы используем `as`, то возвращаемся к ситуации, которая знакома нам по Java. Если типы несовместимы, возбуждается исключение `ClassCastException`.

https://t.me/javalib

Если вы предпочитаете разрешить null-совместимость вместо того, чтобы ловить исключения, Kotlin предлагает альтернативный вариант:




```
val cast: String? = s as? String
if (cast != null) {
  println(cast.toUpperCase())
}
```

Значительная часть мощи Kotlin обусловлена свежим взглядом на повседневную практику программирования, которым Java-разработчики занимались годами. Впрочем, в области конкурентного выполнения язык не просто улучшает удобство и безопасность. В Kotlin есть так называемые *сопрограммы*, или *корутины* (coroutines), которые можно рассматривать как альтернативу классическим многопоточным решениям, широко применяемым в Java.

9.5. КОНКУРЕНТНОСТЬ

Как упоминалось в главе 5, еще самые первые версии JVM поддерживали класс Thread как модель потоков, находящихся под управлением операционной системы. Эта модель интуитивно понятна, но с ней связано множество проблем.

> **ПРИМЕЧАНИЕ** Хотя потоки встроены в язык и экосистему Java так глубоко, что исключить их практически невозможно, переход на новый язык, отличный от Java, позволяет переосмыслить примитивы конкурентности, которые можно в нем использовать.

Хотя Kotlin как язык JVM позволяет работать с потоками, в нем также появилась другая конструкция — так называемые *корутины*. На простейшем уровне корутину можно считать облегченным потоком. Корутины реализуются и планируются на уровне среды выполнения (а не операционной системы), в результате чего они потребляют значительно меньше ресурсов. Если запустить тысячи корутин, это не вызовет никаких проблем, тогда как аналогичное количество потоков парализует систему.

> **ПРИМЕЧАНИЕ** Подход к корутинам в Java будет рассмотрен в главе 18, когда речь пойдет о проекте Loom.

Поддержка корутин в Kotlin частично встроена в язык (возможность приостанавливать функции), но чтобы применять корутины на практике, нужна дополнительная библиотека kotlin-coroutine-core. В главе 11 мы гораздо подробнее поговорим о зависимостях такого рода, а пока ограничимся тем, что в Maven зависимость будет выглядеть так:

```
<dependency>
    <groupId>org.jetbrains.kotlinx</groupId>
    <artifactId>kotlinx-coroutines-core</artifactId>
    <version>1.6.0</version>
</dependency>
```

https://t.me/javalib

А вот эквивалентное определение для Gradle:

```
dependencies {
  implementation("org.jetbrains.kotlinx:kotlinx-coroutines-core:1.6.0")
}
```

Чтобы в Java запустить поток, ему нужно передать объект, который реализует интерфейс `Runnable`. Корутинам в Kotlin тоже нужно как-то получить выполняемый код, но они используют для этого синтаксис лямбда-выражений языка.

Корутина всегда запускается в *области видимости*, которая управляет тем, как корутина будет планироваться и выполняться. Начнем с простейшего варианта — это область видимости `GlobalScope`, которая существует все время, пока ваше приложение выполняется. У `GlobalScope` есть функция `launch`, которой следует передать лямбда-выражение, чтобы запустить корутину:

```
import kotlinx.coroutines.GlobalScope
import kotlinx.coroutines.launch

fun main() {
    GlobalScope.launch {
      println("Внутри!")
    }
    println("Снаружи")
}
```

Импортируем функцию корутины и объект, который будем использовать

Запускаем новую корутину в области видимости GlobalScope, которая существует все время, пока выполняется программа

Выводим сообщение за пределами корутины, чтобы убедиться, что main все еще выполняется

Запуская этот пример, чаще всего мы будем видеть такой вывод:

```
Снаружи
```

Почему корутина не работает? Мы надеялись, что в какой-то момент должно выводиться сообщение `Внутри!`. Однако если присмотреться внимательнее, можно заметить проблему; для этого нужно проследить последовательность событий. Сначала `main` запускает программу. Потом запускается корутина, которая выполняется асинхронно. После этого выводится сообщение `Снаружи`, и программа завершается. Когда `main` заканчивает работу, происходит выход из программы: при этом не учитывается, что какие-то корутины могут ожидать выполнения.

Чтобы получить желаемый результат, нужно добавить паузу перед завершением программы. Для этого можно вставить цикл или запросить у пользователя ввод с консоли. Мы добавим вызов `Thread.sleep(1000)`, чтобы дать корутине время выполнить свою работу:

```
import kotlinx.coroutines.GlobalScope
import kotlinx.coroutines.launch

fun main() {
  GlobalScope.launch {
    println("Внутри!")
  }
  println("Снаружи")
  Thread.sleep(1000)
}
```

Снова запускает корутину

Дает корутине время для выполнения

https://t.me/javalib

Теперь выводятся оба сообщения, хотя их порядок не детерминирован: он зависит от того, насколько быстро запускается корутина и что делает главный поток.

На высоком уровне это мало чем отличается от потоков, которые используются для конкурентного выполнения кода. Однако внутренняя реализация расходует меньше ресурсов операционной системы (корутине не обязательно иметь собственный стек и локальное хранилище), и при этом обеспечивается безопасность таких операций, как отмена корутины.

Следующий пример показывает, как это делается. Мы сохраняем в переменной дескриптор корутины, который возвращается при вызове `launch`. Объект корутины предоставляет функцию `cancel`, которую при желании можно вызвать немедленно:

```
import kotlinx.coroutines.GlobalScope
import kotlinx.coroutines.delay
import kotlinx.coroutines.launch

fun main() {
  val co = GlobalScope.launch {
      delay(1000)
      println("Внутри!")
  }
  co.cancel()
  println("Снаружи")
  Thread.sleep(2000)
}
```

Сохраняет объект корутины, который возвращается из launch

Внутри корутины можно вызвать delay, чтобы приостановить выполнение на заданный промежуток времени

Немедленно отменяет корутину

Здесь можно ждать сколько угодно — вы не увидите вывод корутины

Этот код безопасно останавливает корутину, и выводится только сообщение `Снаружи`. Ситуация заметно контрастирует с методом `stop()` класса `java.lang.Thread`: как упоминалось в главе 5, этот метод давно был объявлен устаревшим из-за своей катастрофической небезопасности.

Почему корутины могут безопасно делать то, что не удается потокам? Все дело в функции `delay`, объявление которой помечено специальным модификатором `suspend` («приостанавливаемая»). Приостанавливаемые функции можно использовать как «островки безопасности», когда при выполнении корутины, например, нужно переключиться на другую задачу или проверить, какие корутины отменены. Это называется *кооперативной многозадачностью*, и выполнение корутины можно отменить именно потому, что ее код «кооперируется» в своих вызовах `suspend`-функций.

Преимущества кооперации не ограничиваются безопасной отменой. Например, Kotlin понимает, когда одна корутина (родительская) запускает другую (дочернюю). Если отменить родительскую корутину, то дочерние автоматически отменяются без дополнительного управления с нашей стороны:

```
import kotlinx.coroutines.GlobalScope
import kotlinx.coroutines.coroutineScope
import kotlinx.coroutines.delay
import kotlinx.coroutines.launch
```

https://t.me/javalib

```
fun main() {
    val co = GlobalScope.launch {
       coroutineScope {
          delay(1000)
          println("Первое")
       }
       coroutineScope {
          delay(1000)
          println("Второе")
       }
    }
    co.cancel()
    Thread.sleep(2000)
}
```

Запускает родительскую корутину, как и в предыдущих примерах

Запускает две дочерние корутины. coroutineScope связывает их с внешней областью видимости, в данном случае глобальной

Отменяет родительскую корутину

И снова, сколько здесь ни ждать, вывод не появится

Если вы видели, какой объем кода на Java приходится писать, чтобы достичь такой же координации, становится очевидно, сколько времени и усилий экономит Kotlin.

Корутины — отличный пример ситуации, где Kotlin пользуется преимуществами того, что он является отдельным языком с собственным компилятором, что позволяет работать с библиотеками, чтобы чисто и компактно реализовать сложное поведение. Тема корутин настолько обширна, что мы вернемся к ним в главе 16, чтобы изучить еще подробнее.

И все же ни один язык не существует в полной изоляции, особенно на JVM. Успех и быстрое распространение Kotlin в значительной степени объясняются тем, что он хорошо совместим с необъятным миром существующего кода на Java.

9.6. СОВМЕСТИМОСТЬ С JAVA

Как вы узнали в главе 4, файл класса занимает центральное место в модели выполнения JVM. Не приходится удивляться тому, что компилятор Kotlin (`kotlinc`) генерирует файлы классов — по аналогии с `javac` для Java, как показано на рис. 9.2.

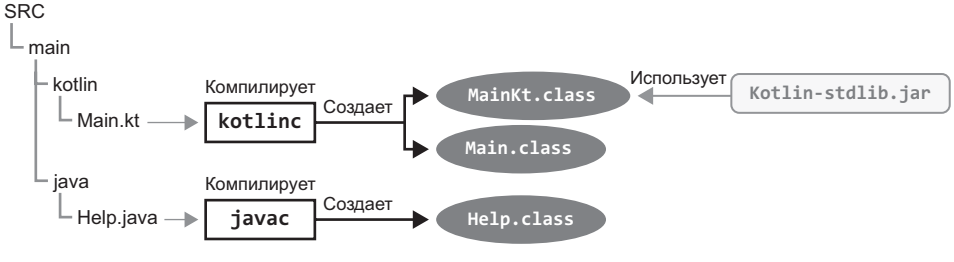


Рис. 9.2. Kotlin и Java совместно генерируют файлы классов

https://t.me/javalib

Основные определения классов в этих языках выглядят похоже, но когда Kotlin предоставляет возможности, которых нет в Java, в сгенерированных файлах классов проявляются более интересные различия. Это признаки *фикций компилятора*, о которых мы упоминали в главе 8.

Например, функции Kotlin верхнего уровня, которые не принадлежат классам, даже не поддерживаются напрямую в формате файла класса JVM. Kotlin заполняет этот пробел, генерируя класс с суффиксом `Kt`, имя которого соответствует компилируемому файлу. Все функции верхнего уровня из этого файла будут принадлежать классу `Kt`.

ПРИМЕЧАНИЕ Имя генерируемого класса можно изменить с помощью аннотации `@file:JvmName("AlternateClassName")` в файле .kt.

Приведем пример:

```
// Main.kt
package com.wellgrounded.kotlin

fun shout() {
  println("Здесь нет классов!")
}
```

Имя генерируемого класса по умолчанию определяется именем файла

Пользователям функции, как обычно, понадобится импортировать ее из пакета

Компилятор сгенерирует файл класса MainKt.class, в котором содержится наша функция. Так как в Java не бывает функций верхнего уровня, в коде на Java эту функцию придется использовать через промежуточный класс:

```
// Help.java
import com.wellgrounded.kotlin.MainKt;

public class Help {
  public static void main(String[] args) {
    MainKt.shout();
  }
}
```

Импортирует класс, который Kotlin создал как обертку для функции

Вызывает функцию, используя синтаксис статических методов Java

Еще одно важное удобство Kotlin — встроенная поддержка свойств. Благодаря использованию `val` и `var` нам не приходится заполнять целые экраны шаблонным кодом методов чтения и записи. Если рассматривать классы Kotlin из Java, мы увидим, что на нижнем уровне эти методы все равно существуют, просто Kotlin упаковывает их для нашего удобства:

```
// Person.kt
class Person(var name: String)

// App.java
public class App {
  public static void main(String[] args) {
    Person p = new Person("Кто-то");
    System.out.println(p.getName());
  }
```

Это изменяемое свойство, потому что оно объявлено с var

Когда класс Kotlin используется в Java, его экземпляры создаются оператором new

Обращение Person.name в Kotlin превращается в Person.getName() в Java

https://t.me/javalib

```
    p.setName("Кто-то другой");
    System.out.println(p.getName());
  }
}
```

Обращение Person.name = "..." в Kotlin превращается в Person.setName("...") в Java. Обратите внимание, что этот метод записи доступен только потому, что в классе Person свойство name объявлено с ключевым словом var, то есть как изменяемое. Если бы свойство name было объявлено с val, то был бы сгенерирован только метод getName()

ПРИМЕЧАНИЕ Этот пример показывает, что во внутренней реализации Kotlin действует по стандартной схеме: создает частное поле и обертывает доступ к нему. Kotlin позволяет обращаться к свойствам в более естественной форме, не допуская прямого доступа к полям.

Многие другие вспомогательные средства Kotlin не проявляются в сгенерированном коде, который используется из другого языка JVM. Например, именованные аргументы работают только в коде Kotlin, потому что в Java просто невозможно обратиться к аргументу по имени.

На первый взгляд может показаться, что это относится и к значениям по умолчанию: в конце концов, вызов из Java требует явно передавать все аргументы функции:

```
// Person.kt
class Person(var name: String) {
  fun greet(words: String = "Привет") {
    println(words)
  }
}
```

В коде Kotlin задается значение по умолчанию для аргумента words

```
// App.java
public class App {
  public static void main(String[] args) {
    Person p = new Person("Кто-то");
    // p.greet();
    p.greet("Как дела?");
  }
}
```

Если попытаться так вызвать функцию со значением по умолчанию, произойдет ошибка компиляции: списки фактических и формальных аргументов имеют разную длину

Можно передать собственное значение

Тем не менее обходной путь все-таки существует, а значит, нам не придется отказываться от удобств Kotlin. Аннотация `@JvmOverloads` заставляет Kotlin явно сгенерировать необходимые перегруженные варианты функции, чтобы ее вызов из других языков JVM выглядел так же:

```
// Person.kt
class Person(var name: String) {
  @JvmOverloads
  fun greet(words: String = "Привет") {
    println(words)
  }
}
```

Снабжаем функцию аннотацией и предоставляем значение по умолчанию, как и раньше

https://t.me/javalib

```
// App.java
public class App {
  public static void main(String[] args) {
    Person p = new Person("Кто-то");
    p.greet();
    p.greet("Как дела?");
  }
}
```

Работает нормально и выводит значение по умолчанию «Привет»

Тоже работает нормально и выводит альтернативное приветствие, которое мы передали функции

Есть и другие аннотации, которые управляют реализацией кода Kotlin на уровне JVM. Одну из них — `@JvmName` — мы уже видели в другом контексте: она применяется как к функциям, так и к файлам и управляет тем, как формируются имена за пределами Kotlin. Аннотация `@JvmField` позволяет обойтись без оберток свойств и предоставить доступ к полю, если это требуется.

Остается упомянуть об аннотации `@JvmStatic`. Как вы уже видели, Kotlin обертывает функции верхнего уровня в классы со специальными именами, к которым можно обращаться как к статическим методам в Java. Даже если вы стараетесь обойтись без статических методов, во всех приложениях на Java придется иметь дело с одним важным статическим методом: это `main`, который запускает приложение.

Если вы создаете приложение на Kotlin, можно пометить его метод `main` аннотацией `@JvmStatic`, чтобы при запуске не приходилось использовать непривычные имена:

```
class App {
  companion object {
    @JvmStatic fun main(args: Array<String>) {
      println("Привет от Kotlin")
    }
  }
}
```

Класс App, который указывается как главный класс для запуска

@JvmStatic означает, что функция будет присутствовать как статический метод в содержащем ее классе, а не только в компаньоне

Сменить язык на проекте — это обычно очень серьезный шаг. Kotlin отчасти облегчает этот шаг, задействуя стандартные паттерны многоязычных проектов на JVM. Как и следовало ожидать, IntelliJ IDEA предоставляет разработчикам дополнительную поддержку. Мы рассмотрим стандартную структуру проекта в главе 11, а пока достаточно знать, что языки, которые используются в проектах, обычно выстраиваются в иерархию каталогов наподобие этой:

```
└── src
    └── main
        ├── java
        │   └── JavaCode.java
        └── kotlin
            └── KotlinCode.kt
```

Такое разделение помогает средствам сборки легко определить, что понадобится для того, чтобы все части кода успешно взаимодействовали.

https://t.me/javalib

Если вы работаете в IntelliJ IDEA, то благодаря добрым людям из JetBrains в вашем распоряжении есть дополнительные удобства. Если щелкнуть правой кнопкой мыши на файле Java, откроется меню с командой, которая преобразует этот файл в Kotlin. Аналогичное преобразование доступно, если вы вставляете код на Java в файл Kotlin. Это позволяет начать миграцию системы с наиболее удобной точки — возможно, с тестов или с модуля, который не слишком тесно связан с остальным приложением.

IDE поможет вам и с другими операциями, однако миграция не сводится к тому, чтобы просто перевести несколько исходных файлов на другой язык. Ваши средства сборки должны знать о Kotlin, чтобы компилировать его код вместе с существующим. Кроме того, стандартную библиотеку Kotlin `kotlin-stdlib` нужно включить в проект как зависимость. О том, как управлять подобными зависимостями, будет рассказано в главе 11.

> **ПРИМЕЧАНИЕ** Хотя IntelliJ IDEA умеет преобразовывать Java в Kotlin, она не работает в обратном направлении. Кроме того, ее преобразование может оказаться не идеальным с точки зрения того, как лучше писать код на Kotlin. Как обычно, перед началом большой миграции стоит привести в готовность систему управления версиями.

Код на Kotlin компилируется в файлы классов и предоставляет бóльшую часть дополнительной функциональности через библиотеки, а значит, даже после того, как в проект добавится новый язык, код все равно продолжит выполняться на старой доброй JVM.

ИТОГИ

- Kotlin — практичный, привлекательный альтернативный язык для JVM.
- Kotlin опирается на многолетний опыт практического использования Java и как новый язык вносит изменения, которые в Java, возможно, никогда не удастся воспроизвести из-за обратной совместимости.
- Kotlin ценит компактность кода. Знакомые конструкции Java почти всегда можно записать на Kotlin короче.
- Безопасность играет ключевую роль в Kotlin. Средства `null`-безопасности встроены прямо в язык, чтобы сократить число исключений `NullPointerException` в эксплуатационном коде.
- Корутины — это привлекательная альтернатива классической модели потоков Java.
- Механизм сценариев Kotlin (`kts`) позволяет создавать сценарии, которые традиционно относились к сфере динамических языков или командной оболочки.
- На Kotlin можно писать даже сценарии сборки, как мы увидим, когда будем рассматривать Gradle в главе 11.

https://t.me/javalib

10 Clojure: другой взгляд на программирование

В ЭТОЙ ГЛАВЕ

- ✓ Понятия идентичности и состояния в Clojure
- ✓ Среда REPL для Clojure
- ✓ Синтаксис, структуры данных и последовательности в Clojure
- ✓ Интеграция Clojure с Java
- ✓ Макросы в Clojure

Стиль программирования на Clojure очень сильно отличается от Java и других языков, которые мы изучали ранее. Clojure — это переработанная для JVM версия одного из самых старых языков программирования — Lisp. Если вы не знакомы с Lisp, не беспокойтесь: перед тем, как приступить к Clojure, мы расскажем все, что необходимо знать о семействе языков Lisp.

ПРИМЕЧАНИЕ Поскольку Clojure сильно отличается от Java, во время чтения этой главы вам могут пригодиться дополнительные источники информации по этому языку. Порекомендуем пару превосходных книг: «Clojure in Action» (Manning, 2016; https://livebook.manning.com/book/clojure-in-action-second-edition) и «The Joy of Clojure» (Manning, 2014; https://livebook.manning.com/book/the-joy-of-clojure-second-edition).

К мощным средствам программирования, унаследованным от классического Lisp, Clojure добавляет замечательные современные технологии, которые актуальны для современных Java-разработчиков. Благодаря этому сочетанию Clojure

https://t.me/javalib

выделяется на фоне других языков для JVM и становится привлекательным кандидатом для разработки приложений. Среди новых технологий Clojure стоит особо выделить инструменты конкурентного программирования (которые будут рассматриваться в главе 16) и структуры данных (которые кратко представлены в этой главе, а более подробно обсуждаются в главе 15).

Для любознательного читателя, которому не терпится узнать все поскорее, скажем сразу: абстракции конкурентного выполнения позволяют писать на Clojure намного более безопасный многопоточный код, чем на Java. Эти абстракции можно совмещать с последовательностями Clojure (которые представляют собой альтернативный подход к коллекциям и структурам данных), в результате чего в вашем распоряжении окажется мощный инструментарий разработки.

Чтобы в полной мере раскрыть всю эту мощь, некоторые важные элементы языка устроены совершенно не так, как в Java. Из-за различий в подходах Clojure интереснее изучать; возможно, эти различия даже изменят ваше отношение к программированию.

> **ПРИМЕЧАНИЕ** Изучение Clojure поможет вам стать более квалифицированным программистом на любом языке. Функциональное программирование действительно важно.

Эта глава начинается с того, как Clojure оперирует состоянием и переменными. После нескольких простых примеров мы представим базовую терминологию языка — *специальные формы*, эквивалентные ключевым словам в таких языках, как Java. На небольшом наборе этих форм строится весь язык.

Мы также подробно изучим синтаксис Clojure для структур данных, циклов и функций. Это позволит нам ввести понятие последовательности — одной из самых мощных абстракций Clojure.

В конце главы рассматриваются две очень интересные возможности: тесная интеграция с Java и великолепная поддержка макросов в Clojure (которая играет ключевую роль в чрезвычайно гибком синтаксисе Lisp). Мы еще вернемся к удобным возможностям Clojure (наряду с примерами на Kotlin и Java), когда пойдет речь о функциональном программировании (глава 15) и расширенных возможностях конкурентного выполнения (глава 16).

10.1. ВВЕДЕНИЕ В CLOJURE

Базовая единица синтаксиса Lisp — вычисляемое выражение. Такие выражения обычно состоят из нуля и более символических имен, заключенных в скобки. Если вычисление проходит без ошибок, выражение называется *формой* (form).

> **ПРИМЕЧАНИЕ** Clojure — компилируемый, а не интерпретируемый язык, но его компилятор очень прост. Также не забывайте, что Clojure использует динамическую типизацию, так что компилятор мало поможет вам с проверкой типов: ошибки в типах проявятся в виде исключений времени выполнения.

https://t.me/javalib

Вот несколько простых примеров форм:

```
0
(+ 3 4)
(list 42)
(quote (a b c))
```

В ядре языка есть совсем немного встроенных форм (они называются специальными формами). В Clojure они играют ту же роль, что ключевые слова в Java, но есть нюансы:

1. В Clojure термин «*ключевое слово*» имеет особый смысл, о котором мы скоро расскажем.
2. В Clojure (как и во всех разновидностях Lisp) можно создавать конструкции, которые будут неотличимы от встроенного синтаксиса.

При работе с кодом на Clojure почти всегда неважно, являются ли используемые формы специальными формами или библиотечными функциями, построенными из них.

В начале знакомства с формами мы рассмотрим одно из важнейших концептуальных отличий Clojure от Java. Речь идет о состоянии, переменных и хранении данных. Как видно из рис. 10.1, в модели памяти и состояния, которую использует Java (как и Kotlin), переменная рассматривается как «ящик» (на самом деле — область памяти), содержимое которого может изменяться во времени.

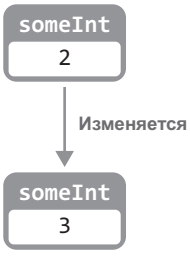


Рис. 10.1. Использование памяти в императивном языке

Такие языки программирования, как Java, относятся к *изменяемым по умолчанию*, потому что мы изменяем состояние программы, которое в Java состоит из объектов. Языки, которые следуют этой модели, часто называются императивными (см. главу 8).

В Clojure дело обстоит иначе: важным понятием языка считается *значение*. Значениями могут быть числа, строки, векторы, отображения, множества и многие другие сущности. После того как значение создано, оно никогда не изменяется. Это принципиально важный момент, поэтому повторим еще раз: после того, как значение в Clojure создано, оно никогда не изменяется, — таким образом, значения являются *неизменяемыми*.

https://t.me/javalib

ПРИМЕЧАНИЕ Неизменяемость характерна для языков функционального программирования, потому что она позволяет описывать свойства функций (например, тот факт, что одни и те же входные данные всегда приводят к одному и тому же выводу) языком теоретической математики.

В Clojure не используется стандартная для императивных языков модель «ящика», содержимое которого может изменяться. На рис. 10.2 показано, как Clojure обходится с состоянием и памятью: он создает ассоциацию между именем и значением.

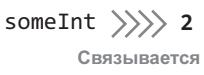


Рис. 10.2. Использование памяти в Clojure

Эта ассоциация называется *связыванием* (binding) и создается с помощью специальной формы `def`. Взгляните на ее синтаксис:

```
(def <имя> <значение>)
```

Если синтаксис выглядит немного странно, не переживайте: он абсолютно нормален для Lisp, и вы быстро к нему привыкнете. Пока можно представить себе, будто скобки расположены немного иначе и на самом деле вы вызываете метод:

```
def(<имя> <значение>)
```

Продемонстрируем `(def)` на классическом примере, в котором используется интерактивная среда Clojure.

10.1.1. Программа «Hello World» на Clojure

Если вы еще не установили Clojure, на Mac это можно сделать следующей командой:

```
brew install clojure/tools/clojure
```

Эта команда устанавливает инструменты командной строки из источника clojure/tools. Инструкции для других операционных систем доступны на сайте *clojure.org*.

ПРИМЕЧАНИЕ В системе Windows поддержка Clojure оставляет желать лучшего: например, `clj` все еще находится на альфа-стадии. Будьте внимательны, выполняя инструкции с сайта.

После того как установка будет завершена, можно запустить интерактивный сеанс Clojure командой `clj`. А если вы собирали Clojure из исходного кода, перейдите в каталог, в котором вы установили Clojure, и выполните следующую команду:

```
java -cp clojure.jar clojure.main
```

https://t.me/javalib

В любом случае на экране появится приглашение оболочки REPL (Read-Evaluate-Print Loop) для Clojure. В оболочке работает интерактивный сеанс, в котором вы будете проводить достаточно много времени, разрабатывая код на Clojure. Он выглядит примерно так:

```
$ clj
Clojure 1.10.1
user=>
```

Часть `user=>` — это приглашение ввода Clojure в сеансе, который можно рассматривать как расширенный отладчик или командную строку. Чтобы завершить сеанс (потеряв все накопленное в нем состояние), используйте традиционное для Unix сочетание клавиш Ctrl+D. Давайте напишем на Clojure программу «Hello World»:

```
user=> (def hello (fn [] "Hello world"))
#'user/hello

user=> (hello)
"Hello world"
user=>
```

Этот код начинается с того, что *идентификатор* `hello` связывается со значением. `(def)` всегда связывает идентификаторы (в Clojure они называются *символическими именами* — symbols) со *значениями*. Во внутренней реализации также создается объект *переменной*, который представляет связывание (и символическое имя):

```
(def hello (fn [] "Hello world"))
 --- ----- ---------------------
  |    |             |
  |    |           значение
  |  символическое имя
  |
специальная форма
```

С каким значением связывается имя `hello`? Вот это значение:

```
(fn [] "Hello world")
```

Это функция, которая в Clojure является подлинным значением (а следовательно, неизменяема). Она не получает аргументов и возвращает строку «Hello world». Пустой список аргументов обозначается квадратными скобками: `[]`.

> **ПРИМЕЧАНИЕ** В Clojure (но не в других разновидностях Lisp) квадратные скобки обозначают линейную структуру данных, которая называется вектором, в данном случае вектор аргументов функции.

После связывания функция выполняется с помощью кода `(hello)`. Это заставляет среду выполнения Clojure вывести результат вычисления функции, то есть строку «Hello World».

https://t.me/javalib

Вспомните, что круглые скобки в языках семейства Lisp означают вычисление функции, так что пример фактически состоит из таких шагов:

- Создать функцию и связать ее с символическим именем `hello`.
- Вызвать функцию, связанную с символическим именем `hello`.

На этом месте вам стоит ввести пример «Hello World» (если вы еще этого не сделали) и убедиться, что он ведет себя именно так. После этого переходите к следующему разделу.

10.1.2. Основы работы с REPL

Оболочка REPL позволяет вводить код на Clojure и выполнять функции Clojure. Это интерактивная среда, и в ней сохраняются результаты более ранних вычислений. Такой тип программирования называется *исследовательским программированием* (exploratory programming), которое, по сути, означает, что вы экспериментируете с кодом. Во многих случаях стоит экспериментировать в REPL и строить функции все большего размера, пока вы не получите нужные структурные элементы.

> **ПРИМЕЧАНИЕ** Ключевой метод функционального программирования — последовательное деление: задача разбивается на меньшие части, пока не станет либо решаемой, либо подходящей для того, чтобы применить к ней стандартное решение (которое уже может присутствовать в стандартной библиотеке).

Присмотримся к синтаксису Clojure поближе. Прежде всего стоит отметить, что связывание символического имени со значением можно изменить другим вызовом `(def)`; выполним эту операцию в REPL. В данном случае используется разновидность `(def)`, которая называется `(defn)`:

```
user=> (hello)
"Hello world"

user=> (defn hello [] "Goodnight Moon")
#'user/hello

user=> (hello)
"Goodnight Moon"
```

Обратите внимание: исходное связывание для `hello` продолжает действовать, пока вы его не измените, — это фундаментальная особенность REPL. При этом мы наблюдаем состояние (оно складывается из того, какие символические имена связываются с какими значениями), и это состояние сохраняется между строками, которые вводит пользователь.

Возможность связать символическое имя с другим значением — это альтернатива изменению состояния в Clojure. Вместо того чтобы позволять изменять содержимое области памяти («ящика») со временем, Clojure разрешает связывать символическое имя с разными неизменяемыми значениями в разные моменты

https://t.me/javalib

времени. Иными словами, переменная может указывать на разные значения на протяжении жизненного цикла программы. Пример показан на рис. 10.3.

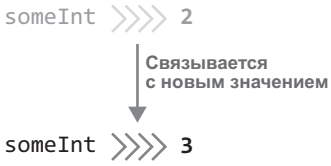


Рис. 10.3. Изменение связывания в Clojure со временем

> **ПРИМЕЧАНИЕ** Отличия между изменяемым состоянием и изменением связывания в разные моменты времени трудноуловимы, но это важная концепция, которую необходимо усвоить. Помните: изменяемое состояние означает, что содержимое «ящика» может измениться, а изменение связывания означает, что мы связываемся с разными «ящиками» в разные моменты времени.

В каком-то отношении это напоминает концепцию конечных ссылок в Java: если вы используете `final int`, то содержимое соответствующей области памяти не может измениться. Так как значения `int` хранятся в виде последовательностей битов, это означает, что значение `int` не может измениться.

Если использовать запись `final AtomicInteger`, содержимое памяти тоже не может измениться. Однако это другой случай, потому что переменная, содержащая атомарное целое число, на самом деле содержит ссылку на объект. Объект `AtomicInteger`, который хранится в куче, может изменить содержащееся в нем значение (в отличие от `Integer`), и это справедливо независимо от того, объявлена ли ссылка на объект как конечная.

В последнем примере мы использовали еще одно понятие Clojure — *макрос* определения функции (`defn`). Макросы относятся к важнейшим концепциям языков семейства Lisp. Они воплощают идею о том, что различия между встроенными конструкциями и обычным кодом должны быть сведены к минимуму.

> **ПРИМЕЧАНИЕ** Макросы позволяют создавать формы, которые ведут себя подобно встроенному синтаксису. Создание макросов — непростая тема, но если вы этому научитесь, то сможете строить невероятно мощные инструменты.

С помощью истинных языковых примитивов системы (специальных форм) можно создавать конструкции, которые расширяют ядро языка так, что вы практически не будете замечать различий между одним и другим.

> **ПРИМЕЧАНИЕ** Макрос (`defn`) служит примером такого рода. Это всего лишь немного более простой способ связать значение функции с символическим именем (и конечно, создать соответствующую переменную). (`defn`) — не специальная форма, а просто макрос, построенный из специальных форм (`def`) и (`fn`).

Мы рассмотрим макросы подробнее в конце главы.

https://t.me/javalib

10.1.3. Ошибки

Что произойдет, если допустить ошибку? Допустим, вы пытаетесь объявить функцию, но вместо этого случайно определяете значение:

```
user=> (def hello "Goodnight Moon")
#'user/hello

user=> (hello)
Execution error (ClassCastException) at user/eval137 (REPL:1).
class java.lang.String cannot be cast to class clojure.lang.IFn
(java.lang.String is in module java.base of loader 'bootstrap';
clojure.lang.IFn is in unnamed module of loader 'app')[1]
```

Здесь стоит обратить внимание на пару моментов. Во-первых, ошибка является исключением времени выполнения. Это означает, что форма `(hello)` откомпилировалась нормально, просто во время выполнения произошел сбой. В эквивалентном коде Java это выглядит примерно так (мы немного упростили код, чтобы он был понятнее для читателей, у которых нет опыта работы с Clojure или реализацией языка):

```
// (def hello "Goodnight Moon ")
var helloSym = Symbol.of("user", "hello");
var hello = Var.of(helloSym, "Goodnight Moon");

// Или просто
// var hello = Var.of(Symbol.of("user", "hello"), "Goodnight Moon");

// #'user/hello

// (hello)
hello.invoke();

// ClassCastException
```

Здесь `Symbol` и `Var` — классы из пакета `clojure.lang`, который предоставляет основную функциональность среды выполнения Clojure. Они похожи на базовые реализации, которые приводятся ниже в упрощенном виде:

```
public class Symbol {
    private final String ns;
    private final String name;

    private Symbol(String ns, String name) {
        this.ns = ns;
        this.name = name;
    }
```

[1] Ошибка времени выполнения (ClassCastException) по адресу user/eval137 (REPL:1). класс java.lang.String нельзя привести к классу clojure.lang.IFn (java.lang.String находится в модуле java.base загрузчика 'bootstrap'; clojure.lang.IFn находится в безымянном модуле загрузчика 'app'). — *Примеч. пер.*

https://t.me/javalib

```
    // toString() и т. д.
}

public class Var implements IFn {
    private volatile Object root;

    public final Symbol sym;
    public final Namespace ns;

    private Var(Symbol sym, Namespace ns, Object root) {
        this.sym = sym;
        this.ns = ns;
        this.root = root;
    }

    public static Var of(Symbol sym, Object root){
        return new Var(sym, Namespace.of(sym), root);
    }

    static public class Unbound implements IFn {
        final public Var v;
        public Unbound(Var v){
            this.v = v;
        }

        @Override
        public String toString(){
            return "Unbound:"+v;
        }
    }

    public synchronized void bindRoot(Object root) {
        this.root = root;
    }

    public synchronized void unBindRoot(Object root) {
        this.root = new Unbound(this);
    }

    @Override
    public Object invoke() {
        return ((IFn)root).invoke();
    }

    @Override
    public Object invoke(Object o1) {
        return ((IFn)root).invoke(o1);
    }

    @Override
    public Object invoke(Object o1, Object o2) {
        return ((IFn)root).invoke(o1, o2);
    }

    @Override
```

https://t.me/javalib

```
    public Object invoke(Object o1, Object o2, Object o3) {
        return ((IFn)root).invoke(o1, o2, o3);
    }
    // ...
}
```

Крайне важный интерфейс `IFn` выглядит примерно так:

```
public interface IFn {
    default Object invoke() {
        return throwArity();
    }
    default Object invoke(Object o1) {
        return throwArity();
    }
    default Object invoke(Object o1, Object o2) {
        return throwArity();
    }
    default Object invoke(Object o1, Object o2, Object o3) {
        return throwArity();
    }
    // ... Много других, включая вариадическую форму

    default Object throwArity(){
        throw new IllegalArgumentException("Wrong number of args passed[1]: "
                + toString());
    }
}
```

`IFn` играет ключевую роль в том, как работают формы Clojure: первый элемент формы рассматривается как функция (или имя функции), которую следует вызвать. Остальные элементы считаются аргументами функции, и вызывается метод `invoke()` с подходящим количеством аргументов («арностью»).

Если переменная Clojure не связана со значением, которое реализует `IFn`, то во время выполнения возбуждается исключение `ClassCastException`. Если значение реализует `IFn`, но форма пытается вызвать его с неправильным количеством аргументов, возбуждается исключение `IllegalArgumentException` (точнее, его подтип `ArityException`).

> **ПРИМЕЧАНИЕ** Не забудьте, что Clojure — язык с динамической типизацией, что проявляется во многих аспектах. Например, все аргументы и возвращаемые типы методов в IFn имеют тип `Object`, а `IFn` не является интерфейсом `@FunctionalInterface` в стиле Java; вместо этого в нем определяются несколько методов для разных арностей.

Этот экскурс во внутреннюю реализацию немного проясняет как синтаксис Clojure, так и принципы работы кода в наших примерах. Осталось исправить ошибки в коде — к счастью, это несложно!

[1] Передано неправильное количество аргументов. — *Примеч. пер.*

https://t.me/javalib

Что же произошло? Идентификатор `hello` связан с чем-то, что не является функцией, и поэтому его невозможно вызвать. В REPL проблема решается простым повторным связыванием:

```
user=> (defn hello [] (println "Dydh da an Nor")) ; "Hello World" по-корнски
#'user/hello

user=> (hello)
Dydh da an Nor
nil
```

Как вы наверняка догадались по приведенному фрагменту, точка с запятой (`;`) означает, что все символы до конца строки считаются комментарием, а (`println`) — функция для вывода строки. Учтите, что (`println`), как и все функции, возвращает значение, которое воспроизводится в REPL в конце выполнения функции.

В Clojure нет инструкций, как в Java, а есть только выражения, так что все функции должны возвращать значение. Если возвращать нечего, используется `nil` — по сути, эквивалент `null` из языка Java. Функции, которые в Java были бы помечены `void`, в Clojure вернут `nil`.

10.1.4. Привыкаем к круглым скобкам

Профессиональной культуре программистов всегда была свойственна немалая доля юмора и причуд. Одна из самых старых шуток гласит, что Lisp — это сокращение от «Lots of Irritating Silly Parentheses», то есть «Множество раздражающих дурацких скобок» (вместо более прозаической правды — это сокращение от «List Processing», то есть «обработка списков»). Эта самокритичная шутка популярна среди программистов на Lisp в том числе потому, что она указывает на неприятный факт: синтаксис Lisp считается трудным для изучения.

В реальности это затруднение сильно преувеличено. Синтаксис Lisp отличается от того, к чему привыкли многие программисты, но это препятствие не настолько серьезно, как его иногда представляют. Кроме того, в Clojure есть ряд нововведений, которые дополнительно снижают порог входа.

Еще раз присмотримся к примеру «Hello World». Чтобы вызвать функцию, которая возвращает значение «Hello World», мы использовали такую запись:

```
(hello)
```

Если функция должна вызываться с аргументами, то вместо выражений вида `myFunction(someObj)` в Clojure используется запись `(myFunction someObj)`. Этот синтаксис называется «польской записью», потому что его придумали польские математики в начале XX века (также говорят «префиксная нотация»).

Если вы изучали историю компиляторов, то возможно, у вас появится вопрос: не связана ли эта запись с таким понятием, как абстрактное синтаксическое дерево (AST)? Да, связана. Программа на Clojure (или на любой другой разновидности

https://t.me/javalib

Lisp), закодированная в польской записи (которую программисты Lisp обычно называют *s-выражением*), является очень простым и прямым представлением AST этой программы.

> **ПРИМЕЧАНИЕ** Это также объясняет, почему компилятор Clojure такой простой. Код на Lisp компилируется с минимальными затратами, потому что его структура чрезвычайно близка к AST.

Программу на Lisp можно рассматривать как написанную непосредственно на уровне AST. Структура данных, которая представляет программу, на самом деле не отличается от кода, так что код и данные взаимозаменяемы. И в этом заключается причина для несколько странной записи: благодаря ей в языках из семейства Lisp стираются различия между встроенными примитивами и пользовательским или библиотечным кодом. Это придает языку такую мощь, ради которой можно пренебречь странностями синтаксиса с точки зрения Java-программистов, осваивающих новый язык. Давайте глубже рассмотрим этот синтаксис и начнем создавать реальные программы на Clojure.

10.2. CLOJURE: СИНТАКСИС И СЕМАНТИКА

В предыдущем разделе мы познакомились со специальными формами `(def)` и `(fn)` (а также с `(defn)`, но это макрос, а не специальная форма). Сейчас вам предстоит освоить еще несколько специальных форм, чтобы сформировать базовую номенклатуру языка.

В Clojure есть множество полезных функций для самых разнообразных задач. Не пугайтесь этого изобилия — берите его на вооружение. Пускай вас вдохновляет то, что для многих практических задач программирования, с которыми вы можете столкнуться в Clojure, уже существуют готовые решения.

В этом разделе мы рассмотрим базовый набор специальных форм, а затем займемся собственными типами данных Clojure (аналогами коллекций Java). После этого мы перейдем к естественному стилю написания кода на Clojure, в котором центральное место занимают функции, а не переменные. Хотя «под капотом» по-прежнему остается JVM со своей объектно-ориентированной природой, особый акцент, который Clojure придает функциям, обладает потенциалом, который не очевиден в чистых объектно-ориентированных языках и выходит далеко за рамки простейших функций `map()`, `filter()` и `reduce()`.

10.2.1. Краткий курс специальных форм

В табл. 10.1 перечислены специальные формы Clojure, которые используются особенно часто. Чтобы извлечь максимум пользы из этой таблицы, бегло просмотрите ее сейчас и возвращайтесь к ней позднее, когда мы начнем рассматривать примеры в разделе 10.3 и далее. В таблице используется традиционный синтаксис регулярных выражений, где `?` представляет единичное необязательное значение, а `*` — серию из нуля и более значений.

https://t.me/javalib

Это не исчерпывающий список специальных форм, и многие из них можно использовать разными способами. Рассматривайте табл. 10.1 скорее как начальную подборку основных сценариев использования, а не как фундаментальный справочник.

Таблица 10.1. Некоторые базовые специальные формы Clojure

Специальная форма	Описание
`(def <символическое_имя> <значение?>)`	Связывает символическое имя со значением (если оно задано); при необходимости создает переменную, которая соответствует символическому имени
`(fn <имя>? [<аргументы>*] <выражение>*)`	Возвращает значение-функцию, которая принимает заданные аргументы и применяет их к выражениям; часто объединяется с `(def)` в формы вида `(defn)`
`(if <условие> <то> <иначе>?)`	Если условие истинно, вычисляет и возвращает «то»; в противном случае вычисляет и возвращает «иначе» (если оно есть)
`(do <выражение>*)`	Вычисляет выражения слева направо и возвращает значение последнего
`(let [<связь>*] <выражение>*)`	Делает значения синонимами локального имени и неявно определяет область видимости; синоним становится доступным во всех выражениях в области видимости `let`
`(quote <форма>)`	Возвращает форму «как есть», ничего не вычисляя; принимает одну форму и игнорирует все остальные аргументы
`(var <символическое_имя>)`	Возвращает переменную, которая соответствует символическому имени (возвращает Clojure-объект JVM, а не значение)

Пара моментов заслуживает дополнительных разъяснений, потому что структура кода на Clojure на первый взгляд совсем не похожа на Java. Во-первых, форма `(do)` — это один из простейших способов сконструировать то, что в Java было бы блоком команд.

Во-вторых, нужно чуть глубже рассмотреть различия между переменной, значением и символическим именем, с которым (временно) связано значение. В следующем простом фрагменте создается переменная с именем `hi`. Это объект JVM (экземпляр типа `clojure.lang.Var`), который находится в куче (как и все объекты) и связывается с объектом `java.lang.String`, содержащим текст «hello»:

```
user=> (def hi "Hello")
#'user/hi
```

Переменной соответствует *символическое имя* `hi` и *пространство имен* `user`, которое Clojure использует для организации программ, — что-то вроде пакета

https://t.me/javalib

Java. Если вы введете в REPL символическое имя само по себе, оно вычислится в значение, с которым связано в настоящий момент:

```
user=> hi
"Hello"
```

В форме `(def)` со значением связывается новое символическое имя, так что в следующем коде:

```
user=> (def bye hi)
#'user/bye
```

символическое имя `bye` связывается со *значением*, которое в настоящее время уже связано с `hi`:

```
user=> bye
"Hello"
```

Фактически в этой простой форме вычисляется `hi`, а символическое имя заменяется на полученное значение.

Однако Clojure этим не ограничивается. Например, значение, с которым связано символическое имя, может быть любым значением JVM. Таким образом, символическое имя можно связать с только что созданной переменной, потому что сама переменная является объектом JVM. Для этого служит специальная форма `(var)`:

```
user=> (def bye (var hi))
#'user/bye

user=> bye
#'user/hi
```

При этом используется тот факт, что обращения к объектам Java/JVM всегда происходят по ссылке, как видно на рис. 10.4.

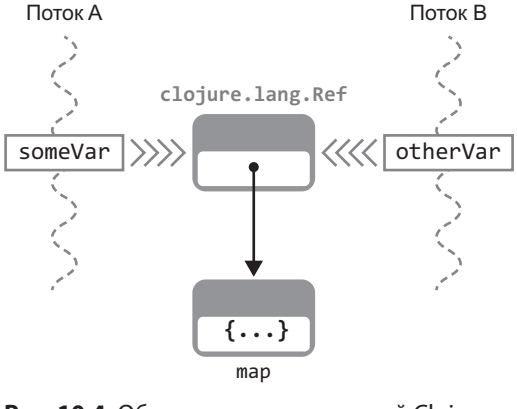


Рис. 10.4. Обращение к переменной Clojure по ссылке

https://t.me/javalib

Чтобы получить обратно значение, которое содержится в переменной, можно воспользоваться формой (deref) (сокращение от «dereference», то есть «разыменование»):

```
user=> (deref bye)
"Hello"
```

Также существует форма (ref), которая применяется для безопасного конкурентного программирования в Clojure и рассматривается в главе 16.

Разобравшись в этих различиях между переменной и значением, с которым она связана в настоящее время, вам будет легче понять форму (quote). Вместо того чтобы вычислять переданную форму, (quote) просто возвращает форму с невычисленными символическими именами.

Итак, вы получили представление о синтаксисе некоторых базовых специальных форм. Перейдем к структурам данных Clojure и посмотрим, как формы могут работать с данными.

10.2.2. Списки, векторы, отображения и множества

В Clojure есть ряд собственных структур данных. Вероятно, самая знакомая из них — *список*, который в Clojure реализуется как односвязный список.

> **ПРИМЕЧАНИЕ** В некоторых отношениях список Clojure похож на LinkedList языка Java, если не считать того, что LinkedList — двусвязный, то есть в каждом элементе хранятся ссылки как на следующий, так и на предыдущий элемент.

Списки обычно заключаются в круглые скобки, что поначалу может сбить с толку, потому что круглые скобки также используются для форм, в частности для вычисления вызовов функций. В результате многие новички допускают такую синтаксическую ошибку:

```
user=> (1 2 3)
Execution error (ClassCastException) at user/eval1 (REPL:1).
class java.lang.Long cannot be cast to class clojure.lang.IFn[1]
(java.lang.Long is in module java.base of loader 'bootstrap';
clojure.lang.IFn is in unnamed module of loader 'app')
```

Проблема в том, что Clojure исключительно гибок в отношении своих значений, поэтому в первом аргументе он ожидает получить значение-функцию (или символическое имя, которое в нее разрешается), чтобы можно было вызвать эту функцию и передать 2 и 3 в аргументах. Единица не является значением-функцией, поэтому Clojure не может вычислить эту форму. Такое s-выражение называется недействительным, а формами Clojure могут быть только действительные s-выражения.

[1] Класс java.lang.Long нельзя привести к классу clojure.lang.IFn. — *Примеч. пер.*

https://t.me/javalib

Проблема решается формой `(quote)`, которая упоминалась в предыдущем разделе и может использоваться в удобном сокращенном виде `'`. Таким образом, если мы хотим закодировать неизменяемый список из трех элементов, которыми являются числа 1, 2 и 3, это можно сделать двумя эквивалентными способами:

```
user=> '(1 2 3)
(1 2 3)

user=> (quote (1 2 3))
(1 2 3)
```

Обратите внимание, что `(quote)` обрабатывает свои аргументы особым образом. В частности, эта форма не пытается вычислять аргумент, поэтому не возникает ошибок из-за того, что в первой позиции нет значения-функции.

Также в Clojure есть векторы, которые похожи на массивы. (Вообще говоря, списки можно рассматривать как аналоги `LinkedList` языка Java, а векторы — как аналоги `ArrayList`, и такое представление будет недалеко от истины.) Векторы можно записывать с помощью квадратных скобок, так что все следующие конструкции эквивалентны:

```
user=> (vector 1 2 3)
[1 2 3]

user=> (vec '(1 2 3))
[1 2 3]

user=> [1 2 3]
[1 2 3]
```

Мы уже встречались с векторами: когда мы объявляли «Hello World» и другие функции, то с помощью вектора обозначались параметры, которые принимает функция. Форма `(vec)` принимает список и создает вектор на его основе, а форма `(vector)` принимает несколько отдельных символических имен и возвращает вектор, составленный из них.

Функция `(nth)` для коллекций принимает два параметра — коллекцию и индекс. Ее можно рассматривать как аналог метода `get()` из интерфейса `List` языка Java. Эту функцию можно применять к векторам и спискам, а также к коллекциям Java и даже к строкам, которые рассматриваются как коллекции символов. Вот пример:

```
user=> (nth '(1 2 3) 1)
2
```

Clojure также поддерживает отображения, которые можно рассматривать как близкие аналоги `HashMap` языка Java (собственно, они даже реализуют интерфейс `Map`). Отображения задаются в виде простого литерала:

```
{ключ1 значение1 ключ2 значение2}
```

https://t.me/javalib

Чтобы получить значение из отображения, используется несложный синтаксис:

```
user=> (def foo {"aaa" "111" "bbb" "2222"})
#'user/foo

user=> foo
{"aaa" "111", "bbb" "2222"}

user=> (foo "aaa")  <-- Этот синтаксис эквивалентен использованию метода get() в Java
"111"
```

Помимо интерфейса `Map`, отображения Clojure также реализуют интерфейс `IFn`, благодаря чему их можно использовать в формах вида `(foo "aaa")`, не возбуждая исключения времени выполнения.

Особую пользу приносят ключи, которые начинаются с двоеточия. В Clojure они называются *ключевыми словами* (keywords).

> **ПРИМЕЧАНИЕ** Конечно, термин «ключевое слово» в Clojure имеет совершенно иной смысл, чем в других языках (включая Java), где этим словосочетанием обозначаются зарезервированные части грамматики языка, которые нельзя использовать в качестве идентификаторов.

Вот что стоит знать о ключевых словах и отображениях:

- В Clojure ключевое слово представляет собой функцию, принимающую один аргумент, которым должно быть отображение.
- Если вызвать функцию по ключевому слову для отображения, она вернет значение, которое соответствует функции по этому ключевому слову в отображении.
- При использовании ключевых слов в синтаксисе существует полезная симметрия, так что допустимы оба варианта — `(my-map :key)` и `(:key my-map)`.
- Если ключевое слово используется как значение, оно возвращает себя.
- Перед тем как использовать ключевые слова, их не нужно объявлять или определять с `def`.
- Помните, что функции Clojure являются значениями, а значит, их можно использовать в качестве ключей в отображениях.
- Чтобы разделять пары «ключ — значение», можно (хотя не обязательно) использовать запятые, потому что Clojure воспринимает их как пробельные символы.
- В отображениях Clojure можно также использовать символические имена, которые не являются ключевыми словами, однако синтаксис ключевых слов чрезвычайно полезен и его стоит задействовать по максимуму.

https://t.me/javalib

Рассмотрим некоторые из перечисленных пунктов в действии:

```
user=> (def martijn {:name “Martijn Verburg”, :city “London”,
:area "Finsbury Park"})
#'user/martijn

user=> (:name martijn)
"Martijn Verburg"

user=> (martijn :area)
"Finsbury Park"

user=> :area
:area

user=> :foo
:foo
```

Вызывает функцию — ключевое слово для отображения

Ищет в отображении значение, связанное с ключевым словом

Если ключевое слово используется как значение, оно возвращает себя

Кроме литералов отображений, в Clojure также есть функция `(map)`. Но будьте внимательны: в отличие от `(list)`, функция `(map)` не создает отображение. Вместо этого она последовательно применяет переданную функцию к каждому элементу коллекции и составляет из возвращенных значений новую коллекцию (хотя на самом деле — последовательность Clojure, которую мы подробнее рассмотрим в разделе 10.4). Конечно, эта функция Clojure действует аналогично методу `map()`, который уже знаком вам по API потоков данных в Java:

```
user=> (def ben {:name "Ben Evans", :city "Barcelona", :area "El Born"})
#'user/ben

user=> (def authors [ben martijn])
#'user/authors

user=> (defn get-name [y] (:name y))
#'user/get-name

user=> (map get-name authors)
("Ben Evans" "Martijn Verburg")

user=> (map (fn [y] (:name y)) authors)
("Ben Evans" "Martijn Verburg")
```

Создает вектор отображений со сведениями об авторах

Отображает функцию get-name на данные

Альтернативная форма с использованием встроенного литерала функции

Существуют другие разновидности `(map)`, которые позволяют работать сразу с несколькими коллекциями, но чаще всего используется форма, которая принимает на вход одну коллекцию.

В Clojure также поддерживаются множества, которые очень похожи на `HashSet` в языке Java. У них есть короткая запись в виде литералов структур данных, которые не поддерживают повторяющиеся ключи (в отличие от `HashSet`):

https://t.me/javalib

```
user=> #{"a" "b" "c"}
#{"a" "b" "c"}

user=> #{"a" "b" "a"}
Syntax error reading source at (REPL:15:15).
Duplicate key: a[1]
```

Эти структуры данных закладывают основу для программ на Clojure.

Опытных Java-разработчиков может удивить, что мы нигде напрямую не упоминали об объектах как о сущностях первого класса. Это не означает, что Clojure не является объектно-ориентированным языком, однако его объектно-ориентированные качества проявляются не совсем так, как в Java. Java предпочитает смотреть на мир через призму явных определений классов для типов данных, задаваемых пользователем, причем классы представляют собой пакеты данных и кода со статической типизацией. Clojure вместо этого приоритизирует функции и формы, хотя на внутреннем уровне JVM они реализуются как объекты.

Это идеологическое различие между Clojure и Java проявляется в том, как пишется код на этих языках. Чтобы в полной мере вникнуть в философию Clojure, необходимо писать программы на этом языке и понимать, какие преимущества можно получить, если отодвинуть на второй план объектно-ориентированные конструкции Java.

10.2.3. Арифметические действия, проверка равенства и другие операции

В Clojure нет операторов в том виде, в каком мы привыкли работать с ними в Java. Как же, например, сложить два числа? В Java это делается просто:

```
3 + 4
```

Однако в Clojure вместо этого приходится использовать функцию:

```
(add 3 4)  <-- Этот код не будет работать сам по себе, если мы не обеспечим функцию add
```

Все это хорошо, но можно сделать еще лучше. В Clojure нет операторов, а значит, для них не нужно резервировать символы. Поэтому функциям можно присваивать более экзотические имена, чем в Java, так что ничто не мешает использовать такую запись:

```
(+ 3 4)  <-- Это типичный пример польской записи, которая упоминалась ранее
```

Во многих случаях функции Clojure являются *вариадическими* (то есть принимают переменное число входных значений), поэтому можно, например, написать такой код:

```
(+ 1 2 3)
```

[1] Синтаксическая ошибка в исходном коде по адресу (REPL:15:15). Дублируется ключ: a. — *Примеч. пер.*

https://t.me/javalib

Значением этой формы будет 6.

Для форм проверки равенства (подобных `equals()` и `==` в Java) ситуация немного усложняется. В Clojure есть две основные формы, связанные с проверкой равенства: `(=)` и `(identical?)`. Обратите внимание, что обе формы служат примерами того, как отсутствие операторов в Clojure позволяет использовать более широкий ассортимент символов в именах функций. Кроме того, `(=)` содержит один знак равенства, потому что в Clojure нет понятия присваивания, которое есть в Java.

Следующий фрагмент кода REPL создает список `list-int` и вектор `vect-int`, после чего проверяет их на равенство:

```
user=> (def list-int '(1 2 3 4))
#'user/list-int

user=> (def vect-int (vec list-int))
#'user/vect-int

user=> (= vect-int list-int)
true

user=> (identical? vect-int list-int)
false
```

Главное здесь в том, что форма `(=)` для коллекций проверяет, содержат ли коллекции одни и те же объекты в одном и том же порядке (что имеет место для `list-int` и `vect-int`), тогда как `(identical?)` проверяет, что они являются одним и тем же объектом.

Возможно, вы также заметили, что в символических именах не используется «верблюжий регистр», потому что так принято в Clojure. Символические имена обычно записываются в нижнем регистре, а слова разделяются дефисами (такой стиль иногда называется «позвоночным регистром»).

True и False в Clojure

В Clojure есть два значения, которые интерпретируются как логическая ложь: `false` и `nil`. Все остальные значения интерпретируются как истина (включая литерал `true`). Так же обстоят дела во многих динамических языках (например, JavaScript), но этот принцип может показаться странным Java-программистам, которые сталкиваются с ним впервые.

Разобравшись с базовыми структурами данных и операторами, давайте объединим некоторые специальные формы и функции, которые мы уже видели, и напишем на Clojure более длинные примеры функций.

10.2.4. Работа с функциями в Clojure

В этом разделе мы перейдем к более содержательному программированию на Clojure. Сначала мы напишем функции, которые работают с данными

https://t.me/javalib

и демонстрируют приоритетную роль функций в языке Clojure. Затем будут рассмотрены циклические конструкции, макросы считывателя и формы диспетчеризации. В конце раздела мы обсудим подход Clojure к функциональному программированию и замыканиям.

Мы начнем с простых примеров, а затем постепенно перейдем к более мощным средствам функционального программирования, которые предоставляет Clojure.

Простые функции Clojure

В следующем листинге определяются три функции: две из них очень просты и получают один аргумент, а третья чуть сложнее.

Листинг 10.1. Определение простых функций Clojure

```
(defn const-fun1 [y] 1)

(defn ident-fun [y] y)

(defn list-maker-fun [x f]
  (map (fn [z] (let [w z]
    (list w (f w))
  )) x))
```

Функция принимает два аргумента, второй из которых — тоже функция

Встроенная анонимная функция

Создает список из двух элементов: значение и результат применения функции f к этому значению

В этом листинге (`const-fun1`) принимает значение и возвращает 1, а (`ident-fun`) принимает значение и возвращает это же значение. Математики называют такие функции, соответственно, *константной* и *тождественной*. Также мы видим, что в определении функции используются литералы векторов, которые обозначают аргументы для функции и для формы (`let`).

Третья функция сложнее предыдущих. Функция (`list-maker-fun`) принимает два аргумента: вектор значений для обработки, который называется `x`, и функцию с именем `f`. Если бы мы писали этот код на Java, он мог бы выглядеть так:

```
public List<Object> listMakerFun(List<Object> x,
                                  Function<Object, Object> f) {
    return x.stream()
            .map(o -> List.of(o, f.apply(o)))
            .collect(toList());
}
```

Анонимной встроенной функции в Clojure соответствует лямбда-выражение в коде Java. Однако не стоит переоценивать эквивалентность двух листингов, Clojure и Java — совершенно разные языки.

> **ПРИМЕЧАНИЕ** Функции, которые принимают другие функции в качестве аргументов, называются функциями высшего порядка. Они будут подробно описаны в главе 15.

Посмотрим, как работает (`list-maker-fun`).

https://t.me/javalib

Листинг 10.2. Работа с функциями

```
user=> (list-maker-fun ["a"] const-fun1)
(("a" 1))

user=> (list-maker-fun ["a" "b"] const-fun1)
(("a" 1) ("b" 1))

user=> (list-maker-fun [2 1 3] ident-fun)
((2 2) (1 1) (3 3))

user=> (list-maker-fun [2 1 3] "a")
java.lang.ClassCastException: java.lang.String cannot be cast to
  clojure.lang.IFn[1]
```

Не забывайте, что когда вы вводите эти выражения в REPL, вы взаимодействуете с компилятором Clojure. Выражение `(list-maker-fun [2 1 3] "a")` не выполняется (хотя и компилируется), потому что `(list-maker-fun)` ожидает получить во втором аргументе функцию, а не строку. Таким образом, хотя компилятор Clojure генерирует байт-код для формы, при запуске возбуждается исключение времени выполнения.

> **ПРИМЕЧАНИЕ** В языке Java можно написать синтаксически правильный код вроде `Integer.parseInt("foo")`, который успешно компилируется, но всегда вызывает сбой на стадии выполнения. В Clojure похожая ситуация.

Пример показывает, что при взаимодействии с REPL все еще проявляются некоторые признаки статической типизации, потому что Clojure — не интерпретируемый язык. Даже в REPL каждая форма Clojure, которую вы вводите, компилируется в байт-код JVM и компонуется с выполняющейся системой. Функция Clojure компилируется в байт-код JVM, когда она определяется, так что исключение ClassCastException возбуждается из-за нарушения статической типизации в JVM.

В листинге 10.3 приводится более длинный код на Clojure — *преобразование Шварца*. Это идиома программирования, которая приобрела популярность благодаря языку Perl в 1990-х годах. Идея заключается в том, чтобы отсортировать вектор на основании не самого́ вектора, а некоторого свойства его элементов. Значения свойства, по которым должна производиться сортировка, определяются вызовом *ключевой функции* от элементов.

Определение преобразования Шварца в листинге 10.3 вызывает ключевую функцию `key-fn`. При фактическом вызове функции `(schwartz)` необходимо передать функцию, которая должна использоваться в качестве ключевой. В этом примере фигурирует уже знакомая форма `(ident-fun)` из листинга 10.1.

[1] java.lang.String нельзя привести к clojure.lang.IFn. — *Примеч. пер.*

https://t.me/javalib

Листинг 10.3. Преобразование Шварца

```
user=> (defn schwartz [x key-fn]
  (map (fn [y] (nth y 0))
    (sort-by (fn [t] (nth t 1))
      (map (fn [z] (let [w z]
        (list w (key-fn w))
      )) x))))
#'user/schwartz

user=> (schwartz [2 3 1 5 4] ident-fun)
(1 2 3 4 5)

user=> (apply schwartz [[2 3 1 5 4] ident-fun])
(1 2 3 4 5)
```

С помощью ключевой функции создает список, состоящий из пар

Сортирует пары на основании значений ключевой функции

Строит новый список посредством свертки: из каждой пары берется только исходное значение

Этот код выполняется за три отдельные фазы, которые на первый взгляд следуют как будто задом наперед. Последовательность действий показана на рис. 10.5.

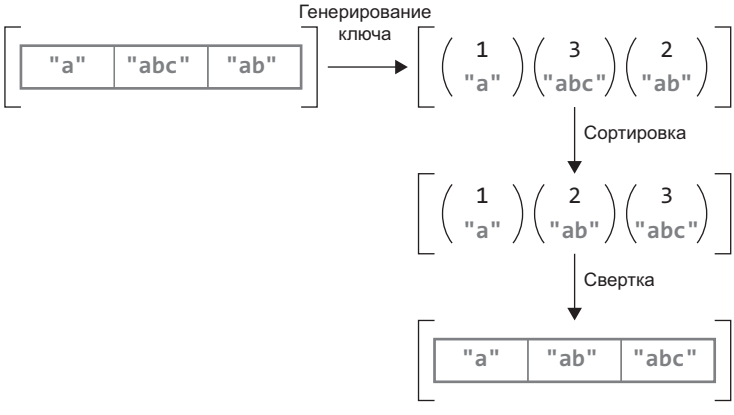


Рис. 10.5. Преобразование Шварца

Обратите внимание, что в листинге 10.3 появилась новая форма `(sort-by)`. Это функция, которая принимает два аргумента: функцию сортировки и сортируемый вектор. Также мы продемонстрировали форму `(apply)` с двумя аргументами: вызываемой функцией и вектором аргументов, которые ей передаются.

Преобразование Шварца имеет интересную историю: когда человек, по имени которого оно было названо, представил свою версию на Perl, он умышленно позаимствовал идею из Lisp. Воплотив ее в коде Clojure, мы сделали полный круг: идея снова вернулась к Lisp!

Преобразование Шварца — хороший пример, к которому мы вернемся позднее. Его сложность позволяет продемонстрировать несколько полезных концепций. А теперь перейдем к теме циклов в Clojure, которые могут показаться вам несколько непривычными.

https://t.me/javalib

10.2.5. Циклы в Clojure

Java предлагает вполне традиционный ассортимент циклов: разработчик может выбрать из циклов `for`, `while` и еще пары разновидностей. Обычно работа цикла заключается в том, что та или иная группа инструкций повторно выполняется до тех пор, пока не будет удовлетворено некоторое условие (часто выраженное в виде изменяемой переменной).

И тут специфика Clojure создает некоторое противоречие: как выразить цикл `for`, если в языке нет изменяемых переменных, которые можно было бы использовать в качестве индекса? В более традиционных версиях Lisp эта проблема часто решается тем, что итеративные циклы переписываются в форме с рекурсией.

Однако JVM не гарантирует оптимизации хвостовой рекурсии (как того требует Scheme и другие разновидности Lisp), так что наивное использование рекурсии грозит переполнить стек. Мы еще вернемся к этой проблеме в главе 15.

Вместо этого в Clojure есть полезные конструкции, которые позволяют организовать цикл, не увеличивая размер стека. Одна из самых распространенных конструкций такого рода — `loop-recur`. Следующий фрагмент показывает, как с ее помощью построить простую конструкцию, аналогичную циклу `for` языка Java:

```
(defn like-for [counter]
  (loop [ctr counter]  <------ Точка входа в цикл
    (println ctr)
    (if (< ctr 10)
      (recur (inc ctr))  <------ Точка возврата, в которой выполнение переходит к началу
      ctr
   )))
```

Форма `(loop)` получает вектор аргументов с локальными именами для символических имен — фактически синонимами, сходными с теми, которые создает `(let)`. Когда выполнение достигает формы `(recur)` (что произойдет в данном примере, только если синоним `ctr` меньше 10), `(recur)` возвращает управление к форме `(loop)`, но с новым значением. Такая конструкция напоминает весьма примитивную форму цикла Java:

```
public int likeFor(int ctr) {
        LOOP: while (true) {
            System.out.println(ctr);
            if (ctr < 10) {
                ctr=ctr+1;
                continue LOOP;
            } else {
                return ctr;
            }
        }
    }
```

Впрочем, для функционального программиста единственная нормальная причина для преждевременного возврата — это выполнение некоторого условия.

https://t.me/javalib

Однако функции возвращают результат последней вычисленной формы, а (if) фактически уже делает это за нас.

В нашем примере (recur) помещается в тело if, а значение counter — в позицию else. Это позволяет строить конструкции в итеративном стиле (например, эквиваленты циклов for и while в Java), но при этом сохранять функциональный «колорит» в реализации. А теперь перейдем к следующей теме — полезному сокращению в синтаксисе Clojure, которое сделает ваши программы еще лаконичнее и выразительнее.

10.2.6. Макросы считывателя и диспетчеризация

У синтаксиса Clojure есть особенности, которые удивляют многих программистов на Java, и одна из них — отсутствие операторов. Ее побочный эффект состоит в том, что в Closure меньше ограничений на выбор символов, которые можно использовать в именах функций. Вы уже видели функции, которые были бы недопустимы в Java, например (identical?); тем не менее мы еще не говорили о том, какие символы допустимы или недопустимы в символических именах.

В табл. 10.2 перечислены символы, которые нельзя использовать в символических именах Clojure. Все эти символы зарезервированы парсером Clojure для собственных целей. Обычно они называются *макросами считывателя* (reader macros) и по сути являются специальными последовательностями символов; они влияют на то, как ведет себя считыватель (первая часть компилятора Clojure).

Например, макрос ; в Clojure обозначает начало однострочного комментария. Когда считыватель видит ;, он немедленно игнорирует все оставшиеся символы в текущей строке и переходит к следующей строке ввода.

> **ПРИМЕЧАНИЕ** Позже мы рассмотрим обычные макросы Clojure — важно не путать их с макросами считывателя.

Макросы считывателя служат только для удобства и компактности синтаксиса и не обеспечивают полноценных средств метапрограммирования общего назначения.

Таблица 10.2. Макросы считывателя

Символ	Название	Описание
‘	Экранирование	Расширяется в (quote); возвращает невычисленную форму
;	Комментарий	Отмечает начало комментария, который продолжается до конца строки (аналог // в Java)
\	Символ	Создает литеральный символ; например, \n — новая строка
@	Разыменование	Расширяется в (deref); принимает переменную и возвращает ее значение (действие, противоположное форме (var)); имеет дополнительный смысл в контексте транзакционной памяти (см. главу 15)

https://t.me/javalib

Символ	Название	Описание
^	Метаданные	Присоединяет к объекту отображение с метаданными; за подробностями обращайтесь к документации Clojure
`	Синтаксическое экранирование	Форма экранирования, часто используемая в определении макросов; за подробностями обращайтесь к разделу о макросах
#	Диспетчеризация	Имеет несколько разных подформ; см. табл. 10.3

Макрос диспетчеризации принимает несколько разных подформ в зависимости от того, что следует за символом `#`, — они представлены в табл. 10.3.

Таблица 10.3. Подформы макроса диспетчеризации

Форма диспетчеризации	Назначение
#'	Расширяется в `(var)`
#{}	Создает литерал множества (см. раздел 10.2.2)
#()	Создает литерал анонимной функции; полезно для одноразового использования, когда `(fn)` кажется слишком громоздким
#_	Пропускает следующую форму; позволяет создавать многострочные комментарии в виде #_(… комментарий …)
#"<шаблон>"	Создает литерал регулярного выражения (по аналогии с объектом `java.util.regex.Pattern`)

И еще пара замечаний по поводу форм диспетчеризации. Форма `#'`, приведенная ниже, объясняет, почему REPL так ведет себя после `(def)`:

```
user=> (def someSymbol)

#'user/someSymbol
```

Форма `(def)` возвращает вновь созданный объект переменной с именем `someSymbol`, который находится в текущем пространстве имен (`user` в REPL), так что `#'user/someSymbol` — это полное значение того, что возвращается из `(def)`.

Литерал анонимной функции `#()` также поддерживает принципиально новый механизм, который делает код компактнее: он опускает вектор аргументов, а вместо него использует специальный синтаксис, чтобы считыватель Clojure мог определить, сколько аргументов потребуется для литерала функции. Синтаксис выглядит как `%N`, где `N` — количество аргументов.

Вернемся к более раннему примеру и посмотрим, как задействовать в нем анонимные функции. Вспомните форму `(list-maker-fun)`, которая принимает два аргумента (список и функцию) и создает новый список, применяя функцию к каждому элементу исходного списка по очереди:

https://t.me/javalib

```
(defn list-maker-fun [x f]
  (map (fn [z] (let [w z]
    (list w (f w))
  )) x))
```

Вместо того чтобы возиться с определением отдельного символического имени, можно вызвать эту функцию с помощью встроенной функции:

```
user=> (list-maker-fun ["a" "b"] (fn [x] x))
(("a" "a") ("b" "b"))
```

Но можно пойти еще дальше и использовать компактный синтаксис `#()`:

```
user=> (list-maker-fun ["a" "b"] #(do %1))
(("a" "a") ("b" "b"))
```

Пример выглядит немного непривычно из-за формы `(do)`, которая встречалась в таблице базовых специальных форм, но он работает. А теперь упростим саму `(list-maker-fun)` при помощи формы `#()`:

```
(defn list-maker-fun [x f]
  (map #(list %1 (f %1)) x))
```

Преобразование Шварца также служит отличным примером того, как этот синтаксис можно использовать в более сложном коде.

Листинг 10.4. Новая реализация преобразования Шварца

```
(defn schwartz [x key-fn]
  (map #(nth %1 0)
    (sort-by #(nth %1 1)
      (map #(let [w %1]
        (list w (key-fn w))
      ) x))))
```

Литералы анонимных функций, соответствующие трем фазам алгоритма

Если использовать `%1` в качестве заменителя для аргумента литерала функции (а также `%2`, `%3` и т. д. для последующих аргументов), код становится значительно проще читать. Подобные визуальные ориентиры существенно помогают программисту — как, например, «стрелка» в лямбда-выражениях языка Java.

Как вы уже видели, Clojure существенно опирается на концепцию функций как основных вычислительных единиц (вместо объектов, которые доминируют в языках вроде Java). Естественная среда для такой концепции — функциональное программирование, которому посвящен следующий раздел.

10.3. ФУНКЦИОНАЛЬНОЕ ПРОГРАММИРОВАНИЕ И ЗАМЫКАНИЯ

Мы собираемся вступить в жуткий мир функционального программирования на Clojure. Хотя, с одной стороны, не такой уж он жуткий, а с другой стороны, мы в него уже вступили. Собственно, на протяжении всей этой главы мы занимались

https://t.me/javalib

функциональным программированием, просто мы этого не акцентировали, чтобы вас не отпугнуть.

Как упоминалось в разделе 8.1.3, концепция функционального программирования выглядит немного расплывчато: ее единственная фундаментальная особенность состоит в том, что с функциями можно работать как со значениями. Функции можно передавать, сохранять в переменных и подвергать разным манипуляциям, как значение `2` или `"hello"`. Однако польза от этого пока неочевидна. Мы выполняли эти операции в самом первом примере: `(def hello (fn [] "Hello world"))`. Мы создали функцию, которая не принимает аргументы и возвращает строку `"Hello world"`, и связали ее с символическим именем `hello`. Эта функция была обычным значением, которое принципиально не отличалось, например, от значения 2.

В листинге 10.3 мы представили преобразование Шварца как пример функции, которая принимает на вход другую функцию. И снова это всего лишь функция, принимающая конкретный тип в одном из своих входных аргументов. Она слегка выделяется лишь тем, что этот тип тоже является функцией.

Вероятно, сейчас также будет уместно представить форму `(filter)`, которая напоминает одноименный метод из Java Streams:

```
user=> (defn gt4 [x] (> x 4))
#'user/gt4
user=> (filter gt4 [1 2 3 4 5 6])
(5 6)
```

Чтобы получить полный набор операций «фильтрация — отображение — свертка», стоит упомянуть также форму `(reduce)`. Чаще всего она встречается в двух разновидностях: первая получает исходное начальное значение, а вторая нет:

```
user=> (reduce + 1 [2 3 4 5])
15
user=> (reduce + [1 2 3 4 5])
15
```

Как насчет замыканий? Ведь это что-то страшное, верно? Опять-таки нет. Посмотрим на простой код, который может напомнить вам некоторые примеры на Kotlin из главы 9:

```
user=> (defn adder [constToAdd] #(+ constToAdd %1))
#'user/adder

user=> (def plus2 (adder 2))
#'user/plus2

user=> (plus2 3)
5

user=> 1:9 user=> (plus2 5)
7
```

Здесь сначала объявляется функция с именем `(adder)`, которая создает другие функции. Если вы знакомы с паттерном «Фабричный метод» (Factory Method)

https://t.me/javalib

в Java, ее можно рассматривать как своего рода аналог этого паттерна в Clojure. Нет ничего странного в функциях, которые возвращают другие функции, — это ключевая часть парадигмы, в соответствии с которой функции являются обычными значениями.

Обратите внимание, что в этом примере используется сокращенная форма `#()` для литерала анонимной функции. Функция `(adder)` принимает число и возвращает функцию, а эта возвращенная функция принимает один аргумент.

Затем с помощью `(adder)` определяется новая форма `(plus2)`. Это функция, которая принимает один числовой аргумент и прибавляет к нему 2. Значение, которое было связано с `constToAdd` внутри `(adder)`, было равно 2. Теперь создадим новую функцию:

```
user=> (def plus3 (adder 3))
#'user/plus3

user=> (plus3 4)
7

user=> (plus2 4)
6
```

Этот пример показывает, что можно создать другую функцию `(plus3)`, которая связывает с `constToAdd` другое значение. Мы говорим, что функции `(plus3)` и `(plus2)` *замыкаются* по значению из своего окружения. Обратите внимание: значения, которые были замкнуты функциями `(plus3)` и `(plus2)`, различаются, а определение `(plus3)` никак не отражается на значении, замкнутом `(plus2)`.

Функции, которые замыкаются по некоторым значениям из своего окружения, называются *замыканиями* (closures); `(plus2)` и `(plus3)` — примеры замыканий. Схема, при которой функция возвращает другую, более простую функцию, замкнутую по чему-либо, очень часто встречается в языках, которые поддерживают замыкания.

> **ПРИМЕЧАНИЕ** Помните, что, хотя Clojure откомпилирует любую синтаксически действительную форму, программа возбудит исключение времени выполнения, если пытаться вызвать функцию с неправильным количеством аргументов. Функцию с двумя аргументами нельзя использовать там, где ожидается функция с одним аргументом.

Нам еще предстоит много рассказать о функциональном программировании в контексте в главе 15. А сейчас займемся одной из выдающихся возможностей Clojure — последовательностями.

10.4. ЗНАКОМСТВО С ПОСЛЕДОВАТЕЛЬНОСТЯМИ CLOJURE

В Clojure есть мощная абстракция — *последовательность* (sequence), которая иногда обозначается сокращением *seq*.

https://t.me/javalib

ПРИМЕЧАНИЕ Последовательности относятся к важным конструкциям Clojure, которые особенно выразительно раскрывают потенциал языка. Последовательности своеобразно контрастируют с тем, как похожие концепции устроены в Java.

Тип последовательности в Clojure приблизительно соответствует коллекциям и итераторам в Java, хотя последовательности обладают несколько иными свойствами; по сути, они объединяют некоторые особенности обоих типов Java в одну концепцию. В их основе лежит идея о том, что тип должен обладать тремя характеристиками:

- Неизменяемость — чтобы последовательности можно было передавать между функциями (и потоками) без каких-либо проблем.
- Более надежная абстракция в стиле итераторов, особенно для многопроходных алгоритмов.
- Возможность использовать *отложенные* (lazy) *последовательности* (об этом позднее).

Из этих трех характеристик больше всего проблем у программистов на Java обычно вызывает неизменяемость. Итераторы в Java изменяемы по своей природе — отчасти из-за того, что они не предоставляют четко отделяемого интерфейса. Собственно, `Iterator` в Java нарушает принцип единственной обязанности, потому что `next()` при вызове выполняет две логически изолированные задачи:

- Возвращает элемент, на который в настоящий момент указывает итератор.
- Изменяет итератор, перемещая указатель к следующему элементу.

Последовательность Clojure опирается на функциональную парадигму, и чтобы избежать изменяемости, она по-другому разделяет функциональность `hasNext()` и `next()`. Создадим слегка упрощенную версию еще одного из важнейших интерфейсов Clojure, а именно `clojure.lang.ISeq`:

```
interface ISeq {
  Object first();
  ISeq rest();
}
```

Возвращает первый объект последовательности

Возвращает новую последовательность, которая содержит все элементы старой последовательности, кроме первого

Теперь последовательность не будет изменяться. Вместо этого при каждом вызове `rest()` то есть там, где мы перемещали бы итератор к следующему значению, создается новое значение-последовательность. Посмотрим, как можно было бы реализовать эту функциональность на Java:

```
public class ArraySeq implements ISeq {
    private final int index;
    private final Object[] values;

    private ArraySeq(int index, Object[] values) {
        this.index = index;
```

Конечные поля

https://t.me/javalib

```
        this.values = values;
    }

    public static ArraySeq of(List<Object> objects) {
        if (objects == null || objects.size() == 0) {
            return Empty.of();
        }
        return new ArraySeq(0, objects.toArray());
    }

    @Override
    public Object first() {
        return values[index];
    }

    @Override
    public ISeq rest() {
        if (index >= values.length - 1) {
            return Empty.of();
        }
        return new ArraySeq(index + 1, values);
    }

    public int count() {
        return values.length - index;
    }
}
```

Фабричный метод, который принимает список

Требует также пустой реализации

Также нам понадобится специальная последовательность, которая представляла бы конец последовательности. Для этого удобно воспользоваться внутренним классом `ArraySeq`:

```
public static class Empty extends ArraySeq {
    private static Empty EMPTY = new Empty(-1, null);

    private Empty(int index, Object[] values) {
        super(index, values);
    }

    public static Empty of() {
        return EMPTY;
    }

    @Override
    public Object first() {
        return null;
    }

    @Override
    public ISeq rest() {
        return of();
    }

    public int count() {
```

https://t.me/javalib

```
        return 0;
    }
}
```

Посмотрим, как это работает:

```
ISeq seq = ArraySeq.of(List.of(10000,20000,30000));
var o1 = seq.first();
var o2 = seq.first();
System.out.println(o1 == o2);
```

Как и ожидалось, вызовы `first()` *идемпотентны*: они не изменяют последовательность и каждый раз возвращают одно и то же значение.

А вот как мы бы написали цикл на Java с помощью `ISeq`:

```
while (seq.first() != null) {
    System.out.println(seq.first());
    seq = seq.rest();
}
```

Этот пример показывает, как бороться с возражением, которое иногда возникает у Java-программистов по поводу неизменяемых последовательностей: «А как насчет мусора?»

Действительно, каждый вызов `rest()` создает новую последовательность, которая является объектом. Тем не менее, если внимательно присмотреться к коду реализации, вы увидите, что мы тщательно избегаем того, чтобы дублировать `values` — хранилище массива. Копировать было бы слишком затратно, поэтому мы этого не делаем.

В действительности на каждом шаге создается крошечный объект, который содержит целое число и ссылку на объект. Если нигде не сохранять эти временные значения, то по мере того, как мы перемещаемся по последовательности, они выйдут из области видимости и очень быстро станут кандидатами для сборки мусора.

> **ПРИМЕЧАНИЕ** Тела методов для Empty не обращаются ни к `index`, ни к `values`. Это позволяет использовать специальные значения (−1 и `null`), до которых нельзя добраться ни из какого экземпляра `ArraySeq`, — это полезно при отладке.

После того как мы изложили элементы теории последовательностей на Java, пора вернуться к Clojure.

> **ПРИМЕЧАНИЕ** Реальный интерфейс `ISeq`, который реализуют все последовательности Clojure, чуть сложнее, чем наши примеры, но основные принципы остаются такими же.

В табл. 10.4 перечислены некоторые базовые функции, относящиеся к работе с последовательностями. Обратите внимание, что ни одна из этих функций не изменяет свои входные аргументы; если функция должна вернуть другое значение, это будет другая последовательность.

https://t.me/javalib

Таблица 10.4. Основные функции последовательностей

Функция	Эффект
(seq <коллекция>)	Возвращает последовательность, которая ведет себя как «представление» данной коллекции
(first <коллекция>)	Возвращает первый элемент коллекции, сначала вызывая для нее (seq) при необходимости; возвращает nil, если коллекция представляет собой nil
(rest <коллекция>)	Возвращает новую последовательность, которая получается из коллекции без первого элемента
(seq? <объект>)	Возвращает true, если объект является последовательностью (то есть реализует ISeq)
(cons <элемент> <коллекция>)	Возвращает последовательность, которая получается из коллекции, если добавить элемент в ее начало
(conj <коллекция> <элемент>)	Возвращает новую коллекцию с новым элементом, который добавляется с соответствующего края: для векторов — с конца, для списков — с начала
(every? <функция-предикат> <коллекция>)	Возвращает true, если (функция-предикат) возвращает логическую истину для каждого элемента в коллекции

Clojure отличается от других разновидностей Lisp, потому что (cons) требует, чтобы вторым аргументом была коллекция (или, вернее, ISeq). В общем случае многие программисты на Clojure отдают предпочтение (conj) перед (cons). Несколько примеров:

```
user=> (rest '(1 2 3))
(2 3)

user=> (first '(1 2 3))
1

user=> (rest [1 2 3])
(2 3)

user=> (seq ())
nil

user=> (seq [])
nil

user=> (cons 1 [2 3])
(1 2 3)

user=> (every? is-prime [2 3 5 7 11])
true
```

Один важный момент: списки Clojure являются последовательностями, а векторы нет. Теоретически (rest) не должен выполняться с векторами. Здесь эта

https://t.me/javalib

функция работает потому, что (rest) сначала вызывает (seq) для вектора, а потом оперирует полученной последовательностью.

ПРИМЕЧАНИЕ Многие функции последовательностей принимают более общие объекты, чем последовательности, и вызывают для них (seq) перед началом работы.

В следующем разделе мы рассмотрим некоторые базовые свойства и примеры использования абстракции последовательности, уделяя особое внимание вариадическим функциям. Позднее, в главе 15, будут представлены отложенные последовательности — важное средство функционального программирования.

10.4.1. Последовательности и функции переменной арности

Рассмотрим одну из самых выдающихся особенностей работы с функциями в Clojure, которую мы решили отложить до этого момента. Речь идет о функциях, которые естественным образом способны принимать переменное количество аргументов (которое иногда называется *арностью* функции). Такие функции называются *вариадическими* и часто применяются при работе с последовательностями.

ПРИМЕЧАНИЕ Java тоже поддерживает вариадические методы: для этого тип последнего параметра метода снабжается пометкой ..., которая означает, что в конце списка параметров допускается любое количество параметров этого типа.

В качестве тривиального примера рассмотрим константную функцию (const-fun1) из листинга 10.1. Она получает один аргумент и отбрасывает его, всегда возвращая значение 1. Но что произойдет, если передать (const-fun1) более одного аргумента?

```
user=> (const-fun1 2 3)
java.lang.IllegalArgumentException:
  Wrong number of args (2) passed to: user$const-fun1 (repl-1:32)[1]
```

Компилятор Clojure не может на стадии компиляции обеспечить статическую проверку количества (и типов) аргументов, которые передаются (const-fun1), и возникает риск исключений.

Такое ограничение выглядит слишком жестким, особенно для функции, которая просто отбрасывает все аргументы и возвращает константу. Как в Clojure создать функцию, которая может принимать произвольное количество аргументов?

Следующий листинг показывает, как сделать такую функцию из (const-fun1). Мы назвали ее (const-fun-arity1) — «константная функция 1 с переменной арностью».

ПРИМЕЧАНИЕ Фактически это кустарная версия функции (constantly) из стандартной библиотеки Closure.

[1] Недопустимое количество аргументов (2) передано функции: user$const-fun1 (repl-1:32). — *Примеч. пер.*

https://t.me/javalib

Листинг 10.5. Функция с переменной арностью

```
user=> (defn const-fun-arity1
  ([] 1)                    ; Определения нескольких функций
  ([x] 1)                   ; с разными сигнатурами
  ([x & more] 1)
)
#'user/const-fun-arity1

user=> (const-fun-arity1)
1

user=> (const-fun-arity1 2)
1

user=> (const-fun-arity1 2 3 4)
1
```

Принципиально здесь то, что за определением функции не следует вектор параметров и форма, которая задает поведение функции. Вместо этого задается список пар, где в каждую пару входит вектор параметров (который фактически является сигнатурой этой версии функции) и реализация этой версии.

Это в некоторой степени похоже на перегрузку методов в Java, а также на поиск по шаблону из главы 3. Но так как Clojure — язык с динамической типизацией, в нем нет эквивалента для шаблонов типов, так что это сходство не настолько сильное, как кажется.

Обычно в таких ситуациях определяются несколько форм для специальных случаев (с нулем, одним или двумя параметрами) и дополнительная форма, последним параметром которой является последовательность. В листинге 10.5 это форма с вектором параметров `[x & more]`. Знак `&` означает, что определяется вариадическая версия функции.

Последовательности принадлежат к числу мощных нововведений Clojure. Собственно, умение «думать в стиле Clojure» во многом заключается именно в том, чтобы представлять себе, как можно применить абстракцию последовательности к той или иной конкретной задаче. Другое важное новшество Clojure — интеграция Clojure с Java; этой теме посвящен следующий раздел.

10.5. ВЗАИМОДЕЙСТВИЕ МЕЖДУ CLOJURE И JAVA

Язык Clojure проектировался с нуля как язык для JVM, который не пытается полностью скрыть специфику JVM от программиста, и этот подход очевидно проявляется во многих местах. Например, на уровне системы типов списки и векторы Clojure реализуют `List` — стандартный интерфейс из библиотеки коллекций Java. Кроме того, библиотеки Java очень легко использовать из кода на Clojure и наоборот. Эти качества чрезвычайно полезны, потому что программисты на Clojure могут пользоваться всем богатством библиотек и инструментария Java, а также высокой производительностью и другими возможностями JVM.

https://t.me/javalib

В этой главе рассматриваются некоторые аспекты этой интеграции, а именно:

- Вызов Java из Clojure.
- Как тип функций Clojure виден в Java.
- Заместители в Clojure.
- Исследовательское программирование в REPL.
- Вызов Clojure из Java.

Для начала посмотрим, как обращаться к методам Java из Clojure.

10.5.1. Вызов Java из Clojure

Рассмотрим фрагмент кода на Clojure, вычисляемого в REPL:

```
user=> (defn lenStr [y] (.length (.toString y)))
#'user/lenStr

user=> (schwartz ["bab" "aa" "dgfwg" "droopy"] lenStr)
("aa" "bab" "dgfwg" "droopy")
```

В этом фрагменте мы применили преобразование Шварца, чтобы отсортировать вектор строк по длине. Для этого мы использовали формы `(.toString)` и `(.length)` — это методы Java, которые здесь вызываются для объектов Clojure. Точка в начале символического имени означает, что среда выполнения должна вызвать метод с указанным именем для следующего аргумента. Это работает потому, что на внутреннем уровне применяется макрос `(.)`, который вам еще не знаком.

Вспомните, что все значения Clojure, которые определяются формой `(def)` или ее разновидностью, помещаются в экземпляры `clojure.lang.Var`, которые могут содержать любое значение типа `java.lang.Object`, так что для значения Clojure можно вызывать любой метод, который доступен для `java.lang.Object`. Кроме того, вызывать статические методы Java (например, `System.getProperty()`) можно с помощью такой формы:

```
(System/getProperty "java.vm.version")
```

А к статическим открытым переменным (например, константам) можно обращаться так:

```
Boolean/TRUE
```

Вот как может выглядеть реализация знакомого примера «Hello World»:

```
user=> (.println System/out "Hello World")
Hello World
nil
```

Откуда взялось последнее `nil`? Вспомните, что все формы Clojure должны возвращать значение, даже если они вызывают `void`-метод языка Java.

https://t.me/javalib

В этих трех примерах мы неявно использовали концепцию пространства имен Clojure, которая похожа на систему пакетов Java и предусматривает отображения сокращенных форм на имена пакетов Java для часто встречающихся случаев (как в последних примерах).

10.5.2. Как устроены вызовы Clojure

Вызов функции в Clojure компилируется в вызов метода JVM, а JVM не гарантирует оптимизацию хвостовой рекурсии, которая обычно применяется в семействе Lisp (особенно в реализациях Scheme). Некоторые диалекты Lisp на JVM исходят из того, что им нужна полноценная хвостовая рекурсия, поэтому они готовы к тому, что функция Lisp не во всех случаях будет вызываться точно так же, как метод Java. Однако Clojure полностью интегрируется с платформой JVM, даже если за это приходится расплачиваться отклонением от обычной практики Lisp.

Если вы хотите создать новый экземпляр объекта Java и работать с ним в Clojure, это легко делается при помощи формы `(new)`. У нее есть альтернативная короткая форма — имя класса, за которым следует точка, что сводится к очередному применению макроса `(.)`:

```
(import '(java.util.concurrent CountDownLatch LinkedBlockingQueue))

(def cdl (new CountDownLatch 2))

(def lbq (LinkedBlockingQueue.))
```

Также здесь используется форма `(import)`, которая позволяет импортировать сразу несколько классов Java из одного пакета в одной строке.

Мы уже упоминали о том, что между системами типов Clojure и Java есть некоторое соответствие. Рассмотрим его подробнее.

10.5.3. Тип Java для значений Clojure

В REPL очень легко просмотреть типы Java для некоторых значений Clojure:

```
user=> (.getClass "foo")
java.lang.String

user=> (.getClass 2.3)
java.lang.Double

user=> (.getClass [1 2 3])
clojure.lang.PersistentVector

user=> (.getClass '(1 2 3))
clojure.lang.PersistentList

user=> (.getClass (fn [] "Hello world!"))
user$eval110$fn__111
```

https://t.me/javalib

Первое, на что стоит обратить внимание, — все значения Clojure являются объектами; примитивные типы JVM недоступны по умолчанию. (Хотя если вы чрезвычайно озабочены производительностью, можно получить доступ и к примитивным типам.) Как и следовало ожидать, строки и числовые значения напрямую отображаются на соответствующие ссылочные типы Java — `java.lang.String`, `java.lang.Double` и т. д.

У анонимной функции «Hello world!» есть имя, которое сообщает, что она является экземпляром динамически сгенерированного класса. Этот класс реализует важнейший интерфейс `IFn`, с помощью которого Clojure показывает, что значение является функцией.

Как упоминалось ранее, последовательности реализуют интерфейс `ISeq`. Обычно они являются одним из конкретных подклассов абстрактного класса `ASeq` или отложенной реализацией `LazySeq` (мы рассмотрим ее в главе 15, когда речь пойдет о расширенном функциональном программировании).

Мы поговорили о типах разных значений, но как насчет памяти для этих значений? Как упоминалось в начале главы, `(def)` связывает символическое имя со значением и тем самым создает переменную, а переменные являются объектами типа `clojure.lang.Var` (который реализует `IFn` среди других интерфейсов).

10.5.4. Заместители в Clojure

В Clojure есть мощный макрос `(proxy)`: он позволяет создать полноценный объект Clojure, который расширяет класс Java (или реализует интерфейс). Например, в следующем листинге мы возвращаемся к одному из предыдущих примеров (с использованием `ScheduledThreadPoolExecutor` из главы 6), но вся основная работа теперь выполняется в малой части кода благодаря компактному синтаксису Clojure.

Листинг 10.6. Новая реализация с планируемыми исполнителями

```
(import '(java.util.concurrent Executors LinkedBlockingQueue TimeUnit))

(def stpe (Executors/newScheduledThreadPool 2))

(def lbq (LinkedBlockingQueue.))

(def msgRdr (proxy [Runnable] []
  (run [] (.println System/out (.toString (.poll lbq))))
))

(def rdrHndl
  (.scheduleAtFixedRate stpe msgRdr 10 10 TimeUnit/MILLISECONDS))
```

Фабричный метод для создания исполнителя

Определяет анонимную реализацию Runnable

Общая форма `(proxy)` выглядит так:

```
(proxy [<надкласс/интерфейсы>] [<аргументы>] <реализации>+)
```

https://t.me/javalib

Первый аргумент-вектор содержит интерфейсы, которые должен реализовать класс-заместитель. Если заместитель также расширяет класс Java (естественно, он может расширять только один класс), то имя этого класса должно быть первым элементом вектора.

Второй аргумент-вектор содержит параметры, которые передаются конструктору надкласса. Часто это пустой вектор, и он, безусловно, будет пустым во всех случаях, когда форма `(proxy)` просто реализует интерфейсы Java.

После этих двух аргументов следуют формы, которые представляют реализации отдельных методов, как того требуют указанные интерфейсы или надклассы. В нашем примере заместитель должен реализовать только `Runnable`, поэтому это единственное символическое имя в первом векторе аргументов. Никакие параметры надкласса не нужны, так что второй вектор пуст (как это часто бывает).

После двух векторов идет список форм, определяющих методы, которые будет реализовывать заместитель. В нашем случае это только метод `run()`, для которого задается определение `(run [] (.println System/out (.toString (.poll lbq))))`. Конечно, эта запись на Clojure — всего лишь эквивалент такого фрагмента Java:

```
public void run() {
    System.out.println(lbq.poll().toString());
}
```

Форма `(proxy)` позволяет простым способом реализовать любой интерфейс Java. Это приводит нас к интересной возможности — использовать REPL для Clojure в качестве площадки для экспериментов с кодом Java и JVM.

10.5.5. Исследовательское программирование в REPL

Основная идея исследовательского программирования заключается в том, что благодаря живому интерактивному окружению, которое предоставляет REPL, и менее объемному коду (вследствие компактного синтаксиса Clojure) REPL может стать отличной средой для того, чтобы не только изучать программирование на Clojure, но и осваивать библиотеки Java.

Возьмем реализации списков в Java. В них присутствует метод `iterator()`, который возвращает объект типа `Iterator`. Но `Iterator` — это интерфейс, так что вам, возможно, будет интересно узнать, какой тип его фактически реализует. При помощи REPL это легко выяснить:

```
user=> (import '(java.util ArrayList LinkedList))
java.util.LinkedList

user=> (.getClass (.iterator (ArrayList.)))
java.util.ArrayList$Itr

user=> (.getClass (.iterator (LinkedList.)))
java.util.LinkedList$ListItr
```

https://t.me/javalib

Здесь форма `(import)` подключает два разных класса из пакета `java.util`. Затем мы используем метод Java `getClass()` из REPL точно так же, как это делалось в разделе 10.5.3. Как видите, итераторы на самом деле предоставляются внутренними классами. Наверное, вас это не удивит: как обсуждалось в разделе 10.4, итераторы тесно связаны с коллекциями, к которым они относятся, поэтому им иногда нужно видеть подробности внутренней реализации этих коллекций.

Обратите внимание, что в этом примере мы не использовали ни одной конструкции Clojure — только немного синтаксиса. Все сущности, с которыми мы оперировали, были полноценными конструкциями Java. Однако предположим, что вы хотите зайти с другой стороны и воспользоваться мощными абстракциями, которые Clojure привносит в программы Java. В следующем подразделе мы увидим, как это делается.

10.5.6. Использование Clojure из Java

Вспомните, что система типов Clojure тесно согласована с системой типов Java. Все структуры данных Clojure — это полноценные коллекции Java, которые реализуют всю обязательную часть интерфейсов Java. Необязательные части обычно не реализуются, потому что они зачастую связаны с изменением структур данных, которое Clojure не поддерживает.

Такое согласование систем типов позволяет использовать структуры данных Clojure в программах на Java. Сама природа Clojure упрощает эту возможность: это компилируемый язык, механизм вызова в котором соответствует аналогичному механизму JVM. Тем самым сводится к минимуму влияние среды выполнения, и оказывается, что с классом, полученным из Clojure, можно работать практически так же, как с любым другим классом Java. Интерпретируемые языки интегрируются намного хуже: обычно для их поддержки нужна минимальная среда выполнения другого языка, отличного от Java.

Следующий пример показывает, как использовать конструкцию последовательности Clojure с обычными строками Java. Чтобы этот код работал, файл clojure.jar должен находиться в classpath:

```
ISeq seq = StringSeq.create("foobar");

while (seq != null) {
  Object first = seq.first();
  System.out.println("Seq: "+ seq +" ; first: "+ first);
  seq = seq.next();
}
```

В этом коде используется фабричный метод `create()` класса `StringSeq`. Он позволяет рассматривать серию символов, входящих в строку, как последовательность. Методы `first()` и `next()` возвращают новые значения, а не изменяют существующую последовательность, как обсуждалось в разделе 10.4.

https://t.me/javalib

В следующем разделе мы поговорим о макросах Clojure. Макросы — мощное средство, которое позволяет опытным программистам фактически модифицировать сам язык Clojure. Эта особенность характерна для таких языков, как Lisp, но для Java-программиста она выглядит довольно непривычно, поэтому мы посвятим ей целый раздел.

10.6. МАКРОСЫ

В главе 8 упоминалось, что языку Java свойственна жесткая грамматика. Наоборот, Clojure поддерживает макросы и активно поощряет использовать их, придавая языку больше гибкости. Это позволяет писать более или менее обычный код, который ведет себя почти так же, как встроенный синтаксис языка.

> **ПРИМЕЧАНИЕ** Макросы есть во многих языках (в том числе в C++), и они почти всегда работают примерно одинаково: для них предусмотрена специальная фаза компиляции исходного кода — часто самая первая.

Например, в языке C компиляция начинается с того, что код обрабатывается *препроцессором*, который удаляет комментарии, встраивает подключенные файлы и расширяет макросы, которые представляют собой *директивы препроцессора* (например, `#include` и `#define`).

Но хотя макросы C были чрезвычайно мощными, они также позволяли писать код с неочевидными усложнениями, который было трудно понимать и отлаживать. Чтобы избежать этих затруднений, в языке Java никогда не было системы макросов или препроцессора.

Работа макросов C основана на простейшей замене текста в фазе препроцессора. Макросы Clojure безопаснее, потому что они работают в синтаксисе самого Clojure. Фактически они позволяют программисту создать специальную разновидность функции, которая особым образом вычисляется на стадии компиляции. Макрос преобразовывает исходный код во время компиляции — на так называемой *фазе расширения макросов*.

> **ПРИМЕЧАНИЕ** Мощь макросов во многом объясняется тем, что код на Clojure записывается как действительная структура данных Clojure, а именно как список форм.

Говорят, что Clojure, как и другие диалекты Lisp (а также несколько других языков), — это *гомоиконический* язык, где программы представляются в том же виде, что и данные. В других языках программирования, таких как Java, исходный код записывается в виде строки, и невозможно определить структуру программы, не разбирая эту строку компилятором Java.

Вспомните, что Clojure компилирует исходный код — в отличие от многих диалектов Lisp, которые относятся к интерпретируемым языкам. Когда загружается исходный код на Clojure, он «на лету» компилируется в байт-код

https://t.me/javalib

JVM. Это может создать поверхностное впечатление, будто Clojure — интерпретируемый язык, но под этой поверхностью прячется компилятор Clojure (очень простой).

> **ПРИМЕЧАНИЕ** Форма Clojure представляет собой список, а макрос — это, по сути, функция, которая не вычисляет свои аргументы, а манипулирует ими, чтобы вернуть другой список, который затем компилируется в форму Clojure.

Чтобы продемонстрировать эту возможность, напишем макрос, который по смыслу противоположен `(if)`. В некоторых языках такие конструкции обозначаются ключевым словом `unless`, поэтому в Clojure мы тоже назовем соответствующую форму `(unless)`. Нам нужна форма, которая выглядит как `(if)`, но ведет себя как ее логическая противоположность:

```
user=> (def test-me false)
#'user/test-me

user=> (unless test-me "yes")
"yes"

user=> (def test-me true)
#'user/test-me

user=> (unless test-me "yes")
nil
```

Обратите внимание, что мы не предусматриваем ветвь `else`: во-первых, без нее пример получается проще, а во-вторых, конструкция «unless ... else» выглядит странно. Если проверка `unless` не проходит, в наших примерах форма будет вычисляться в `nil`.

Если попытаться использовать `(defn)`, первый вариант реализации может выглядеть так (спойлер: он не будет нормально работать):

```
user=> (defn unless [p t]
  (if (not p) t))
#'user/unless

user=> (def test-me false)
#'user/test-me

user=> (unless test-me "yes")
"yes"

user=> (def test-me true)
#'user/test-me

user=> (unless test-me "yes")
nil
```

Здесь на первый взгляд все нормально. Но вспомните, что форма `(unless)` должна работать аналогично `(if)`: в частности, она должна вычисляться, только если условие-предикат истинно. Иначе говоря, `(if)` ведет себя так:

https://t.me/javalib

```
user=> (def test-me true)
#'user/test-me

user=> (if test-me (do (println "Проверка прошла") true))
Проверка прошла
true

user=> (def test-me false)
#'user/test-me

user=> (if test-me (do (println "Проверка прошла") true))
nil
```

Но если пытаться так же использовать нашу функцию `(unless)`, то проблема становится очевидной:

```
user=> (def test-me false)
#'user/test-me

user=> (unless test-me (do (println "Проверка прошла") true))
Проверка прошла
true

user=> (def test-me true)
#'user/test-me

user=> (unless test-me (do (println "Проверка прошла") true))
Проверка прошла
nil
```

Независимо от того, был ли предикат истинным или ложным, форма все равно вычисляется, и поскольку в нашем примере это `(println)`, она выводит текст — по этому признаку мы можем убедиться, что выполнилось вычисление. Чтобы решить эту проблему, необходимо обработать переданные формы, *не вычисляя их*. Фактически перед нами слегка измененный пример отложенного вычисления, которое играет важную роль в функциональном программировании и будет подробно рассмотрено в главе 15. Чтобы объявить новый макрос, используется специальная форма `(defmacro)`:

```
(defmacro unless [p t]
  (list 'if (list 'not p) t))
```

Посмотрим, будет ли эта версия работать правильно:

```
user=> (def test-me true)
#'user/test-me

user=> (unless test-me (do (println "Проверка прошла") true))
nil

user=> (def test-me false)
#'user/test-me

user=> (unless test-me (do (println "Проверка прошла") true))
Проверка прошла
true
```

https://t.me/javalib

Теперь все работает так, как надо: форма `(unless)` выглядит и ведет себя так же, как встроенная специальная форма `(if)`.

Как видите, один из недостатков написания макросов заключается в том, что часто приходится использовать экранирование. Макрос преобразует свои аргументы в новую форму Clojure во время компиляции, поэтому вполне естественно, что выводом должен быть список, то есть `(list)`.

Список содержит символические имена Clojure, которые будут вычисляться на стадии выполнения, поэтому все, что не должно явно вычисляться при расширении макроса, надо экранировать. При этом используется тот факт, что макросы получают свои аргументы во время компиляции, поэтому они доступны в виде невычисленных данных.

В нашем примере приходится экранировать все, что *не* является одним из аргументов: строки, которые соответствуют аргументам, заменяются на символические имена во время расширения. Такой синтаксис быстро становится слишком громоздким. Нет ли более эффективного решения?

Рассмотрим инструмент, который задаст верное направление. Когда вы пишете макросы или занимаетесь их отладкой, вам может пригодиться форма `(macroexpand-1)`. Если ей передается форма макроса, она расширяет макрос и возвращает результат расширения. Если переданная форма не макрос, то возвращается сама эта форма, например:

```
user=> (macroexpand-1 '(unless test-me (do (println "Проверка прошла") true)))
(if (not test-me) (do (println "Проверка прошла") true))
```

Что нам действительно нужно, так это возможность писать макросы, которые выглядят как их расширенная форма без чрезмерного экранирования, которое мы наблюдали в предыдущих примерах.

> **ПРИМЕЧАНИЕ** В результате полное расширение макросов с помощью формы `(macroexpand)` конструируется повторным вызовом предыдущей, более простой формы. Если применение `(macroexpand-1)` оказывается пустой операцией, расширение макроса завершается. Эта возможность опирается на специальный макрос считывателя `` ` `` («синтаксическое экранирование»), который упоминался ранее: по сути, он экранирует все в последующей форме. Если вы хотите, чтобы какие-то значения не экранировались, помечайте их оператором `(~)`. В итоге макрос `(unless)` можно записать в следующем виде:
>
> ```
> (defmacro unless [p t]
>   `(if (not ~p) ~t))
> ```

Эта форма намного яснее и ближе к той, которую мы видим при расширении макроса. Символ ~ становится удобным визуальным ориентиром, который показывает, какие символы будут заменены при расширении. Это хорошо соответствует представлению о макросе как о шаблоне кода, который обрабатывается на стадии компиляции.

https://t.me/javalib

Наряду с синтаксическим экранированием и его отменой в определениях макросов иногда используются важные специальные переменные, чаще всего эти две:

- `&form` — активизируемое выражение;
- `&env` — отображение локальных привязок на момент расширения макроса.

В документации по Clojure вы найдете полное описание информации, которую можно получить из каждой специальной переменной.

Также стоит заметить, что при написании макросов на Clojure важна осторожность. Например, макрос может создавать рекурсивные расширения, которые не завершаются, а *расходятся*, как в следующем примере:

```
(defmacro diverge [t]
  `((diverge ~t) (diverge ~t)))
#'user/diverge

user=> (diverge true)
Syntax error (StackOverflowError) compiling at (REPL:1:1)[1].
null
```

Наконец, убедимся в том, что макросы действительно работают на стадии компиляции. Для этого создадим макрос, который фактически действует как замыкание, которое связывает стадию компиляции со стадией выполнения:

```
user=> (defmacro print-msg-with-compile []
  (let [num (System/currentTimeMillis)]
    `(fn [t#] (println t# " " ~num))))
#'user/print-msg-with-compile

user=> (def p1 (print-msg-with-compile))
#'user/p1

user=> (p1 "aaa")
aaa 1603437421852
nil

user=> (p1 "bbb")
bbb 1603437421852
nil
```

Обратите внимание, что форма `(let)` вычисляется во время компиляции, поэтому значение `(System/currentTimeMillis)` регистрируется при вычислении макроса, связывается с символическим именем `num`, а затем заменяется в расширенной форме на значение, которое было связано, — то есть фактически является константой, определенной во время компиляции.

Хотя мы представили макросы в самом конце главы, на самом деле они повсеместно встречаются в Clojure. Собственно, значительная часть стандартной

[1] Синтаксическая ошибка переполнения стека (StackOverflowError) при компиляции по адресу (REPL:1:1). — *Примеч. пер.*

https://t.me/javalib

библиотеки Clojure реализована в виде макросов. Как опытные разработчики, вы сможете многому научиться, если потратите некоторое время, читая исходный код стандартной библиотеки и наблюдая за тем, как написаны ее ключевые части.

На этом месте стоит предупредить: макросы — мощный инструмент, и некоторые разработчики могут поддаться искушению злоупотреблять ими и применять макросы там, где без них можно обойтись. (То же самое происходит и с другими инструментами, которые «прокачивают» мышление программиста.)

Чтобы не оказаться в такой ситуации, мы настоятельно рекомендуем запомнить пару простых правил использования макросов в Clojure:

- Никогда не пишите макрос, если его цели можно достичь с помощью функции.
- Пишите макросы, когда нужно реализовать функциональность, возможность или паттерн, которого еще нет в языке или стандартной библиотеке.

Первое правило — это, по сути, еще одна формулировка известного принципа «можно — еще не значит нужно».

Второе правило напоминает о том, что макросы существуют по определенной причине: они позволяют сделать то, что не удается никаким другим способом. Квалифицированный программист на Clojure чрезвычайно эффективно применяет макросы там, где это уместно.

В Clojure есть и много других интересных тем помимо макросов, например, как в этом языке устроено динамическое поведение на стадии выполнения. В Java для этого обычно служит наследование классов и интерфейсов и виртуальная диспетчеризация, но эти механизмы являются объектно-ориентированными по своей сути и не очень подходят для Clojure.

Вместо этого аналогичная гибкость в Clojure достигается благодаря *протоколам* и *типам данных* (наряду с заместителями, с которыми вы уже встречались). Есть и другие возможности, например пользовательские схемы диспетчеризации, основанные на *мультиметодах*. Все эти средства чрезвычайно мощны, но к сожалению, они выходят за рамки нашего вводного курса Clojure.

Пожалуй, из всех языков, которые мы рассматриваем в книге, Clojure больше всех отличается от Java. Благодаря происхождению от Lisp, особому акценту на неизменяемости и собственной философии он оказывается совершенно отдельным языком. Но плотная интеграция с JVM, согласованная с JVM система типов (даже когда Clojure предоставляет альтернативы, такие как последовательности) и возможности исследовательского программирования приводят к тому, что Clojure отлично дополняет Java.

Различия между языками, которые мы изучали в этом разделе, наглядно демонстрируют, что платформа Java развивается и остается привлекательной средой для разработки приложений, а виртуальная машина JVM отличается гибкостью и обладает широкими возможностями.

https://t.me/javalib

ИТОГИ

- В Clojure используется динамическая типизация, и Java-программистам не следует забывать об исключениях времени выполнения.
- Стиль исследовательской разработки на базе REPL заметно отличается от того, к чему многие привыкли в традиционных IDE для Java.
- Clojure поддерживает и поощряет программирование с неизменяемыми сущностями.
- Функциональное программирование пронизывает Clojure насквозь — в значительно большей степени, чем Java или Kotlin.
- Последовательности Clojure — функциональный аналог итераторов и коллекций языка Java.
- Макросы определяют преобразования исходного кода Clojure на стадии компиляции.

https://t.me/javalib

Часть IV

Сборка и развертывание

Эта часть книги полностью посвящена эффективной работе с инструментами сборки, тестирования и развертывания приложений на Java. И хотя средства сборки существовали еще задолго до появления Java, эта область продолжает развиваться. В главе 11 мы рассмотрим две самые популярные системы сборки для Java — Maven и Gradle; вы узнаете об их сходствах и различиях. Мы расскажем не только об основных командах, но и о том, как эти системы моделируют окружающий мир и какие возможности для расширения и настройки эти модели предоставляют опытному программисту.

За прошедшие годы контейнеризация стремительно завоевывает отрасль. Все начиналось с набора примитивов Linux, которые применялись в довольно узком сегменте, но Docker и Kubernetes ввели эти технологии в массовую практику. Мы рассмотрим специфику интеграции сред Java-приложений с контейнерами и разберемся, что требует изменений, а что должно оставаться как раньше.

Но прежде чем развертывать приложения, их необходимо протестировать. В чем конкретно заключается это тестирование, зависит от потребностей проекта и может варьироваться в широких пределах. Основательные разработчики знают, что универсальных решений здесь не бывает, поэтому мы познакомимся с разными решениями для тестирования со всеми их достоинствами и недостатками, а также представим терминологию, которая позволит точнее говорить о тестах. Вы узнаете о крупных изменениях в недавно вышедшей основной версии JUnit, а заодно освоите многие новые возможности JDK.

Методы тестирования кода развиваются во многих направлениях. В частности, полезную информацию можно почерпнуть из других технологий и языков помимо Java. В главе 14 вы увидите, как контейнеры, Kotlin и Clojure расширяют инструментарий тестирования. К концу части IV вы будете готовы вывести свое приложение для JVM в реальный мир с полной уверенностью в том, что оно правильно собрано и протестировано.

https://t.me/javalib

11 Сборка в Gradle и Maven

В ЭТОЙ ГЛАВЕ

- ✓ Почему средства сборки важны для опытного разработчика
- ✓ Maven
- ✓ Gradle

Как обсуждалось в главе 4, в поставку JDK входит компилятор, который преобразовывает исходный код на Java в файлы классов. Тем не менее лишь немногие проекты, независимо от размера, обходятся только средствами `javac`. Давайте разберемся, почему любому опытному разработчику стоит владеть инструментарием сборки.

11.1. ПОЧЕМУ СРЕДСТВА СБОРКИ ВАЖНЫ ДЛЯ ОПЫТНОГО РАЗРАБОТЧИКА

Средства сборки стали стандартными инструментами разработки, потому что они:

- Автоматизируют рутинные операции.
- Управляют зависимостями.
- Согласовывают усилия разработчиков.

Существует много систем сборки, но в отрасли сейчас господствуют две: Maven и Gradle. Опытный разработчик должен понимать, какие задачи они решают,

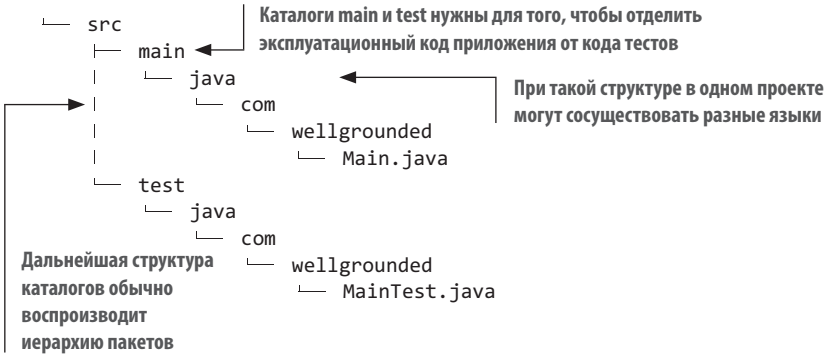
https://t.me/javalib

углубленно разбираться в том, как они устроены, знать, чем они отличаются друг от друга, а также уметь их расширять.

11.1.1. Автоматизация рутинных операций

`javac` может преобразовать любой файл с исходным кодом Java в файл класса, однако сборка типичного Java-проекта этим не ограничивается. Даже правильно перечислять файлы для компилятора в крупном проекте может быть утомительно, если делать это вручную. Средства сборки по умолчанию настроены так, чтобы автоматически искать нужный код, а если вы используете нестандартную структуру, они позволяют легко изменить конфигурацию.

Традиционная иерархия каталогов, которая была популяризирована Maven, а также используется по умолчанию в Gradle, выглядит так:



Как видите, тестирование плотно интегрируется в структуру кода. Язык Java прошел долгий путь с тех времен, когда разработчики спрашивали, действительно ли им нужно писать тесты для своего кода. Системы сборки сыграли ключевую роль в том, чтобы стандартные механизмы тестирования стали повсеместно доступны.

> **ПРИМЕЧАНИЕ** Вероятно, вы уже знаете, как выполнять модульное тестирование кода на Java с помощью JUnit или других библиотек. Другие виды тестирования будут рассматриваться в главе 14.

Хотя в общем случае программа на Java начинает существовать после того, как она откомпилирована в файлы классов, обычно это не конечный этап. К счастью, системы сборки также позволяют упаковывать файлы классов в JAR или другой формат, чтобы программный продукт было удобнее распространять.

11.1.2. Управление зависимостями

На заре истории Java, если вы хотели воспользоваться библиотекой, нужно было где-то найти ее файл JAR, загрузить его и включить в classpath для вашего приложения. Это было сопряжено с проблемами, в частности, из-за того, что не было централизованного авторитетного источника для всех библиотек, поиск

https://t.me/javalib

файлов JAR для редко встречающихся зависимостей иногда превращался в охоту за сокровищами.

Это не устраивало многих разработчиков, поэтому Maven (среди прочих проектов) ввел в экосистему Java репозитории с инструментами, которые могли автоматически находить и устанавливать зависимости. Maven Central до сих пор остается одним из самых популярных реестров для поиска зависимостей Java в интернете. Существуют и другие общедоступные реестры (например, те, которые поддерживает Google или которые размещаются на GitHub), а также частные решения с использованием таких продуктов, как Artifactory.

Загрузка всего кода может занимать много времени, поэтому средства сборки были стандартизованы по некоторым направлениям, чтобы упростить совместное использование артефактов между проектами. Если локальный репозиторий обеспечивает кэширование и другому проекту понадобится та же библиотека, вам не придется загружать ее заново (рис. 11.1). Заодно такой подход экономит дисковое пространство, однако настоящее преимущество состоит в том, что существует единый источник артефактов.



Рис. 11.1. Локальный репозиторий Maven помогает не только искать зависимости в интернете, но и эффективно управлять ими в локальной среде

> **ПРИМЕЧАНИЕ** Возможно, вас интересует, какое место занимают модули в этой картине зависимостей. Модульные библиотеки поставляются в виде файлов JAR с добавлением файла module-info.class, о чем шла речь в главе 2. Модульные файлы JAR можно загрузить из стандартных репозиториев. Реальные различия проявляются тогда, когда вы начинаете компилировать и запускать код с модулями, а не на этапе упаковки и распространения.

Однако реестры не только служат централизованным источником для поиска и загрузки зависимостей, но и помогают управлять *транзитивными*

https://t.me/javalib

зависимостями. В Java эта ситуация обычно встречается, когда используемая библиотека сама зависит от другой библиотеки. Мы уже говорили о транзитивных зависимостях модулей в главе 2, хотя проблема существовала задолго до того, как появились модули Java. На самом деле до появления модулей все было гораздо хуже.

Вспомните, что файлы JAR — это обычные архивированные файлы: в них нет метаданных, которые описывали бы зависимости JAR. Это означает, что зависимости файла JAR представляют собой объединение всех зависимостей всех классов в JAR.

Ситуация усугубляется тем, что формат файлов классов не описывает, какая версия класса нужна, чтобы удовлетворить зависимость; у вас есть только символический дескриптор класса или имя метода, который нужен для компоновки класса (как говорилось в главе 4). Отсюда следуют два обстоятельства:

1. Необходим внешний источник информации о зависимостях.
2. Чем больше проект, тем неизбежно сложнее становится граф транзитивных зависимостей.

На фоне стремительного распространения библиотек и фреймворков с открытым кодом, которыми пользуются разработчики, типичное дерево транзитивных зависимостей в реальном проекте только разрастается.

Потенциальная хорошая новость заключается в том, что ситуация в экосистеме JVM немного лучше, чем, допустим, в JavaScript. У JavaScript нет богатой централизованной библиотеки времени выполнения, которая была бы заведомо всегда доступна, так что многими базовыми функциями приходится управлять как внешними зависимостями. Это создает проблемы, например, когда существует множество несовместимых библиотек, каждая из которых предоставляет свою версию популярной функциональности. Кроме этого, экосистема оказывается настолько хрупкой, что ошибки и злонамеренные атаки оказывают непропорциональный эффект (например, инцидент с left-pad в 2016 году (см. http://mng.bz/5Q64)).

С другой стороны, в Java есть библиотека времени выполнения (JRE) со множеством часто используемых классов, и она доступна в каждой среде Java. Однако потребности реальных приложений выходят за рамки JRE, и у них почти всегда слишком много уровней зависимостей, чтобы ими можно было управлять вручную. Единственное решение — автоматизация.

Конфликты зависимостей

Автоматизация стала настоящим благословением для разработчиков, использующих богатую экосистему открытого исходного кода, но обновление зависимостей не обходится без проблем. Например, на рис. 11.2 изображено дерево зависимостей, которое может привести к неприятностям.

https://t.me/javalib

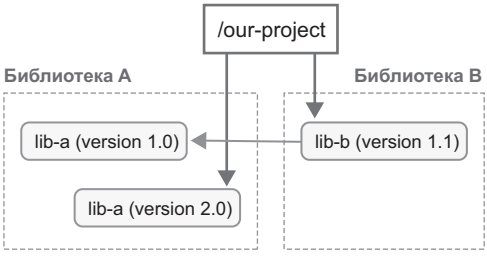


Рис. 11.2. Конфликты транзитивных зависимостей

Мы явно запрашиваем версию 2.0 библиотеки `lib-a`, но зависимость `lib-b` запросила старую версию 1.0 той же библиотеки. Такая ситуация называется *конфликтом зависимостей*, и разные способы ее разрешения могут привести к другим проблемам.

Каких неприятностей можно ожидать из-за расхождений в версиях библиотек? Это зависит от того, что именно изменилось между версиями. Изменения делятся на несколько категорий:

	Тип изменения API	Что меняется между версиями
1	Стабильное	Только поведение
2	Добавление	Появляются новые классы или методы
3	Изменение	Меняются сигнатуры методов или интерфейсы
4	Удаление	Удаляются классы или методы

В случаях 1 и 2 можно даже не заметить, какую версию зависимости выбрала система сборки. В самом распространенном случае 3 между версиями библиотеки изменяется сигнатура метода. В предыдущем примере, если библиотека `lib-a` 2.0 изменила сигнатуру метода, от которого зависит `lib-b`, то при попытке вызвать метод из `lib-b` возбудится исключение `NoSuchMethodError`.

Удаленные методы в случае 4 приводят к тем же исключениям `NoSuchMethodError`. К этой же категории относится «переименование» метода, которое на уровне байт-кода выглядит так, будто мы удалили метод и добавили новый метод, который просто имеет такую же реализацию.

Классы также подвержены проблеме 4 при удалении или переименовании, что приводит к исключению `NoClassDefFoundError`. Еще может оказаться, что если удалить интерфейс из класса, это вызовет неприятное исключение `ClassCastException`.

Этот список проблем, связанных с конфликтами транзитивных зависимостей, ни в коей мере не является исчерпывающим. Все зависит от того, *что* на самом деле изменилось между двумя версиями одного пакета.

Собственно, эта проблема типична для разных языков: как передавать информацию о том, в чем состоят изменения между версиями. Один из

https://t.me/javalib

популярных подходов к решению этой проблемы — *семантическое версионирование* (см. https://semver.org/), которое предоставляет номенклатуру для формулировки требований транзитивных зависимостей, чтобы компьютеры могли разобраться с ними за нас.

В семантическом версионировании используются такие принципы:

- ОСНОВНАЯ версия (1.x → 2.x) увеличивается при критических изменениях в API, как в случаях 3 и 4 выше.
- ДОПОЛНИТЕЛЬНАЯ версия (1.1 → 1.2) увеличивается при дополнениях, которые сохраняют обратную совместимость, как в случае 2.
- Номер ИСПРАВЛЕНИЯ (1.1.0 → 1.1.1) увеличивается при исправлении ошибок.

Хотя такая схема и не абсолютно надежна, она широко применяется в сообществе открытого кода и по крайней мере дает понять, каковы масштабы изменений в том или ином обновлении версии.

Итак, вы получили некоторое представление о том, что управлять зависимостями — не самая простая задача. К счастью, Maven и Gradle предоставляют полезные инструменты в этой области. Позднее в этой главе мы подробно рассмотрим, как каждая система помогает бороться с конфликтами зависимостей.

11.1.3. Согласование усилий разработчиков

Чем сильнее в проекте разрастается объем кода и чем больше разработчиков в нем участвует, тем сложнее обычно становится проект и тем несподручнее с ним работать. Однако вспомогательные инструменты могут снять часть проблем. Например, встроенные средства помогают предусмотреть, что все участники будут компилировать и запускать одни и те же тесты, и это неплохая отправная точка. Тем не менее стоит не только принимать базовые меры, но и задействовать дополнительные полезные возможности.

Тесты — это хорошо, но уверены ли вы, что *весь* ваш код тестируется должным образом? Средства проверки покрытия скажут вам, какой код обеспечен тестами, а какой нет. И хотя в интернете идут бурные дискуссии по поводу того, к какому уровню покрытия следует стремиться, проверка покрытия строк по крайней мере поможет убедиться, что вы не пропустили тест для какой-то специфической ветви условной конструкции.

Язык Java хорошо интегрируется с различными средствами статического анализа. Статический анализ позволяет компилятору проверять различные аспекты вашего кода, которые формально допустимы, но могут преподнести неприятный сюрприз во время реальной эксплуатации: от выявления общих закономерностей (например, если вы переопределяете `equals`, но не переопределяете `hashCode`) до поиска переменных, которые не используются.

https://t.me/javalib

Кроме инструментов, которые проверяют правильность кода, существуют средства контроля стиля и форматирования. Вам доводилось участвовать в спорах о том, где в инструкции должны находиться фигурные скобки? Как расставлять отступы в коде? Если вы один раз согласуете набор правил — пусть даже не все они будут идеальны с вашей точки зрения, — это позволит в дальнейшем сосредоточиться на реальной работе над проектом вместо мелочных споров о том, как должен выглядеть код.

Наконец, средства сборки становятся «точкой схождения» нестандартной функциональности. Нужна ли специальная настройка конфигурации? Нужно ли периодически выполнять какие-либо служебные команды для вашего проекта? Как быть с проверками, которые должны выполняться после сборки, но до развертывания? Все эти задачи уместно встроить в инструменты сборки, чтобы они были доступны всем, кто работает с кодом. Как Maven, так и Gradle предоставляют многочисленные механизмы расширения под вашу логику и потребности.

Надеюсь, мы убедили вас в том, что средства сборки — не простая формальность и что вам стоит потратить время, чтобы разобраться в них. Начнем с самой распространенной системы сборки — Maven.

11.2. MAVEN

В самом начале истории Java стандартной системой сборки был фреймворк Ant. Его задачи кодировались на XML, и он позволял описывать сборку более «Java-центричным» образом, чем инструменты вроде Make. Однако в Ant не хватало структуры конфигурации сборки: из каких шагов она состоит, как они связаны друг с другом, как организуется управление зависимостями. Maven заполняет многие из этих пробелов благодаря концепции стандартизованного *жизненного цикла сборки* и последовательному подходу к работе с зависимостями.

11.2.1. Жизненный цикл сборки

Maven — весьма категоричная система, и эта категоричность особенно наглядно проявляется в жизненных циклах сборки. Вместо того чтобы позволить пользователям определять собственные задачи и порядок их выполнения, Maven предлагает *жизненный цикл по умолчанию*: он состоит из типичных шагов, которые называются *фазами*. Вот самые важные фазы жизненного цикла по умолчанию:

- *Проверка* — проверка, что конфигурация проекта правильная и можно выполнить сборку.
- *Компиляция* — компиляция исходного кода.
- *Тестирование* — запуск модульных тестов.
- *Упаковка* — формирование артефактов (например, файлов JAR).

https://t.me/javalib

- *Верификация* — запуск интеграционных тестов.
- *Установка* — установка пакета в локальном репозитории.
- *Развертывание* — предоставление доступа к пакету другим пользователям, обычно с запуском из среды непрерывной интеграции.

Весьма вероятно, что эта последовательность отражает большинство шагов, которые ваше приложение преодолевает на пути от исходного кода к развернутому продукту или библиотеке. В этом проявляется главное преимущество категоричного подхода Maven: все проекты Maven проходят один и тот же жизненный цикл. Ваши знания о том, как выполнять сборку, становятся более универсальными, чем прежде.

Фазы четко определены в Maven, но у каждого проекта есть специфические особенности. Чтобы их учесть, в модели Maven различные *плагины* присоединяют *цели* к этим фазам. Цель (goal) — это конкретная задача вместе с реализацией ее выполнения.

Кроме жизненного цикла по умолчанию, Maven также определяет жизненный цикл *очистки* (на котором удаляются ненужные артефакты вроде промежуточных результатов сборки) и жизненный цикл *сайта проекта* (на котором генерируется документация).

О том, как можно вмешаться в жизненный цикл, вы узнаете позднее, когда речь пойдет о расширении Maven. А на случай, если вам действительно потребуется сделать все по-своему, Maven позволяет создавать полностью настраиваемые жизненные циклы. Впрочем, это весьма продвинутая тема, которая выходит за рамки книги.

11.2.2. Команды и знакомство с POM

Maven — проект Apache Software Foundation, распространяемый с открытым кодом. Инструкции по установке доступны на сайте проекта по адресу https://maven.apache.org/install.html.

Как правило, Maven устанавливается глобально на рабочей станции и работает на любой не слишком древней JVM (JDK 7 и выше). После установки при запуске вы получите следующий вывод:

```
~: mvn

  [INFO] Scanning for projects...[1]
  [INFO] ------------------------------------------------------------------------
  [INFO] BUILD FAILURE[2]
  [INFO] ------------------------------------------------------------------------
  [INFO] Total time: 0.066 s
  [INFO] Finished at: 2020-07-05T21:28:22+02:00
```

[1] Поиск проектов.... — *Примеч. пер.*

[2] Сборка не удалась. — *Примеч. пер.*

https://t.me/javalib

```
[INFO] ------------------------------------------------------------------------
[ERROR] No goals have been specified for this build. You must specify a
valid lifecycle phase or a goal in the format <plugin-prefix>:<goal> or
<plugin-group-id>:<plugin-artifact-id>[:<plugin-version>]:<goal>.
Available lifecycle phases are: validate, initialize, ....[1]
```

Особый интерес представляет сообщение «No goals have been specified for this build» («Для этой сборки не были заданы цели»). Это означает, что Maven ничего не знает о вашем проекте. Необходимую информацию нужно предоставить в файле pom.xml, на который опирается весь проект Maven.

ПРИМЕЧАНИЕ POM — сокращение от «Project Object Model» («Объектная модель проекта»).

Хотя полноценные файлы pom.xml бывают невероятно длинными и сложными, можно начать с меньшего. Например, относительно минимальный pom.xml выглядит так:

```
<project>
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.wellgrounded</groupId>
  <artifactId>example</artifactId>
  <version>1.0-SNAPSHOT</version>
  <name>example</name>

  <properties>
    <maven.compiler.source>11</maven.compiler.source>
    <maven.compiler.target>11</maven.compiler.target>
  </properties>
</project>
```

Идентифицирует проект

По умолчанию плагины Maven используют Java 1.6. Разумеется, нам нужна более новая версия

В файле pom.xml объявляются два особенно важных поля: `groupId` и `artifactId`. Вместе с версией они образуют *координаты GAV* (Group, Artifact, Version — «группа, артефакт, версия»), которые позволяют глобально и однозначно идентифицировать конкретный выпуск вашего пакета.

`groupId` обычно обозначает компанию, организацию или проект с открытым кодом, который отвечает за библиотеку, тогда как `artifactId` — это имя конкретной библиотеки. В записи координат GAV компоненты часто разделяются двоеточиями: например, `org.apache.commons:collections4:4.4` или `com.google.guava:guava:30.1-jre`.

Эти координаты нужны не только для локальной настройки вашего проекта, но и играют роль адресов для зависимостей, чтобы средства сборки могли их

[1] Для этой сборки не были заданы цели. Требуется указать действительную фазу жизненного цикла или цель в формате: ... Доступные фазы жизненного цикла: проверка, инициализация, ... — *Примеч. пер.*

https://t.me/javalib

успешно найти. В следующих разделах мы подробнее расскажем о том, как технически выражаются эти зависимости.

Maven не только стандартизировал жизненный цикл сборки, но и популяризировал стандартную структуру, которую мы видели в разделе 11.1.1 и повторяем ниже. Если вы будете следовать этим соглашениям, то Maven не потребуется дополнительная информация о вашем проекте, чтобы собрать его:

```
.
├── pom.xml
└── src
    ├── main
    |   └── java
    |       └── com
    |           └── wellgrounded
    |               └── Main.java
    └── test
        └── java
            └── com
                └── wellgrounded
                    └── MainTest.java
```

Обратите внимание на параллельные структуры — src/main/java и src/test/java — с одинаковым отображением каталогов на иерархию пакетов. Эта схема отделяет код тестов от основного кода приложения, чтобы основной код было проще упаковывать для развертывания без кода тестов, который обычно не нужен пользователям пакета.

Существуют и другие стандартные каталоги, кроме этих двух. Например, в каталоге src/main/resources обычно размещаются дополнительные файлы, не относящиеся к коду, которые нужно включить в JAR. Полное описание стандартной структуры Maven доступно в документации по адресу http://mng.bz/6XoG.

Пока вы только осваиваете Maven, имеет смысл придерживаться общепринятых соглашений, стандартных структур и других настроек по умолчанию, которые предлагает Maven. Как упоминалось ранее, это довольно категоричный продукт, поэтому во время обучения лучше оставаться в безопасной зоне. Опытные разработчики Maven часто выходят за границы соглашений и нарушают прочие правила, однако все же не стоит пытаться бегать, пока вы еще не научились ходить.

11.2.3. Сборка

Мы уже видели, что если просто запустить `mvn` из командной строки, программа предупредит вас, что если вы хотите от нее каких-то действий, нужно указать фазу жизненного цикла или цель. Чаще всего вы будете запускать ту или иную фазу, в которую может входить много целей.

https://t.me/javalib

Проще всего начать с компиляции кода, запросив фазу компиляции:

```
~: mvn compile

  [INFO] Scanning for projects...
  [INFO]
  [INFO] -------------------< com.wellgrounded:example >---------------
  [INFO] Building example 1.0-SNAPSHOT
  [INFO] ---------------------------[ jar ]--------------------------
  [INFO]
  [INFO] -- maven-resources-plugin:2.6:resources (default-resources) --
  [INFO] Using 'UTF-8' to copy filtered resources.
  [INFO] Copying 0 resource
  [INFO]
  [INFO] ----- maven-compiler-plugin:3.1:compile (default-compile) ----
  [INFO] Changes detected - recompiling the module!
  [INFO] Compiling 1 source file to ./maven-example/target/classes
  [INFO] ------------------------------------------------------------
  [INFO] BUILD SUCCESS[1]
  [INFO] ------------------------------------------------------------
  [INFO] Total time: 0.940 s
  [INFO] Finished at: 2020-07-05T21:46:25+02:00
  [INFO] ------------------------------------------------------------
```

Хотя в нашем проекте нет ресурсов, плагин maven-resources-plugin из жизненного цикла по умолчанию все равно выполняет эту проверку

Непосредственную компиляцию выполняет плагин maven-compiler-plugin

Maven по умолчанию направляет вывод в каталог target. После выполнения `mvn compile` файлы классов оказываются в каталоге target/classes. Внимательная проверка показывает, что сборке подвергся только код из каталога main. Если вы хотите откомпилировать тесты, используйте фазу `test-compile`.

Жизненный цикл по умолчанию не ограничивается только компиляцией. Например, команда `mvn package` для этого проекта создаст файл JAR target/example-1.0-SNAPSHOT.jar.

И хотя этот JAR можно использовать как библиотеку, при попытке выполнить его командой `java -jar target/example-1.0-SNAPSHOT.jar` выяснится, что Java жалуется на отсутствие основного класса. Чтобы продемонстрировать, как можно наращивать сборку Maven, изменим процесс так, чтобы итоговый файл JAR стал исполняемым приложением.

11.2.4. Управление манифестом

В файле JAR, который создал Maven по команде `mvn package`, не оказалось *манифеста*, который сообщает JVM, где искать метод `main` при запуске. К счастью, Maven поставляется с плагином для построения файлов JAR, который умеет создавать манифесты. Конфигурация плагина определяется в файле pom.xml, — после элемента `properties`, но внутри `project`:

[1] Сборка прошла успешно! — *Примеч. пер.*

https://t.me/javalib

```
<build>
  <plugins>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-jar-plugin</artifactId>
      <version>2.4</version>
      <configuration>
        <archive>
          <manifest>
            <addClasspath>true</addClasspath>
            <mainClass>com.wellgrounded.Main</mainClass>
            <Automatic-Module-Name>
              com.wellgrounded
            </Automatic-Module-Name>
          </manifest>
        </archive>
      </configuration>
    </plugin>
  </plugins>
</build>
```

maven-jar-plugin — имя плагина. Его легко заметить в выводе команды mvn package

У каждого плагина существует свой специализированный элемент configuration с разными дочерними элементами и поддерживаемыми атрибутами

<manifest> настраивает содержимое манифеста итогового файла JAR

Настраивает автоматическое имя модуля

Этот раздел задает главный класс, чтобы система запуска `java` знала, как выполнить JAR напрямую. Мы также добавили автоматическое имя модуля — таковы общепринятые традиции системы модулей. Как упоминалось в главе 2, даже если наш код не модульный (как в данном случае), все равно стоит явно предоставить автоматическое имя модуля, чтобы модульным приложениям было проще использовать наш код.

Настройка конфигурации в элементе `plugin` — вполне стандартная практика в Maven. Для простоты многие плагины по умолчанию любезно предупреждают вас, когда вы пытаетесь использовать неподдерживаемые или непредусмотренные свойства конфигурации, хотя подробности могут зависеть от конкретного плагина.

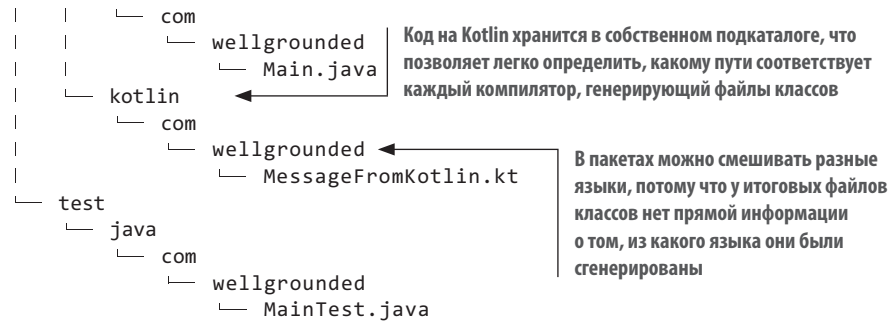
11.2.5. Добавление нового языка

Как упоминалось в главе 8, к достоинствам JVM как платформы относится то, что в пределах одного проекта можно использовать несколько языков. Это может оказаться полезным, если определенный язык лучше подходит для некоторой части приложения, и даже помогает постепенно перевести приложение с одного языка на другой.

Давайте посмотрим, как настроить наш простой проект Maven, чтобы собрать некоторые классы из Kotlin вместо Java. К счастью, стандартная структура уже настроена так, чтобы можно было легко добавлять новые языки:

```
.
├── pom.xml
└── src
    ├── main
    │   ├── java
```

https://t.me/javalib



Maven по умолчанию не знает, как компилировать код на Kotlin (в отличие от Java), поэтому в файл pom.xml необходимо добавить `kotlin-maven-plugin`. За свежей информацией о том, как его использовать, мы рекомендуем обратиться к документации Kotlin по адресу https://kotlinlang.org/docs/maven.html, а здесь просто продемонстрируем эту возможность, чтобы вы знали, чего следует ожидать.

Если проект написан полностью на Kotlin, для компиляции нужно только добавить плагин и присоединить его к цели `compile`:

```
<build>
  <plugins>
    <plugin>
      <groupId>org.jetbrains.kotlin</groupId>
      <artifactId>kotlin-maven-plugin</artifactId>
      <version>1.6.10</version>
      <executions>
        <execution>
          <id>compile</id>
          <goals>
            <goal>compile</goal>
          </goals>
        </execution>
        <execution>
          <id>test-compile</id>
          <goals>
            <goal>test-compile</goal>
          </goals>
        </execution>
      </executions>
    </plugin>
  </plugins>
</build>
```

Текущая версия Kotlin на момент написания книги

Добавляет плагин к целям компиляции основного кода и кода тестов

Ситуация несколько усложняется, если Kotlin и Java используются совместно. Плагин `maven-compiler-plugin`, который применяется по умолчанию и компилирует код на Java, необходимо переопределить, чтобы сначала компилировался код на Kotlin, иначе код на Java не сможет использовать классы Kotlin:

https://t.me/javalib

```
<build>
  <plugins>
    <plugin>
      <groupId>org.jetbrains.kotlin</groupId>
      <artifactId>kotlin-maven-plugin</artifactId>
      <version>1.6.10</version>
      <executions>
        <execution>
          <id>compile</id>
          <goals>
            <goal>compile</goal>
          </goals>
          <configuration>
            <sourceDirs>
              <sourceDir>${project.basedir}/src/main/kotlin</sourceDir>
              <sourceDir>${project.basedir}/src/main/java</sourceDir>
            </sourceDirs>
          </configuration>
        </execution>
        <execution>
          <id>test-compile</id>
          <goals>
            <goal>test-compile</goal>
          </goals>
          <configuration>
            <sourceDirs>
              <sourceDir>${project.basedir}/src/test/kotlin</sourceDir>
              <sourceDir>${project.basedir}/src/test/java</sourceDir>
            </sourceDirs>
          </configuration>
        </execution>
      </executions>
    </plugin>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-compiler-plugin</artifactId>
      <version>3.8.1</version>
      <executions>
        <execution>
          <id>default-compile</id>
          <phase>none</phase>
        </execution>
        <execution>
          <id>default-testCompile</id>
          <phase>none</phase>
        </execution>
        <execution>
          <id>java-compile</id>
          <phase>compile</phase>
          <goals>
            <goal>compile</goal>
          </goals>
        </execution>
        <execution>
          <id>java-test-compile</id>
```

Плагин kotlin-maven-plugin добавляется практически так же, как и прежде, но теперь он знает о путях для Java и Kotlin

Компилятор Kotlin должен знать, где находится код и на Kotlin, и на Java

Блокирует стандартные настройки maven-compiler-plugin для сборки Java, чтобы они не заставляли эту сборку выполняться первой

Повторно применяет maven-compiler-plugin для фаз compile и test-compile. Теперь они добавляются после kotlin-maven-plugin

https://t.me/javalib

```
          <phase>test-compile</phase>
          <goals>
            <goal>testCompile</goal>
          </goals>
        </execution>
      </executions>
    </plugin>
  </plugins>
</build>
```

ПРИМЕЧАНИЕ Эти переопределения могут усложниться, если использовать такие функции Maven, как родительские проекты, в которых могут возникнуть конфликты с дополнительными определениями POM. Скоро мы познакомимся с некоторыми приемами отладки этих проблем.

Проекту понадобится зависимость как минимум от стандартной библиотеки Kotlin, поэтому мы добавим ее явно:

```
<dependencies>
    <dependency>
        <groupId>org.jetbrains.kotlin</groupId>
        <artifactId>kotlin-stdlib</artifactId>
        <version>1.6.10</version>
    </dependency>
</dependencies>
```

После всех этих манипуляций наш многоязычный проект будет собираться и запускаться так же, как прежде.

11.2.6. Тестирование

Если код проходит сборку, логично сделать следующий шаг — протестировать его. Тестирование глубоко интегрировано в жизненный цикл Maven. Собственно, для случаев, когда компиляция вашего основного кода укладывается в одну фазу, Maven «из коробки» поддерживает две отдельные фазы тестирования: `test` и `integration-test`. Фаза `test` используется для типичного модульного тестирования, тогда как `integration-test` выполняется после того, как построены такие артефакты, как файлы JAR, чтобы выполнить сквозную верификацию окончательного результата.

ПРИМЕЧАНИЕ Интеграционные тесты также можно выполнять с JUnit, потому что, несмотря на название, JUnit обладает богатыми возможностями тестирования — не только модульного. Не стоит полагать, будто каждый тест, который выполняет JUnit, автоматически становится модульным! Разные виды тестов подробнее рассматриваются в главе 13.

Почти каждому проекту необходима та или иная форма тестирования. Как можно предположить по категоричной философии Maven, тестирование выполняется (по умолчанию) с помощью почти повсеместно распространенного фреймворка JUnit. Впрочем, существуют плагины и для других фреймворков.

https://t.me/javalib

И хотя стандартные плагины знают о том, как запускать JUnit, библиотеку все равно нужно объявить как зависимость, чтобы Maven понимал, как компилировать тесты. Чтобы добавить библиотеку, включите фрагмент следующего вида в элемент `<project>`:

```
<dependencies>
  <dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter-api</artifactId>
    <version>5.8.1</version>
    <scope>test</scope>
  </dependency>
  <dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter-engine</artifactId>
    <version>5.8.1</version>
    <scope>test</scope>
  </dependency>
</dependencies>
```

<scope> указывает, что библиотека нужна только для фазы test-compile

Когда это будет сделано, можно попытаться запустить модульные тесты. В зависимости от версии Maven даже в самых новых версиях вы можете увидеть странный результат:

```
~:mvn test

  [INFO] Scanning for projects...
  [INFO]
  [INFO] -------------------< com.wellgrounded:example >----------------
  [INFO] Building example 1.0-SNAPSHOT
  [INFO] --------------------------------[ jar ]-------------------------
  [INFO]
  [INFO] .....
  [INFO]
  [INFO] -- maven-surefire-plugin:2.12.4:test (default-test) @ example -
  [INFO] Surefire report dir: ./target/surefire-reports
  -------------------------------------------------------
    TESTS
  -------------------------------------------------------
  Running com.wellgrounded.MainTest
  Tests run: 0, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.001 sec

  Results :

  Tests run: 0, Failures: 0, Errors: 0, Skipped: 0
  [INFO] ------------------------------------------------
  [INFO] BUILD SUCCESS
  [INFO] ------------------------------------------------
  [INFO] Total time: 5.605 s
  [INFO] Finished at: 2021-11-29T09:41:06+01:00
  [INFO] ------------------------------------------------
```

По умолчанию Maven запускает тесты JUnit плагином maven-surefire-plugin

Ни один тест не запускался? Что-то здесь не так!

По соображениям совместимости плагин `maven-surefire-plugin`, который устанавливается по умолчанию, — даже в версии Maven 3.8.4 — не знает

https://t.me/javalib

о существовании JUnit 5. Мы подробнее рассмотрим эти проблемы преобразования в главе 13, а пока просто просто повысим версию плагина до более новой:

```
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-surefire-plugin</artifactId>
  <version>3.0.0-M5</version>
</plugin>
```

После перехода к версии выше чем 2.12 плагины поддерживают JUnit 5 напрямую

После этого результат становится более обнадеживающим:

```
~:mvn test

  [INFO] .....
  -------------------------------------------------------
    TESTS
  -------------------------------------------------------
  Running com.wellgrounded.MainTest
  Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.04 sec

  Results :

  Tests run: 1, Failures: 0, Errors: 0, Skipped: 0

  [INFO] ------------------------------------------------
  [INFO] BUILD SUCCESS
  [INFO] ------------------------------------------------
  [INFO] Total time: 1.010 s
  [INFO] Finished at: 2020-07-06T15:45:22+02:00
  [INFO] ------------------------------------------------------------
```

По умолчанию на фазе `test` плагин Surefire запускает все модульные тесты из стандартного расположения src/test/*. Если вы хотите воспользоваться фазой `integration-test`, рекомендуется применять отдельный плагин, например `maven-failsafe-plugin`. Его сопровождают те же разработчики, которые создали `maven-surefire-plugin`, и он предназначен специально для интеграционного тестирования. Добавим плагин в раздел `<build><plugins>`, в котором мы ранее настраивали манифест:

```
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-failsafe-plugin</artifactId>
  <version>3.0.0-M5</version>
  <executions>
    <execution>
      <goals>
        <goal>integration-test</goal>
        <goal>verify</goal>
      </goals>
    </execution>
  </executions>
</plugin>
```

https://t.me/javalib

`maven-failsafe-plugin` считает интеграционными тестами файлы, имена которых соответствуют следующим шаблонам, хотя это можно перенастроить:

- `**/IT*.java`
- `**/*IT.java`
- `**/*ITCase.java`

Поскольку `maven-surefire-plugin` входит в то же семейство плагинов, он поддерживает это соглашение и не включает эти тесты в фазу `test`.

Интеграционные тесты рекомендуется запускать командой `mvn verify`, как показано ниже, а не `mvn integration-test`. Команда с `verify` включает `post-integration-test` — типичное расположение, где плагины подключают любые операции очистки, если они потребуются после тестов:

```
~: mvn verify

  [INFO] ... вывод компиляции опущен для краткости ...

  [INFO] --- maven-failsafe-plugin:3.0.0-M5:integration-test @ example ---
  [INFO]
  [INFO] -------------------------------------------------------
  [INFO] TESTS
  [INFO] -------------------------------------------------------
  [INFO] Running com.wellgrounded.LongRunningIT
  [INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0,
  [INFO] Time elapsed: 0.032s-in com.wellgrounded.LongRunningIT
  [INFO]
  [INFO] Results:
  [INFO]
  [INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
  [INFO]
  [INFO]
  [INFO] --- maven-failsafe-plugin:3.0.0-M5:verify (default) @ example ---
  [INFO] ------------------------------------------------------------------
  [INFO] BUILD SUCCESS
  [INFO] ------------------------------------------------------------------
```

11.2.7. Управление зависимостями

Ключевой особенностью, которую Maven добавил в экосистему, стал стандартный формат, который позволяет выражать информацию об управлении зависимостями в файле pom.xml. Кроме того, Maven предоставил централизованный репозиторий для библиотек. Maven может обходить файлы pom.xml из вашего проекта и всех его зависимостей, чтобы определить полный набор транзитивных зависимостей, которые необходимы вашему приложению.

Обход дерева с поиском всех необходимых библиотек называется *разрешением зависимостей* (dependency resolution). Этот процесс играет важнейшую роль в управлении современными приложениями, но у него есть свои подводные камни.

https://t.me/javalib

Чтобы понять, где возникают проблемы, вернемся к конфигурации проекта из раздела 11.1.2. В этом примере зависимости проекта образуют дерево, показанное на рис. 11.3.

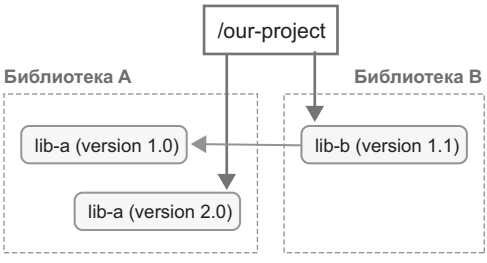


Рис. 11.3. Конфликты транзитивных зависимостей, когда зависимость требует более старой версии

Здесь явно запрашивается версия 2.0 библиотеки `lib-a`, но зависимость `lib-b` запросила более старую версию 1.0. Алгоритм разрешения зависимостей Maven отдает предпочтение той версии, которая находится ближе к корню. В итоге в конфигурации на рис. 11.3 будет использоваться версия `lib-a` 2.0. Как упоминалось в разделе 11.1.2, приложение при этом может работать нормально, но может и пострадать от катастрофического сбоя.

Другая типичная проблемная ситуация обратна предыдущей: зависимость, ближайшая к корню, оказывается *старше* той, которую предполагает транзитивная зависимость (рис. 11.4).

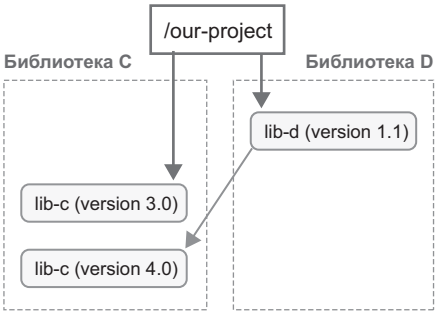


Рис. 11.4. Конфликты транзитивных зависимостей, когда зависимость требует более новой версии

В этой ситуации вполне возможно, что `lib-d` зависит от API из `lib-c`, которого не существовало в версии 3.0, так что если добавить зависимость от `lib-d` в проект, который уже использует `lib-c`, это приведет к исключениям времени выполнения.

ПРИМЕЧАНИЕ С учетом этих рисков мы рекомендуем явно объявлять в pom.xml каждый пакет, с которым напрямую взаимодействует ваш код. Если этого не сделать, а положиться на транзитивные зависимости, обновление прямой зависимости может привести к неожиданным сбоям при сборке.

https://t.me/javalib

Прежде чем решать проблемы с зависимостями, важно знать конкретный перечень этих зависимостей. Для этого Maven предоставляет в ваше распоряжение команду `mvn dependency:tree`:

```
~:mvn dependency:tree
  [INFO] Scanning for projects...
  [INFO]
  [INFO] -------------------< com.wellgrounded:example >---------------
  [INFO] Building example 1.0-SNAPSHOT
  [INFO] ----------------------------[ jar ]--------------------------
  [INFO]
  [INFO] -- maven-dependency-plugin:2.8:tree (default-cli) @ example --
  [INFO] com.wellgrounded:example:jar:1.0-SNAPSHOT
  [INFO] +- org.junit.jupiter:junit-jupiter-api:jar:5.8.1:test
  [INFO] | +- org.opentest4j:opentest4j:jar:1.2.0:test
  [INFO] | +- org.junit.platform:junit-platform-commons:jar:1.8.1:test
  [INFO] | \- org.apiguardian:apiguardian-api:jar:1.1.2:test
  [INFO] \- org.junit.jupiter:junit-jupiter-engine:jar:5.8.1:test
  [INFO]    \- org.junit.platform:junit-platform-engine:jar:1.8.1:test
  [INFO] ------------------------------------------------------------
  [INFO] BUILD SUCCESS
  [INFO] ------------------------------------------------------------
  [INFO] Total time: 0.790 s
  [INFO] Finished at: 2020-08-13T23:02:10+02:00
  [INFO] ------------------------------------------------------------
```

Дерево, которое выводит эта команда, на первом уровне иерархии показывает прямые зависимости от JUnit из файла pom.xml, а затем — собственные транзитивные зависимости JUnit.

У JUnit довольно мало зависимостей, так что для дальнейшего анализа представьте, что ваша команда хочет использовать две внутренние библиотеки, которые обеспечивают поддержку нестандартных тестовых условий. Обе библиотеки зависят от библиотеки `assertj`, но, к сожалению, от разных ее версий:

```
[
[INFO] com.wellgrounded:example:jar:1.0-SNAPSHOT
[INFO] +- org.junit.jupiter:junit-jupiter-api:jar:5.8.1:test
[INFO] |  +- org.opentest4j:opentest4j:jar:1.2.0:test
[INFO] |  +- org.junit.platform:junit-platform-commons:jar:1.8.1:test
[INFO] |  \- org.apiguardian:apiguardian-api:jar:1.1.2:test
[INFO] +- org.junit.jupiter:junit-jupiter-engine:jar:5.8.1:test
[INFO] |  \- org.junit.platform:junit-platform-engine:jar:1.8.1:test
[INFO] +- com.wellgrounded:first-test-helper:1.0.0:test
[INFO] |  \- org.assertj:assertj-core:3.21.0:test
[INFO] \- com.wellgrounded:second-test-helper:2.0.0:test
[INFO]    \- org.assertj:assertj-core:2.9.1:test
```

Первая вспомогательная библиотека требует assertj-core версии 3.21.0

Вторая вспомогательная библиотека требует assertj-core версии 2.9.1

Лучшее из возможных решений — найти новые версии зависимостей, в которых зависимости согласованы между собой. Разумеется, для внутренних библиотек

https://t.me/javalib

такая возможность есть. Она часто доступна даже в более обширном мире разработки с открытым кодом. Тем не менее иногда библиотеки перестают сопровождаться и устаревают, так что вы вполне можете оказаться в ситуации, в которой трудно достать нужное обновление.

Остается искать другие способы разрешить конфликт. Если это не удается сделать естественным образом, возможны два варианта. Учтите, что для обоих решений необходимо найти *какую-нибудь* совместимую версию, которая годится для ваших зависимостей.

Если в одной из ваших зависимостей указывается версия, которая всех устроит, но алгоритм разрешения Maven ее не выбирает, можно заставить Maven исключить определенные части дерева при разрешении. Если обе вспомогательные библиотеки благополучно работают с более новой версией `assertj-core`, то старую версию, которую требует вторая библиотека, можно просто проигнорировать:

```
<dependencies>
  <dependency>
    <groupId>com.wellgrounded</groupId>
      <artifactId>second-test-helper</artifactId>
      <version>2.0.0</version>
      <scope>test</scope>
      <exclusions>
        <exclusion>
          <groupId>org.assertj</groupId>
          <artifactId>assertj-core</artifactId>
        </exclusion>
      </exclusions>
  </dependency>
  <dependency>
    <groupId>com.wellgrounded</groupId>
    <artifactId>first-test-helper</artifactId>
    <version>1.0.0</version>
    <scope>test</scope>
  </dependency>
</dependencies>
```

Исключает устаревшую версию second-test-helper библиотеки assertj-core

Транзитивные зависимости из first-test-helper устанавливаются штатно

Худший вариант — если ни одна из версий библиотеки не окажется универсально совместимой. В этом случае можно задать точную версию в качестве прямой зависимости проекта, как показано в следующем примере. По правилам разрешения Maven выберет эту версию, потому что она ближе к корню проекта. И хотя так мы заставим программу сделать то, что нам нужно, из-за смешения версий библиотек появляется риск ошибок времени выполнения, поэтому важно тщательно протестировать все взаимодействия:

```
<dependencies>
  <dependency>
    <groupId>com.wellgrounded</groupId>
    <artifactId>second-test-helper</artifactId>
    <version>2.0.0</version>
    <scope>test</scope>
  </dependency>
```

Наши зависимости запрашивают другую версию assertj-core

https://t.me/javalib

```
  <dependency>
    <groupId>com.wellgrounded</groupId>
    <artifactId>first-test-helper</artifactId>
    <version>1.0.0</version>
    <scope>test</scope>
  </dependency>
  <dependency>
    <groupId>org.assertj</groupId>
    <artifactId>org.assertj</artifactId>
    <version>3.1.0</version>
    <scope>test</scope>
  </dependency>
</dependencies>
```

Наши зависимости запрашивают другую версию assertj-core

Но мы заставляем алгоритм разрешения использовать нужную нам версию assertj-core

Наконец, стоит заметить, что плагин `maven-enforcer-plugin` можно настроить так, чтобы сборка завершалась неудачей, если в зависимостях обнаружены какие бы то ни было несоответствия. Тогда нам не придется полагаться на некорректное поведение на стадии выполнения, чтобы выявить проблемы (см. http://mng.bz/o2WN). От сбоев при сборке можно избавиться при помощи методов, описанных ранее.

11.2.8. Анализ покрытия

Процесс сборки — отличное место для того, чтобы подключать дополнительные инструменты и проверки. Одна из важнейших проверок анализирует покрытие кода — показатель того, какую долю кода выполняют тесты.

Ведущий инструмент контроля покрытия в экосистеме Java — библиотека JaCoCo (http://mng.bz/nNjv). Ее можно настроить на определенные уровни покрытия в процессе тестирования, и вы получите отчет о том, какие части кода обеспечены (или не обеспечены) тестами.

Чтобы подключить JaCoCo, достаточно добавить плагин в раздел `<build><plugins>` файла pom.xml. Плагин не активизируется сам собой, поэтому нужно указать, в какой момент он должен сработать. В следующем примере он связывается с фазой `test`:

```
<build>
  <plugins>
    <plugin>
      <groupId>org.jacoco</groupId>
      <artifactId>jacoco-maven-plugin</artifactId>
      <version>0.8.5</version>
      <executions>
        <execution>
          <goals>
            <goal>prepare-agent</goal>
          </goals>
        </execution>
        <execution>
          <id>report</id>
          <phase>test</phase>
```

JaCoCo необходимо запустить на ранней стадии процесса. Здесь этот плагин включается в фазу инициализации

Заставляем JaCoCo вывести отчет на фазе тестирования

https://t.me/javalib

```
          <goals>
            <goal>report</goal>
          </goals>
        </execution>
      </executions>
    </plugin>
  </plugins>
</build>
```

По умолчанию отчеты по всем классам сохраняются в target/site/jacoco, как показано на рис. 11.5; полная версия в формате HTML содержится в файле index.html.

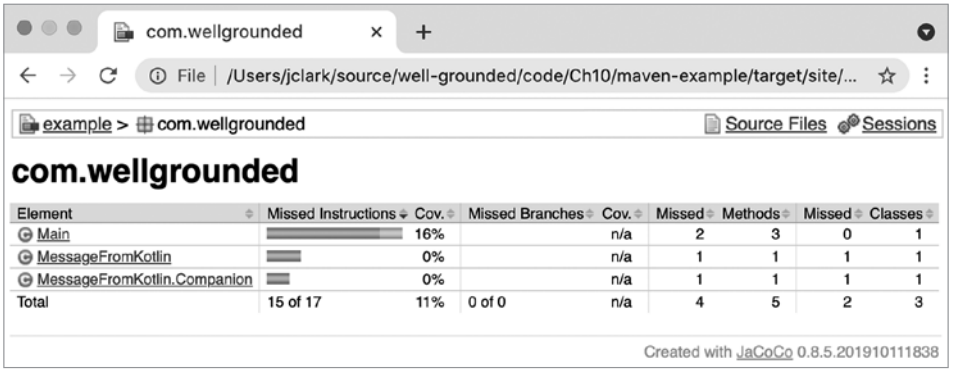


Рис. 11.5. Страница JaCoCo с отчетами о покрытии

11.2.9. За пределами Java 8

В главе 1 мы упоминали о модулях, которые по смыслу относились к Java Enterprise Edition, но были включены в основной JDK. В JDK 9 они были объявлены устаревшими, а в JDK 11 исключены, но остались доступными в виде внешних библиотек:

- `java.activation` (JAF)
- `java.corba` (CORBA)
- `java.transaction` (JTA)
- `java.xml.bind` (JAXB)
- `java.xml.ws` (JAX-WS, а также некоторые сопутствующие технологии)
- `java.xml.ws.annotation` (Common Annotations)

Если ваш проект зависит от каких-либо из этих модулей, переход на более новую версию JDK может привести к сбоям при сборке. К счастью, проблема решается, если добавить несколько простых зависимостей в файл pom.xml:

```
<dependencies>
  <dependency>
    <groupId>com.sun.activation</groupId>  <-- java.activation (JAF)
```

https://t.me/javalib

```
    <artifactId>jakarta.activation</artifactId>
    <version>1.2.2</version>
  </dependency>
  <dependency>
    <groupId>org.glassfish.corba</groupId>  <-- java.corba (CORBA)
    <artifactId>glassfish-corba-omgapi</artifactId>
    <version>4.2.1</version>
  </dependency>
  <dependency>
    <groupId>javax.transaction</groupId>  <-- java.transaction (JTA)
    <artifactId>javax.transaction-api</artifactId>
    <version>1.3</version>
  </dependency>
  <dependency>
    <groupId>jakarta.xml.bind</groupId>  <-- java.xml.bind (JAXB)
    <artifactId>jakarta.xml.bind-api</artifactId>
    <version>2.3.3</version>
  </dependency>
  <dependency>
    <groupId>jakarta.xml.ws</groupId>  <-- java.xml.ws (JAX-WS, а также некоторые сопутствующие технологии)
    <artifactId>jakarta.xml.ws-api</artifactId>
    <version>2.3.3</version>
  </dependency>
  <dependency>
    <groupId>jakarta.annotation</groupId>  <-- java.xml.ws.annotation (Common Annotations)
    <artifactId>jakarta.annotation-api</artifactId>
    <version>1.3.5</version>
  </dependency>
</dependencies>
```

11.2.10. Многоверсионные файлы JAR в Maven

В JDK 9 появилась возможность создавать файлы JAR, в которые упакован разный код для разных JDK. Это позволяет пользоваться новой функциональностью платформы и при этом продолжать поддерживать клиентов со старыми версиями.

В главе 2 мы рассмотрели эту возможность и вручную построили формат JAR, который ее реализует. В этой структуре каталоги разных версий размещаются в каталоге META-INF/versions в файле JAR; именно там JVM, начиная с версии 9, в процессе загрузки проверяет наличие более новых версий заданного класса:

```
.
├── META-INF
│   ├── MANIFEST.MF
│   └── versions
│       └── 11
│           └── wgjd2ed
│               └── GetPID.class
└── wgjd2ed
    ├── GetPID.class
    └── Main.class
```

https://t.me/javalib

При такой структуре в основном каталоге `wgjd2ed` будут размещаться классы, чья версия файла класса соответствует самой старой JVM, с которой может использоваться JAR. (В нашем примере это JDK 8.) Однако классы в META-INF/versions/11 могут компилироваться с более новыми JDK и имеют более свежую версию файла класса. Более старые JDK игнорируют каталог META-INF/versions (а JDK версий 9 и выше понимают, какие версии им можно использовать), поэтому в JAR можно включить новый код, но при этом все будет работать и на более старых JVM. Именно для того, чтобы автоматизировать подобные рутинные процессы, и была разработана система Maven.

И хотя для того, чтобы включить многоверсионную функциональность, не требуется ничего, кроме выходного формата в файле JAR, мы воспроизведем структуру кода для наглядности. Как показано ниже, код в каталоге `src` — это базовая функциональность, которая будет видна любому JDK по умолчанию. Код в каталоге `versions` может заменять некоторые классы альтернативными реализациями:

```
.
├── pom.xml
├── src
│   └── main
│       └── java
│           └── wgjd2ed
│               ├── GetPID.java
│               └── Main.java
└── versions
    └── 11
        └── src
            └── wgjd2ed
                └── GetPID.java
```

По умолчанию Maven найдет и откомпилирует код в src/main, но придется разобраться с парой затруднений:

- Maven должен также найти и тот код, который размещен в каталоге versions.
- Более того, Maven должен будет откомпилировать этот код с учетом того, что он предназначен не для того же JDK, что основной код проекта.

Обе цели достигаются настройкой плагина `maven-compiler-plugin`, который собирает файлы классов Java. В следующем фрагменте добавляются два разных шага `<execution>`: на первом шаге компилируется базовый код, предназначенный для JDK 8, а на втором — код для JDK 11.

ПРИМЕЧАНИЕ При компиляции нужно использовать версию JDK не старше чем последняя версия, которую вы выбрали в качестве целевой. Однако здесь мы явно указываем, что на некоторых шагах сборки должна применяться более старая версия, чем та, которую готов обеспечить компилятор.

https://t.me/javalib

```
<plugins>
  <plugin>
    <groupId>org.apache.maven.plugins</groupId>
    <artifactId>maven-compiler-plugin</artifactId>
    <version>3.8.1</version>
    <executions>
      <execution>
        <id>compile-java-8</id>
        <goals>
          <goal>compile</goal>
        </goals>
        <configuration>
          <source>1.8</source>
          <target>1.8</target>
        </configuration>
      </execution>
      <execution>
        <id>compile-java-11</id>
        <phase>compile</phase>
        <goals>
          <goal>compile</goal>
        </goals>
        <configuration>
          <compileSourceRoots>
            <compileSourceRoot>
              ${project.basedir}/versions/11/src
            </compileSourceRoot>
          </compileSourceRoots>
          <release>11</release>
          <multiReleaseOutput>
            true
          </multiReleaseOutput>
        </configuration>
      </execution>
    </executions>
  </plugin>
</plugins>
```

Шаг выполнения с компиляцией для JDK 8

Компиляция выполняется в JDK 11, поэтому мы явно указываем, что вывод генерируется для JDK 8

Второй шаг выполнения с компиляцией для JDK 11

Сообщает Maven, где находится код для конкретной версии

Атрибуты release и multiReleaseOutput сообщают Maven, для какой версии JDK предназначен код, и заставляют размещать выходные классы в соответствующем многоверсионном каталоге

В результате файл JAR строится и упаковывается с правильной структурой. Остается последний шаг — пометить манифест как многоверсионный. Это делается в настройках `maven-jar-plugin` поблизости от того места, где мы превращали JAR в исполняемый файл в разделе 11.2.4:

```
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-jar-plugin</artifactId>
  <version>3.2.0</version>
  <configuration>
    <archive>
      <manifest>
        <addClasspath>true</addClasspath>
        <mainClass>wgjd2ed.Main</mainClass>
```

https://t.me/javalib

```
      </manifest>
      <manifestEntries>  ◄——  Этот атрибут помечает файл JAR как многоверсионный
        <Multi-Release>true</Multi-Release>
      </manifestEntries>
    </archive>
  </configuration>
</plugin>
```

После этого можно запустить код под разными JDK и посмотреть, работает ли он так, как ожидалось. В нашем примере базовая реализация для JDK 8 выводит дополнительное сообщение о версии, по которому можно определить, что она успешно работает:

```
~:mvn clean compile package
[INFO] Scanning for projects...
[INFO]
[INFO] ----------------< wgjd2ed:maven-multi-release >-------------------
[INFO] Building maven-multi-release 1.0-SNAPSHOT
[INFO] --------------------------------[ jar ]---------------------------------
[INFO]
[INFO] .... Много дополнительных шагов
[INFO]
[INFO] - maven-jar-plugin:3.2.0:jar (default-jar) @ maven-multi-release -
[INFO] Building jar: ~/target/maven-multi-release-1.0-SNAPSHOT.jar
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time: 1.813 s
[INFO] Finished at: 2021-03-05T09:39:16+01:00
[INFO] ------------------------------------------------------------------------

~:java -version
openjdk version "11.0.6" 2020-01-14
OpenJDK Runtime Environment AdoptOpenJDK (build 11.0.6+10)
OpenJDK 64-Bit Server VM AdoptOpenJDK (build 11.0.6+10, mixed mode)

~:java -jar target/maven-multi-release-1.0-SNAPSHOT.jar
75891

# Переключите версию JDK своим обычным способом....

~:java -version
openjdk version "1.8.0_265"
OpenJDK Runtime Environment (AdoptOpenJDK)(build 1.8.0_265-b01)
OpenJDK 64-Bit Server VM (AdoptOpenJDK)(build 25.265-b01, mixed mode)

~:java -jar target/maven-multi-release-1.0-SNAPSHOT.jar
Java 8 version...
76087
```

Теперь все готово к тому, чтобы вы могли использовать новые возможности JDK, не отказываясь от поддержки старых клиентов.

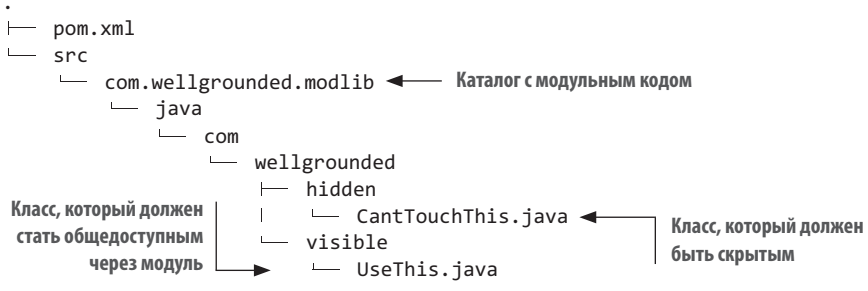
https://t.me/javalib

11.2.11. Maven и модули

В главе 2 мы подробно рассмотрели новую систему модулей JDK. Давайте разберемся, как она влияет на сценарии сборки. Для начала возьмем простую библиотеку, которая предоставляет открытый доступ к одному из своих пакетов, но скрывает другой.

Модульная библиотека

Структура кода в модульных проектах несколько отклоняется от жесткого стандарта Maven. Каталог `main` заменяется именем модуля:



После этих изменений нужно сообщить Maven о том, что исходный код для компиляции следует искать в новом месте:

```
<build>
  <sourceDirectory>src/com.wellgrounded.modlib/java</sourceDirectory>
</build>
```

Последний шаг в создании модульной библиотеки — добавить файл `module-info.java` в корневой каталог кода (на один уровень с каталогом com). В этом файле задается имя модуля и указывается, к чему разрешен внешний доступ:

```
module com.wellgrounded.modlib {
    exports com.wellgrounded.modlib.visible;
}
```

Все остальное в этой простой библиотеке остается по-старому, и если запустить команду `mvn package`, то в каталоге target появится файл JAR. Прежде чем двигаться дальше, можно также поместить эту библиотеку в локальный кэш Maven командой `mvn install`.

> **ПРИМЕЧАНИЕ** Система модулей JDK влияет не на упаковку, а на управление доступом при сборке и во время выполнения. Модульную библиотеку можно распространять как традиционный файл JAR; просто к ней добавляется файл module-info.class, который сообщает модульным приложениям, как с ней взаимодействовать.

Итак, у нас появилась модульная библиотека. Давайте теперь соберем модульное приложение, которое будет ее использовать.

https://t.me/javalib

Модульное приложение

Структура нашего модульного приложения аналогична той, которая использовалась для библиотеки:

```
.
├── pom.xml
└── src
    └── com.wellgrounded.modapp
        └── java
            ├── com
            │   └── wellgrounded
            │       └── Main.java
            └── module-info.java
```

Файл module-info.java для приложения объявляет имя и указывает, что ему требуется пакет, который экспортирует наша библиотека:

```
module com.wellgrounded.modapp {
    requires com.wellgrounded.modlib;
}
```

Впрочем, само по себе это не сообщает Maven о том, где искать файл JAR библиотеки, поэтому мы включаем его в обычный раздел `<dependency>`:

```
dependencies>
  <dependency>
    <groupId>com.wellgrounded</groupId>
    <artifactId>modlib</artifactId>
    <version>2.0</version>
  </dependency>
</dependencies>
```

Наша библиотека из предыдущего раздела, установленная в локальном репозитории Maven

При компиляции и последующем выполнении важно, чтобы эта зависимость помещалась в путь к модулям, а не в classpath. Как Maven решает эту задачу? К счастью, последние версии `maven-compiler-plugin` достаточно умны, чтобы заметить, что: 1) наше приложение содержит файл module-info.java, а следовательно, оно модульное; и 2) зависимость включает module-info.class и поэтому тоже является модулем. Если вы используете свежую версию `maven-compiler-plugin` (версия 3.8 отлично работала на момент написания книги), то Maven сделает все необходимые выводы за вас.

Приложение содержит абсолютно нормальный код на Java, и можно использовать функциональность модульной библиотеки, как и предполагалось:

```
package com.wellgrounded.modapp;

import com.wellgrounded.modlib.visible.UseThis;

public class Main {
  public static void main(String[] args) {
    System.out.println(UseThis.getMessage());
  }
}
```

Импортирование из модуля выполняется так же, как из любого другого пакета

Использует класс из нашего модуля, чтобы вывести сообщение

https://t.me/javalib

Возможно, вы помните, что в библиотеке есть еще один пакет, доступ к которому не предоставлялся. Что произойдет, если изменить приложение и попытаться обратиться к этому пакету?

```
package com.wellgrounded.modapp;

import com.wellgrounded.modlib.visible.UseThis;
import com.wellgrounded.modlib.hidden.CantTouchThis;

public class Main {
  public static void main(String[] args) {
    System.out.println(UseThis.getMessage());
  }
}
```

Библиотека не экспортирует com.wellgrounded.modlib.hidden

При попытке компиляции немедленно выдается сообщение об ошибке:

```
[INFO] - maven-compiler-plugin:3.8.1:compile @ modapp ---
  [INFO] Changes detected - recompiling the module!
  [INFO] Compiling 2 source files to /mod-app/target/classes
  [INFO] -------------------------------------------------------------
  [ERROR] COMPILATION ERROR :
  [INFO] -------------------------------------------------------------
  [ERROR]
    src/com.wellgrounded.modapp/java/com/wellgrounded/Main.java:[4,31]
      package com.wellgrounded.modlib.hidden is not visible (package
      com.wellgrounded.modlib.hidden is declared in module
      com.wellgrounded.modlib, which does not export it)[1]

  [INFO] 1 error
  [INFO] -------------------------------------------------------------
  [INFO] BUILD FAILURE
  [INFO] -------------------------------------------------------------
```

javac и система модулей не позволяют обратиться к тому, что не экспортируется

Инструментарий Maven прошел долгий путь с момента появления модулей в JDK 9. Эта система сборки поддерживает все стандартные сценарии с минимумом дополнительной настройки.

Прежде чем двигаться дальше, сделаем небольшое отступление. В этом разделе файл module-info.class часто служил для Maven сигналом о том, что нужно применять модульные правила. Однако модули — это *необязательная* возможность JDK: этот статус позволяет сохранять совместимость с огромными объемами кода, который был написан до появления модулей.

Что произойдет, если мы соберем приложение с модульной библиотекой, но само приложение не будет с помощью файла module-info.java сообщать о том, что оно использует модули? В таком случае библиотека, хотя она и является модульной,

[1] Пакет com.wellgrounded.modlib.hidden не является видимым (пакет com.wellgrounded.modlib.hidden объявлен в модуле com.wellgrounded.modlib, который его не экспортирует). — *Примеч. пер.*

https://t.me/javalib

будет включена через classpath. Она будет размещена в безымянном модуле наряду с остальным кодом приложения, а все ограничения доступа, которые определены в библиотеке, будут фактически проигнорированы. В сопроводительные материалы этой книги вместе с модульным приложением включено обычное приложение, которое использует библиотеку через classpath, чтобы наглядно продемонстрировать эффект от того, используется ли система модулей.

На этом мы завершаем обзор основных возможностей Maven. Но что, если вам потребуются расширенные возможности, для которых нет готового решения, несмотря на огромный ассортимент плагинов в интернете?

11.2.12. Разработка плагинов для Maven

Даже самые стандартные возможности Maven предоставляются в виде плагинов, и ничто не мешает вам написать собственный плагин, когда существующей функциональности недостаточно. Как вы уже видели, обращаться к плагину можно примерно так же, как подгружать зависимую библиотеку. Поэтому неудивительно, что плагины Maven реализуются в виде отдельных файлов JAR.

Мы начнем соответствующий пример с файла pom.xml. Шаблонный код в целом остается тем же, если не считать пары небольших дополнений:

```
<project>
  <modelVersion>4.0.0</modelVersion>

  <name>A Well-Grounded Maven Plugin</name>
  <groupId>com.wellgrounded</groupId>
  <artifactId>wellgrounded-maven-plugin</artifactId>
  <packaging>maven-plugin</packaging>
  <version>1.0-SNAPSHOT</version>

  <properties>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    <maven.compiler.source>11</maven.compiler.source>
    <maven.compiler.target>11</maven.compiler.target>
  </properties>

  <dependencies>
    <dependency>
      <groupId>org.apache.maven</groupId>
      <artifactId>maven-plugin-api</artifactId>
      <version>3.0</version>
    </dependency>

  <dependency>
    <groupId>org.apache.maven.plugin-tools</groupId>
    <artifactId>maven-plugin-annotations</artifactId>
```

Сообщает Maven, что мы намерены собрать пакет с плагином

-SNAPSHOT — типичный суффикс, который добавляется к еще не опубликованным версиям библиотеки. Он проявляется при подключении библиотеки, потому что когда запрашивается зависимость, необходимо указать полную строку, например 1.0-SNAPSHOT

Зависимости Maven API, которые понадобятся нашей реализации

https://t.me/javalib

```
      <version>3.4</version>
      <scope>provided</scope>
    </dependency>
  </dependencies>
</project>
```

Теперь можно добавлять код. Разместив файл Java в каталоге стандартной структуры, мы реализуем то, что называется `Mojo` — фактически *цель* Maven:

```
package com.wellgrounded;

import org.apache.maven.plugin.AbstractMojo;
import org.apache.maven.plugin.MojoExecutionException;
import org.apache.maven.plugins.annotations.Mojo;

@Mojo(name = "wellgrounded")
public class WellGroundedMojo extends AbstractMojo
{
    public void execute() throws MojoExecutionException
    {
        getLog().info("Расширяем Maven для собственного удовольствия.");
    }
}
```

Наш класс расширяет `AbstractMojo` и при помощи аннотации `@Mojo` сообщает Maven имя цели. В теле метода выполняются необходимые операции. В данном случае мы просто выводим некоторый текст, но на этом месте в вашем распоряжении есть все возможности языка и экосистемы Java, которые нужны, чтобы реализовать вашу цель.

Чтобы протестировать плагин в другом проекте, нужно установить его командой `mvn install`, которая поместит файл JAR в репозиторий локального кэширования. После этого плагин можно подключить к другому проекту точно так же, как все «настоящие» плагины, которые вы уже видели в этой главе:

```
<build>
  <plugins>
    <plugin>
      <groupId>com.wellgrounded</groupId>
      <artifactId>wellgrounded-maven-plugin</artifactId>
      <version>1.0-SNAPSHOT</version>
      <executions>
        <execution>
          <phase>compile</phase>
          <goals>
            <goal>wellgrounded</goal>
          </goals>
        </execution>
      </executions>
    </plugin>
  </plugins>
</build>
```

Обращение к плагину по координатам groupId и artifactId

Наша цель связывается с фазой compile

https://t.me/javalib

Когда это будет сделано, плагин можно увидеть в действии во время компиляции:

```
~: mvn compile
  [INFO] Scanning for projects...
  [INFO]
  [INFO] ------------------< com.wellgrounded:example >--------------
  [INFO] Building example 1.0-SNAPSHOT
  [INFO] --------------------------------[ jar ]---------------------
  [INFO]
  [INFO] - maven-resources-plugin:2.6:resources (default-resources) -
  [INFO] Using 'UTF-8' encoding to copy filtered resources.
  [INFO] skip non existing resourceDirectory /src/main/resources
  [INFO]
  [INFO] --- maven-compiler-plugin:3.1:compile (default-compile) ---
  [INFO] Nothing to compile - all classes are up to date
  [INFO]
  [INFO] --- wellgrounded-maven-plugin:1.0-SNAPSHOT:wellgrounded ---
  [INFO] Расширяем Maven для собственного удовольствия.
  [INFO] ------------------------------------------------------------
  [INFO] BUILD SUCCESS
  [INFO] ------------------------------------------------------------
  [INFO] Total time: 0.872 s
  [INFO] Finished at: 2020-08-16T22:26:20+02:00
  [INFO] ------------------------------------------------------------
```

Плагин выполняется как часть фазы компиляции

Стоит заметить, что если просто подключить плагин без элемента `<executions>`, вы не заметите никаких следов его присутствия в проекте. Нестандартные плагины должны объявлять нужную фазу жизненного цикла в файле pom.xml.

Не всегда легко уследить за жизненным циклом и за тем, какие цели привязаны к той или иной фазе. К счастью, существует плагин, который поможет вам в этом: `buildplan-maven-plugin` проясняет ситуацию с текущими задачами.

Хотя его можно включить в pom.xml, как и любой другой плагин, есть альтернативный вариант. Чтобы не дублировать плагин во всех проектах, укажите его в файле ~/.m2/settings.xml своей учетной записи пользователя, как показано ниже. Файлы settings.xml похожи на файлы pom.xml в Maven, но они не связаны ни с каким конкретным проектом:

```
<settings xmlns="http://maven.apache.org/SETTINGS/1.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/SETTINGS/1.0.0
                      https://maven.apache.org/xsd/settings-1.0.0.xsd">
  <pluginGroups>
    <pluginGroup>fr.jcgay.maven.plugins</pluginGroup>
  </pluginGroups>
</settings>
```

https://t.me/javalib

После этого плагин можно активизировать в любой сборке проекта в Maven:

```
~: mvn buildplan:list

  [INFO] Scanning for projects...
  [INFO]
  [INFO] --------------------< com.wellgrounded:example >-----------------
  [INFO] Building example 1.0-SNAPSHOT
  [INFO] ----------------------------[ jar ]---------------------------
  [INFO]
  [INFO] ---- buildplan-maven-plugin:1.3:list (default-cli) @ example ----
  [INFO] Build Plan for example:
  ------------------------------------------------------------------------
  PLUGIN               | PHASE           | ID                 | GOAL
  ------------------------------------------------------------------------
  jacoco-maven-plugin  | initialize      | default            | prep-agent
  maven-compiler-plugin| compile         | default-compile    | compile
  maven-compiler-plugin| test-compile    | default-testCompile| testCompile
  maven-surefire-plugin| test            | default-test       | test
  jacoco-maven-plugin  | test            | report             | report
  maven-jar-plugin     | package         | default-jar        | jar
  maven-failsafe-plugin| integration-test| default            | int-test
  maven-failsafe-plugin| verify          | default            | verify
  maven-install-plugin | install         | default-install    | install
  maven-deploy-plugin  | deploy          | default-deploy     | deploy
  [INFO] ------------------------------------------------------------------
  [INFO] BUILD SUCCESS
  [INFO] ------------------------------------------------------------------
  [INFO] Total time: 0.461 s
  [INFO] Finished at: 2020-08-30T15:54:30+02:00
  [INFO] ------------------------------------------------------------------
```

ПРИМЕЧАНИЕ Если вы не хотите добавлять плагин ни в pom.xml, ни в settings.xml, можно просто запустить Maven, указав полное имя плагина в качестве параметра! В предыдущем примере можно использовать команду `mvn fr.jcgay.maven.plugins:buildplan-maven-plugin:list`, чтобы Maven загрузил плагин и немедленно выполнил его. Этот прием отлично подходит для нестандартных задач или экспериментов. Документация Maven по созданию плагинов (см. http://mng.bz/v6dx) подробно написана и хорошо сопровождается, так что в нее стоит заглянуть, когда вы будете писать собственные плагины.

Maven остается в числе самых популярных и влиятельных средств сборки для Java. Тем не менее его категоричность нравится не всем. Gradle — самая востребованная из альтернативных систем, поэтому мы посмотрим, как решать те же задачи с ее помощью.

11.3. GRADLE

Система Gradle появилась позднее Maven, и она совместима с большей частью инфраструктуры управления зависимостями, которую предлагает Maven. Gradle

https://t.me/javalib

поддерживает стандартную структуру каталогов и тоже предоставляет жизненный цикл сборки по умолчанию для проектов JVM, но в отличие от Maven, все возможности Gradle можно полностью настраивать по своему усмотрению.

Вместо XML Gradle использует декларативный предметно-ориентированный язык (DSL) поверх реального языка программирования (Kotlin или Groovy). Обычно это приводит к тому, что в простых случаях логика сборки оказывается компактнее, а в сложных вам доступны гибкие возможности настройки.

У Gradle также есть механизмы для повышения производительности, которые предотвращают лишнюю работу и позволяют обрабатывать задачи поэтапно. В результате сборка часто выполняется быстрее и лучше масштабируется. Давайте сделаем первые шаги и посмотрим, как выполнять команды Gradle.

11.3.1. Установка Gradle

Gradle можно установить с официального сайта (https://gradle.org/install). Свежие выпуски работают только на JVM версии 8 и выше. После установки Gradle можно запустить в командной строке, и по умолчанию программа выведет справку:

```
~: gradle

  > Task :help

  Добро пожаловать в Gradle 7.3.3.[1]

  Чтобы запустить сборку, выполните gradlew <задача> ...

  Чтобы просмотреть список доступных задач, выполните gradlew tasks

  Чтобы просмотреть подробные сведения о задаче, выполните gradlew help
    --task <задача>

  Чтобы просмотреть список параметров командной строки, выполните gradlew --help

  Подробное руководство по использованию Gradle см. по адресу:
    https://docs.gradle.org/7.3.3/userguide/command_line_interface.html

  За поддержкой обращайтесь по адресу https://help.gradle.org

  BUILD SUCCESSFUL in 606ms
  1 actionable task: 1 executed
```

Как видите, приступить к работе несложно, но установить одну глобальную версию Gradle — не идеальное решение. Разработчики часто собирают несколько

[1] Как и подавляющее большинство инструментов, Gradle выводит информацию на английском языке. В этой книге избранные важные фрагменты вывода переведены на русский язык, чтобы читателю было удобнее в них разобраться. Имейте в виду, что если вы запустите gradle (и прочие команды) в настоящей командной строке, вы не увидите надписей по-русски! — *Примеч. ред.*

https://t.me/javalib

разных проектов так, чтобы каждый из них использовал собственную версию Gradle.

Чтобы это устроить, Gradle вводит понятие *обертки* (wrapper). Задача `gradle wrapper` фиксирует в проекте конкретную версию Gradle, к которой затем можно обращаться командой `./gradlew` или `gradlew.bat`. Обертки `gradlew` считаются хорошей практикой, потому что они помогают избежать несовместимости версий, так что вам редко придется выполнять команду `gradle` напрямую.

> **ПРИМЕЧАНИЕ** В систему управления исходным кодом рекомендуется включать результаты обертки `gradle` и `gradlew*`, но при этом не включать локальный кэш `.gradle`.

Если вы пользуетесь обертками, то каждый, кто загружает ваш проект, получит систему сборки с правильным версионированием, не устанавливая дополнительные компоненты.

11.3.2. Задачи

Ключевым понятием Gradle является *задача* (task): это единица работы, которую можно запустить на выполнение. Задачи могут зависеть от других задач, их можно настраивать с помощью сценариев, а также добавлять через систему плагинов Gradle. Задачи напоминают цели Maven, хотя на концептуальном уровне они больше похожи на функции. У задач есть четко определенный ввод и вывод, их можно комбинировать и выстраивать в цепочки. Если цели Maven должны быть связаны с конкретной фазой жизненного цикла сборки, то задачи Gradle можно запускать и использовать так, как вам удобнее.

Gradle предоставляет превосходные средства интроспекции, и важнейшее из них — метазадача `./gradlew tasks`, которая выводит список текущих доступных задач в проекте. Даже если вы еще ничего не объявили, команда `tasks` выводит такой список:

```
~: ./gradlew tasks

  > Task :tasks
  ------------------------------------------------------------
  Задачи, запускаемые из корневого проекта
  ------------------------------------------------------------

  Задачи по настройке сборки
  -----------------
  init - Инициализирует новую сборку Gradle
  wrapper - Генерирует файлы обертки Gradle

  Справочные задачи
   ----------
  buildEnvironment - Выводит все зависимости сценария сборки в корневом проекте
  components - Выводит компоненты, произведенные корневым проектом
  dependencies - Выводит все зависимости, объявленные в корневом проекте
  dependencyInsight - Выводит подсказку для зависимости в корневом проекте
```

https://t.me/javalib

```
dependentComponents - Выводит зависимые компоненты в корневом проекте
help - Выводит справочную информацию
model - Выводит модель конфигурации корневого проекта [инкубационная
        функциональность]
outgoingVariants - Выводит исходящие варианты корневого проекта
projects - Выводит подпроекты корневого проекта
properties - Выводит свойства корневого проекта
tasks - Выводит задачи, выполняемые из корневого проекта
```

Если запустить любую задачу с флагом `--dry-run`, выводится список задач, которые Gradle выполнил бы, но сами задачи не выполняются. Это может быть полезно, когда нужно разобраться в логике системы сборки, отладить неисправные плагины или выполнить нестандартные задачи.

11.3.3. Сценарии сборки

Центральный компонент сборки Gradle — *сценарий сборки* (buildscript). В этом кроется ключевое отличие Gradle от Maven: отличается не только формат, но и вся философия. POM-файлы Maven пишутся на XML, тогда как в Gradle сценарий сборки представляет собой исполняемый код на языке программирования, который часто называется *предметно-ориентированным языком* (Domain-Specific Language, DSL). Современные версии Gradle поддерживают как Groovy, так и Kotlin.

Groovy или Kotlin

DSL-языки Gradle начинались с Groovy. Как вы узнали из главы 8, Groovy — динамический язык для JVM, который неплохо позволяет создавать гибкие и компактные сценарии сборки. Однако после выхода Gradle 5.0 добавился другой вариант — язык Kotlin, который мы подробно рассматривали в главе 9.

> **ПРИМЕЧАНИЕ** Сценарии сборки, написанные на Kotlin, используют расширение .gradle.kts вместо .gradle.

Это логично, потому что Kotlin сейчас доминирует в сфере разработки для Android, а Gradle является официальной системой сборки этой платформы. Если использовать один и тот же язык во всех частях проекта, это позволит существенно упростить разработку.

Для наших целей также важно, что Kotlin ближе к Java, чем Groovy. Если вы уже владеете Java, но только начинаете осваивать экосистему Gradle, то имеет смысл писать сценарии сборки на Kotlin, чтобы минимизировать языковой барьер.

Конечно, Groovy никто не отменял, однако здесь мы задействуем уже имеющийся опыт Kotlin и будем использовать этот язык в следующих примерах. Все, что мы покажем в этой главе, можно выразить аналогичным образом в сценарии сборки на Groovy, и при этом Gradle будет вести себя точно так же. В документации Gradle все примеры приводятся на обоих DSL-языках.

https://t.me/javalib

11.3.4. Использование плагинов

Все параметры задач в Gradle определяются с помощью плагинов. Как было показано ранее, если вывести список задач для пустого проекта Gradle, в нем ничего не говорится о сборке, тестировании или развертывании. Всю эту функциональность обеспечивают плагины.

Вместе с Gradle поставляется множество плагинов, и чтобы их использовать, достаточно включить соответствующее объявление в build.gradle.kts. Ключевую роль играет плагин `base`:

```
plugins {
  base
}
```

Если просматривать список после того, как включен плагин `base`, вы увидите стандартные задачи жизненного цикла сборки:

```
~:./gradlew tasks

  > Task :tasks

  ------------------------------------------------------------
  Задачи, запускаемые из корневого проекта
  ------------------------------------------------------------

  Задачи сборки
  -----------
  assemble - Собирает выводы проекта
  build - Собирает выводы проекта и запускает тесты
  clean - Удаляет каталог build

  ... Другие задачи опущены для краткости

  Задачи проверки
  ------------------
  check - Запускает все проверки

  ...

  BUILD SUCCESSFUL in 640ms
  1 actionable task: 1 executed
```

Теперь можно переходить к сборке проекта Gradle для нашего кода.

11.3.5. Сборка

Хотя Gradle позволяет настраивать все как вам заблагорассудится, по умолчанию он предполагает такую же структуру кода, которая сформировалась и приобрела популярность благодаря Maven. Во многих (а может быть, даже почти во всех) проектах лучше не изменять эту структуру, хотя такая возможность и существует.

https://t.me/javalib

Начнем с элементарной библиотеки Java. Для ее сборки мы создадим такое дерево исходного кода:

```
.
├── build.gradle.kts
├── gradle
│   └── wrapper
│       ├── gradle-wrapper.jar
│       └── gradle-wrapper.properties
├── gradlew
├── gradlew.bat
├── settings.gradle.kts
└── src
    └── main
        └── java
            └── com
                └── wellgrounded
                    └── AwesomeLib.java
```

Эти файлы были автоматически созданы командой gradle wrapper

Нам понадобится более осведомленный плагин, чем `base`, который ничего не знает о Java. В нашем случае, когда используется простой файл JAR, мы применим плагин Gradle `java-library`. Он дополняет `base` всем необходимым; на самом деле на практике редко встречаются сборки Gradle, где `base` фигурирует сам по себе. Дело в том, что плагины могут применять другие плагины, чтобы пользоваться их возможностями, по аналогии с композицией в объектно-ориентированном программировании:

```
plugins {
  `java-library`
}
```

Если в именах плагинов есть специальные символы (как дефис в нашем случае), они заключаются в обратные (не прямые!) кавычки

В результате набор задач сборки расширяется:

```
Задачи сборки
-----------
  assemble - Собирает выводы проекта
  build - Собирает выводы проекта и запускает тесты
  buildDependents - Собирает выводы проекта и зависимых проектов и запускает
                    тесты
  buildNeeded - Собирает выводы проекта и проектов, от которых он зависит,
                и запускает тесты
  classes - Собирает главные классы
  clean - Удаляет каталог build
  jar - Собирает архив jar, содержащий главные классы
  testClasses - Собирает тестовые классы
```

В терминологии Gradle задача `assemble` компилирует и упаковывает файл JAR. При пробном запуске (с ключом `--dry-run`) выводятся все этапы, часть из которых отсутствует в списке задач по умолчанию:

```
./gradlew assemble --dry-run
  :compileJava SKIPPED
  :processResources SKIPPED
```

https://t.me/javalib

```
  :classes SKIPPED
  :jar SKIPPED
  :assemble SKIPPED
```

Команда `./gradlew assemble` формирует результат в каталоге `build`:

```
.
└── build
    ├── classes
    │   └── java
    │       └── main
    │           └── com
    │               └── wellgrounded
    │                   ├── Main.class
    └── libs
        └── wellgrounded.jar
```

Создание приложения

Простой файл JAR — хорошее начало, но рано или поздно вы захотите запустить приложение. Это потребует более тонкой настройки, хотя все, что для этого нужно, по-прежнему доступно по умолчанию.

Сейчас мы сменим плагины и укажем Gradle, какой класс в приложении является главным. Заодно в этом фрагменте мы продемонстрируем некоторые удобные возможности Kotlin:

```
plugins {
  application
}

application {
  mainClass.set("wgjd.Main")
}

tasks.jar {
  manifest {
    attributes("Main-Class" to application.mainClass)
  }
}
```

В Kotlin круглые скобки не обязательны, если последний аргумент — лямбда-выражение

Плагин, который умеет компилировать и запускать приложения на Java

Задача для сборки файла JAR с измененным манифестом

Синтаксис to в Kotlin позволяет объявить хеш-отображение на месте (литерал хеша)

Команда `./gradlew build` соберет такой же итоговый файл JAR, что и раньше, но если выполнить команду `java -jar build/libs/wellgrounded.jar`, то наша тестовая программа запустится. Кроме того, плагин `application` также поддерживает команду `./gradlew run`, которая напрямую загружает и выполняет основной класс.

> **ПРИМЕЧАНИЕ** Для плагина `application` достаточно задать только значение `mainClass`, но если опустить конфигурацию tasks.jar, то вы получите файл JAR, который `./gradlew run` сможет запустить, а `java -jar` — нет. Так поступать определенно не рекомендуется!

https://t.me/javalib

Теперь у нас есть все необходимое, чтобы изучить другую ключевую особенность Gradle, которая позволяет избежать лишней работы и сократить время сборки.

11.3.6. Эффективность

Чтобы выполнять сборку как можно быстрее, Gradle старается по возможности избежать лишней работы. Одна из стратегий — инкрементная сборка. Каждая задача Gradle объявляет свои входные и выходные данные. На основании этих сведений Gradle проверяет, изменилось ли что-нибудь с момента последней сборки. Если изменений нет, Gradle пропускает задачу и повторно использует результаты предыдущей сборки.

> **ПРИМЕЧАНИЕ** При использовании Gradle нет необходимости регулярно запускать `clean`, потому что Gradle следит за тем, чтобы выполнять необходимую — и только необходимую — работу по производству результатов сборки.

В этом можно убедиться на примере нашего приложения: обратите внимание, как различается время сборки при первом полном запуске (с принудительной очисткой) и при втором запуске:

```
~: ./gradlew clean build

  BUILD SUCCESSFUL in 2s
  13 actionable tasks: 13 executed

  ~: ./gradlew build

  BUILD SUCCESSFUL in 804ms
  12 actionable tasks: 12 up-to-date
```

Инкрементная сборка может повторно использовать только результаты последнего выполнения задачи, которые расположены в том же месте файловой системы. Gradle действует еще эффективнее: кэш сборок позволяет повторно использовать результаты задач из более ранних сборок или даже из сборок, которые выполнялись в других расположениях.

Эта возможность включается в проекте с помощью флага командной строки `--build-cache`. Мы видим, что даже следующая сборка с очисткой выполняется быстрее, потому что она может повторно использовать кэшированные результаты предыдущих выполнений:

```
~: ./gradlew clean build --build-cache

  BUILD SUCCESSFUL in 2s
  13 actionable tasks: 13 executed

  ~: ./gradlew clean build --build-cache

  BUILD SUCCESSFUL in 1s
  13 actionable tasks: 6 executed, 7 from cache
```

https://t.me/javalib

Быстродействие — ключевая особенность Gradle, благодаря которой сборка остается быстрой, даже если код растет в объеме. Существуют и другие возможности, которые выходят за рамки этой книги: инкрементная компиляция Java, демон Gradle, параллельное выполнение задач и тестов и т. д.

Один в поле не воин. Вряд ли от приложения будет много пользы, если оно не использует зависимости от других библиотек. Это важный аспект Gradle, в котором система серьезно отличается от Maven.

11.3.7. Зависимости в Gradle

Чтобы включать зависимости в приложение, нужно сначала сообщить Gradle, из каких репозиториев их можно загружать. Встроенные функции `mavenCentral` (см. ниже) и `google` позволяют обращаться к соответствующим источникам, но можно использовать и более подробные API, чтобы настраивать другие репозитории, включая частные:

```
repositories {
  mavenCentral()
}
```

После этого можно включать зависимости в стандартном формате координат, который популяризовал Maven. По аналогии с элементом `<scope>`, который в Maven управляет тем, где используется та или иная зависимость, в Gradle применяются *конфигурации зависимостей*. Каждая конфигурация отслеживает конкретный набор зависимостей. Плагины определяют, какие конфигурации доступны, и конфигурации добавляются путем вызова функций. Например, чтобы подключить библиотеку журнального вывода SLF4J (http://www.slf4j.org/), используются следующие конфигурации:

```
dependencies {
    implementation("org.slf4j:slf4j-api:1.7.30")
    runtimeOnly("org.slf4j:slf4j-simple:1.7.30")
}
```

В этом примере мы предполагаем, что наш код напрямую вызывает классы и методы из `slf4j-api`, поэтому он подключается через конфигурацию `implementation`, которая делает его доступным при компиляции и запуске приложения. Однако также предположим, что приложение никогда не должно напрямую вызывать методы из `slf4j-simple`, а может вызывать их исключительно через `slf4j-api`. Для этого `slf4j-simple` подключается с помощью `runtimeOnly`: в этом случае код не будет доступен во время компиляции, что предотвращает некорректное использование библиотеки. В Maven аналогичную задачу с зависимостями решает элемент `<scope>`.

Деление зависимостей на те, которые используются напрямую, и те, которые просто должны присутствовать в classpath во время выполнения, — не единственный способ классификации. Например, авторам библиотек важно отличать те библиотеки, которые мы используем, от тех, которые являются частью нашего

https://t.me/javalib

открытого API. Если зависимость входит в открытый API проекта, ее можно пометить как `api`. В следующем примере мы объявляем, что Guava — часть открытого API нашего проекта:

```
dependencies {
  api("com.google.guava:guava:31.0.1-jre")
}
```

Конфигурации могут расширять друг друга (это примерно похоже на расширение классов при наследовании), и Gradle активно задействует эту возможность. Например, при создании classpath Gradle использует конфигурации `compileClasspath` и `runtimeClassp ath`, которые расширяют `implementation` и `runtimeOnly`. Вам не нужно напрямую добавлять что-либо в конфигурации `*Classpath`: зависимости, которые добавляются к их базовым конфигурациям, формируют итоговую конфигурацию classpath (рис. 11.6).

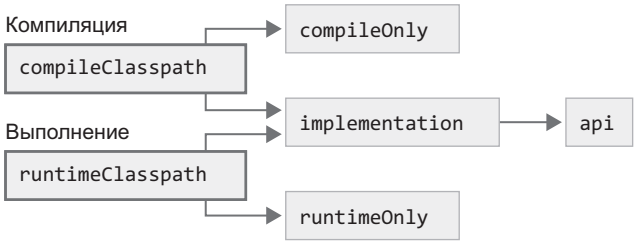


Рис. 11.6. Иерархия конфигураций Gradle

В табл. 11.1 перечислены основные конфигурации, которые доступны при использовании плагина Java, поставляемого с Gradle. Если конфигурация расширяет другие конфигурации, это тоже указано в таблице. Полный список конфигураций доступен в документации плагина Java по адресу http://mng.bz/445B.

> **ПРИМЕЧАНИЕ** В Gradle версии 7 были удалены некоторые давно устаревшие конфигурации, например compile и runtime. В интернете еще встречаются упоминания о них, однако в наше время следует переходить на более новые варианты — `implementation` (или `api`) и `runtimeOnly`.

Таблица 11.1. Типичные конфигурации зависимостей Gradle

Имя	Назначение	Расширяет
`api`	Первичные зависимости, которые являются частью внешнего, общедоступного API проекта	
`implementation`	Первичные зависимости, которые используются при компиляции и запуске	

https://t.me/javalib

Имя	Назначение	Расширяет
`compileOnly`	Зависимости, которые нужны только во время компиляции	
`compileClasspath`	Конфигурация, с помощью которой Gradle ищет пути к классам при компиляции	`compileOnly`, `implementation`
`runtimeOnly`	Зависимости, которые нужны только во время выполнения	
`runtimeClasspath`	Конфигурация, с помощью которой Gradle ищет пути к классам во время выполнения	`runtimeOnly`, `implementation`
`testImplementation`	Зависимости, которые используются во время компиляции и запуска тестов	`implementation`
`testCompileOnly`	Зависимости, которые нужны только во время компиляции тестов	
`testCompileClasspath`	Конфигурация, с помощью которой Gradle ищет пути к классам при компиляции тестов	`testCompileOnly`, `testImplementation`
`testRuntimeOnly`	Зависимости, которые нужны только во время выполнения тестов	`runtimeOnly`
`testRuntimeClasspath`	Конфигурация, с помощью которой Gradle ищет пути к классам при запуске тестов	`testRuntimeOnly`, `testImplementation`
`archives`	Список выходных файлов JAR проекта	

Как и Maven, Gradle использует информацию пакетов, чтобы построить дерево транзитивных зависимостей. Однако алгоритм разрешения конфликтов версий, который Gradle применяет по умолчанию, отличается от подхода Maven «побеждает то, что ближе к корню». В процессе разрешения Gradle обходит все дерево зависимостей, чтобы определить все запрашиваемые версии для всех пакетов. Из полного набора запрашиваемых версий Gradle по умолчанию использует *наивысшую* доступную версию.

Такой подход избегает неожиданностей, которые характерны для Maven, например, когда после изменения порядка или глубины пакетов разрешение может привести к другому результату. Gradle также может настраивать процесс разрешения, пользуясь дополнительной информацией, например расширенными ограничениями версий. Если определенным ограничениям не удается соответствовать, то сборка завершается неудачей, а Gradle выдает четкое сообщение об этом, а не выбирает автоматически версию, которая может создать проблемы.

https://t.me/javalib

Gradle предоставляет богатые API, которые позволяют управлять его поведением при разрешении и переопределять это поведение. Также в Gradle встроены мощные средства интроспекции, с помощью которых можно «заглянуть под капот», когда что-то пойдет не так. Когда вы сталкиваетесь с неприятными проблемами транзитивных зависимостей, на помощь приходит команда `./gradlew dependencies`:

```
~: ./gradlew dependencies

testImplementation - Implementation only dependencies for compilation 'test'
\--- org.junit.jupiter:junit-jupiter-api:5.8.1 (n)

... Другие конфигурации опущены для краткости

testRuntimeClasspath - Runtime classpath of compilation 'test'
+--- org.junit.jupiter:junit-jupiter-api:5.8.1
|    +--- org.junit:junit-bom:5.8.1
|    |    +--- org.junit.jupiter:junit-jupiter-api:5.8.1 (c)
|    |    +--- org.junit.jupiter:junit-jupiter-engine:5.8.1 (c)
|    |    +--- org.junit.platform:junit-platform-commons:1.8.1 (c)
|    |    \--- org.junit.platform:junit-platform-engine:1.8.1 (c)
|    +--- org.opentest4j:opentest4j:1.2.0
|    \--- org.junit.platform:junit-platform-commons:1.8.1
|         \--- org.junit:junit-bom:5.8.1 (*)
\--- org.junit.jupiter:junit-jupiter-engine:5.8.1
     +--- org.junit:junit-bom:5.8.1 (*)
     +--- org.junit.platform:junit-platform-engine:1.8.1
     |    +--- org.junit:junit-bom:5.8.1 (*)
     |    +--- org.opentest4j:opentest4j:1.2.0
     |    \--- org.junit.platform:junit-platform-commons:1.8.1 (*)
     \--- org.junit.jupiter:junit-jupiter-api:5.8.1 (*)

testRuntimeOnly - Runtime only dependencies for compilation 'test'
\--- org.junit.jupiter:junit-jupiter-engine:5.8.1 (n)
```

В крупном проекте этот вывод может быть весьма обширным. Режим `dependencyInsight` позволяет сосредоточиться на конкретной зависимости, которая вас интересует:

```
~: ./gradlew dependencyInsight \
       --configuration testRuntimeClasspath \
       --dependency junit-jupiter-api

> Task :dependencyInsight
org.junit.jupiter:junit-jupiter-api:5.8.1 (by constraint)
   variant "runtimeElements" [
      org.gradle.category = library
      org.gradle.dependency.bundling = external
      org.gradle.jvm.version = 8 (compatible with: 11)
      org.gradle.libraryelements = jar
      org.gradle.usage = java-runtime
      org.jetbrains.kotlin.localToProject = public (not requested)
      org.jetbrains.kotlin.platform.type = jvm
```

https://t.me/javalib

```
      org.gradle.status = release (not requested)
   ]

org.junit.jupiter:junit-jupiter-api:5.8.1
+--- testRuntimeClasspath
+--- org.junit:junit-bom:5.8.1
|    +--- org.junit.platform:junit-platform-engine:1.8.1
|    |    +--- org.junit:junit-bom:5.8.1 (*)
|    |    \--- org.junit.jupiter:junit-jupiter-engine:5.8.1
|    |         +--- testRuntimeClasspath
|    |         \--- org.junit:junit-bom:5.8.1 (*)
|    +--- org.junit.platform:junit-platform-commons:1.8.1
|    |    +--- org.junit.platform:junit-platform-engine:1.8.1 (*)
|    |    +--- org.junit:junit-bom:5.8.1 (*)
|    |    \--- org.junit.jupiter:junit-jupiter-api:5.8.1 (*)
|    +--- org.junit.jupiter:junit-jupiter-engine:5.8.1 (*)
|    \--- org.junit.jupiter:junit-jupiter-api:5.8.1 (*)
\--- org.junit.jupiter:junit-jupiter-engine:5.8.1 (*)

(*) - dependencies omitted (listed previously)
```

Разрешать конфликты зависимостей бывает достаточно сложно. Лучший вариант (если он возможен) — использовать средства управления зависимостями Gradle, которые находят несоответствия и обновляют сборку до взаимно совместимых версий. Ах, если бы в нашем мире это было всегда возможно!

Вернемся к одному из предыдущих примеров, где две версии внутренней вспомогательной библиотеки подключали `assertj` с несовместимыми основными версиями. Библиотека `first-test-helper` зависела от `assertj-core` 3.21.0, а `second-test-helper` требовала версии 2.9.1.

Механизм `constraints` в Gradle позволяет указать, как процесс разрешения должен выбирать версии:

```
dependencies {
  testImplementation("org.junit.jupiter:junit-jupiter-api:5.8.1")
  testRuntimeOnly("org.junit.jupiter:junit-jupiter-engine:5.8.1")

  testImplementation(
      "com.wellgrounded:first-test-helper:1.0.0")
  testImplementation(
      "com.wellgrounded:second-test-helper:2.0.0")

  constraints {
    testImplementation(
      "org.assertj:assertj-core:3.1.0") {
        because("Более новые версии несовместимы...")
    }
  }
}
```

Все зависимости просто запрашивают нужные им версии, как и прежде

Если Gradle не выполнит это ограничение, разрешение завершится неудачей

Хорошая практика состоит в том, чтобы с помощью because документировать, почему мы вмешиваемся. Gradle может использовать этот текст, в отличие от комментариев в сценарии, которые полезны только человеку, читающему код

Если вам важно указать точную версию, можно воспользоваться конструкцией `strictly`, которая переопределяет любые другие результаты разрешения:

https://t.me/javalib

```
dependencies {
  testImplementation("org.junit.jupiter:junit-jupiter-api:5.8.1")
  testRuntimeOnly("org.junit.jupiter:junit-jupiter-engine:5.8.1")
  testImplementation(
    "com.wellgrounded:first-test-helper:1.0.0")
  testImplementation(
    "com.wellgrounded:second-test-helper:2.0.0")

  testImplementation("org.assertj:assertj-core") {
    version {
      strictly("3.1.0")
    }
  }
}
```

Все зависимости просто запрашивают нужные им версии, как и прежде

Принудительно выбирает версию 3.1.0. Здесь не подойдет ни версия 3.1, ни любая другая похожая версия

Если этих механизмов окажется недостаточно или если в списке зависимостей библиотеки просто допущена ошибка, также можно заставить Gradle просто игнорировать заданную группу или артефакт при помощи `exclude`:

```
dependencies {
  testImplementation("org.junit.jupiter:junit-jupiter-api:5.8.1")
  testRuntimeOnly("org.junit.jupiter:junit-jupiter-engine:5.8.1")

  testImplementation(
    "com.wellgrounded:first-test-helper:1.0.0")
  testImplementation(
    "com.wellgrounded:second-test-helper:2.0.0") {
    exclude(group = "org.assertj")
  }
}
```

Будет выбрана зависимость из first-test-helper

Gradle проигнорирует зависимости org.assertj из second-test-helper

Впрочем, это более радикальный вариант, и в таком виде он действует только на зависимость, к которой применяется `exclude`. Если вам удастся найти решение с `constraints`, вероятно, оно будет лучше работать в долгосрочной перспективе.

Как упоминалось в предыдущих разделах, принудительно назначать версии зависимостей стоит только в качестве крайней меры, и нужно специально позаботиться о том, чтобы избежать исключений времени выполнения. Качественный набор тестов поможет сэкономить время и проконтролировать, чтобы ваши разношерстные библиотеки слаженно работали друг с другом.

11.3.8. Добавление кода на Kotlin

Как обсуждалось в главе 8 и в этой главе (в разделе о Maven), возможность включить в проект другой язык — большое преимущество JVM.

На примере добавления кода на Kotlin мы продемонстрируем сильные стороны сценарного подхода Gradle по сравнению с более статической конфигурацией Maven на основе XML. Стандартная многоязыковая структура для нашего исходного кода будет выглядеть так:

https://t.me/javalib

```
.
├── build.gradle.kts
├── gradle
│   └── wrapper
│       ├── gradle-wrapper.jar
│       └── gradle-wrapper.properties
├── gradlew
├── gradlew.bat
├── settings.gradle.kts
└── src
    ├── main
    │   ├── java
    │   │   └── com
    │   │       └── wellgrounded
    │   │           └── Main.java
    │   └── kotlin
    │       └── com
    │           └── wellgrounded
    │               └── kotlin
    │                   └── MessageFromKotlin.kt
    └── test
        └── java
            └── com
                └── wellgrounded
                    └── MainTest.java
```

Дополнительный код на Kotlin размещается в подкаталогах kotlin

Поддержка Kotlin включается через плагин Gradle в файле build.gradle.kts:

```
plugins {
   application
   id("org.jetbrains.kotlin.jvm") version "1.6.10"
}
```

И это все! Благодаря гибкости Gradle плагин может изменить порядок сборки и добавить необходимые зависимости `kotlin-stdlib`, не требуя от нас дополнительных действий.

11.3.9. Тестирование

Задача `assemble`, о которой мы говорили ранее, компилирует и упаковывает основной код, но нужно также откомпилировать и запустить тесты. Для этого по умолчанию настроена задача `build`:

```
./gradlew build --dry-run
  :compileJava SKIPPED
  :processResources SKIPPED
  :classes SKIPPED
  :jar SKIPPED
  :assemble SKIPPED
  :compileTestJava SKIPPED
  :processTestResources SKIPPED
  :testClasses SKIPPED
```

https://t.me/javalib

```
  :test SKIPPED
  :check SKIPPED
  :build SKIPPED
```

Добавим тестовый сценарий в стандартную структуру каталогов:

```
src
  └── test
      └── java
          └── com
              └── wellgrounded
                  └── MainTest.java
```

Затем необходимо добавить тестовый фреймворк в подходящую конфигурацию зависимостей, чтобы он был доступен для нашего кода. Также надо сообщить Gradle, что при запуске тестовых задач следует использовать JUnit:

```
dependencies {
  ....
  testImplementation("org.junit.jupiter:junit-jupiter-api:5.8.1")
  testRuntimeOnly("org.junit.jupiter:junit-jupiter-engine:5.8.1")
}

tasks.named<Test>("test") {
  useJUnitPlatform()
}
```

Конфигурация `testImplementation` делает библиотеку `org.junit.jupiter` доступной при сборке и выполнении тестов, но не основного кода. Когда после этого мы запустим команду `./gradlew build`, то увидим, что библиотека загружается в локальный кэш, если ее раньше там не было.

Полные листинги с трассировками стека, включая отчет в формате HTML, сохраняются в каталоге `build/reports/test`.

11.3.10. Автоматизация статического анализа

Сборка — отличное место для того, чтобы добавлять функциональность, которая помогает обезопасить проект. Одним из видов проверок, помимо модульного тестирования, является статический анализ. В этой категории существуют разные инструменты, но начать проще со SpotBugs (https://spotbugs.github.io/) — это наследник FindBugs. У большинства инструментов такого рода есть плагины как для Maven, так и для Gradle, так что этот раздел всего лишь дает представление о возможностях:

```
plugins {
  application
  id("com.github.spotbugs") version "4.3.0"
}
```

Если намеренно создать проблему в коде (например, реализовать `equals` для класса, не переопределяя `hashCode`), типичная проверка `./gradlew` сообщит об обнаруженной проблеме:

https://t.me/javalib

```
~:./gradlew check

  > Task :spotbugsTest FAILED
  FAILURE: Сборка не удалась, возбуждено исключение

  * Что пошло не так:
  Не удалось выполнить задачу ':spotbugsTest'.
  > Сбой при выполнении SpotBugsRunnerForWorker
     > Проверка не прошла: обнаруженных нарушений SpotBugs:
        2. Отчеты SpotBugs находятся в build/reports/spotbugs/test.xml
* Что можно сделать:
Запустить с параметром --stacktrace, чтобы получить трассировку стека
Запустить с параметром --info или --debug для более подробного журналирования
Запустить с параметром --scan, чтобы получить полную подсказку

* За дополнительной информацией обращайтесь по адресу https://help.gradle.org

BUILD FAILED in 1s
5 actionable tasks: 3 executed, 2 up-to-date
```

Как и при сбоях модульных тестов, файлы отчетов находятся в каталоге build/reports/spotbugs. В исходном состоянии SpotBugs генерирует только файлы XML, которые удобны для компьютеров, но не так полезны для большинства людей. Плагин можно настроить так, чтобы он генерировал отчет в формате HTML:

```
tasks.withType<com.github.spotbugs.snom.SpotBugsTask>()
  .configureEach {
    reports.create("html") {
      isEnabled = true
      setStylesheet("fancy-hist.xsl")
    }
}
```

tasks.withType выполняет поиск задач типобезопасным способом

configureEach выполняет блок так, как если бы мы написали tasks.spotbugsMain { }, а затем tasks.spotbugsTest { } с тем же кодом

Остальная конфигурация взята из документации проекта на GitHub (http://mng.bz/Qvdm)

11.3.11. За пределами Java 8

В главе 1 мы упоминали о модулях, которые по смыслу относились к Java Enterprise Edition, но были включены в основной JDK. В JDK 9 они были объявлены устаревшими, а в JDK 11 исключены, но остались доступными в виде внешних библиотек:

- `java.activation` (JAF)
- `java.corba` (CORBA)
- `java.transaction` (JTA)
- `java.xml.bind` (JAXB)
- `java.xml.ws` (JAX-WS, а также некоторые сопутствующие технологии)
- `java.xml.ws.annotation` (Common Annotations)

https://t.me/javalib

Если ваш проект зависит от каких-либо из этих модулей, переход на более новую версию JDK может привести к сбоям при сборке. К счастью, проблема решается, если добавить несколько простых зависимостей в файл build.gradle.kts:

```
dependencies {
  implementation("com.sun.activation:jakarta.activation:1.2.2")
  implementation("org.glassfish.corba:glassfish-corba-omgapi:4.2.1")
  implementation("javax.transaction:javax.transaction-api:1.3")
  implementation("jakarta.xml.bind:jakarta.xml.bind-api:2.3.3")
  implementation("jakarta.xml.ws:jakarta.xml.ws-api:2.3.3")
  implementation("jakarta.annotation:jakarta.annotation-api:1.3.5")
}
```

11.3.12. Gradle и модули

Как и Maven, Gradle полностью поддерживает систему модулей JDK. Давайте посмотрим, что нужно изменить, чтобы собирать модульные проекты с Gradle.

Модульная библиотека

У модульной библиотеки обычно бывает два основных структурных различия: в каталоге src имя каталога main меняется на имя модуля, а в корневой каталог модуля добавляется файл module-info.java:

```
.
├── build.gradle.kts
├── gradle
│   └── wrapper
│       ├── gradle-wrapper.jar
│       └── gradle-wrapper.properties
├── gradlew
├── gradlew.bat
├── settings.gradle.kts
└── src
    └── com.wellgrounded.modlib
        ├── java
        │   ├── com
        │   │   └── wellgrounded
        │   │       ├── hidden
        │   │       │   └── CantTouchThis.java
        │   │       └── visible
        │   │           └── UseThis.java
        │   └── module-info.java
        └── resources
```

Имя каталога соответствует имени модуля

Этот пакет останется скрытым

Объявления module-info.java для этого модуля

Этот пакет будет экспортироваться для использования за пределами модуля

Gradle не находит измененный каталог с исходным кодом автоматически, поэтому в файле build.gradle.kts надо указать, где его искать:

```
sourceSets {
  main {
    java {
```

https://t.me/javalib

```
      setSrcDirs(listOf("src/com.wellgrounded.modlib/java"))
    }
  }
}
```

Файл module-info.java содержит типичные объявления, которые мы демонстрировали ранее в этой главе и в главе 2. Мы зададим имя модуля и выберем один пакет (но не оба!) для экспортирования:

```
module com.wellgrounded.modlib {
    exports com.wellgrounded.modlib.visible;
}
```

И это все, что нужно сделать, чтобы библиотеку можно было использовать как модуль. Затем мы обратимся к библиотеке из модульного приложения.

Модульное приложение

Когда речь заходит о тестировании модульного приложения в Maven, то чтобы обеспечить доступ к нашей библиотеке из приложения, проще всего установить ее в локальном репозитории Maven. Для этого в Gradle можно использовать плагин `maven-publish`, но есть и другой способ, механику которого стоит понимать.

Наше модульное приложение имеет стандартную структуру, изображенную ниже. Чтобы его было проще тестировать, мы расположим каталоги верхнего уровня рядом друг с другом:

```
mod-lib                                  <-- Исходный код библиотеки mod-lib размещается
└── ...                                      на одном уровне с приложением mod-app

mod-app
├── build.gradle.kts
├── gradle
│   └── wrapper
│       ├── gradle-wrapper.jar
│       └── gradle-wrapper.properties
├── gradlew
├── gradlew.bat
├── settings.gradle.kts
└── src
    └── com.wellgrounded.modapp          <-- Имя каталога соответствует
        └── java                             имени модуля
            ├── com
            │   └── wellgrounded
            │       └── Main.java
            └── module-info.java         <-- Файл module-info.java — признак того,
                                             что приложение является модульным
```

В файле module-info.java задается имя модуля и его требования:

```
module com.wellgrounded.modapp {         <-- Имя модуля
    requires com.wellgrounded.modlib;    <-- Модуль требует пакет, который экспортируется
}                                            из нашей библиотеки
```

https://t.me/javalib

Чтобы протестировать локальную библиотеку, а не устанавливать ее, мы пока будем обращаться к ней локально, как показано в следующем фрагменте кода. Для этого можно воспользоваться функцией `files` в том месте, где мы раньше указывали координаты GAV для нашей зависимости. Разумеется, такой метод не годится в ситуации, когда все будет готово к распространению и развертыванию; это всего лишь простой прием для локального тестирования:

```
dependencies {
  implementation(files("../mod-lib/build/libs/gradle-mod-lib.jar"))
}
```

Текущим версиям Gradle нужна подсказка о том, что мы поручаем им определить, какие зависимости являются модульными, чтобы правильно разместить их в пути к модулям вместо classpath. Возможно, со временем такой режим станет использоваться по умолчанию, но на момент написания книги (Gradle 7.3) его нужно запросить явно:

```
java {
  modularity.inferModulePath.set(true)
}
```

Наконец, как и в случае с библиотекой, необходимо сообщить Gradle, где находятся файлы, которые размещены не по стандартам Maven:

```
sourceSets {
  main {
    java {
      setSrcDirs(listOf("src/com.wellgrounded.modapp/java"))
    }
  }
}
```

После всего этого `./gradlew build run` выдает ожидаемый результат. Если вы попытаетесь использовать библиотечный пакет, который не экспортировался, возникнет ошибка на стадии компиляции:

```
> Task :compileJava FAILED
/mod-app/src/com.wellgrounded.modapp/java/com/wellgrounded/Main.java:4:
error: package com.wellgrounded.modlib.hidden is not visible

import com.wellgrounded.modlib.hidden.CantTouchThis;

  (package com.wellgrounded.modlib.hidden is declared in module
   com.wellgrounded.modlib, which does not export it)
1 error
```

JLink

Как мы узнали в главе 2, модули позволяют создать оптимизированную среду, в которой работает приложение и которая содержит только необходимые зависимости. Это возможно благодаря тому, что система модулей предоставляет конкретные гарантии относительно того, какие модули использует код, так что

https://t.me/javalib

вспомогательные инструменты могут сконструировать необходимый минимальный набор модулей.

Эту возможность наиболее ясно демонстрирует программа `jlink`. Для модульного приложения она может сгенерировать полностью функциональный образ JVM, который будет выполняться независимо от JVM, установленной в системе.

> **ПРИМЕЧАНИЕ** JLink работает только с полностью модуляризованными приложениями. Если приложение загружает хотя бы часть кода через classpath, JLink не сможет успешно построить полный и безопасный образ.

Вернемся к приложению из главы 2, на примере которого мы знакомились с JLink, и посмотрим, как плагины Gradle упрощают управление проектом. Приложение, код которого доступен в сопроводительных материалах книги, использует классы JDK, чтобы подключиться ко всем работающим процессам JVM на компьютере и вывести разнообразную информацию о них.

В модульном приложении, которое мы собираемся упаковать, стоит обратить особое внимание на объявления в его собственном файле module-info.java. Они указывают, какие компоненты JLink должен добавить в свой образ, чтобы наша сборка успешно работала:

```
module wgjd.discovery {
  exports wgjd.discovery;

  requires java.instrument;
  requires java.logging;
  requires jdk.attach;
  requires jdk.internal.jvmstat;
}
```

Внимание: здесь фигурирует пакет jdk.internal, к которому мы обращаемся!

Но прежде чем мы возьмемся за JLink, понадобится еще кое-что настроить, чтобы перейти от ручной компиляции к сборке Gradle. Сначала нужно применить изменения модуляризации, которые рассматривались в предыдущем разделе. Но даже после этого программа все равно не будет компилироваться:

```
~:./gradlew build

> Task :compileJava FAILED
/gradle-jlink/src/wgjd.discovery/wgjd/discovery/VMIntrospector.java:4:
error: package sun.jvmstat.monitor is not visible[1]
  import sun.jvmstat.monitor.MonitorException;
                  ^
  (package sun.jvmstat.monitor is declared in module jdk.internal.jvmstat,
   which does not export it to module wgjd.discovery)

/gradle-jlink/src/wgjd.discovery/wgjd/discovery/VMIntrospector.java:5:
error: package sun.jvmstat.monitor is not visible
```

[1] Ошибка: пакет sun.jvmstat.monitor не является видимым. — *Примеч. пер.*

https://t.me/javalib

```
  import sun.jvmstat.monitor.MonitoredHost;
                    ^
  (package sun.jvmstat.monitor is declared in module jdk.internal.jvmstat,
   which does not export it to module wgjd.discovery)

/gradle-jlink/src/wgjd.discovery/wgjd/discovery/VMIntrospector.java:6:
error: package sun.jvmstat.monitor is not visible
  import sun.jvmstat.monitor.MonitoredVmUtil;
                    ^
  (package sun.jvmstat.monitor is declared in module jdk.internal.jvmstat,
   which does not export it to module wgjd.discovery)

/gradle-jlink/src/wgjd.discovery/wgjd/discovery/VMIntrospector.java:7:
error: package sun.jvmstat.monitor is not visible
  import sun.jvmstat.monitor.VmIdentifier;
                    ^
  (package sun.jvmstat.monitor is declared in module jdk.internal.jvmstat,
   which does not export it to module wgjd.discovery)

4 errors

FAILURE: Build failed with an exception.
```

Система модулей сообщает, что мы нарушаем правила, пытаясь использовать классы из `jdk.internal.jvmstat`. Наш модуль `wgjd.discovery` не входит в список допустимых модулей `jdk.internal.jvmstat`. Если мы сознательно хотим нарушить правила и понимаем риски, на которые идем, можно воспользоваться ключом `--add-exports`, чтобы включить модуль в список. Эта регулировка осуществляется с помощью флага компилятора и в конфигурации Gradle выглядит так:

```
tasks.withType<JavaCompile> {
  options.compilerArgs = listOf(
      "--add-exports",
      "jdk.internal.jvmstat/sun.jvmstat.monitor=wgjd.discovery")
}
```

На этот раз компиляция проходит нормально, и мы можем обратиться к JLink, чтобы упаковать результат. В наше время наибольшей популярностью пользуется плагин `org.beryx.jlink`, который в документации называется «The Badass JLink Plugin» (https://badass-jlink-plugin.beryx.org). Мы добавим его в проект Gradle при помощи строки плагина.

```
plugins {
  id("org.beryx.jlink") version("2.23.3")
}
```

Этот плагин автоматически применяет application, так что нам не нужно повторять это объявление

После того как мы добавили это объявление, в списке появляется задача `jlink`, которую можно немедленно запустить. Результат появится в каталоге build/image:

https://t.me/javalib

```
build/image/
├── bin
|   ├── gradle-jlink
|   ├── gradle-jlink.bat
|   ├── java
|   └── keytool
├── conf
|   └── ... various configuration files
├── include
|   └── ... require headers
├── legal
|   └── ... license and legal information for all included modules
├── lib
|   └── ... library files and dependencies for our image
└── release
```

build/image/bin/java — это наша специализированная JVM, для которой доступны только зависимости модуля нашего приложения. Ее можно запустить из терминала точно так же, как вы обычно запускаете команду `java`:

```
~:build/image/bin/java -version
openjdk version "11.0.6" 2020-01-14
OpenJDK Runtime Environment AdoptOpenJDK (build 11.0.6+10)
OpenJDK 64-Bit Server VM AdoptOpenJDK (build 11.0.6+10, mixed mode)
```

Можно передавать наш модуль этой JVM при запуске, но вместо этого можно запускать удобный стартовый сценарий, который плагин услужливо сгенерировал в каталоге build/image/bin/gradle-jlink. (Имя сценария соответствует имени проекта; сам сценарий приводится ниже.) Впрочем, с только что созданным образом не все в порядке:

```
~:build/image/bin/gradle-jlink

Java processes:
PID Display Name VM Version Attachable
Exception in thread "main" java.lang.IllegalAccessError:
 class wgjd.discovery.VMIntrospector (in module wgjd.discovery) cannot
   access class sun.jvmstat.monitor.MonitorException (in module
   jdk.internal.jvmstat) because module jdk.internal.jvmstat does not
   export sun.jvmstat.monitor to module wgjd.discovery
 wgjd.discovery/wgjd.discovery.VMIntrospector.accept(VMIntrospector.java:19)
 wgjd.discovery/wgjd.discovery.Discovery.main(Discovery.java:26)
```

Сообщение об ошибке выглядит знакомо — это еще одно проявление той же проблемы с доступом, которую мы раньше решали с помощью параметров компилятора. По-видимому, при запуске приложения тоже нужно сообщать о наших махинациях с модулем. К счастью, плагин позволяет широко настраивать параметры — как для запуска `jlink`, так и для сгенерированных итоговых сценариев:

https://t.me/javalib

```
jlink {
  launcher{
    jvmArgs = listOf(
                "--add-exports",
                "jdk.internal.jvmstat/sun.jvmstat.monitor=wgjd.discovery")
  }
}
```

С этим добавлением сценарий запуска работает нормально:

```
~:build/image/bin/gradle-jlink
Java processes:
PID    Display Name    VM Version       Attachable
833 wgjd.discovery/wgjd.discovery.Discovery       11.0.6+10 true
276 org.jetbrains.jps.cmdline.Launcher /Applications/IntelliJ IDEA CE.app...
```

Стоит заметить, что образ, который генерирует JLink, по умолчанию предназначается для той же операционной системы, в которой выполняется JLink, — это демонстрируется в следующем примере кода. Однако это не обязательно: можно создавать и кросс-платформенные образы. Главное, чтобы были доступны файлы из установки JDK для целевой платформы. Эти файлы легко загружаются из таких источников, как сайт Eclipse Adoptium по адресу https://adoptium.net/:

```
jlink {
  targetPlatform("local",
                 System.getProperty("java.home"))
  targetPlatform("linux-x64",
                 "/linux_jdk-11.0.10+9")

  launcher{
    jvmArgs = listOf(
                 "--add-exports",
                 "jdk.internal.jvmstat/sun.jvmstat.monitor=wgjd.discovery")
  }
}
```

Собирает образ для платформы, на которой работает локальный JDK

Собирает образ для загруженного Linux JDK

Когда вы начинаете создавать образы для конкретных платформ, плагин добавляет дополнительные каталоги в build/image. Очевидно, чтобы протестировать эти результаты, вам придется перенести их в соответствующую систему.

Последнее возможное препятствие при использовании JLink — ограничения, которые связаны с автоматически назначаемыми именами модулей. Хотя для миграции было бы полезно, если бы можно было просто включить имя в манифест файла JAR, JLink не поддерживает такую возможность.

Впрочем, плагин Badass JLink вам с этим поможет. Он перепаковывает любые модули с автоматически назначенными именами в модуль, который JLink способен обработать. Эта возможность подробно описана в документации (*http://mng.bz/XZ2Y*), и от нее может зависеть работоспособность JLink.

https://t.me/javalib

11.3.13. Настройка Gradle

Одна из самых сильных сторон Gradle — неограниченная гибкость. Без плагинов Gradle даже не имеет понятия о жизненном цикле сборки. Вы можете добавлять задачи и изменять конфигурацию существующих задач с минимумом ограничений. Нет необходимости держать в проекте каталог scripts со сторонними инструментами: благодаря Gradle ваши особые потребности можно интегрировать прямо в повседневный инструментарий сборки и тестирования.

Нестандартные задачи

Нестандартную задачу можно определить прямо в файле build.gradle.kts:

```
tasks.register("wellgrounded") {
  println("конфигурируем…")
  doLast {
    println("Привет от Gradle")
  }
}
```

Если запустить задачу, вы получите следующий результат:

```
~: ./gradlew wellgrounded
  конфигурируем…

  > Task :wellgrounded
  Привет от Gradle
```

Строка `println("конфигурируем…")` запускается во время подготовки задачи, но содержимое блока `doLast` выполняется при фактическом запуске задачи. Чтобы убедиться в этом, можно провести пробный запуск:

```
~: ./gradlew wellgrounded --dry-run
  конфигурируем…
  :wellgrounded SKIPPED
```

Задачи можно настраивать так, чтобы они зависели от других задач:

```
tasks.register("wellgrounded") {
  println("конфигурируем…")
  dependsOn("assemble")
  doLast {
    println("Привет от Gradle")
  }
}
```

Этот метод подходит и для задач, которые вы не создавали: проведите поиск и добавьте задачу как зависимость:

```
  tasks {
    named<Task>("help") {
      dependsOn("wellgrounded")
    }
  }
```

https://t.me/javalib

```
~: ./gradlew help
  конфигурируем…

  > Task :wellgrounded
  Привет от Gradle

  > Task :help

  Welcome to Gradle 7.3.3.

  To run a build, run gradlew <task> ...

  To see a list of available tasks, run gradlew tasks

  To see more detail about a task, run gradlew help --task <task>

  To see a list of command-line options, run gradlew --help

  For more detail on using Gradle, see
    https://docs.gradle.org/7.3.3/userguide/command_line_interface.html

  For troubleshooting, visit https://help.gradle.org
```

Писать нестандартные задачи прямо в файле сборки — исключительно мощная возможность. Однако если размещать их в build.gradle.kts, вы столкнетесь с довольно жесткими ограничениями: такие задачи не удается совместно использовать в разных проектах, и для них трудно писать автоматизированные тесты. Чтобы избежать таких проблем, можно создавать плагины Gradle.

Создание собственных плагинов

Плагины Gradle реализуются в коде JVM. Их можно размещать непосредственно в проекте в виде исходных файлов, а можно подключать через библиотеки. Многие плагины были написаны на Groovy (Gradle изначально поддерживал этот язык сценариев), но сейчас можно использовать любой язык для JVM. Если вы планируете распространять свой плагин, то имеет смысл писать его на Java, чтобы обеспечить наибольшую совместимость и минимизировать проблемы с идиомами конкретных языков.

Плагины можно кодировать прямо в сценарии сборки, и мы продемонстрируем основные API, которые используют этот прием. А когда вы будете готовы распространять код, его можно выделить в отдельный проект. Вот эквивалент уже знакомой задачи `wellgrounded`:

```
class WellgroundedPlugin : Plugin<Project> {
    override fun apply(project: Project) {
        project.task("wellgrounded") {
            doLast {
                println("Привет от Gradle")
            }
        }
    }
}

apply<WellgroundedPlugin>()
```

Наследует Plugin

Использует уже знакомые API уровня проекта и реализацию задачи

Применяйте apply, чтобы использовать плагин: он не вызывается автоматически, как предыдущее определение задачи

https://t.me/javalib

Если оформлять задачи в виде плагинов, это не только облегчает совместный доступ, но и открывает больше возможностей для настройки конфигурации. В стандартном объекте Gradle, который представляет проект, предусмотрено конкретное место, чтобы хранить конфигурации плагинов, — свойство `extensions`. К этим расширениям можно добавлять наши собственные объекты `Extension`:

```
open class WellgroundedExtensions {
  var count: Int = 1
}

class WellgroundedPlugin : Plugin<Project> {
  override fun apply(proj: Project) {
    val extensions = proj.extensions
    val ext = extensions.create<WellgroundedExtensions>("wellgrounded")
    proj.task("wellgrounded") {
      doLast {
        repeat(ext.count) {
          println("Привет от Gradle")
        }
      }
    }
  }
}

apply<WellgroundedPlugin>()

configure<WellgroundedExtensions> {
  count = 4
}
```

В плагинах доступна вся мощь языка программирования.

Если выделить плагин в другую библиотеку, его можно будет включить в сборку при помощи того же механизма, с помощью которого мы раньше подключали плагин SpotBugs:

```
plugins {
  id("com.wellgrounded.gradle") version "1000.0"
}

apply<WellgroundedPlugin>()

configure<WellgroundedExtensions> {
  count = 4
}
```

ИТОГИ

- Средства сборки играют фундаментальную роль в построении программных продуктов на Java в реальном мире. Они автоматизируют рутинные операции, упрощают управление зависимостями, помогают разработчикам согласованно выполнять свои задачи и, что особенно важно, гарантируют,

https://t.me/javalib

что при сборке одного проекта на разных машинах будут получены одинаковые результаты.

- Maven и Gradle — самые распространенные системы сборки в экосистеме Java, и большинство задач можно решать в любой из этих систем.
- Maven определяет конфигурацию в формате XML в сочетании с плагинами, написанными на языках JVM.
- Gradle предоставляет декларативный язык сборки на основе реального языка программирования (Kotlin или Groovy), который обеспечивает компактную запись логики сборки для простых случаев и гибкость для сложных.
- Разрешение конфликтов зависимостей — одна из главных функций в любой системе сборки. И Maven, и Gradle предоставляют средства, которые помогают разрешать конфликты версий библиотек. Gradle обеспечивает более мощные инструменты для борьбы с типичными проблемами управления зависимостями.
- В Gradle есть средства повышения производительности — например, режим инкрементной сборки, который ускоряет сборку.
- Модули, о которых шла речь в главе 2, требуют некоторых изменений в сценариях сборки и в структуре исходного кода, однако средства сборки успешно поддерживают эти изменения.

https://t.me/javalib

12 Выполнение Java в контейнерах

В ЭТОЙ ГЛАВЕ

- ✓ Почему контейнерно-ориентированная разработка важна для опытного разработчика Java?
- ✓ Различия между ОС, виртуальной машиной, контейнером и инструментами координации
- ✓ Docker
- ✓ Kubernetes
- ✓ Практическое руководство по выполнению рабочей нагрузки Java в контейнерах
- ✓ Производительность и наблюдаемость в контейнерах

Контейнеры Docker (https://www.docker.com/) стали фактическим стандартом упаковки Java-приложений для развертывания, а Kubernetes (https://kubernetes.io/) (k8s) остается самым популярным инструментом координации этих контейнеров. Вам необходимо знать об этих технологиях, тем более о том, как они взаимодействуют с Java, особенно если вы развертываете ПО на мощностях любого крупного облачного провайдера.

ПРИМЕЧАНИЕ Хотя в этой нише существуют и другие продукты, Docker и Kubernetes занимают доминирующее положение на рынке контейнеров и средств координации соответственно.

https://t.me/javalib

12.1. ПОЧЕМУ КОНТЕЙНЕРЫ ВАЖНЫ ДЛЯ ОПЫТНЫХ РАЗРАБОТЧИКОВ

Чтобы лучше разобраться, что такое контейнеры и почему они важны для опытных Java-разработчиков, мы рассмотрим следующие темы:

- Управляющие операционные системы, виртуальные машины и контейнеры.
- Преимущества контейнеров.
- Недостатки контейнеров.

12.1.1. Управляющие операционные системы, виртуальные машины и контейнеры

С самого начала компьютерной эпохи мы добавляли слои абстракции между программным обеспечением и оборудованием, на котором оно работает. Контейнеры — очередной естественный шаг в этом направлении. Давайте обзорно рассмотрим эти слои абстракции, чтобы понять, какое место среди них занимают контейнеры.

Физическое оборудование

Начнем с самого нижнего уровня — *физического оборудования*, на котором не установлена управляющая операционная система. «Пустая» машина предоставляет любому программному продукту, который можно на ней запустить, конечный набор ресурсов: процессор, оперативную память, жесткий диск, сетевые коммуникации и т. д.

> **ПРИМЕЧАНИЕ** Концепция конечных ресурсов чрезвычайно важна. Слишком часто у разработчиков возникает иллюзия того, что контейнеры каким-то волшебным образом предоставляют в их распоряжение неограниченные ресурсы!

Всегда помните, что под операционной системой, виртуальными машинами и контейнерами находится уровень физического оборудования с *конечными ресурсами*.

Управляющая операционная система или гипервизор типа 1

В современных центрах обработки данных на физическом оборудовании устанавливается либо управляющая операционная система (например, Linux), либо гипервизор типа 1 (например, VMWare ESXi или Microsoft Hyper-V). *Гипервизором* (hypervisor) называется программный продукт, который позволяет создавать виртуальные машины и управлять ими. Гипервизоры могут функционировать на разных уровнях стека. Гипервизор типа 1 устанавливается непосредственно на физическом оборудовании и работает как облегченная операционная система, которая выделяет бóльшую часть аппаратных ресурсов в распоряжение запущенных на ней виртуальных машин.

https://t.me/javalib

Системы, которые работают на первом уровне (традиционные ОС или гипервизоры), обычно бывают легковесными, а их функциональность в основном сводится к тому, чтобы обеспечивать безопасность и поддерживать абстракции более высоких уровней, которые устанавливаются поверх этих систем. Естественно, на работу управляющей операционной системы расходуется некоторая доля ресурсов процессора, оперативной памяти и сети.

Гипервизоры типа 2

Если на физическом оборудовании установлена традиционная операционная система (например, Linux), то на следующем уровне обычно находится гипервизор типа 2. Гипервизоры (типа 1 или типа 2) отвечают за управление ресурсами нижележащего оборудования для виртуальных машин (VM) с гостевыми операционными системами.

Типичный пример: на физической машине с 32 Гбайт оперативной памяти и 16-ядерным процессором установлена управляющая ОС Linux, а под ней работает гипервизор типа 2. В свою очередь, он управляет четырьмя VM, и на каждой из них выполняется гостевая операционная система Linux, которая как будто оснащена 8 Гбайт оперативной памяти и четырьмя вычислительными ядрами. Современные гипервизоры обычно сами потребляют мало ресурсов. Если гипервизор типа 1 установлен непосредственно на физическом оборудовании, то под управлением этого гипервизора можно запускать системы следующего уровня (виртуальные машины) без дополнительного вмешательства.

Виртуальные машины

Каждая виртуальная машина полностью автономна. С точки зрения пользователя, у нее есть собственный процессор, оперативная память, сетевые и дисковые ресурсы. Когда вы подключаетесь к серверу в среде реальной эксплуатации, вполне вероятно, что вы имеете дело с VM, а не с физическим сервером.

В автономной VM работает собственная ОС, которая называется *гостевой операционной системой*. В прошлом виртуальным машинам приходилось расплачиваться производительностью за поддержку подобных изолированных сред, но со временем развитие технологий позволило преодолеть многие проблемы.

Помните, что мы говорили о конечных ресурсах? Каждая виртуальная машина является именно виртуальной. Если гипервизор настроен неправильно, или если виртуальным машинам выделяется больше ресурсов, чем существует физически, или если вам приходится делить виртуальную машину с другими пользователями (очень частая ситуация в облачных средах), то производительность становится непредсказуемой.

Контейнерные движки

В прежние времена контейнерные движки обычно запускались поверх гостевой операционной системы. На одном движке могло выполняться несколько контейнеров.

https://t.me/javalib

На этом уровне проявляется одно из главных различий между виртуальной машиной и контейнерами, потому что одна из ключевых функций контейнерного движка заключается в том, чтобы обеспечить всем выполняемым контейнерам совместный доступ к одному и тому же ядру операционной системы. Эта конфигурация гораздо менее ресурсоемка, чем модель VM, где каждому экземпляру предоставляется полная копия ОС. Впрочем, это преимущество требует серьезной поддержки со стороны разных компонентов самого́ ядра Linux.

Контейнеры

Наконец-то мы добрались до контейнеров. Контейнер можно рассматривать как специально сконструированную изолированную среду, в которой запускается приложение. У контейнера есть своя файловая система, и в нем выполняется как минимум один процесс. Хотя процессы в контейнере могут взаимодействовать с ядром ОС, существует много ограничений, которые обеспечивают изоляцию контейнера от окружающего мира: ограничения на использование памяти, процессора, сети (как по нагрузке, так и по видимости), дискового пространства и т. д.

В контейнере можно запустить приложение на Java, хранилище данных или другую службу, которая вам нужна. Давайте посмотрим на все уровни абстракции, которые при этом задействуются.

На рис. 12.1 на нижнем уровне абстракции находится управляющая операционная система. Над ней расположен гипервизор, а поверх него — гостевая ОС, контейнерный движок, контейнер и приложение на Java. Вам не кажется, что это перебор? Для чисто контейнерной среды так и есть, поэтому в последние годы стали появляться выделенные контейнерные среды (рис. 12.2), в которых нет уровней гипервизора и гостевой ОС.

Так намного лучше! Между тем многие разработчики не до конца представляют себе, как устроена их целевая среда. Возможно, вам стоит связаться со своими системными администраторами, чтобы точно выяснить, какова она в вашем случае и какая часть конечных ресурсов физического оборудования выделяется на каждом уровне.



Рис. 12.1. Целевая среда для приложений Java

https://t.me/javalib



Рис. 12.2. Целевая среда для приложений Java в выделенной контейнерной среде

Несмотря на все сложности с уровнями абстракции, Java-разработчику имеет смысл сосредоточить свое внимание в основном на контейнере, который будет целевым объектом развертывания. У такого подхода есть ряд важных преимуществ.

12.1.2. Преимущества контейнеров

Если запуск контейнеров сопряжен с такими многоступенчатыми техническими ухищрениями, почему же они стали новым стандартом развертывания? Одно из ключевых преимуществ контейнеров заключается в том, что они способны применять ограничения, изолируя выполняемые процессы друг от друга. В прошлом, если вы развертывали два приложения Java на одном узле, возникала высокая вероятность, что они будут мешать друг другу — займут слишком много процессорного времени или захватят больше памяти, чем положено. С этим можно было бороться разными средствами, однако теперь эти средства встроены в архитектуру контейнеров. Собственно, когда есть система ограничений, которым можно доверять, это позволяет эффективнее использовать вычислительные ресурсы и безопасно запускать на узлах больше программ, чем удавалось до распространения контейнеров.

Изоляция — настолько важный фактор, что в оставшейся части этой главы отношения между контейнерами, узлами и процессами будут изображаться на диаграммах в виде вложенных, а не размещенных друг над другом компонентов. Впрочем, на практике успешно используются оба способа визуализации, и многое зависит от контекста.

Благодаря контейнерам индустрия также перешла к более систематичным принципам упаковки для развертывания. Раньше царил полный хаос в том, как компоненты приложения переносились в среду развертывания, как разработчики управляли зависимостями операционной системы и даже как они управляли запуском программы. Контейнеры наводят порядок во всех этих процедурах, поэтому прежние нагромождения инструментов и самодельных сценариев становятся ненужными. Контейнеры также изолируют среду разработки от содержимого контейнера. Контейнерному движку уже не нужно отвлекаться на то, как вы структурировали содержимое контейнера, ему достаточно знать, как запуститься, когда потребуется. Упаковка контейнерных образов — это важнейший пример схемы IaaS (Infrastructure as a Service, то есть «инфраструктура

https://t.me/javalib

как услуга») с декларативным описанием уровней системы, которое находится под управлением службы контроля версий и больше не требует скрупулезного императивного конструирования.

Наконец, еще одно преимущество, которое обусловлено этими систематичными принципами упаковки, — это экосистема, развившаяся вокруг контейнеров. В наши дни почти каждый значительный программный продукт, который вы запускаете, с большой вероятностью уже был упакован в контейнер на Docker Hub или где-то еще. Больше не нужны огромные файлы README с инструкциями по установке или нестандартные сценарии установки.

Но ведь не может быть, чтобы у контейнеров были только одни достоинства? Каковы слабые стороны выполнения в контейнерах?

12.1.3. Недостатки контейнеров

Как выясняется, первое указанное выше преимущество контейнеров — встроенная изоляция — становится одним из факторов, которые затрудняют их использование. Основная задача контейнера — изолировать свое содержимое от окружающего мира, однако вы, разработчик, тоже относитесь к этому окружающему миру. Многие приемы и средства, к которым вы привыкли за пределами контейнеров, приходится пересматривать и перенастраивать, чтобы использовать с контейнерами.

Эта проблема проявляется особенно ярко, когда вы интегрируете контейнеры в процессы локальной разработки. Может появиться ощущение, что увеличение длительности сборки и затраты времени на перемещения огромных контейнерных образов не всегда окупаются.

И хотя контейнеры предоставляют систематизированный интерфейс для упаковки и запуска приложений, развертывание в реальной ситуации иногда сталкивается с проблемами. Например, если приложение рассчитано на полный доступ к диску на своем узле, то может понадобиться специальная конфигурация, чтобы необходимые файлы стали доступны контейнеру. Если мы разделяем по контейнерам процессы, которые должны взаимодействовать друг с другом, то придется явно настроить механизмы их взаимодействия. Сохранение и применение конфигураций такого рода относится к числу важнейших задач систем координации, таких как Kubernetes. Однако следует помнить, что о системе Kubernetes, которая будет кратко представлена в этой главе, вполне можно написать целые книги, а ее экосистема продолжает стремительно развиваться.

Хотя контейнеры постепенно становятся технологией массового использования, опытный разработчик должен уметь анализировать их сильные и слабые стороны, чтобы подобрать правильный баланс для своей системы. В следующем разделе вы узнаете, как пользоваться этими инструментами, и получите представление о том, где их уместно применять.

https://t.me/javalib

12.2. ОСНОВЫ РАБОТЫ С DOCKER

Хотя многие технологии, на которые опираются контейнеры, существовали ранее, в Docker были добавлены удобные инструменты и абстракции, которые привели к массовому использованию контейнеризации. Здесь мы рассмотрим два основных аспекта функциональности, которые предоставляет Docker: построение образов и запуск контейнеров, а также разберемся, как использовать их в контексте практической Java-разработки.

12.2.1. Построение образов Docker

Контейнер Docker запускается из *образа*. Образ (image) — это фактически «моментальный снимок», который фиксирует все зависимости файловой системы, необходимые для выполнения программы. В образ входят платформенные библиотеки, среды выполнения языков, инструменты и, что самое важное, конкретная версия вашего программного продукта.

Docker-файл (обычно он называется Dockerfile) — типичный формат, в котором хранится последовательность действий для сборки образа. Dockerfile для простейшего возможного образа — абсолютно пустого — выглядит так:

```
FROM scratch
```

Образ строится командой `docker build`:

```
$ docker build .

[+] Building 0.1s (3/3) FINISHED
 => [internal] load build definition Dockerfile    0.0s
 => => transferring dockerfile: 55B                0.0s
 => [internal] load .dockerignore                  0.0s
 => => transferring context: 2B                    0.0s
 => exporting to image                             0.0s
 => writing image sha256:71de1148337f4d1845be0...  0.0s

Use 'docker scan' to run Snyk tests against images to find vulnerabilities
and learn how to fix them
```

Идентификатор 71de114… в формате SHA256 однозначно идентифицирует полученный образ. Вскоре вы увидите, как задать более удобное имя

Конечно, от пустого образа толку мало. На практике существует множество базовых образов, в которых уже установлены полезные программы. Стандартным источником этих образов является Docker Hub (https://hub.docker.com/). Позднее мы вернемся к тому, как выбрать правильный базовый образ Java, а пока посмотрим, как построить образ на основе версии Eclipse Temurin от OpenJDK, которую предоставляет Adoptium. Для этого возьмем образ `eclipse-temurin:11`, содержащий новейшую версию Java 11 на момент выхода книги:

```
FROM eclipse-temurin:11
RUN java -version
```

https://t.me/javalib

Свежие версии Docker по умолчанию динамически скрывают вывод, когда образ строится в интерактивном терминале. Чтобы лучше видеть, что происходит, мы воспользуемся ключом `--progress plain`:

```
$ docker build --progress plain .

=1 [internal] load build definition from Dockerfile
=1 sha256:261a2389333859f063c39502b306e984de49700a9...
=1 transferring dockerfile: 36B done
=1 DONE 0.0s

=2 [internal] load .dockerignore
=2 sha256:909e36a5a9cd7cc4e95e7926f84f982542233925d...
=2 transferring context: 2B done
=2 DONE 0.0s

=3 [internal] load docker.io/library/eclipse-temurin:11
=3 sha256:6a73b62137bbf64760945abf21baf23bf909644cf...
=3 DONE 0.5s

=4 [1/2] FROM docker.io/library/eclipse-temurin:11...
=4 sha256:f225b618d7ad96bd25e0182d6e89aa8e77643f42f...
=4 CACHED

=5 [2/2] RUN java -version
=5 sha256:556476b43b8626a27892422f8688979c4ba1e6029...
=5 0.38 openjdk version "11.0.13" 2021-10-19
=5 0.38 OpenJDK Runtime Environment Temurin-11.0.13+8 (build 11.0.13+8)
=5 0.38 OpenJDK 64-Bit Server VM Temurin-11.0.13+8 (build 11.0.13+8)
=5 DONE 0.4s

=6 exporting to image
=6 sha256:e8c613e07b0b7ff33893b694f7759a10d42e180f2...
=6 exporting layers 0.0s done
=6 writing image sha256:9796a789e295989cec550f... done
=6 DONE 0.0s

Use 'docker scan' to run Snyk tests against images to find vulnerabilities
and learn how to fix them
```

Внутренние действия, которые Docker выполняет при подготовке к сборке образов

Загружается запрошенный базовый образ

Наша команда RUN выполняется в процессе сборки, и мы видим ее вывод

Можно заметить, что код выполняется дольше — по крайней мере в первый раз, потому что Docker загружает соответствующий базовый образ с Docker Hub. Команда `RUN`, которую мы указали, добавляет новый шаг к сборке этого образа. `RUN` позволяет выполнить любую действительную команду в контейнерной среде. Если команда изменяет файловую систему, эти изменения сохраняются как часть итогового образа. Наш пример не изменяет файловую систему, но команду `RUN` также часто используют для того, чтобы загружать файлы (например, с помощью `curl`), устанавливать пакеты операционных систем при помощи стандартных менеджеров пакетов или вносить другие локальные изменения.

Другой важный аспект сборки образов Docker проявляется, если снова выполнить ту же команду сборки, не изменяя Dockerfile:

https://t.me/javalib

```
$ docker build --progress plain .

=1-4 excluded for length...

=5 [2/2] RUN java -version
=5 sha256:556476b43b8626a27892422f8688979c4ba1e602907a09d62a39a2
=5 CACHED

=6 exporting to image
=6 sha256:e8c613e07b0b7ff33893b694f7759a10d42e180f2b4dc349fb57dc
=6 exporting layers done
=6 writing image sha256:9796a789e295989cec5550fb3c17bc6c1d9c0867 done
=6 DONE 0.0s
```

Docker сообщает, что шаг пропущен, потому что результат уже есть в кэше

Каждая из начальных команд (таких, как `FROM` и `RUN`) в Dockerfile создает так называемый *уровень* (layer). Поскольку выполнение команд может занимать много времени, уровни кэшируются, и Docker всеми силами старается избежать лишней работы.

Теперь, когда в нашем контейнере есть среда Java, в нем можно запускать код. Мы создадим простой файл HelloDocker.java в том же каталоге, что Dockerfile. Для простоты вместо полной сборки мы пока используем механизм однофайлового выполнения Java. Основной код выглядит так:

```
public class HelloDocker {
  public static void main(String[] args) {
    System.out.println("Привет, Docker!");
  }
}
```

Затем можно сделать так, чтобы Docker включил этот файл в образ, и назначить команду по умолчанию для контейнеров, в которых запускается этот образ:

```
FROM eclipse-temurin:11
RUN java -version

COPY HelloDocker.java .

CMD ["java", "HelloDocker.java"]
```

Копирует наш файл в текущий рабочий каталог, назначенный Docker

Назначает команду по умолчанию для образа. Обратите внимание, что каждый аргумент командной строки содержится в отдельных кавычках

`COPY` (как и более сложная инструкция — `ADD`) берет файлы из локального окружения сборки и помещает их в контейнер. `ADD` поддерживает множество параметров, включая загрузку из удаленного источника и автоматическую распаковку файлов TAR, но, как правило, лучше использовать простую инструкцию `COPY` там, где ею можно обойтись.

`CMD` указывает на следующую стадию в жизненном цикле образа. Мы строим образы не просто так — в них должны выполняться программы. Как упоминалось ранее, у каждого образа есть уникальный идентификатор SHA256, но с такими идентификаторами неудобно работать и они изменяются при каждой сборке. Прежде чем запускать образ, назначим ему более практичное имя:

https://t.me/javalib

```
$ docker build -t hello .

... Предыдущие шаги сборки опущены для краткости

=8 exporting to image
=8 sha256:e8c613e07b0b7ff33893b694f7759a10d42e...
=8 exporting layers done
=8 writing image sha256:666fdc7613189865b9a5f2... done
=8 naming to docker.io/library/hello done
=8 DONE 0.0s
```

Идентификатор SHA256 нашего образа

Имя, которое мы назначили итоговому образу

Сейчас образ `hello` доступен только локально, но мы уже видели из строки `FROM`, что образы можно использовать совместно. Для этого служит так называемый *реестр контейнеров*. Когда мы запросили базовый образ `eclipse-temurin:11`, Docker по умолчанию обратился за этим образом в реестр Docker Hub (https://hub.docker.com/). Существуют и другие реестры контейнеров; более того, вы можете наладить собственный внутренний реестр, чтобы размещать образы своих приложений.

Сохранять образы в реестре и загружать их оттуда можно с помощью команд `docker push` и `docker pull` соответственно, как показано в следующем примере. Если вы работаете с нестандартным реестром, его надо указать перед именем образа и меткой:

```
$ docker pull k8s.gcr.io/echoserver:1.4
1.4: Pulling from echoserver
6d9e6e7d968b: Pull complete
...
7abee76f69c0: Pull complete
Digest: sha256:5d99aa1120524c801bc8c1a7077e8f5ec122ba16b6dda1a...
Status: Image is up to date for k8s.gcr.io/echoserver:1.4
k8s.gcr.io/echoserver:1.4
```

k8s.gcr.io — домен реестра, echoserver — имя образа, а 1.4 — метка

Если реестр требует аутентификации, для нее вам может понадобиться команда `docker login`. Впрочем, общедоступные образы на Docker Hub не требуют этого шага.

Можно еще многое рассказать о том, как строить качественные образы Docker, и мы вернемся к некоторым вопросам позднее. А пока посмотрим, как превратить эти образы в работающие контейнеры.

12.2.2. Запуск контейнеров Docker

При всех бурных обсуждениях и ажиотаже вокруг Docker и контейнеров основная идея проста: нам надо выполнить четко определенный процесс в жестко контролируемой среде. Эта среда в основном определяется образом, который вы создали. Контейнер можно запустить командой `docker run`:

```
$ docker run hello
Привет, Docker!
```

https://t.me/javalib

Выполняя эту команду, Docker создал новую файловую систему на основании нашего образа, применил различные ограничения (например, по ресурсам процессора и памяти), а затем запустил процесс по умолчанию, который мы задали в инструкции `CMD`. Наша программа выводит сообщение и завершается, но с таким же успехом она могла бы запустить сервер и продолжать работать неограниченно долго.

На рис. 12.3 демонстрируется процесс `java`, который мы указали в `CMD` для образа. Учтите, что узел, который здесь изображен как внешний уровень, на самом деле может быть отделен от физического оборудования множеством других уровней.

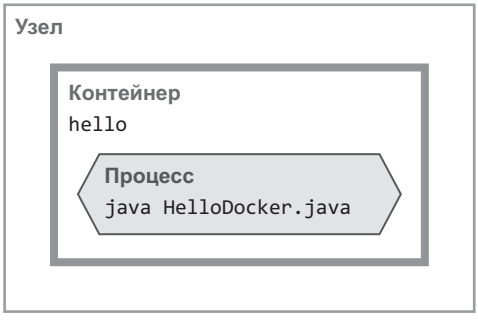


Рис. 12.3. Запуск базового контейнера

В `CMD` определяется команда, которая выполняется по умолчанию при запуске контейнера, однако Docker позволяет запустить образ и с любой другой командой на ваше усмотрение. Ранее упоминалось о том, что у контейнера есть текущий рабочий каталог, как у интерактивных терминалов. Чтобы узнать путь к этому каталогу, его можно запросить у контейнера командой `pwd`:

```
$ docker run hello pwd
/
```

Как показано на рис. 12.4, если при запуске контейнера выполняется альтернативная команда, то процесс, заданный в `CMD` по умолчанию, не запускается.

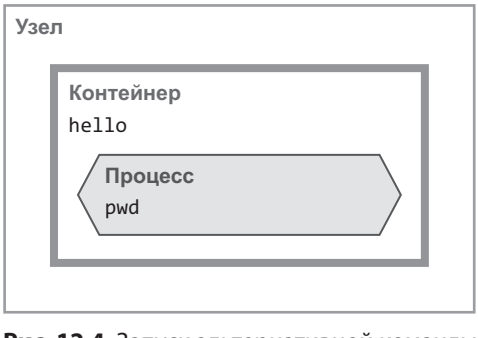


Рис. 12.4. Запуск альтернативной команды в контейнере

https://t.me/javalib

Конфигурацию можно «зашить» в образ в виде файлов, но часто бывает желательно определять ее во время выполнения. Один из принципов методологии 12-факторного приложения (https://12factor.net/) — авторитетного набора идей, которые касаются разработки приложений (в том числе контейнерных), — рекомендует определять конфигурацию на уровне *окружения*, чтобы один и тот же собранный ресурс (в нашем случае образ) можно было развертывать в других местах, не вмешиваясь в код.

Как показано в следующем фрагменте, при запуске контейнера его переменные окружения можно изменить с помощью флага `-e`, который можно передать несколько раз. В коде приложения эти переменные можно прочитать стандартными средствами, например методом `System.getenv()`:

```
$ docker run -e MY_VAR=here -e OTHER_VAR=there hello env
PATH=/opt/java/openjdk/bin:/usr/local/sbin:/usr/local/bin:...
HOSTNAME=f25762652561
MY_VAR=here
OTHER_VAR=there
LANG=en_US.UTF-8
LANGUAGE=en_US:en
LC_ALL=en_US.UTF-8
JAVA_VERSION=jdk-11.0.13+8
JAVA_HOME=/opt/java/openjdk
HOME=/root
```

Запускает стандартную команду env, чтобы просмотреть окружение контейнера

Полный список переменных окружения

Прежде чем переходить к более реалистичным примерам сборки приложений Java в контейнерах, рассмотрим еще один полезный прием — интерактивное выполнение образа. Раньше мы уже видели, как можно изменить команду, которая по умолчанию запускается в контейнере. Этот же механизм позволяет запустить в контейнере командную оболочку (например, `bash`) для дополнительной отладки. Для этого `docker run` необходимо передать специальные флаги, а именно `-i` для подключения `STDIN`, чтобы ввод достигал контейнера, и `-t`, чтобы контейнер запускал интерактивный сеанс TTY:

```
$ docker run -it hello bash
root@b770c2ac829c: ls *.java
HelloDocker.java
root@b770c2ac829c:
```

Входит в интерактивную командную оболочку, которая позволяет взаимодействовать с контейнером

Таким образом мы можем видеть окружающую среду точно так же, как ее видят приложения, которые развернуты в контейнере.

Скопировать в контейнер однофайловое приложение в стиле «Hello, World!» — это, конечно, большое достижение, но давайте рассмотрим более реалистичные сценарии совместного использования Docker и Java.

12.3. РАЗРАБОТКА ПРИЛОЖЕНИЙ НА JAVA С DOCKER

В этом разделе мы изучим различные практические аспекты разработки приложений на Java с Docker. Для начала мы чуть глубже рассмотрим базовые образы

https://t.me/javalib

JVM и разберемся, как строить собственные образы. Затем будут представлены некоторые соображения о том, как настраивать, запускать и осуществлять отладку контейнеров. В нашем контейнере должна откуда-то взяться JVM, что приводит нас к вопросу выбора базового образа.

12.3.1. Выбор базового образа

Какой базовый образ будет «самым правильным» для запуска вашего приложения на JVM? На этот вопрос нет однозначного ответа. Чтобы выяснить, какой образ вам подойдет, необходимо рассмотреть такие вопросы:

- Продукт какого разработчика вам нужен?
- Какая операционная система должна работать внутри контейнера?
- На каких системных архитектурах должен работать образ?

На выбор поставщика образа также влияет ряд факторов, которые мы упоминали в главе 1, например:

- Доступность службы поддержки и контракты.
- Политика обновлений безопасности и их своевременность.
- Особые аспекты развертывания на облачных платформах — сборка OpenJDK от Microsoft для Azure, Amazon Corretto для AWS.

Сборки, специфические для облачных провайдеров, базируются на OpenJDK, но могут включать средства оптимизации производительности и другие улучшения, которые приспособлены к облачным средам этого провайдера. Также они часто обеспечивают дополнительные преимущества, связанные с поддержкой и периодичностью выпуска.

Многие разработчики поддерживают в своих контейнерах более одной операционной системы. На практике часто встречаются Debian, Ubuntu или Alpine, а также другие разновидности Linux. От операционной системы часто зависит, через какой менеджер пакетов будут устанавливаться собственные зависимости и какие дополнительные инструменты будут доступны в контейнере. Если ваши требования не предусматривают какую-то конкретную операционную систему, стоит выбирать более массовые варианты (такие, как Debian или Ubuntu), чтобы испытывать меньше трудностей с поиском и обновлением пакетов.

> **ПРИМЕЧАНИЕ** Будьте особенно осторожны с Alpine Linux. До недавнего времени не было ни одного официального образа для Java на Alpine. Проверьте, предоставляет ли ваш поставщик Java образы для Alpine.

Если вам нужно работать в операционной системе, которую поставщик не предоставляет напрямую, не отчаивайтесь. В таких случаях можно самостоятельно построить образ, в котором вы установите JDK вручную с помощью

https://t.me/javalib

типичных системных средств управления пакетами. Помните, что базовые образы и сборки Docker всего лишь включают правильные составляющие в файловую систему контейнера. Часто того же результата можно добиться и другими способами.

И еще одно замечание о системной архитектуре образов. В последнее время микросхемы на базе ARM становятся все популярнее, особенно на облачных платформах. Хотя они позволяют добиться высокой производительности, учтите, что вам понадобятся образы, построенные специально для этой архитектуры. Если вам нужно запускать контейнеры на разных архитектурах, возможно, в итоге придется собрать и опубликовать несколько образов; впрочем, Docker уже довольно хорошо с этим справляется.

12.3.2. Сборка образа в Gradle

Как мы видели в главе 11, любому сколько-нибудь масштабному проекту на Java пойдет на пользу качественный инструментарий сборки. Для демонстрационных целей мы рассмотрим, как построить образ на базе сборки Gradle, но в сопроводительных материалах книги доступно аналогичное решение для Maven.

Образ должен содержать по крайней мере все файлы JAR приложения (или файлы классов) и все зависимости для classpath. В нашем примере приложение зависит от `org.apache.commons:commons-lang3`:

```
plugins {
  application
  java
}

application {
  mainClass.set("com.wellgrounded.Main")
}

tasks.jar {
  manifest {
    attributes("Main-Class" to application.mainClass)
  }
}

repositories {
  mavenCentral()
}

dependencies {
  implementation("org.apache.commons:commons-lang3:3.12.0")
}
```

Обычной команды `build` или `assemble` в этом случае недостаточно, но в настройках Gradle по умолчанию все необходимое упаковано в `installDist`:

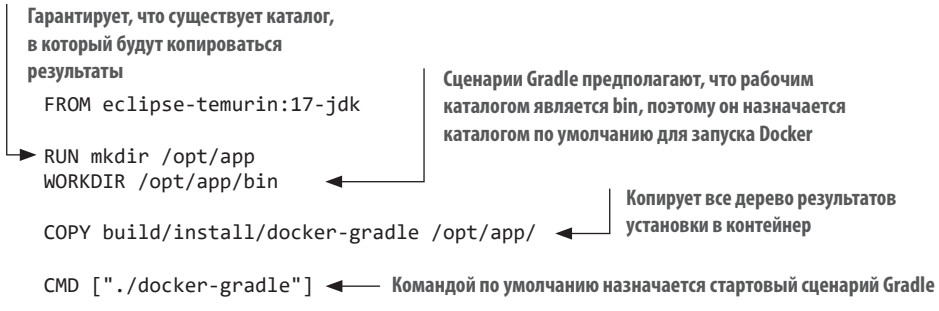
```
$ ./gradlew installDist
```

https://t.me/javalib

Вот упрощенный результат сборки, которую выполнила эта команда:

```
build
└── install
    └── docker-gradle
        ├── bin
        │   ├── docker-gradle
        │   └── docker-gradle.bat
        └── lib
            ├── commons-lang3-3.12.0.jar
            └── docker-gradle.jar
```

Мы могли бы просто взять файлы JAR и запускать их из контейнера, но в Gradle есть вспомогательные сценарии для запуска приложений. Попробуем ими воспользоваться:



За дополнительной информацией о стартовых сценариях Gradle обращайтесь к документации плагина `Application` (http://mng.bz/yvxJ).

Этот метод предполагает, что вы локально установили JDK для сборки с помощью Gradle, прежде чем копировать результаты в образ. В следующем разделе мы покажем, как полностью выполнить сборку в Docker.

12.3.3. Сборка в Docker

Главная задача контейнеров — создать изолированную, воспроизводимую среду, в которой будут выполняться ваши приложения. Это помогает эффективно развертывать службы, но этим преимущества контейнеров не исчерпываются. Многим проектам свойственна классическая проблема — как настроить локальную среду для разработки. Если вам когда-нибудь доводилось продираться через файлы README со сложными пошаговыми описаниями установки и следить за тем, чтобы у вас были правильные версии всех компонентов, то вы знаете, о чем идет речь. Контейнеры помогают избавиться от этих проблем. Посмотрим, как изменить процесс сборки, чтобы полнее использовать преимущества изоляции.

Пока в нашем Dockerfile фигурирует только один образ, который мы пытаемся сконструировать. Однако Docker позволяет определить в одном файле несколько образов, и что еще важнее, — копировать данные между ними. Благодаря

https://t.me/javalib

этому мы можем сконструировать образ, в котором будет собрано наше приложение, — и этот образ будет полностью отделен от JDK вашей локальной системы, — а затем скопировать результаты сборки в образ развертывания. Такое решение выигрывает и с точки зрения безопасности, и в отношении размера образа.

Этот процесс называется *многоступенчатой сборкой*, и вы можете наблюдать его в действии, когда Dockerfile содержит несколько инструкций `FROM`. Строки `FROM`, которые соответствуют промежуточным стадиям сборки, также включают ключевое слово `AS`, чтобы идентифицировать их для последующего использования в Dockerfile, тогда как основной итоговый образ остается прежним:

```
FROM eclipse-temurin:17-jdk AS build

RUN mkdir /project
WORKDIR /project

COPY . .

RUN ./gradlew clean installDist


FROM eclipse-temurin:17-jre

RUN mkdir /opt/app

COPY --from=build \
      /project/build/install/docker-gradle-multi \
      /opt/app/

WORKDIR /opt/app/bin
CMD ["./docker-gradle-multi"]
```

Контейнер с именем build, в котором будет запускаться компиляция

Создает каталог для исходного кода и назначает его рабочим каталогом по умолчанию

Копирует полный проект в контейнер

Собирает наше приложение (в данном случае Gradle-версию) — как и прежде, локально

Образ развертывания теперь может просто использовать JRE, которая заметно меньше

COPY --from=build берет файлы из образа build, а не из локальной файловой системы

Теперь, чтобы собрать приложение для развертывания, среде непрерывной интеграции требуется только Docker, без установки JDK. Как показано на рис. 12.5, все компоненты, необходимые для сборки, остаются полностью в пределах контейнеров.

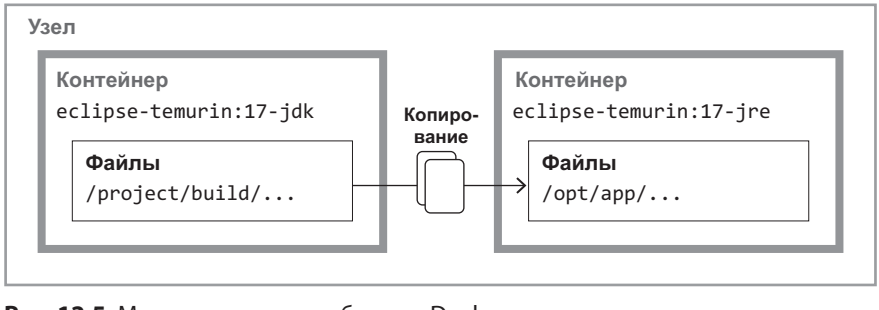


Рис. 12.5. Многоступенчатая сборка в Docker

https://t.me/javalib

Стоит указать, что такая конфигурация для подобных сборок близка к минимальной, но у нее есть обратная сторона — время сборки. Как упоминалось ранее, каждая команда Docker создает уровень, который кэшируется; но если не проявить необходимой осторожности, кэш может стать недействительным, когда это не нужно.

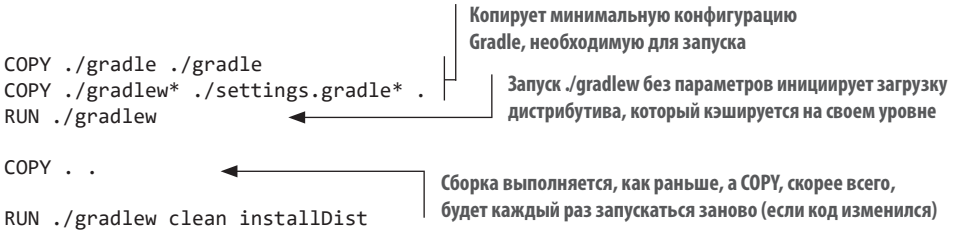
Например, кэширование может нарушиться из-за тех строк в Dockerfile, где мы копируем полный каталог проекта в контейнер. Если какой-нибудь файл меняется, даже совсем незначительно, то строка `COPY . .` становится недействительной и все, что следует за ней, приходится выполнять заново. Однако может оказаться, что некоторые локальные файлы никак не влияют на сборку: например, в образ контейнера сборки незачем включать историю `git`, файлы IDE и вывод локальной сборки. К счастью, эти файлы можно исключить из обработки Docker, для чего в одном каталоге с Dockerfile следует создать файл .dockerignore. Его простой формат может быть вам уже знаком, если вам доводилось работать с файлами .gitignore. Как показано в следующем фрагменте, каждая строка задает шаблон (в котором допускаются стандартные подстановочные символы командной оболочки), и при копировании Docker будет игнорировать файлы, имена которых соответствуют этим шаблонам:

```
.git
.idea/
*.iml
*.class

# Игнорировать папки сборки
out/
build/
target/
.gradle/
```

Вторая, менее очевидная проблема возникает с оберткой Gradle. Если посмотреть на вывод при запуске сборки, можно заметить, что значительное время тратится на то, чтобы загрузить правильный дистрибутив. Поскольку наши контейнеры запускаются без локального кэша Gradle, эта загрузка повторяется при каждом запуске.

Чтобы избежать повторных загрузок, надо разбить первое выполнение Gradle на отдельные уровни, которые размещаются до того, как весь проект копируется в контейнер. Требуется скопировать минимум данных, которые необходимы Gradle для выполнения загрузки, чтобы кэш этого уровня становился недействительным, только если изменяется обертка Gradle (например, если мы обновляем ее версию):



https://t.me/javalib

Это всего лишь малая часть разнообразных приемов оптимизации, которые можно применять при конструировании образов контейнеров. Главное здесь — тщательно продумать, какие компоненты относятся к тому или иному уровню. Если разные части вашей системы изменяются в разном темпе, их часто имеет смысл разнести по разным уровням.

Мы продемонстрировали достаточно топорный подход к построению образов Docker. Как нетрудно предположить, существует огромное количество плагинов для Maven и Gradle, которые помогут вам просто упаковать эту функциональность, не редактируя Dockerfile вручную. Бывают даже такие средства, как Jib (https://github.com/GoogleContainerTools/jib), которые вообще обходятся без инструментов Docker. Все они полезны, однако опытному разработчику стоит глубоко понимать, как строятся контейнеры, даже если он постоянно пользуется инструментами, которые упрощают эту процедуру.

12.3.4. Порты и узлы

Контейнеры предоставляют приложению не только изолированную файловую систему, но и сетевые ресурсы. Допустим, мы добавили в приложение код для запуска стандартного сервера HTTP, хотя бы базового, который предоставляется в JDK как `com.sun.net.httpserver.HttpServer`. Запустив контейнер командой `docker run`, мы обнаружим, что обратиться с запросом к этой конечной точке HTTP не удается.

Чтобы решить эту проблему, необходимо, чтобы Docker открыл доступ к порту. Этого можно добиться напрямую, добавив параметры в команду `run`:

```
$ docker run -p 8080:8080 hello
```

Параметр `-p` принимает пару портов, разделенных двоеточием. Первое значение — это порт, который должен быть доступен *вне* контейнера, а второе — порт, который наша программа прослушивает *внутри* контейнера. Если переключиться на другой терминал (или в браузер), то вы увидите, что подключение работает, как показано ниже и проиллюстрировано на рис. 12.6:

```
$ curl http://localhost:8080/hello
Hello from HttpServer
```

Как вы могли предположить по формату параметра и рис. 12.6, номера двух портов не обязаны совпадать. Если воспользоваться для запуска следующей командой:

```
$ docker run -p 9000:8080 hello
```

Порт 9000 будет виден за пределами контейнера и связан с портом 8080 процесса внутри контейнера

мы увидим ответ сервера через порт 9000, тогда как порт 8080 перестанет быть доступным:

https://t.me/javalib

```
$ curl http://localhost:9000/hello
Hello from HttpServer

$ curl http://localhost:8080/hello
curl: (7) Failed to connect to localhost port 8080: Connection refused
```

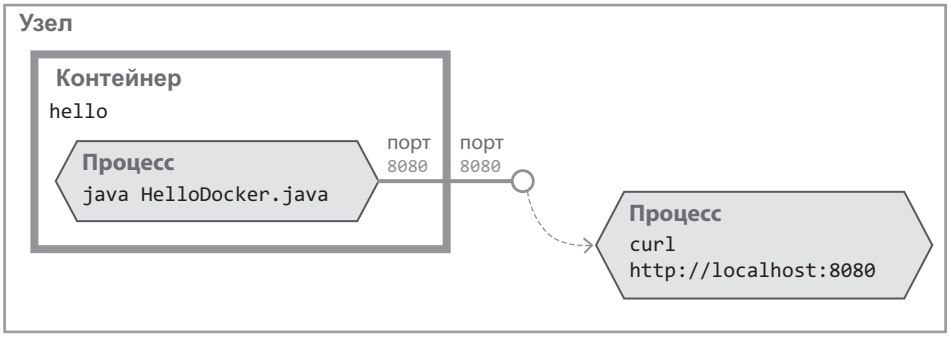


Рис. 12.6. Предоставление доступа к порту в Docker

Настройка доступа к портам является настолько фундаментальной частью развертывания контейнеров, что Dockerfile позволяет указать, какие порты должен предоставлять образ:

```
EXPOSE 8080
```

Если внести это изменение, снова выполнить `docker build` и запустить образ, не указывая порты, результат может оказаться неожиданным: Docker по умолчанию не открывает доступ к портам `EXPOSE`. Но если применить ключ `-P` сам по себе (в верхнем регистре и без аргументов), Docker связывает каждый порт `EXPOSE` в образе со случайным (*эфемерным*) портом. Поскольку никак нельзя угадать, какой порт будет назначен эфемерным, нам понадобится новая команда, чтобы его найти, — `docker ps`:

```
$ docker run -P hello

... В другом терминале, часть столбцов опущена...
$ docker ps
CONTAINER ID   IMAGE      COMMAND              PORTS
94d7f125caad   hello      "./docker-gradle"    0.0.0.0:55031->8080/tcp
```

Значение `0.0.0.0:55031->8080/tcp` говорит о том, что порт 55031 вне контейнера ассоциирован с внутренним портом 8080.

Возня с эфемерными портами на первый взгляд кажется неудобным излишеством, особенно во время тестирования, потому что порт изменяется. Но на самом деле эта особенность играет важную роль, когда контейнеры запускаются в условиях реальной эксплуатации. Представьте, что у вас есть узел, который вы хотите использовать максимально эффективно, запуская на нем много

https://t.me/javalib

контейнеров Java. Каждое из этих приложений может захотеть использовать один и тот же порт, но узел сможет назначить этот порт только кому-то одному. Хотя механизм эфемерных портов требует дополнительной координации с другими частями системы, он позволяет контейнерам сохранять упрощенное представление о мире («Я работаю на порте 8080») и при этом сосуществовать с другими приложениями в более широкой и сложной среде.

Итак, мы научились обращаться извне к нашему приложению, которое локально выполняется в контейнере. А как насчет обратного направления — когда нашему контейнеру потребуется обратиться к внешней службе (например, базе данных)?

В среде реальной эксплуатации рекомендуется явно задавать в конфигурации местонахождение служб и обращаться к ним через обычные балансировщики нагрузки и DNS. Сведения о внешних службах можно внедрить в контейнеры через переменные окружения или другие системы обнаружения служб, но главное, не делать никаких предположений о том, где расположены эти службы относительно вашего контейнера.

Правда, в локальной среде разработки все становится сложнее, потому что там подобная инфраструктура обычно недоступна. Если вы работаете с Docker для Mac или Windows, внутри контейнеров можно использовать имя `host.docker.internal`, которое автоматически указывает на ваш узел. В Docker для Linux можно задать ту же настройку, если запустить контейнер с флагом `--add-host host.docker.internal:host-gateway`. В таких случаях, если ваше приложение настроено так, чтобы получать соответствующую информацию о расположении через переменные окружения, вы можете из контейнера обращаться по этому имени узла.

На случай, если в вашей конкретной среде этот способ не работает, внутри контейнера существует IP-адрес для вашего узла. Такие команды, как `sudo ip addr show`, могут предоставить некоторые сведения о расположении, хотя такое решение оказывается слишком утомительным.

Контейнеры поддерживают множество сетевых настроек, выходящих за рамки темы этой книги, однако отдельные параметры, которые связаны с вышеописанным затруднением, можно регулировать с помощью программы Docker Compose. Давайте посмотрим, как контейнеры помогают решать проблемы с обращением к внешним ресурсам из локальной среды.

12.3.5. Локальная разработка с Docker Compose

Наряду с ужасающими списками установки для новых проектов многие приложения также требуют, чтобы в среде выполнения работало еще много других служб. Возможно, у вас есть база данных, кэш, хранилище NoSQL или даже другие ваши собственные приложения, которые должны выполняться, чтобы ваше приложение могло работать локально.

Программа Docker Compose предназначена для того, чтобы объявлять и запускать наборы контейнеров. Эта система позволяет зафиксировать конкретный набор служб и запустить их вместе, а также умеет сохранять состояние этих

https://t.me/javalib

контейнеров, чтобы их можно было останавливать и перезапускать, не настраивая все с нуля.

Если вам кажется, что это похоже на функциональность таких средств координации, как Kubernetes, то вы абсолютно правы. Две системы дублируют друг друга в некоторых аспектах управления контейнерами. Однако Docker Compose рассчитана на выполнение на одной машине и поэтому вряд ли подойдет для большинства сред реальной эксплуатации.

> **ПРИМЕЧАНИЕ** Docker Compose изначально была отдельным инструментом, но теперь она интегрирована как команда в docker. Если где-то в интернете вы встречаете старые пособия, где предлагают запустить `docker-compose`, просто замените «-» на пробел.

По умолчанию конфигурация описывается в файле docker-compose.yml. Для начала сообщим Docker Compose о нашем приложении:

```
version: "3.9"      <-- Версия файла Docker Compose
services:
  app:              <-- Объявляет запускаемую службу с именем app
    build: .        <-- Заставляет Docker Compose выполнить типичную команду docker build
    ports:              в текущем каталоге, чтобы сгенерировать образ для этой службы
      - "8080:8080" <-- Объявления портов, как при ручном
                        запуске docker run
```

Если запустить Docker Compose командой `docker compose up`, сначала отображается уже знакомый вывод `build`, а затем новый вывод, когда запускается контейнер:

```
[+] Running 2/2
 - Network docker-gradle_default Created 0.1s
 - Container docker-gradle-app-1 Created 0.1s
Attaching to docker-gradle-app-1
docker-gradle-app-1 | (Howdy,Docker)
```

Как вы вскоре увидите, в файле docker-compose.yml может фигурировать несколько служб. Чтобы не запутаться в них, вывод каждой службы предваряется ее именем, которое по умолчанию составляется из текущего каталога и имени службы (в нашем случае `docker-gradle-app-1`).

Допустим, нашему приложению требуется экземпляр Redis. Добавим его с новым ключом `redis` в разделе `services`:

```
version: "3.9"
services:
  app:
    build: .
    ports:
      - "8080:8080"
  redis:
    image: "redis:alpine"   <-- Образ redis:alpine из Docker Hub
```

https://t.me/javalib

Теперь, когда мы запустим приложение, Docker Compose извлечет образ `redis:alpine` и запустит его параллельно с контейнером нашего приложения. Рисунок 12.7 и следующий фрагмент вывода показывают, как эти контейнеры взаимодействуют друг с другом:

```
[+] Running 7/7
 - redis Pulled                                  5.0s
   - 59bf1c3509f3 Pull complete                  1.2s
   - 719adce26c52 Pull complete                  1.2s
[+] Running 2/2
 - Container docker-gradle-redis-1 Created       0.2s
 - Container docker-gradle-app-1 Created         0.0s
Attaching to docker-gradle-app-1, docker-gradle-redis-1
docker-gradle-redis-1 | # oO0Oo Redis is starting...
docker-gradle-redis-1 | # Redis version=6.2.6, ...
docker-gradle-redis-1 | * monotonic clock: POSIX ...
docker-gradle-redis-1 | # Warning: no config file...
docker-gradle-redis-1 | * Running mode=standalone, ...
docker-gradle-redis-1 | # Server initialized
docker-gradle-redis-1 | * Ready to accept connections
docker-gradle-app-1 | (Howdy,Docker)
```

Вывод контейнера Redis

Вывод контейнера приложения

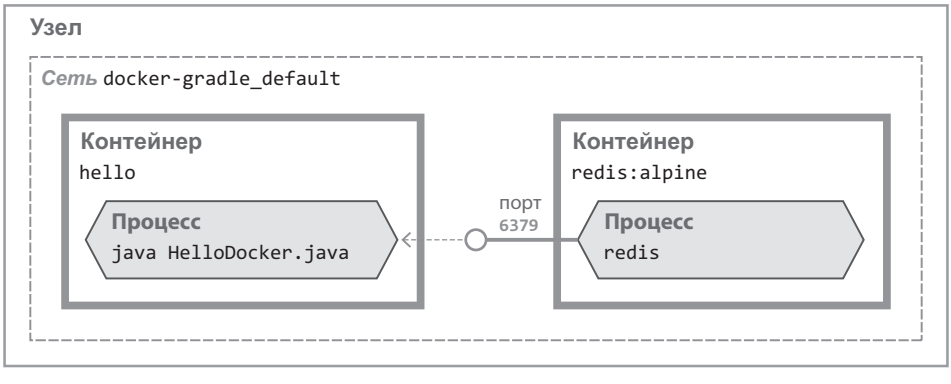


Рис. 12.7. Работающие контейнеры Docker Compose

И это уже удобно: мы можем локально получить конкретную версию баз данных и других внешних служб без ручной установки. Но Docker Compose добавляет еще одну полезную возможность, позволяющую предотвратить многие сетевые проблемы, с которыми мы сталкивались ранее. Во время исходного запуска выводилось сообщение `Network docker-gradle_default Created`. Это означало, что Docker Compose создал новое отдельное сетевое пространство имен `docker-gradle_default`. Эту сеть совместно используют все службы, которые запускает Docker Compose. Кроме того, имена служб, которые мы указали в docker-compose.yml, — `app` и `redis` — отображаются как настоящие имена узлов внутри всех контейнеров.

https://t.me/javalib

Если мы будем проектировать приложение в соответствии с принципами 12-факторной методологии и передавать местонахождение Redis в переменной окружения, все можно будет полностью настроить в docker-compose.yml:

```
version: "3.9"
services:
  app:
    build: .
    ports:
      - "8080:8080"
    environment:
      REDIS_URL: redis://redis:6379
  redis:
    image: "redis:alpine"
```

Первое вхождение redis — схема URL, а второе — имя узла

Здесь мы рассказали о возможностях Docker Compose лишь на самом поверхностном уровне. Все наиболее часто используемые настройки, которыми управляется `docker run`, можно задать в docker-compose.yml — это поможет вам нарастить темп локальной разработки и повысить ее эффективность.

12.3.6. Отладка в Docker

Если программа ведет себя неподобающе, иногда приходится переступать границы, которые задают контейнеры. Ранее мы видели, как команда `docker ps` определяет порт, доступ к которому предоставляется контейнером. Но этим возможности команды не ограничиваются. В частности, по умолчанию контейнеру назначается случайно сгенерированное «человеческое» имя, по которому к нему можно обращаться:

```
$ docker ps
CONTAINER ID   IMAGE    COMMAND             ...  PORTS      NAMES
c103de6e6634   hello    "./docker-gradle"   ...  8080/tcp   vigilant_austin
```

К этому контейнеру можно обращаться по имени `vigilant_austin`. Если вы не хотите, чтобы имя менялось при каждом запуске контейнера, его можно явно присвоить с помощью параметра `--name имя_контейнера` команды `docker run`. Его стоит объединить с параметром `--rm`, чтобы контейнер удалялся при выходе, а иначе имя окажется недоступным при повторном запуске.

Зная имя контейнера, можно заниматься другими операциями по отладке. Команда `docker exec` позволяет выполнять команды в действующем контейнере. Как было показано ранее для `docker run -it`, можно даже запустить интерактивную командную оболочку внутри контейнера, как в следующем примере (предполагается, что в контейнере установлена оболочка bash или что-то похожее):

```
$ docker run --name hello-container --rm hello

# В другом терминале запустите командную оболочку в контейнере
$ docker exec -it hello-container bash
```

https://t.me/javalib

```
root@18a5f04bb4c8: ps aux
USER PID %CPU %MEM COMMAND
root   1  1.6  1.9 /opt/java/openjdk/bin/java -cp /opt/app/lib/docker-gradle
root  37  0.1  0.1 bash
root  47  0.0  0.1 ps aux
```

Важно помнить, что exec не запускает новый контейнер, а присоединяется к уже существующему. На рис. 12.8 показано, как сосуществуют процессы внутри одного контейнера.

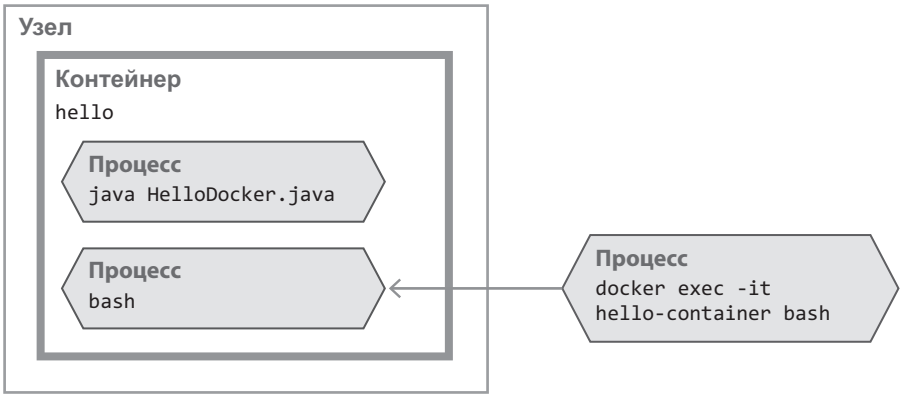


Рис. 12.8. Выполнение docker exec в контейнере

Впрочем, наши возможности не ограничиваются базовыми командами Unix. Например, с помощью команд `jps` и `jcmd` можно получить информацию о JVM, которые работают в контейнере:

```
root@18a5f04bb4c8: jps
1 Main
148 Jps

root@18a5f04bb4c8: jcmd 1 VM.version
1:
OpenJDK 64-Bit Server VM version 17.0.1+12
JDK 17.0.1
```

В главе 7 мы узнали, как собирать подробную информацию с помощью инструментария JFR (JDK Flight Recorder). Запустив командную оболочку в работающем контейнере, можно собрать данные JFR несколькими простыми командами. Если режим записи JFR еще не активен, его можно включить так:

```
root@4f146639fcfc: jcmd 1 JFR.start
1:
Started recording 1. No limit specified, using maxsize=250MB as default.
```

После того как приложение накопит информацию за какой-то период времени, ее можно сохранить в файле журнала внутри контейнера:

https://t.me/javalib

```
root@4f146639fcfc: jcmd 1 JFR.dump name=1 filename=./capture.jfr
1:
Dumped recording "1", 293.3 kB written to:
```

Чтобы проанализировать содержимое файла в автономном режиме, его надо скопировать из контейнера. В управляющей системе это можно сделать командой `docker cp`, как показано в следующем примере. И снова имя контейнера поможет указать, откуда копировать файл:

```
$ docker cp hello-container:/opt/app/bin/capture.jfr .
```

Первый параметр cp — источник файла, а второй — приемник. Формат параметров: имя_контейнера:путь. Второй параметр — локальное расположение, поэтому мы не указываем имя контейнера, а используем только путь

Теперь файл capture.jfr появился в вашей локальной системе и его можно открыть с помощью JMC (JDK Mission Control).

Поскольку Docker предоставляет доступ к своему API, команды `docker` могут обращаться к удаленному узлу вместо локального окружения. Подробнее о настройке этой функции можно узнать из документации Docker по адресу https://docs.docker.com/.

Все эти оболочки и параметры командной строки удобны для того, чтобы на низком уровне анализировать, что происходит в контейнерах. Но что, если вы просто хотите задать точку останова в IDE для приложения Java в локальном контейнере? К счастью, средства удаленной отладки JDK предоставляют все необходимое для этого:

```
docker run --rm \
  -p 8090:8090 \
  -e JAVA_TOOL_OPTIONS=\
  '-agentlib:jdwp=transport=dt_socket,server=y,suspend=n,address=*:8090' \
  --name hello-container \
  hello
```

Помимо обычного вывода при запуске приложения вы увидите сообщение о том, что доступен порт удаленной отладки:

```
Listening for transport dt_socket at address: 8090
```

После этого вы сможете пользоваться средствами IDE, чтобы отлаживать удаленную JVM через порт 8090. Все должно работать так же, как при отладке приложения в локальном окружении, но из комфортного, ограниченного мира контейнера.

12.3.7. Протоколирование в Docker

Как неоднократно упоминалось выше, нужно определенное изменение менталитета, чтобы эффективно работать с системой, где контейнеры отделяют свое содержимое от среды управляющего узла. Многие спотыкаются на задаче ведения

https://t.me/javalib

журнала. Независимо от того, пользуетесь ли вы каким-нибудь из популярных фреймворков протоколирования (логирования) или просто направляете вывод в `System.out`, службы обычно генерируют вывод при выполнении. Не хотелось бы терять доступ к этой информации только потому, что мы переместились в контейнер.

Можно настроить протоколирование вручную теми способами, которые уже встречались в этой главе. Просто записывайте журнальную информацию на диск, как и прежде, а когда вам понадобится ее анализировать, обращайтесь к файлам с помощью `docker exec` или `docker cp`:

```
// Запуск контейнера
$ docker run --rm --name hello-container hello

// В другой оболочке создается локальная копия файла.
// Предполагается, что журнал хранится в файле /log/application.log
$ docker cp hello-container:/log/application.log .

// Также можно непрерывно присоединять данные к файлу
$ docker exec hello-container tail -f /log/application.log
```

Тем не менее при таком решении оказывается не очень удобно извлекать информацию, а также есть риск потерять данные, если контейнер будет удален преждевременно.

Стандартная практика — независимо от того, работаете ли вы с контейнерами или нет, — заключается в том, чтобы пересылать журналы от приложения в единый приемник, которым может быть централизованное хранилище, служба индексирования (например, Elasticsearch) или даже полностью внешний провайдер протоколирования.

Если вы пытаетесь по-простому вести протоколирование в локальном файле в контейнере, необходимо ответить на вопрос о том, где будет работать приложение, которое пересылает журнальные данные. Если размещать его в контейнере, на это будет тратиться дополнительная память и ресурсы, которые необходимо учитывать. В общем случае лучше не запускать несколько задач в одном контейнере. Контейнеры позволяют смонтировать том так, чтобы обеспечить совместный доступ к файлам журналов между контейнерами и управляющим узлом, но такую систему нужно специально настраивать и она не всегда обеспечивает удовлетворительную производительность.

Более эффективное решение основано на том факте, что Docker сохраняет все, что контейнеры выводят в стандартные выходные потоки `STDOUT` и `STDERR`. На узле эти потоки сохраняются в определенных местах для каждого работающего контейнера. Это упрощает конфигурацию, потому что можно установить средства пересылки журнальных данных на узле, а затем заставить каждый контейнер направлять вывод в `STDOUT`, а не в файлы. Также это решение совместимо с существующими библиотеками протоколирования (такими, как `log4j2`), у которых есть средства присоединения для записи в `CONSOLE` именно для этой цели.

https://t.me/javalib

Подобная настройка инфраструктуры для того, чтобы запускать контейнеры и регистрировать их журнальные данные, демонстрирует типичные проблемы, которые возникают, когда контейнеры масштабируются за пределы одного узла. Систематическое решение подобных проблем — одно из ключевых преимуществ системы Kubernetes, которой будет посвящен следующий раздел.

12.4. KUBERNETES

Наше введение в Docker дает лишь поверхностное представление о конфигурации и настройке контейнеров. В среде реальной эксплуатации вам может понадобиться множество экземпляров контейнеров. Если пытаться управлять большими комплексами контейнеров только с помощью команд `docker`, ситуация быстро выйдет из-под контроля, — при том, что в средах эксплуатации нередко задействованы сотни отдельных контейнеров. Эти задачи необходимо автоматизировать. Автоматизация такого рода обозначается общим термином «*оркестрация*» (другими словами, координация), и хотя для нее существует много инструментов, доминирующее положение среди них занимает Kubernetes.

Kubernetes (название часто сокращается до *K8s*) — проект с открытым кодом, который происходит от внутреннего проекта по оркестрации контейнеров в компании Google. По сути, он предоставляет стандартный, управляемый через API инструментарий для того, чтобы описывать желательное состояние системы и поддерживать это состояние со временем.

В Kubernetes система моделируется как набор разнотипных *объектов*. В ней работают *контроллеры*, которые отслеживают состояние системы и применяют изменения (например, создают новый контейнер, когда старый уничтожается), чтобы привести фактическое состояние к желаемому.

Полное описание Kubernetes выходит далеко за рамки книги, но чтобы получить некоторое представление о том, как она работает, мы рассмотрим основные типы объектов и узнаем, как использовать их в контексте того, что вам уже известно о контейнерах.

- *Кластер* — отдельная установка Kubernetes на чем угодно — от одной машины до сотен узлов.
- *Узел* — одна машина (виртуальная или физическая), входящая в кластер.
- *Под* — развертываемая единица из одного или нескольких контейнеров.
- *Развертывание* — декларативный механизм развертывания блоков.
- *Служба* — объект, который предоставляет доступ к контейнерам в кластере.

Чтобы освоить эти понятия на практике, мы воспользуемся `minikube` — локальной средой разработки из проекта Kubernetes. Текущую версию инструкций по ее установке для вашей ОС можно найти по адресу https://minikube.sigs.k8s.io/docs/start/.

https://t.me/javalib

После установки можно запустить локальный кластер командой `minikube start`, как показано ниже. Учтите, что при первом запуске может уйти несколько минут на то, чтобы загрузить необходимые образы:

```
$ minikube start

  minikube v1.25.2 on Darwin 11.6.2
  Using the docker driver based on existing profile
  Starting control plane node minikube in cluster minikube
  Pulling base image ...
  Downloading Kubernetes v1.23.1 preload ...
  > preloaded-images-k8s-v17-v1...: 504.44 MiB / 504.44 MiB 100.00%
  Restarting existing docker container for "minikube" ...
  Preparing Kubernetes v1.23.1 on Docker 20.10.12 ...
  * kubelet.housekeeping-interval=5m
  Verifying Kubernetes components...
  * Using image kubernetesui/dashboard:v2.3.1
  * Using image kubernetesui/metrics-scraper:v1.0.7
  * Using image gcr.io/k8s-minikube/storage-provisioner:v5
  Enabled addons: storage-provisioner, default-storageclass, dashboard

  Done! kubectl is now configured to use "minikube" cluster and "default"
  namespace by default
```

Наш кластер Kubernetes работает локально. Его можно остановить командой `minikube stop`, а если вы закончили эксперименты — удалить командой `minikube delete`.

Хотя Kubernetes предоставляет REST API, с помощью которого внешние системы могут с ней взаимодействовать, существует удобная обертка, которая лучше подходит для человека, — команда `kubectl`. С ее помощью можно просматривать, создавать и редактировать объекты в кластере. Например, `minikube` создает объекты узлов по умолчанию, а команда `kubectl describe node` позволяет просмотреть, как они устроены. В следующем листинге приведена лишь часть вывода, потому что полный вывод занял бы слишком много места:

```
$ kubectl describe node

Name:               minikube
Roles:              control-plane,master
Labels:             kubernetes.io/arch=amd64
                    kubernetes.io/hostname=minikube
                    kubernetes.io/os=linux
Addresses:
  InternalIP: 192.168.49.2
  Hostname: minikube

Non-terminated Pods:        (12 in total)
  Namespace                 Name
  ---------                 ----
  kube-system               coredns-64897985d-n8fzv
  kube-system               etcd-minikube
  kube-system               kube-apiserver-minikube
```

Сама Kubernetes выполняется в указанных здесь подах на узле

https://t.me/javalib

```
  kube-system               kube-controller-manager-minikube
  kube-system               kube-proxy-4zvll
  kube-system               kube-scheduler-minikube
  kube-system               storage-provisioner
  kubernetes-dashboard      dashboard-metrics-scraper-58549894f-bcjh4
  kubernetes-dashboard      kubernetes-dashboard-ccd587f44-mq8zv

Events:                                          <-- Информация о событиях может пригодиться при отладке, если с узлом возникают непредвиденные проблемы
  Type    Reason                   Message
  ----    ------                   -------
  Normal  Starting                 Starting kubelet.
  Normal  NodeHasSufficientMemory  Node status is: NodeHasSufficientMemory
  Normal  NodeHasNoDiskPressure    Node status is: NodeHasNoDiskPressure
  Normal  NodeHasSufficientPID     Node status is: NodeHasSufficientPID
```

Итак, после того как `minikube` предоставил кластер и узел, можно запускать приложения. Для простоты мы воспользуемся образом `k8s.gcr.io/echoserver:1.4`, который всего лишь возвращает информацию об отправленных ему запросах HTTP.

> **ПРИМЕЧАНИЕ** `minikube` поддерживает работу с локальными образами, но при этом запускает отдельного демона Docker, так что управлять образами становится сложнее. Если вы собираетесь использовать `minikube` в локальной разработке, обращайтесь к документации README по адресу https://github.com/kubernetes/minikube. Для простоты мы будем ограничиваться в своих примерах общедоступными образами.

Наша первая цель — получить под в кластере, где будет выполняться контейнер `echoserver`. Для этого мы заставим `kubectl` создать объект развертывания, как показано в следующем примере. Этот объект сообщает кластеру Kubernetes, что желаемое состояние заключается в том, чтобы наш под выполнялся. Управляющий цикл Kubernetes замечает, что желаемое состояние не соответствует реальному, и запускает поды, чтобы устранить это несоответствие:

```
$ kubectl create deployment echoes --image=k8s.gcr.io/echoserver:1.4
deployment.apps/echoes created
```

Чтобы проверить кластер и убедиться в том, что наше развертывание действительно существует, мы воспользуемся стандартной командой `kubectl get`. Команда работает с любыми типами объектов в системе:

```
$ kubectl get deployments
NAME     READY   UP-TO-DATE   AVAILABLE   AGE
echoes   1/1     1            1           55s
```

Просматривая информацию о подах, мы вскоре увидим, что кластер привел фактическое состояние в соответствие с запрашиваемым. На рис. 12.9 представлено итоговое состояние нашего единственного пода.

```
$ kubectl get pods
NAME                      READY   STATUS     RESTARTS   AGE
echoes-7989cff4bc-7m4df   1/1      Running   0          78s
```

https://t.me/javalib

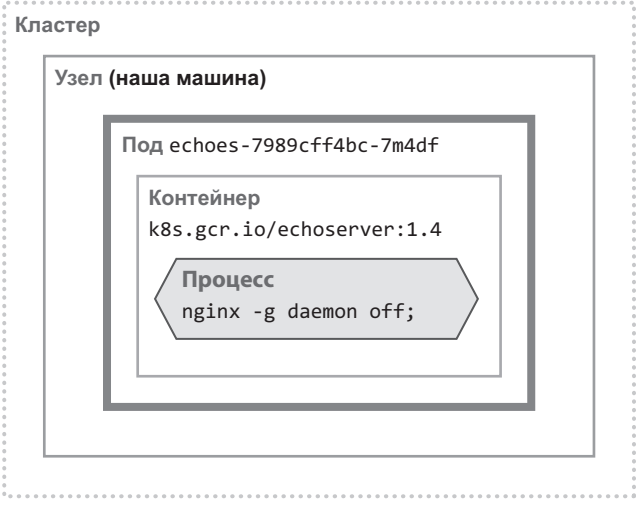


Рис. 12.9. Кластер Kubernetes с одним выполняемым подом

Команда `kubectl create deployment` позволяет легко начать работу, но ее аргументы дают лишь поверхностное представление о возможностях конфигурации Kubernetes. Естественное представление объекта Kubernetes задается в формате YAML, и оценить общую картину позволяет команда `kubectl edit deployment echoes`, которая показана в следующем примере. Команда откроет ваш редактор по умолчанию, в котором отобразится текущее состояние объекта в формате YAML. Если внести изменения в файл, они применятся при выходе из редактора. Мы не будем подробно рассматривать все эти настройки, так что за дополнительной информацией обращайтесь к документации по адресу http://mng.bz/M5m2:

```
# Please edit the object below. Lines beginning with a '#' will be ignored,
# and an empty file will abort the edit. If an error occurs while saving
# this file will be reopened with the relevant failures.
#
apiVersion: apps/v1
kind: Deployment
metadata:
  annotations:
    deployment.kubernetes.io/revision: "1"
  creationTimestamp: "2022-02-01T08:26:32Z"
  generation: 1
  labels:
    app: echoes
  name: echoes  <-- Имя echoes, которое мы присвоили объекту развертывания
  namespace: default
  resourceVersion: "1310"
  uid: e8b775f6-243e-46c1-9275-dadaecf2db3b
spec:  <-- spec описывает желаемое состояние развертывания
```

https://t.me/javalib

```
  progressDeadlineSeconds: 600
  replicas: 1
  revisionHistoryLimit: 10
  selector:
    matchLabels:
      app: echoes
  strategy:
    rollingUpdate:
      maxSurge: 25%
      maxUnavailable: 25%
    type: RollingUpdate
  template:
    metadata:
      creationTimestamp: null
      labels:
        app: echoes
    spec:
      containers:
      - image: k8s.gcr.io/echoserver:1.4
        imagePullPolicy: IfNotPresent
        name: echoserver
        resources: {}
        terminationMessagePath: /dev/termination-log
        terminationMessagePolicy: File
      dnsPolicy: ClusterFirst
      restartPolicy: Always
      schedulerName: default-scheduler
      securityContext: {}
      terminationGracePeriodSeconds: 30
status:
  availableReplicas: 1
  conditions:
  - lastTransitionTime: "2022-02-01T08:26:33Z"
    lastUpdateTime: "2022-02-01T08:26:33Z"
    message: Deployment has minimum availability.
    reason: MinimumReplicasAvailable
    status: "True"
    type: Available
  - lastTransitionTime: "2022-02-01T08:26:32Z"
    lastUpdateTime: "2022-02-01T08:26:33Z"
    message: ReplicaSet "echoes-7989cff4bc" has successfully progressed.
    reason: NewReplicaSetAvailable
    status: "True"
    type: Progressing
  observedGeneration: 1
  readyReplicas: 1
  replicas: 1
  updatedReplicas: 1
```

Это важное значение определяет желаемое количество подов, которые должны выполняться (вскоре мы рассмотрим его подробнее)

Мы запросили, чтобы в подах выполнялся этот образ

status сообщает, как в данный момент выглядит состояние развертывания. Обратите внимание, что здесь тоже есть значение replicas, которое показывает, сколько реплик сейчас выполняется

Что произойдет, если в разделе `spec` изменить значение `replicas` с `1` на `3`? Kubernetes увидит, что состояние развертывания не соответствует фактическому состоянию кластера, и запустит новые контейнеры. На рис. 12.10 показан результат после того, как контейнеры получат возможность запуститься.

https://t.me/javalib

```
$ kubectl get pods
NAME                        READY   STATUS    RESTARTS   AGE
echoes-7989cff4bc-7m4df     1/1     Running   0          7m38s
echoes-7989cff4bc-7qn47     1/1     Running   0          8s
echoes-7989cff4bc-cmngm     1/1     Running   0          8s
```

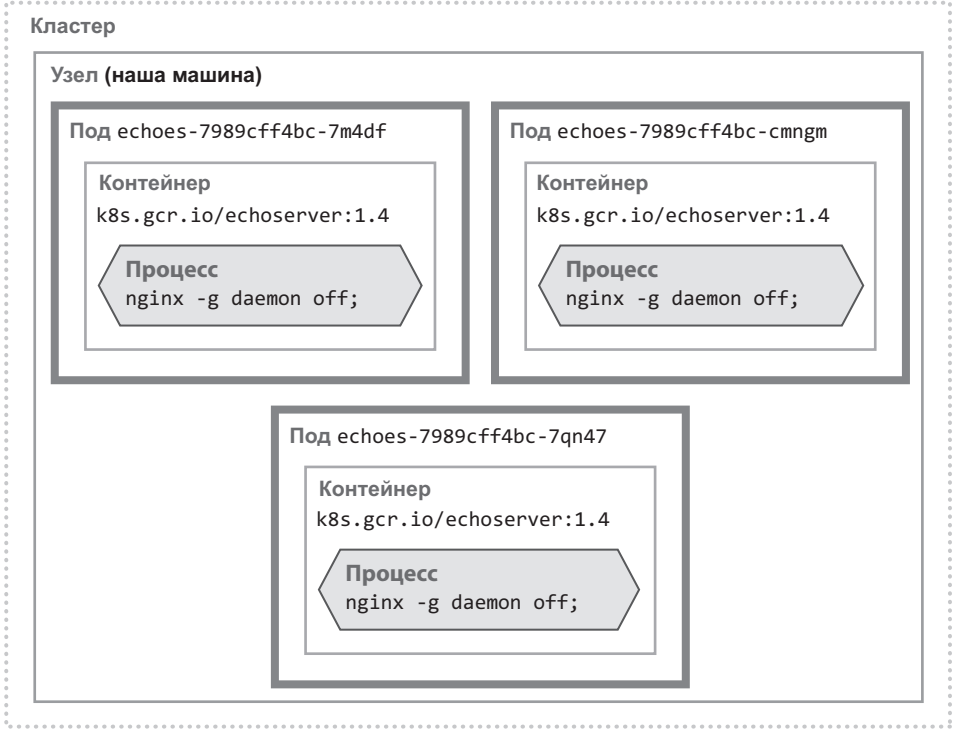


Рис. 12.10. Кластер Kubernetes с несколькими выполняемыми блоками

Скорее всего, вам не придется вручную редактировать файлы YAML на работающем кластере Kubernetes. Обратите внимание, что все соответствующие инструменты — системы CI/CD, генерируемые или курируемые манифесты Kubernetes — это, по сути, вспомогательные средства, которые создают за вас правильную разметку YAML и вызовы API.

В локальной системе уже знакомые команды `docker ps` и `docker exec` тоже помогут глубже проанализировать выполняемые контейнеры. Если известно имя контейнера, в `kubectl` можно применить чуть более удобную команду, которая запускает командную оболочку внутри пода:

```
$ kubectl exec echoes-7989cff4bc-7m4df -- bash
root@echoes-7989cff4bc-7m4df: uname -a
Linux echoes-7989cff4bc-7m4df 5.10.76-linuxkit #1 SMP \
  Mon Nov 8 10:21:19 2021 x86_64 x86_64 x86_64 GNU/Linux
```

https://t.me/javalib

Остается сделать последний шаг, чтобы это развертывание стало более полезным. По умолчанию мы вообще не можем обращаться к подам в кластере. Если просмотреть вывод `docker ps` для контейнеров, вы увидите, что открытых портов нет.

В Kubernetes эта проблема решается с помощью абстракции *служб* — интерфейсов общего вида, которые работают с балансировкой нагрузки и маршрутизацией трафика, входящего в кластер. Дальнейшие подробности выходят за рамки этой книги, но мы настроим простейшую службу NodePort командой `kubectl expose`:

```
$ kubectl expose deployment echoes --type=NodePort --port=8080
service/echoes exposed
```

Как показано на рис. 12.11, эта команда создает в кластере новый объект, который представляет `NodePort`.

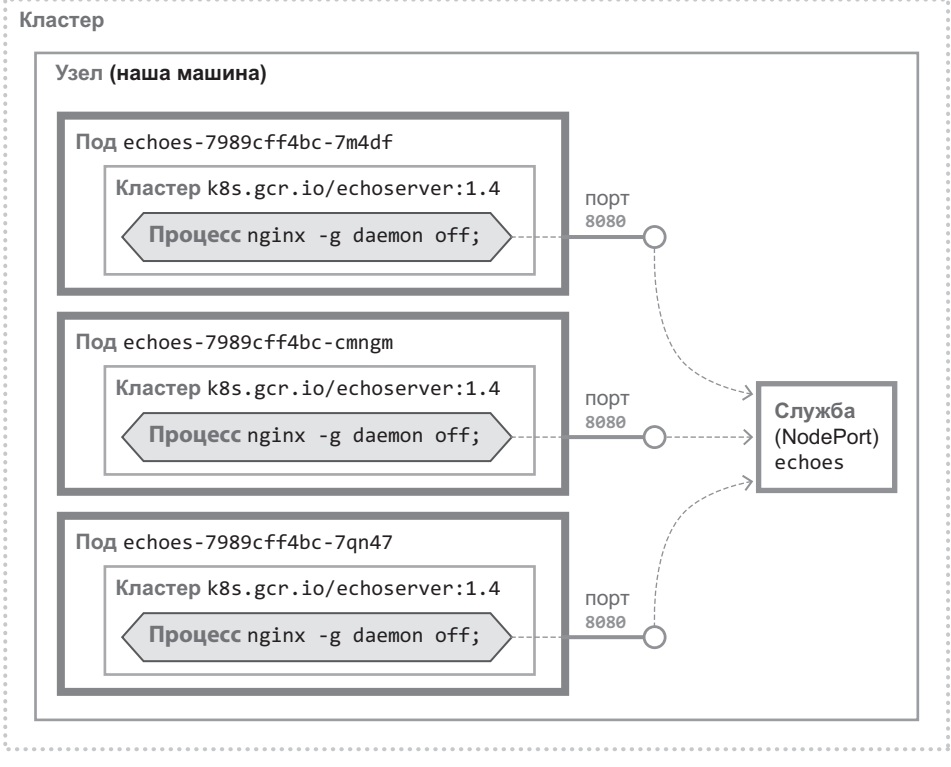


Рис. 12.11. NodePort и службы в кластере Kubernetes

Просматривая список служб, мы видим, что в кластере настроен `NodePort`:

```
$ kubectl get services
NAME         TYPE        CLUSTER-IP       EXTERNAL-IP PORT(S)          AGE
echoes       NodePort    10.108.182.100   <none>      8080:31980/TCP   12s
kubernetes   ClusterIP   10.96.0.1        <none>      443/TCP          35m
```

https://t.me/javalib

Во внутренней реализации это означает, что порт 8080 доступен для каждого узла в кластере и перенаправляет трафик нашим подам. Теперь, когда поды могут принимать входящий трафик, нужен способ подавать этот трафик из-за пределов кластера. Для этого `kubectl` предлагает механизм переадресации портов:

```
$ kubectl port-forward service/echoes 7080:8080
Forwarding from 127.0.0.1:7080 -> 8080
Forwarding from [::1]:7080 -> 8080
Handling connection for 7080
```

Запустив переадресацию в терминале, можно открыть адрес *127.0.0.1:7080* в браузере, и вы увидите, что по запросу возвращается эхо-ответ. На рис. 12.12 показано, как трафик поступает к подам через различные компоненты.

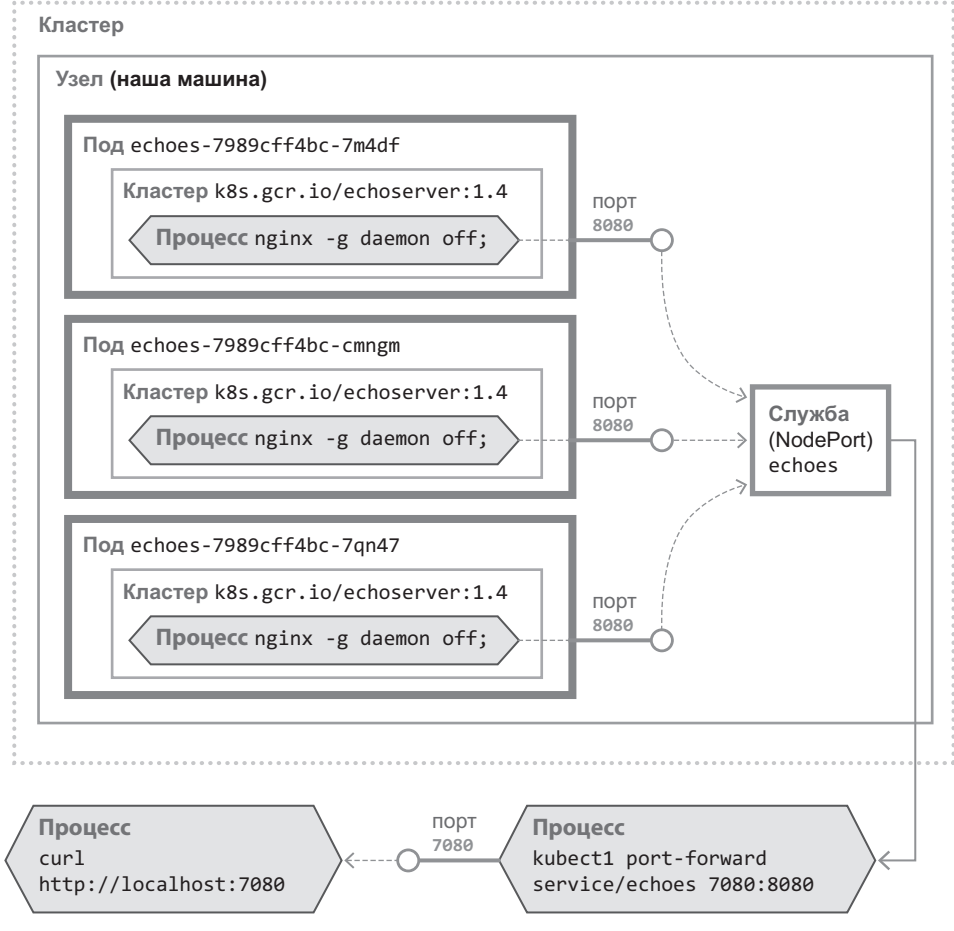


Рис. 12.12. Переадресация портов в кластере Kubernetes

https://t.me/javalib

Kubernetes существенно повышает сложность по сравнению с простым локальным запуском контейнеров Docker, но эта система координации также решает многие проблемы, которые возникают при масштабировании контейнерных конфигураций. Впрочем, независимо от того, используете вы Kubernetes или нет, для вас почти всегда важна производительность служб. Давайте посмотрим, как добиться, чтобы контейнеры эффективно работали в условиях реальной эксплуатации.

12.5. НАБЛЮДАЕМОСТЬ И ПРОИЗВОДИТЕЛЬНОСТЬ

Технология Java изначально разрабатывалась для мира, в котором JVM запускались на физическом оборудовании в вычислительных центрах, а разработчик был относительно изолирован от нюансов среды развертывания (и мог даже вообще ничего не знать о них). Однако в мире происходят фундаментальные изменения. Технологии развертывания в облачных средах, особенно контейнеры, быстро набирают популярность, хотя в разных областях индустрии это происходит в разном темпе.

С контейнерами становится труднее понимать все подробности того, что происходит в современном приложении. Например, в контейнерах обычно не запускаются такие службы, как демон `ssh`, из-за чего не удается подключиться к контейнеру, чтобы наблюдать за его работой изнутри. Вместо этого все данные о работоспособности приложения должны экспортироваться из контейнера.

Принцип DevOps, который обозначается термином «*наблюдаемость*» (observability), происходит из нескольких отдельных направлений современной практики разработки, включая как мониторинг производительности (APM), так и отслеживание работы систем координации (таких, как Kubernetes). Наблюдаемость направлена на то, чтобы получать детализированные сведения о поведении систем наряду с обширным контекстом. Ее методы крайне эффективно помогают понимать и настраивать производительность приложений Java в контейнерах.

12.5.1. Наблюдаемость

В целом наблюдаемость сводится к относительно простому набору принципов:

1. Снабжайте системы и приложения средствами измерительного контроля, которые собирают актуальные данные.
2. Отправляйте собранные данные системе, которая может их сохранять и анализировать (в том числе обрабатывать запросы).
3. Обеспечивайте визуализацию данных и общие аналитические выводы о системе.

https://t.me/javalib

Запросы и визуализация — ключевые механизмы наблюдаемости. Часто говорят, что она позволяет «получать ответы на вопросы, которые вы даже не планировали задавать», — а это удается, только если вы собрали достаточно данных, чтобы можно было точно моделировать внутреннее состояние системы.

> **ПРИМЕЧАНИЕ** Принципы, лежащие в основе наблюдаемости, происходят из теории управления системами — по сути, от вопроса «Насколько эффективно внутреннее состояние системы можно наблюдать извне?»

В конечном итоге цель состоит в том, чтобы делать выводы, которые касаются системы в целом и позволяют принимать решения о *конкретных действиях*. Эта практика должна прийти на смену фрагментарным представлениям, которые опираются на то, как ведут себя один-два избранных компонента системы.

Таким образом, хотя разрешение инцидентов — самая очевидная сфера для практического применения наблюдаемости (в конце концов, сама идея наблюдаемости происходит именно из этой сферы), верно и то, что наблюдаемость потенциально можно внедрить в гораздо более широкой области. Если собирать правильные данные, то они будут представлять интерес практически для всех, кто участвует в производстве продукта, а не только для специалистов по надежности, службы поддержки и персонала DevOps.

Наблюдаемость особенно актуальна для контейнеризованных приложений, потому что их артефакты развертывания обычно устроены сложнее, чем у традиционных локальных приложений. Приложения, которые развертываются на облачных платформах, обычно содержат больше служб и компонентов, имеют более сложную топологию и чаще изменяются (из-за таких практик, как непрерывное развертывание).

Свою роль играет и растущая популярность специфических облачных технологий, которые функционируют по новым принципам. К этой категории относится Kubernetes, а также средства внесерверной обработки данных (например, AWS Lambda). В этом новом мире становится значительно сложнее выявлять корневые причины и разрешать инциденты.

О данных наблюдаемости часто говорят в контексте «трех столпов» — простой ментальной модели (по мнению некоторых, *слишком* простой), которая особенно полезна для разработчиков без опыта в области наблюдаемости. Под «тремя столпами» понимаются следующие характеристики:

- *Распределенная трассировка* — регистрация информации об отдельном вызове службы, соответствующем одному запросу со стороны пользователя.
- *Метрики* — значения, которые описывают некоторую активность через те или иные промежутки времени.
- *Журналы* (*логи*) — неизменяемые протоколы дискретных событий, которые происходят в конкретные моменты времени. (Журналы могут быть в формате простого текста, структурированном или двоичном виде.)

https://t.me/javalib

Все основные библиотеки и компоненты измерительного контроля распространяются с открытым кодом, и многие из них сопровождаются отраслевыми организациями, такими как CNCF (Cloud Native Compute Foundation).

OpenTelemetry

OpenTelemetry (https://opentelemetry.io/), крупный проект под управлением CNCF, представляет собой набор стандартов, форматов, клиентских библиотек и связанных с ними программных компонентов, которые обеспечивают наблюдаемость. Стандарты явно сформулированы как кросс-платформенные и не привязаны к какому-либо технологическому стеку.

Таким образом формируется фреймворк, который интегрируется с ОС и коммерческими продуктами и может получать данные наблюдаемости из приложений, написанных на многих языках. Так как реализации распространяются с открытым кодом, они находятся на разных уровнях технологической зрелости в зависимости от того, насколько востребован OpenTelemetry в сообществе того или иного языка.

Проект OpenTelemetry появился в результате слияния двух предшествующих проектов с открытым кодом — OpenTracing и OpenCensus. Хотя OpenTelemetry все еще находится в стадии становления, он набирает популярность, и его задействуют все больше приложений и разработчиков.

В контексте нашей темы важно то, что реализация OpenTelemetry для Java и JVM — одна из самых зрелых, и у нее немало преимуществ перед традиционным APM-мониторингом. В частности, открытый стандарт обеспечивает следующие возможности:

- Значительное ослабление привязки к технологиям конкретного разработчика.
- Открытая спецификация протоколов передачи данных.
- Клиентские компоненты с открытым кодом.
- Стандартизированные архитектурные паттерны.
- Повышение количества и качества серверных компонентов с открытым кодом.

OpenTelemetry состоит из нескольких подпроектов, которые находятся на разном уровне зрелости в отношении своего жизненного цикла.

На момент написания этой книги спецификация распределенной трассировки Distributed Tracing существует в версии 1.0 и активно применяется в реальных системах. Она полностью замещает OpenTracing, который был официально объявлен архивным. Проект Jaeger — одна из самых популярных подсистем распределенной трассировки — тоже упразднил свои клиентские библиотеки и по умолчанию использует протоколы OpenTelemetry.

Проект OpenTelemetry Metrics не настолько развит, но он достиг версии 1.0 и статуса общедоступности (GA, General Availability). На момент написания

https://t.me/javalib

книги протокол был стабильным, и функциональность API была зафиксирована.

Наконец, спецификация Logging на момент написания книги все еще находится в черновом варианте и пока еще нуждается в определенной доработке.

В целом можно будет считать, что проект OpenTelemetry дойдет до версии 1.0 (GA), когда стандарт Metrics достигнет версии 1.0 наряду с Tracing.

Библиотеки Java для OpenTelemetry можно развертывать в приложениях вручную (когда разработчик сознательно выбирает, в каких частях приложения должен применяться измерительный контроль), а можно автоматически (с помощью агента Java). Компоненты Java для OpenTelemetry доступны в нескольких проектах на GitHub, например http://mng.bz/aJyJ.

Полное обсуждение того, как реализовывать полноценные решения из области наблюдаемости (на базе OpenTelemetry или другого технологического стека), выходит за рамки книги, но опытному разработчику Java стоит тщательно изучить эту область.

Наблюдаемость имеет отношение к некоторым нюансам производительности, о которых могут не знать специалисты, не связанные с миром Java или виртуальных машин. В следующем разделе мы поговорим об этом подробнее.

12.5.2. Производительность в контейнерах

Многие разработчики, которые переносят свои приложения на Java в контейнеры, стараются минимизировать размер контейнеров. Это выглядит логично, потому что облачные приложения обычно тарифицируются в соответствии с тем, сколько памяти и процессорного времени они используют.

Тем не менее JVM — чрезвычайно динамичная платформа, и при запуске она автоматически устанавливает некоторые важные параметры на основании наблюдаемых свойств машины, на которой она работает.

Например, к этим свойствам относится тип и количество процессоров, а также объем физической памяти. Приложение может (и будет!) вести себя поразному, если выполняется на машинах разной мощности, а это относится и к контейнерам. Вот примеры свойств, которые могут меняться в зависимости от машины:

- Режим JVM Intrinsics — метод JIT, который использует особые возможности процессора (например, поддержку векторов).
- Размер внутренних пулов потоков (таких, как «общий пул»).
- Количество потоков, используемых для сборки мусора.

Из этого списка видно, что неправильно выбранный размер образа контейнера может вызвать проблемы со сборкой мусора или другими потоковыми операциями. Тем не менее настоящая проблема лежит глубже.

https://t.me/javalib

Текущие версии Java, включая Java 17, выполняют динамические проверки и автоматически («эргономически») выбирают, какой сборщик мусора использовать, если он не указан явно в командной строке:

- Для машин «серверного класса» выбирается G1 (Parallel для Java 8).
- Для машин, не относящихся к «серверному классу», выбирается Serial.

К «серверному классу» относятся машины, у которых не менее двух физических процессоров и не менее 2 Гбайт памяти . Таким образом, если приложение Java запускается на более слабом оборудовании и вы не указали конкретный алгоритм сборки мусора, то будет использоваться Serial. Обычно это не то, что нужно разработчикам, и отсюда следует рекомендация:

> **ПОДСКАЗКА** Всегда запускайте приложения Java в контейнерах, у которых не менее двух процессоров и 2 Гбайт памяти.

Важно понимать, что жизненный цикл традиционного приложения Java состоит из нескольких фаз: инициализация, интенсивная загрузка классов, разогрев (с JIT-компиляцией), а затем долгосрочное устойчивое состояние (которое продолжается днями и неделями) с относительно незначительной загрузкой классов и JIT-компиляцией. Эта модель конфликтует со спецификой облачных развертываний, где срок жизни контейнеров часто бывает гораздо короче, а размеры кластеров можно регулировать динамически.

В этой новой реальности приходится следить за тем, чтобы Java сохраняла конкурентоспособность по нескольким ключевым направлениям, включая следующие:

- Затраты памяти.
- Интенсивность операций.
- Время запуска.

К счастью, в настоящее время ведутся исследования и появляются разработки, которые помогают добиться того, чтобы платформа продолжала оптимизироваться для этих характеристик, — подробнее об этом в главе 18.

ИТОГИ

- Контейнеры радикально изменили способы упаковки и развертывания приложений. Работа с ними требует новых идей и приемов.
- Контейнеры представляют дополнительный уровень абстракции поверх классических операционных систем, гипервизоров и виртуальных машин, с которыми мы сталкивались до этого.
- Docker — самый популярный инструмент для того, чтобы собирать, публиковать и запускать образы контейнеров.

https://t.me/javalib

- Образ контейнера определяется в Dockerfile. Готовый образ содержит ваше приложение и полную среду для его выполнения, включая JVM, собственные зависимости и дополнительные инструменты.
- Контейнеры создают дополнительную прослойку при работе с сетью. Нужны специальные настройки для того, чтобы контейнеризованные приложения предоставляли порты для доступа, а из внешнего мира к ним можно было обращаться.
- В крупномасштабных комплексах контейнеров требуется контролировать множество разных факторов. Чтобы автоматизировать и систематически выполнять эту работу, используются средства оркестрации, самое популярное из которых — Kubernetes.
- Kubernetes предоставляет полнофункциональный расширяемый API, который позволяет объявить желаемое состояние системы и достичь его во время выполнения. Координацией можно управлять с помощью командной строки или REST API, на основе которого построена огромная экосистема вспомогательных средств.
- Ключевая особенность контейнеров — контроль ограничений по ресурсам. Ограничения по затратам памяти и ресурсам процессора могут существенно влиять на производительность приложений, которые работают в контейнере.

https://t.me/javalib

13 Основы тестирования

В ЭТОЙ ГЛАВЕ

- ✓ Зачем нужно тестирование
- ✓ Как проводить тестирование
- ✓ Разработка через тестирование
- ✓ Тестовые дублеры
- ✓ От JUnit 4 к 5

В последние годы автоматизированное тестирование ПО все чаще рассматривается как неотъемлемая часть процесса разработки. Благодаря тестам, которые запускаются как локально самими разработчиками, так и в средах сборки и непрерывной интеграции, можно убедиться, что системы ведут себя правильно. Наряду с этим стремительно развиваются многочисленные инструменты, практики и методологии.

Как и в любой технологии, здесь нет «серебряной пули»: ни один подход к тестированию не покроет все возможные ситуации. Поэтому важно понимать, *зачем* вы тестируете, — это поможет узнать, *как* лучше тестировать.

13.1. ЗАЧЕМ НУЖНО ТЕСТИРОВАНИЕ

За словом «тест» кроется великое множество намерений, ради которых мы изучаем поведение кода. Приведем неполный (и местами перекрывающийся) список возможных целей:

https://t.me/javalib

- Убедиться, что логика отдельного метода правильна.
- Убедиться, что два объекта в коде взаимодействуют друг с другом.
- Убедиться, что библиотека или другая внешняя зависимость ведет себя так, как ожидалось.
- Убедиться, что часть системы производит или потребляет правильные данные.
- Убедиться, что система правильно работает с внешним компонентом (например, базой данных).
- Убедиться, что сквозное поведение системы соответствует важным бизнес-сценариям.
- Задокументировать предположения для специалистов, которые позднее будут заниматься сопровождением (потому что тесты не теряют актуальности, как это бывает с комментариями и документацией).
- Повлиять на архитектуру системы, выявив сильные зацепления и обязанности объектов.
- Автоматизировать пострелизные проверки по контрольным спискам, чтобы их не приходилось выполнять вручную.
- Найти непредвиденные граничные случаи в коде методом случайного ввода.

Даже такой короткий список задач тестирования показывает, что простая идея «протестировать код» на самом деле не всегда проста. Таким образом, приступая к тестированию, необходимо задать себе следующие вопросы:

- Ради какой цели я собираюсь тестировать эту часть кода?
- Какие средства позволят мне достичь этой цели наиболее точно и эффективно?

13.2. КАК ПРОВОДИТЬ ТЕСТИРОВАНИЕ

Одна из самых популярных моделей для обсуждения разных видов тестирования — пирамида тестирования, изображенная на рис. 13.1. Она впервые появилась в книге Майка Кона (Mike Cohn) «Succeeding with Agile. Software Development Using Scrum»[1] (Addison-Wesley Professional, 2009) и выражает один из принципов, по которому можно распределять ресурсы между разными типами тестирования, чтобы получить от них максимальную отдачу.

И хотя в интернете бушуют споры относительно точных границ между этими типами тестирования, основные идеи этой модели весьма полезны.

[1] Майк Кон. «Scrum: гибкая разработка ПО».

https://t.me/javalib



Рис. 13.1. Пирамида тестирования

> **ПРИМЕЧАНИЕ** Тип тестирования не определяется инструментами, которые вы применяете. Если вы работаете с JUnit, это само по себе не означает, что вы пишете модульный тест, а если вы используете библиотеку спецификаций, это еще не гарантирует, что вы действительно создаете эффективные приемочные тесты, которые принесут пользу заказчикам. То, к какому типу относится тест, определяется тем, что именно вы хотите проверить и доказать.

В основании пирамиды лежат *модульные тесты*, или *юнит-тесты*. Это узконаправленные тесты, каждый из которых проверяет один аспект системы. Но что означает «один аспект»? Чтобы это понять, проще всего посмотреть, как тестируемый код обходится с внешними зависимостями. Если тест обращается к базе данных, а потом применяет некоторую логику к результату, то тестируется уже не «один аспект»: вы тестируете чтение из базы данных *и* работоспособность вашей логики. К внешним зависимостям также могут относиться сетевые службы или файлы.

Чтобы не нарушать принцип «одного аспекта», часто используют прием с *тестовыми дублерами*, например, чтобы модульный тест обращался к суррогатному объекту вместо настоящей базы данных. Этот прием подробнее рассматривается в следующем разделе, но основная идея состоит в том, что существует много разновидностей таких суррогатов, и при работе с ними нужно учитывать немало важных деталей.

Модульные тесты привлекательны по многим причинам, поэтому их традиционно изображают как самый большой сегмент пирамиды. Вот некоторые из этих причин:

- *Скорость:* если у теста нет внешних зависимостей, его выполнение не займет много времени.
- *Целенаправленность:* когда мы тестируем отдельную программную единицу, смысл теста часто оказывается понятнее, чем в случае комплексных тестов со сложной настройкой.
- *Предсказуемость:* минимизация внешних зависимостей, особенно зависимостей от внешнего состояния, делает модульные тесты более детерминированными.

https://t.me/javalib

Если все так великолепно, почему же мы не занимаемся только модульными тестами? Дело в том, что у них есть ограничения, которые не позволяют им эффективно работать на всех уровнях, где нужно тестирование. К этим ограничениям относятся:

- *Сильное зацепление* (coupling): поскольку модульные тесты по определению нацелены на конкретную реализацию, они оказываются слишком сильно связанными с решениями, которые заложены в эту реализацию. Нередко случается так, что целый набор модульных тестов становится недействительным, когда меняется нижележащая реализация.
- *Упускаются содержательные взаимодействия:* было бы заманчиво представлять код как набор объектов, каждый из которых просто занимается своим делом. Однако в реальном ПО разные его части взаимодействуют друг с другом, а модульные тесты не охватывают эти взаимодействия.
- *Внутренняя направленность:* часто цель тестирования состоит в том, чтобы убедиться, что конечные пользователи программы получат правильные результаты. Правильность отдельного метода совершенно не гарантирует, что пользователи будут довольны.

Интеграционные тесты, которые расположены на следующем уровне пирамиды, свободны от ограничений модульных тестов в отношении зависимостей и позволяют сосредоточиться на том, насколько успешно разные компоненты системы взаимодействуют между собой.

С помощью интеграционных тестов, как и с помощью модульных, можно проверять отдельные части системы. При этом некоторые зависимости (например, внешние службы) могут по-прежнему заменяться тестовыми дублерами, тогда как другие (например, базы данных) участвуют в тестировании напрямую. Здесь принципиально то, что область действия тестов выходит за рамки одной программной единицы («модуля»).

Между модульными и интеграционными тестами не всегда удается провести четкую границу. Впрочем, следующие примеры очевидно заступают на территорию интеграционного тестирования:

- Вы используете экземпляр базы данных и вызываете код, который обращается к данным.
- Вы налаживаете специализированный служебный сервер HTTP и тестируете запросы к нему.
- Вы выполняете реальные запросы к другой службе (которая не обязательно находится в тестовой среде).

У интеграционных тестов есть ряд приятных особенностей, например:

- *Расширенное покрытие:* интеграционный тест охватывает заведомо больше вашего кода и кода библиотек, от которых зависит ваш код.

https://t.me/javalib

- *Дополнительные проверки:* некоторые виды ошибок можно обнаружить только с реальными зависимостями. Например, синтаксическую ошибку в инструкции SQL будет трудно выявить, не обращаясь к настоящей базе данных.

Конечно, здесь тоже есть как сильные, так и слабые стороны. Если недостаточно хорошо управлять интеграционными тестами, они могут создать серьезные неприятности, например:

- *Низкая скорость:* обращаться к реальной базе данных — намного более долгий процесс, чем читать значение из памяти. Умножьте эту задержку на сотни и тысячи тестов, и вы обнаружите, что рискуете не дождаться результатов тестов.
- *Недетерминированные результаты:* внешние зависимости повышают вероятность того, что важное состояние может измениться между запусками теста. Например, записи, оставшиеся в базе данных, могут изменить результат, который возвращает инструкция SQL.
- *Ложная уверенность:* иногда интеграционные тесты используют зависимости, которые чем-то отличаются от зависимостей настоящей системы. Например, если тестовая база данных имеет не ту версию, что в среде реальной эксплуатации, интеграционный тест может ошибочно засвидетельствовать, что все в порядке, хотя на самом деле это не так.

Несмотря на эти затруднения, интеграционные тесты остаются критически важной частью общего процесса тестирования.

Сквозные тесты идут еще дальше интеграционных; их функция — воспроизвести полноценное взаимодействие пользователя с системой. Например, они могут программно имитировать действия пользователя, который работает с веб-браузером или другим приложением, или обеспечивать рабочей нагрузкой полностью развернутый экземпляр службы в тестовой среде. Сквозные тесты обладают преимуществами, которые трудно воспроизвести на более низких уровнях:

- *«Реальное» взаимодействие с пользователем:* хороший сквозной тест приближен к тому, что видят живые люди. Это позволяет напрямую проверить, соответствует ли система высокоуровневым ожиданиям пользователей.
- *«Реальная» среда:* многие сквозные тесты выполняются в тестовых, инсценировочных и даже эксплуатационных средах. Таким образом они проверяют, что код работает за пределами удобной и полностью управляемой среды разработки.
- *Доступность UI:* многие методы сквозного тестирования (например, программное управление браузером) позволяют проверять такие аспекты системы, которые было бы трудно воспроизвести на других уровнях (например, была ли отрисована кнопка, чтобы пользователь мог на ней щелкнуть).

https://t.me/javalib

Однако сквозные тесты не только позволяют добиться большей реалистичности, но и добавляют новые трудности:

- *Еще более низкая скорость:* если модульные тесты выполняются почти мгновенно, и даже интеграционные тесты часто занимают меньше секунды, то сквозной тест, где симуляция пользователя «реалистично» ходит по сайту, неизбежно требует намного больше времени.
- *Нестабильные тесты:* исторически инструменты сквозного тестирования, особенно относящиеся к UI, считаются нестабильными: они часто требуют повторных попыток и долгих тайм-аутов, чтобы предотвратить лишние сбои.
- *Хрупкие тесты:* поскольку сквозные тесты расположены на вершине пирамиды, изменения на нижележащих уровнях могут привести к сбоям. Безобидные на первый взгляд текстовые правки могут случайно «сломать» тщательно подготовленный тест.
- *Усложняется отладка:* поскольку для сквозного тестирования часто нужен дополнительный уровень управления тестами, в результате становится еще сложнее определить, что же пошло не так.

Глядя на пирамиду тестирования, вы можете спросить: «В какой пропорции следует распределять тесты по уровням?» Единственно правильного ответа на этот вопрос не существует, потому что потребности разных проектов и систем различаются. Однако пирамида поможет вам оценить плюсы и минусы разных вариантов того, как можно проверить функциональность вашей системы в полном объеме.

Хотя разработка через тестирование — не единственно возможный подход, опытные разработчики убеждаются в том, что она помогает четко разграничивать разные уровни тестирования по мере эволюции системы.

13.3. Разработка через тестирование

Разработка через тестирование (TDD, Test-Driven Development) уже давно стала одной из стандартных методологий производства ПО. Ее основная идея заключается в том, что вы пишете тесты параллельно с реализацией, а не после нее, и эти тесты влияют на архитектуру кода. Один из часто рекомендуемых методов TDD, который называется *предварительной разработкой тестов* (test first), требует написать тест, который не проходит, до того, как писать основной код, а затем проводить рефакторинг по мере надобности. Например, если вы реализуете конкатенацию двух строковых объектов (`"foo"` и `"bar"`), то сначала пишется тест, который проверяет, что результат равен `"foobar"`, и позволяет вам убедиться, что реализация верна. Хотя большинство разработчиков пишут тесты, очень часто они делают это после реализации, лишаясь основных преимуществ TDD.

Несмотря на повсеместное распространение TDD, многие разработчики до сих пор не понимают, зачем применять эту практику. У разработчиков остается вопрос: «Для чего писать код, управляемый тестами? Какая от этого польза?»

https://t.me/javalib

Мы полагаем, что главная цель того, чтобы писать код, управляемый тестами, — *избавиться от страха и неопределенности*. Кент Бек (один из создателей тестового фреймворка JUnit) выразительно описывает этот фактор в своей книге «Test-Driven Development: By Example»[1]):

- Страх порождает нерешительность.
- Страх заставляет вас меньше общаться.
- Страх заставляет вас пугаться отзывов о вашей работе.
- Страх делает вас раздражительными.

TDD избавляет вас от страха, и в результате опытные Java-разработчики становятся более уверенными, коммуникабельными, отзывчивыми и вообще довольными жизнью. Иначе говоря, TDD помогает избавиться от менталитета, который проявляется в таких пагубных подходах:

- Приступая к новой работе: «Не знаю, с чего начать, поэтому просто начну писать код как попало».
- При изменении существующего кода: «Не знаю, как поведет себя существующий код, поэтому боюсь его трогать».

У TDD также немало других преимуществ, которые не всегда бросаются в глаза:

- *Более чистый код:* вы пишете только тот код, который необходим.
- *Улучшение архитектуры:* некоторые разработчики называют TDD «дизайном через тестирование» (test-driven design).
- *Улучшение API:* тесты становятся дополнительным клиентом для реализации, что помогает выявлять проблемные места на ранней стадии.
- *Повышение гибкости:* TDD стимулирует программирование, ориентированное на интерфейсы.
- *Документирование через тесты:* поскольку никакой код не пишется без тестов, для каждого компонента программы есть примеры использования в тестах.
- *Быстрая обратная связь:* вы узнаете об ошибках немедленно, а не в процессе эксплуатации.

Разработчиков, которые только начинают осваивать TDD, нередко смущает, что эта методология кажется не предназначенной для «обычных» разработчиков. Создается ощущение, будто TDD используют только последователи некой вымышленной «Секты Agile» или другого эзотерического движения и что методология не принесет пользы, если вы не будете неукоснительно соблюдать абсолютно

[1] Кент Бек. «Экстремальное программирование. Разработка через тестирование». — СПб., издательство «Питер».

https://t.me/javalib

каждый ее принцип. Как мы вскоре убедимся, такое ощущение совершенно не соответствует действительности. TDD — методология для каждого разработчика.

13.3.1. TDD в двух словах

TDD проще всего реализуется на уровне модульного тестирования, и если вы не знакомы с TDD, оно станет хорошей отправной точкой. Мы тоже начнем с этой точки, но пройдем дальше, в частности, чтобы показать, как TDD работает на стыке модульного и интеграционного тестирования.

> **ПРИМЕЧАНИЕ** Разбираться в существующем коде, для которого почти (или совсем) нет тестов, — задача не из простых. Написать все тесты задним числом практически невозможно. Гораздо лучше просто писать тесты для каждой единицы функциональности, которую вы добавляете. За дополнительной информацией обращайтесь к превосходной книге Майкла Физерса (Michael Feathers) «Working Effectively with Legacy Code»[1].

Мы начнем с ключевой концепции TDD — цикла «красный — зеленый — рефакторинг». Для практических примеров возьмем код, который вычисляет выручку от продажи театральных билетов. Мы воспользуемся фреймворком JUnit; если у вас еще нет опыта работы с ним, рекомендуем обратиться к официальному руководству (https://junit.org/junit5/docs/current/user-guide), а за более подробным описанием — к книге Кэтэлина Тудосе (Cătălin Tudose) «JUnit in Action» (Manning, 2020; http://mng.bz/gwOR).

13.3.2. Пример TDD с одним сценарием использования

Если вы уверенно практикуете TDD, этот простой пример можно пропустить, хотя не исключено, что даже из него вы почерпнете что-то новое. Допустим, вам поручено написать стопроцентно надежный метод, который вычисляет выручку от продажи театральных билетов. Бухгалтерия театра сообщает вам простые исходные правила расчета:

- Базовая цена билета — 30 долларов.
- Общая выручка = количество проданных билетов × цена.
- Театр вмещает 100 зрителей.

Поскольку театр еще не внедрил полноценную систему электронных продаж, пользователю пока приходится вручную вводить количество проданных билетов.

Если вам приходилось применять TDD на практике, вам, скорее всего, знакомы три основных этапа этой методологии: красный, зеленый, рефакторинг. Но если у вас еще нет соответствующего опыта или если вам хочется освежить память, приведем определение этих этапов от Кента Бека:

[1] Майкл Физерс. «Эффективная работа с унаследованным кодом».

https://t.me/javalib

- *Красный* — напишите небольшой тест, который не проходит (*сбойный тест*).
- *Зеленый* — как можно быстрее заставьте тест работать (*проходящий тест*).
- *Рефакторинг* — устраните дублирование (*уточненный проходящий тест*).

Рассмотрим псевдокод, который дает представление о реализации `TicketRevenue`, которую мы пытаемся построить:

```
estimateRevenue(int numberOfTicketsSold)
  if (numberOfTicketsSold меньше 0 ИЛИ  больше 100)
    Обработать ошибку и выйти
  else
    revenue = 30 * numberOfTicketsSold;
    return revenue;
  endif
```

Важный момент: на этом этапе не стоит слишком глубоко вдаваться в детали реализации. В конечном итоге именно тесты будут определять процесс проектирования кода и отчасти его реализацию.

Сбойный тест (красный)

Цель этого шага — написать тест, который не проходит. Собственно, он даже не будет компилироваться, потому что класс `TicketRevenue` еще не написан!

После непродолжительного мозгового штурма с бухгалтерией вы понимаете, что нужно предусмотреть тесты для пяти случаев: количество проданных билетов отрицательное, 0, 1, от 2 до 100 и больше 100.

> **ПРИМЕЧАНИЕ** Хорошее эмпирическое правило при написании тестов (особенно с числами) — рассмотреть стандартные случаи: 0 (null), 1 и «много» (N). Также можно пойти дальше и учесть другие ограничения для N, например отрицательное значение или значение больше максимально возможного.

Допустим, сначала вы решили написать тест, который проверяет выручку от продажи одного билета. Тест JUnit будет выглядеть примерно так (помните: на этой стадии мы не пытаемся написать идеальный, проходящий тест):

```
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;

import java.math.BigDecimal;

import static org.junit.jupiter.api.Assertions.*;

public class TicketRevenueTest {
  private TicketRevenue venueRevenue;

  @BeforeEach
  public void setUp() {
    venueRevenue = new TicketRevenue();
  }
```

https://t.me/javalib

```
  @Test
  public void oneTicketSoldIsThirtyInRevenue() {   <-- Один проданный билет
    var expectedRevenue = new BigDecimal("30");
    assertEquals(expectedRevenue, venueRevenue.estimateTotalRevenue(1));
  }
}
```

Как видно из кода, тест достаточно ясно выражает, что выручка от продажи одного билета должна быть равна 30.

Однако прямо сейчас этот тест не компилируется, потому что вы еще не написали класс `TicketRevenue` с методом `estimateTotalRevenue(int numberOfTicketsSold)`. Можно добавить фиктивную реализацию просто для того, чтобы тест прошел компиляцию и чтобы его можно было запускать:

```
public class TicketRevenue {
  public BigDecimal estimateTotalRevenue(int i) {
    return BigDecimal.ZERO;
  }
}
```

Может показаться немного странным, что тест раскрывает изменяемое поле `venueRevenue`, хотя раньше мы рекомендовали отдавать предпочтение неизменяемости. Дело в том, что совместно используемое поле позволяет определить общую настройку для разных тестовых случаев (которые вскоре появятся). Нашим тестам не нужна такая же степень защиты, как продакшен-коду, а чем яснее мы выразим, какие элементы остаются одинаковыми во всех тестовых случаях, тем эффективнее пойдет вся работа.

Итак, теперь тест компилируется, и его можно запустить из вашей любимой IDE или командной строки. Во втором случае классические системы сборки — Gradle и Maven — предоставляют удобные средства для запуска тестов (`gradle test` или `mvn test`).

> **ПРИМЕЧАНИЕ** В IDE бывают собственные инструменты для запуска тестов JUnit, но как правило, все они сводятся к тому, чтобы щелкнуть правой кнопкой мыши по классу теста и выбрать пункт Run test («Запустить тест»). Если так и сделать, IDE покажет окно или панель с сообщением о том, что тест не прошел, потому что вызов `estimateTotalRevenue(1)` вернул не ожидаемое значение 30, а 0.

Итак, у нас есть сбойный тест. На следующем шаге нужно сделать так, чтобы он проходил (зеленый этап).

Проходящий тест (зеленый)

На этом этапе нужно обеспечить, чтобы тест проходил, — при этом реализация не обязана быть оптимальной. Для этого в классе `TicketRevenue` надо улучшить реализацию `estimateTotalRevenue()`, чтобы она не просто возвращала 0.

Помните, что на этой стадии достаточно, чтобы тест стал «зеленым», и не обязательно писать идеальный код. Первый вариант решения может выглядеть примерно так:

https://t.me/javalib

```
import java.math.BigDecimal;

public class TicketRevenue {
  public BigDecimal estimateTotalRevenue(int numberOfTicketsSold) {
    BigDecimal totalRevenue = BigDecimal.ZERO;
    if (numberOfTicketsSold == 1) {
      totalRevenue = new BigDecimal("30");  <-- Реализация, которая проходит тест
    }

    return totalRevenue;
  }
}
```

Если теперь запустить тест, то он пройдет успешно, и в большинстве IDE это будет обозначено зеленой полосой или галочкой. Даже в командной строке выводится обнадеживающее зеленое сообщение, которое свидетельствует о том, что с кодом все хорошо.

Выходит, можно сказать: «Готово!» и перейти к следующей части работы? Ни в коем случае! Скорее всего, вам, как и нам, не терпится привести полученный код в порядок, так что давайте займемся этим прямо сейчас.

Рефакторинг

Цель этого шага — просмотреть реализацию, которую вы написали на скорую руку ради прохождения теста, и добиться, чтобы она следовала общепринятой практике. Разумеется, код пока получился не настолько чистым и аккуратным, каким должен быть. Его можно переработать, чтобы в дальнейшем и вам самим, и другим людям было удобно с ним обращаться.

Помните: после того, как у вас появился «зеленый» тест, *можно проводить рефакторинг без каких-либо опасений*. При этом вы не рискуете потерять бизнес-логику, которая уже реализована.

> **ПОДСКАЗКА** Если вы изначально написали проходящий тест, это даст вам и вашим коллегам еще одно преимущество: процесс разработки в целом ускорится. Остальные участники команды могут немедленно взять первую версию кода и начать тестировать ее вместе с полной кодовой базой (имеются в виду интеграционные тесты и не только).

В этом примере лучше не использовать «магические числа»: цену билета `30` стоит заменить на именованную сущность, поэтому мы правим код:

```
import java.math.BigDecimal;

public class TicketRevenue {

  private final static int TICKET_PRICE = 30;  <-- Избавились от «магических чисел»

  public BigDecimal estimateTotalRevenue(int numberOfTicketsSold) {
    BigDecimal totalRevenue = BigDecimal.ZERO;

    if (numberOfTicketsSold == 1) {
```

https://t.me/javalib

```
      totalRevenue = new BigDecimal(TICKET_PRICE *
                                    numberOfTicketsSold);
    }

    return totalRevenue;
  }
}
```

Переработали процедуру вычисления

После рефакторинга код улучшился, однако он, очевидно, не покрывает все возможные сценарии (отрицательное значение, 0, от 2 до 100 и более 100 проданных билетов).

Не пытайтесь угадывать, как должна выглядеть реализация для других тестируемых случаев, — проектирование и реализация должны определяться дальнейшими тестами. В следующем разделе мы продемонстрируем дизайн через тестирование, добавив в пример с продажей билетов другие сценарии.

Пример TDD с несколькими сценариями

В одном из стилей TDD мы добавляли бы по одному тесту для разного количества проданных билетов — отрицательного, 0, от 2 до 100 и более 100 (в соответствии с тестовыми случаями, которые мы перечислили ранее). Однако ничто не мешает написать сразу целый набор тестовых случаев, особенно если они связаны с исходным тестом.

Учтите, что и в этом случае очень важно следовать циклу «красный — зеленый — рефакторинг». После того как вы добавите все тестовые случаи, у вас может получиться такой тестовый класс со сбойными тестами (красный):

```
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;

import java.math.BigDecimal;

import static org.junit.jupiter.api.Assertions.*;

public class TicketRevenueTest {
  private TicketRevenue venueRevenue;
  private BigDecimal expectedRevenue;

  @BeforeEach
  public void setUp() {
    venueRevenue = new TicketRevenue();
  }

  @Test
  public void failIfLessThanZeroTicketsAreSold() {
    assertThrows(IllegalArgumentException.class,
                 () -> venueRevenue.estimateTotalRevenue(-1));
}

  @Test
  public void zeroSalesEqualsZeroRevenue() {
```

Продано отрицательное количество билетов

Продано 0 билетов

https://t.me/javalib

```
    assertEquals(BigDecimal.ZERO, venueRevenue.estimateTotalRevenue(0));
  }

  @Test
  public void oneTicketSoldIsThirtyInRevenue() {   <-- Продан 1 билет
    expectedRevenue = new BigDecimal("30");
    assertEquals(expectedRevenue, venueRevenue.estimateTotalRevenue(1));
  }

  @Test
  public void tenTicketsSoldIsThreeHundredInRevenue() {   <-- Продано N билетов
    expectedRevenue = new BigDecimal("300");
    assertEquals(expectedRevenue, venueRevenue.estimateTotalRevenue(10));
  }

  @Test                                                    Продано больше
  public void failIfMoreThanOneHundredTicketsAreSold() {   100 билетов
    assertThrows(IllegalArgumentException.class,
                 () -> venueRevenue.estimateTotalRevenue(101));
  }
}
```

Исходная базовая реализация, с которой все эти тесты проходят, может выглядеть примерно так:

```
import java.math.BigDecimal;

public class TicketRevenue {
  public BigDecimal estimateTotalRevenue(int numberOfTicketsSold)
    throws IllegalArgumentException {

    if (numberOfTicketsSold < 0) {
      throw new IllegalArgumentException(   <-- Исключительные случаи
                  "Должно быть > -1");
    }

    if (numberOfTicketsSold == 0) {
      return BigDecimal.ZERO;
    }

    if (numberOfTicketsSold == 1) {
      return new BigDecimal("30");
    }

    if (numberOfTicketsSold == 101) {
      throw new IllegalArgumentException(   <-- Исключительные случаи
                  "Должно быть < 101");
    }

    return new BigDecimal(30 * numberOfTicketsSold);   <-- Продано N билетов
  }
}
```

При такой реализации тесты становятся зелеными.

https://t.me/javalib

И снова, в соответствии с циклом TDD, этот код можно подвергнуть рефакторингу. Например, можно объединить недопустимые значения `numberOfTicketsSold` (< 0 или > 100) в одну команду `if`, а для всех остальных допустимых значений `numberOfTicketsSold` вычислять выручку по формуле (`TICKET_PRICE * numberOfTicketsSold`). Возможно, после рефакторинга у вас получится нечто такое:

```
import java.math.BigDecimal;

public class TicketRevenue {

  private final static int TICKET_PRICE = 30;

  public BigDecimal estimateTotalRevenue(int numberOfTicketsSold)
    throws IllegalArgumentException {

    if (numberOfTicketsSold<0|| numberOfTicketsSold > 100) {
      throw new IllegalArgumentException(  <-- Исключительный случай
                  "# Количество должно быть == 1..100");
    }

    return new BigDecimal(TICKET_PRICE *  <-- Все остальные случаи
                          numberOfTicketsSold);
  }
}
```

Класс `TicketRevenue` стал намного компактнее, и при этом с ним по-прежнему проходят все тесты! Вы завершили полный цикл «красный — зеленый — рефакторинг» и можете уверенно переходить к следующей части бизнес-логики. Также можно начать цикл заново, если окажется, что вы с бухгалтером упустили какие-нибудь важные сценарии (например, переменную цену билета).

13.4. ТЕСТОВЫЕ ДУБЛЕРЫ

Продолжая писать код по принципам TDD, вы быстро столкнетесь с ситуацией, когда ваш код обращается к некоторой зависимости или подсистеме (часто сторонней). В этом случае обычно стоит позаботиться о том, чтобы тестируемый код был изолирован от этой зависимости, потому что код тестов должен проверять только то, что вы непосредственно разрабатываете. Кроме того, тесты должны выполняться как можно быстрее, особенно если вы пишете модульные, а не интеграционные тесты. Обращение к сторонней зависимости или подсистеме (например, базе данных) может занимать много времени, отчего потеряется характерное для TDD преимущество быстрой обратной связи. Чтобы решить эту проблему, используются *тестовые дублеры*.

В этом разделе вы узнаете, как тестовые дублеры помогают эффективно изолировать зависимости и подсистемы. На примерах мы разберем четыре разновидности дублеров: пустышки (dummy), заглушки (stub), суррогаты (fake) и макеты, или имитации (mock). Мы рассмотрим не только преимущества тестовых дублеров, но и то, какие затруднения с ними связаны и как их преодолевать.

https://t.me/javalib

Нам нравится простое объяснение тестовых дублеров, которое привел Джерард Месарош (Gerard Meszaros) в книге «xUnit Test Patterns»[1], поэтому мы охотно его процитируем: «Тестовый дублер (по аналогии с дублерами в кино) — обобщенный термин, обозначающий любой объект-заменитель, который используется вместо реального объекта для целей тестирования».

Месарош определяет четыре разновидности тестовых дублеров, которые перечислены в табл. 13.1.

Таблица 13.1. Четыре разновидности тестовых дублеров

Тип	Описание
Пустышка	Объект, который передается, но не используется; обычно служит для того, чтобы выполнить формальные требования к списку параметров метода
Заглушка	Объект, который всегда возвращает один и тот же заготовленный результат; также может хранить фиктивное состояние
Суррогат	Работоспособная реализация (не обязательно эксплуатационного качества), которая заменяет фактическую реализацию
Макет	Объект, который представляет серию ожиданий и выдает заготовленные ответы

В этих разновидностях тестовых дублеров будет намного проще разобраться, если проработать примеры кода, в которых они используются. Давайте сделаем это, начиная с пустышки.

13.4.1. Пустышка

Пустышка (dummy) — простейший из четырех типов тестовых дублеров. Напомним, что его назначение — фигурировать в списках параметров или удовлетворять обязательным требованиям к полям, когда вы твердо уверены в том, что объект никогда не будет использоваться. Во многих случаях даже можно передать пустой объект (а то и `null`, хотя безопасность при этом не гарантируется).

Вернемся к сценарию с театральными билетами. Хорошо иметь оценку выручки от одной точки продажи билетов, но у владельцев театра появились более амбициозные планы. Они заговорили о том, что моделирование продаж и выручки хочется улучшить, а значит, стоит ждать новых требований и большей сложности.

Вам поручают отслеживать проданные билеты, а также предусмотреть возможность скидки 10 % на некоторые билеты. Похоже, вам понадобится класс `Ticket` с методом, который будет вычислять цену со скидкой. Вы начинаете знакомый цикл TDD со сбойного теста, сосредоточившись на новом методе `getDiscountPrice()`. Вы знаете, что классу понадобятся два конструктора: для

[1] Джерард Месарош. «Шаблоны тестирования xUnit. Рефакторинг кода тестов».

https://t.me/javalib

билета со стандартной ценой и для билета, цена которого может изменяться. В итоге объект `Ticket` будет ожидать два аргумента:

- *Имя клиента* — строка, которая вообще не будет использоваться в этом тесте.
- *Обычная цена* — значение `BigDecimal`, которое будет использоваться в тесте.

Вы абсолютно уверены, что в методе `getDiscountPrice()` не будет фигурировать имя клиента. Это означает, что конструктору можно передать пустышку (в данном случае первую попавшуюся строку `"Вася"`), как показано в следующем фрагменте:

```
import org.junit.jupiter.api.Test;

import java.math.BigDecimal;

import static org.junit.jupiter.api.Assertions.*;

public class TicketTest {

  private static String dummyName = "Вася";  <-- Создает пустышку

  @Test
  public void tenPercentDiscount() {
    Ticket ticket = new Ticket(dummyName,    <-- Передает пустышку
                               new BigDecimal("10"));
    assertEquals(new BigDecimal("9.0"), ticket.getDiscountPrice());
  }
}
```

Как видите, идея пустышки вполне тривиальна. Чтобы выразить ее максимально четко, в следующем коде используется частичная реализация класса `Ticket`:

```
import java.math.BigDecimal;

public class Ticket {

  public static final int BASIC_TICKET_PRICE = 30;  <-- Цена по умолчанию
  private static final BigDecimal DISCOUNT_RATE =   <-- Скидка по умолчанию
                                     new BigDecimal("0.9");

  private final BigDecimal price;
  private final String clientName;

  public Ticket(String clientName) {
    this.clientName = clientName;
    price = new BigDecimal(BASIC_TICKET_PRICE);
  }

  public Ticket(String clientName, BigDecimal price) {
    this.clientName = clientName;
    this.price = price;
  }

  public BigDecimal getPrice() {
```

https://t.me/javalib

```
    return price;
  }

  public BigDecimal getDiscountPrice() {
    return price.multiply(DISCOUNT_RATE);
  }
}
```

Пустышки сбивают с толку отдельных разработчиков, которые склонны искать сложность там, где ее нет. Тем не менее в пустышках нет никакой хитрости: вы просто используете простейший произвольный объект, чтобы не возбудилось `NullPointerException` и программа могла запускаться.

Перейдем к заглушкам — это следующая разновидность тестовых дублеров в порядке возрастания сложности.

13.4.2. Заглушка

Заглушку (stub) обычно используют тогда, когда имеет смысл подменить реальную реализацию объектом, который каждый раз возвращает один и тот же ответ. Вернемся к примеру с продажей билетов, чтобы продемонстрировать работу заглушек.

Класс `Ticket` успешно реализован, вы возвращаетесь после заслуженных выходных — и немедленно обнаруживаете отчет об ошибке: оказывается, тест `tenPercentDiscount()` время от времени не проходит. Заглянув в кодовую базу, вы видите, что класс `Ticket` теперь использует конкретный класс `HttpPrice`, который реализует недавно добавленный интерфейс `Price`. Как подсказывает имя, `HttpPrice` обращается к внешнему сайту и в разные моменты времени может возвращать разные значения (или выдавать ошибку).

Эти перемены не только приводят к сбою теста, но и размывают цель нашего тестирования. Вспомните, что изначально мы просто писали модульные тесты, которые проверяют механизм вычисления 10-процентной скидки!

> **ПРИМЕЧАНИЕ** Разумеется, обращение к стороннему сайту с ценами не входит в обязанности этого теста. Класс `HttpPrice` и его сторонняя служба `HttpPricingService` должны покрываться отдельными интеграционными тестами.

Чтобы вернуть тест к стабильному целостному состоянию, заменим класс `HttpPrice` заглушкой. Сперва рассмотрим текущий код, три фрагмента которого представлены ниже:

```
import org.junit.jupiter.api.Test;

import java.math.BigDecimal;

import static org.junit.jupiter.api.Assertions.*;

public class TicketTest {

  private static String dummyName = "Вася";
```

https://t.me/javalib

```
  @Test
  public void tenPercentDiscount() {
    Price price = new HttpPrice();   <-- HttpPrice реализует интерфейс Price
    Ticket ticket = new Ticket(dummyName, price);   <-- Создает экземпляр Ticket
    assertEquals(new BigDecimal("9.0"),   <-- Тест может не пройти
                 ticket.getDiscountPrice());
  }
}
```

В следующем фрагменте приведена новая реализация класса `Ticket`:

```
import java.math.BigDecimal;

public class Ticket {
  private final String clientName;
  private final Price priceSource;
  private final BigDecimal discountRate;

  private BigDecimal faceValue = null;

  public Ticket(String clientName,
                Price price,
                BigDecimal discountRate) {   <-- Измененный конструктор
    this.clientName = clientName;
    this.priceSource = price;
    this.discountRate = discountRate;
  }

  public BigDecimal getPrice() {
    if (faceValue == null) {
      faceValue = priceSource.getInitialPrice();   <-- Новый вызов getInitialPrice
    }

    return faceValue;
  }

  public BigDecimal getDiscountPrice() {
    return faceValue.multiply(discountRate);   <-- Вычисления не изменились
  }
}
```

Полная реализация класса `HttpPrice` слишком сильно отвлекла бы нас от главного, поэтому просто предположим, что он вызывает другой класс `HttpPricingService`:

```
import java.math.BigDecimal;

public interface Price {
  BigDecimal getInitialPrice();
}

public class HttpPrice implements Price {
  @Override
```

https://t.me/javalib

```
  public BigDecimal getInitialPrice() {
    return HttpPricingService.getInitialPrice(); ◄── Возвращает случайные результаты
  }
}
```

Теперь давайте подумаем, что же мы собираемся тестировать. Наша цель — убедиться, что умножение в методе `getDiscountPrice()` класса `Ticket` работает так, как предполагается. Чтобы это проверить, никакие внешние сайты не нужны.

Интерфейсы `Price` предоставляют ту границу, на которой стоит заменить капризный экземпляр `HttpPrice` стабильной реализацией `StubPrice`:

```
import org.junit.jupiter.api.Test;

import java.math.BigDecimal;

import static org.junit.jupiter.api.Assertions.*;

public class TicketTest {
  @Test
  public void tenPercentDiscount() {
    Price price = new StubPrice(); ◄── Заглушка StubPrice
    Ticket ticket = new Ticket(price); ◄── Создает Ticket
    assertEquals(new BigDecimal("9.0"), ◄── Проверяет price
                 ticket.getDiscountPrice());
  }
}
```

Простой маленький класс `StubPrice` будет стабильно возвращать исходную цену, равную 10:

```
import java.math.BigDecimal;

public class StubPrice implements Price {
  @Override
  public BigDecimal getInitialPrice() {
    return new BigDecimal("10"); ◄── Возвращает стабильную цену
  }
}
```

Теперь тест снова проходит, и, что не менее важно, можно без опасений приступать к рефакторингу остальной реализации.

Заглушки — полезная разновидность тестовых дублеров, но иногда бывает нужно, чтобы заглушка выполняла какую-то реальную работу, приближенную к тому, что происходит в режиме эксплуатации. В таких ситуациях в качестве тестовых дублеров используются суррогаты.

13.4.3. Суррогаты

Суррогат (fake) можно рассматривать как улучшенную заглушку, которая делает почти то же самое, что продакшен-код, но использует упрощенное поведение, чтобы удовлетворить требованиям к тестированию.

https://t.me/javalib

Суррогаты особенно полезны, когда вы хотите, чтобы ваш код взаимодействовал с чем-то очень похожим на реальную стороннюю подсистему или зависимость, которая будет использоваться в среде эксплуатации.

Представьте, что в нашем приложении с продажей билетов уровень базы данных предоставляет простой интерфейс для работы с билетами:

```
package com.wellgrounded;

public interface TicketDatabase {
    Ticket findById(int id);
    Ticket findByName(String name);
    int count();

    void insert(Ticket ticket);
    void delete(int id);
}
```

Наш класс, управляющий отдельным спектаклем, должен взаимодействовать с этим интерфейсом базы данных и управлять различными функциями, например проверять, не распроданы ли все билеты:

```
package com.wellgrounded;

import java.math.BigDecimal;

public class Show {
    private TicketDatabase db;
    private int capacity;

    public Show(TicketDatabase db, int capacity) {
        this.db = db;
        this.capacity = capacity;
    }

    public void addTicket(String name, BigDecimal amount) {
        if (db.count() < capacity) {
            var ticket = new Ticket(name, amount);
            db.insert(ticket);
        } else {
            throw new RuntimeException("Все билеты проданы");
        }
    }
}
```

Нам хотелось бы провести модульное тестирование метода `addTicket`, не задействуя полноценный экземпляр реляционной базы данных. Соответствующий тест мог бы выглядеть так:

```
package com.wellgrounded;

import org.junit.jupiter.api.Test;
import java.math.BigDecimal;
import static org.junit.jupiter.api.Assertions.*;
```

https://t.me/javalib

```
public class ShowTest {
    @Test
    public void plentyOfSpace() {
        var db = new FakeTicketDatabase();
        var show = new Show(db, 5);

        var name = "Новая продажа";
        show.addTicket(name, BigDecimal.ONE);

        var mine = db.findByName(name);
        assertEquals(name, mine.getName());
        assertEquals(BigDecimal.ONE, mine.getAmount());
    }
}
```

FakeTicketDatabase не существует, но в духе TDD мы напишем код, который должен проходить тест

Хотя результата можно было добиться при помощи заглушек, такое решение обладало бы рядом недостатков. Нам пришлось бы заменять заглушками методы `count` и `insert`, которые даже не видны в тесте, и в итоге получилось бы, что тест загромождён низкоуровневыми подробностями и отклоняется от настоящей цели. Однако это еще не все затруднения: каждый тест также должен обеспечивать, чтобы значение `count` оставалось согласованным с количеством вызовов `insert`. И что еще хуже, итоговый вызов `findByName`, который должен проверять, что данные сохранились, тоже пришлось бы заменять заглушкой. Но это означает, что само тестовое утверждение теряет смысл: оно удовлетворяется независимо от того, правилен ли код реализации! Заглушки не позволяют нам полноценно проверить этот набор тесно связанных действий.

Суррогат предоставляет альтернативу — реальную, хотя и упрощенную реализацию. Необходимый интерфейс легко реализуется оберткой вокруг простой коллекции `HashMap`:

```
package com.wellgrounded;

import java.util.HashMap;

class FakeTicketDatabase implements TicketDatabase {
    private HashMap<Integer, Ticket> tickets =
                                        new HashMap<>();
    private Integer nextId = 1;

    @Override
    public Ticket findByName(String name) {
        var found = tickets.values()
                .stream()
                .filter(ticket -> ticket.getName().equals(name))
                .findFirst();
        return found.orElse(null);
    }

    @Override
    public int count() {
        return tickets.size();
    }
```

Отображение имитирует базу данных на время всего срока существования модульного теста

Необходимо воспроизвести такую функциональность, как автоматическое наращивание идентификаторов в базе данных

https://t.me/javalib

```
    @Override
    public void insert(Ticket ticket) {
        tickets.put(nextId, ticket);
        nextId++;
    }

    // Остальные методы доступны в сопроводительных материалах
}
```

Суррогатные объекты, особенно когда они совместно используются в проекте с сильными интерфейсами, могут эффективно сопровождать модульные тесты. Суррогаты уместны не везде: если бы интерфейс базы данных позволял передавать предложения SQL для дополнительной фильтрации, наш суррогат с этим бы не справился. Тем не менее суррогаты — полезный инструмент; просто следите за тем, чтобы реализация не становилась слишком большой или сложной, потому что каждая строка кода, который вы пишете, потенциально может стать источником дефектов.

13.4.4. Макеты

Макеты, или имитации (mock objects), в чем-то похожи на уже знакомые вам заглушки, однако заглушки обычно весьма примитивны. Например, они часто имитируют методы так, чтобы те всегда возвращали один и тот же результат. Такой подход не позволяет моделировать поведение, которое зависит от состояния.

Например, вы пытаетесь следовать методологии TDD и пишете систему анализа текста. Один из модульных тестов заставляет классы анализа текста подсчитать количество вхождений подстроки «Java11» в некотором сообщении из блога. Но поскольку блог — сторонний ресурс, возникает целый ряд возможных сбойных сценариев, не имеющих отношения к алгоритму подсчета, который вы реализуете. Иначе говоря, тестируемый код не изолирован, а обращения к стороннему ресурсу могут отнимать время. Вот несколько стандартных сценариев, которые приводят к сбою:

- Вашему коду не удается выйти в интернет, чтобы получить сообщение из блога, из-за ограничений доступа, которые действуют в вашей организации.
- Сообщение в блоге перемещено без перенаправления.
- Сообщение в блоге отредактировано так, что в нем стало больше или меньше вхождений «Java11».

Написать такой тест на заглушках почти невозможно, и для каждого тестового случая его код получился бы слишком обширным. На помощь приходит *макет* — особая разновидность тестового дублера, которую можно считать программируемой заглушкой. Пользоваться макетом очень просто: когда вы готовите макет, вы сообщаете ему, какую последовательность вызовов он должен ожидать и как реагировать на каждый вызов.

https://t.me/javalib

Рассмотрим макеты в действии на простом примере с продажей билетов, задействовав популярную библиотеку макетов Mockito (https://site.mockito.org):

```
import org.junit.jupiter.api.Test;
import java.math.BigDecimal;
import static org.junit.jupiter.api.Assertions.*;
import static org.mockito.Mockito.*;

public class TicketTest {
  @Test
  public void tenPercentDiscount() {
    Price price = mock(Price.class);  <-- Создает макет

    when(price.getInitialPrice())
      .thenReturn(new BigDecimal("10"));  <-- Программирует макет для тестирования

    Ticket ticket = new Ticket(price, new BigDecimal("0.9"));
    assertEquals(new BigDecimal("9.0"), ticket.getDiscountPrice());

    verify(price).getInitialPrice();
  }
}
```

Чтобы создать макет, мы вызвали статический метод `mock()` с объектом класса того типа, который мы имитируем. Затем мы запрограммировали, как макет должен себя вести: метод `when()` указывает, какой метод нужно «отлавливать», а `thenReturn()` описывает ожидаемый результат. Наконец, с помощью `verify()` мы убедились, что вызвали нужные методы макетируемого объекта. Тем самым можно гарантировать, что мы не пришли к правильному результату по неправильному пути.

Эта проверка демонстрирует серьезные различия между заглушкой и макетом. Основное назначение заглушки — возвращать заготовленные значения, а цель макета — *проверять поведение* (например, какие именно вызовы были сделаны). На практике метод `mock()` из Mockito позволяет создавать и обычные заглушки, если не использовать `verify()`, но при этом важно, чтобы вы как программист понимали, что именно вы тестируете.

Макет можно использовать как обычный объект и передать при вызове конструктора `Ticket` без лишних церемоний. В результате макеты становятся чрезвычайно мощным инструментом TDD, и некоторые специалисты почти не применяют другие разновидности тестовых дублеров, предпочитая делать практически все при помощи макетов. Но как и у многих мощных инструментов, у макетов есть подводные камни, о которых необходимо знать.

13.4.5. Проблемы с макетами

Один из самых больших недостатков тестовых дублеров заключается именно в том, что они фиктивны, поэтому могут вести себя не так, как объекты в реальной системе. Если не помнить об этом, у вас может сохраняться приятное чувство,

https://t.me/javalib

будто вы обеспечили полное тестовое покрытие, — пока вы не столкнетесь с суровой реальностью.

Различия в поведении между макетами и реальной системой проявляются в разных формах, в том числе:

- Различия в возвращаемых данных, особенно в сложных вложенных объектах.
- Реальные данные сериализуются (десериализуются) не так, как тестовые.
- По-разному упорядочиваются элементы в коллекциях.
- Разная реакция на ошибки: возбуждаются исключения не тех типов (или не возбуждаются вовсе).

И хотя единого решения всех этих проблем не существует, их часто удается выявить, перейдя на уровень интеграционных тестов. И снова, если вы твердо представляете себе, *что именно* вы тестируете в каждом наборе тестов, вы сможете сосредоточить модульные тесты на локализованной логике, а в остальных местах применять интеграционное тестирование, чтобы покрыть взаимодействия с зависимостями.

Вдумчивое проектирование интерфейсов — еще один фактор, улучшающий тестовые дублеры. Если класс службы возвращает не просто сырую строку содержимого от вызова HTTP, а конкретный объект, то у тестовых дублеров остается меньше возможностей для отклонения. Когда для исключений, которые возбуждает ваш класс, предусмотрены точные подклассы в форме оберток над более примитивными типами исключений, это не только делает ваш код выразительнее, но и позволяет проще строить более точные макеты.

Если злоупотреблять макетами, можно прийти к тому, что тесты начнут слишком близко воспроизводить эксплуатационный код. Тогда у каждой строки реального кода будет строка-двойник в тестовой конфигурации, воспроизводящая конкретный ожидаемый вызов. Такие тесты получаются крайне хрупкими, а правка кода часто приводит к целому каскаду вроде бы несвязанных изменений.

Хрупкость макетов распространяется вплоть до уровня отдельных аргументов. Хотя фреймворки помогают чрезвычайно точно описать передаваемые значения, подумайте, действительно ли вашему тесту нужно проверять этот аргумент. Как мы продемонстрировали в разделе про пустышки, в некоторых тестовых случаях конкретное значение может не играть роли. Механизмы макетных фреймворков позволяют эффективно выражать описания вроде «любое целое число», и если значение действительно может быть любым, такие описания одновременно поясняют, что важно для вашего теста, и позволяют эксплуатационному коду проще эволюционировать. Оставляйте тестам немного пространства для маневра!

Если для запуска тестов требуется много сложной подготовки, это тоже может свидетельствовать о проблемах. Макетирование, особенно в сочетании с внедрением зависимостей, способно легко и непринужденно превратить класс в свалку зависимостей. Если вы настраиваете все необходимое для теста дольше, чем он потом отрабатывает и проверяет результаты, — весьма вероятно, что

https://t.me/javalib

ваши классы слишком сложны и было бы неплохо заняться рефакторингом. При подготовке тестов также легко обнаружить нарушения *закона Деметры*, который утверждает, что объекты должны располагать информацией только о своих непосредственных соседях. Если на этапе подготовки теста вам приходится возиться с объектами, которые расположены через несколько уровней, это может означать, что ваши объекты уделяют чересчур много внимания тому, что их непосредственно не касается.

Тестовые дублеры — ценный инструмент для опытного разработчика. Ранее мы уже пользовались библиотекой JUnit, но не рассматривали ее подробно. Давайте познакомимся с ней поближе и узнаем, какие новые возможности появились в JUnit 5 — новейшей основной версии.

13.5. ОТ JUNIT 4 К 5

JUnit — реализация на базе JVM тестового фреймворка в стиле xUnit; изначально ее разработали Кент Бек и Эрих Гамма. Предложенный ими подход к модульному тестированию оказался гибким и простым в использовании, благодаря чему библиотека JUnit стала одной из самых популярных в экосистеме JVM.

Однако долгая история и массовая популярность также послужили причиной многочисленных ограничений. Версия JUnit 4, которая вышла в 2006 году, не могла использовать такие возможности, как лямбда-выражения, не нарушая совместимость. В 2017 году была выпущена JUnit 5; благодаря обновлению основной версии в ней удалось внедрить серьезные изменения.

> **ПРИМЕЧАНИЕ** Значительная часть следующей главы будет посвящена другим инструментам и методам, однако опытный Java-разработчик почти наверняка будет сталкиваться с кодом JUnit, который имеет смысл перенести на более современную версию.

Одно из самых заметных изменений JUnit 5 — способ упаковки. Если предыдущие версии были монолитными и содержали как API для создания тестов, так и средства поддержки запуска и отчетов по этим тестам, то JUnit 5 разбит на более узконаправленные пакеты. Также у JUnit 5 больше нет внешней зависимости от Hamcrest, которая была у JUnit 4.

JUnit 5 поставляется в совершенно новом пакете `org.junit.jupiter`, благодаря чему обе версии могут сосуществовать во время миграции. Вскоре мы подробнее рассмотрим, как это организовать.

У JUnit 5 есть две основные зависимости:

- `org.junit.jupiter.junit-jupiter-api` — используется из кода тестов, чтобы предоставить все необходимые аннотации и вспомогательные средства для написания тестов.
- `org.junit.jupiter.junit-jupiter-engine` — стандартное ядро для запуска тестов JUnit 5. Эта зависимость необходима только во время выполнения,

https://t.me/javalib

а не во время компиляции, причем ее можно дополнять или заменять другими исполнителями тестов.

В Gradle эти зависимости подключаются примерно так (с аннотацией, которая заставляет Gradle использовать новые компоненты JUnit при запуске тестов):

```
dependencies {
  testImplementation("org.junit.jupiter:junit-jupiter-api:5.7.1")
  testRuntimeOnly("org.junit.jupiter:junit-jupiter-engine:5.7.1")
}

tasks.named<Test>("test") {
  useJUnitPlatform()
}
```

Эквивалент для Maven приведен ниже. Плагины Maven `surefire` и `failsafe` умеют автоматически работать с JUnit 5, если у вас установлена достаточно новая версия (рекомендуется 2.22 или выше):

```
<project>
  <dependencies>
    <dependency>
      <groupId>org.junit.jupiter</groupId>
      <artifactId>junit-jupiter-api</artifactId>
      <version>5.7.1</version>
      <scope>test</scope>
    </dependency>
    <dependency>
      <groupId>org.junit.jupiter</groupId>
      <artifactId>junit-jupiter-engine</artifactId>
      <version>5.7.1</version>
      <scope>test</scope>
    </dependency>
  </dependencies>
</project>
```

Если добавить все это в проект JUnit 4 и просто запустить тесты, результат может обескуражить: скорее всего, тесты пройдут, но если внимательно присмотреться к отчетам, окажется, что *ни один тест на самом деле не запускался*! Дело в том, что в JUnit 5 изменилась аннотация для пометки тестовых случаев.

> **ПРИМЕЧАНИЕ** JUnit 5 предоставляет свою аннотацию `@Test` в пакете `org.junit.jupiter.api`. По умолчанию JUnit 5 не распознает существующие тесты, помеченные старой версией `@Test` из `org.junit`.

Далее возможны два пути. Первый: во всех местах, где импортировались старые аннотации, изменить код под новую версию. Постепенно, класс за классом, ваш набор тестов начнет работать в JUnit 5. Такие преобразования могут потребовать дополнительной работы, о которой пойдет речь ниже.

Альтернативное решение — подключить дополнительную зависимость времени выполнения от `junit-vintage-engine`. Этот пакет использует расширенные возможности JUnit 5, которые позволяют интегрировать разные исполнители

https://t.me/javalib

и вспомогательные классы, чтобы обеспечить обратную совместимость с тестами JUnit 4 (и даже 3).

Конфигурация для Gradle:

```
dependencies {
  testImplementation("org.junit.jupiter:junit-jupiter-api:5.7.1")
  testRuntimeOnly("org.junit.jupiter:junit-jupiter-engine:5.7.1")

  testRuntimeOnly("org.junit.vintage:junit-vintage-engine:5.7.1")
}
```

Конфигурация для Maven:

```
<project>
  <dependencies>
    <dependency>   <-- Поддержка для запуска тестов JUnit 4 параллельно с JUnit 5
      <groupId>junit</groupId>
      <artifactId>junit</artifactId>
      <version>4.13</version>
      <scope>test</scope>
    </dependency>
    <dependency>
      <groupId>org.junit.vintage</groupId>
      <artifactId>junit-vintage-engine</artifactId>
      <version>5.7.1</version>
      <scope>test</scope>
    </dependency>
    <dependency>   <-- Основные зависимости, API и ядро JUnit 5
      <groupId>org.junit.jupiter</groupId>
      <artifactId>junit-jupiter-api</artifactId>
      <version>5.7.1</version>
      <scope>test</scope>
    </dependency>
    <dependency>
      <groupId>org.junit.jupiter</groupId>
      <artifactId>junit-jupiter-engine</artifactId>
      <version>5.7.1</version>
      <scope>test</scope>
    </dependency>
  </dependencies>
</project>
```

Этот метод упростит переход, потому что вы сможете включить JUnit 5, а потом постепенно преобразовывать один тест за другим, а не все сразу. Однако стоит заметить, что старые версии поддерживаются с определенными ограничениями, которые хорошо документированы в руководстве пользователя JUnit: http://mng.bz/5Q61.

Кроме нового механизма упаковки, некоторые классы были переименованы, чтобы новые имена точнее отражали их назначение:

- `@Before` → `@BeforeEach`
- `@After` → `@AfterEach`

https://t.me/javalib

- `@BeforeClass` → `@BeforeAll`
- `@AfterClass` → `@AfterAll`
- `@Category` → `@Tag`
- `@Ignored` → `@Disabled` (или может обрабатываться с `ExecutionCondition` в новой модели расширений)
- `@RunWith`, `@Rule` и `@ClassRule` заменены новой моделью расширений.

Еще одно важное новшество JUnit 5 — новая модель расширений, которая объединяет множество отдельных возможностей из предыдущих версий JUnit. Эти возможности позволяли разным классам совместно использовать поведение (например, такие этапы, как подготовительная фаза, очистка и настройка ожиданий), но они не очень хорошо сочетались друг с другом.

Например, вот простой тест JUnit 4, который должен запустить сервер перед тем, как выполнять тесты. Он использует класс `ExternalResource` с аннотацией `@Rule`, чтобы указать, в каких точках цикла тестирования надо запускать соответствующий код:

```
package com.wellgrounded;

import org.junit.Rule;
import org.junit.Test;
import org.junit.rules.ExpectedException;
import org.junit.rules.ExternalResource;

import static org.junit.Assert.*;

public class PasswordCheckerTest {
    private PasswordChecker checker = new PasswordChecker();

    @Rule
    public ExternalResource passwordServer =
                              new ExternalResource() {
        @Override
        protected void before() throws Throwable {
            super.before();
            checker.reset();
            checker.start();
        }

        @Override
        protected void after() {
            super.after();
            checker.stop();
        }
    };

    @Test
    public void ok() {
        assertTrue(checker.isOk("abcd1234!"));
    }
}
```

@Rule требует, чтобы этот код запускался до и после каждого теста

JUnit предоставляет класс ExternalResource специально для таких нестандартных сценариев подготовки или очистки

https://t.me/javalib

Переопределения `ExternalResource` можно легко перенести в другое место, чтобы разные тесты использовали их совместно.

Вместо этого в JUnit 5 цикл тестирования разбивается на меньшие интерфейсы, которые вам нужно реализовать. Затем эти расширения можно применять на уровне классов или тестовых методов:

```
package com.wellgrounded;

import org.junit.jupiter.api.extension.AfterEachCallback;
import org.junit.jupiter.api.extension.BeforeEachCallback;
import org.junit.jupiter.api.extension.ExtensionContext;

public class PasswordCheckerExtension
    implements AfterEachCallback, BeforeEachCallback {

    private PasswordChecker checker;

    PasswordCheckerExtension(PasswordChecker checker) {
        this.checker = checker;
    }

    @Override
    public void beforeEach(ExtensionContext context) {
        checker.reset();
        checker.start();
    }

    @Override
    public void afterEach(ExtensionContext context) {
        checker.stop();
    }
}
```

Мы реализуем AfterEachCallback и BeforeEachCallback, как это делалось ранее для каждого тестового метода. Также существуют интерфейсы AfterAllCallback и BeforeAllCallback, которые заменяют функциональность @ClassRule

Поскольку наше расширение работало с полем класса теста, его необходимо передать при конструировании

Обратные вызовы, которые выполняют подготовку и очистку, как и раньше

Имея такой класс, можно применить его к тесту:

```
package com.wellgrounded;

import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.RegisterExtension;

import static org.junit.Assert.*;

public class PasswordCheckerTest {
    private static PasswordChecker checker = new PasswordChecker();

    @RegisterExtension
    static PasswordCheckerExtension ext =
            new PasswordCheckerExtension(checker);

    @Test
    public void ok() {
```

@RegisterExtension позволяет создать экземпляр расширения для теста

Чтобы поле тестового класса работало с @RegisterExtension, оно должно быть объявлено открытым

https://t.me/javalib

```
        assertTrue(checker.isOk("abcd1234!"));
    }
}
```

Если расширение не требует параметров при конструировании, его также можно применить к определению метода или класса с помощью аннотации `@ExtendWith`:

```
@ExtendWith(CustomConfigurationExtension.class)
public class PasswordCheckerTest {
    // ....
}
```

Благодаря тому что JUnit 5 начал требовать более новых версий JDK, удалось навести порядок с некоторыми проблемными местами, в которых правила когда-то были стандартным подходом. В JUnit 4 и более ранних версиях было два способа проверить, возбудил ли тестовый метод исключение:

```
package com.wellgrounded;

import org.junit.Rule;
import org.junit.Test;
import org.junit.rules.ExpectedException;

import static org.junit.Assert.*;

public class PasswordCheckerTest {
    private PasswordChecker checker = new PasswordChecker();

    @Rule
    public ExpectedException ex = ExpectedException.none();

    @Test
    public void nullThrows() {
        ex.expect(IllegalArgumentException.class);
        checker.isOk(null);
    }

    @Test(expected = IllegalArgumentException.class)
    public void alsoThrows() {
        checker.isOk(null);
    }
}
```

Проверка исключений на основе правил для тестового метода

Аннотация @Test с конфигурацией ожидаемого исключения

Лямбда-выражения позволяют записывать такие конструкции новым способом, который больше похож на типичные тестовые утверждения:

```
package com.wellgrounded;

import org.junit.jupiter.api.Test;

import static org.junit.Assert.*;

public class PasswordCheckerTest {
    private static PasswordChecker checker = new PasswordChecker();
```

https://t.me/javalib

```
    @Test
    public void nullThrows() {
        assertThrows(IllegalArgumentException.class, () -> {
            checker.isOk(null);
        });
    }
}
```

Решение с `assertThrows` предпочтительнее, чем старый аргумент `expected` у тестовых аннотаций или правило `ExpectedException`, по двум причинам. Во-первых, тестовые утверждения более адресны: они расположены там же, где код, который мы фактически тестируем, а не где-то ранее в методе. Во-вторых, `assertThrows` возвращает исключение, не заставляя нас сооружать `try/catch` на месте, поэтому становится легче формулировать типичные тестовые утверждения без специальной инфраструктуры. Хотя наследие JUnit 4 в экосистеме таково, что эта версия продолжит существовать (и сопровождаться) еще довольно долго, пользователям JUnit стоит изучить, что предлагает новая версия.

Впрочем, тестовые библиотеки — это лишь часть общей картины, особенно когда вы пишете интеграционные тесты. В следующей главе мы рассмотрим, как с помощью некоторых полезных средств решать классические проблемы тестирования с внешними зависимостями.

ИТОГИ

- Мы рассмотрели цели и разновидности тестирования, а также остановились на том, почему важно понимать, что именно вы тестируете, чтобы определиться с подходом.
- Мы познакомились с методологией разработки через тестирование (TDD) и выяснили, как она помогает поэтапно развивать архитектуру системы.
- Мы узнали, какие бывают виды тестовых дублеров, для чего они нужны и, что еще важнее, где могут возникнуть проблемы, если неправильно их использовать.
- После долгих лет ожидания вышла новая версия JUnit, которая решает давние архитектурные проблемы, часто обусловленные ограничениями устаревших JDK. Мы в общих чертах рассказали о том, как перейти на новейшую версию.

https://t.me/javalib

14 Тестирование за пределами JUnit

В ЭТОЙ ГЛАВЕ

- ✓ Интеграционное тестирование с Testcontainers
- ✓ Спецификационное тестирование в Spek и Kotlin
- ✓ Тестирование на основе свойств с помощью Clojure

В предыдущей главе мы рассмотрели общие принципы, на которые стоит опираться при тестировании. Теперь мы глубже изучим конкретные подходы к тому, как повысить качество тестов в разных ситуациях. Независимо от того, какие цели вы преследуете — тщательнее протестировать зависимости, улучшить взаимодействия в тестовом коде или даже выявить краевые случаи, которые не были обнаружены ранее, — экосистема JVM предоставляет множество инструментов, которые в этом помогают, и в этой главе будут представлены лишь некоторые из них. Начнем с классической проблемы — как эффективно обращаться с внешними зависимостями при интеграционном тестировании.

14.1. ИНТЕГРАЦИОННОЕ ТЕСТИРОВАНИЕ С TESTCONTAINERS

Чем дальше мы поднимаемся по пирамиде тестирования от изолированных модульных тестов, тем больше препятствий нам встречается. Чтобы провести интеграционное тестирование взаимодействия с реальной базой данных,

https://t.me/javalib

нужен доступ к реальной базе данных! Реалистичное тестирование такого рода дает много преимуществ, но и подготовка тестов значительно усложняется. У таких внешних систем есть состояние, поэтому повышается вероятность, что тесты не будут проходить, — не только из-за проблем с нашим кодом, но и из-за того, что между тестами каким-то образом сохранилось непредвиденное состояние.

За прошедшие годы описанные проблемы пытались решать по-разному: от баз данных в памяти до фреймворков, запускающих тесты полностью в транзакциях, которые выполняют очистку после себя. Однако эти решения часто создают собственные краевые случаи и трудности.

Контейнеризация, описанная в главе 12, предоставляет интересный новый подход к проблеме. Так как контейнеры эфемерны, их можно эффективно разворачивать специально для запуска отдельного набора тестов. А поскольку они инкапсулируют реальные базы данных и другие службы, с которыми мы хотим взаимодействовать, это позволяет избежать трудноуловимых нюансов несогласованности, которым подвержены базы данных в памяти.

14.1.1. Установка testcontainers

Один из простейших способов задействовать контейнеры в тестировании предлагает библиотека `testcontainers` (https://www.testcontainers.org/). Она предоставляет API, через который можно управлять контейнерами непосредственно из кода тестов, и поддерживает широкий набор модулей для типичных зависимостей. Базовую функциональность обеспечивает файл JAR `org.testcontainers.testcontainers`; вот его пример для Maven:

```
<dependency>
  <groupId>org.testcontainers</groupId>
  <artifactId>testcontainers</artifactId>
  <version>1.15.3</version>
  <scope>test</scope>
</dependency>
```

А вот конфигурация для Gradle:

```
testImplementation "org.testcontainers:testcontainers:1.15.3"
```

14.1.2. Пример с Redis

Если помните, мы оставили приложение для продажи билетов в той точке, где оно загружало цены от службы HTTP. Нам хотелось бы обеспечить кэш для этих значений. И хотя о правильном кэшировании можно было бы написать отдельную главу, представим, что мы решили использовать внешний кэш, а не просто хранить значения в памяти. Типичное хранилище данных для таких случаев — Redis (https://redis.io/), который отличается невероятно быстрым доступом к операциям чтения, записи и удаления пар «ключ — значение», а также более сложных структур данных.

https://t.me/javalib

Интерфейс `Price`, которому мы уже поручили получать данные через службу HTTP, достаточно гибок для того, чтобы можно было добавить кэширование как отдельную обязанность:

```
package com.wellgrounded;

import redis.clients.jedis.Jedis;

import java.math.BigDecimal;

public class CachedPrice implements Price {

    private final Price priceLookup;
    private final Jedis cacheClient;

    private static final String priceKey = "price";

    CachedPrice(Price priceLookup, Jedis cacheClient) {
        this.priceLookup = priceLookup;
        this.cacheClient = cacheClient;
    }

    @Override
    public BigDecimal getInitialPrice() {
        String cachedPrice = cacheClient.get(priceKey);
        if (cachedPrice != null) {
            return new BigDecimal(cachedPrice);
        }

        BigDecimal price =
            priceLookup.getInitialPrice();
        cacheClient.set(priceKey,
                        price.toPlainString());
        return price;
    }
}
```

Имя ключа, с которым будет кэшироваться цена в Redis

Для доступа к Redis мы используем библиотеку Jedis (https://github.com/redis/jedis)

Проверяет, есть ли цена в кэше

Если цены в кэше нет, используется значение, полученное по запросу

Кэширует только что полученное значение

В этой точке стоит сделать паузу и задуматься над тем, какой аспект системы мы хотим протестировать. Основное назначение класса `CachedPrice` — взаимодействие между Redis и нижележащим механизмом запроса цены. Здесь принципиально то, как мы работаем с Redis, и Testcontainers позволяет провести тестирование с реальной системой:

```
package com.wellgrounded;

import org.junit.jupiter.api.Test;
import org.testcontainers.containers.GenericContainer;
import org.testcontainers.junit.jupiter.*;
import org.testcontainers.utility.DockerImageName;
import redis.clients.jedis.*;

import java.math.BigDecimal;

import static org.junit.jupiter.api.Assertions.assertEquals;
```

https://t.me/javalib

```
@Testcontainers
public class CachedPriceTest {
    private static final DockerImageName imageName =
            DockerImageName.parse("redis:6.2.3-alpine");
    @Container
    public static GenericContainer redis = new GenericContainer(imageName)
            .withExposedPorts(6379);

    // Дальнейшие тесты...
}
```

В этой начальной секции теста демонстрируется простейшая форма подключения Testcontainers. Аннотация `@Testcontainers` применяется к тестовому классу в целом, сообщая библиотеке, что она должна отслеживать требуемые контейнеры во время выполнения теста. Затем поле с пометкой `@Container` запрашивает запуск конкретного образа контейнера `"redis:6.2.3-alpine"` на стандартном порте Redis 6379.

Когда выполняется этот тестовый класс (рис. 14.1), Testcontainers запускает запрошенный контейнер. Testcontainers ожидает в течение стандартного таймаута (60 секунд), пока будет доступен первый переадресованный порт, чтобы мы могли быть уверены, что с контейнером можно взаимодействовать. Затем поле `redis` позволяет получить реквизиты (например, имя узла и порты), которые можно использовать в тесте.

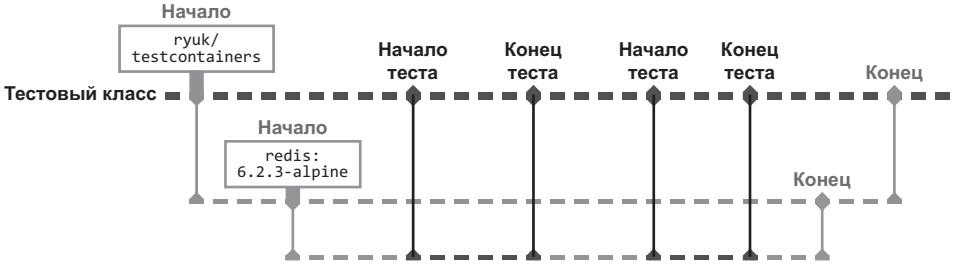


Рис. 14.1. Выполнение Testcontainers

После того как Redis запущен в контейнере, можно переходить к самим тестам. Мы тестируем взаимодействие между Redis и службой получения цены (а не то, как фактически реализована эта служба), поэтому, чтобы упростить тестирование, можно повторно использовать предыдущую реализацию `StubPrice`, которая всегда возвращает значение 10:

```
@Test
    public void cached() {
        var jedis = getJedisConnection();
        jedis.set("price", "20");

        CachedPrice price =
            new CachedPrice(new StubPrice(), jedis);
        BigDecimal result = price.getInitialPrice();
```

Задает цену, которая отличается от цены в Redis

Передает StubPrice как результат поиска цены, поэтому возвращается 10, а не 20

https://t.me/javalib

```
    assertEquals(new BigDecimal("20"), result);
}

@Test
public void noCache() {
    var jedis = getJedisConnection();
    jedis.del("price");

    CachedPrice price = new CachedPrice(new StubPrice(), jedis);
    BigDecimal result = price.getInitialPrice();
    assertEquals(new BigDecimal("10"), result);
}

private Jedis getJedisConnection() {
    HostAndPort hostAndPort = new HostAndPort(
                                  redis.getHost(),
                                  redis.getFirstMappedPort());
    return new Jedis(hostAndPort);
}
```

Проверяет, что было получено кэшированное значение

Удаляет ранее кэшированное значение в Redis вызовом del

Вспомогательный метод для настройки экземпляра Jedis

Обратите внимание, как метод `getJedisConnection` использует конфигурацию из Testcontainers, чтобы подключиться к Redis. Хотя `redis.getHost()` может вернуть стандартное значение (например, `localhost`), это не гарантируется в каждой среде. Лучше запросить такие значения у Testcontainers и защититься от их непредвиденных изменений в будущем.

Хотя автоматизированное развертывание контейнеров достаточно удобно, стоит понимать, как управлять развертыванием напрямую. Это особенно актуально, если инициализация контейнера требует некоторого времени, как мы увидим в дальнейших примерах с реляционными базами данных и схемами.

Аннотация `@Container` понимает, когда она применяется к статическому полю, а когда — к полю экземпляра (рис. 14.2). В первом случае контейнер будет инициализироваться один раз за все время выполнения тестового класса. Если же поле остается на уровне экземпляра, то каждый отдельный тест будет запускать и останавливать контейнер.

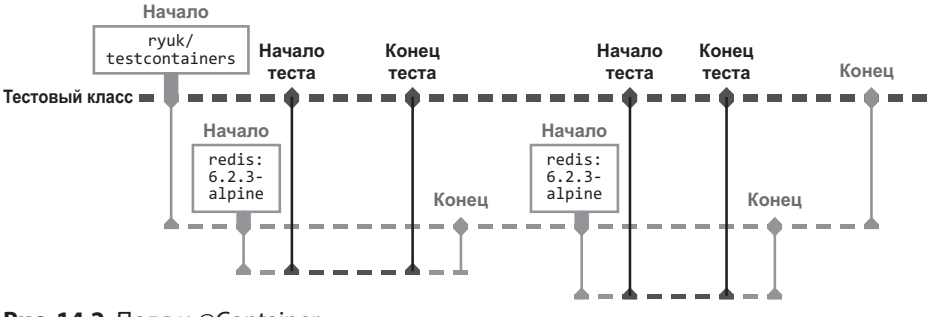


Рис. 14.2. Поля и @Container

Отсюда следует еще один возможный способ управлять сроком жизни контейнера. Если мы хотим, чтобы контейнер запускался всего один раз для всего набора

https://t.me/javalib

тестов, нужно отказаться от аннотации `@Container` и использовать API, который напрямую предоставляется самим объектом `GenericContainer`:

```
private static final DockerImageName imageName =
        DockerImageName.parse("redis:6.2.3-alpine");

public static GenericContainer redis = new GenericContainer(imageName)
        .withExposedPorts(6379);

    @BeforeAll
    public void setUp() {
        redis.start();
    }
```

start можно безопасно вызвать несколько раз для одного и того же экземпляра: контейнер будет запущен только один раз для каждого объекта

Вам не обязательно предоставлять `tearDown`, чтобы явно остановить контейнер: библиотека Testcontainers сделает это автоматически за вас.

Хотя в предыдущем примере для каждого теста вызывается `start`, объект `redis` можно переместить в место, где его смогут безопасно использовать несколько классов тестов.

14.1.3. Получение логов контейнеров

Если вы запустите эти тесты в командной строке или в IDE, то по умолчанию не увидите никакого вывода от контейнеров. В нашем простом примере с Redis это не создает проблем, но при более сложной конфигурации или в целях отладки вам, возможно, захочется иметь больше информации об этих контейнерах. Чтобы помочь в этом, Testcontainers позволяет обращаться к потокам `STDOUT` и `STDERR` запускаемых контейнеров.

Эта поддержка основана на интерфейсе `Consumer<>` из JDK, и с библиотекой поставляются несколько реализаций. Вы можете подключиться к стандартным провайдерам протоколирования или, как мы скоро увидим, получить доступ к низкоуровневым журнальным данным.

Вам может показаться неудобным, что журнальные данные (логи) контейнеров направляются в основной вывод, но вас также вряд ли обрадует перспектива принимать какие-то специальные меры каждый раз, когда эти данные вам понадобятся. Одно из возможных решений — добавить код, который всегда будет сохранять их в специальном месте (например, в файле в выводе сборки):

Снова используем аннотацию @Container для инициализации, потому что так проще

```
Container
public static GenericContainer redis =
    new GenericContainer(imageName)
        .withExposedPorts(6379);
```

Экземпляр потребителя будет собирать логи в ходе выполнения теста

```
public static ToStringConsumer consumer =
                                new ToStringConsumer();
@BeforeAll
public static void setUp() {
```

https://t.me/javalib

```
    redis.followOutput(consumer,
                       OutputType.STDOUT,
                       OutputType.STDERR);
}

@AfterAll
public static void tearDown() throws IOException {
    Path log = Path.of("./build/tc.log");
    byte[] bytes = consumer.toUtf8String().getBytes();
    Files.write(log, bytes,
                StandardOpenOption.CREATE);
}
```

Присоединяет потребитель к нашему контейнеру, запрашивая STDOUT и STDERR

Записывает данные в удобное место

Использует java.nio.Files, чтобы простым способом записать содержимое файла

14.1.4. Пример с Postgres

Пример с Redis получился довольно простым, потому что в нем нет зависимостей, сохранение данных имеет вре́менную природу, а контейнер запускается быстро. Но как насчет камня преткновения в традиционном интеграционном тестировании — реляционной базы данных? Часто данные, которые размещены в реляционном хранилище, оказываются наиболее критичными для основной функциональности приложения, но его тестирование обычно чревато устареванием данных, неудобным макетированием и ложными положительными результатами.

Testcontainers поддерживает много разных хранилищ данных. Они упакованы в отдельные модули, которые необходимо подключать к программе. В следующем примере мы будем использовать модуль Postgres, но на сайте Testcontainers (https://www.testcontainers.org/modules/databases/) можно найти длинный перечень других вариантов.

Чтобы подключиться к новой базе данных в Maven, мы добавим модуль Postgres как тестовую зависимость, а также основной драйвер Postgres:

```
<dependency>
  <groupId>org.postgresql</groupId>
  <artifactId>postgresql</artifactId>
  <version>42.2.1</version>
</dependency>
<dependency>
  <groupId>org.testcontainers</groupId>
  <artifactId>postgresql</artifactId>
  <version>1.15.3</version>
  <scope>test</scope>
</dependency>
```

То же в Gradle:

```
implementation("org.postgresql:postgresql:42.2.1")
testImplementation("org.testcontainers:postgresql:1.15.3")
```

Важно, чтобы эта версия соответствовала версии базовой библиотеки `org.testcontainers:testcontainers`, которую вы используете.

https://t.me/javalib

Доступ к контейнеру Postgres инкапсулируется в классе `PostgreSQLContainer`, вспомогательные методы которого позволяют настраивать такую информацию, как имя базы данных и реквизиты пользователя:

```
public static DockerImageName imageName =
            DockerImageName.parse("postgres:9.6.12"));

    @Container
    public static PostgreSQLContainer postgres =
        new PostgreSQLContainer<>(imageName)
            .withDatabaseName("theater_db")
            .withUsername("theater")
            .withPassword("password");
```

Здесь действуют все те же факторы управления жизненным циклом, с небольшим дополнительным затруднением: чтобы реляционную базу данных можно было использовать, к ней необходимо применить схему. Миграцию многих популярных баз данных можно осуществить на программном уровне, но мы будем напрямую использовать JDBC, чтобы показать, что никакого волшебства здесь нет.

Прежде всего нам надо подключиться к экземпляру контейнера. С помощью классов JDBC мы настраиваем его параметрами из нашего объекта `postgres` библиотеки Testcontainers:

```
private static Connection getConnection() throws SQLException {
    String url = String.format(
            "jdbc:postgresql://%s:%s/%s",
            postgres.getHost(),
            postgres.getFirstMappedPort(),
            postgres.getDatabaseName());

    return DriverManager.getConnection(url,
                                       postgres.getUsername(),
                                       postgres.getPassword());
}
```

ПРИМЕЧАНИЕ Testcontainers также позволяет модифицировать строки подключения и может автоматически запускать контейнеры для баз данных. Это удобно, но не так наглядно для демонстрационных целей. Впрочем, эта возможность особенно полезна, если Testcontainers интегрируется в существующие наборы тестов.

Наладив подключение, перед запуском тестов следует убедиться, что у базы есть схема. В области видимости одного тестового класса эта задача решается с помощью `@BeforeAll`:

```
@BeforeAll
public static void setup() throws SQLException, IOException {
    var path = Path.of("src/main/resources/init.sql");
    var sql = Files.readString(path);
    try (Connection conn = getConnection()) {
        conn.createStatement().execute(sql);
    }
}
```

В нашем примере определения схемы находятся в файле SQL

Применяет схему SQL

https://t.me/javalib

При наличии схемы можно применить тесты к полноценной, хотя и пустой базе данных Postgres:

```
@Test
public void emptyDatabase() throws SQLException {
    try (Connection conn = getConnection()) {
        Statement st = conn.createStatement();
        ResultSet result = st.executeQuery("SELECT * FROM prices");
        assertEquals(0, result.getFetchSize());
    }
}
```

Если в программе используются прочие абстракции, например объекты DAO (Data Access Object), репозитории или другие способы чтения из базы данных, все они должны нормально работать с подключением к контейнеру.

14.1.5. Пример сквозного тестирования с Selenium

Использовать внешние ресурсы в контейнерах — естественная практика для интеграционного тестирования, хотя аналогичные приемы применяются и при сквозном тестировании. Многое зависит от конкретной системы, однако при сквозных тестах часто стоит использовать браузер, чтобы убедиться, что веб-приложение работает, как ожидалось.

Традиционно управление браузером на программном уровне считалось рискованным делом. Соответствующие методы по-прежнему остаются ненадежными и медленными, но Testcontainers избавляет от хлопот с установкой и настройкой, позволяя запустить браузер в контейнере и управлять им удаленно.

Как и в примере с Postgres, нужно подключить зависимости. Для этого в Maven есть модуль поддержки Testcontainers вместе с библиотеками для того, чтобы удаленно управлять экземпляром браузера из тестов:

```
<dependency>
  <groupId>org.testcontainers</groupId>
  <artifactId>selenium</artifactId>
  <version>1.15.3</version>
  <scope>test</scope>
</dependency>
<dependency>
  <groupId>org.seleniumhq.selenium</groupId>
  <artifactId>selenium-remote-driver</artifactId>
  <version>3.141.59</version>
  <scope>test</scope>
</dependency>
<dependency>
  <groupId>org.seleniumhq.selenium</groupId>
  <artifactId>selenium-chrome-driver</artifactId>
  <version>3.141.59</version>
  <scope>test</scope>
</dependency>
```

https://t.me/javalib

Для Gradle:

```
testImplementation("org.testcontainers:selenium:1.15.3")
testImplementation(
  "org.seleniumhq.selenium:selenium-remote-driver:3.141.59")
testImplementation(
  "org.seleniumhq.selenium:selenium-chrome-driver:3.141.59")
```

Поддерживаются и другие браузеры, в пакетах с соответствующими именами

За настройку экземпляров браузеров отвечают специализированные классы. Здесь мы передаем `ChromeOptions`, чтобы указать, что запускается именно этот браузер:

```
@Container
public static BrowserWebDriverContainer<?> chrome =
    new BrowserWebDriverContainer<>()
        .withCapabilities(new ChromeOptions());
```

С этим экземпляром можно писать тесты, которые посещают веб-страницы и анализируют результаты:

```
@Test
public void checkTheSiteOut() {
    var url = "https://github.com/well-grounded-java";
    RemoteWebDriver driver = chrome.getWebDriver();
    driver.get(url);

    WebElement title =
        driver.findElementByTagName("h1");
    assertEquals("well-grounded-java", title.getText());
}
```

Переходим к проекту well-grounded-java на GitHub

После того как страница загрузится, проверяем содержимое первого элемента <h1>

Этот простой пример уже демонстрирует хрупкость, присущую сквозному тестированию. Что, если GitHub изменит дизайн и добавит на страницу еще один элемент `<h1>`? А если изменится текст заголовка? Если вы тестируете собственное приложение, проблема будет не столь серьезной, но сильное зацепление с представлением останется источником проблем.

Если при выполнении внутри контейнера что-то идет не так, как вы ожидали, бывает трудно понять, почему это происходит. К счастью, есть способы получать визуальную обратную связь.

Прежде всего, можно создавать снимки экрана в определенные моменты времени:

```
@Test
public void checkTheSiteOut() {
    RemoteWebDriver driver = chrome.getWebDriver();
    driver.get("https://github.com/well-grounded-java");

    File screen = driver.getScreenshotAs(OutputType.FILE);
}
```

https://t.me/javalib

Возвращаемый файл — временный, и он будет удален в конце теста, однако после того как он создан, вы можете скопировать его туда, куда вам нужно.

Довольно часто статического изображения оказывается недостаточно, и можно запросить автоматическую видеозапись сеанса:

```
private static final File tmpDirectory = new File("build");

@Container
    public static BrowserWebDriverContainer<?> chrome =
        new BrowserWebDriverContainer<>()
            .withCapabilities(new ChromeOptions())
            .withRecordingMode(RECORD_ALL,
                               tmpDirectory,
                               VncRecordingFormat.MP4);
```

Как и в случае с контейнерными журналами, этот режим будет создавать видеозаписи в выходном каталоге сборки при каждом запуске тестов. Если вдруг начнутся неприятности, все необходимое для отладки будет у вас под рукой.

Конечно, этот раздел дает лишь поверхностное представление о возможностях Testcontainers. А теперь пришло время оставить JUnit и попробовать писать тесты в другой, более удобочитаемой форме.

14.2. СПЕЦИФИКАЦИОННОЕ ТЕСТИРОВАНИЕ В SPEK И KOTLIN

Методы, классы и аннотации в JUnit используются так, что это выглядит чрезвычайно естественно с точки зрения Java-разработчика. Но независимо от того, сознаете вы это или нет, от этого также зависит, по какому принципу мы формулируем и группируем наши тесты. И хотя это не обязательно, часто у нас получается решение, в котором один тестовый класс соответствует классу в эксплуатационной версии, а для каждого метода реализации предусмотрены целые нагромождения тестовых методов.

Альтернативный подход — написание *спецификаций*. Эта практика происходит из таких фреймворков, как RSpec и Cucumber, и призвана не отражать структуру кода, а описывать, как работает система на высоком уровне. Такой подход больше соответствует тому, как живые люди формулируют требования.

Тестирование такого рода доступно в Kotlin через фреймворк Spek (https://www.spekframework.org/). Как вы увидите, многие встроенные средства Kotlin позволяют отразить в спецификации совершенно иную структуру и дух тестов.

Spek устанавливается в соответствии с типичным процессом для зависимостей. Этот фреймворк полностью сосредоточивается на структуре спецификаций и задействует возможности экосистемы для такой функциональности, как тестовые утверждения и запуск тестов. В качестве примера мы выберем для демонстрации

https://t.me/javalib

утверждения и исполнитель тестов из JUnit 5, но вам не обязательно ограничиваться ими, если вы предпочитаете другие библиотеки.

В Maven достаточно сообщить о файлах спецификаций плагину `maven-surefire-plugin` из раздела 11.2.6, для чего следует включить `Spek` в шаблон имен файлов, как показано далее. Также понадобится поддержка Kotlin, которая рассматривалась в разделе 11.2.5 (здесь не повторяется для краткости):

```
<build>
  <plugins>
    <plugin>
      <artifactId>maven-surefire-plugin</artifactId>
      <version>2.22.2</version>
      <configuration>
        <includes>
          <include>**/*Spek*.*</include>
        </includes>
      </configuration>
    </plugin>
  </plugins>
</build>

<dependencies>
  <dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter-api</artifactId>
    <version>5.7.1</version>
    <scope>test</scope>
  </dependency>
  <dependency>
    <groupId>org.spekframework.spek2</groupId>
    <artifactId>spek-dsl-jvm</artifactId>
    <version>2.0.15</version>
    <scope>test</scope>
  </dependency>
  <dependency>
    <groupId>org.spekframework.spek2</groupId>
    <artifactId>spek-runner-junit5</artifactId>
    <version>2.0.15</version>
    <scope>test</scope>
  </dependency>
</dependencies>
```

Используются нестандартные имена файлов, поэтому необходимо указать Maven, что выполнять

Использует API тестовых утверждений JUnit

Использует интеграцию Spek с исполнителями тестов JUnit

В Gradle нужно подключить стандартную задачу `test` и уведомить платформу JUnit о ядре Spek, как показано в следующем фрагменте. Возможно, вы обнаружите, что тесты командной строки увидят спецификации, даже если не указать ядро, но для других систем, например IDE, соответствующая строка может быть необходима:

```
dependencies {
  testImplementation(
      "org.junit.jupiter:junit-jupiter-api:5.7.1")
  testImplementation("org.spekframework.spek2:spek-dsl-jvm:2.0.15")
```

Использует API тестовых утверждений JUnit

https://t.me/javalib

```
  testRuntimeOnly(
      "org.spekframework.spek2:spek-runner-junit5:2.0.15")
}

tasks.named<Test>("test") {
  useJUnitPlatform() {
    includeEngines("spek2")
  }
}
```

Использует интеграцию Spek с исполнителями тестов JUnit

Ищет задачу test с типом Test, чтобы можно было обращаться к useJUnitPlatform и следующим методам

Уведомляет JUnit о ядре Spek для лучшей интеграции с IDE

Теперь можно писать первую спецификацию. Вспомним, как мы раньше тестировали класс `InMemoryCachedPrice`, и посмотрим, как Spek изменяет структуру и логику тестирования:

```
import org.spekframework.spek2.Spek
import org.junit.jupiter.api.Assertions.assertEquals
import java.math.BigDecimal

object InMemoryCachedPriceSpek : Spek({
    group("пустой кэш") {
        test("получает значение по умолчанию") {
            val stubbedPrice = StubPrice()
            val cachedPrice = InMemoryCachedPrice(stubbedPrice)

            assertEquals(BigDecimal(10), cachedPrice.initialPrice)
        }

        test("получает то же значение при повторном вызове") {
            val stubbedPrice = StubPrice()
            val cachedPrice = InMemoryCachedPrice(stubbedPrice)

            val first = cachedPrice.initialPrice
            val second = cachedPrice.initialPrice
            assertTrue(first === second)
        }
    }
})
```

=== — оператор Kotlin, проверяющий равенство ссылок; иначе говоря, эта строка проверяет, что оба вызова используют один и тот же объект, а не просто одинаковые значения

Первая спецификация описывает поведение для пустого кэша. При этом используется ряд специфических особенностей Kotlin. Прежде всего спецификация объявляется как одиночный объект (singleton), а не как класс. Это помогает прояснить вопросы, касающиеся жизненного цикла теста, которые иногда возникают в JUnit в зависимости от того, содержит ли исполнитель теста один экземпляр теста на класс или на каждый отдельный тестовый метод.

Основная спецификация объявляется в лямбда-выражении, которое передается в качестве параметра классу Spek. В этом лямбда-выражении доступны две важные функции — `group` и `test`. Для каждой из них задается полное строковое описание на обычном человеческом языке: здесь не нужен ни «верблюжий регистр», ни подчеркивания, ни другие трюки. Функция `group` группирует

https://t.me/javalib

взаимосвязанные вызовы `test`. При необходимости конструкции `group` могут быть вложенными.

Если бы суть спецификационного тестирования сводилась к такому переформатированию, оно было бы не особо интересным. Однако группировка позволяет не только назначать имена, но и объявлять *оснастки* (fixtures), которые позволяют разным тестам использовать общие настройки:

```
object InMemoryCachedPriceSpek : Spek({
    group("пустой кэш") {
        lateinit var stubbedPrice : Price
        lateinit var cachedPrice : InMemoryCachedPrice

        beforeEachTest {
            stubbedPrice = StubPrice()
            cachedPrice = InMemoryCachedPrice(stubbedPrice)
        }

        test("получает значение по умолчанию") {
            assertEquals(BigDecimal(10), cachedPrice.initialPrice)
        }

        test("получает то же значение при повторном вызове") {
            val first = cachedPrice.initialPrice
            val second = cachedPrice.initialPrice
            assertTrue(first === second)
        }
    }
})
```

В группе `"пустой кэш"` объявляются две оснастки: заглушка `stubbedPrice`, которая используется при настройке кэша, и тестируемый экземпляр `cachedPrice`. У всех вызовов `test`, входящих в группу, будут идентичные представления этих оснасток.

Оснастки рекомендуется объявлять с помощью `lateinit` и инициализировать в `beforeEachTest`. Отложенная инициализация нужна потому, что на самом деле Spek отрабатывает спецификацию за две фазы: исследование и последующее выполнение.

На фазе исследования выполняется лямбда-выражение на верхнем уровне спецификации. Лямбда-выражения `group` вычисляются по неотложной схеме, но вызовы `test` не выполняются, а только помечаются для последующего запуска. После того как все группы спецификации вычислены, выполняются лямбда-выражения `test`. Такое разделение позволяет лучше контролировать контекст каждой группы перед выполнением каждого отдельного теста:

```
object InMemoryCachedPriceSpek : Spek({
    group("пустой кэш") {
        lateinit var stubPrice : Price
        lateinit var cachedPrice : InMemoryCachedPrice
```

Запускается на фазе исследования

https://t.me/javalib

```
        beforeEachTest {
            stubPrice = StubPrice()
            cachedPrice = InMemoryCachedPrice(stubPrice)
        }

        test("получает значение по умолчанию") {
            assertEquals(BigDecimal(10),
                         cachedPrice.initialPrice)
        }

        test("получает то же значение при повторном вызове") {
            val first = cachedPrice.initialPrice
            val second = cachedPrice.initialPrice
            assertTrue(first === second)
        }
    }
})
```

Запускается на фазе выполнения

Пользоваться `lateinit` немного неудобно, поэтому Spek упаковывает эту функциональность с помощью *делегирования свойств* Kotlin. За каждой оснасткой может следовать вызов `memoized` и лямбда-выражение, которое вырабатывает значение.

ПРИМЕЧАНИЕ *Мемоизированным* (memoized) называется значение, которое вычисляется однократно и кэшируется для использования в будущем.

Не используйте этот прием для результатов действий, которые вы тестируете, — это должно происходить внутри самих лямбда-выражений `test`:

```
object InMemoryCachedPriceSpek : Spek({
    val stubbedPrice : Price by memoized { StubPrice() }

    group("пустой кэш") {
        val cachedPrice by memoized { InMemoryCachedPrice(stubbedPrice) }

        test("получает значение по умолчанию") {
            assertEquals(BigDecimal(10), cachedPrice.initialPrice)
        }
        test("получает то же значение при повторном вызове") {
            val first = cachedPrice.initialPrice
            val second = cachedPrice.initialPrice
            assertTrue(first === second)
        }
    }
})
```

Фаза исследования, которая реализуется простым выполнением кода на Kotlin, позволяет применить более простую параметризацию, чем в JUnit. Можно обойтись без дополнительных аннотаций и рефлексивного поиска и просто создать цикл, в котором повторяются вызовы `test`:

```
object InMemoryCachedPriceSpek : Spek({
    group("параметризованный пример") {
        listOf(1, 2, 3).forEach {
```

https://t.me/javalib

```
            test("тест $it") {
                assertNotEquals(it, 0)
            }
        }
    }
})
```

Подстановка it на каждой итерации цикла дает тесты «тест 1», «тест 2» и «тест 3»

Если вы уже сталкивались со спецификационным тестированием в других экосистемах (например, RSpec в Ruby или Jasmine в JavaScript), вы можете заменить методы `group` и `test` на `describe` и `it`, чтобы код был еще больше похож на обычный текст:

```
object InMemoryCachedPriceSpek : Spek({
    val stubbedPrice : Price by memoized { StubPrice() }

    describe("пустой кэш") {
        val cachedPrice by memoized { InMemoryCachedPrice(stubbedPrice) }

        it("получает значение по умолчанию") {
            assertEquals(BigDecimal(10), cachedPrice.initialPrice)
        }

        it("получает то же значение при повторном вызове") {
            val first = cachedPrice.initialPrice
            val second = cachedPrice.initialPrice
            assertEquals(true, first === second)
        }
    }
})
```

Другой распространенный формат спецификаций — синтаксис Gherkin (https://cucumber.io/docs/gherkin/reference/), который обрел популярность благодаря программе тестирования Cucumber. Этот синтаксис объявляет спецификацию в виде серии команд `given-when-then`: при таком-то *условии* (given), *когда* (when) происходит такое-то действие, *тогда* (then) мы видим такие-то последствия. Благодаря такой структуре спецификации часто читаются как текст на естественном языке, а не только как код.

Вот как приведенный выше тест можно сформулировать в стиле Gherkin: *при условии*, что кэш пуст, *когда* вычисляется цена, *тогда* запросить значение по умолчанию. В Spek синтаксис для Gherkin позволяет выразить это так:

```
object InMemoryCachedPriceSpekGherkin : Spek({
    Feature("кэширование") {
        val stubbedPrice by memoized { StubPrice() }

        lateinit var cachedPrice : Price
        lateinit var result : BigDecimal

        Scenario("пустой кэш") {
            Given("кэш пуст") {
                cachedPrice = InMemoryCachedPrice(stubbedPrice)
            }
```

https://t.me/javalib

```
            When("вычисляется цена") {
                result = cachedPrice.initialPrice
            }

            Then("запрашивается значение по умолчанию") {
                assertEquals(BigDecimal(10), result)
            }
        }
    }
})
```

Вы наверняка заметили, что при таком подходе появляется дополнительная группировка от Cucumber: спецификация делится на разделы `Feature` и `Scenario`, а уже затем применяется структура `given-when-then`.

Спецификации — это альтернативный способ организации тестового кода, который лучше передает смысл тестирования для тех, кто будет читать код позднее. Однако спецификации все равно требуют записывать все случаи вручную. Clojure обладает особыми возможностями для анализа того, как выбирать тестовые данные.

14.3. ТЕСТИРОВАНИЕ НА ОСНОВЕ СВОЙСТВ В CLOJURE

В отличие от Java и Kotlin, в стандартную библиотеку Clojure входит тестовый фреймворк `clojure.test`. И хотя мы не будем рассматривать эту библиотеку подробно, давайте познакомимся с ее основными принципами, прежде чем переходить к более экзотическим частям экосистемы тестирования Clojure.

14.3.1. clojure.test

Мы будем запускать тесты в оболочке Clojure REPL, как и в главе 10. Если вы пропустили эту главу или прошло слишком много времени, то вам стоит освежить в памяти основы Clojure, чтобы лучше понимать тесты.

Хотя библиотека `clojure.test` поставляется непосредственно с Clojure, она не упаковывается с кодом автоматически, а запрашивается директивой `require`. Если ввести в REPL следующие инструкции, то все функции `clojure.test` станут доступными с префиксом `test`, который здесь объявлен с ключевым словом `:as`:

```
user=> (require '[clojure.test :as test])
nil
user=> (test/is (= 1 1))
true
```

Кроме того, с помощью `:refer` можно выбрать конкретные функции, чтобы использовать их без префикса:

```
user=> (require '[clojure.test :refer [is]])
nil
user=> (is (= 1 1))
true
```

https://t.me/javalib

Функция `is` — основа для проверки тестовых утверждений в `clojure.test`. Вы уже видели, что если утверждение выполняется, то функция возвращает `true`. А если нет?

```
user=> (is (= 1 2))

FAIL in () (NO_SOURCE_FILE:1)
expected: (= 1 2)
    actual: (not (= 1 2))
false
```

С `is` можно использовать любой предикат. Например, так можно убедиться, что функция возбуждает ожидаемое исключение:

```
user=> (defn oops [] (throw (RuntimeException. "Oops")))
#'user/oops

user=> (is (thrown? RuntimeException (oops)))
#error {
  :cause "Oops"
  :via
  [{:type java.lang.RuntimeException
    :message "Oops"
    :at [user$oops invokeStatic "NO_SOURCE_FILE" 1]}]
    ...
```

Функция, которая всегда возбуждает исключение RuntimeException

Получаем значение #error, а не сообщение FAIL. Это означает, что тестовое утверждение выполнено

Сообщение об ошибке также содержит полную трассировку стека, которая здесь не приводится для краткости

С этими тестовыми утверждениями можно начинать составлять тесты. Для этого в первую очередь служит функция `deftest`:

```
user=> (require '[clojure.test :refer [deftest]])
nil
user=> (deftest one-is-one (is (= 1 1)))
#'user/one-is-one
```

После того как тест определен, его нужно запустить. Для этого можно воспользоваться функцией `run-tests`, которая находит все тесты, определенные в текущем пространстве имен. В REPL автоматически существует пространство имен по умолчанию `user`, и именно в него `deftest` включает тест:

```
user=> (require '[clojure.test :refer [run-tests]])
nil
user=> (run-tests)

Testing user

Ran 1 tests containing 1 assertions.
0 failures, 0 errors.
{:test 1, :pass 1, :fail 0, :error 0, :type :summary}
```

Очевидно, что писать и запускать тесты в REPL удобно на этапе обучения, но для долгосрочного использования в проекте этот вариант не подходит. В какой-то момент стоит создать исполнитель тестов, хотя, в отличие от мира Java, где

https://t.me/javalib

однозначно лидирует JUnit, в Clojure есть несколько альтернативных решений. Назовем некоторые из них:

- Leiningen (https://leiningen.org/) — популярный инструмент сборки Clojure с поддержкой тестирования подобно Maven и Gradle.
- `test-runner` от Cognitect Labs (https://github.com/cognitect-labs/test-runner) — простой исполнитель тестов, который опирается исключительно на встроенную поддержку зависимостей в Clojure.
- Kaocha (https://github.com/lambdaisland/kaocha) — полнофункциональный исполнитель тестов, ориентированный на модульную архитектуру процесса тестирования.

Однако далее в этом разделе мы продолжим работать в REPL и рассмотрим интересную возможность, которая поставляется вместе с Clojure, — спецификации данных.

14.3.2. clojure.spec

Хотя интеграция Clojure с JVM означает, что можно напрямую работать с классами и объектами, в функциональном программировании поведение не так тесно связывается с данными. Часто встречаются функции, которые работают со структурами данных, состоящими из базовых примитивов, особенно отображений, которые реализуют поведение хранилища данных, в то время как в объектно-ориентированном программировании оно ассоциируется с классами.

В результате развиваются средства, которые тестируют форму и содержимое встроенных структур данных. В стандартной библиотеке такая возможность предоставляется библиотекой `clojure.spec`. Как и в случае с `clojure.test`, ее нужно подключить инструкцией `require`:

```
user=> (require '[clojure.spec.alpha :as spec])
nil
```

> **ПРИМЕЧАНИЕ** Хотя в `clojure.spec` есть термин «спецификация», он используется совершенно не в том смысле, который знаком нам по Spek в Kotlin. `clojure.spec` специфицирует *данные*, а не *поведение*.

Получив доступ к библиотеке, можно делать утверждения о разных значениях при помощи функции `valid?`. Она принимает функцию-предикат, применяет ее к значению и возвращает логический флаг:

```
user=> (spec/valid? even? 10)
true
user=> (spec/valid? even? 13)
false
```

Функция `conform` предоставляет следующий уровень проверки, как показано в примере кода ниже. Если значение удовлетворяет предикату, функция его

https://t.me/javalib

же и возвращает, в противном случае возвращается ключевое слово `:clojure.spec.alpha/invalid`:

```
user=> (spec/conform even? 10)
10
user=> (spec/conform even? 13)
:clojure.spec.alpha/invalid
```

Несколько проверок можно объединить функцией `and`. Это можно сделать напрямую, написав собственные функции-предикаты, но если использовать версию из `clojure.spec`, как показано в следующем примере, библиотека сама поймет вашу комбинацию. Вскоре мы увидим, как извлечь из этого дополнительную информацию:

```
user=> (spec/conform (spec/and int? even?) 10)
10
user=> (spec/conform (spec/and int? even?) 13)
:clojure.spec.alpha/invalid
user=> (spec/conform (spec/and int? even?) "not int")
:clojure.spec.alpha/invalid
```

Вполне логично, что наряду с `and` существует парная функция `or`. Но если применять ее так же, как `and`, происходит нечто странное:

```
user=> (spec/conform (spec/or int? string?) 10)
Unexpected error (AssertionError) macroexpanding spec/or at (REPL:1:15).
Assert failed: spec/or expects k1 p1 k2 p2..., where ks are keywords
(c/and (even? (count key-pred-forms)) (every? keyword? keys))
```

Сообщение об ошибке указывает, что `or` ожидает, чтобы между переданными предикатами были ключевые слова. Для простой логической функции такое требование выглядит странно. Однако оно станет более понятным, если внимательнее присмотреться к результатам `conform` при следующих условиях `or`:

```
user=> (spec/conform (spec/or :a-number int? :a-string string?) "hello")
[:a-string "hello"]
user=> (spec/conform (spec/or :a-number int? :a-string string?) 10)
[:a-number 10]
user=> (spec/conform (spec/or :a-number int? :a-string string?) nil)
:clojure.spec.alpha/invalid
```

Библиотека не только сообщает, что значение соответствует спецификации, но и указывает, какая ветвь условия `or` ей удовлетворяет. Проверка не ограничивается простым результатом «да/нет»; мы также узнаем, *почему* значение прошло проверку.

Повторение спецификаций быстро становится однообразным, а в реальном приложении это превращается в очевидный признак «кода с душком». `clojure.spec` позволяет регистрировать спецификации с ключевым словом, принадлежащим к пространству имен. После этого можно вызывать `conform` с ключевым словом:

https://t.me/javalib

```
user=> (spec/def :well/even (spec/and int? even?))
:well/even
user=> (spec/conform :well/even 10)
10
user=> (spec/conform :well/even 11)
:clojure.spec.alpha/invalid
```

В Clojure REPL также есть удобная функция `doc`, которая хорошо интегрируется с нашими спецификациями. Получив зарегистрированное ключевое слово, функция выводит аккуратно отформатированную версию спецификации:

```
user=> (doc :well/even)
-------------------------
:well/even
Spec
(and int? even?)
```

И хотя `conform` сообщает о том, как произошло успешное совпадение, ключевое слово `:clojure.spec.alpha/invalid` не раскрывает подробностей о сбоях. Функция `explain` использует дополнительную информацию, которая доступна спецификациям, чтобы сообщить, почему то или иное значение не прошло:

```
user=> (spec/explain :well/even 10)
Success!
nil
user=> (spec/explain :well/even 11)
11 - failed: even? spec: :well/even
nil
user=> (spec/explain :well/even "")
"" - failed: int? spec: :well/even
nil
```

Определив для значений спецификации, которые можно использовать повторно, мы можем применять их в модульных тестах напрямую:

```
(deftest its-even
    (is (spec/valid? :well/even 4)))

(deftest its-not-even
    (is (not (spec/valid? :well/even 5))))
```

До этого момента наши спецификации проверяли отдельные значения. Однако при работе с отображениями встает дополнительный вопрос: соответствует ли форма предоставленных данных нашим ожиданиям? Для проверки можно воспользоваться функцией `keys`.

Представим, что часть системы продажи билетов из наших примеров написана на Clojure. Нужно убедиться в том, что у любого билета, который мы передаем, есть ключи `id` и `amount`. Также нужно учесть необязательный ключ `notes`. Спецификацию можно определить примерно так:

```
user=> (spec/def :well/ticket (spec/keys
                                :req [:ticket/id :ticket/amount]
                                :opt [:ticket/notes]))
:well/ticket
```

https://t.me/javalib

Обратите внимание, что для всех ключей указано пространство имен `:ticket`. Такая форма для ключей отображений в Clojure считается удачной, потому что она позволяет, например, отличить значение `amount` для стоимости билетов от `amount` для количества мест в театре. Но если вам непременно нужно использовать ключи без пространств имен, то такие функции, как `req`, предлагают альтернативные версии с суффиксом `-un` (например, `req-un`).

Если вызвать `conform` для отображения, функция проверит, что указанные ключи есть. Проверка также допускает, чтобы наряду с обязательными ключами фигурировали необязательные, как показано в следующем примере:

```
user=> (spec/conform :well/ticket
                     {:ticket/id 1
                      :ticket/amount 100
                      :ticket/notes "Примечание"})
#:ticket{:id 1, :amount 100, :notes "Примечание"}

user=> (spec/conform :well/ticket
                     {:ticket/id 1
                      :ticket/amount 100
                      :ticket/other-stuff true})
#:ticket{:id 1, :amount 100, :other-stuff true}

user=> (spec/conform :well/ticket {:ticket/id 1})
:clojure.spec.alpha/invalid
```

Впрочем, польза от ключей с пространствами имен особенно наглядно проявляется в том, насколько гладко они работают при проверке значений, о которой говорилось выше. Если у имени ключа есть зарегистрированная спецификация, то это значение будет проверено при вызове `conform`:

```
user=> (spec/def :ticket/amount int?)
:ticket/amount

user=> (spec/conform :well/ticket
                      {:ticket/id 1 :ticket/amount 100})
#:ticket{:id 1, :amount 100]}

user=> (spec/conform :well/ticket {:ticket/id 1 :ticket/amount "100"})
:clojure.spec.alpha/invalid
```

`clojure.spec` предоставляет богатый набор средств для проверки данных. Тем не менее особое внимание Clojure к взаимодействию с данными этим не ограничивается.

14.3.3. test.check

При написании тестов много времени уходит на то, чтобы выбрать данные, с которыми код будет тестироваться. Конструируем ли мы типичные объекты или подбираем краевые значения — мы тратим значительные усилия на поиск того, что же следует тестировать.

Тестирование на основе свойств разворачивает эти принципы в противоположную сторону. Вместо того чтобы конструировать примеры для выполнения, мы

https://t.me/javalib

определяем *свойства*, которые должны быть истинными для наших функций, а затем подаем случайные данные, чтобы убедиться, что эти свойства действительно истинны.

> **ПРИМЕЧАНИЕ** Недавний ажиотаж вокруг тестирования на основе свойств в значительной мере объясняется популярностью библиотеки QuickCheck для Haskell (https://hackage.haskell.org/package/QuickCheck). Аналогичные средства существуют и в других языках, например Hypothesis (https://hypothesis.readthedocs.io/en/latest/) в Python. В Clojure эта функциональность предоставляется библиотекой `test.check`.

Эта парадигма существенно отличается от традиционного модульного тестирования, которое знакомо многим разработчикам. При тестировании, которым мы занимались до сих пор, результаты должны быть полностью детерминированными. Любая неопределенность при запуске тестов означает, что тест плохо написан и его нужно исправить.

Почему же тестирование на основе свойств использует другой подход: оно не только допускает случайные данные, но и полагается на них? Во-первых, хотя входные данные случайны, сбой не является признаком некачественного теста — он лишь указывает на то, что ошибочным было наше понимание системы в том виде, как мы выразили его свойствами, которые определили. По сути, тесты на основе свойств помогают найти краевые случаи, которые вы могли упустить при ручном выборе данных.

Впрочем, это не означает, что от традиционных модульных тестов нужно полностью отказаться; разумнее дополнить их тестами на основе свойств, особенно в областях, где входным данным свойственна большая изменчивость, которая может вас подвести.

В отличие от `clojure.test` и `clojure.spec`, `test.check` — это отдельный пакет, не входящий в стандартную библиотеку Clojure. Чтобы использовать его в REPL, нужно предоставить Clojure эту зависимость. Для этого проще всего поместить файл с именем deps.edn в тот же каталог, из которого запускается `clj`. Этот файл заставит Clojure загрузить библиотеку из репозитория Maven:

```
{
  :deps { org.clojure/test.check {:mvn/version "1.1.0"}}
}
```

После создания файла deps.edn нужно перезапустить REPL для `clj`. При первом запуске REPL появятся сообщения о том, что загружаются необходимые файлы JAR.

Тестирование на основе свойств состоит из двух больших задач: как определять свойства, которые проверяются для вашего кода, и как генерировать случайные данные для проверки. Начнем с того, как настраивать генераторы данных; это может подсказать нам, какие свойства следует проверять.

Основные средства `test.check` для создания случайных данных находятся в пакете `generators`. Подключим весь пакет и назначим ему синоним `gen`, чтобы меньше печатать:

https://t.me/javalib

```
user=> (require '[clojure.test.check.generators :as gen])
nil
```

Точками входа для генерирования случайных данных служат две главные функции: `generate` и `sample`. Функция `generate` принимает одно значение, а `sample` — набор значений. Каждой из этих функций необходим генератор; в частности, можно использовать много встроенных генераторов, которые входят в пакет. Например, подбрасывание монеты можно моделировать генератором случайных логических значений:

```
user=> (gen/generate gen/boolean)
false

user=> (gen/sample gen/boolean)
(true false true false false false true true false false)

user=> (gen/sample gen/boolean 5)
(true true true true true)
```

Базовые генераторы, которые предоставляет `test.check`, покрывают большу́ю часть потребностей по работе с примитивными типами в Clojure. Ниже представлена небольшая подборка примеров. За дополнительной информацией и необязательными параметрами, которые принимают некоторые из этих генераторов, обращайтесь по адресу http://mng.bz/6XoD:

```
user=> (gen/sample gen/nat)
(0 1 0 2 3 5 5 7 4 5)
```
← Небольшие натуральные (целые неотрицательные) числа

```
user=> (gen/sample gen/small-integer)
(0 -1 1 1 2 4 0 5 -7 -8)
```
← Небольшие целые числа, включая отрицательные

```
user=> (gen/sample gen/large-integer)
(-1 0 -1 -3 3 -1 -8 9 26 -249)
```
← Большие целые числа, включая отрицательные

```
user=> (gen/sample (gen/choose 10 20))
(11 20 17 16 11 16 14 19 14 13)
```
← Выбор из диапазона целых чисел

```
user=> (gen/sample gen/any)
(#{} (true) (-3.0) () (Xs/B 553N -4460N) {} #{-3 W_/R? :? \} () #{} [])
```
← Любое значение Clojure

```
user=> (gen/sample gen/string)
("" "" "" "ØI_" "" "rý" "?HODÄ" "?fÿí'ß" "ü<Ò29eXÔ" "?ÅÆk0®<")
```
← Любая действительная строка Clojure

```
user=> (gen/sample gen/string-alphanumeric)
("" "" "3" "G" "pB9" "e2" "oRt98" "l8" "T61T75k4" "b8505NXt")
```
← Любая строка из алфавитно-цифровых символов

```
user=> (gen/sample (gen/elements [:a :b :c]))
(:b :c :b :a :c :b :a :c :a :b)
```
← Выбор из списка элементов

```
user=> (gen/sample (gen/list gen/nat))
(() (1) (1) (0 2 1) (0 3) (3 3) (1) (1 6 5 1 2 4 4) (4 7 3 4 7 0) (3 2))
```
← Создает список на основе предоставленного генератора

Эти генераторы могут пригодиться для так называемого *нечеткого тестирования* (fuzzing). Эта практика часто применяется в области безопасности и состоит

https://t.me/javalib

в том, чтобы передавать разнообразные (и прежде всего недопустимые!) данные и понять, где это может нарушить работу системы. Примеры, которые мы придумываем для тестирования, часто получаются недостаточно изощренными, особенно если мы имитируем входные данные из внешнего мира. Генераторы позволяют легко подкрепить тестирование данными, которые вы не смогли бы выдумать из головы.

Представьте, что приложение для продажи билетов позволяет вводить произвольный текст для заметок, но при этом извлекает из него ключевые слова. Если приложение работает в интернете, эта функция не должна возбуждать непредвиденные исключения. Ее нечеткое тестирование можно провести так:

```
user=> (defn validate-input [s]
; представьте здесь реализацию, которая не должна возбуждать исключения
)
#'user/validate-input

user=> (deftest never-throws
         (doall (map (gen/sample gen/string)
                     validate-input)))

user=> (run-tests)

Testing user

Ran 1 tests containing 0 assertions.
0 failures, 0 errors.
{:test 1, :pass 0, :fail 0, :error 0, :type :summary}
```

doall гарантирует, что в режиме отложенных вычислений Clojure не будет игнорировать map, возвращаемое значение которой нигде не используется

Нечеткое тестирование может стать полезным первым шагом, но очевидно, что в тестировании на основе свойств можно задействовать и более интересные свойства функций, чем свойство «не ломаться внезапно».

Вернемся к системе продажи билетов. Теперь заказчики хотят реализовать новую функцию, которая позволит продавать билеты в формате аукциона. У фирмы, специализирующейся в области машинного обучения, был приобретен сложный алгоритм, чтобы максимизировать количество людей, которые купят билеты по цене из заданного набора цен спроса. Алгоритм гарантирует, что он не будет предлагать цену вне диапазона заявок.

Мы еще не получили код, но нужно подготовиться к проверке того, выполнили ли разработчики эту гарантию. Для начала мы реализуем заглушку, которая случайным образом выбирает одну цену из заданного списка цен спроса:

```
user=> (defn bid-price [prices] (rand-nth prices))
#'user/bid-price
user=> user=> (bid-price [1 2 3])
1
user=> (bid-price [1 2 3])
3
```

https://t.me/javalib

Давайте посмотрим, как с помощью `test.check` можно определить свойства функции ставок. Кроме генераторов, которые мы подключили ранее, нужно подключить функции из `clojure.test.check` и `clojure.test.check.properties`:

```
user=>(require '[clojure.test.check :as tc])
nil
user=>(require '[clojure.test.check.properties :as prop])
nil
```

Первое свойство, которое нужно проверить (и самое важное для владельцев театра!), — что функция никогда не будет возвращать цену ниже всех ставок покупателей:

```
user=>(def bigger-than-minimum
  (prop/for-all [prices (gen/not-empty (gen/list gen/nat))]
    (<= (apply min prices) (bid-price prices))))
#'user/bigger-than-minimum
```

В этом маленьком фрагменте происходит столько всего, что его лучше рассматривать по частям. Сначала `def bigger-than-minimum` задает имя свойства, по которому к нему можно будет обращаться в будущем. Важно помнить, что здесь мы только определяем свойство, а не проверяем его.

В следующей строке объявляется `prop/for-all`; так указывается, какое свойство надо проверять. Далее следует список, который определяет, как будут генерироваться данные и с чем должны связываться эти значения. `prices` поочередно получает каждое сгенерированное значение от инструкции генератора `(gen/not-empty (gen/list gen/nat))` . В данном случае запрашивается непустой список натуральных (целых неотрицательных) чисел.

Наконец, последняя строка выражает фактическую логику свойства. `(<= (apply min prices) (bid-price prices))` находит минимальное значение в сгенерированном списке, вызывает функцию ставки для того же списка и убеждается, что ставка не меньше минимальной цены.

Теперь с помощью `test.check` можно протестировать набор сгенерированных значений по этому свойству. Функция `quick-check` принимает количество итераций для проверки и проверяемое свойство:

```
user=> (tc/quick-check 100 bigger-than-minimum)
{:result true, :pass? true, :num-tests 100,
 :time-elapsed-ms 13, :seed 1631172881794}
```

Свойство прошло проверку! Второе условие — что мы не предлагаем цену больше, чем любая ставка, — реализуется простым расширением уже написанного кода:

```
user=>(def smaller-than-maximum
  (prop/for-all [prices (gen/not-empty (gen/list gen/nat))]
    (>= (apply max prices) (bid-price prices))))
#'user/smaller-than-maximum

user=>(tc/quick-check 100 smaller-than-maximum)
```

https://t.me/javalib

```
{:result true, :pass? true, :num-tests 100,
 :time-elapsed-ms 13, :seed 1631173295156}
```

Хорошо, что свойства проходят проверку, но давайте посмотрим, что получится, если их нарушить. Для этого проще всего добавить небольшое увеличение в функцию ставки и заново проверить свойство:

```
user=>(defn bid-price [prices] (+ (rand-nth prices) 2))
#'user/bid-price

user=>(tc/quick-check 100 smaller-than-maximum)
{:shrunk {:total-nodes-visited 3, :depth 1, :pass? false, :result false,
:result-data nil, :time-shrinking-ms 1, :smallest [(0)]},
:failed-after-ms 5, :num-tests 1, :seed 1631173486892, :fail [(2)] }
```

Теперь результат выглядит иначе! Как и следовало ожидать, проверка не прошла, причем у нас есть вся нужная информация о сбое. В частности, ключ `:smallest [(0)]` соответствует точному значению, на котором в ходе выполнения произошел сбой. Конструкция `:seed` уже встречалась в предшествующих результатах. Если вы захотите снова проверить свойство с теми же самыми сгенерированными значениями, передайте это значение `:seed` при вызове:

```
user=>(tc/quick-check 100 smaller-than-maximum
        :seed 1631173486892)
{:shrunk {:total-nodes-visited 3, :depth 1, :pass? false, :result false,
:result-data nil, :time-shrinking-ms 1, :smallest [(0)]},
:failed-after-ms 5, :num-tests 1, :seed 1631173486892, :fail [(2)] }
```

Если повторно передать то же самое начальное значение, произойдет тот же сбой

В ответе особенно интересен ключ `:shrunk`. Когда `test.check` обнаруживает сбой, функция не просто останавливается и выдает отчет. Она проходит через процесс *сжатия* — создания меньших перестановок из сбойных сгенерированных данных, — чтобы найти минимальный случай. Эта возможность исключительно полезна, особенно с более сложными случайными данными. Если знать наименьшее, самое простое входное значение, которое приводит к сбою, это сильно упростит отладку.

`test.check` интегрируется с базовой библиотекой `clojure.test`. Функция `defspec` позволяет одновременно определить тест (как в `deftest`) и свойство:

```
user=> (require '[clojure.test.check.clojure-test :refer [defspec]])
nil

user=> (defspec smaller-than-maximum
  (prop/for-all [prices (gen/not-empty (gen/list gen/nat))]
    (>= (apply max prices) (bid-price prices))))
#'user/smaller-than-maximum

user=> (run-tests)
Testing user
{:result true, :num-tests 100, :seed 1631516389835,
 :time-elapsed-ms 36, :test-var "smaller-than-maximum"}
```

https://t.me/javalib

```
Ran 1 tests containing 1 assertions.
0 failures, 0 errors.
{:test 1, :pass 1, :fail 0, :error 0, :type :summary}
```

Самым сложным в тестировании на основе свойств часто оказывается не написать код, а определить сами свойства. Хотя в нашем примере с билетами и во многих элементарных алгоритмах (например, сортировке) свойства выводятся очевидно, в сценариях из реального мира часто все не так просто.

Вот несколько идей относительно того, где искать свойства в ваших системах:

- *Проверка данных и границы:* если у функции есть условие, которое вы бы стали проверять во время выполнения (например, ограничения для значения, длина списка или содержимое строки), — перед вами готовый кандидат для определения свойства.
- *Циклические преобразования данных:* во многих системах данные преобразуются из одних форматов в другие. Допустим, вы получаете данные в одном формате в веб-запросе, и их необходимо преобразовать в другой формат, чтобы сохранить в базе данных. В таких случаях можно определять свойства, которые показывают, что значение успешно прошло нужную серию преобразований и вернулось в исходную форму без потерь.
- *Оракулы:* иногда приходится заменять реализацию существующей функциональности — чтобы повысить производительность, улучшить удобочитаемость или еще с какой-либо целью. Если у вас есть альтернативное решение, которое считается «правильным», из него можно извлечь много свойств для сравнения, хотя бы только на время разработки замены.

14.3.4. clojure.spec и test.check

`test.check` предоставляет обширный набор генераторов для примитивов в Clojure, но в программах почти всегда приходится работать с более сложными структурами. Писать специализированные генераторы для таких структур может быть утомительно и сложно.

К счастью, `clojure.spec` помогает заполнить этот пробел. Она позволяет в общем виде описывать структуры данных более высокого уровня и умеет автоматически преобразовывать их в генераторы, совместимые с `test.check`, которые было бы слишком хлопотно определять вручную.

Составим определения структуры данных для продажи билетов — как требования к отображению, так и ограничения на значения:

```
user=> (spec/def :well/ticket (spec/keys
                                :req [:ticket/id :ticket/amount]
                                :opt [:ticket/примечание]))
:well/ticket

user=> (spec/def :ticket/amount int?)
:ticket/amount
```

https://t.me/javalib

```
user=> (spec/def :ticket/id int?)
:ticket/id

user=> (spec/def :ticket/notes string?)
:ticket/notes
```

Функция `gen` из `clojure.spec.alpha` преобразует спецификацию в генератор. Затем этот генератор можно передать тем же функциям `test.check`, которые использовались ранее, чтобы создать случайные данные:

```
user=> (gen/generate (spec/gen :well/ticket))
#:ticket{:notes "fZBvSkOAWERawpNz", :id -3, :amount 233194633}
```

Этот случайно сгенерированный билет уже проявляет краевые случаи, которые мы могли не учесть в спецификации: действительно ли нужны отрицательные идентификаторы? Может, стоит ограничить допустимую стоимость билета? Похоже, над спецификацией и тестированием еще нужно потрудиться!

ИТОГИ

- В тестировании нет единственно правильного подхода. У каждого метода есть свои сильные и слабые стороны. Код тестов — отличное место, где можно смешивать и сочетать разные библиотеки и языки, чтобы наилучшим образом раскрыть их потенциал.
- Другие языки (например, Kotlin и Clojure) позволяют применять такие приемы тестирования, которые труднее реализовать на Java.
- Интеграционное тестирование — взаимодействие с хранилищами данных и другими службами — может страдать от ненадежности и ошибок. Testcontainers обеспечивает плавную интеграцию с такими внешними зависимостями; при этом используется технология контейнеров, описанная в главе 12.
- От того, как мы описываем спецификации, может зависеть, как мы представляем себе свои системы. Фреймворк Spek для Kotlin, а также другие фреймворки для спецификационного тестирования — альтернатива тестам в стиле JUnit, которые ориентируются на код. В этой главе мы увидели, как спецификации упрощают обмен информацией во время тестирования.
- Наконец, мы рассмотрели тестирование на основе свойств в Clojure — подход, который полностью отличается от классической схемы «написать пример и проверить результат». Возможности этого стиля тестирования — от генерирования случайных данных и определения глобальных свойств системы до сжатия сбоев к минимальному возможному вводу — открывают новые горизонты для повышения качества в наших программных продуктах.

https://t.me/javalib

Часть V

Передовые рубежи Java

В этой части книги сводятся воедино многие понятия и приемы, которые встречались в других частях.

Опираясь на материал ознакомительного курса по Clojure из главы 10, мы углубимся в функциональное программирование за пределами базовых схем «отображение — фильтр — свертка». Мы подробно разберемся, почему архитектура и история Java создают определенные преграды для функционального стиля. Затем мы увидим, как расширенные средства функциональных языков в Kotlin и Clojure позволяют сделать код проще и эффективнее.

В развитие темы конкурентного выполнения из части II также будут представлены другие возможности создания безопасных высокопроизводительных приложений. Мы рассмотрим недавно появившуюся в Java функциональность `Fork/Join`, корутины Kotlin и агенты Clojure, в результате чего в вашем арсенале появится еще больше инструментов, чтобы управлять многоядерным, многопоточным миром современных вычислений.

Далее речь пойдет о внутреннем устройстве JVM. Если вас когда-нибудь интересовало, почему рефлексия работает медленно или как динамические языки могут уживаться с JVM, глава 17 поможет во многом разобраться.

В завершение вы ознакомитесь со сводкой основных используемых проектов OpenJDK: вы узнаете, каковы цели проектов и чего стоит ожидать от них в следующих версиях.

https://t.me/javalib

15 Продвинутое функциональное программирование

В ЭТОЙ ГЛАВЕ

- ✓ Понятия функционального программирования
- ✓ Ограничения функционального программирования на Java
- ✓ Продвинутое функциональное программирование на Kotlin
- ✓ Продвинутое функциональное программирование на Clojure

В этой книге уже шла речь о концепциях функционального программирования, однако в этой главе мы сведем их воедино и расширим свой инструментарий. Хотя о *функциональном программировании* много говорят в отрасли, это понятие все же остается довольно расплывчатым. Единственное, на чем все сходятся, — в языках функционального программирования (ФП) код представляется как элемент данных первого класса, то есть блок отложенных вычислений допустимо представить как значение, которое можно присвоить переменной.

Конечно, такое определение получается до нелепости широким: по существу, под него подходят все массовые языки за последние 30 лет (за единичными исключениями). Таким образом, когда разные группы программистов обсуждают функциональное программирование, они имеют в виду разные вещи. У каждой группы есть свое негласное понимание того, какие *еще* свойства других языков можно отнести к функциональному программированию.

Другими словами, как и в случае с объектно-ориентированным программированием, не существует общепринятого определения того, что же считать «языком

https://t.me/javalib

функционального программирования». А если считать такими все языки подряд, то понятие теряет смысл.

Опытные разработчики привыкли скорее представлять языки программирования как точки на оси (а еще лучше — в многомерном пространстве возможных характеристик языка). Одни языки просто являются более или менее функциональными, чем другие, и между одними и другими нет четкой границы. Познакомимся с некоторыми понятиями из типичного инструментария языков функционального программирования на более глубоком уровне, чем «код — это данные».

15.1. ВВЕДЕНИЕ В ПОНЯТИЯ ФУНКЦИОНАЛЬНОГО ПРОГРАММИРОВАНИЯ

В последующем материале часто упоминаются *функции*, но ни в языке Java, ни в JVM такого понятия не существует: весь исполняемый код должен выражаться в виде *методов*, которые определяются, компонуются и загружаются в составе *класса*. В других языках, не привязанных к JVM, исполняемый код может пониматься по-другому, так что давайте условимся, что в этой главе под функцией будет подразумеваться участок исполняемого кода, который приблизительно соответствует методу Java.

15.1.1. Чистые функции

Функция называется *чистой* (pure function), если она не изменяет состояние никакой другой сущности. Иногда говорят, что такая функция *не имеет побочных эффектов*, иными словами, она ведет себя как воплощение математической функции: принимает аргументы, никак не изменяет их и возвращает результат, который зависит только от переданных значений.

С понятием чистоты связана идея *ссылочной прозрачности* (referential transparency). Пожалуй, этот термин выбран не вполне удачно: он не имеет никакого отношения к ссылкам в понимании программиста на Java. Здесь имеется в виду, что вызов функции можно заменить результатом любого предыдущего вызова той же функции с теми же аргументами.

Очевидно, что все чистые функции обладают ссылочной прозрачностью, однако могут существовать функции, которые тоже ею обладают, но не являются чистыми. Чтобы убедиться, что нечистая функция обладает ссылочной прозрачностью, потребуется формальное доказательство, основанное на анализе кода. И если понятие чистоты относится к коду, то понятие неизменяемости относится к данным, и это следующая концепция функционального программирования, которую мы рассмотрим.

15.1.2. Неизменяемость

Неизменяемость (immutability) означает, что после того, как объект создан, его состояние нельзя изменить. Объекты в Java по умолчанию являются изменяемыми.

https://t.me/javalib

Ключевое слово `final` используется в Java по-разному, но здесь для нас важно то, что оно позволяет запретить изменение полей после создания. Другие языки могут предпочитать неизменяемость и обозначать это предпочтение разными способами: например, в Rust, чтобы сделать переменную изменяемой, к ней нужно явно применить модификатор `mut`.

Неизменяемость делает код проще для понимания: объекты имеют тривиальную модель состояния просто потому, что они конструируются в единственном состоянии, в котором они когда-либо будут существовать. Среди прочего, это означает, что их можно безопасно копировать и совместно использовать — даже из разных потоков.

> **ПРИМЕЧАНИЕ** Вы можете спросить: а не бывает ли «почти неизменяемых» данных, которые сохраняют некоторые (возможно, большинство) положительных сторон неизменяемости? И действительно, примером таких данных может послужить класс Java `CompletableFuture`, который нам уже встречался. Мы подробнее рассмотрим его в следующей главе.

Одно из следствий заключается в том, что поскольку неизменяемые объекты нельзя модифицировать, изменение состояния в такой системе можно выразить только одним способом: опираясь на неизменяемое значение, сконструировать совершенно новое неизменяемое значение, которое будет почти таким же, как старое, но с другими значениями некоторых полей. (Возможно, это будет сделано с помощью методов `with*()`.)

Например, в API `java.time` чрезвычайно широко используются неизменяемые данные, а новые экземпляры можно создавать с помощью `with*()`:

```
LocalDate ld = LocalDate.of(1984, Month.APRIL, 13);
LocalDate dec = ld.withMonth(12);
System.out.println(dec);
```

У неизменяемости есть свои недостатки — прежде всего это касается подсистемы памяти, потому что когда создается новое значение, компоненты старого значения приходится копировать. Это означает, что модификация на месте часто обходится намного меньшими затратами с точки зрения производительности.

15.1.3. Функции высшего порядка

Понятие функций высшего порядка на самом деле очень простое и выражается таким принципом: если функцию можно представить как элемент данных, то с ней можно делать все то же самое, что и с любым другим значением.

Функцию высшего порядка можно определить как функцию, которая обладает хотя бы одним из следующих свойств:

- Ее можно передавать в качестве параметра.
- Она может возвращать функцию в качестве значения.

https://t.me/javalib

Например, возьмем статический метод, который принимает объект Java `String` и генерирует на его основе объект-функцию:

```
public static Function<String, String> makePrefixer(String prefix) {
    return s -> prefix +": "+ s;
}
```

Мы продемонстрировали простейший способ создавать объекты-функции. Давайте рассмотрим другой статический метод, который на этот раз получает объект-функцию на входе:

```
public static String doubleApplier(String input,
                                   Function<String, String> f) {
    return f.apply(f.apply(input));
}
```

Объединив оба метода, мы приходим к следующему простому примеру:

```
var f = makePrefixer("NaNa");
System.out.println(doubleApplier("Batman", f));
```

Создает объект-функцию

Объект-функция передается другому методу как параметр

Тем не менее это далеко не все, что Java умеет по части функционального программирования, и мы убедимся в этом в следующем разделе.

15.1.4. Рекурсия

Функция называется *рекурсивной*, если она вызывает сама себя хотя бы на некоторых ветвях выполнения внутри функции. С рекурсивными функциями связана одна из самых старых шуток в программировании: «Чтобы понять рекурсию, надо сначала понять рекурсию».

Однако если придерживаться академической строгости, шутку следовало бы записать в таком виде: чтобы понять рекурсию, необходимо сначала понять:

1) рекурсию и
2) что в физически реализуемой системе любая цепочка рекурсивных вызовов должна в какой-то момент завершиться и вернуть значение.

Второй пункт важен: чтобы функции могли вызывать другие функции, в языках программирования используется стек, который занимает место в памяти. Таким образом, слишком глубокая рекурсия потенциально может потребовать слишком много памяти и привести к аварийному завершению программы.

В компьютерной теории рекурсия интересна и важна по многим причинам. Одна из самых важных причин заключается в том, что рекурсию можно использовать как основу для изложения теории вычислений и таких идей, как *полнота по Тьюрингу* (упрощенно говоря, она заключается в том, что все нетривиальные вычислительные системы имеют одинаковые теоретические возможности для выполнения вычислений).

https://t.me/javalib

15.1.5. Замыкания

Замыкание (closure) обычно определяется как лямбда-выражение, которое «захватывает» некоторое состояние из окружающего контекста. Тем не менее, чтобы понять это определение, нужно сначала разобраться, что такое захват (capturing).

Когда мы создаем значение и присваиваем его локальной переменной (связываем его с этой переменной), переменная существует и может использоваться до какой-то будущей точки в коде. Этой точкой может быть конец функции или блока, в котором эта переменная была объявлена. Область кода, в которой переменная существует и доступна для использования, называется *областью видимости* (scope) переменной.

А если мы создаем значение-функцию, то локальные переменные, объявленные в теле функции, по-прежнему будут находиться в области видимости, когда это значение-функция будет вызываться, а это случится позже той точки, в которой оно было объявлено. Если в объявлении значения-функции упоминается переменная (или другое состояние, например поле), которая объявлена за пределами области видимости тела функции, то говорят, что значение-функция *замыкается* по состоянию, и такое значение-функция называется *замыканием*.

Когда впоследствии замыкание вызывается, в нем полностью доступны все захваченные переменные, даже если вызов происходит не в той области видимости, в которой замыкание было объявлено.

Например, в Java это может выглядеть так:

```
public static Function<Integer, Integer> closure() {
    var atomic = new AtomicInteger(0);
    return i -> atomic.addAndGet(i);
}
```

Этот статический метод является функцией высшего порядка, которая возвращает замыкание Java: она возвращает лямбда-выражение, которое ссылается на переменную `atomic`, объявленную локальной внутри метода, то есть в той же области видимости, в которой объявлено само лямбда-выражение. Замыкание, которое возвращается из `closure()`, может вызываться повторно и накапливать состояние при каждом вызове.

15.1.6. Отложенные вычисления

Отложенные вычисления кратко упоминались в главе 10. По сути, *отложенное вычисление* (или «ленивое» вычисление, lazy evaluation) позволяет отложить вычисление значения выражения до того момента, когда это значение потребуется. С другой стороны, *неотложное вычисление* (оно же строгое, или «жадное», вычисление, eager evaluation) выполняется там, где оно встречается в коде.

Идея отложенных вычислений проста: если работу можно не выполнять, то не выполняйте ее! Вроде бы простой принцип, но он заметно сказывается на том,

https://t.me/javalib

как вы пишете свои программы и насколько эффективно они работают. Ключевая часть дополнительной сложности заключается в том, что программа должна отслеживать, какая работа уже была (или не была) проделана ранее.

Отложенные вычисления доступны не во всех языках, и многие программисты до сих пор имели дело только с неотложными вычислениями, и в этом нет ничего предосудительного.

Например, Java не поддерживает отложенные вычисления на уровне языка, поэтому будет трудно привести понятный пример. Чтобы полноценно обсудить эту тему, придется подождать, когда мы перейдем к Kotlin.

Тем не менее, хотя многие программисты на Java не воспринимают отложенные вычисления как естественную концепцию, это в высшей степени полезный и мощный механизм в функциональном программировании. Собственно, в некоторых функциональных языках, например в Haskell, отложенные вычисления используются по умолчанию.

15.1.7. Каррирование и частичное применение

К сожалению, каррирование (currying) не имеет никакого отношения к карри из индийской кухни. Это прием программирования, названный по имени Хаскелла Карри (в честь которого также назван язык программирования Haskell). Чтобы объяснить суть каррирования, начнем с конкретного примера.

Представьте неотложно вычисляемую чистую функцию, которая принимает два аргумента. Если предоставить оба аргумента, вы получите значение, и вызов функции можно будет везде заменить на это значение (вследствие ссылочной прозрачности). Но что произойдет, если предоставить не оба, а только один из двух аргументов?

На интуитивном уровне это можно понимать так, будто мы создаем новую функцию, которой теперь нужен только один аргумент, чтобы вычислить результат. Эта новая функция называется *каррированной*, или частично применяемой. Java не поддерживает каррирование напрямую, поэтому конкретный пример снова откладывается на будущее.

Если заглянуть еще дальше, можно упомянуть о том, что некоторые языки программирования поддерживают концепцию функций со списками множественных аргументов (или синтаксис, который позволяет их имитировать). В таком случае каррирование можно рассматривать как преобразование функций. В математической записи это выражается так: функция с несколькими аргументами, которая вызывается в форме `f(a, b)`, преобразуется в функцию с одним аргументом, которая вызывается в форме `(g(a))(b)`, где `g(a)` — частично применяемая функция.

Как вы наверняка уже поняли к настоящему моменту, разные языки, встречавшиеся нам ранее, поддерживают функциональное программирование в разной мере. Например, в Clojure прекрасно поддерживаются многие концепции,

https://t.me/javalib

которые обсуждались в этом разделе. Но с Java дело обстоит совершенно иначе, как вскоре будет показано.

15.2. ОГРАНИЧЕНИЯ ФУНКЦИОНАЛЬНОГО ПРОГРАММИРОВАНИЯ НА JAVA

Начнем с хорошей новости: Java определенно преодолевает довольно низкую планку «представления кода как данных» благодаря типам из `java.util.function`, а также обширной интроспективной поддержке в среде выполнения (например, с помощью рефлексии и дескрипторов методов).

> **ПРИМЕЧАНИЕ** До Java 8 объекты-функции иногда имитировались с помощью внутренних классов, например в таких библиотеках, как Google Guava. Таким образом, формально Java был способен представлять код в виде данных и до Java 8.

Начиная с версии 8, язык Java уже не ограничивается минимальными средствами: появляются потоки данных, а с ними — и сильно ограниченный пласт отложенных операций. Тем не менее, несмотря на потоки данных, Java нельзя считать органически функциональной средой. Отчасти это объясняется историей платформы и проектировочными решениями, принятыми десятки лет назад.

> **ПРИМЕЧАНИЕ** Не забывайте, что Java — императивный язык 25-летней давности, который продолжает постепенно эволюционировать. Некоторые из его API «дружат» с функциональным программированием, неизменяемыми данными и т. д., а некоторые — нет. Такова реальность работы с языком, которому жизненно важно и идти в ногу со временем, и сохранять обратную совместимость.

Таким образом, Java, пожалуй, можно описать как «слегка функциональный язык программирования». Он поддерживает базовые идиомы функционального программирования и предоставляет доступ к основным паттернам (например, «фильтрация — отображение — свертка») через API потоков данных, но более сложные функциональные возможности либо поддерживаются не полностью, либо вовсе отсутствуют. Разберемся с ними поподробнее.

15.2.1. Чистые функции

Как было показано в главе 4, байт-код Java выполняет множество различных операций, включая арифметические действия, манипуляции со стеком и управление потоком выполнения, а также может передавать управление, записывать и читать данные. Для опытных разработчиков, которые уже понимают байт-код JVM, это означает, что о чистоте методов можно судить по эффекту от выполнения байт-кода, а именно: в языке JVM метод считается чистым, если он обладает следующими свойствами:

- Не изменяет объект или статическое состояние (не содержит `putfield` или `putstatic`).

https://t.me/javalib

- Не зависит от внешнего изменяемого объекта или статического состояния.
- Не вызывает никакие нечистые методы.

Этот набор условий создает довольно серьезные ограничения и дает понять, что JVM сложно использовать как основу для чисто функционального программирования.

Также остается вопрос о семантике (то есть назначении) разных интерфейсов, входящих в JDK. Например, `Callable` (из `java.util.concurrent`) и `Supplier` (из `java.util.function`) делают, по сути, одно и то же: они выполняют некоторые вычисления и возвращают значение:

```
@FunctionalInterface
public interface Callable<V> {
    V call() throws Exception;
}

@FunctionalInterface
public interface Supplier<T> {
    T get();
}
```

Оба интерфейса помечаются аннотацией `@FunctionalInterface` и обычно используются как целевой тип для лямбда-выражений. У них одинаковые сигнатуры, не считая того, что исключения обрабатываются по-разному.

Тем не менее роли этих интерфейсов различны: `Callable` подразумевает, что вызываемый код потенциально может выполнять нетривиальную работу, чтобы создать значение, которое будет возвращаться. С другой стороны, имя `Supplier` предполагает, что выполняется меньше работы, — возможно, просто возвращается кэшированное значение.

15.2.2. Изменяемость

Java — это изменяемый язык: изменяемость интегрирована в его архитектуру с самого начала. Отчасти это объясняется историческими причинами: в конце 1990-х (когда появился Java) объем памяти компьютеров был весьма ограничен (по современным стандартам). При неизменяемой модели данных значительно увеличилась бы нагрузка на подсистему управления памятью, а сборка мусора происходила бы намного чаще, от чего пострадала бы производительность.

Поэтому создатели Java сделали выбор в пользу изменения вместо создания измененных копий. Таким образом, модификацию «на месте» можно считать проектировочным решением, которое обусловлено соображениями производительности 25-летней давности.

Однако на самом деле ситуация еще хуже. Java обращается ко всем составным данным по ссылке, а ключевое слово `final` применяется к ссылке, а *не* к данным. Например, если оно применяется к полю, то этому полю можно присвоить значение только один раз.

https://t.me/javalib

А это значит, что даже если объект состоит только из конечных полей, его составное состояние все равно может оказаться изменяемым, потому что в объекте может храниться конечная ссылка на другой объект, который содержит изменяемые поля. Это приводит к проблеме поверхностной неизменяемости, о которой мы говорили в главе 5.

> **ПРИМЕЧАНИЕ** Для программистов на C++: в Java отсутствует понятие константы; слово `const` входит в категорию ключевых, но не используется.

Например, вот слегка расширенная версия неизменяемого класса `Deposit` из главы 5:

```
public final class Deposit implements Comparable<Deposit> {
    private final double amount;
    private final LocalDate date;
    private final Account payee;

    private Deposit(double amount, LocalDate date, Account payee) {
        this.amount = amount;
        this.date = date;
        this.payee = payee;
    }

    @Override
    public int compareTo(Deposit other) {
        return Comparator.nullsFirst(LocalDate::compareTo)
                         .compare(this.date, other.date);
    }

    // Другие методы
}
```

Неизменяемость этого класса основана на предположении о том, что `Account` и все его транзитивные зависимости тоже неизменяемы. А следовательно, простор для маневра ограничен: по сути, модели данных Java и JVM по своей природе плохо укладываются в концепцию неизменяемости.

В байт-коде можно увидеть, что конечные поля представлены блоком метаданных полей в следующем виде:

```
$ javap -c -p out/production/resources/ch13/Deposit.class
Compiled from "Deposit.java"
public final class ch13.Deposit
    implements java.lang.Comparable<ch13.Deposit> {

    private final double amount;

    private final java.time.LocalDate date;

    private final ch13.Account payee;
// ...
}
```

https://t.me/javalib

Пытаться применять неизменяемые решения к состоянию в Java — все равно что черпать воду из тонущей лодки. Каждую отдельную ссылку придется проверять на изменяемость, и если вы упустите хотя бы одну, весь граф объекта получится изменяемым.

Что еще хуже, рефлексия JVM и другие подсистемы тоже позволяют обойти неизменяемость:

```
var account = new Account(100);
var deposit = Deposit.of(42.0, LocalDate.now(), account);
try {
    Field f = Deposit.class.getDeclaredField("amount");
    f.setAccessible(true);
    f.setDouble(deposit, 21.0);
    System.out.println("Значение: "+ deposit.amount());
} catch (NoSuchFieldException e) {
    e.printStackTrace();
} catch (IllegalAccessException e) {
    e.printStackTrace();
}
```

Все вместе это означает, что ни Java, ни JVM на уровне среды не предоставляют сколько-нибудь значительную поддержку программирования с неизменяемыми данными. Языкам с более строгими требованиями, таким как Clojure, приходится выполнять значительный объем работы в своей собственной среде выполнения.

15.2.3. Функции высшего порядка

Понятие функций высшего порядка не должно быть чем-то неожиданным для программистов на Java. Ранее уже приводился пример статического метода `makePrefixer()`, который принимает строку с префиксом и возвращает объект-функцию. Перепишем код и заменим статическую фабрику на другой объект-функцию:

```
Function<String, Function<String, String>> prefixer =
                                           prefix -> s -> prefix +": "+ s;
```

Возможно, этот код трудно прочитать с первого раза, поэтому добавим немного дополнительных конструкций, которые на самом деле не обязательны, но помогают лучше понять, что здесь происходит:

```
Function<String, Function<String, String>> prefixer = prefix -> {
    return s -> prefix +": "+ s;
};
```

В этой расширенной версии видно, что `prefix` — аргумент функции, а возвращаемое значение — это лямбда-выражение (на самом деле — замыкание Java), которое реализует `Function<String, String>`.

Обратите внимание на тип функции `Function<String, Function<String,String>>`: у него есть два параметра-типа, которые определяют входной и выходной тип.

https://t.me/javalib

Второй (выходной) параметр-тип — это всего лишь еще один тип (в данном случае еще один тип функции). Это признак, по которым в Java можно опознать тип функции высшего порядка: `Function` (или другой функциональный тип), одним из параметров-типов которого является `Function`.

Наконец, подчеркнем, что синтаксис языка играет роль: в конце концов, объекты-функции можно создавать и как анонимные реализации:

```
public class PrefixerOld
    implements Function<String, Function<String, String>> {

    @Override
    public Function<String, String> apply(String prefix) {
        return new Function<String, String>() {
            @Override
            public String apply(String s) {
                return prefix +": "+ s;
            }
        };
    }
}
```

Такой код был бы допустимым даже в Java 5, если бы в то время существовал тип `Function` (и даже в Java 1.1, если убрать аннотации и обобщения). Однако выглядит он просто ужасно. Очень трудно разобраться в его структуре, поэтому многие программисты считают, что настоящее функциональное программирование появилось только в Java 8.

15.2.4. Рекурсия

Компилятор `javac` преобразовывает исходный код на Java непосредственно в байт-код. Как показано ниже, это относится и к рекурсивным вызовам:

```
public static long simpleFactorial(long n) {
    if (n <= 0) {
        return 1;
    } else {
        return n * simpleFactorial(n - 1);
    }
}
```

Это компилируется в такой байт-код:

```
public static long simpleFactorial(long);
    Code:
       0: lload_0
       1: lconst_0
       2: lcmp
       3: ifgt 8
       6: lconst_1
       7: lreturn
       8: lload_0
```

https://t.me/javalib

```
       9: lload_0
      10: lconst_1
      11: lsub
      12: invokestatic #37             // Method simpleFactorial:(J)J
      15: lmul
      16: lreturn
```

Конечно, у такого подхода есть серьезные ограничения. В данном случае вызов вроде `simpleFactorial(100000)` приведет к ошибке `StackOverflowError` из-за вызова `invokestatic` со смещением 12, из-за которого в стек помещается дополнительный кадр интерпретатора для каждого рекурсивного вызова.

> **ПРИМЕЧАНИЕ** *Рекурсивный* метод вызывает сам себя. Методом с *хвостовой рекурсией* (tail-recursive) называется метод, в котором вызов самого себя является последней операцией.

Попробуем разобраться, как избежать рекурсивного вызова. Одно из возможных решений — переписать код факториала в хвостовой рекурсивной форме, что на Java проще всего сделать при помощи частного вспомогательного метода:

```
public static long tailrecFactorial(long n) {
    if (n <= 0) {
        return 1;
    }
    return helpFact(n, 1);
}

private static long helpFact(long i, long j) {
    if (i == 0) {
        return j;
    }
    return helpFact(i - 1, i * j);
}
```

Входной метод `tailrecFactorial()` не выполняет никакой рекурсии, а просто настраивает хвостовой рекурсивный вызов и скрывает от пользователя подробности более сложной сигнатуры. Байт-код метода в основном тривиален, но давайте приведем его для полноты изложения:

```
public static long tailrecFactorial(long);
    Code:
       0: lload_0
       1: lconst_0
       2: lcmp
       3: ifgt  8
       6: lconst_1
       7: lreturn
       8: lload_0
       9: lconst_1
      10: invokestatic #49             // Method helpFact:(JJ)J
      13: lreturn
```

https://t.me/javalib

Как видите, здесь нет циклов, а есть только один переход `if` в байте 3. Все самое интересное (и рекурсия) происходит в `helpFact()`. Этот метод компилируется `javac` в байт-код с рекурсивным вызовом:

```
private static long helpFact(long, long);
    Code:
       0: lload_0
       1: lconst_0
       2: lcmp
       3: ifne           8
       6: lload_2
       7: lreturn
       8: lload_0
       9: lconst_1
      10: lsub
      11: lload_0
      12: lload_2
      13: lmul
      14: invokestatic #49                 // Method helpFact:(JJ)J
      17: lreturn
```

Значения long занимают по 8 байт, поэтому им требуются два слота локальных переменных

Возврат из ветви i == 0

Хвостовой рекурсивный вызов

Однако в такой форме мы видим, что в этом методе существуют два пути. Простой путь `i == 0` начинается с байт-кода 0, проходит через условие `if` со смещением 3 и возвращает `j` в байт-коде 7. В более общем случае выполнение проходит с 0 до 3, а затем с 8 до 14, что приводит к рекурсивному вызову.

Таким образом, на единственном пути, где вызывается метод, этот вызов является рекурсивным и всегда оказывается последним перед `return`, то есть вызов находится в *хвостовой позиции*. Тем не менее его *можно* откомпилировать в следующий байт-код, который избегает вызова:

```
private static long helpFact(long, long);
    Code:
       0: lload_0
       1: lconst_0
       2: lcmp
       3: ifne           8
       6: lload_2
       7: lreturn
       8: lload_0
       9: lconst_1
      10: lsub
      11: lload_0
      12: lload_2
      13: lmul
      14: lstore_2
      15: lstore_0
      16: goto           0
```

Значения long занимают по 8 байт, поэтому им требуются два слота локальных переменных

Возврат из ветви i == 0

Сбрасывает локальные переменные

Переходит в начало метода

А теперь плохая новость: `javac` не производит такой код автоматически, даже если такая возможность существует. Это еще один пример того, как компилятор старается преобразовать исходный код на Java в байт-код как можно точнее.

https://t.me/javalib

ПРИМЕЧАНИЕ В проекте Resources из сопроводительных материалов книги приведен пример того, как с помощью библиотеки ASM генерируется класс, который реализует предыдущую последовательность операций байт-кода, потому что javac сам по себе не будет этого делать.

Для полноты изложения следует указать, что на практике реализация факториальной функции, где используется `long` с рекурсивными вызовами (вместо перезаписи кадров в стеке), не создаст проблем: факториал растет так быстро, что место в переменной `long` исчерпается задолго до того, как переполнится размер стека:

```
$ java TailRecFactorial 20
2432902008176640000

$ jshell
jshell> 2432902008176640000L + 0.0
$1 ==> 2.43290200817664E18

jshell> Long.MAX_VALUE + 0.0
$2 ==> 9.223372036854776E18
```

Таким образом, `factorial(21)` уже больше максимального числа типа `long`, которое можно выразить в JVM. Тем не менее, хотя этот тривиальный пример относительно безопасен, он не отменяет того неприятного факта, что все рекурсивные алгоритмы в Java потенциально подвержены переполнению стека.

Эта конкретная проблема присуща языку Java, а не JVM. Другие языки для JVM могут решать ее иначе, например, с помощью аннотации или ключевого слова. Примеры такого рода вы увидите далее в этой главе, когда пойдет речь о рекурсии в Kotlin и Clojure.

15.2.5. Замыкания

Как вы уже знаете, замыкание — это, по сути, лямбда-выражение, которое захватывает некоторое видимое состояние из области видимости, где лямбда-выражение было объявлено, например:

```
int i = 42;
Function<String, String>f=s->s+i;
// i=37;
System.out.println(f.apply("Привет, "));
```

Если запустить этот пример, то выводится сообщение `Привет, 42`, как и ожидалось. Но если раскомментировать строку, в которой `i` присваивается значение 37, произойдет нечто иное: программа вообще перестанет компилироваться.

Чтобы понять, почему так получается, рассмотрим байт-код, в который компилируется эта программа. Как будет показано в главе 17, тела лямбда-выражений в Java преобразуются в частные статические методы. В данном случае тело лямбда-выражения компилируется так:

https://t.me/javalib

```
private static java.lang.String lambda$main$0(int, java.lang.String);
    Code:
       0: aload_1
       1: iload_0
       2: invokedynamic #32, 0  // InvokeDynamic #1:makeConcatWithConstants:
                                // (Ljava/lang/String;I)Ljava/lang/String;
       7: areturn
```

Присмотритесь к сигнатуре `lambda$main$0()`: она получает *два* параметра, а не один. Первый параметр — это переданное значение `i`, то есть 42 на момент создания замыкания, а второй — `String`, который лямбда-выражение принимает при выполнении. Замыкания Java содержат копии *значений*, то есть последовательности битов (будь то примитивы или ссылки на объекты), а не *переменные*.

ПРИМЕЧАНИЕ В языке Java используется исключительно передача по значению — в базовом языке нет механизмов передачи по ссылке или по имени.

Чтобы увидеть, к чему приведут изменения в захваченном состоянии за пределами области видимости тела замыкания (или в других областях видимости), захваченное состояние должно быть изменяемым объектом:

```
var i = new AtomicInteger(42);
Function<String, String>f=s->s+ i.get();
i.set(37);
// i=new AtomicInteger(37);
System.out.println(f.apply("Привет, "));
```

Можно повторно присвоить значение состоянию изменяемого объекта

Не будет компилироваться

На самом деле в ранних версиях Java в замыканиях могли захватываться только переменные, которые были явно помечены как `final`. Однако начиная с Java 8, это ограничение было ослаблено до переменных, которые являются *фактически конечными*, то есть используются как конечные, даже если в их объявлении нет ключевого слова `final`.

Вообще говоря, это указывает на более глубокую проблему. JVM использует общую кучу, частные по отношению к методу локальные переменные и частный по отношению к методу стек — и это все. В отличие от некоторых других языков ни в JVM, ни в языке Java нет понятия *окружения* или таблицы символических имен, как нет и возможности передать ссылку на элемент такой таблицы.

Другим языкам для JVM, в которых есть такие концепции, приходится реализовывать их в собственной среде выполнения, потому что JVM не предоставляет для них никакой встроенной поддержки. На этом основании некоторые теоретики языков программирования приходят к выводу, что механизмы Java нельзя считать истинными замыканиями, потому что им требуется дополнительный уровень косвенности. Программисту на Java приходится изменять состояние значения-объекта, вместо того чтобы иметь возможность напрямую изменять захваченную переменную.

https://t.me/javalib

15.2.6. Отложенные вычисления

В базовом языке Java напрямую не поддерживаются отложенные (lazy) вычисления для обычных значений. Однако есть одно интересное место, где отложенные вычисления все же применяются, — это API потоков данных (Streams). В приложении Б кратко описаны некоторые аспекты потоков данных, если вам потребуется освежить их в памяти.

> **ПРИМЕЧАНИЕ** Отложенные вычисления играют определенную роль в некоторых частях JVM и ее программного окружения (например, в отдельных аспектах загрузки классов).

При вызове `stream()` для коллекции Java создается объект `Stream`, который фактически представляет собой отложенное представление комбинации элементов. Некоторые потоки данных также могут представляться как коллекции Java, хотя в общем случае потоки данных имеют более общую природу и не каждый из них можно представить в виде коллекции.

Вернемся к типичному конвейеру Java с операциями `filter()` и `map()`:

```
             stream()  filter()  map()     collect()
Collection -> Stream -> Stream -> Stream -> Collection
```

Метод `stream()` возвращает объект `Stream`. Методы `map()` и `filter()`, как почти все операции над `Stream`, выполняются в отложенном режиме. На другом конце конвейера находится операция `collect()`, которая *материализует* текущее содержимое `Stream` обратно в `Collection`. Этот *терминальный* метод является неотложным (eager), так что полный конвейер выглядит так:

```
             отлож.    отлож.    отлож.    неотлож.
Collection -> Stream -> Stream -> Stream -> Collection
```

Не считая материализации обратно в коллекцию, платформа может полностью контролировать, какую часть потока данных задействовать в вычислениях. Это открывает путь к различным методам оптимизации, которые недоступны при чисто неотложных вычислениях.

В каком-то смысле отложенный, функциональный режим потоков данных Java можно уподобить перемещению через гиперпространство в научно-фантастическом фильме. Вызов `stream()` срабатывает как прыжок из «обычного пространства» в гиперпространство, в котором действуют другие правила — функциональная парадигма вместо объектно-ориентированной и отложенные (lazy) вычисления вместо неотложных (eager).

В конце конвейера терминальная операция над потоком данных возвращает нас из отложенного функционального мира в «обычное пространство». Это происходит, когда поток либо материализуется в `Collection` (например, с помощью `toList()`), либо агрегируется в результате `reduce()` или другой операции.

https://t.me/javalib

Отложенные вычисления требуют повышенного внимания со стороны программиста, но это бремя в основном ложится на плечи создателей библиотек, например разработчиков JDK. Тем не менее Java-разработчику стоит знать и соблюдать правила, связанные с некоторыми аспектами отложенной природы потоков данных. Например, реализации некоторых интерфейсов `java.util.function` (таких, как `Predicate` и `Function`) не должны изменять внутреннее состояние или создавать побочные эффекты. Если ваши реализации или лямбда-выражения будут нарушать это правило, это может привести к большим проблемам.

Еще одно важное качество потоков данных состоит в том, что сами объекты потоков данных (экземпляры `Stream`, которые служат промежуточными объектами в цепочке потоковых вызовов) являются одноразовыми. После обхода они должны считаться недействительными. Иначе говоря, не надо пытаться сохранить или повторно использовать объект потока данных, потому что результаты почти наверняка будут некорректными, а такие попытки могут привести к ошибке.

> **ПРИМЕЧАНИЕ** Помещать объект потока данных во временную переменную — это почти всегда признак «кода с душком». Впрочем, такой прием может принести пользу в процессе разработки при отладке нетривиальной проблемы с обобщениями. Главное, после отладки не забудьте удалить эти временные переменные.

Благодаря отложенным вычислениям потоки также позволяют моделировать данные более общего характера, чем коллекции. Например, можно сконструировать бесконечный поток, вызвав `Stream.generate()` в сочетании с функцией-генератором. Посмотрим, как это делается:

```
public class DaySupplier implements Supplier<LocalDate> {
    private LocalDate current = LocalDate.now().plusDays(1);

    @Override
    public LocalDate get() {
        var tmp = current;
        current = current.plusDays(1);
        return tmp;
    }
}

final var tomorrow = new DaySupplier();
Stream.generate(() -> tomorrow.get())
      .limit(10)
      .forEach(System.out::println);
```

Этот код создает бесконечный поток дней (или по крайней мере настолько большой, насколько потребуется). Представить его в виде коллекции было бы невозможно — это привело бы к исчерпанию памяти. Это показывает, что потоки данных имеют более общую природу.

Этот пример также демонстрирует, что ограничения Java, такие как передача по значению, несколько сужают пространство проектировочных решений. `LocalDate` — неизменяемый класс, а значит, нам нужно создать класс

https://t.me/javalib

с изменяемым полем `current`, а затем изменить `current` внутри метода `get()`, чтобы получить метод с состоянием, который сможет генерировать последовательность объектов `LocalDate`.

В языках, которые поддерживают передачу по ссылке, можно было бы обойтись без типа `DaySupplier`, объявив `current` как локальную переменную в одной области видимости с переменной `tomorrow`, которая могла бы быть лямбда-выражением.

15.2.7. Каррирование и частичное применение

Вы уже знаете, что Java не поддерживает каррирование на уровне языка, но мы можем в общих чертах рассмотреть, как добавить соответствующие средства. Например, вот как объявлен интерфейс `BiFunction` из `java.util.function`:

```
@FunctionalInterface
public interface BiFunction<T, U, R> {
    R apply(T t, U u);

    default <V> BiFunction<T, U, V> andThen(
                                  Function<? super R, ? extends V> after) {

        Objects.requireNonNull(after);
        return (T t, U u) -> after.apply(apply(t, u));
    }
}
```

Обратите внимание, как методы по умолчанию (одна из возможностей интерфейсов) используются в определении `andThen()` — дополнительного метода наряду со стандартным `apply()` для `BiFunction`. Таким же способом можно было бы реализовать некоторый вариант поддержки каррирования, например, определив два новых метода по умолчанию:

```
default Function<U, R> curryLeft(T t) {
        return u -> this.apply(t, u);
    }

    default Function<T, R> curryRight(U u) {
        return t -> this.apply(t, u);
    }
```

Эти методы определяют два способа создания объектов Java `Function` — функций с одним аргументом на основе исходного определения `BiFunction`. Обратите внимание на то, что они реализованы в виде замыканий. Мы просто захватываем переданное значение и сохраняем его на будущее — до того момента, когда функция будет реально применяться. Затем эти дополнительные методы по умолчанию можно использовать примерно так:

```
BiFunction<Integer, LocalDate, String> bif =
                                       (i, d) -> "Счетчик для "+d+"="+i;

Function<LocalDate, String> withCount = bif.curryLeft(42);
Function<Integer, String> forToday = bif.curryRight(LocalDate.now());
```

https://t.me/javalib

Правда, такой синтаксис получается несколько неуклюжим: он требует двух разных методов для двух разных каррированных версий, и они должны называться по-разному из-за стирания типов (type erasure). Причем даже после этого полученная возможность вряд ли хорошо подходит для широкого применения, так что этот подход так и не был реализован, и в Java, как упоминалось выше, нет встроенной поддержки каррирования.

15.2.8. Система типов Java и коллекции

В завершение печальной истории о том, что мешает использовать Java для функционального программирования, поговорим о системе типов и коллекциях языка Java. В этих областях можно выделить три основные проблемы с точки зрения функционального программирования:

- в системе типов нет единого корня;
- `Void`;
- архитектура коллекций Java.

Прежде всего, в системе типов Java нет единого корня (то есть нет общего надтипа у `Object` и `int`). Из-за этого на Java невозможно написать `List<int>`, что в результате приводит к автоупаковке и сопутствующим проблемам.

> **ПРИМЕЧАНИЕ** Многие разработчики жалуются на то, что параметры-типы обобщенных типов стираются во время компиляции, но на самом деле проблемы с обобщениями в коллекциях чаще возникают из-за того, что в системе типов нет единого корня.

`void` — еще одна проблема Java, связанная с тем, что в системе типов нет единого корня. Это ключевое слово означает, что метод не возвращает значения (или, другими словами, что в момент возврата стек вычисления метода пуст). Таким образом, семантика `void` заключается в том, что метод выполняет всю свою работу исключительно ради побочных эффектов — в определенном смысле это противоположно понятию чистоты.

Существование `void` означает, что в Java имеются как инструкции, так и выражения и в нем невозможно реализовать принцип «все является выражением», к которому так ревностно относятся некоторые традиции функционального программирования.

> **ПРИМЕЧАНИЕ** В главе 18 рассматривается проект Valhalla, который среди прочего дает проектировщикам языка Java потенциальную возможность исправить отсутствие единого корня в системе типов.

Другая проблема связана с формой и природой интерфейсов коллекций Java. Они были добавлены в версию языка 1.2 (она же Java 2), которая вышла в декабре 1998 года и проектировалась без расчета на функциональное программирование.

https://t.me/javalib

Функциональное программирование плохо применимо к коллекциям Java прежде всего потому, что они фундаментально ориентированы на изменяемость. Интерфейсы коллекций обширны и явно включают такие методы, как эти примеры из `List<E>`:

- `boolean add(E e);`
- `E remove(int index)`.

Это изменяющие методы: их сигнатура подразумевает, что сам объект коллекции изменяется на месте.

У соответствующих методов неизменяемых списков были бы сигнатуры вида `List<E> add(E e)`, которые возвращают новую, измененную копию списка. Реализовать `remove()` было бы труднее, потому что Java не позволяет возвращать из методов несколько значений.

> **ПРИМЕЧАНИЕ** Настоящая проблема здесь в том, что `remove()` не соответствует принципам функционального программирования, — этот пример очень похож на ситуацию с Iterator, которая рассматривалась в разделе 10.4.

Из всего сказанного следует, что для любой реализации коллекций неявно подразумевается изменяемость. Существует кошмарный трюк с `UnsupportedOperationException`, представленный в разделе 6.5.3, но серьезный Java-разработчик не будет заниматься такими вещами.

В других языках тип коллекции и изменяемость не связаны друг с другом: например, они представляются разными интерфейсами (или разными *типажами* в языках, которые поддерживают эту концепцию). В результате, чтобы обозначить свою изменяемость или неизменяемость на уровне типа, реализации могут реализовывать (или не реализовывать) разные интерфейсы.

На заднем плане всего этого стоит одно из главных преимуществ Java, которое остается важнейшим принципом проектирования языка, — обратная совместимость. Из-за нее бывает трудно (или невозможно) изменить некоторые особенности языка, чтобы сделать его более функционально-ориентированным. Например, разработчики языка не пытались включать дополнительные функциональные методы непосредственно в интерфейсы коллекций, а вместо этого ввели потоки данных `Stream` — новую разновидность контейнера, у которой нет неявной семантики коллекций.

Конечно, сколько ни добавляй новые типы контейнеров и API, от этого никак не изменятся миллионы и миллиарды строк уже существующего кода, в котором используются коллекции. Новые типы также ничем не помогут в распространенной ситуации, когда API уже выражается в понятиях типа коллекции.

> **ПРИМЕЧАНИЕ** Эта проблема характерна не только для разделения между потоками данных и коллекциями. Например, механизм рефлексии появился в Java 1.1, еще до коллекций. В результате API рефлексии неудобно использовать, потому что в качестве контейнеров элементов там применяются массивы.

https://t.me/javalib

В этом разделе были приведены некоторые неутешительные факты о поддержке функционального программирования в Java. Основной вывод таков: в Java доступны простые функциональные паттерны (такие, как «фильтрация — отображение — свертка»). Они находят разнообразные применения, а также хорошо обобщаются для конкурентных (и даже распределенных) приложений, но по сути, это предел того, что можно сделать в Java. А теперь перейдем к альтернативным языкам для JVM и посмотрим, как в них обстоит дело с функциональным программированием.

15.3. ФУНКЦИОНАЛЬНОЕ ПРОГРАММИРОВАНИЕ НА KOTLIN

Мы уже показали, как в современных версиях Java реализуются элементарные, типичные паттерны парадигмы функционального программирования. Вряд ли вас удивит, что Kotlin добавляет в эту копилку более компактный синтаксис и некоторые дополнительные возможности.

В этом разделе будут кратко представлены основные моменты, но если вы захотите изучить тему глубже, обращайтесь к книге Марко Вермёлена (Marco Vermeulen), Рунара Бьярнасона (Rúnar Bjarnason) и Пола Кьюзано (Paul Chiusano) «Functional Programming in Kotlin» (Manning, 2021, http://mng.bz/o2Wr).

15.3.1. Чистые функции и функции высшего порядка

В разделе 9.2.4 были представлены функции Kotlin. В этом языке функции являются частью системы типов и выражаются в синтаксисе вида `(Int) -> Int`, где в круглых скобках указываются типы аргументов, а справа от стрелки — тип возвращаемого значения.

В таком формате легко записываются сигнатуры функций, которые принимают другие функции в качестве аргументов или возвращают функции. Kotlin органически поощряет использование таких функций высшего порядка. Большáя часть API для работы с коллекциями (в частности, `map` и `filter`) на самом деле построены на основе функций высшего порядка, как мы видели в разделах об аналогичных возможностях API языков Java и Clojure.

Однако функции высшего порядка полезны не только для коллекций и потоков данных. Например, в классическом функциональном программировании есть функция, известная под именем `compose`: она возвращает функцию, которая вызывает каждую из функций, переданных ей в аргументах:

```
fun compose(callFirst: (Int) -> Int,
            callSecond: (Int) -> Int): (Int) -> Int {
  return { callSecond(callFirst(it)) }
}
```

compose возвращает функцию, так что при выполнении этой строки не вызываются callFirst и callSecond

https://t.me/javalib

```
val c = compose({ it*2},{it+10})
c(10)
```

Два лямбда-выражения передаются с помощью сокращенной записи it (см. главу 9), чтобы не указывать явно единственный аргумент лямбда-выражений

Вызываем и запускаем функцию, которую возвращает compose; эта функция возвращает 30

В Kotlin можно по-разному обращаться с функциями в зависимости от того, что вам нужно. Можно объявлять лямбда-выражения, как показано ранее (с учетом многих нюансов и особенностей, которые обсуждались в главе 9). Также можно обратиться к именованной функции с использованием синтаксиса `::`, как показано ниже:

```
fun double(x: Int): Int {
  return x * 2
}

val c = compose(::double, { it + 10 })
c(10)
```

Тот же результат, что и в предыдущем примере

Оператор `::` позволяет ссылаться не только на функции верхнего уровня, но и на функции, принадлежащие конкретному экземпляру объекта:

```
data class Multiply(var factor: Int) {
  fun apply(x: Int): Int=x*factor
}

val m = Multiply(2)
val c = compose(m::apply, { it + 10 })
c(10)
```

Ссылается на метод apply класса Multiply, специально связанный с экземпляром m

К сожалению, как и в Java, в Kotlin нет встроенной возможности гарантировать, что та или иная функция будет чистой. Хотя функции определяются на верхнем уровне (вне какого-либо класса), а `val` обеспечивает неизменяемость данных, эти особенности устраняют часть проблем, но не гарантируют ссылочную прозрачность ваших функций.

15.3.2. Замыкания

Одна особенность лямбда-выражений, которая неочевидна на первый взгляд, заключается в том, как они взаимодействуют с окружающим кодом. Например, следующий код работает, хотя `local` и не объявляется в лямбда-выражении:

```
var local = 0
val lambda = { println(local) }
lambda()
```

Выводит 0

Этот механизм называется *замыканием* (то есть лямбда-выражение *замыкается* по значениям, которые ему видны). Важно, что, в отличие от Java, лямбда-выражение может обращаться не только к значениям переменных — во внутренней реализации хранится ссылка на сами переменные:

https://t.me/javalib

```
var local = 0
val lambda = { println(local) }
lambda()  <-- Выводит 0

local = 10
lambda()  <-- Выводит 10 — обновленное значение local на момент выполнения лямбда-выражения
```

Замыкание по переменным остается даже тогда, когда сами переменные выходят из области видимости. В следующем примере функция возвращает лямбда-выражение, а в нем сохраняется ссылка на переменную, которая в обычной ситуации была бы недоступна:

```
fun makeLambda(): () -> Unit {
  val inFunction = "Я из функции makeLambda"
  return { println(inFunction) }
}

val lambda = makeLambda()
lambda()
```

В обычной ситуации inFunction вышла бы из области видимости при завершении makeLambda

Поскольку лямбда-выражение замыкается по inFunction, эта переменная все еще доступна — но только внутри лямбда-выражения

ПРИМЕЧАНИЕ Если лямбда-выражения выносят ссылки за пределы их типичной области видимости, это может стать источником непредвиденной утечки объектов!

От того, где объявлено лямбда-выражение, зависит, что оно может захватывать в своем замыкании. Например, если оно объявлено в классе, оно может замыкаться по свойствам объекта:

```
class ClosedForBusiness {
  private val amount = 100
  val check = { println(amount) }
}

fun getTheCheck(): () -> Unit {
  val closed = ClosedForBusiness()
  return closed.check
}

val check = getTheCheck()
check()
```

В переменной check сохраняется лямбда-выражение, которое замыкается по частному свойству amount

Эта функция возвращает ранее упомянутое лямбда-выражение, которое хранит ссылку на amount. В результате экземпляр closed продолжает существовать, хотя обычно он не должен существовать после завершения функции

При вызове выводит 100. Когда переменная check выйдет из области видимости, экземпляр closed наконец станет доступным для сборщика мусора

Замыкания с функциями высшего порядка открывают широкие возможности для создания новых функций на базе старых.

15.3.3. Каррирование и частичное применение

Ситуация с каррированием в Kotlin выглядит примерно так же, как в Java. Рассмотрим пример:

```
fun add(x: Int, y: Int): Int {
  return x + y
}
```

https://t.me/javalib

```
fun partialAdd(x: Int): (Int) -> Int {
  return { add(x, it) }
}

val addOne = partialAdd(1)
println(addOne(1))
println(addOne(2))

val addTen = partialAdd(10)
println(addTen(1))
println(addTen(2))
```

На самом деле это просто синтаксический трюк, который работает, потому что оператор `()` разворачивается в вызов метода `apply()`. Соответствующий байт-код остается точно таким же, как в примере на Java. Можно представить себе вспомогательный синтаксис, который позволяет автоматически создавать каррированные функции, — например, он мог бы выглядеть так:

```
val addOne: (Int) -> Int = add(1, _)
println(addOne(10))
```

Тем не менее в базовом языке такая возможность не поддерживается напрямую. Некоторые сторонние библиотеки предоставляют похожие, хотя и не столь лаконичные средства, которые обычно опираются на механизм расширения.

15.3.4. Неизменяемость

В разделе 15.1.2 мы охарактеризовали неизменяемость как один из ключевых факторов успеха функционального программирования. Если чистая функция возвращает данные, которые должны быть одинаковыми для заданных входных данных, то возможность последующего изменения объекта нарушает те гарантии, которые предоставляет чистота.

Основная особенность Kotlin, которая помогает достичь неизменяемости, — объявление `val`. Оно гарантирует, что значение свойства можно задать только во время конструирования объекта. Объявление `val` фактически эквивалентно комбинации `final var` в Java, но оно также применимо к свойствам и записывается намного удобнее.

В главе 9 рассмотрены многие ситуации, в которых Kotlin допускает `val` или `var`, но чтобы добиться успеха в функциональном программировании, рекомендуется ориентироваться на неизменяемость и отдавать предпочтение `val` перед `var`. Встроенная поддержка свойств в Kotlin также позволяет отказаться от шаблонных методов чтения, которые необходимы в Java:

```
class Point(val x: Int, val y: Int)

val point = Point(10, 20)
println(point.x)
// point.x = 20 // Не компилируется, потому что значение x неизменяемо!
```

https://t.me/javalib

Впрочем, у неизменяемых объектов есть один существенный недостаток: если вам в самом деле понадобится что-нибудь изменить, это может оказаться непросто. Чтобы сохранить неизменяемость, приходится создавать новые экземпляры, но это утомительно и ненадежно. В Java проблему часто позволяют решить статические фабричные методы, объекты-строители или методы `with*()`, которые снижают уровень шума в коде.

Конструкция `data class` в Kotlin — хорошая альтернатива этим решениям. Кроме конструктора и операций проверки равенства, которые рассматривались в разделе 9.3.1, у класса данных также есть метод `copy`. Используя его вместе с именованными аргументами Kotlin, можно сгенерировать нужный вам новый экземпляр, записывая только те изменения, которые необходимы:

```
data class Point(val x: Int, val y: Int)

val point = Point(10, 20)
val offsetPoint = point.copy(x = 100)
```

По поводу `copy` стоит сделать пару важных предупреждений. Во-первых, это поверхностное копирование. Если одно из полей содержит объект, то копируется не сам объект, а ссылка на него. Как и в Java, если хотя бы одно звено цепочки объектов может меняться, то гарантии нарушаются. Чтобы сохранять неизменяемость, все задействованные объекты должны выполнять соответствующие требования, но сам по себе язык не заставит их это сделать.

Также следует помнить, что `copy` создает копию объекта исключительно на основании конструктора класса. Если нарушить правила и включить в объекты дополнительные `var`-поля, то `copy` не будет о них знать и в любой копии они получат только значения по умолчанию:

```
data class Point(val x: Int, val y: Int) {
  var shown: Boolean = false
}

val point = Point(10, 20)
point.shown = true

val newPoint = point.copy(x = 100)
println(newPoint.shown)
```

Выводит false, потому что copy не учитывает поля, которые не задействованы в конструкторе

Но ведь вы ни за что не позволите изменяемым полям просочиться в ваши замечательные неизменяемые объекты, верно?

Управление изменяемостью объектов — важный первый шаг, но бóльшая часть реального кода имеет дело с коллекциями, а не с отдельными экземплярами. В главе 9 было показано, что функции Kotlin для конструирования коллекций (`listOf`, `mapOf` и т. д.) возвращают интерфейсы (такие, как `kotlin.collections.List` и `kotlin.collections.Map`), которые, в отличие от своих аналогов из `java.util`, доступны только для чтения. Это хорошее начало, но, к сожалению, оно не дает необходимых гарантий.

https://t.me/javalib

На неизменяемость этих объектов нельзя положиться, потому что существуют изменяемые интерфейсы, которые их расширяют. Везде, где можно передать `List`, также можно передать `MutableList`:

```
fun takesList(list: List<String>) {
  println(list.size)
}

val mutable = mutableListOf("Всем", "привет")
takesList(mutable)

mutable.add("от меня")
takesList(mutable)
```

В обоих вызовах `takesList` получает один и тот же объект, но результаты различаются. Функциональная частота нарушена!

> **ПРИМЕЧАНИЕ** Неизменяющие вспомогательные методы вроде `listOf` используют нижележащие коллекции JDK и возвращают объекты, доступные только для чтения. Например, `listOf` по умолчанию использует реализацию списка на базе массива, в который нельзя добавлять новые элементы. Однако все портится, если смешивать изменяемые интерфейсы Kotlin со стандартными интерфейсами, доступными только для чтения.

Из-за того что эти коллекции реализованы через классы JDK, также возникают проблемы при преобразовании между интерфейсами. Kotlin стремится к чистому взаимодействию с коллекциями Java, а это означает, что результат `listOf()` можно привести как к изменяемым интерфейсам Kotlin, так и к классической реализации `java.util.List<T>`, которая может попытаться изменить коллекцию! Следующий код компилируется без помех, но во время выполнения происходит сбой:

```
fun takesMutableList(muted: MutableList<Int>) {
  muted.add(4)
}

val list = listOf(1,2,3)
takesMutableList(list as MutableList<Int>)
```

Этот вызов возбуждает исключение java.lang.UnsupportedOperationException

Отсутствие реальной неизменяемости создает особые проблемы, когда речь заходит о конкурентном выполнении. Как рассказывалось в главе 6, приходится приложить существенные усилия, чтобы обеспечить безопасность изменяемых коллекций между потоками. Но если бы у нас был по-настоящему неизменяемый экземпляр коллекции, мы могли бы свободно распределять его между разными потоками выполнения, не сомневаясь, что все потоки будут видеть его одинаково.

Хотя в стандартной библиотеке Kotlin нет неизменяемых *перманентных* структур, библиотека `kotlinx.collections.immutable` (http://mng.bz/nNjg) предоставляет разнообразную подборку таковых. Многочисленные аналоги также встречаются в таких популярных библиотеках, как Guava и Apache Commons.

В чем проявляется «перманентность» для коллекций? Как неоднократно упоминалось ранее, неизменяемость означает, что когда объект нужно «изменить»,

https://t.me/javalib

вместо этого создается его новый экземпляр. Для больших коллекций это может оказаться весьма неэффективным. Перманентные коллекции используют неизменяемость, чтобы сократить затраты на изменение: они конструируются так, чтобы безопасно совместно использовать неизменяемые части, которые хранятся в памяти. Чтобы внести изменения, все равно приходится создавать новые объекты, однако они могут быть намного меньше полной копии, как показано ниже:

```
import kotlinx.collections.immutable.persistentListOf

val pers = persistentListOf(1, 2, 3)
val added = pers.add(4)
println(pers)   <-------  Выводит [1, 2, 3]
println(added)  <-------  Выводит [1, 2, 3, 4]
```

Ядро библиотеки реализует две группы интерфейсов — `ImmutableList` и `PersistentList`. Аналогичные пары существуют для отображений, множеств и коллекций общего назначения. `ImmutableList` расширяет `List`, но в отличие от базового интерфейса, он гарантирует неизменяемость любого экземпляра. `ImmutableList` можно использовать везде, где передаются списки и нужно обеспечить неизменяемость. `PersistentList` строится на основе `ImmutableList` и предоставляет методы, которые работают по принципу «изменить и вернуть».

В библиотеку также входят знакомые расширения для того, чтобы преобразовывать другие коллекции в перманентные версии:

```
val mutable = mutableListOf(1,2,3)
val immutable = mutable.toImmutableList()
val persistent = mutable.toPersistentList()
```

Возникает вопрос: почему бы не преобразовать все `listOf` в `persistentListOf` «на всякий случай»? Но перманентные структуры данных неспроста не выбраны в качестве реализации по умолчанию. Хотя они снижают затраты на копирование, они все еще работают медленнее, чем классические изменяемые структуры данных. Меньше копирования — еще не значит, что его нет вообще! Насколько высоки накладные затраты?

Хочется надеяться, что глава 7 убедила вас, что это можно выяснить только одним способом: провести измерения на ваших практических примерах. Но если вам нужен конкурентный доступ к коллекции из разных потоков, то стоит сравнить, какие результаты показывают эти перманентные структуры по сравнению со стандартным копированием с синхронизацией. Теперь в вашем распоряжении есть средства Kotlin для неизменяемости, и мы можем посмотреть, какие возможности они открывают в мире рекурсивных функций.

15.3.5. Хвостовая рекурсия

В разделе 15.2.4 рассматривались рекурсивные функции в Java. Они страдали от серьезного ограничения: каждый последующий рекурсивный вызов добавляет кадр в стек, что со временем приведет к исчерпанию доступной памяти. У Kotlin

https://t.me/javalib

те же проблемы с базовой рекурсией; чтобы убедиться в этом, достаточно перевести функцию `simpleFactorial` на Kotlin (обратите внимание, что выражение `if` используется как возвращаемое значение):

```
fun simpleFactorial(n: Long): Long {
  return if (n <= 0) {
    1
  } else {
    n * simpleFactorial(n - 1)
  }
}
```

Для этой функции генерируется следующий байт-код, эквивалентный тому, что `javac` генерировал для Java:

```
public final long simpleFactorial(long);
    Code:
       0: lload_1
       1: lconst_0
       2: lcmp
       3: ifgt 10
       6: lconst_1
       7: goto 22
      10: lload_1
      11: aload_0
      12: checkcast #2                  // class Factorial
      15: lload_1
      16: lconst_1
      17: lsub
      18: invokevirtual #19             // Method simpleFactorial:(J)J
      21: lmul
      22: lreturn
```

Не считая небольшой дополнительной проверки (байт 12) и `goto` вместо нескольких инструкций `lreturn`, код практически не изменился. Рекурсивный вызов со смещением 18, где выполняется `invokevirtual` для `simpleFactorial`, рано или поздно переполнит стек:

```
java.lang.StackOverflowError
    at Factorial.simpleFactorial(factorial.kts:32)
    at Factorial.simpleFactorial(factorial.kts:32)
    at Factorial.simpleFactorial(factorial.kts:32)
  ...
```

И хотя в общем случае эта проблема неизбежна, Kotlin может помочь, если мы имеем дело с хвостовой рекурсивной функцией. Напомним: так называется функция, у которой рекурсия является последней операцией во всей функции. Ранее мы показали, как на уровне байт-кода можно сбросить состояние и с помощью `goto` вернуться в начало функции, вместо того чтобы добавлять новый кадр в стек. Таким образом рекурсивный вызов преобразуется в цикл, где нам не грозит переполнение стека. Java не дает такой возможности, но в Kotlin это можно сделать.

https://t.me/javalib

ПРИМЕЧАНИЕ Любую рекурсивную функцию можно переписать в хвостовом рекурсивном виде. При этом могут потребоваться дополнительные параметры, переменные и различные трюки для преобразования, но теоретически это всегда возможно. А поскольку всякую хвостовую рекурсивную функцию можно преобразовать в обычный цикл, получается, что любую рекурсивную функцию тоже можно реализовать в итеративной форме, используя только циклические конструкции.

Чтобы при вычислении факториала переместить рекурсивный вызов в конечную позицию, понадобятся некоторые ухищрения. Как и в Java, мы разобьем функцию на две: для пользователей сохраняется удобная форма с одним аргументом, а более сложная рекурсивная функция, которая теперь должна получать несколько аргументов, становится отдельной функцией:

```
fun tailrecFactorial(n: Long): Long {
  return if (n <= 0) {
    1
  } else {
    helpFact(n, 1)
  }
}

tailrec fun helpFact(i: Long, j: Long): Long {
  return if (i == 0L) {
    j
  } else {
    helpFact(i - 1,i*j)
  }
}
```

Благодаря пометке tailrec Kotlin знает, что здесь ожидается хвостовая рекурсия

Входная функция `tailrecFactorial` не преподносит никаких сюрпризов в байткоде. Она проверяет начало диапазона, а затем переходит к вспомогательной функции с хвостовой рекурсией:

```
public final long tailrecFactorial(long);
    Code:
       0: lload_1
       1: lconst_0
       2: lcmp
       3: ifgt 10
       6: lconst_1
       7: goto 19
      10: aload_0
      11: checkcast #2                  // class Factorial
      14: lload_1
      15: lconst_1
      16: invokevirtual #10             // Method helpFact:(JJ)J
      19: lreturn
```

Байты 0–3 проверяют условие раннего возврата, который реализован в байтах 6–7. Если надо выполнить рекурсивный вызов, то далее загружаются значения, которые нужны, чтобы вызвать `invokevirtual` для `helpFact` в байте 16.

https://t.me/javalib

Самое важное отличие, которое создает ключевое слово `tailrec`, проявляется в байт-коде `helpFact`:

```
public final long helpFact(long, long);
    Code:
       0: lload_1
       1: lconst_0
       2: lcmp
       3: ifne 10
       6: lload_3
       7: goto 26
      10: aload_0
      11: checkcast #2                  // class Factorial
      14: pop
      15: lload_1
      16: lconst_1
      17: lsub
      18: lload_1
      19: lload_3
      20: lmul
      21: lstore_3
      22: lstore_1
      23: goto 0
      26: lreturn
```

Бóльшая часть кода метода выполняет логические проверки и арифметические действия, но все самое интересное происходит в байте 23. Вместо того чтобы вызвать `invokevirtual` для `helpFact` и запустить рекурсию, мы просто выполняем переход `goto 0` и начинаем выполнение функции заново. Без инструкций `invoke` пропадает риск переполнения стека, и это замечательно. Кто сказал, что команды `goto` всегда вредны?

Хвостовая рекурсия становится элегантным решением, если вашу функцию можно переписать в подходящей форме. Но что, если запросить `tailrec` для функции, где нет хвостовой рекурсии?

```
tailrec fun simpleFactorial(n: Long): Long {
  return if (n <= 0) {
    1
  } else {
    n * simpleFactorial(n - 1)
  }
}
```

Некорректный запрос tailrec в ситуации, когда последним действием функция не вызывает саму себя; в данном случае последняя операция — это умножение на результат рекурсивного вызова

Kotlin распознает проблему и предупреждает, что байт-код не удается преобразовать так, чтобы использовать хвостовую рекурсию. Предупреждение прямо указывает на ошибочный, не хвостовой вызов:

```
factorial.kts:28:1: warning: a function is marked as tail-recursive
                            but no tail calls are found[1]
```

[1] Предупреждение: функция помечена как функция с хвостовой рекурсией, но хвостовые вызовы не обнаружены. — *Примеч. пер.*

https://t.me/javalib

```
tailrec fun simpleFactorial(n: Long): Long {
^
factorial.kts:32:9: warning: recursive call is not a tail call[1]
    n * simpleFactorial(n - 1)
        ^
```

Выдавать предупреждение — не самое эффективное решение для компилятора, потому что может случиться, что мы сначала создадим реализацию с хвостовой рекурсией, а потом слегка изменим ее так, что возможность хвостовой рекурсии пропадет. Если процесс сборки не выловит эту проблему, код может попасть в реальную эксплуатацию, где приведет к ошибке `StackOverflowError` во время выполнения. Возможно, было бы лучше, если бы запрос `tailrec` для функции, в которой нет хвостовой рекурсии, приводил к ошибке компиляции, как в некоторых других языках (например, Scala).

15.3.6. Отложенное вычисление

Как упоминалось ранее в этой главе, многие функциональные языки (например, Haskell) в значительной мере полагаются на отложенные («ленивые») вычисления. Kotlin как один из языков для JVM не задействует отложенные вычисления в своей основной модели выполнения, но обеспечивает для них поддержку первого класса там, где это потребуется, через интерфейс `Lazy<T>`. Тем самым предоставляется стандартная структура для ситуаций, в которых требуется отложить часть обработки (или полностью пропустить ее).

Обычно программисты не реализуют `Lazy<T>` самостоятельно, а конструируют экземпляры с помощью функции `lazy()`. В простейшей форме `lazy()` получает лямбда-выражение, от возвращаемого типа которого зависит тип `T` возвращаемого интерфейса. Лямбда-выражение не выполняется до тех пор, пока `value` не будет запрошено явно. Также можно проверить, вычислялось ли значение ранее:

```
val lazing = lazy {
  42
}

println("init? ${lazing.isInitialized()}")
println("value = ${lazing.value}")
println("init? ${lazing.isInitialized()}")
```

Проверяет, инициализировалась ли функция; выводит false

Если обратиться к value, лямбда-выражение выполняется и сохраняет результат

Проверяет, инициализировалась ли функция; выводит true

Иногда мы хотим не только отложить вычисления, которые не нужны прямо сейчас, но и организовать выполнение в нескольких потоках. В таких случаях `lazy()` принимает перечисление `LazyThreadSafetyMode`, которое помогает управлять этим выполнением. Перечисление состоит из значений `SYNCHRONIZED` (используется по умолчанию для `lazy()`), `PUBLICATION` и `NONE`:

- `SYNCHRONIZED` синхронизирует выполнение инициализирующего лямбда-выражения с помощью самого экземпляра `Lazy<T>`.

[1] Предупреждение: рекурсивный вызов не находится в конце функции. — *Примеч. пер.*

https://t.me/javalib

- `PUBLICATION` вместо этого разрешает несколько конкурентных выполнений инициализирующего лямбда-выражения, но сохраняет только первое обнаруженное значение.
- `NONE` не выполняет синхронизацию; поведение при конкурентных обращениях не определено.

ПРИМЕЧАНИЕ `LazyThreadSafetyMode.NONE` следует использовать, только если вы: 1) на основании измерений сделали вывод, что синхронизация в отложенных экземплярах действительно ухудшает производительность, и 2) можете как-то гарантировать, что никогда не будете обращаться к объекту из разных потоков. Между остальными вариантами — `SYNCHRONIZED` и `PUBLICATION` — стоит выбирать в зависимости от того, подходит ли вам, что инициализирующее лямбда-выражение будет выполняться много раз в конкурентном режиме.

Интерфейс `Lazy<T>` хорошо совместим с другой расширенной возможностью Kotlin — *делегируемыми свойствами*. Определяя свойство в классе, вместо значения или собственных методов чтения и записи можно предоставить объект с ключевым словом `by`. Этот объект должен содержать реализацию `getValue()`, а также (для свойств `var`) `setValue()`. Как мы покажем далее, `Lazy<T>` соответствует этой спецификации, что позволяет легко откладывать инициализацию свойств в классах, не повторяя шаблонный код и не отступая от естественного синтаксиса Kotlin:

```
class Procrastinator {
  val theWork: String by lazy {
    println("Хорошо, сейчас сделаю…")
    "Наконец-то готово"
  }
}

val me = Procrastinator()

println(me.theWork)
println(me.theWork)
println(me.theWork)
```

Диагностическое сообщение, помогающее убедиться, что все работает как надо

Первый вызов theWork выполнит лямбда-выражение и выведет итоговое сообщение

Последующие вызовы theWork, как видно из двух завершающих строк, просто возвращают то же самое, уже вычисленное значение

Как и неизменяемость, отложенные вычисления хорошо подходят для объектов, которые вы сами создаете, но с коллекциями и итераторами возникают вопросы. В следующем разделе вы увидите, как Kotlin помогает управлять последовательностью выполнения при обходе коллекций с помощью интерфейса `Sequence<T>`.

15.3.7. Последовательности

Функции коллекций Kotlin часто оказываются удобными, но они предполагают, что функция неотложно применяется ко всей коллекции. Как показано ниже, для каждого шага в цепочке функций создается промежуточная коллекция, а это может оказаться лишней работой, если полный результат вам на самом деле не нужен.

https://t.me/javalib

```
val iter = listOf(1, 2, 3)
val result = iter
  .map { println("1@ $it"); it.toString() }
  .map { println("2@ $it"); it + it }
  .map { println("3@ $it"); it.toInt() }
```

Генерирует промежуточную коллекцию ["1", "2", "3"]

Генерирует промежуточную коллекцию ["11", "22", "33"]

Генерирует финальный результат [11, 22, 33]

Отслеживая процесс выполнения на рис. 15.1, можно увидеть, что каждый шаг цепочки вызовов `map` выполняется с целым списком, прежде чем запустится следующий `map`.

```
val iter = listOf(1, 2, 3)
val result = iter
    .map { it.toString() }
    .map { it + it }
    .map { it.toInt() }
```

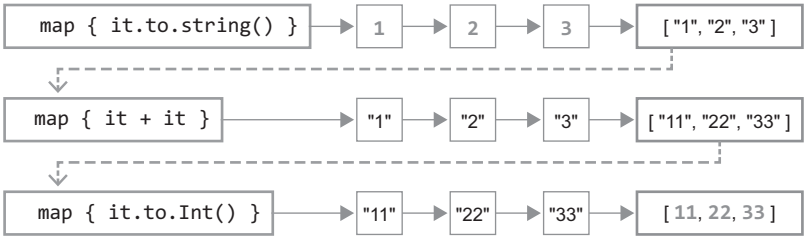


Рис. 15.1. Стандартный обход в коллекциях

В результате получится такой вывод:

```
1@ 1
1@ 2
1@ 3
2@ 1
2@ 2
2@ 3
3@ 11
3@ 22
3@ 33
```

Кроме того, что это может вылиться в напрасную трату ресурсов, также возможны ситуации с потенциально бесконечным объемом ввода. Что, если вы захотите обрабатывать числа, пока пользователь не дает команду остановиться? В этом случае не получится создать список заранее и обрабатывать его полное содержимое шаг за шагом.

Для таких задач в Kotlin существуют *последовательности*. В основе последовательности лежит интерфейс `Sequence<T>`, который похож на `Iterable<T>`, но при этом предоставляет множество новых возможностей.

https://t.me/javalib

Можно создать новую последовательность функцией `sequenceOf()`, а затем начать применять к ней функции, как и в случае с уже привычными коллекциями. В следующем примере мы преобразовали список в последовательность и сохранили диагностический вывод, который помогает проследить за происходящим:

```
val seq = sequenceOf(1, 2, 3)
val result = seq
    .map { println("1@ $it"); it.toString() }
    .map { println("2@ $it"); it + it }
    .map { println("3@ $it"); it.toInt() }
```

Обратите внимание: + здесь выполняет конкатенацию строк, а не арифметическое сложение

Если запустить эту короткую программу, вы увидите, что она ничего не выводит. Главная особенность последовательностей — отложенное вычисление. Нигде в этой программе не требуются результаты вызовов `map`, поэтому Kotlin просто не запускает эти `map`! Но если преобразовать последовательность в список, как показано в следующем примере, программе придется все вычислять, и вы увидите, как выполняются операции с последовательностью:

```
val seq = sequenceOf(1, 2, 3)
val result = seq
    .map { println("1@ $it"); it.toString() }
    .map { println("2@ $it"); it + it }
    .map { println("3@ $it"); it.toInt() }
    .toList()
```

Из рис. 15.2 видно, что каждый элемент последовательности проходит через цепочку `map` по отдельности — сначала 1, затем 2 и т. д., после чего выполняется переход к обработке следующего элемента.

Результат выглядит так:

```
1@ 1
2@ 1
3@ 11
1@ 2
2@ 2
3@ 22
1@ 3
2@ 3
3@ 33
```

Это интересно, но на примере относительно небольших статических списков выглядит не слишком убедительно. Без полноценных измерений мы не можем уверенно утверждать, но можем предположить, что в этом случае бюрократия, связанная с последовательностями, может свести на нет весь выигрыш от того, что в памяти не создаются промежуточные коллекции. Мощь последовательностей становится нагляднее, если рассмотреть альтернативные способы их создания.

Начнем с функции `asSequence()`. Как нетрудно предположить, она преобразует итерируемые объекты в последовательность. Правда, функция работает не только с привычными списками и коллекциями, но и с интервалами.

https://t.me/javalib

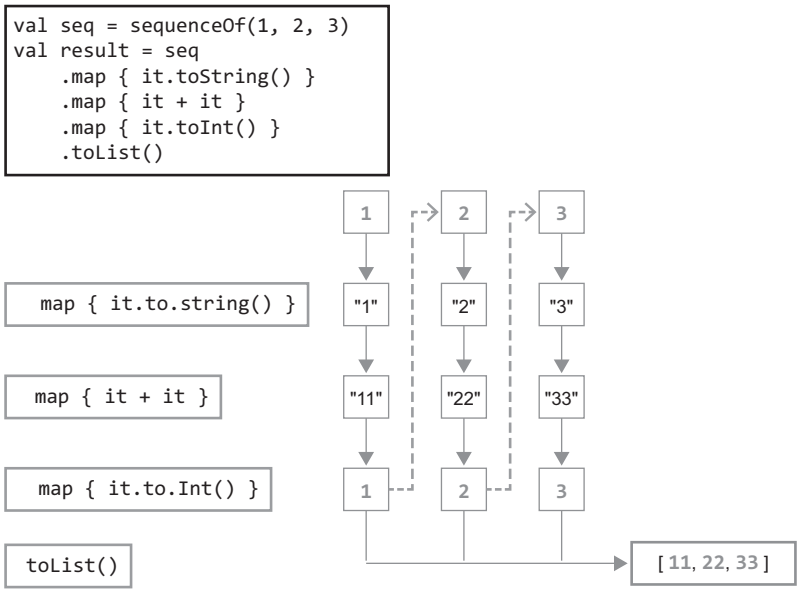


Рис. 15.2. Операции с последовательностью

Числовые интервалы Kotlin уже встречались в разделе 9.2.6, где они использовались с выражениями `when`, чтобы проверить принадлежность. Однако по интервалам также можно итерироваться. Например, их можно объединить с `asSequence()`, чтобы создавать длинные числовые списки, не вводя объемный код и не выделяя лишнюю память:

```
(0..1000000000)
  .asSequence()
  .map{it*2}
```

Интервал хранит в памяти только начало и конец, так что это объявление не создает миллиард элементов

Но что, если даже интервалы (большие, но все-таки не бесконечные) кажутся вам слишком ограниченными? Вас выручит `generateSequence()` из `kotlin.sequences` в стандартной библиотеке. Функция создает новый объект обобщенной последовательности с необязательным начальным значением. Каждый раз, когда ей нужен следующий элемент, она выполняет предоставленное лямбда-выражение, передавая ему предыдущее значение:

```
generateSequence(0) { it+2}
```

Бесконечная последовательность четных чисел

С бесконечным `Iterable<T>` было бы невозможно объединять методы в цепочки, потому что первый вызов для этого объекта никогда не вернет управление. `Sequence<T>` получает только то, что необходимо, и откладывает остальное на будущее. Такой подход хорошо сочетается с функцией `take()`, у которой можно запросить конкретное количество элементов, чтобы получить составленную из них новую, ограниченную последовательность:

https://t.me/javalib

```
generateSequence(0) { it+2}
    .take(3)
    .forEach { println(it) }
```

Создает последовательность из первых трех элементов

Принудительно запускает вычисление по последовательности и выводит полученные результаты

Операция `forEach()` для последовательностей относится к *терминальным*: она переводит последовательность из отложенного состояния в неотложное и обрабатывает все ее элементы. Раньше вы уже видели другую терминальную операцию `toList()`, которая обязательно перебирает каждый элемент, чтобы построить список.

Kotlin предлагает и другой вариант создания последовательности на тот случай, если ее по какой-то причине не удается выстроить, опираясь только на один начальный элемент. Комбинация `sequence()` и `yield()` позволяет конструировать совершенно произвольные последовательности:

```
val yielded = sequence {
  yield(1)
  yield(100)
  yieldAll(listOf(42, 3))
}
```

yieldAll() принимает итерируемый объект того же типа, что мы собираемся генерировать, и по запросу последовательно генерирует каждый элемент

Как обычно для последовательностей, лямбда-выражение, которое определяет следующий элемент, выполняется в отложенном режиме. Необычно здесь то, что для каждого вызова, в результате которого должен получиться очередной элемент, лямбда-выражение выполняется только до следующего `yield`, после чего приостанавливается. Когда будет запрошен следующий элемент, лямбда-выражение возобновляет выполнение с той же точки, где приостановилось, и опять выполняется до следующего `yield`, как показано на рис. 15.3.

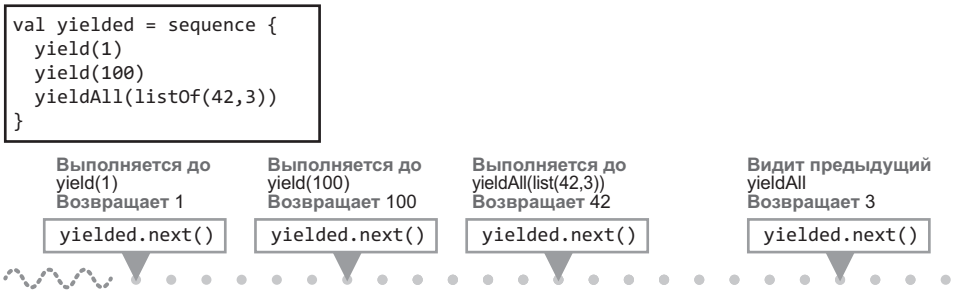


Рис. 15.3. Временна́я ось выполнения yield

`yield` использует такую возможность Kotlin, как *приостанавливаемые функции* (suspend functions). Как можно догадаться по названию, это функции, в которых Kotlin распознает точки выполнения кода, где выполнение можно приостановить и возобновить. В данном случае Kotlin видит, что каждый раз, когда мы генерируем значение, выполнение лямбда-выражения `sequence` нужно приостановить,

https://t.me/javalib

пока не будет запрошено следующее значение. Хотя наш код выглядит как простое лямбда-выражение, компилятор Kotlin на самом деле выполняет много дополнительной работы «под капотом».

Приостанавливаемые функции тесно связаны с альтернативной моделью конкурентного выполнения в Kotlin — корутинами, которые были представлены в разделе 9.4 и подробнее рассматриваются в главе 16. Интересно заметить, что функциональность, которую мы часто рассматриваем в контексте конкурентности, также открывает уникальные возможности в области функционального программирования.

15.4. ФУНКЦИОНАЛЬНОЕ ПРОГРАММИРОВАНИЕ НА CLOJURE

В главе 10 мы познакомились с основами функционального программирования на языке Clojure, в котором используются формы вида `(map)`. Также были кратко описаны такие понятия, как неизменяемость и функции высшего порядка, потому что они играют фундаментальную роль в модели программирования на Clojure.

В этом разделе мы расскажем о практике функционального программирования в Clojure, но не по той же схеме, по какой строился материал, посвященный Java и Kotlin. Вместо этого мы пойдем дальше этих основ и покажем, как работают некоторые расширенные функциональные возможности Clojure, начиная со списковых включений.

15.4.1. Включения

В функциональном программировании важная роль отводится концепции *включений* (comprehensions). Она происходит из математики, где множества часто задаются записью следующего вида:

```
{ x ∈ ℕ : (x / 2) ∈ ℕ }
```

В этой записи ∈ означает «принадлежит» (или «является элементом»), ℕ — бесконечное множество натуральных чисел (1, 2, 3 и т. д.), а двоеточие (:) определяет условие или ограничение.

Таким образом, это включение описывает множество натуральных чисел, которые обладают особым свойством: каждое число этого множества при делении на 2 дает число, которое тоже является натуральным. Конечно, мы уже знаем это множество под другим названием — четные числа.

Главное здесь в том, что мы не задавали множество четных чисел, перечисляя его элементы (это было бы невозможно, потому что их количество бесконечно). Вместо этого мы начали с множества натуральных чисел и указали дополнительное условие, по которому из него выбираются элементы для нового множества.

https://t.me/javalib

Если это напоминает вам применение таких функциональных средств, как `filter`, вы не ошибаетесь: одно с другим очень тесно связано. Однако функциональные языки часто поддерживают как включения, так и `filter` — `map`, потому что каждое из этих средств оказывается концептуально удобнее в определенных ситуациях.

Для реализации *списковых включений* в Clojure используется форма `(for)`, которая возвращает список (в некоторых случаях — итератор). Вот почему, когда мы рассказывали о циклах в Clojure в главе 10, мы не упоминали цикл `(for)`: на самом деле это не цикл. Рассмотрим его в действии:

```
user=> (for [x [1 2 3]] (* x x))
(1 4 9)
```

Форма `(for)` получает два аргумента: вектор аргументов и форму, представляющую значения, которые нужно сгенерировать как часть общего списка, возвращаемого `(for)`.

Вектор аргументов содержит пару (или несколько пар) элементов: переменную, которая будет использоваться в определении генерируемых значений, и последовательность с входными данными. Можно считать, что переменная связывается с каждым из элементов последовательности по очереди. Конечно, то же самое можно записать в виде отображения:

```
user=> (map #(* % %) [1 2 3])
(1 4 9)
```

Когда же следует использовать `(for)`? Эта форма особенно полезна, когда приходится строить более сложные структуры:

```
(for [num [1 2 3]
      ch [:a :b :c]]
  (str num ch))
("1:a" "1:b" "1:c" "2:a" "2:b" "2:c" "3:a" "3:b" "3:c")
```

Такого же результата можно добиться с помощью отображения, но конструкция получится более сложной и громоздкой, тогда как с `(for)` она проста и незамысловата. Чтобы получить эффект фильтра, также можно использовать дополнительный квалификатор `:when`, который может действовать как ограничение:

```
user=> (for [x (range 8) :when (even? x)] x)
(0 2 4 6)
```

А теперь посмотрим, как в Clojure реализуется отложенное выполнение, особенно применительно к последовательностям.

15.4.2. Отложенные последовательности

В Clojure отложенное выполнение чаще всего встречается при работе с последовательностями, а не с отдельными значениями. Для последовательностей оно означает, что в нашем распоряжении нет полного списка всех значений, входящих в последовательность, но мы можем получать значения тогда, когда в них

https://t.me/javalib

возникнет необходимость (например, вызывая функцию, которая генерирует их по требованию).

В случае коллекций Java такая идея потребовала бы нестандартной реализации `List`, которую не удалось бы эффективно воплотить без больших объемов шаблонного кода. С другой стороны, если использовать реализацию `ISeq`, можно написать код следующего вида:

```
public class SquareSeq implements ISeq {
    private final int current;

    private SquareSeq(int current) {
        this.current = current;
    }

    public static SquareSeq of(int start) {
        if (start < 0) {
            return new SquareSeq(-start);
        }
        return new SquareSeq(start);
    }

    @Override
    public Object first() {
      return Integer.valueOf(current * current);
    }

    @Override
    public ISeq rest() {
      return new SquareSeq(current + 1);
    }
}
```

Здесь не предусмотрено хранилище для значений; вместо этого каждый новый элемент последовательности генерируется по требованию, что позволяет моделировать бесконечные последовательности. Кроме того, взгляните на следующий пример:

```
public class IntGeneratorSeq implements ISeq {
  private final int current;
  private final Function<Integer, Integer> generator;

  private IntGeneratorSeq(int seed,
                          Function<Integer, Integer> generator) {
    this.current = seed;
    this.generator = generator;
  }

  public static IntGeneratorSeq of(int seed,
                                   Function<Integer, Integer> generator) {
    return new IntGeneratorSeq(seed, generator);
  }

  @Override
  public Object first() {
```

https://t.me/javalib

```
    return generator.apply(current);
  }

  @Override
  public ISeq rest() {
    return new IntGeneratorSeq(generator.apply(current), generator);
  }
}
```

В этом коде результат применения функции используется для того, чтобы предоставить начальное значение следующей последовательности. Если функция-генератор является чистой, то все отлично работает, но разумеется, в Java никто не даст таких гарантий. Рассмотрим мощные макросы Clojure, которые помогают создавать отложенные последовательности с минимальными усилиями.

Подумайте, как бы вы представили отложенную, потенциально бесконечную последовательность. Один из очевидных вариантов — использовать функцию, которая генерирует элементы последовательности. Функция должна обладать двумя качествами:

- возвращать следующий элемент последовательности;
- принимать фиксированное конечное количество аргументов.

Математики говорят, что такие функции определяют *рекурсивное соотношение*. Из теории таких соотношений непосредственно следует, что в подобных ситуациях уместно применять рекурсию.

Вообразите, будто на вашем компьютере нет ограничений на размер стека и другие ресурсы и вы можете создать два потока выполнения: один генерирует бесконечную последовательность, а другой ее использует. После этого можно определить в генерирующем потоке отложенную последовательность с помощью рекурсии; соответствующий псевдокод будет выглядеть примерно так:

```
(defn infinite-seq <vec-args>
  (let [new-val (seq-fn <vec-args>)]
    (cons new-val (infinite-seq <new-vec-args>))
  ))
```

В реальной системе это не будет работать, потому что рекурсивный вызов (infinite-seq) приведет к переполнению стека

Чтобы стек не переполнился, можно добавить конструкцию, которая заставляет Clojure оптимизировать рекурсию и применять ее только в случае необходимости: речь идет о макросе `(lazy-seq)`. В следующем листинге рассмотрен короткий пример, который определяет отложенную последовательность k, $k + 1$, $k + 2$, ... для некоторого числа k.

Листинг 15.1. Пример отложенной последовательности

```
(defn next-big-n [n] (let [new-val (+ 1 n)]
  (lazy-seq                                    ; Маркер отложенной последовательности
    (cons new-val (next-big-n new-val))        ; Бесконечная рекурсия
  )))
```

https://t.me/javalib

```
(defn natural-k [k]
  (concat [k] (next-big-n k)))  ◄—— concat ограничивает рекурсию

1:57 user=> (take 10 (natural-k 3))
(3 4 5 6 7 8 9 10 11 12)
```

Ключевые моменты этого кода — форма `(lazy-seq)`, которая отмечает точку, где может возникнуть бесконечная рекурсия, и форма `(concat)`, которая безопасно с ней справляется. После этого с помощью формы `(take)` можно извлечь из отложенной последовательности необходимое количество элементов. Отложенные последовательности — чрезвычайно мощная конструкция, и когда вы накопите опыт, вы увидите, что это очень полезный инструмент в арсенале Clojure.

15.4.3. Каррирование в Clojure

Каррирование функций в Clojure сопряжено с дополнительными затруднениями по сравнению с другими языками. Это связано с тем, что многие формы Clojure являются вариадическими, как обсуждалось в главе 10.

Вариадические функции усложняют ситуацию, потому что из-за них возникают вопросы типа: «Что имеет в виду пользователь — каррировать форму с двумя аргументами или вычислить форму с одним аргументом?» — особенно из-за того, что в Clojure функции вычисляются неотложно.

Проблема решается формой `(partial)`, которая, кстати, является настоящей функцией Clojure, а не макросом. Рассмотрим ее в действии:

```
user=> (filter (partial = :b) [:a :b :c :b :d])
(:b :b)
```

Функция `=` получает один или несколько аргументов, так что в неотложном режиме `(=:b)` вычислилась бы в `true`, но `(partial)` преобразует ее в каррированную функцию. При вызове `(filter)` она распознается как функция с одним аргументом (фактически перегруженная версия с одним аргументом), а затем используется, чтобы проверять каждый элемент в следующем векторе.

> **ПРИМЕЧАНИЕ** Форма `(partial)` сама по себе каррирует только первый параметр формы. Если вы хотите каррировать другой параметр, `(partial)` нужно будет объединить с другой формой, которая переставляет список аргументов перед тем, как применять функцию.

Бесспорно, Clojure — самый функциональный из трех языков, которые мы рассматривали, и если это описание вызвало у вас интерес, перед вами простираются необъятные возможности для исследований. То, что мы здесь рассмотрели, — всего лишь начало, но и оно позволяет убедиться, что сама JVM может быть хорошей платформой для функционального программирования, — проблема скорее в языке Java, на котором трудно программировать в функциональном стиле.

В этой главе мы заглянули в мир функционального программирования намного глубже, чем предусматривает традиционная парадигма

https://t.me/javalib

«фильтрация — отображение — свертка» потоков данных Java. Для этого пришлось выйти за пределы Java и обратиться к другим языкам.

Конечно, можно пойти еще дальше. Существуют две основные школы функционального программирования: школа динамической типизации, которую представляет Clojure (и другие языки за пределами JVM, например Erlang), и школа статической типизации, к которой относится Kotlin, хотя, пожалуй, еще более удачным примером будет Scala (и Haskell среди языков, которые не выполняются на JVM).

Как бы то ни было, одно из главных преимуществ Java — обратная совместимость — также оказывается потенциальным недостатком языка. Код на Java, откомпилированный для версии 1.0 (более 25 лет назад), по-прежнему будет в неизменном виде выполняться на современных JVM. Однако за это выдающееся достижение приходится расплачиваться. Java API и даже архитектура байт-кода и JVM вынуждены жить с проектировочными решениями, которые сейчас трудно или невозможно изменить и которые плохо совместимы с функциональным программированием. Это одна из главных причин, по которым разработчикам, желающим использовать функциональный стиль, часто приходится переходить на другие языки для JVM, отличные от Java.

ИТОГИ

- Паттерн «фильтрация — отображение — свертка» — отправная, а не конечная точка функционального программирования.
- Язык Java не очень хорошо приспособлен для функционального стиля, потому что в нем нет таких встроенных возможностей, как отложенное вычисление, каррирование и оптимизация хвостовой рекурсии.
- Другие языки для JVM могут эффективнее поддерживать функциональное программирование благодаря таким средствам, как `Lazy<T>` в Kotlin и отложенные последовательности в Clojure.
- На уровне JVM также есть определенные проблемы (например, изменяемость данных по умолчанию), которые невозможно принципиально исправить сменой языка программирования.

https://t.me/javalib

16 Продвинутое конкурентное программирование

В ЭТОЙ ГЛАВЕ

- ✓ Fork/Join API
- ✓ Алгоритмы перехвата работы
- ✓ Конкурентность и функциональное программирование
- ✓ Внутренняя реализация корутин Kotlin
- ✓ Конкурентное выполнение в Clojure
- ✓ Программная транзакционная память
- ✓ Агенты

Эта глава опирается на несколько тем из предшествующих глав. В частности, понятия функционального программирования из предыдущей главы объединяются с конкурентными библиотеками Java из главы 6. Также мы уделим внимание альтернативным языкам и рассмотрим некоторые новые аспекты конкурентности в Kotlin и Clojure.

ПРИМЕЧАНИЕ Понятия из этой главы, такие как корутины и агенты (или акторы), начинают играть все более значительную роль в области конкурентного выполнения Java.

https://t.me/javalib

Начнем с небольшого отступления: API Fork/Join языка Java. Этот фреймворк позволяет решать определенные типы конкурентных задач эффективнее, чем исполнители, с которыми вы встречались в главе 6.

16.1. ФРЕЙМВОРК FORK/JOIN

Как упоминалось в главе 7, скорость процессоров (или, выражаясь точнее, количество транзисторов на микросхемах) значительно возросла за последние годы. Однако производительность ввода/вывода прогрессирует более скромными темпами, так что ожидание ввода/вывода стало обыденным явлением. А это наводит на мысль о том, что вычислительный потенциал компьютера можно было бы использовать эффективнее. Именно эту задачу пытается решить фреймворк Fork/Join (F/J, в дословном переводе — «разбиение и слияние»).

F/J обеспечивает прозрачное для пользователя автоматическое планирование задач в пуле потоков. Для этого необходимо, чтобы задачи можно было разбивать по запросу пользователя. F/J опирается на понятие «малых» и «больших» задач, что чрезвычайно естественно для фреймворка.

Рассмотрим основные факты и фундаментальные принципы, относящиеся к F/J:

- Фреймворк вводит новую разновидность служб-исполнителей — `ForkJoinPool`.
- `ForkJoinPool` оперирует единицами конкурентного выполнения `ForkJoinTask`, которые «меньше», чем `Thread`.
- Задачи `ForkJoinTask` можно планировать под управлением `ForkJoinPool` с меньшими затратами ресурсов.
- F/J использует задачи двух видов (оба представляются экземплярами `ForkJoinTask`):
  - «малые» задачи — такие, которые можно выполнить сразу, не потребляя слишком много процессорного времени;
  - «большие» задачи — такие, которые необходимо разделять на части (возможно, за несколько приемов), прежде чем их можно будет выполнить напрямую;
- Фреймворк предоставляет базовые методы, которые позволяют разбивать большие задачи.
- Фреймворк обеспечивает автоматическое планирование и перепланирование.

У F/J есть одна ключевая особенность: предполагается, что облегченные задачи могут порождать другие экземпляры `ForkJoinTask`, которые будут планироваться в том же пуле потоков, который выполнял их родительскую задачу. Этот принцип иногда характеризуется фразой «*разделяй и властвуй*».

https://t.me/javalib

Начнем с простого примера использования F/J, а затем вкратце рассмотрим специфику задач, которые хорошо подходят для этого типа конкурентной обработки. Далее мы обсудим механизм «перехвата работы», который применяется в F/J, и его актуальность в более широком контексте.

16.1.1. Простой пример F/J

В качестве простого примера того, что умеет фреймворк F/J, рассмотрим такую ситуацию: есть некоторое количество объектов транзакций, созданных в разное время. Здесь мы будем представлять их классом `Transaction` — это результат эволюции класса `TransferTask`, который упоминался в главах 5 и 6:

```
public class Transaction implements Comparable<Transaction> {
    private final Account sender;
    private final Account receiver;
    private final int amount;
    private final long id;
    private final LocalDateTime time;

    private static final AtomicLong counter = new AtomicLong(1);

    Transaction(Account sender, Account receiver,
                int amount, LocalDateTime time) {
        this.sender = sender;
        this.receiver = receiver;
        this.amount = amount;
        this.id = counter.getAndIncrement();
        this.time = time;
    }

    public static Transaction of(Account sender, Account receiver,
                                 int amount) {
        return new Transaction(sender, receiver,
                               amount, LocalDateTime.now());
    }

    @Override
    public int compareTo(Transaction other) {
        return Comparator.nullsFirst(LocalDateTime::compareTo)
                         .compare(this.time, other.time);
    }

    // Методы чтения и другие методы (equals, hashcode и т. д.) опущены
}
```

Требуется получить список транзакций, отсортированный по времени. Для этого мы применим F/J как инструмент многопоточной сортировки, которая, по сути, представляет собой разновидность сортировки слиянием.

В нашем примере используется `RecursiveAction` — специализированный подкласс `ForkJoinTask`. Он проще универсального класса `ForkJoinTask`, потому что

https://t.me/javalib

в нем явно выражено отсутствие общего результата (транзакции будут переупорядочиваться на месте) и делается акцент на рекурсивной природе задач.

Класс `TransactionSorter` позволяет упорядочивать список обновлений с помощью метода `compareTo()` для объектов `Transaction`. Метод `compute()` (который нужно реализовать, потому что это абстрактный метод надкласса `RecursiveAction`) упорядочивает массив транзакций по времени создания, как показано в следующем листинге.

Листинг 16.1. Сортировка с помощью RecursiveAction

```
public class TransactionSorter extends RecursiveAction {
    private static final int SMALL_ENOUGH = 32;
    private final Transaction[] transactions;
    private final int start, end;
    private final Transaction[] result;

    public TransactionSorter(List<Transaction> transactions) {
        this(transactions.toArray(new Transaction[0]),
            0, transactions.size());
    }

    public TransactionSorter(Transaction[] transactions) {
        this(transactions, 0, transactions.length);
    }

    public TransactionSorter(Transaction[] txns, int start, int end) {
        this.start = start;
        this.end = end;
        this.transactions = txns;
        this.result = new Transaction[this.transactions.length];
    }

    /**
     * Этот метод реализует простую сортировку слиянием (Mergesort).
     * Если вам интересны детали реализации, обратитесь
     * к подходящему учебнику.
     *
     * @param left
     * @param right
     */
    private void merge(TransactionSorter left, TransactionSorter right) {
        int i = 0;
        int lCount = 0;
        int rCount = 0;

        while (lCount < left.size() && rCount < right.size()) {
            int comp = left.result[lCount].compareTo(right.result[rCount]);
            result[i++] = (comp < 0)
                    ? left.result[lCount++]
                    : right.result[rCount++];
        }
```

32 и менее элементов сортируются последовательно

https://t.me/javalib

```
        while (lCount < left.size()) {
            result[i++] = left.result[lCount++];
        }

        while (rCount < right.size()) {
            result[i++] = right.result[rCount++];
        }
    }

    public int size() {
        return end - start;
    }

    public Transaction[] getResult() {
        return result;
    }

    @Override
    protected void compute() {  <-- Метод, определенный в RecursiveAction
        if (size() < SMALL_ENOUGH) {
            System.arraycopy(transactions, start, result, 0, size());
            Arrays.sort(result, 0, size());
        } else {
            int mid = size() / 2;
            TransactionSorter left =
                new TransactionSorter(transactions, start, start + mid);
            TransactionSorter right =
                new TransactionSorter(transactions, start + mid, end);
            invokeAll(left, right);

            merge(left, right);
        }
    }
}
```

Сортировщиком можно управлять с помощью кода вроде того, который приведен ниже. Этот код генерирует транзакции и перемешивает их, прежде чем передавать сортировщику. На выходе получаются переупорядоченные обновления:

```
var transactions = new ArrayList<Transaction>();
var accs = new Account[] {
              new Account(1000),
              new Account(1000)};

for (var i = 0;i<256;i=i+1){
    transactions.add(Transaction.of(accs[i % 2], accs[(i + 1) % 2], 1));
    Thread.sleep(1);
}
Collections.shuffle(transactions);

var sorter = new TransactionSorter(transactions);
var pool = new ForkJoinPool(4);

pool.invoke(sorter);
```

https://t.me/javalib

```
for (var txn : sorter.getResult()) {
    System.out.println(txn);
}
```

Перспективы F/J выглядят соблазнительно, но на практике не каждая проблема легко сводится к такой простой форме, как только что рассмотренная многопоточная реализация `MergeSort`.

Это пример антипаттерна «Простые случаи такие простые», при котором разработчик может соблазниться простой на первый взгляд технологией, которая позволяет решить простую задачу с минимальными усилиями, но скрывает тот факт, что технология плохо масштабируется или обобщается для более сложных случаев. Давайте разберемся, какие задачи стоит решать с помощью F/J, а для каких лучше подойдут другие средства.

16.1.2. Параллелизация задач для F/J

Вот несколько примеров задач, которые хорошо подходят для F/J:

- Моделирование движения большого количества простых объектов (например, эффекты частиц).
- Анализ файлов журналов.
- Операции с данными, где некоторая величина вычисляется по агрегированным входным данным (например, операции «отображения — свертки»).

Иными словами, можно сказать, что F/J эффективен для задач, которые можно разбить на части, как на рис. 16.1.

Чтобы определить, можно ли успешно разбить задачу, попробуйте ответить на следующие вопросы:

- Могут ли подзадачи работать без явного взаимодействия или синхронизации между собой?
- Вычисляют ли подзадачи некоторое значение по своим данным, не изменяя их (то есть являются ли они чистыми функциями)?
- Является ли подход «разделяй и властвуй» естественным для выделения подзадач?

Если вы ответили на эти вопросы «Да!» или «В основном, хотя есть краевые случаи», — возможно, ваша задача хорошо подходит для F/J. Однако если вы ответили «Может быть» или «Пожалуй, нет», — не исключено, что F/J покажет не лучшие результаты, а другие подходы сработают эффективнее.

Спроектировать хороший многопоточный алгоритм непросто, и фреймворк F/J работает не в любых условиях. В конечном итоге вам самим придется решать, относится ли ваша задача к той области, где F/J наиболее полезен. Если нет, будьте готовы заняться разработкой собственного решения, для которого, скорее всего, вам пригодится превосходный инструментарий `java.util.concurrent`.

https://t.me/javalib

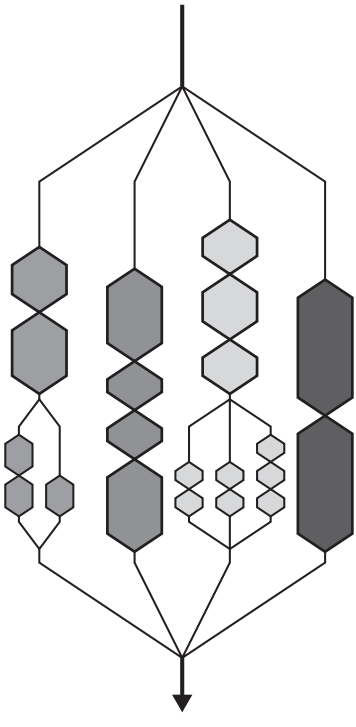


Рис. 16.1. Разбиение и слияние задач (Fork/Join)

16.1.3. Алгоритмы перехвата работы

`ForkJoinTask` является надклассом по отношению к `RecursiveAction`. Это обобщенный класс по возвращаемому типу действия (таким образом, `RecursiveAction` расширяет `ForkJoinTask<Void>`). В результате `ForkJoinTask` прекрасно подходит для операций «отображение — свертка», которые сокращают набор данных и либо возвращают результат, либо действуют посредством побочного эффекта (как в случае с `RecursiveAction`).

Планированием объектов типа `ForkJoinTask` занимается `ForkJoinPool` — новая разновидность службы-исполнителя, предназначенная специально для таких облегченных задач. Служба поддерживает список задач для каждого потока, и если одна задача завершается, служба может переназначить задачи от полностью загруженного потока к простаивающему. На рис. 16.2 показано, как это происходит.

Без алгоритма *перехвата работы* (work-stealing algorithm) могли бы возникнуть проблемы планирования из-за того, что задачи делятся на два типа по размеру. В общем случае задачи разного размера могли бы требовать существенно разного времени выполнения.

https://t.me/javalib

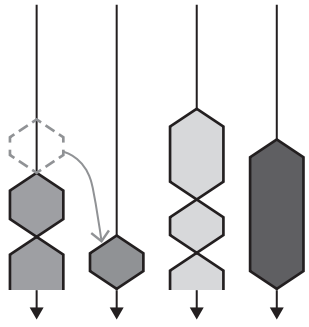


Рис. 16.2. Перехват работы: когда второй поток завершает свою задачу, служба переназначает ему задачу от все еще занятого первого потока

Например, предположим, что в одном потоке работает очередь выполнения, состоящая только из малых задач, а в другом выполняются только большие задачи. Если малая задача выполняется в пять раз быстрее большой, то поток с малыми задачами часто будет простаивать, пока не завершится поток больших задач.

> **ПРЕДУПРЕЖДЕНИЕ** Перехват работы опирается на предположение о том, что задачи не зависят друг от друга. Если это не так, то результаты вычислений могут различаться между запусками.

Перехват работы был реализован именно для того, чтобы обойти эту проблему и предусмотреть, что все пулы потоков будут нагружаться работой на протяжении жизненного цикла задания F/J. Все происходит полностью автоматически, и чтобы пользоваться преимуществами перехвата работы, вам не нужно предпринимать специальных усилий. Это еще один пример того, как среда выполнения помогает разработчикам управлять конкурентным выполнением, чтобы его не приходилось координировать вручную.

> **ПРИМЕЧАНИЕ** `ForkJoinPool` также используется в популярных библиотеках Java и JVM, например в системе Akka, которая обеспечивает конкурентность на основе акторов в Scala и Java.

Для взаимодействия с `ForkJoinPool` класс предоставляет следующие основные методы:

- `execute()` — запускает асинхронное выполнение.
- `invoke()` — запускает выполнение и ожидает результата.
- `submit()` — запускает выполнение и возвращает в качестве результата предназначенный объект.

Начиная с Java 8, в среде выполнения есть общий пул, к которому можно обращаться с помощью `ForkJoinPool.commonPool()`. Он предоставляется в основном ради функциональности перехвата работы — вряд ли стоит ожидать, что многие программы будут пользоваться им для рекурсивной декомпозиции.

https://t.me/javalib

У общего пула есть ряд настраиваемых свойств, которые позволяют, например, регулировать уровень параллелизма (то есть количество используемых потоков) и определять, каким фабричным классом будут создаваться новые потоки для общего пула.

16.2. КОНКУРЕНТНОСТЬ И ФУНКЦИОНАЛЬНОЕ ПРОГРАММИРОВАНИЕ

В главе 5 мы познакомились с понятием неизменяемых объектов и узнали, что они чрезвычайно полезны в конкурентном программировании, потому что обходят фактор *общего изменяемого состояния*, который лежит в основе многих проблем конкурентного выполнения. Можно предположить, что средства функционального программирования, которые опираются на неизменяемость, способны сыграть важную роль в разработке конкурентных приложений. И это действительно так, хотя в контексте конкурентного программирования также понадобится немного расширить понятие неизменяемости.

16.2.1. Снова о CompletableFuture

В главе 6 был представлен класс `CompletableFuture`. Этот тип не является неизменяемым, но модель его состояния очень проста:

- Сначала он находится в незавершенном состоянии.
- Любая попытка получить от него значение вызовом `get()` блокируется.
- Затем в какой-то момент времени происходит событие публикации.
- В результате этого события объекту присваивается значение, и оно передается любым потокам, которые были заблокированы по вызову `get()`.
- Опубликованное значение теперь остается неизменяемым.

На рис. 16.3 представлен преднамеченный объект (future) и событие публикации.

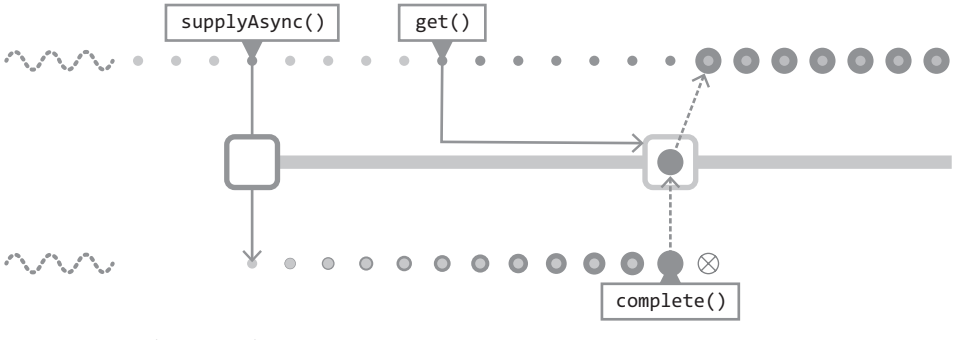


Рис. 16.3. Событие публикации присваивает значение и передает его всем потокам, которые были заблокированы по get()

https://t.me/javalib

Важное преимущество `CompletableFuture` заключается в том, что его результат можно использовать в функциях, и он будет вычисляться в отложенном режиме, то есть функция не будет выполняться до тех пор, пока не появится значение.

Такие функции можно создавать синхронно или асинхронно. Пожалуй, это проще всего продемонстрировать на примерах. Вернемся к классу `NumberService` из главы 6 и используем для него реализацию-пустышку:

```
public class NumberService {
    public static long findPrime(int n) {
        try {
            Thread.sleep(5_000);
        } catch (InterruptedException e) {
            throw new CancellationException("interrupted");
        }
        return 42;
    }
}
```

Очевидно, что, хотя этот класс и называется `findPrime`, он не находит простые числа. Однако его будет вполне достаточно, чтобы продемонстрировать, как ведут себя потоки. Для этого понадобится написать небольшой фрагмент кода:

```
var n = 1000;
var future =
  CompletableFuture.supplyAsync(() -> {
  System.out.println("Начинаем с: "+ Thread.currentThread().getName());
  return NumberService.findPrime(n);
});
var f2 = future.thenApply(l -> {
  System.out.println("Применяем к: "+ Thread.currentThread().getName());
  return l * 2;
});
var f3 = future.thenApplyAsync(l -> {
  System.out.println("Асинхронно применяем к: "+ Thread.currentThread().
getName());
  return l * 3;
});

try {
  System.out.println("F2: "+ f2.get());
  System.out.println("F3: "+ f3.get());
} catch (InterruptedException | ExecutionException e) {
  e.printStackTrace();
}
```

Предоставляет вычисления, которые должны выполняться асинхронно

Предоставляет функцию, которую нужно применить к результату асинхронных вычислений

Предоставляет другую функцию, которую нужно асинхронно применить к результату

Запустив этот код, мы получим такой результат:

```
Начинаем с thread: ForkJoinPool.commonPool-worker-19
Применяем к thread: ForkJoinPool.commonPool-worker-19
Асинхронно применяем к thread: ForkJoinPool.commonPool-worker-5
F2: 84
F3: 126
```

https://t.me/javalib

Преднамеченный объект `f2`, который использует `thenApply()`, выполняется в том же потоке, что и `future`, тогда как `f3` (который использует `thenApplyAsync()`) выполняется в другом потоке из пула.

Можно заметить, что по умолчанию при выполнении всего кода `CompletableFuture` используется общий пул. Как видно из предыдущего вывода, этот пул доступен по имени `ForkJoinPool.commonPool`.

В некоторых обстоятельствах разработчикам может понадобиться альтернативный пул потоков. Например, для стандартного пула нельзя настроить максимальное количество используемых потоков, а это может быть неприемлемо для некоторых рабочих нагрузок. К счастью, у фабричных методов `CompletableFuture` (таких, как `supplyAsync()`) существуют перегруженные версии, которые принимают явный аргумент `Executor`. Это позволяет преднамеченному объекту выполняться в отдельном пуле потоков.

Кроме метода `thenApply()`, `CompletableFuture` также предоставляет `thenCompose()`. Некоторые разработчики плохо понимают, чем различаются эти два метода, и этот момент стоит пояснить.

Вспомните, что `thenApply()` принимает в качестве аргумента объект `Function`, который выполняет отображение `T -> U`. Эта функция синхронно применяется к потоку, в котором выполняется `CompletableFuture` после того, как завершился исходный преднамеченный объект.

С другой стороны, `thenCompose()` принимает объект `Function`, который выполняет отображение `T` -> CompletableFuture<U>. (Фактически возвращается значение типа `CompletionStage<U>`, а не `CompletableFuture<U>`, но пока не будем обращать внимания на эти тонкости.) По сути, это асинхронная функция, и она может выполняться в другом потоке. Рассмотрим конкретный пример:

```
Function<Long, CompletableFuture<Long>>f=l->
    CompletableFuture.supplyAsync(() -> {
        System.out.println("Применяем к потоку: " +
                           Thread.currentThread().getName());
        return l * 2;
    });
```

Мы *могли бы* передать эту функцию методу `thenApply()`, но тогда результатом был бы `CompletableFuture<CompletableFuture<Long>>`. С другой стороны, `thenCompose()` сводит результат в `CompletableFuture<Long>`. Это похоже на то, как работает метод `flatMap()` в API потоков данных Java: он применяет к объекту потока данных функцию, которая возвращает `Stream<T>`, но не возвращает `Stream<Stream<T>>`, а объединяет отдельные потоки данных в один.

`CompletableFuture` также поддерживает метод `join()`, который фактически работает так же, как с потоками, но возвращает значение. Кроме того, можно «скрепить» преднамеченные объекты, если определить код, который должен выполняться после того, как выполнится один или оба из этих объектов, например:

https://t.me/javalib

```
var n = 1000;
var future = CompletableFuture.supplyAsync(() -> {
    System.out.println("Запускаем: "+ Thread.currentThread().getName());
    return NumberService.findPrime(n);
});

var future2 = CompletableFuture.supplyAsync(() -> {
    System.out.println("Запускаем: "+ Thread.currentThread().getName());
    return NumberService.findPrime(n);
});

Runnable dontKnow = () -> System.out.println("Один из преднамеченных объектов
                                              завершился");
future.runAfterEither(future2, dontKnow);
```

Теперь объединим материал о функциональном программировании из предыдущей главы с изложенными здесь идеями.

Если функции, которые мы собираемся применять к результату `CompletableFuture`, являются чистыми и не зависят ни от чего, кроме входных значений, то применение функции к преднамеченному объекту должно работать так же, как ее применение к результату. Это можно сформулировать по-другому: если рассматривать преднамеченный объект как контейнерный тип, который содержит значение, то контейнер «прозрачен» для функции, которая применяется к значению, когда оно появится.

В частности, здесь становятся актуальными два основных преимущества ссылочной прозрачности (то есть использования чистых функций):

- мемоизация;
- транспортируемость.

Мемоизация означает, что вызов любой чистой функции можно заменить на уже вычисленное значение: не нужно заново вызывать функцию с теми же аргументами, потому что ответ уже известен. Транспортируемость проявляется в том, что при вычислении чистой функции неважно, в каком потоке она выполняется, и результат ее применения не зависит от того, применяется ли она синхронно или асинхронно.

> **ПРИМЕЧАНИЕ** Как уже упоминалось ранее, язык Java далек от чистоты, так что вы сможете получить эти преимущества, только если будете внимательно следить за тем, чтобы всюду использовать только чистые функции и неизменяемые данные.

Хотя сейчас мы обсуждаем конкурентное функциональное программирование, будет уместно сказать несколько слов о *параллельных потоках данных* — это часть API потоков данных, которую многие программисты понимают неправильно.

https://t.me/javalib

16.2.2. Параллельные потоки данных

В главе 5 упоминался закон Амдала — один из фундаментальных результатов в области конкурентного выполнения применительно к *параллелизму данных*. Параллелизм имеет смысл использовать при больших объемах данных, которые устроены похожим образом и должны обрабатываться более или менее одинаково. В общем случае параллелизм данных полезен, когда выполняются все перечисленные условия:

- Есть большой объем данных, которые должны обрабатываться одинаково (или почти одинаково).
- Упорядочение не играет роли.
- Элементы данных не зависят друг от друга.
- Вы убедились, что этот шаг обработки является «узким местом».

Когда параллельные потоки данных появились в Java 8, они вызвали большой энтузиазм у многих разработчиков. Но как вы вскоре увидите, реальность несколько разошлась с исходными ожиданиями. Показанный ниже API кажется довольно простым:

```
// Просто заменяем stream() на parallelStream()
List<String> origins = musicians
    .parallelStream()
    .filter(artist -> artist.getName().startsWith("The"))
    .map(artist -> artist.getNationality())
    .collect(toList());
```

Во внутренней реализации всю работу координирует фреймворк F/J, и алгоритм перехвата работы распределяет вычисления по нескольким ядрам. Может показаться, что следующие утверждения слишком хороши, чтобы быть правдой:

- Работой управляет фреймворк.
- API получился явно выраженным, но ненавязчивым.
- Существует распределение по данным.
- `parallelStream()` позволяет переключаться между последовательным и параллельным режимами.
- Скорость обработки увеличивается «бесплатно».

И действительно, на самом деле ситуация обстоит иначе. Первая и совершенно очевидная проблема — закон Амдала. Чтобы разбить последовательную задачу на набор фрагментов, которые можно выполнять параллельно, нужна определенная работа, то есть затраты вычислительного времени. Чем больше будет подготовки и взаимодействия, тем меньше выигрыш от использования нескольких процессоров — в этом заключается суть закона Амдала.

Не существует универсального способа легко и надежно оценить относительные затраты на разбиение по сравнению с затратами на линейное выполнение.

https://t.me/javalib

Фреймворк возлагает на разработчика когнитивную нагрузку: ему нужно принять решение о том, оправданно ли применение параллелизма. Этот факт уже наводит на мысль, что «автоматический параллелизм» не настолько совершенен, как могло показаться. Конечному пользователю приходится беспокоиться о многочисленных подробностях, которые в идеале должны быть абстрагированы.

Всего один пример: разбиение и повторное объединение потоков должно происходить в пуле потоков внутри JVM. Чем больше потоков создает JVM, тем сильнее они конкурируют за процессорное время. API потоков данных сам по себе не знает, сколько других экземпляров параллельных потоков данных существует внутри текущего процесса. Это приводит нас к двум возможным, но одинаково неудачным стратегиям:

- Создать новый, выделенный пул потоков для каждого вызова параллельных потоков данных.
- Создать один экземпляр пула потоков (частного для JVM) и заставить все вызовы параллельных потоков данных его использовать.

Первый случай чреват тем, что потоки данных будут неограниченно плодиться, в результате чего в конце концов JVM исчерпает ресурсы или откажет. По этой причине в Java 8 существует единственный общий пул потоков, который используют параллельные потоки данных, — `ForkJoinPool.commonPool()`. Это потенциально приводит к состязаниям за общие ресурсы (а как мы видели в главе 7, это и есть настоящий корень многих проблем с быстродействием).

Существует обходное решение: если выполнять параллельный поток данных как задачу с `ForkJoinPool`, то поток данных будет использовать этот специализированный пул, а не общий:

```
// Использовать специализированный пул
var forkJoinPool = new ForkJoinPool(4);
List<String> origins2 = forkJoinPool.submit(() -> musicians
    .parallelStream()
    .filter(artist -> artist.getName().startsWith("The"))
    .map(artist -> artist.getNationality())
    .collect(toList())).get();
forkJoinPool.shutdown();
```

Обратите внимание, что `forkJoinPool` необходимо явно завершить, иначе он останется в памяти в ожидании новых задач, что приведет к утечке памяти и потоков.

В общем случае лучший совет, который можно дать по поводу параллельных потоков данных, — не применяйте параллелизм вслепую. Обязательно убедитесь, что его использование оправданно. Как обычно, для этого необходимо провести измерения и удостовериться, что операции с потоками данных действительно являются узким местом, а уже потом выяснять, помогут ли здесь параллельные потоки данных.

https://t.me/javalib

К сожалению, не существует единого класса задач, в котором параллельные потоки данных обычно приносили бы пользу. Каждый случай нужно анализировать и тестировать индивидуально. Только тогда можно попытаться применить параллелизм к потоку данных и на основании измерений доказать, что это приводит к оправданным улучшениям.

16.3. КАК РАБОТАЮТ КОРУТИНЫ KOTLIN

Как упоминалось в главе 9, в Kotlin есть *корутины* (coroutines) — альтернатива модели потоков (`Thread`). Корутины можно рассматривать как «облегченные потоки», которые потребляют меньше ресурсов, чем полноценные потоки операционной системы. Этот механизм отчасти похож на фреймворк Fork/Join языка Java. Чтобы понять, как Kotlin обеспечивает альтернативные средства выполнения, нужно глубже разобраться, что при этом происходит «под капотом».

16.3.1. Как работают корутины

Начнем с измененной версии примера из главы 9:

```
package com.wgjd

import kotlinx.coroutines.GlobalScope
import kotlinx.coroutines.delay
import kotlinx.coroutines.launch

fun main() {
  GlobalScope.launch {
    delay(1000)
    sus()
  }

  Thread.sleep(2000)
}

suspend fun sus() {
  println("Стоп машина!" )
}
```

Здесь `GlobalScope.launch` запускает новую корутину. Чтобы указать, как она должна выполняться, используются области видимости (scopes), о которых мы подробнее поговорим в следующем разделе.

Корутина, которую мы сейчас создали, ожидает в течение секунды (из-за функции `delay`), а затем вызывает нашу функцию `sus`. Далее с помощью `Thread.sleep` мы выдерживаем двухсекундную паузу, чтобы дать корутине время отработать до того, как вся программа завершится.

Одно из отличий корутин от потоков заключается в том, что корутины могут приостанавливаться в определенных точках. Как Kotlin узнает, где можно сделать остановку? Об этом сообщает ключевое слово `suspend`, которое предваряет функцию

https://t.me/javalib

`sus` (а также библиотечную функцию `delay`). Такие приостанавливаемые функции отмечают блоки кода, которые Kotlin рассматривает как единицы выполнения.

Когда приостанавливаемые функции отмечают фрагменты кода корутины, между которыми возможна приостановка, Kotlin может создать *конечный автомат*, который будет управлять выполнением корутины. Наш сгенерированный код отслеживает переходы в конечном автомате, а блоки, помеченные как `suspend`, предоставляют этапы работы автомата.

Преобразуем нашу корутину в конечный автомат:

```
GlobalScope.launch {
  delay(1000)
  sus()
}
```

Ниже перечислены этапы корутины, изображенные на рис. 16.4:

- Вызов `launch` создает новый экземпляр корутины.
- Корутина выполняется до `delay`.
- Управление возвращается Kotlin, который ожидает 1 секунду, как было запрошено.
- Корутина возобновляет работу после `delay` и выполняется до `sus`.
- Управление снова возвращается Kotlin, но на этот раз пауза не требуется.
- Корутина возобновляет работу после `sus` и завершается.

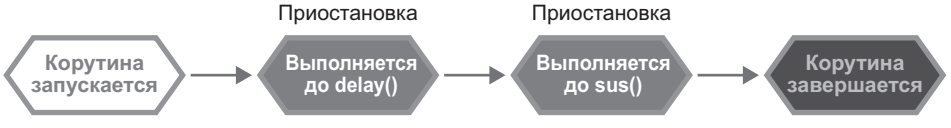


Рис. 16.4. Конечный автомат корутины

Разбиение на этапы помогает понять, почему корутины иногда описываются термином «кооперативная многозадачность». Код между вызовами приостанавливаемых функций выполняется синхронно, а точки `suspend` предоставляют единственную возможность приостановить работу и выполнить другие шаги корутины. Представьте, что между `delay` и `sus` программа входит в бесконечный цикл. Этот цикл навсегда заблокирует поток, в котором он выполняется.

Конечный автомат — не просто абстракция: Kotlin напрямую генерирует код, который все это делает, и его можно просмотреть. Чтобы понять, как происходит преобразование, давайте присмотримся к выводу предыдущей функции. (Ради компактности и ясности мы опустим некоторые второстепенные подробности.)

Результаты компиляции нашего приложения содержат больше файлов классов, чем можно было бы ожидать. Например, вот содержимое каталога build системы Gradle:

https://t.me/javalib

```
build
└── classes
    └── kotlin
        └── main
            └── com
                └── wgjd
                    ├── MainKt$main$1.class
                    └── MainKt.class
```

Файл MainKt.class уже встречался в главе 9. Kotlin прозрачно создает класс, в котором будут храниться функции верхнего уровня, потому что JVM сама по себе не поддерживает свободные функции: код всех методов должен размещаться в некотором классе.

Однако здесь появился еще один новый класс — MainKt$main$1.class. Дизассемблирование этого файла класса показывает, что Kotlin создал «секретные» классы для нашей корутины подобно тому, как он сделал это для функций верхнего уровня. Сгенерированный класс представляет одно выполнение нашей корутины. Как мы вскоре увидим, в этом классе наш код смешивается со служебным кодом, чтобы выполнить операции, которые мы записали в виде корутины:

```
final class com.wgjd.MainKt$main$1
    extends kotlin.coroutines.jvm.internal.SuspendLambda
    implements kotlin.jvm.functions.Function2<
        kotlinx.coroutines.CoroutineScope,
        kotlin.coroutines.Continuation<? super kotlin.Unit>,
        java.lang.Object> {
```

Сгенерированный класс получает функциональность от класса SuspendLambda языка Kotlin

Сгенерированный класс реализует конкретный интерфейс, который будет использоваться вызывающим кодом, чтобы вызвать нашу корутину

Если посмотреть на байт-код, который сгенерировался для нашей функции `main` в MainKt.class, можно увидеть, что Kotlin создает экземпляр нашей корутины, а затем запускает его:

```
Compiled from "Main.kt"
public final class com.wgjd.MainKt {

  public static final void main();
    Code:
       0: getstatic     #41  // Field
                             // kotlinx/coroutines/GlobalScope.INSTANCE:
                             // Lkotlinx/coroutines/GlobalScope;
       3: checkcast     #43  // class kotlinx/coroutines/CoroutineScope
       6: aconst_null
       7: aconst_null
       8: new           #45  // class com/wgjd/
                             // MainKt$main$1
      11: dup
      12: aconst_null
      13: invokespecial #49  // Method com/wgjd/MainKt$main$1."<init>":
                             // (Lkotlin/coroutines/Continuation;)V
```

Получает экземпляр GlobalScope, чтобы начать работать с корутиной

Создает и инициализирует новый экземпляр сгенерированного класса корутины

https://t.me/javalib

```
      16: checkcast     #51  // class kotlin/jvm/functions/Function2
      19: iconst_3
      20: aconst_null
      21: invokestatic  #57  // Method kotlinx/coroutines/
                             // BuildersKt.launch$
                             // default:(Lkotlinx/coroutines/CoroutineScope;
                             // Lkotlin/coroutines/CoroutineContext;
                             // Lkotlinx/coroutines/CoroutineStart;
                             // Lkotlin/jvm/functions/Function2;
                             // ILjava/lang/Object;)Lkotlinx/coroutines/Job;
      24: pop
      25: ldc2_w        #58  // long 2000l
      28: invokestatic  #65  // Method java/lang/Thread.sleep:(J)V
      31: return
```

Вызывает метод launch, передавая ему область видимости и экземпляр корутины (он передается как параметр Function2)

Затем код метода `launch` начинает вызывать методы для сгенерированного экземпляра корутины, выполняя конечный автомат. Рассмотрим байт-код, который его реализует. Прежде всего, экземпляр корутины содержит два разных поля, в которых хранится область видимости корутины и текущая позиция конечного автомата:

```
final class com.wgjd.MainKt$main$1
    extends kotlin.coroutines.jvm.internal.SuspendLambda
    implements kotlin.jvm.functions.Function2<
        kotlinx.coroutines.CoroutineScope,
        kotlin.coroutines.Continuation<? super kotlin.Unit>,
        java.lang.Object> {

  java.lang.Object L$0;
  int label;
```

CoroutineScope для текущего выполнения

Целое число, представляющее текущий шаг конечного автомата

Главный механизм работы конечного автомата — метод, который выбирает следующий шаг на основании этих полей. Именно для этой цели Kotlin генерирует метод `invokeSuspend`, который содержит смесь нашего кода и кода конечного автомата, отслеживающего ход выполнения. В течение всего времени жизни корутины Kotlin повторно вызывает `invokeSuspend` каждый раз, когда корутина готова к следующему шагу выполнения.

В следующем фрагменте кода представлено начало `invokeSuspend` с первым шагом конечного автомата (от начала корутины до вызова `delay`):

```
public final java.lang.Object invokeSuspend(java.lang.Object);
    Code:
       0: invokestatic  #36  // Method kotlin/coroutines/intrinsics/
                             // IntrinsicsKt.getCOROUTINE_SUSPENDED:
                             // ()Ljava/lang/Object;
       3: astore_3
       4: aload_0
       5: getfield      #40  // Field label:I
       8: tableswitch   {    // 0 to 2
                     0: 36
                     1: 69
                     2: 104
               default: 122
```

Определяет следующий шаг конечного автомата

https://t.me/javalib

```
    }
36: aload_1   <-- Начало первого шага (до вызова delay)
37: invokestatic  #46  // Method kotlin/ResultKt.throwOnFailure:
                       // (Ljava/lang/Object;)V
40: aload_0
41: getfield      #48  // Field p$:Lkotlinx/coroutines/CoroutineScope;
44: astore_2
45: ldc2_w        #49  // long 1000l
48: aload_0
49: aload_0
50: aload_2
51: putfield      #52  // Field L$0:Ljava/lang/Object;
54: aload_0
55: iconst_1
56: putfield      #40  // Field label:I
59: invokestatic  #58  // Method kotlinx/coroutines/DelayKt.delay:
                       // (JLkotlin/coroutines/Continuation;)
                       // Ljava/lang/Object;
62: dup
63: aload_3
64: if_acmpne     82
67: aload_3
68: areturn
69: aload_0   <-- Начало второго шага (до вызова sus), который
                  произойдет при следующем вызове invokeSuspend
70: getfield      #52  // Field L$0:Ljava/lang/Object;
73: checkcast     #60  // class kotlinx/coroutines/CoroutineScope

// Дальнейшие шаги опущены для экономии места.
// Полный код можно найти в сопроводительных материалах книги
```

После сбора информации о текущей корутине байт 5 загружает номер следующего выполняемого шага из поля `label`. Затем в байте 8 используется код операции `tableswitch`, который нам до сих пор не встречался. Этот код проверяет значение в стеке и выполняет переход в зависимости от значения. Это первый проход `invokeSuspend`, поэтому `label` содержит значение 0, и происходит переход к байту 36. Дальше код выполняется линейно. В байтах 55 и 56 состояние `label` обновляется значением 1, и осуществляется переход к следующему шагу. В байте 59 вызывается `delay`, после чего в байте 68 происходит возврат `areturn` из `invokeSuspend`.

В этой точке управление получает код на Kotlin, от которого теперь зависит, когда выполнять следующий шаг корутины. Когда он определит, что настал подходящий момент, он вызывает `invokeSuspend` для того же экземпляра корутины. Состояние `label` устанавливается в значение 1, и мы переходим вперед, чтобы выполнить код второго шага, — после `delay`, но до вызова `sus`.

И хотя для повседневного использования корутин не обязательно опускаться на уровень байт-кода, понимать этот механизм очень полезно. Основательному разработчику не стоит успокаиваться, когда кажется, будто что-то работает «волшебным образом». В конце концов, это всего лишь код, который выполняется по одной инструкции за раз, и у нас есть инструменты, чтобы в нем разобраться.

https://t.me/javalib

Этот анализ также поможет ответить на некоторые вопросы из раздела 15.3.7, где встречалась функция `yield`, которую мы использовали при определении последовательности Kotlin. В тот момент мы намеренно обошли вопрос о том, как лямбда-выражение, которое определяет последовательность, может «приостановить» свое выполнение. На самом деле при этом используется такой же генерируемый конечный автомат на основе приостанавливаемых функций. Каждый последующий вызов `invokeSuspend` получает следующий элемент из последовательности.

16.3.2. Области видимости и диспетчеризация корутин

Хотя корутины предоставляют другую абстракцию вместо стандартной модели потоков операционной системы, на внутреннем уровне код все еще выполняется в каком-то потоке. За то, как эта работа распределяется и координируется из корутин Kotlin, отвечают *диспетчеры* и области видимости корутин.

Вернемся к нашему примеру и посмотрим, где фактически выполняется каждый шаг нашей корутины:

```
package com.wgjd

import kotlinx.coroutines.GlobalScope
import kotlinx.coroutines.delay
import kotlinx.coroutines.launch

fun main() {
  GlobalScope.launch {
    println("On thread ${Thread.currentThread().name}")
    delay(500)

    println("On thread ${Thread.currentThread().name}")
    delay(500)

    println("On thread ${Thread.currentThread().name}")
  }

  Thread.sleep(2000)
}
```

Результат не детерминирован, но он выглядит примерно так:

```
On thread DefaultDispatcher-worker-1
On thread DefaultDispatcher-worker-2
On thread DefaultDispatcher-worker-1
```

Имя потока содержит два интересных значения: имя диспетчера (`DefaultDispatcher`) и номер используемого потока из доступного пула. Когда мы запрашивали конкретную область видимости (в нашем случае это `GlobalScope`, которая существует на протяжении всего времени работы приложения), при этом также выбирался диспетчер, который определял, как фактически планируется работа.

https://t.me/javalib

Если вы хотите лучше контролировать диспетчеризацию работы, можно не использовать `GlobalScope`, а создать собственный отдельный экземпляр `CoroutineScope`, как показано ниже. Как правило, область видимости связывается с отдельным объектом в системе, у которого есть свой собственный срок жизни. При конструировании нестандартных областей видимости нужно передать контекст, а стандартные фабричные методы в основном называются в соответствии с тем, как они настраивают диспетчеризацию:

```
val context: CoroutineContext = newFixedThreadPoolContext(3, "Новый-пул")

CoroutineScope(context).launch {
    println("В потоке ${Thread.currentThread().name}")
    delay(500)

    println("В потоке ${Thread.currentThread().name}")
    delay(500)

    println("В потоке ${Thread.currentThread().name}")
}
```

Вполне ожидаемый вывод показывает, что корутина выполняется в разных потоках, как и прежде:

```
В потоке Новый-пул-1
В потоке Новый-пул-2
В потоке Новый-пул-1
```

> **ПРИМЕЧАНИЕ** Обратите внимание, что `newFixedThreadPoolContext` и другие сопутствующие функции считаются устаревшими, но на момент написания книги их замена еще была недоступна. За самой актуальной информацией обращайтесь к документации корутин Kotlin по адресу https://kotlin.github.io/kotlinx.coroutines.

Классы контекста не только содержат диспетчер для корутин, но и позволяют предоставить такую дополнительную информацию, как имя корутины (что упрощает отладку в IDE, особенно когда несколько корутин используют один и тот же диспетчер) и обобщенный обработчик ошибок. Элементы добавляются в существующий объект контекста с помощью функции `plus`:

```
val context: CoroutineContext = newFixedThreadPoolContext(3, "Новый-пул")
  .plus(CoroutineExceptionHandler { _, thrown ->
      println(thrown.message + "!!!!") })
  .plus(CoroutineName("Наша корутина"))

CoroutineScope(context).launch {
  throw RuntimeException("Отказ")
}
```

Если возбудится исключение, то корутины не только автоматически отменятся, но и сработает наш обработчик `CoroutineExceptionHandler`, который выведет сообщение «Отказ!!!!». Вероятно, для реальных приложений стоит рассмотреть

https://t.me/javalib

более основательную стратегию обработки ошибок, но имейте в виду, что корутины предоставляют для этого необходимые средства.

На практике нередко бывает, что различные шаги выполняются параллельно, но их результаты нужно собрать перед тем, как продолжать. Внутри корутин такую возможность обеспечивает функция `async`, которая возвращает `Deferred<T>` — по сути, корутину `Job`, с помощью которой тоже можно ожидать значение и получить его:

```
GlobalScope.launch {
    val result: Deferred<Int> = async {
        10;
    }

    println("Получили ${result.await()}")
}
```

Как и следовало ожидать, выводит «Получили 10»

В разделе 9.5 было показано, что когда возникают ошибки, по умолчанию корутины отменяют всю иерархию корутин:

```
val failed = GlobalScope.launch {
    launch { throw RuntimeException("Failing...") }
}

Thread.sleep(2000)
println("Отмена: ${failed.isCancelled}")
```

Дает время, чтобы завершить выполнение

Выводит «Отмена: true»

Там, где это нужно, такое поведение оказывается в высшей степени мощным. Тем не менее иногда оно нежелательно. Если в некоторых дочерних корутинах могут происходить безопасные сбои, их можно упаковать в `supervisorScope`, как показано ниже. Эта конструкция ведет себя как типичная обертка корутины, но без «всплывающей» отмены:

```
val supervised = GlobalScope.launch {
  supervisorScope {
    launch { throw RuntimeException("Failing...") }
  }
}

Thread.sleep(2000)

println("Отмена: ${supervised.isCancelled}")
```

Снова дает время для завершения

Выводит «Отмена: false» — это указывает на то, что супервизор разрешил дочерней корутине отказывать, не отменяя при этом родительскую

С помощью корутины можно реализовать множество сценариев конкурентного выполнения. Тем не менее это далеко не единственный альтернативный инструментарий. Посмотрим, что в этой области может предложить Clojure.

16.4. КОНКУРЕНТНОЕ ВЫПОЛНЕНИЕ В CLOJURE

В основу модели состояния Java заложена идея изменяемости объектов. Как мы убедились в главе 5, это вызывает проблемы с безопасностью в конкурентном коде. Нам приходится выстраивать достаточно сложные стратегии блокировки,

https://t.me/javalib

чтобы, пока один поток изменяет состояние какого-либо объекта, другие потоки не могли увидеть его промежуточное (так называемое *противоречивое*) состояние. Такие стратегии трудно разрабатывать, трудно отлаживать и еще труднее тестировать.

В Clojure используется другой подход, и абстракции параллелизма в каких-то отношениях работают не на таком низком уровне, как в Java. Например, может показаться странным, что пулы потоков находятся под управлением среды выполнения Clojure и у разработчика нет почти никакого контроля над ними. Но поскольку вы фактически разрешаете платформе (в данном случае среде выполнения) выполнять за вас служебные операции, это освобождает вашу голову для более важных задач, например общего проектирования.

В целом философия Clojure заключается в том, чтобы по умолчанию изолировать потоки друг от друга, что в значительной мере обеспечивает типобезопасную конкурентность по умолчанию. Благодаря принципу «ни с кем ничем не делиться» и неизменяемым значениям Clojure преодолевает многие проблемы Java и может сосредоточиться на том, как сделать совместное использование состояния безопасным при конкурентном выполнении.

> **ПРИМЕЧАНИЕ** Чтобы улучшить безопасность, среда выполнения Clojure предоставляет механизмы координации между потоками. Мы настоятельно рекомендуем использовать эти механизмы вместо того, чтобы пытаться применять идиомы Java или создавать собственные конкурентные конструкции.

Рассмотрим первый из важных элементов конкурентного выполнения в Clojure — перманентные структуры данных.

16.4.1. Перманентные структуры данных

Перманентной структурой данных (persistent data structure) называется структура, которая сохраняет предыдущие версии при изменении. Такие структуры обладают потоковой безопасностью, потому что операции с ними не изменяют структуру, которую видят текущие читающие стороны, а всегда возвращают обновленный объект.

Все коллекции Clojure являются перманентными и позволяют эффективно создавать измененные копии с помощью *структурного общего доступа*. Коллекции по своей природе потоково-безопасны и спроектированы с расчетом на эффективность.

Важно заметить, что перманентные коллекции Clojure не позволяют переставлять или удалять элементы на месте. Если ваша программа вызовет соответствующие методы интерфейсов коллекций Java (например, `List` или `Map`), возбудится исключение `UnsupportedOperationException`. Правильный способ создавать перманентные коллекции — использовать такие операции, как `(cons)` и `(conj)`, в соответствии с традициями Lisp.

https://t.me/javalib

Все коллекции поддерживают следующие базовые методы:

- `count` получает размер коллекции.
- `cons` и `conj` добавляют элементы в коллекцию.
- `seq` возвращает последовательность, с помощью которой можно обходить коллекцию.

Таким образом, благодаря `seq` с любой коллекцией можно использовать любые функции последовательностей. Давайте рассмотрим пример с `PersistentVector`, но сначала продемонстрируем этот класс Clojure в коде Java, чтобы показать, как `(cons)` позволяет строить вектор многократным добавлением новых элементов:

```
var aList = new ArrayList<PersistentVector>();
var vector = PersistentVector.EMPTY;
for (int i=0; i < 32;i=i+1){
    vector = vector.cons(i);
    aList.add(vector);
}
System.out.println(aList);
```

Результат выглядит примерно так:

```
[[0], [0 1], [0 1 2], [0 1 2 3],

...

[0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27
 28 29 30 31]]
```

Этот пример показывает, что при необходимости можно хранить все более ранние версии перманентного вектора. Также это означает, что вектор можно передавать между потоками и каждый поток сможет его изменять, не затрагивая другие потоки.

Давайте в общих чертах посмотрим, как реализована эта структура данных. Вспомните, что структуры данных на основе массивов (такие, как векторы) в других языках обычно располагаются в одном непрерывном блоке памяти. Такая реализация позволяет быстро выполнять индексируемые операции (например, доступ по индексу), но если понадобится создать измененную копию вектора (сохранив при этом оригинал), придется скопировать весь исходный массив.

В Clojure структура `PersistentVector` реализуется совершенно иначе. Clojure хранит элементы вектора блоками по 32 элемента. Это позволяет при добавлении элемента копировать только текущий *хвостовой блок* из 32 элементов. Если в вектор добавляется более 32 элементов, то создается структура, которая называется *узлом* и содержит массив из 32 элементов, каждый из которых является ссылкой на узел, содержащий полный массив из 32 элементов.

https://t.me/javalib

ПРИМЕЧАНИЕ `PersistentVector` — одна из основных абстракций Clojure. Она используется настолько широко, что ее реализация написана на Java (а не на Clojure), потому что это необходимо, чтобы инициализировать базовую среду выполнения Clojure.

Определение класса `PersistentVector` выглядит примерно так (слегка упрощено для ясности):

```
public class PersistentVector extends APersistentVector
                              implements IObj, ... {

    // ...

    public final PersistentVector.Node root;
    public final Object[] tail;
    private final int cnt;

    // ...

    public static final PersistentVector EMPTY;
    public static final PersistentVector.Node EMPTY_NODE;

    // ...
}
```

Внутренний класс `PersistentVector.Node` определяется следующим образом:

```
public static class Node implements Serializable {
    public final transient AtomicReference<Thread> edit;
    public final Object[] array;

    public Node(AtomicReference<Thread> edit, Object[] array) {
        this.edit = edit;
        this.array = array;
    }

    Node(AtomicReference<Thread> edit) {
        this.edit = edit;
        this.array = new Object[32];
    }
}
```

Обратите внимание, что поле `array` — это просто `Object[]`. В этом проявляется динамическая типизация Clojure: обобщений здесь нет.

Кроме того, в ядре Clojure часто используются поля `public final` и не всегда определяются методы доступа. Это дает нам возможность заглянуть внутрь структуры данных и посмотреть, как работают узлы, например:

```
var vector = PersistentVector.EMPTY;
for (int i = 0;i<32;i=i+1){
    vector = vector.cons(i);
```

https://t.me/javalib

```
}

System.out.println(Arrays.toString(vector.tail));
System.out.println(Arrays.toString(vector.root.array));
System.out.println("----------------");
for (int i=32; i < 64;i=i+1){
    vector = vector.cons(i);
}

System.out.println(Arrays.toString(vector.tail));
System.out.println(Arrays.toString(vector.root.array));
var earlier = (PersistentVector.Node)(vector.root.array[0]);
System.out.println("Хвостовая часть: "+ Arrays.toString(earlier.array));
```

Результат выглядит так:

```
[0, 1, 2, 3, ... 31]
[null, null, null, null, ... null]
----------------
[32, 33, 34, 35, ... 63]
[clojure.lang.PersistentVector$Node@783e6358, null, null, null, ... null]
Хвостовая часть: [0, 1, 2, 3, ... 31]
```

После добавления 64 элементов `tail` содержит `[32, … 63]`, а `root.array` содержит один узел `PersistentVector.Node`, в поле `array` которой хранятся элементы `[0, … 31]`.

На рис. 16.5 показано, как выглядит вектор, в котором от 0 до 32 элементов.

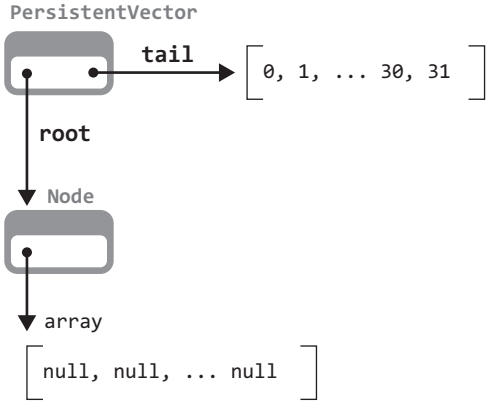


Рис. 16.5. Перманентный вектор с 32 элементами

Если количество элементов больше 32, но меньше 64, то структура выглядит так, как на рис. 16.6.

Если вектор содержит от 64 до 96 элементов, он выглядит так, как показано на рис. 16.7, и т. д.

https://t.me/javalib

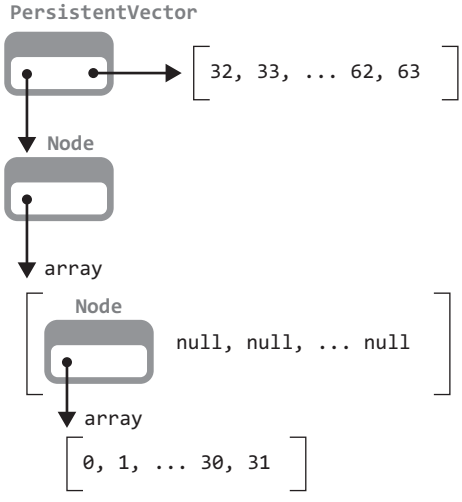


Рис. 16.6. Перманентный вектор с 64 элементами

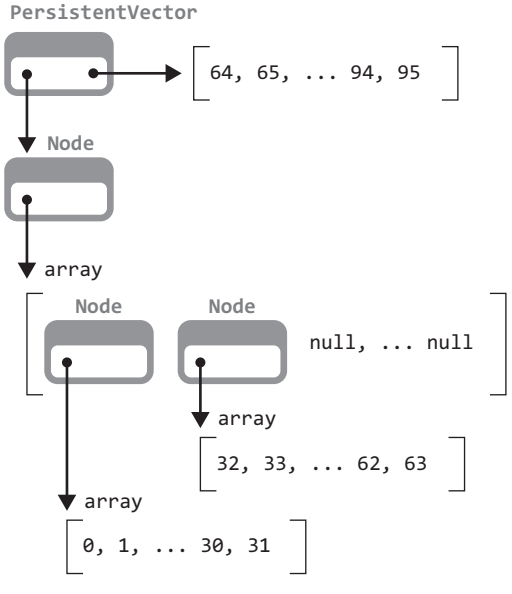


Рис. 16.7. Перманентный вектор с 96 элементами

Возникает резонный вопрос: что случится, когда все элементы массива узла будут заполнены? Это происходит, когда вектор содержит 32 + (32 × 32) = = 1056 элементов. Возможно, вы ожидали увидеть число 1024, но не забывайте, что еще есть хвостовая часть, содержащая 32 элемента (это типичный эффект «смещения на единицу»). Целиком заполненный `PersistentVector` с одним уровнем `Node` изображен на рис. 16.8.

https://t.me/javalib

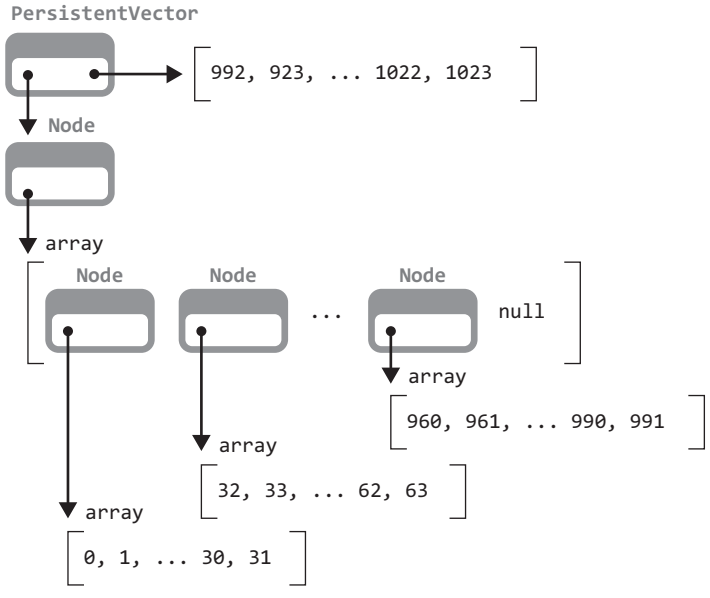


Рис. 16.8. Перманентный вектор с 1056 элементами

Если продолжать добавлять элементы, структура дерева вырастет еще на один уровень, как показано на рис. 16.9.

Чтобы понаблюдать за происходящим на программном уровне, можно запустить такой код:

```
var vector = PersistentVector.EMPTY;
for (int i = 0; i < 1088; i = i + 1) {
    vector = vector.cons(i);
}
System.out.println(Arrays.toString(vector.tail));
System.out.println(Arrays.toString(vector.root.array));
System.out.println();

var v0 = (PersistentVector.Node) (vector.root.array[0]);
var v1 = (PersistentVector.Node) (vector.root.array[1]);
System.out.println("r.a[0]:"+ Arrays.toString(v0.array));
System.out.println("r.a[1]:"+ Arrays.toString(v1.array));
System.out.println();

var v0A0 = (PersistentVector.Node)(((PersistentVector.Node)v0).array[0]);
var v0A31 = (PersistentVector.Node)(((PersistentVector.Node)v0).array[31]);
var v1A0 = (PersistentVector.Node)(((PersistentVector.Node)v1).array[0]);
System.out.println("r.a[0].a[0] : " + Arrays.toString(v0A0.array));
System.out.println("r.a[0].a[31] : " + Arrays.toString(v0A31.array));
System.out.println("r.a[1].array[0]:"+ Arrays.toString(v1A0.array));
```

Результат выглядит так:

```
[1056, 1057, 1058, 1059, ... 1087]
[clojure.lang.PersistentVector$Node@2344fc66,
```

https://t.me/javalib

```
clojure.lang.PersistentVector$Node@458ad742, null, null, ... null]

r.a[0] : [clojure.lang.PersistentVector$Node@735f7ae5,
          clojure.lang.PersistentVector$Node@180bc464, ... ,
          clojure.lang.PersistentVector$Node@617c74e5]
r.a[1] : [clojure.lang.PersistentVector$Node@6ea12c19, null, ... , null]

r.a[0].a[0] : [0, 1, ... , 31]
r.a[0].a[31] : [992, 993, ... , 1023]
r.a[1].a[0] : [1024, 1025, ... , 1055]
```

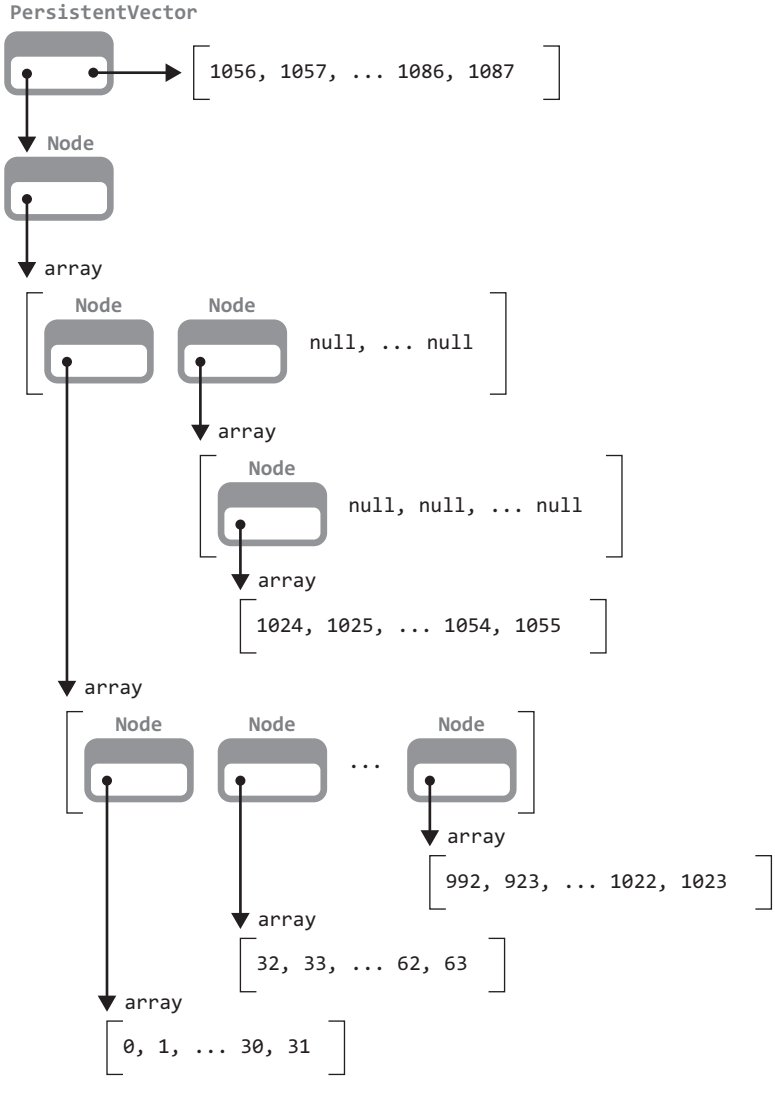


Рис. 16.9. Перманентный вектор со множеством элементов

https://t.me/javalib

Обратите внимание, что в этом примере поле `array` объектов `v0` и `v1` вместо элементов `Integer` содержит объекты `Node`. Поле `array` объявлено с типом `Object[]` именно для того, чтобы поддерживать подобную динамическую типизацию.

> **ПРИМЕЧАНИЕ** Если мы продолжим добавлять элементы, то в структуре будут появляться все новые и новые уровни. Таким образом `PersistentVector` позволяет обрабатывать векторы произвольного размера (хотя и с возрастающими затратами на разрешение ссылок для больших векторов).

Тем не менее, чтобы язык конкурентного программирования был успешным, он не должен ограничиваться структурами данных. Например, абсолютно необходимо понятие моделей конкурентного выполнения, и к счастью, в Clojure оно есть!

На самом деле в Clojure есть несколько методов, которые предоставляют разные модели конкурентного выполнения: преднамеченные объекты и `pcalls`, ссылки (`ref`) и агенты. Рассмотрим их поочередно, начиная с более простых.

16.4.2. Преднамеченные объекты и pcalls

Прежде всего подчеркнем, что для запуска новых потоков всегда можно воспользоваться тем, что Clojure тесно связан с Java. Все, что осуществимо на Java, также осуществимо на Clojure, и на Clojure очень легко пишется конкурентный код Java.

Тем не менее в Clojure есть улучшенные формы для некоторых абстракций Java. Например, Clojure реализует очень чистый подход к понятию преднамеченных объектов, о которых в контексте Java говорилось в главе 6. Простой пример приведен в следующем листинге.

Листинг 16.2. Преднамеченные объекты в Clojure

```
user=> (def simple-future
    (future (do
        (println "Строка 0")
        (Thread/sleep 10000)
        (println "Строка 1")
        (Thread/sleep 10000)
        (println "Строка 2"))))

#'user/simple-future
Строка 0  <-- Начинает выполняться немедленно
user=> (future-done? simple-future)
user=> false
Строка 1
user=> @simple-future  <-- Блокируется при разыменовании
Строка 2
nil
user=>
```

В этом листинге преднамеченный объект создается формой `(future)`. Сразу же после создания он начинает выполняться в фоновом потоке, и поэтому в Clojure

https://t.me/javalib

REPL мы видим вывод `Строка 0` (и позднее `Строка 1`) — код начал выполняться в другом потоке.

Затем можно проверить, завершился ли код, с помощью `(future-done?)` — это неблокирующий вызов (как `isDone()` в Java). Однако если попытаться разыменовать преднамеченный объект, это приведет к тому, что вызывающий поток блокируется до завершения функции.

Фактически это тонкая обертка Clojure над объектом Java `Future`, с чуть более чистым синтаксисом. Clojure также предоставляет вспомогательные формы, которые могут быть очень полезны разработчикам конкурентных приложений. В частности, к таким формам относится простая функция `(pcalls)`, которая получает переменное количество функций с нулем аргументов и выполняет их параллельно.

> **ПРИМЕЧАНИЕ** `(pcalls)` отчасти напоминает вспомогательный метод `ExecutorService.invokeAll()` в Java.

Вызовы выполняются в пуле потоков, которым управляет среда выполнения, и возвращают отложенную последовательность с результатами. Если попытаться обратиться к еще не завершенным элементам последовательности, то обращающийся поток заблокируется.

В листинге 16.3 создается функция `(wait-with-for)`, которая принимает один аргумент. Она построена в форме цикла подобно примеру из раздела 10.2.5. С помощью этой функции можно создать ряд функций с нулем аргументов — `(wait-1)`, `(wait-2)` и т. д., которые затем можно передать `(pcalls)`.

Листинг 16.3. Параллельные вызовы в Clojure

```
user=> (defn wait-with-for [limit]
  (let [counter 1]
    (loop [ctr counter]
      (Thread/sleep 500)
      (println (str "Ctr=" ctr))
    (if (< ctr limit)
      (recur (inc ctr))
    ctr))))
#'user/wait-with-for

user=> (defn wait-1 [] (wait-with-for 1))
#'user/wait-1

user=> (defn wait-2 [] (wait-with-for 2))
#'user/wait-2

user=> (defn wait-3 [] (wait-with-for 3))
#'user/wait-3

user=> (def wait-seq (pcalls wait-1 wait-2 wait-3))
#'user/wait-seq
Ctr=1
```

https://t.me/javalib

```
Ctr=1
Ctr=1
Ctr=2
Ctr=2
Ctr=3

user=> (first wait-seq)
1

user=> (first (next wait-seq))
2
```

Если поток приостанавливается всего на 500 миллисекунд, функции `wait` завершаются очень быстро. Поэкспериментировав с тайм-аутом (например, увеличив его до 10 секунд), легко убедиться в том, что отложенная последовательность `wait-seq`, которую возвращает `(pcalls)`, обладает описанным выше блокирующим поведением.

Такой способ доступа к простым многопоточным конструкциям хорошо подходит для случая, когда вам не нужно совместно использовать состояние, но во многих приложениях потоки должны взаимодействовать в процессе работы. В Clojure существуют модели для таких ситуаций, и в следующем разделе рассматривается одна из них — совместно используемое состояние, которое включается формой `(ref)`, обрабатывающейся в транзакции.

16.4.3. Программная транзакционная память

Первый (и самый очевидный) способ совместно использовать состояние — не делать этого. Собственно, конструкция `var`, которой мы пользовались до сих пор, на самом деле не допускает совместного использования. Если два разных потока унаследуют одно имя для `var` и переприсвоят ей значение внутри потока, эти значения будут видны только внутри этих отдельных потоков, а другие потоки не смогут к ним обратиться.

Создатели Clojure приняли такое решение намеренно, но предоставили альтернативный способ совместно использовать состояние между потоками — ссылки `(ref)`. Этот механизм полагается на модель, которую среда выполнения предоставляет для изменений состояния, которые должны быть видны в нескольких потоках. Эта модель фактически вводит дополнительный уровень косвенности между символическим именем и значением, то есть символическое имя связывается не напрямую со значением, а со ссылкой на значение.

Такая система по своей сути является транзакционной, а изменения в используемом значении координирует среда выполнения Clojure. Это иллюстрируется на рис. 16.10.

Эта косвенность означает, что перед тем, как ссылку можно будет изменить, ее нужно поместить в транзакцию. Когда транзакция завершается, все обновления либо вступают в силу, либо отменяются — это очевидная аналогия с транзакциями в базах данных.

https://t.me/javalib

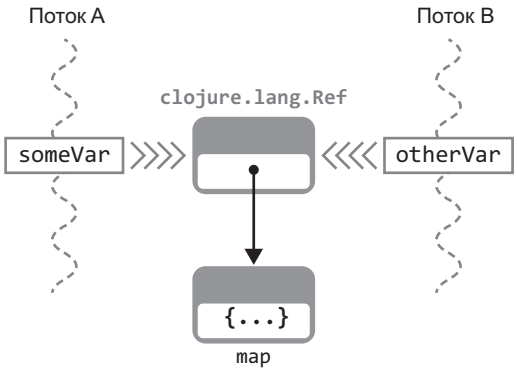


Рис. 16.10. Программная транзакционная память

Описание выглядит немного абстрактно, поэтому вернемся к примеру с классом `Account`, который рассматривался в главах 5 и 6. Вспомните, что для предотвращения таких проблем конкурентности, как потерянные обновления, в Java приходилось защищать блокировками все потенциально ненадежные данные:

```
// ...

private final Lock lock = new ReentrantLock();
private int balance;

public boolean withdraw(final int amount) {
    // Пропущена часть кода: проверяем, что amount > 0,
    // и если нет, возбуждаем исключение

    lock.lock();
    try {
        if (balance >= amount) {
            balance = balance - amount;
            return true;
        }
    } finally {
        lock.unlock();
    }
    return false;
}

// ...
```

Если мы попробуем написать что-то подобное на Clojure, то столкнемся со своего рода концептуальной проблемой.

В Java по умолчанию используется изменяемое состояние, и это проявляется в приведенном выше коде. Метод `withdraw()` получает один параметр `amount`, и происходит один из трех результатов:

- Значение `amount` меньше либо равно нулю — возбуждается исключение `IllegalArgumentException`, потому что это некорректная сумма для списания.

https://t.me/javalib

- Списание проходит успешно — баланс обновляется, возвращается `true`.
- Списание завершается неудачей (недостаточно средств) — баланс не обновляется, возвращается `false`.

Если не учитывать некорректный случай, здесь происходят две разные операции: обновляется изменяемое состояние, а через возвращаемое значение передается информация об успехе или неудаче операции.

В функциональном программировании мы бы не обновляли изменяемое состояние, а возвращали новое значение с обновленным состоянием. Но если списание завершается неудачей, как пользователь кода узнает, был ли баланс обновлен? Можно представить себе решение, в котором возвращаемый код заменяется парой возвращаемых кодов и, возможно, обновленным значением, но это выглядит довольно неуклюже.

Вместо этого начнем с несколько иной однопоточной версии, которая приводится в следующем листинге. Здесь форма `(debit)` работает с отображением, которое представляет банковский счет, и возвращает новое отображение, если списание прошло успешно (а если нет — возбуждает исключение).

Листинг 16.4. Простая модель банковского счета в Clojure

```
(defn make-new-acc [account-name opening-balance]
  {:name account-name :bal opening-balance})

(defn debit [account amount]
  (let [balance (:bal account) my-name (:name account)]
    (if (<= amount 0)
      (throw (AssertionError. "Withdrawal amount cannot be < 0")))
    (if (> balance amount)
      (make-new-acc my-name (- balance amount))
      (throw (AssertionError. "Withdrawal amount cannot exceed balance"))
    )))

(debit (make-new-acc "Ben" 5000) 1000)
```

Обратите внимание, насколько этот код короче, чем версия на Java, — правда, он все еще однопоточный. Запуск кода приведет к ожидаемому результату: вы получите отображение с балансом 4000.

Несмотря на относительную простоту, такое решение нельзя считать полностью удовлетворительным, ведь на самом деле мы решаем другую проблему с другой семантикой. Посмотрим, не улучшится ли ситуация, если мы обобщим код в конкурентную версию.

Чтобы сделать код конкурентным, нам понадобятся ссылки Clojure. Они создаются формой `(ref)` и представляют собой объекты JVM с типом `clojure.lang.Ref`. Обычно ссылки указывают на отображение Clojure, в котором хранится состояние.

https://t.me/javalib

Также нам понадобится форма (dosync), которая создает транзакцию. Внутри транзакции мы используем форму (alter), которая позволяет изменить содержимое ссылки. В следующем листинге показано, как использовать ссылки в многопоточной реализации нашей конструкции банковского счета.

Листинг 16.5. Многопоточная обработка счетов

```
user=> (defn safe-debit [ref-account amount]
  (dosync
    (alter ref-account debit amount)
    ref-account))
#'user/safe-debit

user=> (def my-acc (make-new-acc "Ben" 5000))
#'user/my-acc

user=> (def r-my-acc (ref my-acc))
#'user/r-my-acc

user=> (safe-debit r-my-acc 1000)
#object[clojure.lang.Ref 0x6b1e7ad3 {:status :ready,
                                     :val {:name "Ben", :bal 4000}}]
```

Как говорилось выше, форма (alter) воздействует на ссылку, применяя к ней функцию с аргументами. Значение, с которым выполняется действие, видно этому потоку во время транзакции — это называется *внутритранзакционным значением*. Возвращаемым значением становится новое значение ссылки после возвращения из (alter). Это значение невидимо за пределами потока, в котором выполняется (alter), вплоть до выхода из блока транзакции, определенного (dosync).

Другие транзакции могут выполняться одновременно с этой. В таком случае программная транзакционная память (STM) Clojure отслеживает их и разрешает зафиксировать транзакцию, только если она не противоречит другим транзакциям, которые были зафиксированы с момента ее запуска. Если транзакция окажется противоречивой, Clojure ее отменит и может попытаться повторить с обновленным представлением системы.

Такие повторные попытки могут создать проблемы, если транзакция вызывает побочные эффекты (например, запись в файл журнала или другой вывод). Вам придется заботиться о том, чтобы транзакционные части были как можно проще и чище в смысле функционального программирования (то есть без побочных эффектов).

В многопоточных конфигурациях такое оптимистичное транзакционное поведение иногда «утяжеляет» всю систему. В некоторых конкурентных приложениях взаимодействия между потоками требуются относительно редко и происходят скорее в асимметричном режиме. К счастью, Clojure предоставляет еще один механизм конкурентного выполнения, который требует намного меньше участия со стороны пользователя. Этот механизм рассматривается в следующем разделе.

https://t.me/javalib

16.4.4. Агенты

Агенты — альтернативный примитив конкурентного выполнения в Clojure. Агент Clojure не обращается к совместно используемому состоянию, а представляет собой асинхронный исполняемый объект, ориентированный на сообщения. Это отчасти напоминает понятие *акторов* в других языках (таких, как Scala и Erlang).

Агент — это контекст исполнения, который может принимать сообщения (в форме функций), отправленные ему из другого (или того же) потока. Новые агенты объявляются функцией `(agent)`, а сообщения им можно отправлять с помощью `(send)`.

> «Придется отправлять их с посыльным, — думала она. — Вот будет смешно! Подарки собственным ногам! И адрес какой странный!»
>
> *Льюис Кэрролл*

Сами агенты — это не потоки, а исполняемые объекты, которые «меньше» потока. Они планируются в пуле потоков, которым управляет среда выполнения Clojure (пул обычно недоступен программисту напрямую).

> **ПРИМЕЧАНИЕ** Агенты Clojure потенциально имеют продолжительный срок жизни, в отличие от короткоживущих объектов задач в пулах потоков Java.

Среда выполнения также гарантирует, что значения агента, которые видны снаружи, будут изолированными и атомарными. Это означает, что пользовательский код будет видеть значение агента только в состоянии «до» или «после».

> **ПРИМЕЧАНИЕ** Еще один пример исполняемых объектов, меньших потока, будет представлен в главе 18.

В следующем листинге приведен простой пример использования агентов; он немного напоминает код, с помощью которого ранее демонстрировались преднамеченные объекты.

Листинг 16.6. Агенты в Clojure

```
user=> (defn wait-and-log [coll str-to-add]
  (do (Thread/sleep 10000)
    (let [my-coll (conj coll str-to-add)]
      (Thread/sleep 10000)
      (conj my-coll str-to-add))))
#'user/wait-and-log

user=> (def str-coll (agent []))
#'user/str-coll

user=> (send str-coll wait-and-log "foo")
#object[clojure.lang.Agent 0x38499e48 {:status :ready, :val []}]

user=> @str-coll
[]
```

https://t.me/javalib

```
// Сделать паузу, чтобы программа успела обработать сообщение

user=> @str-coll
["foo" "foo"]
```

Вызов (`send`) пересылает агенту вызов (`wait-and-log`). Если разыменовать его с помощью REPL, можно убедиться, что вы никогда не видите промежуточное состояние агента: появляется только итоговое состояние (где строка `"foo"` фигурирует дважды).

Может показаться странным, что в решении с агентом мы отправляем сообщение агенту, который запланирован в потоке из пула под управлением Clojure, когда оба потока уже совместно используют адресное пространство. Но здесь проявляется один из принципов конкурентного выполнения, который уже встречался нам несколько раз: дополнительная абстракция может быть полезна, если она делает код проще и нагляднее.

Нигде эта синергия не проявляется так очевидно, как в механизме Clojure, который делегирует среде выполнения многие низкоуровневые аспекты многопоточности и управления конкурентным выполнением. Это освобождает программиста от рутины и позволяет ему сосредоточиться на качественном многопоточном проектировании и высокоуровневых концепциях. Здесь можно провести аналогию со средствами сборки мусора Java, которые освобождают вас от технических подробностей управления памятью.

ИТОГИ

- Каждый язык по-своему расширяет основные принципы выполнения.
- В Java библиотека Fork/Join позволяет декомпозировать задачи и перехватывать работу; в Kotlin применяются корутины, реализованные благодаря продвинутым приемам компиляции; а в Clojure есть разновидность модели акторов, воплощенная в форме агентов.
- Работа с состоянием играет ключевую роль в конкурентном программировании и по-разному реализована в этих трех языках.
- В Java по умолчанию применяется концепция изменяемости с такими усовершенствованиями, как `CompletableFuture`.
- Kotlin уделяет неизменяемости больше внимания, но при этом на фундаментальном уровне опирается на то же представление общего, изменяемого состояния.
- Clojure ставит на первое место неизменяемость на основе программной транзакционной памяти, но за это приходится расплачиваться менее знакомой моделью программирования и ослабленной интеграцией с коллекциями Java.

https://t.me/javalib

17 Внутреннее устройство современной JVM

В ЭТОЙ ГЛАВЕ

- ✓ Знакомство с внутренним устройством JVM
- ✓ Внутренняя реализация рефлексии
- ✓ Дескрипторы методов
- ✓ `invokedynamic`
- ✓ Недавние изменения во внутренней реализации
- ✓ `Unsafe`

Виртуальная машина Java (JVM) — чрезвычайно сложная среда выполнения, которая десятилетиями ставила на первое место стабильность и технические решения коммерческого уровня. В результате многим разработчикам никогда не приходилось углубляться во внутреннее устройство JVM, потому что в большинстве случаев это просто не нужно.

Тем не менее эта глава написана для любознательных читателей, которые хотят узнать больше, заглянуть за кулисы и разобраться в подробностях того, как реализована JVM. Начнем с вызова методов.

https://t.me/javalib

17.1. ЗНАКОМСТВО С ВНУТРЕННИМ УСТРОЙСТВОМ JVM: ВЫЗОВ МЕТОДОВ

Для начала рассмотрим простой пример с классами `Pet`, `Cat` и `Bear` и интерфейсом `Furry`. Их иерархия изображена на рис. 17.1.

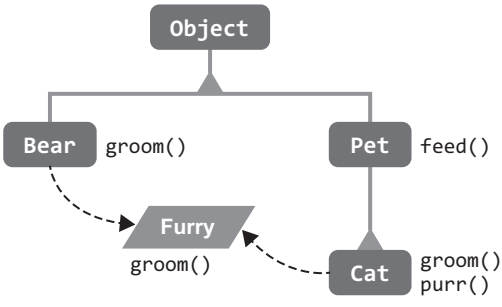


Рис. 17.1. Простая иерархия наследования

Также допустим, что существуют другие подклассы `Pet` (например, `Dog` и `Fish`), которые для простоты не представлены на диаграмме. На этом примере мы подробно объясним, как работают разные коды операций вызова, начиная с `invokevirtual`.

17.1.1. Вызов виртуальных методов

Самая распространенная разновидность вызова методов — вызов метода экземпляра для объекта конкретного класса (или подкласса) с помощью байт-кода `invokevirtual`. Этот механизм вызова также называется *диспетчеризацией виртуального метода* (или просто виртуальной диспетчеризацией): какой конкретно метод будет вызван — это определяется на стадии выполнения (а не на стадии компиляции) в зависимости от фактического типа объекта во время выполнения. Когда JVM выполняет следующий фрагмент кода:

```
Pet p = getPet();
p.feed();
```

только в той точке, где метод должен выполняться, среда выполнения определяет, какую именно реализацию `feed()` вызывать.

Реализации могут различаться в зависимости от того, содержит ли переменная `p` экземпляр `Cat` или `Dog` (или `Pet`, если этот надкласс не является абстрактным). Также возможно, что в разные моменты выполнения программы `getPet()` возвращает объекты разных подтипов `Pet`. Это неважно, потому что среда будет выбирать вызываемую реализацию каждый раз, когда метод должен выполняться. Собственно, это описание соответствует тому, как методы Java работали всегда, даже когда вы только начинали изучать язык.

https://t.me/javalib

Чтобы эта схема функционировала, во внутренней реализации JVM хранит так называемую *v-таблицу* (для каждого класса) с определениями методов, соответствующих этому типу (в C++ аналогичная структура обычно называется таблицей виртуальных функций). Эта таблица размещается в специальной области памяти JVM, которая называется *метапространством* и где хранятся все метаданные, необходимые виртуальной машине.

> **ПРИМЕЧАНИЕ** В Java 7 и ранее эти метаданные размещались в области кучи Java, которая называлась «постоянным поколением» (permgen).

Чтобы разобраться, как используется v-таблица, необходимо кратко рассмотреть метаданные JVM для класса. В Java все объекты существуют в куче Java, а операции с ними выполняются только по ссылке.

В HotSpot различные внутренние структуры данных, которые находятся в куче, обозначаются общим термином *oop* (сокращение от «ordinary object pointer», «указатель на обычный объект»).

У каждого объекта Java должен быть *заголовок объекта*, который содержит две разновидности метаданных:

- Метаданные, которые относятся к конкретному экземпляру класса («mark-слово»).
- Метаданные, общие для всех экземпляров класса («klass-слово»).

Для экономии места хранится только одна копия метаданных уровня класса, а каждый объект, принадлежащий классу, содержит ссылку на нее — *klass-слово*. На рис. 17.2 изображена ссылка Java, которая хранится в локальной переменной и указывает на начало заголовка объекта Java в куче.

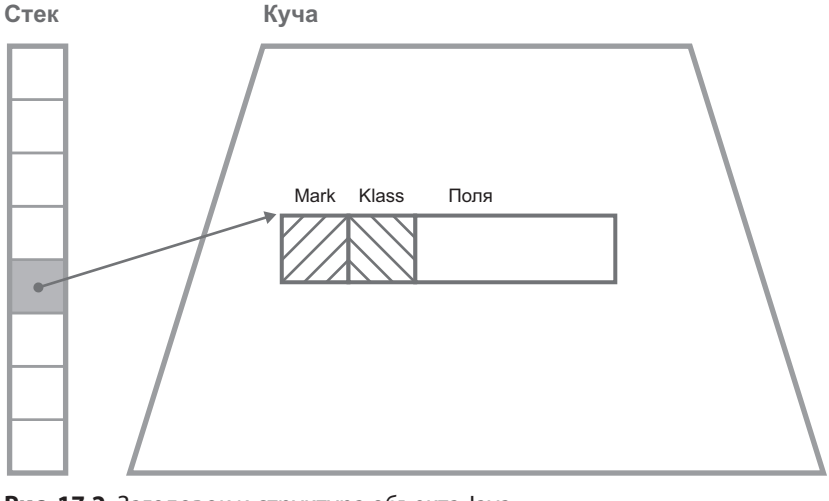


Рис. 17.2. Заголовок и структура объекта Java

https://t.me/javalib

Термином «klass» обозначается внутреннее представление класса Java в JVM во время выполнения, которое хранится в метапространстве. Это представление содержит всю информацию, которая необходима JVM, чтобы работать с классом во время выполнения (например, определения методов и структуру полей).

Часть информации из klass доступна Java-программисту через объект `Class<?>`, соответствующий текущему типу, но klass и `Class` — совершенно разные вещи. В частности, представление klass содержит информацию, которая специально сделана недоступной для обычного кода приложения.

> **ПРИМЕЧАНИЕ** Вариант написания «klass» был выбран намеренно, потому что он позволяет отличить внутреннюю структуру данных от других понятий, которые обозначаются словом «class» в письменной документации, но, к сожалению, не в устной речи. В коде вам также могут встретиться слова «clazz» и «clz» — обычно так называются переменные Java, которые содержат объект `Class`.

Виртуальную диспетчеризацию, которую реализует байт-код `invokevirtual`, можно объяснить в понятиях внутренних структур JVM, таких как klass и прежде всего его v-таблица. Когда JVM встречает инструкцию `invokevirtual` для выполнения, она извлекает объект-получатель и любые аргументы для метода из стека выполнения текущего метода.

> **ПРИМЕЧАНИЕ** Получатель (receiver object) — это объект, для которого вызывается метод экземпляра.

Структура заголовка объекта JVM начинается с mark-слова, за которым немедленно следует klass-слово. Таким образом, чтобы найти метод для выполнения, JVM переходит по указателю в метапространство, где она по v-таблице из klass определяет, какой именно код нужно выполнять. Этот процесс изображен на рис. 17.3.

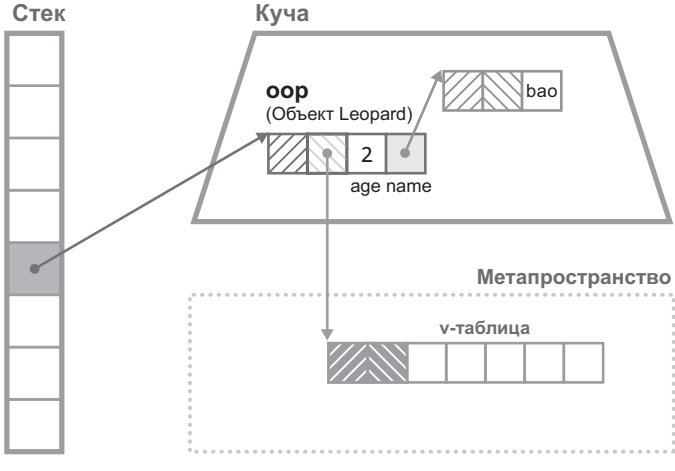


Рис. 17.3. Поиск реализации метода

https://t.me/javalib

Если в klass нет определения для метода, то JVM переходит по указателю к klass, который соответствует непосредственному надклассу, и делает повторную попытку. На этом процессе основана перегрузка методов в JVM.

Чтобы v-таблицы работали эффективно, они имеют специальную структуру. В v-таблице для каждого klass первыми идут методы, которые определены родительским типом, и они размещаются точно в таком же порядке, как в v-таблице родительского типа. Методы, которые специфичны для этого типа и не объявлены родительским типом, размещаются в конце v-таблицы.

Когда подкласс переопределяет метод, он сохраняется в v-таблице с таким же смещением, как у переопределяемой реализации. В результате поиск переопределенных методов становится полностью тривиальным, потому что их смещение в v-таблице будет таким же, как в таблице родителя. На рис. 17.4 изображена структура v-таблицы для некоторых классов из нашего примера.

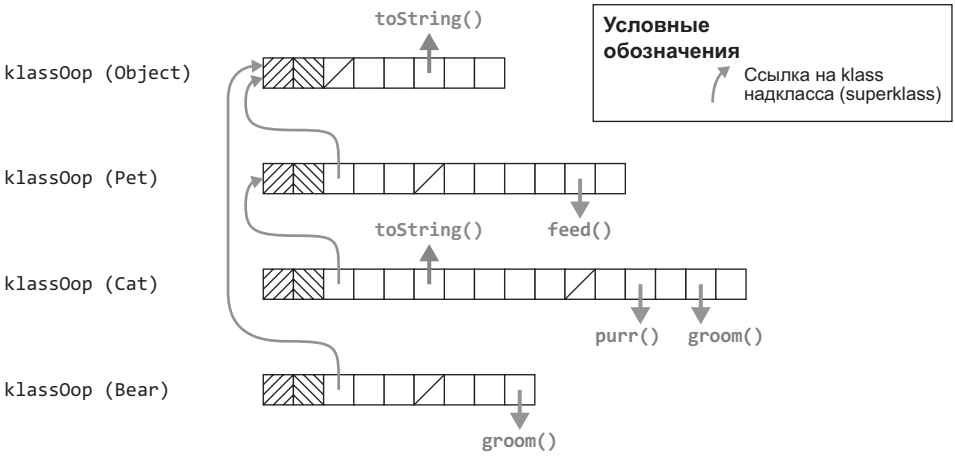


Рис. 17.4. Структура v-таблицы

Таким образом, если вызвать `Cat::feed`, JVM не найдет переопределение в классе `Cat` и проследует по указателю на надкласс в klass для `Pet`. В нем есть реализация `feed()`, поэтому именно этот код будет вызван.

> **ПРИМЕЧАНИЕ** Эта структура v-таблицы, как и эффективная реализация переопределения, хорошо работает, потому что Java реализует только одиночное наследование классов. У каждого типа есть только один непосредственный надкласс (кроме Object, у которого нет надкласса).

17.1.2. Вызов методов интерфейсов

В случае `invokeinterface` ситуация немного усложняется. Например, обратите внимание на то, что метод `groom()` не обязательно находится в одной и той же позиции в v-таблице для каждой реализации `Furry`. Разные смещения `Cat::groom`

https://t.me/javalib

и `Bear::groom` обусловлены тем, что классы `Cat` и `Bear` имеют разную иерархию наследования. В результате JVM приходится выполнять дополнительный поиск, когда метод вызывается для объекта, для которого во время компиляции известен только тип интерфейса.

> **ПРИМЕЧАНИЕ** Хотя поиск реализации для методов интерфейсов требует немного больше работы, не стоит заниматься микрооптимизацией, отказываясь от интерфейсов. Помните, что в JVM есть JIT-компилятор и он фактически устраняет различия в производительности этих двух случаев.

Рассмотрим очередной пример:

```
Cat tom = new Cat();
Bear pooh = new Bear();
Furry f;

tom.groom();
pooh.groom();
f = tom;
f.groom();
f = pooh;
f.groom();
```

Для этого фрагмента генерируется следующий байт-код:

```
0: new           #2                  // class ch15/Cat
       3: dup
       4: invokespecial   #3         // Method ch15/Cat."<init>":()V
       7: astore_1
       8: new             #4         // class ch15/Bear
      11: dup
      12: invokespecial   #5         // Method ch15/Bear."<init>":()V
      15: astore_2
      16: aload_1
      17: invokevirtual   #6         // Method ch15/Cat.groom:()V
      20: aload_2
      21: invokevirtual   #7         // Method ch15/Bear.groom:()V
      24: aload_1
      25: astore_3
      26: aload_3
      27: invokeinterface #8, 1      // InterfaceMethod
                                     // ch15/Furry.groom:()V
      32: aload_2
      33: astore_3
      34: aload_3
      35: invokeinterface #8, 1      // InterfaceMethod
                                     // ch15/Furry.groom:()V
```

Может показаться, что два вызова со смещениями 27 и 35 должны соответствовать одному и тому же участку кода на Java, но на самом деле они вызывают разные методы, потому что содержимое `f` меняется во время выполнения.

https://t.me/javalib

Со смещением 27 на самом деле вызывается `Cat::groom`, тогда как со смещением 35 — `Bear::groom`.

17.1.3. Вызов «специальных» методов

Зная, как работают `invokevirtual` и `invokeinterface`, будет проще понять `invokespecial`. Если метод вызывается инструкцией `invokespecial`, он не проходит виртуальную диспетчеризацию. Вместо этого JVM ищет запрашиваемый метод только в конкретном месте конкретной v-таблицы.

Инструкция `invokespecial` используется в двух случаях: при вызове методов надкласса и при вызовах тела конструктора (который в байт-коде преобразуется в метод `<init>`). В обеих ситуациях виртуальная диспетчеризация и возможность переопределения явно исключаются.

Следует упомянуть о двух особых случаях, которые вроде бы наводят на мысль об использовании `invokespecial` (этот механизм также называется *точной диспетчеризацией*). Первый случай — частные методы: они не могут переопределяться, и вызываемый метод точно известен во время компиляции класса, так что может показаться, что их тоже следует вызывать с помощью `invokespecial`. Тем не менее ситуация сложнее, чем кажется. Это проще пояснить на конкретном примере:

```
public class ExamplePrivate {

  public void entry() {
    callThePrivate();
  }

  private void callThePrivate() {
    System.out.println("Частный метод");
  }
}
```

Для начала откомпилируем этот пример в Java 8. Декомпиляция результата с помощью `javap` дает следующий результат:

```
$ javap -c ch15/ExamplePrivate.class
Compiled from "ExamplePrivate.java"
public class ch15.ExamplePrivate {
  public ch15.ExamplePrivate();
    Code:
       0: aload_0
       1: invokespecial #1      // Method java/lang/Object."<init>":()V
       4: return
  public void entry();
    Code:
       0: aload_0
       1: invokespecial #2      // Method callThePrivate:()V
       4: return
}
```

https://t.me/javalib

Обратите внимание, что `javap` вызывается без ключа `-p`, так что в выводе отсутствует декомпиляция частного метода. Пока все хорошо: частный метод действительно вызывается инструкцией `invokespecial`. Но если перекомпилировать программу на Java 11 и присмотреться повнимательнее, мы увидим другой результат:

```
$ javap -c ch15/ExamplePrivate.class
Compiled from "ExamplePrivate.java"
public class ch15.ExamplePrivate {
  public ch15.ExamplePrivate();
    Code:
       0: aload_0
       1: invokespecial #1  // Method java/lang/Object."<init>":()V
       4: return

  public void entry();
    Code:
       0: aload_0
       1: invokevirtual #2 // Method callThePrivate:()V
       4: return
}
```

Теперь используется invokevirtual

Как видите, в современном Java вызовы частных методов обрабатываются иначе. Эта тема подробнее объясняется в разделе 17.5.3.

17.1.4. Конечные методы

Другой особый случай — вызовы конечных (`final`) методов. На первый взгляд может показаться, что такие вызовы тоже будут преобразованы в инструкции `invokespecial` — в конце концов, они не могут переопределяться, а вызываемая реализация известна во время компиляции. Тем не менее спецификация языка Java по этому поводу говорит следующее:

Изменение метода, объявленного как `final`*, таким образом, что он перестает быть объявленным как* `final`*, не нарушает совместимость с ранее существующими двоичными файлами.*

Допустим, код в одном классе содержит вызов конечного метода из другого класса, который был откомпилирован в `invokespecial`. Затем, если изменить класс с конечным методом так, что метод перестанет быть конечным (и перекомпилировать), то метод можно будет переопределить в подклассе.

Теперь допустим, что экземпляр подкласса был передан вызывающему методу в первом классе. Сработает инструкция `invokespecial`, и на этот раз будет вызвана не та реализация метода. Это нарушает правила объектной ориентации Java (строго говоря, это нарушает принцип подстановки Лисков). Поэтому вызовы конечных методов должны компилироваться в инструкции `invokevirtual`.

ПРИМЕЧАНИЕ На практике HotSpot применяет оптимизацию, которая позволяет обнаруживать конечные методы и выполнять их чрезвычайно эффективно.

https://t.me/javalib

Мы представили основы внутреннего устройства HotSpot с точки зрения диспетчеризации виртуальных методов. На этом месте вам, возможно, захочется перечитать описание JIT-компиляции из главы 7, особенно разделы, посвященные мономорфной диспетчеризации и встраиванию. Возможно, вы начнете глубже понимать эти методы после того, как узнали некоторые подробности их реализации.

17.2. ВНУТРЕННЯЯ РЕАЛИЗАЦИЯ РЕФЛЕКСИИ

Рефлексия упоминалась в главе 4 как механизм, который позволяет динамически оперировать объектами и вызовами методов во время выполнения. Теперь, когда вы знаете о v-таблицах, можно заглянуть глубже и посмотреть, как рефлексия реализуется в JVM.

Вспомните, что из объекта класса можно получить объект `java.lang.reflect.Method`, а затем вызвать его следующим образом (обработка исключений опущена):

```
Class<?> clazz = // ... Какой-то класс
Method m = clazz.getMethod("toString");
Object ret = m.invoke(this);
System.out.println(ret);
```

Но какую сущность представляет этот объект `Method`? Это буквально «возможность динамически вызвать некоторый метод во время выполнения». Динамический вызов означает, что в откомпилированном коде вы увидите только инструкцию `invokevirtual` для метода `invoke()` объекта `Method`:

```
0: ldc           #7  // ... some class
 2: astore_1
 3: aload_1
 4: ldc           #24 // String toString
 6: iconst_0
 7: anewarray     #26 // class java/lang/Class
10: invokevirtual #28 // Method java/lang/
                      // Class.getMethod:
                      // (Ljava/lang/String;[Ljava/lang/Class;)
                      // Ljava/lang/reflect/Method;
13: astore_2
14: aload_2
15: aload_0
16: iconst_0
17: anewarray     #2  // class java/lang/Object
20: invokevirtual #32 // Method java/lang/reflect/
                      // Method.invoke:
                      // (Ljava/lang/Object;[Ljava/lang/Object;)
                      // Ljava/lang/Object;
23: astore_3
```

Вызов getMethod() является вариадическим, ему передается массив нулевого размера объектов Class

При вызове методу invoke() передается массив нулевого размера объектов Object (аргументы)

Обратите внимание, что здесь нет байт-кода, который бы ссылался на `toString()` по дескриптору метода (такому, как `java/lang/Object.toString:()Ljava/lang/String;`); здесь фигурирует только строка `toString`.

https://t.me/javalib

Теперь вспомните, что объекты классов (например, `String.class`) — это обычные объекты Java: они обладают свойствами обычных объектов и представляются внутренними структурами *oop*. У объекта класса есть объект `Method` для каждого метода класса, и эти объекты методов, опять же, представляют собой обычные объекты Java.

ПРИМЕЧАНИЕ Объекты `Method` создаются в отложенном режиме после загрузки класса. Иногда следы этого эффекта видны в отладчике кода в IDE.

Как же в JVM фактически реализуется рефлексия? Давайте посмотрим на фрагмент исходного кода класса `Method`, особенно на некоторые из его полей:

```
private Class<?>            clazz; ◄── Класс, которому принадлежит этот метод
private int                 slot;  ◄── Смещение в v-таблице, по которому хранится метод
//  Гарантированно будет интернироваться виртуальной машиной
// в реализации рефлексии версии 1.4
private String              name;
private Class<?>            returnType;
private Class<?>[]          parameterTypes;
private Class<?>[]          exceptionTypes;
private int                 modifiers;
// Поддержка обобщений и аннотаций
private transient String    signature;
// Хранилище общей информации с отложенной инициализацией
private transient MethodRepository genericInfo;
private byte[]              annotations;
private byte[]              parameterAnnotations;
private byte[]              annotationDefault;
private volatile MethodAccessor methodAccessor;  ◄── Делегат, который передает вызов
```

Вы уже знаете, что когда в Java вызывается метод экземпляра, при этом задействуется поиск по v-таблице. Таким образом, на концептуальном уровне мы воспользуемся дуализмом между v-таблицей и массивом объектов `Method`, который хранится в объекте `Class`. Этот дуализм продемонстрирован на рис. 17.5,

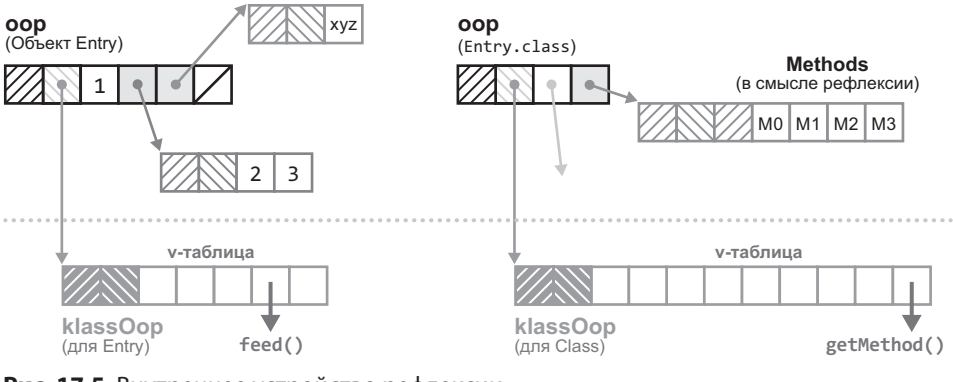


Рис. 17.5. Внутреннее устройство рефлексии

https://t.me/javalib

где массив объектов `Method`, хранящийся в `Entry.class`, сопоставляется с v-таблицей klassOop для `Entry`.

Посмотрим, как на основе этого дуализма `Method` реализует рефлексию. Как выясняется, ключевая роль принадлежит объекту `MethodAccessor`.

> **ПРИМЕЧАНИЕ** Часть приведенного ниже кода упрощена и основана на более ранней версии Java, чтобы облегчить понимание. Текущая версия продакшен-кода в Java 11 и выше устроена гораздо сложнее.

Метод `invoke()` для `Method` выглядит примерно так:

```
public Object invoke(Object obj, Object... args)
    throws IllegalAccessException, IllegalArgumentException,
           InvocationTargetException {
  if (!override) {  <-- Проверяет безопасность доступа (если не setAccessible())
    if (!Reflection.quickCheckMemberAccess(clazz, modifiers)) {
      Class<?> caller = Reflection.getCallerClass();
      checkAccess(caller, clazz, obj, modifiers);
    }
  }
  MethodAccessor ma = methodAccessor;  <-- Нестабильное чтение метода доступа
  if (ma == null) {
    ma = acquireMethodAccessor();
  }
  return ma.invoke(obj, args);  <-- Делегирует экземпляру MethodAccessor
}
```

При первом вызове этого метода `acquireMethodAccessor()` создает экземпляр `DelegatingMethodAccessorImpl`, который содержит ссылку на `NativeMethodAccessorImpl`. Это классы, которые определены в `sun.reflect` и оба реализуют `MethodAccessor`. Они не входят в API модуля `java.base`, и их нельзя вызвать напрямую.

Ниже приведен полный код `DelegatingMethodAccessorImpl`:

```
class DelegatingMethodAccessorImpl extends MethodAccessorImpl {
  private MethodAccessorImpl delegate;

  DelegatingMethodAccessorImpl(MethodAccessorImpl delegate) {
    setDelegate(delegate);
  }

  public Object invoke(Object obj, Object[] args)
        throws IllegalArgumentException, InvocationTargetException {
    return delegate.invoke(obj, args);
  }

  void setDelegate(MethodAccessorImpl delegate) {
    this.delegate = delegate;
  }
}
```

https://t.me/javalib

А вот код `NativeMethodAccessorImpl`:

```
class NativeMethodAccessorImpl extends MethodAccessorImpl {
  private Method method;
  private DelegatingMethodAccessorImpl parent;
  private int numInvocations;

  // …

  public Object invoke(Object obj, Object[] args)
          throws IllegalArgumentException, InvocationTargetException {

    if (++numInvocations >
          ReflectionFactory.inflationThreshold()) {
      MethodAccessorImpl acc = (MethodAccessorImpl)
          new MethodAccessorGenerator()
            .generateMethod(method.getDeclaringClass(),
                            method.getName(),
                            method.getParameterTypes(),
                            method.getReturnType(),
                            method.getExceptionTypes(),
                            method.getModifiers());
      parent.setDelegate(acc);
    }

    return invoke0(method, obj, args);
  }

  private static native Object invoke0(Method m, Object obj, Object[] args);

  // ...
}
```

Входим сюда, когда достигнут порог количества вызовов

С помощью MethodAccessorGenerator создаем собственный класс, который реализует рефлексивный вызов

Заменяет текущий объект как делегат экземпляром нового собственного класса

Если порог еще не достигнут, выполняем стандартный вызов

Таким образом, мы используем делегирующий метод доступа, который можно обогатить новым динамически сгенерированным методом доступа в байт-коде. Этот прием изображен на рис. 17.6. Обратите внимание, что наш собственный класс метода доступа должен быть подклассом `MethodAccessorImpl`, чтобы не возникло проблем с приведением типов.

Скажем пару слов о производительности: в этом механизме есть два гипотетических источника замедления. С одной стороны, низкоуровневый метод доступа использует низкоуровневый вызов, который выполняется медленнее, чем вызов метода Java, и не может JIT-компилироваться. С другой стороны, динамическое генерирование байт-кода в `MethodAccessorGenerator` потенциально работает медленно и может оказаться неудачным компромиссом, которого лучше избегать, если метод вызывается с помощью рефлексии только один раз. Этот прием, когда объект метода доступа загружается в отложенном режиме, а затем динамически модифицируется на стороне вызова, еще встретится нам в другом обличии позднее в этой главе.

Также заметим, что отражение несовместимо со встраиванием и стандартными разновидностями диспетчеризации методов, которые JVM хорошо

https://t.me/javalib

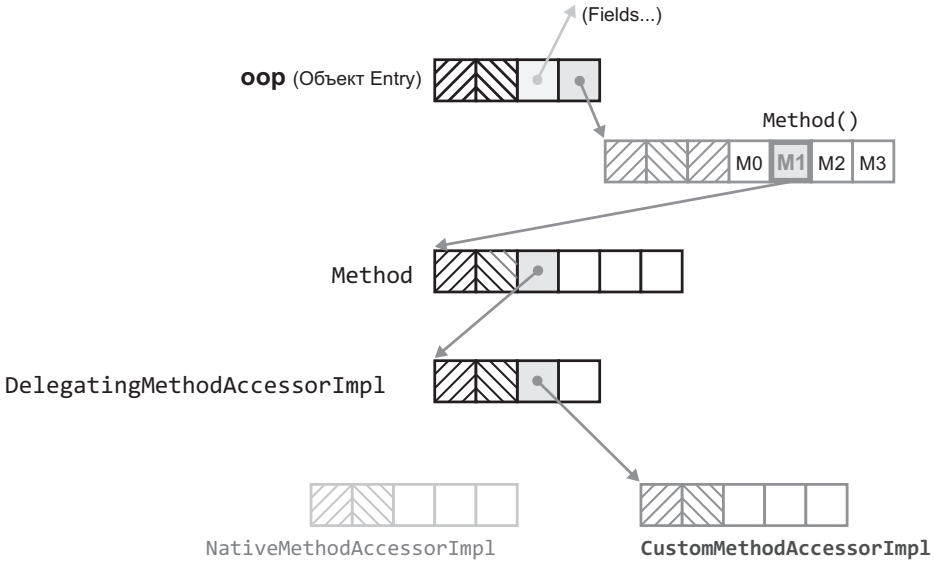


Рис. 17.6. Реализация рефлексии

умеет оптимизировать. После модификации сторона вызова для метода `DelegatingMethodAccessorImpl` называется *мегаморфной* (у одного метода возможно много разных реализаций), потому что у каждого экземпляра `Method` оказывается свой, динамически сгенерированный объект метода доступа. Это означает, что для рефлексивных вызовов некоторые механизмы первичной оптимизации JVM будут работать не лучшим образом.

Таким образом, делегирование в сочетании с модификацией, которая подменяет низкоуровневый метод доступа, — это компромисс, направленный на то, чтобы достичь приемлемой производительности для редко вызываемых рефлексивных методов и при этом сохранить некоторые преимущества JIT. Этот компромисс, как и другие проблемы рефлексии, обсуждавшиеся в главе 4, привели к поискам более эффективного решения проблем с динамическими вызовами и изобретению облегченных объектов методов. В следующем разделе мы представим первый результат этих поисков — API дескрипторов методов.

17.3. ДЕСКРИПТОРЫ МЕТОДОВ

API дескрипторов методов был добавлен в Java 7. Центральное место в этом интерфейсе занимает пакет `java.lang.invoke` и особенно класс `MethodHandle`. Экземпляры этого типа предоставляют возможность вызвать метод, и их можно запускать напрямую подобно объектам `java.lang.reflect.Method`.

API был разработан в рамках проекта по внедрению на JVM инструкции `invokedynamic`, которая рассматривалась в разделе 17.4. Но объекты дескрипторов

https://t.me/javalib

методов находят применение во фреймворках и в обычном пользовательском коде далеко за пределами типичных сценариев использования `invokedynamic`.

Сначала мы представим основы технологии дескрипторов методов, а затем рассмотрим расширенный пример, который сравнивает их с некоторыми альтернативами и предоставляет краткую сводку различий.

17.3.1. MethodHandle

Что такое дескриптор метода (`MethodHandle`)? Официальный ответ гласит, что это типизованная ссылка на метод, которую можно запустить непосредственно. Также можно сказать, что `MethodHandle` — объект, который предоставляет возможность безопасно вызвать метод.

> **ПРИМЕЧАНИЕ** `MethodHandle` во многом похож на объект `Method` из `java.lang.reflect`, но его API в целом удобнее и менее громоздок, а также в нем исправлен ряд важных недочетов проектирования.

Обсудим два главных вопроса, которые связаны с дескрипторами методов: как их получить и как их использовать. Второй вопрос очень прост, и сейчас вы увидите пример, в котором вызывается дескриптор метода. Допустим, у нас есть статический вспомогательный метод `getTwoArgMH()`, который возвращает дескриптор метода, а этот дескриптор принимает объект-получатель `obj` и один аргумент вызова `arg0`, а возвращает `String`.

Позже мы объясним, как получить дескриптор метода с такой сигнатурой, а пока будем предполагать, что вспомогательный метод создает его за нас. Рассмотрим пример:

```
MethodHandle mh = getTwoArgMH();

try {
  String result = mh.invokeExact(obj, arg0);
} catch (Throwable e) {
  e.printStackTrace();
}
```

Получает дескриптор метода от вспомогательного метода

Выполняет вызов, передавая получатель и один аргумент

Это напоминает рефлексивный вызов метода наподобие тех, с которыми мы познакомились в разделе 4.5.1. Правда, мы вызываем `invokeExact()` для `MethodHandle`, а не `invoke()` для `Method`, но в остальном все выглядит похоже. Тем не менее это можно сделать, только если у нас уже есть объект дескриптора метода, — а как же его получить?

Чтобы получить дескриптор метода, необходимо провести *поиск по контексту*, а контекст обычно можно получить с помощью статического вспомогательного метода `MethodHandles.lookup()`. Он возвращает контекст поиска для текущего выполняемого метода, а уже из этого контекста можно получить дескрипторы методов, вызывая один из методов `find*()`, например `findVirtual()` или `findConstructor()`.

https://t.me/javalib

Объект контекста поиска может предоставить дескриптор любого метода, видимого из контекста выполнения, в котором был создан поиск. Однако нам понадобится не только контекст поиска для методов, но и способ представить сигнатуру метода, для которого нужен дескриптор.

Вспомните интерфейс `Callable` из главы 6. Он представляет блок кода, который необходимо выполнить, как и дескрипторы методов. Однако проблема с `Callable` заключается в том, что он умеет моделировать только методы, которые не получают аргументов.

Чтобы моделировать все типы методов, можно было бы создавать другие интерфейсы с разным количеством параметров-типов. В итоге получится набор интерфейсов следующего вида:

```
Function0<R>
Function1<R, P>
Function2<R, P1, P2>
Function3<R, P1, P2, P3>
...
```

Такой подход быстро приведет к тому, что у вас расплодится слишком много интерфейсов. Некоторые языки идут по этому пути (например, Scala), но Java поступает по-другому.

Также поучительно посмотреть, как эту задачу решает Clojure. Интерфейс `IFn` содержит методы `invoke`, которые представляют функции любой возможной арности (включая вариадическую форму для функций, принимающих более 20 аргументов). Мы встречали упрощенную версию `IFn` в главе 10.

Однако Clojure использует динамическую типизацию, так что все методы `invoke` получают `Object` для каждого параметра и возвращают тоже `Object` при этом не возникает сложности, связанной с обработкой обобщений. Формы Clojure также можно довольно естественным образом записывать в вариадическом виде: если вызвать форму с неправильной арностью, Clojure возбуждает исключение времени выполнения. В Java не используется ни один из этих приемов.

Вместо этого дескрипторы методов Java реализуют подход, который позволяет моделировать любую сигнатуру метода, не плодя огромного количества мелких классов. Для этого используется новый класс `MethodType`.

17.3.2. MethodType

`MethodType` — это неизменяемый объект, который представляет сигнатуру типов метода. С каждым дескриптором метода связан экземпляр `MethodType`, который содержит возвращаемый тип и типы аргументов. Однако он не содержит ни имени метода, ни типа получателя (это тип, для которого вызывается метод экземпляра).

Простой способ получать новые экземпляры `MethodType` обеспечивают фабричные методы класса `MethodType`. Вот несколько примеров:

https://t.me/javalib

```
var mtToString = MethodType.methodType(String.class);
var mtSetter = MethodType.methodType(void.class, Object.class);
var mtStringComparator = MethodType.methodType(int.class,
                                               String.class, String.class);
```

Это экземпляры `MethodType`, которые представляют сигнатуры типов `toString()`, метода записи (для члена класса типа `Object`) и метода `compareTo()`, который определен интерфейсом `Comparator<String>`. В общем случае экземпляр строится по той же схеме: сначала передается возвращаемый тип, а потом типы аргументов (в виде объектов `Class`):

```
MethodType.methodType(RetType.class, Arg0Type.class, Arg1Type.class, ...);
```

Как видите, разные сигнатуры методов теперь можно представлять как обычные объекты и не нужно определять новый тип для каждой сигнатуры. Также это позволяет легко обеспечить безопасность типов, насколько это возможно. Чтобы узнать, можно ли вызывать дескриптор того или иного метода с определенным набором аргументов, проанализируйте экземпляр `MethodType`, который относится к этому дескриптору.

> **ПРИМЕЧАНИЕ** Передавать один объект `MethodType` намного удобнее, чем громоздкий `Class[]`, который вам приходится использовать в случае рефлексии.

Теперь, когда мы узнали, как объекты `MethodType` решают проблему «плодящихся интерфейсов», давайте посмотрим, как создавать дескрипторы методов, которые ссылаются на методы наших типов.

17.3.3. Как получать дескрипторы методов

Посмотрим, как получить дескриптор метода, который указывает на метод `toString()` текущего класса. Нам нужно, чтобы объект `mtToString` точно соответствовал сигнатуре `toString()`, то есть имел возвращаемый тип `String` и не принимал аргументы. Соответствующий экземпляр `MethodType` создается как `MethodType.methodType(String.class)`:

```
public MethodHandle getToStringMH() {
  MethodHandle mh;
  var mt = MethodType.methodType(String.class);
  var lk = MethodHandles.lookup(); ◄── Получает контекст поиска

  try {
    mh = lk.findVirtual(getClass(), "toString", mt); ◄── Находит дескриптор по контексту
  } catch (NoSuchMethodException | IllegalAccessException mhx) {
    throw (AssertionError)new AssertionError().initCause(mhx);
  }

  return mh;
}
```

Чтобы получить от объекта поиска дескриптор метода, необходимо предоставить класс, в котором содержится нужный метод, имя метода и объект `MethodType`,

https://t.me/javalib

который представляет нужную сигнатуру. `MethodType` необходим для перегруженных методов.

Контекст поиска часто используют, чтобы находить методы текущего класса, но на самом деле контекст позволяет получить дескрипторы методов, принадлежащих *любому* типу, включая типы JDK. Конечно, если вы получаете дескрипторы из класса, который находится в другом пакете или модуле, контекст поиска будет видеть только методы, которые вам доступны (например, открытые методы открытых классов экспортированных пакетов). Это важная особенность API дескрипторов методов: права доступа к дескриптору метода проверяются, когда метод найден, а не тогда, когда дескриптор выполняется.

После того как вы получили дескриптор метода, его всегда можно безопасно вызвать, потому что дополнительных проверок прав доступа не будет. Дескриптор метода можно создать в одном контексте, где доступ разрешен, а затем передать в другой контекст, где доступ запрещен, — и он все равно выполнится. В этом заключается важное отличие от рефлексии.

> **ПРИМЕЧАНИЕ** В отличие от рефлексивных вызовов, контроль доступа для вызова дескрипторов методов невозможно обойти. Здесь не существует аналога трюка с `setAccessible()`, который демонстрировался в главе 4.

Теперь у вас есть дескриптор метода, и самое естественное, что с ним можно сделать, — выполнить. API предоставляет для этого два основных метода: `invokeExact()` и `invoke()`.

Метод `invokeExact()` требует, чтобы типы аргументов точно совпадали с теми, которые ожидает получить используемый метод. Метод `invoke()`, обнаружив какие-либо расхождения, выполняет преобразования, чтобы привести типы в соответствие (например, упаковку или распаковку в случае необходимости).

После этого краткого введения мы перейдем к более длинному примеру того, как дескрипторы методов позволяют заменить другие механизмы, например рефлексию и мелкие внутренние классы, которые используются в качестве заместителей.

17.3.4. Рефлексия, заместители и дескрипторы методов

Если вам доводилось разбираться в кодовой базе, где интенсивно используется рефлексия, то скорее всего, вы знаете, сколько проблем создает рефлексивный код. В этом подразделе мы покажем, как дескрипторы методов помогают заменить большой объем шаблонного рефлексивного кода и немного упростить вашу работу.

Чтобы объяснить различия между дескрипторами методов и другими решениями, мы продемонстрируем три варианта обращения к частному методу `callThePrivate()` из-за пределов класса. Сравним два стандартных приема — рефлексию и внутренний класс-заместитель — с современным подходом на основе `MethodHandle`. Примеры всех трех решений содержатся в следующем листинге.

https://t.me/javalib

Листинг 17.1. Три способа предоставления доступа

```
public class ExamplePrivate {
    // Какое-то состояние...

    public void entry() {
        callThePrivate();
    }

    private void callThePrivate() {   <-- Частный метод, к которому нужно предоставить доступ
        System.out.println("Private method");
    }

    public Method makeReflective() {
        Method meth = null;

        try {
            Class<?>[] argTypes = new Class[] { Void.class };
            meth = ExamplePrivate.class
                        .getDeclaredMethod("callThePrivate", argTypes);
            meth.setAccessible(true);
        } catch (IllegalArgumentException |
                 NoSuchMethodException |
                 SecurityException e) {
            throw (AssertionError)new AssertionError().initCause(e);
        }

        return meth;
    }

    public static class Proxy {
        private Proxy() {}

        public static void invoke(ExamplePrivate priv) {
            priv.callThePrivate();
        }
    }

    public MethodHandle makeMh() {
        MethodHandle mh;
        var desc = MethodType.methodType(void.class);   <-- Создаем MethodType: здесь можно использовать конкретный тип, а не выполнять упаковывание

        try {
            mh = MethodHandles.lookup()
                      .findVirtual(ExamplePrivate.class,
                          "callThePrivate", desc);   <-- Поиск MethodHandle
        } catch (NoSuchMethodException | IllegalAccessException e) {
            throw (AssertionError)new AssertionError().initCause(e);
        }
        return mh;
    }
}
```

Класс из этого примера обращается к частному методу `callThePrivate()` тремя разными способами. В реальном коде в каждом отдельном случае обычно

https://t.me/javalib

используется только один из этих способов; мы приводим все три только для того, чтобы обсудить различия между ними. На практике вас как пользователя API не должно касаться, какой способ используется в конкретном случае.

Из табл. 17.1 видно, что главное преимущество рефлексии заключается в том, что она знакома разработчикам. Возможно, заместители проще для понимания в простых случаях, но мы полагаем, что дескрипторы методов объединяют лучшие качества обоих подходов, и настоятельно рекомендуем использовать дескрипторы во всех новых приложениях.

Таблица 17.1. Сравнение технологий косвенного обращения к методам в Java

Фактор	Рефлексия	Внутренний класс или лямбда-выражение	Дескриптор метода
Управление доступом	Необходимо использовать setAccessible(). Может быть запрещено активным диспетчером безопасности	Внутренние классы могут обращаться к методам с ограниченным доступом	Полный доступ ко всем методам, разрешенным из текущего контекста. Нет проблем с диспетчерами безопасности
Дисциплина типов	Отсутствует. Неприятное исключение при несоответствии	Статическая. Может быть слишком жесткой. Может потребоваться много метапространства для всех заместителей	Безопасность типов во время выполнения. Практически (или вовсе) не потребляет метапространства
Быстродействие	Медленно по сравнению с альтернативными вариантами	Так же быстро, как любой другой вызов метода	Так же быстро, как любой другой вызов метода

Дескрипторы методов обеспечивают еще одну возможность — определять текущий класс из статического контекста. Если вы когда-нибудь писали код протоколирования (например, для log4j) вроде такого:

```
Logger lgr = LoggerFactory.getLogger(MyClass.class);
```

то вы уже знаете, что этот код довольно хрупкий. Если провести рефакторинг и перенести его в надкласс или подкласс, начнутся проблемы из-за явного имени класса. Однако с дескрипторами методов можно написать так:

```
Logger lgr = LoggerFactory.getLogger(MethodHandles.lookup().lookupClass());
```

В этом коде выражение `lookupClass()` можно считать аналогом `getClass()`, которое используется в статическом контексте. Это особенно полезно при работе с фреймворками журналов, где обычно приходится по одному протоколирующему объекту на класс.

> **ПРИМЕЧАНИЕ** API дескрипторов методов оказался очень успешным — настолько, что в Java 18 (но не 11 или 17) реализацию рефлексии переработали так, что она теперь опирается на дескрипторы методов, а не на технологии из предыдущего раздела.

https://t.me/javalib

Пополнив свой инструментарий технологией дескрипторов методов и вооружившись практическими знаниями байт-кода из главы 4, перейдем к подробностям операции `invokedynamic`. Она появилась в Java 7 и до настоящего момента остается единственным кодом операции, который был специально добавлен в набор инструкций JVM. Изначально `invokedynamic` предназначалась для того, чтобы другие языки помимо Java могли полнее использовать потенциал платформы JVM, но как вы вскоре увидите, эта инструкция стала важным фактором перемен на самой платформе.

17.4. INVOKEDYNAMIC

В этом разделе рассматривается одна из самых технически сложных возможностей современного Java. Но несмотря на чрезвычайную мощь, она предназначена в первую очередь не для обычных программистов, а скорее для тех, кто разрабатывает фреймворки и создает реализации альтернативных языков.

Поэтому при первом чтении этот раздел вполне можно пропустить. Чтобы использовать его материал наиболее эффективно, важно прочитать и понять описания инструкций `invoke` ранее в этой главе — это поможет усвоить те правила, которые мы здесь собираемся нарушать.

Мы подробно рассмотрим, как работает `invokedynamic`, а также разберем примеры декомпиляции точки вызова, которая использует новый байт-код. Учтите, что вам не обязательно полностью понимать эту тему, чтобы использовать языки и фреймворки, где задействована `invokedynamic`, но эта глава посвящена внутренней реализации, поэтому мы уделим внимание техническим подробностям.

Как можно догадаться по имени, `invokedynamic` — новая разновидность инструкций для вызова методов. Она заставляет JVM отложить до стадии выполнения решение о том, какой конкретно метод вызывать.

На первый взгляд ничего особенного в этом нет: в конце концов, `invokevirtual` и `invokeinterface` тоже выбирают вызываемую реализацию только во время выполнения. Однако выбор метода у этих операций ограничен правилами наследования и системой типов языка Java, так что хотя бы какая-то информация о *целевом объекте вызова* (call target) должна быть известна во время компиляции.

Инструкция `invokedynamic` была создана, чтобы ослабить эти ограничения. Для этого она вызывает вспомогательный метод (так называемый *метод-инициатор*, или BSM (bootstrap method)), который принимает решение о том, какой метод нужно вызвать в итоге.

> **ПРИМЕЧАНИЕ** Целевой метод для `invokedynamic` вообще не обязан подчиняться правилам иерархии наследования Java — все решает пользователь.

Чтобы сделать возможной такую гибкость, операции `invokedynamic` обращаются к специальному разделу пула констант класса, который содержит расширенные элементы для поддержки динамических вызовов — BSM. Они критически важны

https://t.me/javalib

для `invokedynamic`, и каждой точке вызова `invokedynamic` соответствует элемент для BSM в пуле констант.

BSM получает информацию о точке вызова и компонует динамический вызов. BSM принимает не менее трех аргументов и возвращает объект `CallSite`. Стандартные аргументы имеют следующие типы:

- `MethodHandles.Lookup` — объект поиска для класса, в котором находится точка вызова.
- `String` — имя, упомянутое в `NameAndType`.
- `MethodType` — разрешенный дескриптор типа из `NameAndType`.

За этими аргументами следуют дополнительные аргументы, которые нужны BSM и в документации называются *дополнительными статическими аргументами*. Возвращаемая точка вызова содержит `MethodHandle`, который является результатом вызова точки вызова и будет выполняться как фактический вызов `invokedynamic`.

> **ПРИМЕЧАНИЕ** Чтобы BSM можно было связывать с конкретной точкой вызова `invokedynamic`, в формат файлов класса был добавлен новый тип элемента пула констант, который тоже называется `InvokeDynamic`.

Говорят, что точка вызова инструкции `invokedynamic` «отвязана» во время загрузки класса. Это значит, что с точкой вызова не связан ни один целевой метод, и он не будет связан, пока выполнение не достигнет этой точки (то есть пока JVM не попытается скомпоновать, а затем выполнить эту конкретную инструкцию `invokedynamic`).

В этот момент вызывается BSM, чтобы определить, какой метод должен быть фактически вызван. BSM всегда возвращают объект `CallSite` (содержащий `MethodHandle`), который «привязывается» к точке вызова. После того как `CallSite` скомпонован, можно фактически вызвать метод — благодаря дескриптору `MethodHandle`, который хранится в `CallSite`.

В простейшем случае использования `ConstantCallSite` после того, как поиск был выполнен один раз, он не будет повторяться. Вместо этого при всех последующих вызовах целевой метод точки вызова будет вызываться напрямую без каких-либо дополнительных действий. Такое поведение напоминает `CompletableFuture<CallSite>`. На практике это означает, что точка вызова теперь стабильна, а следовательно, хорошо взаимодействует с другими подсистемами JVM, например JIT-компилятором. Возможны и более сложные варианты, такие как `MutableCallSite` (и даже `VolatileCallSite`), которые позволяют перекомпоновать точку вызова, чтобы позднее она могла указывать на другой целевой метод.

На протяжении срока жизни программы у непостоянной точки вызова может смениться много разных дескрипторов методов в качестве целей. Возможность

https://t.me/javalib

сменить метод, который вызывается в конкретной точке вызова, может сыграть важную роль для других языков помимо Java.

Например, в JavaScript или Ruby для отдельного объекта того или иного типа можно определить методы, которых нет у других экземпляров класса. В Java это невозможно: класс определяет набор методов, на основании которого строится v-таблица при загрузке класса, и все экземпляры совместно используют одну и ту же v-таблицу. `invokedynamic` с изменяемыми точками вызова позволит эффективно реализовать эту функциональность, отсутствующую в Java.

Следует указать, что нельзя заставить `javac` генерировать `invokedynamic` для обычных вызовов методов: вызов метода Java всегда преобразуется в один из четырех стандартных кодов `invoke`, представленных в главе 4. Вместо этого фреймворки и библиотеки Java (в том числе входящие в JDK) используют `invokedynamic` для различных целей, в частности для лямбда-выражений. Присмотримся к ним поближе.

17.4.1. Реализация лямбда-выражений

Лямбда-выражения повсеместно используются в программах на Java, но на самом деле многие Java-программисты не знают, как они реализованы. Разберемся в этом, начиная с простого примера:

```
public class LambdaExample {
    private static final String HELLO = "Hello World!";

    public static void main(String[] args) throws Exception {
        Runnable r = () -> System.out.println(HELLO);
        Thread t = new Thread(r);
        t.start();
        t.join();
    }
}
```

Можно предположить, что лямбда-выражения — это просто «синтаксический сахар» для анонимной реализации `Runnable`. Но если откомпилировать приведенный выше класс, вы увидите, что сгенерирован только один файл `LambdaExample.class` и нет второго файла класса, где был бы размещен внутренний класс (как обсуждалось в главе 8). А значит, все не так просто.

Если декомпилировать класс, то можно убедиться, что тело лямбда-выражения на самом деле скомпилировалось в частный статический метод, который находится в основном классе:

```
private static void lambda$main$0();
    Code:
       0: getstatic     #7   // Field
                             // java/lang/System.out:Ljava/io/PrintStream;
       3: ldc           #9   // String Hello World!
       5: invokevirtual #10  // Method java/io/PrintStream.println:
                             // (Ljava/lang/String;)V
       8: return
```

https://t.me/javalib

При этом метод `main` выглядит так:

```
public static void main(java.lang.String[]) throws java.lang.Exception;
    Code:
       0: invokedynamic #2, 0   // InvokeDynamic #0:run:
                                // ()Ljava/lang/Runnable;
       5: astore_1
       6: new           #3      // class java/lang/Thread
       9: dup
      10: aload_1
      11: invokespecial #4      // Method java/lang/Thread."<init>"
                                // :(Ljava/lang/Runnable;)V
      14: astore_2
      15: aload_2
      16: invokevirtual #5      // Method java/lang/Thread.start:()V
      19: aload_2
      20: invokevirtual #6      // Method java/lang/Thread.join:()V
      23: return
```

`invokedynamic` действует как вызов необычной формы фабричного метода. Вызов возвращает экземпляр некоторого типа, реализующего интерфейс `Runnable`. Точный тип в байт-коде не указан, да это и неважно. На самом деле возвращаемого типа как такового не существует во время компиляции; он создается по требованию во время выполнения.

Мы знаем, что с точками вызова `invokedynamic` всегда связываются методы-инициаторы. В нашем простом примере с `Runnable` в соответствующем разделе файла класса используется один BSM:

```
BootstrapMethods:
  0: #28 REF_invokeStatic java/lang/invoke/LambdaMetafactory.metafactory:
       (Ljava/lang/invoke/MethodHandles$Lookup;Ljava/lang/String;
        Ljava/lang/invoke/MethodType;Ljava/lang/invoke/MethodType;
        Ljava/lang/invoke/MethodHandle;Ljava/lang/invoke/MethodType;)
        Ljava/lang/invoke/CallSite;
    Method arguments:
      #29 ()V
      #30 REF_invokeStatic LambdaExample.lambda$main$0:()V
      #29 ()V
```

Этот код довольно трудно прочитать, так что попробуем расшифровать его. Методом-инициатором для этой точки вызова является элемент #28 в пуле констант — элемент типа `MethodHandle`. Он указывает на статический метафабричный метод `LambdaMetafactory.metafactory()` из пакета `java.lang.invoke`. Этот метод принимает несколько аргументов, но они в основном обеспечиваются дополнительными статическими аргументами, которые принадлежат BSM (элементы #29 и #30).

Единственное лямбда-выражение генерирует три статических аргумента, которые передаются BSM: сигнатуру лямбда-выражения, дескриптор метода для фактического целевого объекта вызова лямбда-выражения (то есть тела лямбда-выражения) и форму сигнатуры со стиранием.

https://t.me/javalib

Перейдем к `java.lang.invoke` и посмотрим, как платформа использует *метафабрики*, чтобы динамически создавать классы, которые фактически реализуют целевые типы для наших лямбда-выражений. BSM (то есть вызов метода `metafactory`), как обычно, возвращает объект точки вызова. Когда выполняется инструкция `invokedynamic`, дескриптор метода, содержащийся в точке вызова, вернет экземпляр класса, который реализует целевой тип лямбда-выражения.

> **ПРИМЕЧАНИЕ** Если инструкция `invokedynamic` не выполнится, то динамически создаваемый класс не будет создан.

Исходный код метода `metafactory` относительно прост:

```
public static CallSite metafactory(MethodHandles.Lookup caller,
                                   String invokedName,
                                   MethodType invokedType,
                                   MethodType samMethodType,
                                   MethodHandle implMethod,
                                   MethodType instantiatedMethodType)
        throws LambdaConversionException {
    AbstractValidatingLambdaMetafactory mf;
    mf = new InnerClassLambdaMetafactory(caller, invokedType,
                                         invokedName, samMethodType,
                                         implMethod,
                                         instantiatedMethodType,
                                         false, EMPTY_CLASS_ARRAY,
                                         EMPTY_MT_ARRAY);
    mf.validateMetafactoryArgs();
    return mf.buildCallSite();
}
```

Объект поиска соответствует контексту, в котором находится инструкция `invokedynamic`. В нашем случае это тот же класс, в котором было определено лямбда-выражение, так что у контекста поиска есть необходимые права, чтобы обратиться к частному методу, в который было откомпилировано тело лямбда-выражения.

Имя и тип вызываемого метода предоставляет JVM, и они относятся к подробностям реализации. Три последних параметра — дополнительные статические аргументы из BSM.

В текущей реализации лямбда-выражений метафабрика делегирует вызов коду, который с помощью внутренней теневой копии библиотек байт-кода ASM создает внутренний класс, который реализует целевой тип. В будущем эта схема может измениться.

Наконец, следует заметить, что ничто не мешает вам создать собственный класс, который будет использовать `invokedynamic` для каких-то особых целей. Однако чтобы сконструировать такой класс, необходимо воспользоваться библиотекой, которая манипулирует байт-кодом и может создать файл `.class`, содержащий инструкцию `invokedynamic`. Хорошим вариантом является библиотека ASM (http://asm.ow2.org/), о которой мы уже упоминали. Это библиотека коммерческого

https://t.me/javalib

уровня, которая используется во многих известных фреймворках Java (включая сам JDK).

На этом наше обсуждение `invokedynamic` завершается, и мы переходим к менее масштабным, но все равно важным изменениям во внутренней реализации.

17.5. НЕБОЛЬШИЕ ВНУТРЕННИЕ ИЗМЕНЕНИЯ

Иногда мелкие изменения оказывают большое влияние на язык. В этом разделе мы представим три небольших внутренних изменения в реализации, которые либо улучшают производительность, либо исправляют пережитки прошлого на платформе. Начнем со строк.

17.5.1. Конкатенация строк

Как известно, в Java экземпляры `String` фактически неизменяемы. Что же происходит при конкатенации двух строк оператором +? JVM должна создать новый объект `String`, но это сложнее, чем может показаться на первый взгляд.

Возьмем простой класс с методом `main()`:

```
public static void main(String[] args) {
  String str = "foo";
  if (args.length > 0) {
    str = args[0];
  }
  System.out.println("вот моя строка: " + str);
}
```

Байт-код Java 8 для этого относительно простого метода выглядит так:

```
public static void main(java.lang.String[]);
Code:
  0: ldc #17            // String foo
  2: astore_2
  3: aload_1
  4: arraylength
  5: ifle 12  <-------  Если массив пуст, перейти вперед к инструкции 12
  8: aload_1
  9: iconst_0
 10: aaload
 11: astore_2
 12: getstatic #19      // Field java/lang/System.out:
                        // Ljava/io/PrintStream;          | Загружает System.out в стек
 15: new #25            // class java/lang/StringBuilder |
 18: dup                                                 |
 19: ldc #27            // String вот моя строка:        | Создает StringBuilder
 21: invokespecial #29  // Method java/lang/             |
                        // StringBuilder."<init>"        |
                        // :(Ljava/lang/String;)V        |
 24: aload_2
 25: invokevirtual #32  // Method java/lang/StringBuilder.append
```

https://t.me/javalib

```
                        // (Ljava/lang/String;)Ljava/lang/StringBuilder;
 28: invokevirtual #36  // Method java/lang/                  Создает строку с помощью
                        // StringBuilder.toString:           StringBuilder
                        // ()Ljava/lang/String;
 31: invokevirtual #40  // Method java/io/                   Выводит строку
                        // PrintStream.println:
                        // (Ljava/lang/String;)V
 34: return
```

В этом коде заслуживают внимания несколько моментов. В частности, кажется немного странным, что здесь фигурирует `StringBuilder`: мы запросили конкатенацию строк, но байт-код сообщает, что на самом деле создаются дополнительные объекты, а затем для них вызываются `append()` и `toString()`.

В инструкциях 15–23 представлен паттерн создания объектов (`new`, `dup`, `invokespecial`) для временного объекта `StringBuilder`, но в данном случае в конструировании также задействуется инструкция `ldc` (load constant) после `dup`. В этой разновидности паттерна вызывается «не-`void`» конструктор, в данном случае `StringBuilder(String)`.

Причина всего этого в том, что строки Java фактически неизменяемы. Операция конкатенации не может изменить содержимое строки; вместо этого приходится создавать новый объект, и `StringBuilder` — удобный способ сделать это.

Однако байт-код для Java 11 выглядит совершенно иначе:

```
public static void main(java.lang.String[]);
Code:
  0: ldc           #2      // String foo
  2: astore_1
  3: aload_0
  4: arraylength
  5: ifle 12
  8: aload_0
  9: iconst_0
 10: aaload
 11: astore_1
 12: getstatic     #3      // Field java/lang/System.out:
                           // Ljava/io/PrintStream;
 15: aload_1
 16: invokedynamic #4, 0   // InvokeDynamic #0:makeConcatWithConstants:
                           // (Ljava/lang/String;)Ljava/lang/String;
 21: invokevirtual #5      // Method java/io/PrintStream.println:
                           // (Ljava/lang/String;)V
 24: return
```

Первые 12 инструкций идентичны байт-коду для Java 8, но дальше ситуация меняется: полностью исчез временный объект `StringBuilder`, зато появился `invokedynamic` в инструкции 16. Конечно, для этого требуется метод-инициатор:

```
BootstrapMethods:
  0: #23 REF_invokeStatic java/lang/invoke/StringConcatFactory.
```

https://t.me/javalib

```
    makeConcatWithConstants:(Ljava/lang/invoke/MethodHandles$Lookup;
    Ljava/lang/String;Ljava/lang/invoke/MethodType;
    Ljava/lang/String;[Ljava/lang/Object;)Ljava/lang/invoke/CallSite;
  Method arguments:
    #24 вот моя строка: \u0001
```

Это динамический вызов статического фабричного метода `makeConcatWithConstants()`, который принадлежит классу `StringConcatFactory` из пакета `java.lang.invoke`. Фабричный метод получает строку — *рецепт* конкатенации для этих конкретных аргументов — и производит объект `CallSite`, который скомпонован со специальным методом для этого конкретного случая.

Все эти механизмы находятся глубоко в коде реализации JVM, и обычным программам на Java практически никогда не приходится вызывать эти методы напрямую. Вместо этого программы используют код JDK и библиотеки или фреймворки, которые это делают.

> **ПРИМЕЧАНИЕ** Статический аргумент метода-инициатора включает символ `\u0001` (кодовая точка Юникода 0001). Это обычный аргумент, который интерполируется в рецепт конкатенации.

Точки вызова `invokedynamic` можно использовать повторно, и при необходимости реализующие классы могут создаваться динамически. Реализациям также доступны частные API, такие как конструкторы `String`, обходящиеся без копирования, которые было бы невозможно предоставить как часть `StringBuilder`.

17.5.2. Компактные строки

Когда вы только начали изучать Java, то узнали о примитивных типах и о том, что `char` занимает 2 байта. Нетрудно предположить, что во внутренней реализации строк Java используется массив `char[]`, в котором хранятся отдельные символы строки.

Тем не менее это не совсем так. Действительно, в Java 8 и более ранних версиях содержимое `String` представляется в виде `char[]`. Однако это представление — не самое эффективное; поскольку это может быть неочевидно на первый взгляд, давайте разберемся чуть глубже.

Двухбайтовый тип Java `char` (который представляет символ UTF-16) избыточен для любой строки, которая содержит только символы западноевропейских языков, потому что в таких строках первый байт каждого значения `char` всегда равен нулю. Таким образом почти половина памяти пропадает даром, — при том, что такие строки чрезвычайно широко распространены.

Чтобы решить эту проблему, в Java 9 была введена оптимизация, которая позволяет выбрать одно из двух (в настоящее время) представлений на уровне строки. Каждую строку можно закодировать в Latin-1 (для западноевропейских языков) или в UTF-16 (исходное представление).

https://t.me/javalib

ПРИМЕЧАНИЕ Кодировка Latin-1 также известна по номеру своего стандарта ISO-8859-1. Не путайте с ISO-8851 — международным стандартом для определения содержания влаги в масле.

Во внутренней реализации представление строки изменилось на `byte[]`; строка из *n* символов представляется *n* байтами в кодировке Latin-1 или `n*2` байтами в UTF-16. Соответствующий код в `java.lang.String` выглядит так:

```
private final byte[] value;

/**
 * Идентификатор кодировки, используемой для кодирования байтов в
 * {@code value}. В этой реализации поддерживаются следующие значения:
 *
 * LATIN1
 * UTF16
 *
 * @implПримечание Виртуальная машина доверяет этому полю, и к нему применяется
 * свертка констант, если экземпляр String является константным.
 * Переопределение этого поля после конструирования создаст проблемы.
 */
private final byte coder;

static final byte LATIN1 = 0;
static final byte UTF16 = 1;
```

В зависимости от природы рабочей нагрузки это может обеспечить существенную экономию в распространенном случае Latin-1. С другой стороны, если приложение в основном работает на восточноазиатских языках (китайском, японском, корейском, вьетнамском), то это внутреннее изменение не даст никакой экономии.

На практике в приложениях, где используются западные языки, при переходе с Java 8 на 11 только одно это изменение может сэкономить до 30 и даже 40 % памяти в куче. А чем меньше куча, тем меньше контейнер, так что это может ощутимо сократить затраты на облачные вычисления в ваших приложениях.

Завершая обсуждение, скажем пару слов о том, как это изменение влияет на производительность. Этот пример наглядно демонстрирует, почему к анализу производительности следует относиться как к экспериментальной науке и измерять все от начала до конца. Две разные реализации `String` увеличивают размер выполняемого кода, потому что строковые операции теперь имеют дело с двумя разными реализациями — для Latin-1 и для UTF-16, так что программе приходится узнавать значение `coder` и переходить по той или иной ветви в зависимости от результата.

Однако ключевой вопрос производительности — в том, помогает ли дополнительный код, то есть оправдываются ли его преимущества теми затратами,

https://t.me/javalib

которые создает дополнительный «налог на сложность». Можно отметить такие преимущества компактных строк:

- уменьшается размер кучи;
- потенциально ускоряется сборка мусора;
- улучшается локализация кэша для строк Latin-1.

Также можно задаться вопросом, влияют ли дополнительные операции сравнения и ветвления на объем фактически выполняемого кода. Вспомните, что JIT-компилятор проводит много неочевидной оптимизации и может эффективно использовать время ожидания промахов кэша, так что дополнительные инструкции, необходимые для сокращения строк, могут фактически оказаться бесплатными.

> **ПРИМЕЧАНИЕ** В общем случае считать количество выполненных инструкций — не лучший подход к анализу производительности Java.

Эта система «сдержек и противовесов», влияние которых можно определить только в результате измерения крупномасштабных наблюдаемых характеристик, в точности соответствует модели быстродействия, которая обсуждалась в главе 7.

17.5.3. Общие контексты

Механизм общих контекстов (nestmates), описанный в документе JEP 181, по сути, упраздняет трюк, который использовался еще со времен Java 1.1 для работы с внутренними классами. Следующий пример показывает, какие изменения были внесены в байт-код для поддержки общих контекстов:

```
public class Outer {
    private int i = 0;

    public class Inner {
        public int i() {
            return i;
        }
    }
}
```

Если откомпилировать этот код, вы получите два отдельных файла класса — независимо от того, выполнялась ли компиляция в Java 8 или 17. Однако байт-код в этих двух случаях различается, и мы проанализируем эти различия с помощью `javap`. Вот байт-код для Java 8:

```
Compiled from "Outer.java"
public class Outer {
  private int i;

  public Outer();
```

https://t.me/javalib

```
    Code:
       0: aload_0
       1: invokespecial #2          // Method java/lang/Object."<init>":()V
       4: aload_0
       5: iconst_0
       6: putfield      #1          // Field i:I
       9: return

  static int access$000(Outer);  <-- Компилятор вставил этот связующий метод
    Code:
       0: aload_0
       1: getfield      #1          // Field i:I
       4: ireturn
}
```

Для внутреннего класса генерируется отдельный файл класса:

```
Compiled from "Outer.java"
public class Outer$Inner {
  final Outer this$0;

  public Outer$Inner(Outer);
    Code:
       0: aload_0
       1: aload_1
       2: putfield      #1         // Field this$0:LOuter;
       5: aload_0
       6: invokespecial #2         // Method java/lang/Object."<init>":()V
       9: return

  public int i();
    Code:
       0: aload_0
       1: getfield      #1         // Field this$0:LOuter;
       4: invokestatic  #3         // Method  <-- Здесь вызывается связующий метод
                                   // Outer.access$000:(LOuter;)I
       7: ireturn
}
```

Обратите внимание, что во внешний класс был добавлен *синтетический* метод доступа (или связующий метод) `access$000()`, чтобы предоставить доступ в пределах пакета к частному полю, к которому мы обращаемся из внутреннего класса. Теперь посмотрим, что происходит с байт-кодом, если перекомпилировать исходный код в Java 17 (или 11):

```
Compiled from "Outer.java"
public class Outer {
  private int i;

  public Outer();
    Code:
       0: aload_0
       1: invokespecial #2         // Method java/lang/Object."<init>":()V
       4: aload_0
```

https://t.me/javalib

```
       5: iconst_0
       6: putfield      #1         // Field i:I
       9: return
}
```

Синтетический метод доступа полностью исчез, зато посмотрите, как теперь выглядит внутренний класс:

```
Compiled from "Outer.java"
public class Outer$Inner {
  final Outer this$0;

  public Outer$Inner(Outer);
    Code:
       0: aload_0
       1: aload_1
       2: putfield      #1        // Field this$0:LOuter;
       5: aload_0
       6: invokespecial #2        // Method java/lang/Object."<init>":()V
       9: return

  public int i();
    Code:
       0: aload_0
       1: getfield      #1        // Field this$0:LOuter;
       4: getfield      #3        // Field Outer.i:I
       7: ireturn
}
```

Теперь код обращается к частному полю напрямую:

```
SourceFile: "Outer.java"
NestMembers:
  Outer$Inner
InnerClasses:
  public #6= #5 of #3;            // Inner=class Outer$Inner of class Outer
```

В Java 11 появилась концепция общих контекстов (nests), которые, по сути, обобщают существующее понятие вложенных классов. В предыдущих версиях Java, чтобы два класса могли использовать один и тот же контекст управления доступом, исходный код одного класса должен был физически размещаться внутри исходного кода другого класса. По новой схеме группа файлов классов может образовать общий контекст, члены которого совместно используют общий механизм управления доступом и имеют неограниченный прямой (а также рефлексивный) доступ друг к другу, в том числе и к частным членам.

> **ПРИМЕЧАНИЕ** Общие контексты внесли некоторые коррективы в понятие `private`, как было показано ранее, когда шла речь про `invokespecial`.

Это в самом деле небольшое изменение, но оно не только позволяет избавиться от былых «костылей» реализации, но также необходимо для дальнейшей эволюции платформы. Перейдем к более серьезным изменениям и обсудим один из

https://t.me/javalib

важнейших (и имеющих скверную репутацию) аспектов внутреннего строения JVM — класс `Unsafe`.

17.6. UNSAFE

Если какая-то функциональность или поведение на платформе Java кажутся «волшебными», скорее всего, это реализуется одним из трех механизмов: рефлексия, загрузка класса (включая сопутствующие преобразования кода) или `Unsafe`.

Опытный пользователь Java постарается разобраться во всех трех механизмах, даже если он прибегает к ним только при необходимости. Принцип «можно — еще не значит нужно» применим к проектированию программных продуктов в той же степени, как в любой другой области.

Из трех перечисленных механизмов `Unsafe`, пожалуй, является самым потенциально опасным (и самым мощным), потому что он позволяет делать то, что нельзя сделать никаким другим способом и что нарушает устоявшиеся правила платформы. Например, `Unsafe` позволяет коду на Java:

- Напрямую обращаться к аппаратным средствам процессора.
- Создать объект, не запуская его конструктор.
- Создать по-настоящему анонимный класс без обычной верификации.
- Вручную управлять памятью за пределами кучи.
- Выполнять много других «невозможных» операций.

В Java 8 класс `Unsafe` (`sun.misc.Unsafe`) отчетливо предупреждает о своей природе — не только именем класса, но и именем пакета, в котором он находится. Пакет `sun.misc` — это внутреннее расположение, которое тесно связано с реализацией и к которому лучше не обращаться напрямую из кода на Java.

> **ПРИМЕЧАНИЕ** В Java 9 и более поздних версиях опасность `Unsafe` выражена еще более явно, потому что функциональность перемещена в модуль с именем `jdk.unsupported`.

Официально предполагается, что библиотеки не должны напрямую связываться с этими подробностями реализации. В подтверждение этой позиции специалисты, которые сопровождают Java, традиционно отмечают, что если конечные пользователи нарушают правила и вторгаются во внутренние подробности реализации, они делают это на свой страх и риск.

Однако неудобная правда заключается в том, что API, даже будучи неподдерживаемым, получил широкое распространение среди авторов библиотек. Он не входит в официальный стандарт Java и фактически стал свалкой нестандартных, хотя и необходимых платформенных средств различной степени безопасности.

Рассмотрим классический пример применения `Unsafe`: использование аппаратной функции сравнения с обменом (CAS, Compare And Swap). Эта возможность

https://t.me/javalib

есть практически во всех современных процессорах, однако, как всем известно, она не входит в модель памяти Java (JMM).

Вспомните класс `Account`, который упоминался в главе 5. По техническим причинам в этом разделе мы будем предполагать, что баланс представлен значением `int` вместо `double`. Интерфейс `Account` определяется так:

```
public interface Account {
    boolean withdraw(int amount);

    void deposit(int amount);

    int getBalance();

    boolean transferTo(Account other, int amount);
}
```

Мы реализуем этот интерфейс двумя разными способами. Сначала будем придерживаться правил и воспользуемся синхронизацией. Два метода интерфейса в `SynchronizedAccount` выглядят так:

```
public class SynchronizedAccount implements Account {
    private int balance;

    public SynchronizedAccount(int openingBalance) {
        balance = openingBalance;
    }

    @Override
    public int getBalance() {
        synchronized (this) {
            return balance;
        }
    }

    @Override
    public void deposit(int amount) {
        // Проверить, что amount > 0, если нет - возбудить исключение
        synchronized (this) {
            balance = balance + amount;
        }
    }
```

Сравните с атомарной реализацией с `Unsafe`, в которой значительно больше шаблонного кода из-за того, что обращаться к классу `Unsafe` приходится рефлексивно:

```
public class AtomicAccount implements Account {
    private static final Unsafe unsafe;
    private static final long balanceOffset;

    private volatile int balance = 0;
    static {
```

Наша копия объекта Unsafe

Числовое значение смещения указателя на поле balance относительно начала объекта

Настоящее поле balance

https://t.me/javalib

```
        try {
            Field f = Unsafe.class.getDeclaredField("theUnsafe");
            f.setAccessible(true);
            unsafe = (Unsafe) f.get(null);
            balanceOffset = unsafe.objectFieldOffset(
                                AtomicAccount.class
                                  .getDeclaredField("balance"));
        } catch (Exception ex) { throw new Error(ex); }
    }

    public AtomicAccount(int openingBalance) {
        balance = openingBalance;
    }

    @Override
    public double getBalance() {
        return balance;
    }

    @Override
    public void deposit(int amount) {
        // Проверить, что amount > 0, если нет - возбудить исключение
        unsafe.getAndAddInt(this,
                            balanceOffset, amount);
    }

    // ...
```

Рефлексивно ищет объект Unsafe

Вычисляет смещение указателя

Небезопасное чтение баланса — без блокировок

Обновляет balance методом сравнения с обменом

В этом примере выполняется ряд операций, которые, казалось бы, невозможны в Java. Сначала вычисляется смещение указателя — смещение, по которому хранится поле `value` относительно начала объектов `AtomicAccount`. Не существует последовательности инструкций байт-кода JVM, которая бы предоставляла такую возможность, — это может делать только низкоуровневый код, который напрямую обращается к внутренним структурам данных JVM. В данном случае для этого применяется метод `objectFieldOffset()` объекта `Unsafe`.

Далее, с `balance` выполняется атомарное сложение без блокировки. Это невозможно в рамках JMM (Java Memory Model), потому что `volatile` позволяет выполнить только одну операцию (чтение *или* запись), а сложение требует как чтения, так и записи. Чтобы понять, как оно осуществляется, рассмотрим код метода `getAndAddInt()` из `Unsafe`:

```
public final int getAndAddInt(Object o, long offset, int delta) {
    int v;
    do {
        v = getIntVolatile(o, offset);
    } while (!compareAndSetInt(o, offset, v, v + delta));
    return v;
}
```

Программное нестабильное обращение

Низкоуровневая операция сравнения с обменом

В этом коде мы задаем режим обращения к памяти (в данном случае `volatile`), вместо того чтобы он устанавливался в зависимости от того, как объявлена

https://t.me/javalib

переменная. Мы также напрямую обращаемся к памяти по смещению указателя, минуя обращение к полю, — это еще одно действие, которое невозможно в обычном коде на Java.

ПРИМЕЧАНИЕ Реализация в JDK 11 использует «внутренний объект `Unsafe`» из соображений инкапсуляции — здесь приведен именно этот код.

Логика сравнения с обменом выглядит так: для объекта `o`, содержащего поле с заданным смещением `offset` от начала заголовка объекта, за одну операцию процессора выполнить следующие действия:

1. Сравнить текущее состояние области памяти (4 байта) с целым числом `v`.
2. Если значение `v` совпадает, заменить его на значение `v + delta`.
3. Если замена была выполнена успешно, вернуть `true`, иначе вернуть `false`.

Замена может завершиться неудачей, если поток, который активно выполняется на другом процессоре, обновит содержимое этой области памяти между нестабильным чтением и сравнением с обменом. В этом случае метод `compareAndSetInt()` вернет `false`, а цикл `do-while` сделает новую попытку.

Таким образом, эти типы операций свободны от блокировки, но не от циклов. Поля с высокой частотой обращений, с которыми работают многие потоки, могут потребовать нескольких проходов цикла, прежде чем атомарное сложение завершится успехом, но зато пропадает риск «потерянного обновления».

ПРИМЕЧАНИЕ Если вы внимательно изучите код JDK, то увидите, что эта реализация очень близка к тому, как JDK на самом деле поступает с `AtomicInteger`.

Для полноты информации также рассмотрим реализацию `withdraw()`:

```
@Override
public boolean withdraw(int amount) {
    // Проверить, что amount > 0, если нет - возбудить исключение
    var currBal = balance;  <-- Нестабильное чтение balance
    var newBal = currBal - amount;
    if (newBal >= 0) {
        if (unsafe.compareAndSwapInt(this,
                                     balanceOffset,
                                     currBal, newBal)) {
            return true;
        }
    }
    return false;
}
```

Пытается обновить balance низкоуровневой операцией сравнения с обменом

Этот случай немного отличается от предыдущего, потому что для обновления баланса напрямую используется низкоуровневый API. Это необходимо из-за ограничения, согласно которому баланс не должен стать отрицательным. Для этого требуются дополнительные операции, такие как сравнение с `newBal`.

https://t.me/javalib

Для внесения средств можно использовать API более высокого уровня, который выполняет попытки в цикле, пока операция не будет завершена успешно. Однако это удается потому, что на счет всегда можно внести деньги независимо от его состояния. Если бы мы использовали такой прием для снятия средств, то попытки снятия могли бы повторяться бесконечно, если на счете недостаточно средств для выполнения запроса.

Вместо этого мы применяем алгоритм, в котором делается одна попытка, и если операция сравнения с обменом не удается, то все снятие тоже завершается неудачей. В результате предотвращается ситуация гонки, если две операции снятия попытаются получить одни и те же средства, — однако появляется побочный эффект: попытка снятия, которая должна завершиться успехом, может ошибочно завершиться неудачей из-за того, что в другом потоке выполняется внесение (которое изменяет баланс после нестабильного чтения).

ПРИМЕЧАНИЕ Можно включить в код снятия цикл `for`, чтобы снизить вероятность ошибочных отказов, но такой цикл должен гарантированно быть конечным.

Как показывают тесты, два этих подхода существенно различаются по производительности: реализация с `Unsafe` на современном оборудовании работает приблизительно в 2–3 раза быстрее. Однако это не повод применять такие решения в коде, предназначенном для конечных пользователей. Как упоминалось ранее, многие (возможно, практически все) современные фреймворки уже используют `Unsafe`. Маловероятно, что программировать непосредственно через `Unsafe` будет более эффективно, чем использовать средства выбранного вами фреймворка.

Что еще важнее, такой подход нарушает правила спецификации Java для пользовательского кода: не использовать внутренние средства, которые не обязательно следуют правилам. В следующем разделе мы поговорим о том, как последние версии Java попытались сократить неподдерживаемый API и заменить его полностью поддерживаемыми альтернативами.

17.7. ЗАМЕНА UNSAFE ПОДДЕРЖИВАЕМЫМИ API

Напомним, что в главе 2 были представлены модули Java. Этот механизм инкапсуляции предоставляет формализованную функциональность экспортирования и не позволяет вызывать код из внутренних пакетов. Как это влияет на класс `Unsafe` и код, в котором он используется?

Если учесть, сколько фреймворков и библиотек зависят от `Unsafe`, они не смогут обновиться до версий Java, которые не поддерживают рефлексивный доступ к `Unsafe`.

ПРИМЕЧАНИЕ К объекту `Unsafe` приходится обращаться рефлексивно, потому что платформа уже блокирует прямой доступ к нему для кода, который не входит в JDK.

https://t.me/javalib

В Java 11 и выше система модулей предоставляет модуль `jdk.unsupported`. Его объявление выглядит так:

```
module jdk.unsupported {
    exports sun.misc;
    exports sun.reflect;
    exports com.sun.nio.file;

    opens sun.misc;
    opens sun.reflect;
}
```

Этот код предоставляет доступ любому приложению, которое явно зависит от модуля `unsupported`, и (что очень важно) также предоставляет неограниченный рефлексивный доступ к `sun.misc` — пакету, содержащему `Unsafe`. И хотя это помогает перевести `Unsafe` в форму, совместимую с модулями, возникает законный вопрос: долго ли будет поддерживаться этот «черный ход»?

Класс `Unsafe` не останется с нами навсегда: настоящее решение заключается в том, что разработчики платформы Java создают новые, поддерживаемые API, которые могут заменить «безопасные» средства `sun.misc.Unsafe`. А когда авторы библиотек Java смогут полноценно мигрировать на новые API, модуль `jdk.unsupported` будет закрыт или удален.

> **ПРИМЕЧАНИЕ** Закрытие `Unsafe` затронет всех пользователей *очень* широкого спектра фреймворков: не будет преувеличением утверждать, что практически каждое нетривиальное приложение в экосистеме Java в той или иной степени косвенно зависит от Unsafe.

Один из главных API `Unsafe`, которые необходимо удалить, — режимы программного доступа к памяти, такие как `getIntVolatile()`. Его заменит API `VarHandle`, о котором пойдет речь в следующем разделе.

17.7.1. VarHandle

API `VarHandle`, который появился в Java 9, старается расширить концепцию дескрипторов методов, чтобы обеспечить похожую функциональность для полей и обращений к памяти. Вспомните, о чем говорилось в главе 5: в JMM предоставляются только два режима обращения к памяти: обычный и нестабильный (`volatile`), в котором данные заново считываются из основной памяти, при этом кэши процессора игнорируются, а выполнение приостанавливается до завершения чтения. Мало того, язык Java позволяет представить эти режимы только на уровне полей. Все обращения осуществляются в обычном режиме, если только поле не объявлено явно с модификатором `volatile`; в этом случае *весь* доступ к нему происходит в нестабильном режиме. Но что делать, если этих мер недостаточно для современных приложений?

https://t.me/javalib

ПРИМЕЧАНИЕ Нестабильность — это одна из языковых фикций Java. Память — это всего лишь память, и не бывает отдельных банков памяти, которые состояли бы из модулей со стабильным и нестабильным доступом.

Одной из важных целей `VarHandle` была возможность использовать новые механизмы обращения к памяти, то есть предоставить поддерживаемую (и более совершенную) замену `Unsafe` — например, альтернативные средства выполнения сравнения с обменом или нестабильный доступ общего вида.

Чтобы увидеть, как работает `VarHandle`, рассмотрим небольшой пример. Он показывает, как с помощью `VarHandle` можно было бы заменить `Unsafe` в классе банковского счета:

```
public class VHAccount implements Account {
    private static final VarHandle vh;
    private volatile int balance = 0;

    static {
        try {
            var l = MethodHandles.lookup();
            vh = l.findVarHandle(VHAccount.class,
                                 "balance", int.class);
        } catch (Exception ex) { throw new Error(ex); }
    }

    @Override
    public void deposit(int amount) {
        // Проверить, что amount > 0, если нет - возбудить исключение
        vh.getAndAdd(this, amount);
    }

    // …
}
```

Создает объект Lookup

Получает объект VarHandle для balance и кэширует его

Обращается к полю с помощью VarHandle, применяя семантику нестабильной памяти

Этот код функционально эквивалентен версии с `Unsafe`, но в нем используются только полностью поддерживаемые API.

Использование `MethodHandles.Lookup` — это важное изменение. В отличие от рефлексии, которая не может обращаться к частным полям без `setAccessible()`, у объекта `Lookup` есть те же права, что и у вызывающего контекста, включая доступ к частному полю `balance`.

Переход от рефлексии к дескрипторам методов и полей означает, что теперь из неподдерживаемого API можно исключить многие методы, которые были в `Unsafe` в Java 8, в том числе:

- `compareAndSwapInt()`
- `compareAndSwapLong()`
- `compareAndSwapObject()`

https://t.me/javalib

В VarHandle есть аналоги этих методов, а также полезные методы доступа. Кроме того, класс содержит методы чтения и записи для примитивных типов и объектов как в обычном, так и в нестабильном режиме доступа, а также инструменты, с помощью которых можно реализовывать эффективные методы обновления, такие как:

- getAndAddInt()
- getAndAddLong()
- getAndSetInt()
- getAndSetLong()
- getAndSetObject()

Другая важная цель VarHandle — обеспечить низкоуровневый доступ к новым режимам упорядоченных обращений к памяти, которые доступны в JDK 9 и выше. Эти новые режимы затрагивают обработку барьеров при конкурентном выполнении и также требуют некоторых обновлений в JMM.

В целом можно отметить определенный прогресс в создании альтернатив для API Unsafe, который считался фактическим стандартом. Например, в дополнение к VarHandle функциональность getCallerClass() из Unsafe теперь доступна в API обхода стека, который определен в JEP 259 (https://openjdk.java.net/jeps/259). Тем не менее в этой области остается еще много работы.

17.7.2. Скрытые классы

Скрытые классы описаны в JEP 371 (https://openjdk.java.net/jeps/371). Эта внутренняя функциональность предназначена для разработчиков платформы и фреймворков, и ее задача — предоставить поддерживаемый API для одного из самых типичных сценариев использования Unsafe: программистам часто нужно создавать «на ходу» классы, к которым другие классы не могут обращаться напрямую (но с которыми можно работать косвенно).

Иногда такие классы называются *анонимными*, и в Unsafe их можно создавать методом defineAnonymousClass(). Однако этот термин сбивает разработчиков с толку, потому что в контексте обычного кода приложений на Java он обозначает вложенную реализацию некоего интерфейса, который объявляет свой статический тип интерфейсом, как в следующем примере:

```
public class Scratch {
    public void foo() {
        Runnable r = new Runnable() {
            @Override
            public void run() {
                System.out.println("До появления лямбда-выражений можно было
                                    только так!");
            }
        };
    }
}
```

https://t.me/javalib

Обычно это называется «анонимной реализацией `Runnable`», хотя такие классы на самом деле не анонимны: компилятор генерирует класс с именем вида `Scratch$1`, то есть полноценный класс Java. И хотя этого имени нет в исходном коде Java, можно найти класс по этому имени, получить к нему доступ с помощью рефлексии и после этого использовать как любой другой класс.

Скрытый класс тоже не является полностью анонимным: у него есть имя, которое можно узнать, если напрямую вызвать `getName()` для его объекта `Class`. Это имя также отображается в других местах, включая диагностические программы, JVMTI (JVM Tool Interface) или события JFR (JDK Flight Recorder). Однако скрытые классы нельзя найти с помощью загрузчика классов или любым другим способом, которым осуществляется поиск обычного класса, включая рефлексию (например, через `Class.forName()`).

Имена скрытых классов назначаются таким образом, чтобы явно поместить их в другое пространство имен, отделив от имен обычных классов. В результате имена имеют довольно своеобразную форму, из-за которой класс становится фактически невидимым для всех остальных классов.

Дело в том, что у обычных классов JVM, как правило, есть две формы имени: двоичное имя (`com.acme.Gadget`), которое возвращается, если вызвать `getName()` для объекта класса, и внутренняя форма (`com/acme/Gadget`). Но имена скрытых классов назначаются по другой схеме: при вызове `getName()` для объекта класса, который относится к скрытому классу, возвращается имя вида `com.acme.Gadget/1234`. Это и не двоичное имя, и не внутренняя форма, причем любая попытка создать обычный класс с таким именем завершится неудачей. Рассмотрим небольшой пример создания скрытого класса:

```
var fName = "/Users/ben/projects/books/resources/Ch15/ch15/Concat.class";
var buffy = Files.readAllBytes(Path.of(fName));
var lookup = MethodHandles.lookup();
var hiddenLookup = lookup.defineHiddenClass(buffy, true);
var klazz = hiddenLookup.lookupClass();
System.out.println(klazz.getName());
```

Одно из преимуществ того, что скрытые классы именуются по такой схеме (и выделяются в отдельную категорию), состоит в том, что на них не распространяется стандартный жесткий контроль, которым управляет механизм загрузки классов JVM. Такой подход соответствует общему принципу, по которому скрытые классы предназначены для авторов фреймворков и других разработчиков, которым нужна функциональность, выходящая за рамки обычных сверхнадежных проверок, которые применяются к обычным классам Java.

ПРИМЕЧАНИЕ Скрытые классы появились в Java 15, а в Java 11 они недоступны.

Помимо прочего, JEP 371 объявляет устаревшим метод `defineAnonymousClass()` из `Unsafe`, рассчитывая вовсе упразднить его в какой-нибудь будущей версии. Это чисто внутреннее изменение: не стоит полагать, что от появления

https://t.me/javalib

внутренних классов язык программирования Java как-то изменится (по крайней мере в обозримом будущем). Тем не менее реализации таких классов, как `LambdaMetaFactory`, `StringConcatFactory`, и других «гибких фабричных методов» уже можно обновлять, задействовав новые API.

ИТОГИ

- Java предоставляет средства для интроспекции на стадии выполнения, к которым значительно труднее получить доступ в таких языках, как C++. Вот основные из этих средств:
  - рефлексия;
  - дескрипторы методов;
  - `invokedynamic`;
  - `Unsafe`.

https://t.me/javalib

18 Будущее Java

В ЭТОЙ ГЛАВЕ

- ✓ Проект Amber
- ✓ Проект Panama
- ✓ Проект Loom
- ✓ Проект Valhalla
- ✓ Java 18

В этой главе рассказывается о том, как будет развиваться язык и платформа Java после выпуска Java 17: о будущих обновлениях, которые еще не реализованы. Новые направления в развитии языка и платформы управляются JEP, но там речь идет о внедрении конкретных возможностей. На более высоком уровне внутри OpenJDK существует несколько больших долгоиграющих проектов по внедрению крупных изменений, которые в настоящее время находятся в состоянии активной разработки и будут поставлены в ближайшие годы.

Сначала мы по очереди рассмотрим каждый из этих проектов, а затем и Java 18. Начнем с проекта Amber, где уделим внимание поиску по шаблону и объясним, почему это столь важное обновление.

https://t.me/javalib

18.1. ПРОЕКТ AMBER

Из всех текущих крупных проектов в составе OpenJDK проект Amber находится ближе всего к завершению. К его преимуществам относится то, что он относительно понятен в контексте повседневных задач разработчика. Из кодекса проекта:

> Цель проекта Amber — исследовать и культивировать небольшие аспекты языка Java, ориентированные на производительность...
>
> *Project Amber,* https://openjdk.java.net/projects/amber/

Основные цели проекта:

- автоматическое выведение типов локальных переменных (завершено);
- выражения `switch` (завершено);
- записи (завершено);
- запечатанные типы (завершено);
- поиск по шаблону.

Как видите, многие из этих возможностей были опубликованы в Java 17 — и оказались чрезвычайно полезными!

Последний значительный компонент Amber, который остается незавершенным, — поиск по шаблону. Как рассказывалось в главе 3, он реализуется поэтапно, и первым этапом было использование шаблонов типов в `instanceof`. Также мы рассматривали предварительную версию шаблонов в разделе, посвященном `switch`.

Разумно ожидать, что шаблоны в `switch` будут проходить тот же жизненный цикл, что и другие компоненты Amber: первая предварительная версия, затем вторая и после этого публикация в качестве стандартной функциональности.

На будущее запланированы новые предложения JEP. Завершение спецификации поиска по шаблону не единственное, что нас ожидает; в язык предполагается добавить и другие разновидности шаблонов. А это значит, что в модели, когда новая LTS-версия выпускается через каждые два года, у всей предварительной функциональности, которая была изначально опубликована в Java 18 и 19, будет время созреть к следующей LTS-версии Java 21 в сентябре 2023 года.

Например, вы уже видели, как эффективно используются запечатанные типы в текущей, предварительной версии поиска по шаблону. Без запечатанных типов даже уже существующая форма поиска по шаблону была бы не такой полезной. Кроме того, некоторые важные сценарии применения записей в шаблонах пока еще не реализованы. В частности, *декомпозиционные шаблоны* (deconstruction patterns) позволят разбивать запись на компоненты как часть шаблона.

https://t.me/javalib

ПРИМЕЧАНИЕ Если вы программировали на Python, JavaScript или других языках, возможно, вы знакомы с декомпозицией. В Java используется похожая идея, но она подчиняется действующей системе типов Java.

Это возможно, потому что записи определяются своей семантикой: запись — это буквально совокупность своих составляющих. Таким образом, если запись можно построить, объединив ее компоненты, это значит, что ее также можно разложить на компоненты без семантических последствий.

На момент написания книги эта функциональность еще не вошла ни в основную ветвь разработки JDK, ни даже в репозитории JDK, относящиеся к Amber. Предполагается, что синтаксис должен выглядеть так:

```
FXOrder order = // ...

// ПРЕДУПРЕЖДЕНИЕ: это предварительный синтаксис!!!

var isMarket = switch (order) {
    case MarketOrder(int units, CurrencyPair pair, Side side,
                     LocalDateTime sent, boolean allOrNothing) -> true;
    case LimitOrder(int units, CurrencyPair pair, Side side,
                     LocalDateTime sent, double price, int ttl) -> false;
};
```

Обратите внимание, что в этом коде явно фигурируют типы компонентов записи. Однако не исключено, что финальная версия позволит компилятору автоматически выводить типы.

Также перспективно выглядит возможность декомпозиции массивов, потому что они ведут себя как контейнеры для элементов без дополнительной семантики. Декомпозиция может выглядеть примерно так:

```
// ПРЕДУПРЕЖДЕНИЕ: это предварительный синтаксис!!!

if (o instanceof String[] { String s1, String s2, ... }){
    System.out.println(s1 + s2);
}
```

Обратите внимание, что в обоих примерах мы не объявляли связывание для контейнера, будь то запись или массив.

Также стоит упомянуть о том, как на эту функциональность влияет механизм сериализации Java. В общем случае сериализация Java создает проблемы, потому что она нарушает некоторые базовые принципы того, как должна работать инкапсуляция в Java.

> Сериализация фактически предоставляет невидимый, но открытый конструктор и невидимый, но открытый набор методов доступа к внутреннему состоянию.
>
> *Брайан Гетц*

https://t.me/javalib

К счастью, и записи, и массивы очень просты: это всего лишь прозрачные носители своего содержимого, а значит, работая с ними, нет необходимости разбираться во всех тонкостях механизма сериализации. Чтобы сериализовать и десериализовать записи, всегда можно воспользоваться открытым API и каноническим конструктором. На этом основании даже выдвигались весьма радикальные предложения: например, отказаться от механизма сериализации полностью или частично и расширить декомпозицию на некоторые (а возможно, даже на все) классы Java.

В целом проект Amber придерживается примерно такой позиции: если вы знакомы с этой функциональностью из других языков программирования — отлично. А если не знакомы, все равно не беспокойтесь: она проектировалась так, чтобы хорошо интегрироваться с уже известным вам языком Java и чтобы вам было проще начать использовать ее в своем коде.

И малые, и большие нововведения функциональности Amber могут положительно отразиться на вашем коде. Если вы начнете пользоваться ими, то скоро убедитесь, что они приносят реальную пользу в разработке. А теперь пора переходить к следующему крупному проекту, которому присвоено кодовое название Panama.

18.2. ПРОЕКТ PANAMA

На странице проекта Panama его цель описывается так:

> Совершенствовать и расширять связи между виртуальной машиной Java и четко определенными, но «сторонними» (не принадлежащими Java) API, включая многие интерфейсы, часто используемые программистами на C.
>
> *Project Panama,* https://openjdk.org/projects/panama/

Название проекта выбрано по аналогии с Панамским перешейком: подобно тому, как эта узкая полоска земли соединяет два больших массива суши, проект Panama соединяет JVM и низкоуровневый код. В проект входят документы JEP из двух основных областей:

- API внешних функций и памяти;
- API векторов.

В этом разделе рассматривается только API внешних функций, а API векторов еще не готов для полноценного обсуждения — позже в этой главе мы объясним почему.

18.2.1. API внешних функций и памяти

Хотя интерфейс JNI (Java Native Interface) для взаимодействия с внешним кодом существует еще со времен Java 1.1, примерно с тех же времен его критикуют за серьезные недостатки:

https://t.me/javalib

- С JNI связано много формальностей и лишних артефактов.
- JNI по-настоящему хорошо взаимодействует только с библиотеками, которые написаны на C и C++.
- JNI никак не пытается автоматизировать соответствия между системами типов Java и C.

Проблема лишних артефактов хорошо понятна разработчикам: как и API Java для низкоуровневых методов, JNI требует заголовочного файла C (`.h`), который формируется на основе Java API, и файла реализации C, который обращается с вызовом к внешней библиотеке. Другие проблемы менее очевидны: например, тот факт, что с помощью низкоуровневых методов нельзя вызвать функцию на языке, в котором соглашения вызова отличаются от тех, что заложены в основу JVM.

За годы, прошедшие с момента появления JNI, предпринимались попытки предоставить более удачную альтернативу, например JNA. Однако в других языках (не работающих на JVM) взаимодействие с низкоуровневым кодом поддерживалось значительно лучше. Например, репутация Python как хорошего языка для машинного обучения в значительной степени опирается на то, что в нем просто упаковывать низкоуровневые библиотеки и получать доступ к ним из кода на Python.

API внешних функций Panama пытается заполнить этот пробел, обеспечивая прямую поддержку Java в следующих областях:

- Выделение внешней памяти.
- Манипуляции со структурированной внешней памятью.
- Управление жизненным циклом внешних ресурсов.
- Вызов внешних функций.

API находится в пакете `jdk.incubator.foreign` одноименного модуля `jdk.incubator.foreign`. Он построен на основе дескрипторов `MethodHandle` и `VarHandle`, которые встречались в главе 17.

> **ПРИМЕЧАНИЕ** В Java 17 API внешних функций содержится в инкубационном модуле — этот механизм обновления Java рассматривался в главе 1. Чтобы примеры из этого раздела работали, необходимо явно добавить инкубационный модуль в путь к модулям.

К первой части API относятся такие классы, как `MemorySegment`, `MemoryAddress` и `SegmentAllocator`, — они позволяют выделять память и выполнять операции с ней вне кучи. Предполагается, что это станет более совершенной заменой API `ByteBuffer` и `Unsafe`. API внешних функций должен избежать недостатков `ByteBuffer`, которые, в частности, сказываются на производительности и возникают из-за того, что размер сегментов ограничен 2 Гбайт, а API не

https://t.me/javalib

проектировался специально для работы за пределами кучи. В то же время API внешних функций должен быть безопаснее, чем `Unsafe`, который предоставляет практически неограниченный доступ к памяти, из-за чего ошибки очень легко приводят к отказу JVM.

> **ПРИМЕЧАНИЕ** В оставшейся части раздела мы предполагаем, что вы уже знакомы с основными понятиями языка C и знаете, как строить программы на C/C++ из исходного кода и как в этих языках устроены фазы компиляции, компоновки и т. д.

Посмотрим, как работают эти средства. Для этого вам понадобится загрузить версию раннего доступа проекта Panama по адресу https://jdk.java.net/panama/. Хотя инкубационные модули входят в JDK 17, в нем нет важной утилиты `jextract`, которая нам здесь понадобится.

Когда вы установите Panama, проверьте его командой `jextract -h`. Результат должен выглядеть примерно так:

```
WARNING: Using incubator modules:
         jdk.incubator.jextract, jdk.incubator.foreign
Non-option arguments:
[String] -- header file

Option                          Description
------                          -----------
-?, -h, --help                  вывести справку
-C <String>                     передать аргумент для clang
-I <String>                     указать путь к файлам include
-d <String>                     указать, где размещать сгенерированные файлы
--dump-includes <String>        сбросить включенные символы в указанный файл
--header-class-name <String>    имя заголовочного файла
--include-function <String>     имя включаемой функции
--include-macro <String>        имя включаемого макроса констант
--include-struct <String>       имя включаемого определения структуры
--include-typedef <String>      имя включаемого определения типа
--include-union <String>        имя включаемого определения объединения
--include-var <String>          имя включаемой глобальной переменной
-l <String>                     указать библиотеку
--source                        генерировать исходный код на Java
-t, --target-package <String>   целевой пакет для указанных заголовочных файлов
```

В нашем примере будет использоваться простая библиотека для работы с форматом PNG, написанная на C, — LibSPNG (https://libspng.org/).

Пример: LibSPNG

Для начала мы с помощью утилиты `jextract` получим набор базовых пакетов Java, которые будем использовать в примере. Вот соответствующий синтаксис командной строки:

```
$ jextract --source -t <целевой пакет Java> -l <имя библиотеки> \
    -I <путь к /usr/include> <путь к заголовочному файлу>
```

https://t.me/javalib

На Mac команда будет выглядеть примерно так:

```
$ jextract --source -t org.libspng \
  -I /Applications/Xcode.app/Contents/Developer/Platforms/MacOSX.platform/ \
      Developer/SDKs/MacOSX.sdk/usr/include \
  -l spng /Users/ben/projects/libspng/spng/spng.h
```

В зависимости от точной версии заголовочного файла, который вы указали, эта команда может выдать ряд предупреждений, но если она выполнится успешно, то в текущем каталоге будет создана структура каталогов с большим количеством классов Java, объединенных в пакет `org.libspng`, — к нему позднее можно будет обращаться из нашей программы на Java.

Также нам понадобится создать совместно используемый объект, с которым программа будет компоноваться при запуске. Для этого лучше всего следовать инструкциям по сборке, которые включены в проект: http://mng.bz/v6dJ.

В процессе установки в проекте локально генерируется файл libspng.dylib, который устанавливается в общем системном каталоге. При запуске проекта убедитесь в том, что файл входит в один из путей, которые перечислены в системном свойстве `java.library.path`, или напрямую включите ваш каталог в значение этого свойства. На Mac для нашего примера по умолчанию используется каталог ~/Library/Java/Extensions/. После того как код сгенерирован, а библиотека установлена, можно переходить к программированию на Java.

Цель проекта Panama — предоставить статические методы Java, имена которых (и версии Java для типов параметров) соответствуют символическим именам библиотеки C, которую мы собираемся использовать для компоновки. Таким образом, символические имена в сгенерированном коде Java будут составлены по правилам C и не будут похожи на имена Java.

Для программиста Java все выглядит так, будто он вызывает функции C напрямую (насколько это возможно). На самом деле «за кулисами» Panama занимается «черной магией» и применяет такие средства, как дескрипторы методов, чтобы замаскировать сложность. В обычных обстоятельствах большинству разработчиков не приходится беспокоиться о том, как конкретно работает Panama.

В качестве примера рассмотрим программу, которая с помощью библиотеки C читает основные данные из файла PNG. Код будет оформлен в виде полноценной модульной сборки. Дескриптор модуля module-info.java выглядит так:

```
module wgjd.png {
  exports wgjd.png;

  requires jdk.incubator.foreign;
}
```

Мы подключаем пакет `org.libspng`, который автоматически сгенерирован по коду C, и единственный экспортируемый пакет wgjd.png. Исходный код состоит из одного файла, содержимое которого мы приводим полностью, потому что

https://t.me/javalib

команды импортирования и другие подробности важны для понимания происходящего:

```
package wgjd.png;

import jdk.incubator.foreign.MemoryAddress;
import jdk.incubator.foreign.MemorySegment;
import jdk.incubator.foreign.SegmentAllocator;
import org.libspng.spng_ihdr;

import static jdk.incubator.foreign.CLinker.toCString;
import static jdk.incubator.foreign.ResourceScope.newConfinedScope;
import static org.libspng.spng_h.*;

public class PngReader {
    public static void main(String[] args) {
        if (args.length < 1) {
            System.err.println("Usage: pngreader <fname>");
            System.exit(1);
        }

        try (var scope = newConfinedScope()) {
            var allocator = SegmentAllocator.ofScope(scope);

            MemoryAddress ctx = spng_ctx_new(0);
            MemorySegment ihdr = allocator.allocate(spng_ihdr.$LAYOUT());

            spng_set_crc_action(ctx, SPNG_CRC_USE(),
                                     SPNG_CRC_USE());

            int limit = 1024 * 1024 * 64;
            spng_set_chunk_limits(ctx, limit, limit);

            var cFname = toCString(args[0], scope);
            var cMode = toCString("rb", scope);
            var png = fopen(cFname, cMode);
            spng_set_png_file(ctx, png);

            int ret = spng_get_ihdr(ctx, ihdr);

            if (ret != 0) {
                System.out.println("spng_get_ihdr() error: " +
                                   spng_strerror(ret));
                System.exit(2);
            }

            final String colorTypeMsg;
            final byte colorType = spng_ihdr.color_type$get(ihdr);

            if (colorType ==
                    SPNG_COLOR_TYPE_GRAYSCALE()) {
                colorTypeMsg = "grayscale";
            } else if (colorType ==
```

Классы Panama для управления памятью в стиле C

Обертка Java для функции C в spng.h

Читает данные блоками по 64 Мбайт

Копирует содержимое строки Java в строку C

Обертка Java для функции из стандартной библиотеки C

Обертка Java для константы C

https://t.me/javalib

```
                    SPNG_COLOR_TYPE_TRUECOLOR()) {
                colorTypeMsg = "truecolor";
            } else if (colorType ==
                    SPNG_COLOR_TYPE_INDEXED()) {
                colorTypeMsg = "indexed color";
            } else if (colorType ==
                    SPNG_COLOR_TYPE_GRAYSCALE_ALPHA()) {
                colorTypeMsg = "grayscale with alpha";
            } else {
                colorTypeMsg = "truecolor with alpha";
            }

            System.out.println("File type: " + colorTypeMsg);
        }
    }
}
```

Обертка Java для константы C

Для сборки используется следующий сценарий Gradle:

```
plugins {
  id("org.beryx.jlink") version("2.24.2")
}

repositories {
  mavenCentral()
}

application {
  mainModule.set("wgjd.png")
  mainClass.set("wgjd.png.PngReader")
}

java {
  modularity.inferModulePath.set(true)
}

sourceSets {
  main {
    java {
      setSrcDirs(listOf("src/main/java/org",
                        "src/main/java/wgjd.png"))
    }
  }
}

tasks.withType<JavaCompile> {
  options.compilerArgs = listOf()
}

tasks.jar {
  manifest {
    attributes("Main-Class" to application.mainClassName)
  }
}
```

https://t.me/javalib

Этот сценарий можно выполнить такой командой:

```
$ java --add-modules jdk.incubator.foreign \
    --enable-native-access=ALL-UNNAMED \
    -jar build/libs/Panama.jar <FILENAME>.png
```

В результате программа выводит основные метаданные графического файла.

Работа с низкоуровневой памятью в Panama

Один из ключевых вопросов при работе с памятью — срок жизни низкоуровневой памяти. В C нет сборщика мусора, поэтому всю память нужно выделять и освобождать вручную. Конечно, эти операции сопряжены с высоким риском ошибок и к тому же выглядят совершенно неестественно для программиста на Java.

Чтобы обойти эту проблему, Panama предоставляет несколько классов, которые служат как дескрипторы Java, обеспечивающие доступ к операциям языка C по управлению памятью. Важнейший из этих классов — `ResourceScope`, который позволяет обеспечить детерминированное освобождение ресурсов. Он используется по обычной для Java схеме `try`-с-ресурсами. Например, в предыдущем примере для низкоуровневой работы с памятью использовался срок жизни с лексической областью видимости:

```
try (var scope = newConfinedScope()) {
    var allocator = SegmentAllocator.ofScope(scope);

    // ...
}
```

Объект `allocator` — это экземпляр реализации интерфейса `SegmentAllocator`. Он создается из области видимости с помощью фабричного метода, а уже на основе `allocator` можно создавать объекты `MemorySegment`.

Объекты, которые реализуют интерфейс `MemorySegment`, представляют непрерывные блоки памяти. Как правило, они соответствуют блокам низкоуровневой памяти, но также можно создавать сегменты памяти на основе массивов в куче. Это похоже на объекты `ByteBuffer` в Java NIO API.

> **ПРИМЕЧАНИЕ** Panama API также содержит `MemoryAddress` — по сути обертку Java для указателя C (выраженного значением `long`).

Когда область видимости автоматически закрывается, выполняется обратный вызов `allocator`, который должен детерминированно освободить все удерживаемые ресурсы. Так паттерн «Захват ресурсов является инициализацией» (RAII, Resource Acquisition Is Initialization), который в Java реализуется конструкцией `try`-с-ресурсами, переносится в низкоуровневый код. Объекты области видимости и `allocator` хранят ссылки на низкоуровневые ресурсы и автоматически освобождают их при выходе из блока `try`-с-ресурсами.

https://t.me/javalib

Также можно использовать неявный механизм, чтобы низкоуровневая память освобождалась, когда объект `MemorySegment` уничтожается сборщиком мусора. Конечно, при этом освобождение будет происходить недетерминированно — в зависимости от того, когда выполняется сборка мусора. В общем случае рекомендуется использовать явные области видимости, особенно если вы не знакомы с потенциальными рисками работы с памятью вне кучи.

На момент написания книги утилита `jextract` поддерживает заголовочные файлы только языка C. А значит, чтобы использовать ее из низкоуровневых языков (например, Rust), придется сначала сгенерировать заголовочный файл C. В идеале рано или поздно должна появиться программа, которая генерирует такие файлы автоматически, — как `rust-bindgen`, только наоборот.

Со временем в `jextract` может появиться поддержка других языков. Эта утилита базируется на системе LLVM, которая уже не зависит от языка, так что теоретически ее можно расширять для любого языка, с которым совместима LLVM и в котором используются соглашения вызова функций C.

API внешних функций и памяти вскоре будет опубликован как второй выпуск в форме инкубационной функциональности в составе Java 18. (О том, что такое инкубационная и предварительная функциональность, см. главу 1.) Предположительно он будет воплощен в виде окончательной, стандартизированной функциональности в составе Java 19.

API векторов не настолько проработан, в первую очередь потому, что его создатели решили, что лучше подождать, пока не созреет функциональность проекта Valhalla (см. далее в этой главе). А следовательно, этот API не выйдет из инкубации, пока Valhalla не станет стандартной функциональностью.

18.3. ПРОЕКТ LOOM

По словам создателей проекта Loom из OpenJDK, его задача —

> обеспечить простые в использовании, высокопроизводительные и облегченные средства конкурентного выполнения и новых моделей программирования на платформе Java.
>
> *Project Loom,* https://wiki.openjdk.org/display/loom/Main

Зачем же понадобился новый подход к конкурентному выполнению? Чтобы ответить на этот вопрос, необходимо взглянуть на Java в исторической перспективе.

Одна из интересных точек зрения на Java заключается в том, что это язык и платформа из конца 1990-х, и в этом продукте был сделан ряд стратегических, субъективных ставок на то, как будет развиваться область разработки ПО. Эти ставки в основном оправдались, хотя остается открытым вопрос о том, объясняется это проницательностью или простым везением.

https://t.me/javalib

Для примера возьмем многопоточность. Платформа Java стала первой массовой программной платформой, где потоки были интегрированы в базовый язык. До этой интеграции самые передовые продукты обеспечивали взаимодействие между потоками с помощью множественных процессов и других неудовлетворительных механизмов (кто-нибудь еще помнит общую память Unix?).

На уровне операционной системы потоки представляют собой независимо планируемые единицы выполнения, которые принадлежат процессу. У каждого потока есть собственный счетчик команд и стек вызова, но все потоки одного процесса используют общую кучу.

Кроме того, куча Java устроена как непрерывное подмножество кучи процесса (по крайней мере в реализации HotSpot — в других JVM ситуация может быть иной), так что модель памяти потоков на уровне ОС органически переносится в область языка Java.

Понятие потоков естественным образом приводит к концепции облегченного переключения контекста. Переключение между двумя потоками внутри одного процесса менее затратно, чем переключение между разными процессами. Прежде всего это связано с тем, что таблицы преобразования виртуальных адресов памяти в физические в основном одинаковы для потоков одного процесса.

> **ПРИМЕЧАНИЕ** Создать поток тоже дешевле, чем процесс. Впрочем, насколько дешевле, — зависит от подробностей конкретной операционной системы.

В общем случае спецификация Java не требует, чтобы потоки Java соответствовали потокам операционной системы (при том, что в управляющей ОС может вообще не быть понятия потоков, что раньше было обычным делом). Более того, в самых первых версиях Java потоки JVM отображались на потоки ОС (или *платформенные* потоки) в так называемых *зеленых потоках*, или *потоках M:1* (потому что реализация фактически использовала только один платформенный поток).

Однако эта практика ушла в небытие в эпоху Java 1.2/1.3 (и чуть ранее в ОС Sun Solaris). Современные версии Java, которые работают в массовых операционных системах, реализуют правило, согласно которому один поток Java соответствует ровно одному потоку операционной системы. Вызов `Thread.start()` инициирует вызов системной функции создания потока (например, `clone()` в Linux) и фактически создает новый поток ОС.

Главная цель проекта Loom из OpenJDK — внедрить новые объекты `Thread`, которые могут выполнять код, но не соответствуют специализированным потокам ОС, — или, другими словами, создать модель выполнения, в которой объект, который представляет контекст исполнения, — это не обязательно та же самая сущность, планированием которой управляет операционная система.

Таким образом, в некоторых отношениях Loom возвращается к чему-то вроде зеленых потоков. Тем не менее за прошедшие годы мир заметно изменился,

https://t.me/javalib

а в компьютерной отрасли порой появляются идеи, которые опережают свое время.

Например, технологию EJB (Enterprise Java Beans) часто интерпретируют как разновидность виртуализированной/ограниченной среды, которая с помощью виртуализации слишком усердно старается разорвать связь с окружением. Может быть, эту технологию стоит рассматривать как прототипную форму идей, которые позднее найдут применение в современных системах PaaS — и в меньшей степени в Docker и Kuberntes?

Таким образом, если Loom возвращается (частично) к идее зеленых потоков, то понять суть проекта поможет вопрос: «Что же изменилось в среде, чтобы возвращаться к старой идее, которая в прошлом оказалось не слишком удачной?»

Чтобы ответить на этот вопрос, рассмотрим пример. Давайте попробуем добиться отказа JVM, создав слишком много потоков. Не запускайте код из этого примера, если вы не готовы столкнуться с возможным сбоем:

```
//
// Не запускайте этот код: он может "уронить" вашу JVM или компьютер
//
public class CrashTheVM {
    private static void looper(int count) {
        var tid = Thread.currentThread().getId();
        if (count > 500) {
            return;
        }
        try {
            Thread.sleep(10);
            if (count % 100 == 0) {
                System.out.println("Идентификатор потока: "+ tid +" : "+ count);
            }
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
        looper(count + 1);
    }

    public static Thread makeThread(Runnable r) {
        return new Thread(r);
    }

    public static void main(String[] args) {
        var threads = new ArrayList<Thread>();
        for(inti=0;i< 20_000;i=i+1){
            var t = makeThread(() -> looper(1));
            t.start();
            threads.add(t);
            if (i % 1_000 == 0) {
                System.out.println(i + " поток запущен");
            }
        }
        // Соединяем все потоки
```

https://t.me/javalib

```
        threads.forEach(t -> {
            try {
                t.join();
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        });
    }
}
```

Код запускает 20 000 потоков и выполняет в каждом из них минимальный объем вычислений — или, по крайней мере, пытается выполнить. На практике программа обычно отказывает или заставляет компьютер зависнуть задолго до того, как все эти операции завершатся.

> **ПРИМЕЧАНИЕ** Теоретически этот пример может отработать до завершения, если компьютер или ОС работает в режиме лимитирования и не успевает создавать потоки достаточно быстро, чтобы они столкнулись с нехваткой ресурсов.

И хотя этот пример нельзя назвать полностью репрезентативным, он дает представление о том, что происходит, например, в среде веб-сервера, где на каждое подключение выделяется отдельный поток. От современного высокопроизводительного веб-сервера вполне разумно ожидать, что он справится с 20 000 подключений в конкурентном режиме, — и все же этот пример наглядно показывает, что архитектура «один поток на подключение» в данном случае себя не оправдывает.

> **ПРИМЕЧАНИЕ** Loom можно рассматривать и под другим углом: возможно, современной программе на Java потребуется отслеживать намного больше контекстов выполнения, чем она сможет создать потоков.

Еще один фактор заключается в том, что потоки потенциально обходятся дороже, чем мы привыкли думать, и стали узким местом для масштабирования в современных приложениях на JVM. Разработчики годами пытались решить эту проблему — либо контролируя затраты на создание потоков, либо представляя контексты выполнения в виде сущностей, которые не являются потоками.

В качестве одного из решений предлагалась методология SEDA (Staged Event Driven Architecture) — упрощенно говоря, система, в которой объект предметной области перемещается из пункта A в пункт Z по многоэтапному конвейеру, на котором подвергается различным преобразованиям. В распределенной системе это можно реализовать на базе системы сообщений, а в одном процессе — с помощью блокирующих очередей и пула потоков для каждого этапа.

На каждом этапе обработка объекта предметной области описывается объектом Java, который содержит код, реализующий преобразование на этом этапе. Чтобы такая схема правильно работала, код должен гарантированно завершаться

https://t.me/javalib

(то есть не содержать бесконечных циклов), причем фреймворк не должен обеспечивать это принудительно.

У такого подхода есть ряд существенных недостатков, и один из них заключается в том, что от программистов потребуется высокая дисциплина, чтобы эффективно использовать эту архитектуру. Рассмотрим более совершенную альтернативу.

18.3.1. Виртуальные потоки

Чтобы повысить эффективность современных крупномасштабных приложений, проект Loom добавляет в JVM новые конструкции:

- виртуальные потоки;
- ограниченные продолжения;
- устранение хвостовых вызовов.

Главное в этом списке — *виртуальные потоки*, которые с точки зрения программиста должны выглядеть как обычные потоки. Тем не менее это не тонкие обертки для потоков в соотношении 1:1; среда исполнения Java реализует их в пользовательском пространстве и управляет ими. Предполагается, что виртуальные потоки обеспечат ряд преимуществ:

- Создание и блокировка потоков обходятся дешево.
- Можно использовать стандартные планировщики Java (пулы потоков).
- Для стека не требуются структуры данных на уровне ОС.

Если операционная система перестает участвовать в жизненном цикле виртуального потока, тем самым устраняется узкое место масштабирования. Приложения для JVM иногда имеют дело с миллионами и даже миллиардами объектов, — тогда почему мы должны ограничиваться всего несколькими тысячами объектов, которые планирует ОС (это одна из возможных интерпретаций потоков)? Снять это ограничение и дать возможность использовать новые стили конкурентного программирования — главная цель проекта Loom.

Рассмотрим виртуальные потоки в действии. Загрузите бета-версию Loom (https://jdk.java.net/loom/) и запустите `jshell` (включив режим предварительного просмотра, чтобы активировать функциональность Loom):

```
$ jshell --enable-preview
|  Welcome to JShell -- Version 18-loom
|  For an introduction type: /help intro
jshell> Thread.startVirtualThread(() -> {
  ...> System.out.println("Hello World");
  ...> });
Hello World
$1 ==> VirtualThread[<unnamed>,<no carrier thread>]

jshell>
```

https://t.me/javalib

Конструкция виртуального потока видна в выводе. Также здесь используется новый статический метод `startVirtualThread()`, чтобы запустить лямбда-выражение в новом контексте исполнения, которым является виртуальный поток. Все просто!

Общее правило заключается в том, что существующие кодовые базы должны продолжать работать точно так же, как до появления Loom. Или, другими словами, виртуальные потоки можно использовать по желанию. Необходимо сделать консервативное предположение о том, что всему существующему коду Java будет по-прежнему достаточно облегченной обертки для потоков ОС, которая до настоящего времени оставалась единственным возможным вариантом.

Виртуальные потоки открывают новые горизонты и в других отношениях. Раньше создать новый поток в языке Java можно было двумя основными способами:

- Создать подкласс `java.lang.Thread` и вызвать унаследованный метод `start()`.
- Создать экземпляр `Runnable` и передать его конструктору `Thread`, после чего запустить полученный объект.

Если меняется представление о том, что такое поток, то имеет смысл пересмотреть и методы, с помощью которых создаются потоки. Только что мы рассмотрели новый статический фабричный метод для *автономных* виртуальных потоков, но существующему API потоков тоже не помешают некоторые усовершенствования.

18.3.2. Строители потоков

Важное место среди новых понятий занимает класс `Thread.Builder`, добавленный как внутренний класс `Thread`. В `Thread` появились два новых фабричных метода, которые открывают доступ к строителям платформенных и виртуальных потоков:

```
jshell> var tb = Thread.ofPlatform();
tb ==> java.lang.ThreadBuilders$PlatformThreadBuilder@312b1dae

jshell> var tb = Thread.ofVirtual();
tb ==> java.lang.ThreadBuilders$VirtualThreadBuilder@506e1b77
```

Рассмотрим класс-строитель в действии, заменив метод `makeThread()` в нашем примере следующим кодом:

```
// Код, специфичный для Loom
public static Thread makeThread(Runnable r) {
    return Thread.ofVirtual().unstarted(r);
}
```

Здесь вызывается метод `ofVirtual()`, чтобы явно создать виртуальный поток, который выполнит наш объект `Runnable`. Конечно, вместо этого можно было воспользоваться фабричным методом `ofPlatform()` и получить объект традиционного потока, который планируется ОС. Но какая от этого польза?

https://t.me/javalib

Если в наш предыдущий пример подставить виртуальную версию `makeThread()` и перекомпилировать его с версией Java, которая поддерживает Loom, то полученный код можно будет запустить. На этот раз программа отработает до завершения без каких-либо проблем. Это хороший пример философии Loom в действии: изменения, которые необходимо внести в приложение, ограничиваются точками, в которых создаются потоки.

Один из факторов, которые помогают разработчикам переходить со старых парадигм на новую потоковую библиотеку, заключается в том, что подклассы `Thread` не могут быть виртуальными. Следовательно, код, где наследуется `Thread`, продолжит создавать традиционные потоки ОС.

> **ПРИМЕЧАНИЕ** Со временем виртуальные потоки станут более распространенными, и разработчики перестанут обращать внимание на различия между виртуальными потоками и потоками ОС. Это должно способствовать отказу от механизма наследования, потому что он всегда создает поток, планируемый ОС.

Такой подход призван защитить существующий код, в котором наследуется `Thread`, в соответствии с принципом «наименьшей неожиданности».

Другие части библиотеки потоков тоже потребуется обновить, чтобы они лучше поддерживали Loom. Например, `ThreadBuilder` также может строить экземпляры `ThreadFactory`, которые можно передавать различным исполнителям:

```
jshell> var tb = Thread.ofVirtual();
tb ==> java.lang.ThreadBuilders$VirtualThreadBuilder@312b1dae

jshell> var tf = tb.factory();
tf ==> java.lang.ThreadBuilders$VirtualThreadFactory@506e1b77

jshell> var tb = Thread.ofPlatform();
tb ==> java.lang.ThreadBuilders$PlatformThreadBuilder@1ddc4ec2

jshell> var tf = tb.factory();
tf ==> java.lang.ThreadBuilders$PlatformThreadFactory@b1bc7ed
```

Чтобы виртуальный поток мог выполниться, его нужно присоединить к реальному потоку ОС, — такие потоки называются *потоками-носителями* и уже встречались в выводе `jshell` в одном из предыдущих примеров. Однако на протяжении своего жизненного цикла один виртуальный поток может выполняться в нескольких разных потоках-носителях. Это отчасти похоже на то, как обычный поток может в разное время выполняться на разных ядрах физического процессора, — оба случая демонстрируют примеры планирования выполнения.

18.3.3. Программирование с помощью виртуальных потоков

Появление виртуальных потоков меняет менталитет. Программисты, которые писали конкурентные приложения на Java в его текущем состоянии, привыкли иметь дело (сознательно или неосознанно) с неизбежными ограничениями на масштабирование потоков.

https://t.me/javalib

Мы привыкли создавать объекты задач, часто на базе `Runnable` или `Callable`, и передавать их исполнителям, которые работают на основе пулов потоков, которые, в свою очередь, существуют для того, чтобы экономить драгоценные ресурсы потоков. А что, если вся ситуация неожиданно изменится?

В сущности, проект Loom пытается устранить ограничения на масштабирование потоков за счет введения нового понятия потока, который требует меньших затрат и не отображается напрямую на поток ОС. Тем не менее эти новые сущности все еще выглядят и ведут себя как традиционные потоки, к которым программисты на Java уже привыкли.

Вместо того чтобы заставлять разработчиков осваивать совершенно новый стиль программирования (например, стиль передачи продолжения, `Promise`/`Future` или обратные вызовы), среда выполнения Loom сохраняет для виртуальных потоков модель программирования, знакомую нам по существующим потокам. Проще говоря, виртуальные потоки — они потоки и есть (по крайней мере в том, что касается программиста).

Виртуальные потоки являются *вытесняющими*, потому что пользовательскому коду не приходится явно уступать управление. Точки планирования выбираются виртуальным планировщиком и JDK. Пользователи не должны делать никаких предположений о том, когда это произойдет, потому что передача управления относится к подробностям реализации. Тем не менее, чтобы понимать, чем виртуальные потоки отличаются от традиционных, желательно разбираться в основах теории операционных систем, которая лежит в основе планирования.

Когда ОС планирует платформенные потоки, она выделяет потоку *квант* процессорного времени. По истечении этого кванта генерируется аппаратное прерывание, и ядро получает возможность принять управление, снять исполняемый платформенный (пользовательский) поток и заменить его на другой.

> **ПРИМЕЧАНИЕ** Именно по этому принципу Unix (как и некоторые другие операционные системы) всегда разделял процессорное время между разными задачами — даже десятки лет назад, когда у компьютеров было всего одно вычислительное ядро.

Тем не менее виртуальные потоки обрабатываются не так, как платформенные. Ни один из существующих планировщиков для виртуальных потоков не использует кванты времени, чтобы вытеснять выполняемые виртуальные потоки.

> **ПРИМЕЧАНИЕ** Использовать кванты времени для вытеснения виртуальных потоков технически возможно, и виртуальная машина уже способна перехватывать контроль над выполнением потоков Java, например она делает это в безопасных точках JVM.

Вместо этого виртуальные потоки автоматически уступают свой поток-носитель, когда выполняется блокирующий вызов (например, ввод/вывод). Это происходит под контролем библиотеки и среды выполнения, а не под явным управлением программиста.

https://t.me/javalib

Таким образом, вместо того чтобы заставлять программистов явно управлять уступкой управления или возиться со сложностями неблокирующих операций или операций на базе обратного вызова, Loom позволяет писать код в традиционном потоково-последовательном стиле. В качестве дополнительного преимущества такое решение позволяет отладчикам и профилировщикам работать в обычном режиме. Правда, разработчикам инструментов и сред выполнения придется приложить специальные усилия, чтобы их продукты поддерживали виртуальные потоки, но это лучше, чем создавать дополнительную когнитивную нагрузку для конечных Java-разработчиков. В частности, этот принцип отличается от подхода `async/await`, который принят в некоторых языках программирования.

Разработчики Loom ожидают, что поскольку виртуальные потоки не нужно объединять в пулы, они никогда *не будут* объединяться в пулы, так что допускается неограниченное создание виртуальных потоков. Для этого появился *неограниченный исполнитель*, к которому можно обратиться через новый фабричный метод `Executors.newVirtualThreadPerTaskExecutor()`. По умолчанию для виртуальных потоков используется тот же планировщик с перехватом работы, что и для `ForkJoinPool`.

> **ПРИМЕЧАНИЕ** Интересно, что здесь возможности перехвата работы Fork/Join оказываются намного важнее, чем рекурсивная декомпозиция задач.

Архитектура Loom в ее сегодняшнем состоянии полагается на то, что разработчик осознает, что разные виды потоков в приложениях требуют лишних вычислительных затрат. Проще говоря, если многочисленным потокам постоянно требуется процессорное время, ваше приложение столкнется с нехваткой ресурсов, и даже самое умное планирование здесь не поможет. С другой стороны, если ожидается, что всего несколько потоков будут интенсивно расходовать ресурсы процессора, их следует объединить в отдельный пул и связать с платформенными потоками.

Предполагается, что виртуальные потоки будут хорошо работать в случае, когда выполняется много потоков и при этом они лишь изредка интенсивно расходуют процессорное время. Планировщик с перехватом работы должен сгладить нагрузку на процессор, и в реальном коде рано или поздно будет вызвана операция, которая проходит точку уступки управления (например, блокирующий ввод/вывод).

18.3.4. Когда проект Loom станет доступным?

Разработка Loom ведется в отдельном репозитории, а не в основной ветви JDK. Двоичные файлы ранней версии доступны, но они не идеальны, и время от времени случаются отказы (впрочем, это происходит все реже). Базовый API обретает форму, но он еще далек от завершения.

JEP 425 (https://openjdk.java.net/jeps/425) предлагает интегрировать виртуальные потоки в качестве предварительной функциональности, но на момент написания

https://t.me/javalib

книги этот документ еще не был запланирован для публикации в какой-либо версии. Разумно предположить, что если его не включат как предварительную функциональность в Java 19, то финальной версии виртуальных потоков не будет в составе Java 21 (которая, вероятно, станет следующей LTS-версией Java). Еще предстоит проделать значительную работу над API, которые строятся на основе виртуальных потоков, например, по части структурированного параллелизма и других, более сложных подсистем.

У разработчиков всегда возникает вопрос о производительности, но на него обычно трудно дать ответ, когда новая технология еще находится на ранних стадиях. Для Loom мы еще не достигли точки, в которой имеет смысл сравнивать решения, и текущую производительность нельзя считать показательной для финальной версии.

Тем не менее нынешний прототип дает достаточно возможностей для того, чтобы начать экспериментировать и получить первое представление о том, какими путями пойдет развитие Java в будущем. Давайте теперь обратимся к последнему из четырех крупных проектов OpenJDK, рассматриваемых в этой главе, — Valhalla.

18.4. ПРОЕКТ VALHALLA

> Привести поведение структуры памяти JVM в соответствие с моделью стоимости современного оборудования.
>
> *Брайан Гетц*

Чтобы понять, где современная модель структуры памяти Java доходит до своего предела и начинает проявлять несовершенство, начнем с примера. На рис. 18.1 изображен массив примитивных целых чисел. Поскольку это значения примитивного типа, а не объекты, они размещаются в смежных ячейках памяти.

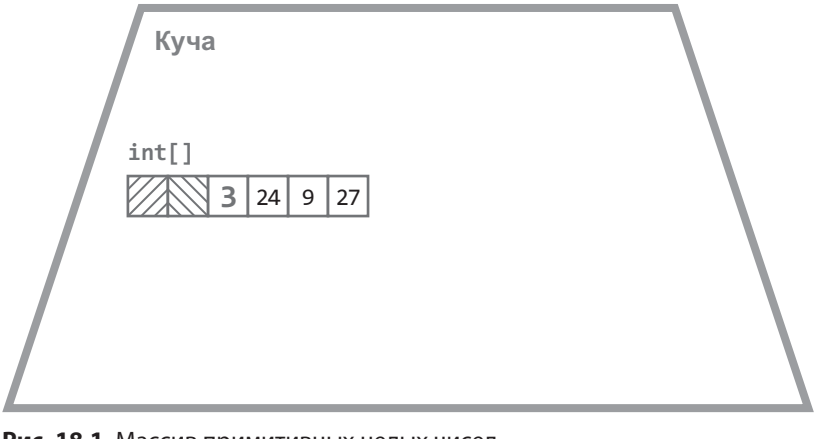


Рис. 18.1. Массив примитивных целых чисел

https://t.me/javalib

Чтобы увидеть, что меняется в случае массива объектов, рассмотрим массив упакованных целых чисел. Массив объектов `Integer` будет представлять собой массив ссылок, как показано на рис. 18.2.

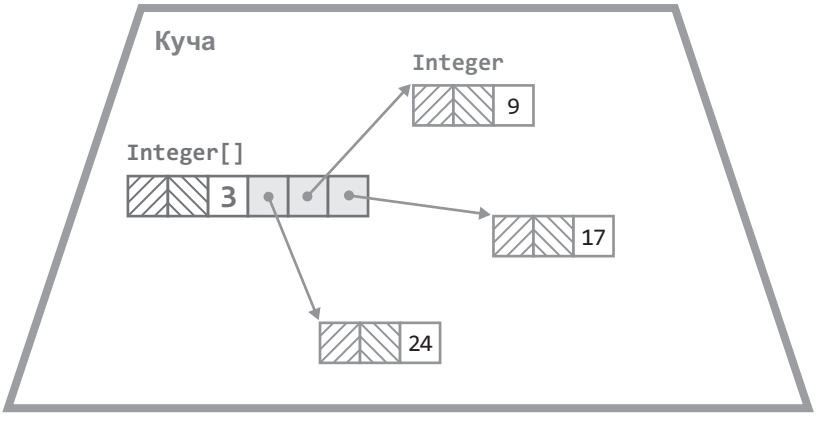


Рис. 18.2. Массив объектов Integer

Поскольку каждое значение `Integer` является объектом, у него должен быть заголовок объекта, как рассказывалось в предыдущей главе. Иногда говорят, что каждый объект Java облагается «заголовочным налогом».

Более 20 лет структура памяти на платформе Java устроена именно таким образом. С одной стороны, она организована просто, но с другой стороны, подразумевает компромисс по производительности: работа с массивами объектов подразумевает неизбежные косвенные обращения по указателям и соответствующие промахи кэша.

Для примера рассмотрим класс, который представляет точку в трехмерном пространстве, — тип `Point3D`. Он состоит лишь из трех пространственных координат, и в Java 17 его можно представить типом объекта с тремя полями (или эквивалентной записью):

```
public final class Point3D {
    private final double x;
    private final double y;
    private final double z;

    public Point3D(double a, double b, double c) {
        x = a;
        y = b;
        c = z;
    }

    // Дополнительные методы: методы чтения, toString() и т. д.
}
```

https://t.me/javalib

В HotSpot массив этих объектов размещается в памяти так, как показано на рис. 18.3.

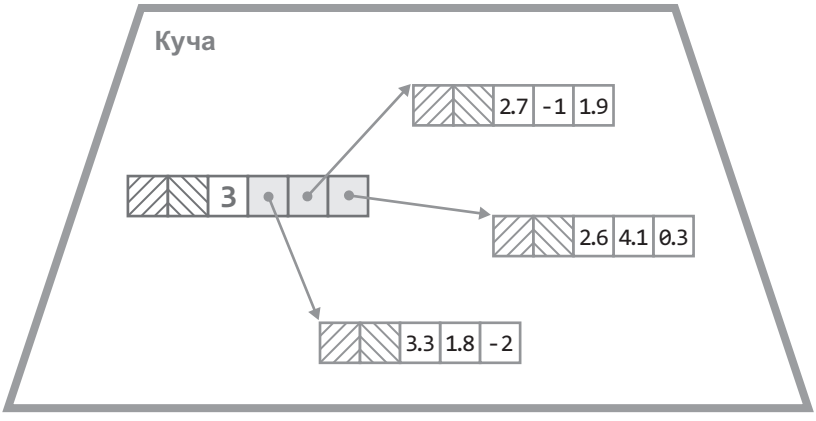


Рис. 18.3. Массив объектов Point3D

При обработке этого массива обращение к каждому элементу для того, чтобы получить координаты точки, происходит через дополнительный уровень косвенности. Это вызывает промахи кэша для каждой точки в массиве и, как следствие, снижает производительность, казалось бы, «на пустом месте».

Для программистов, которые уделяют особое внимание производительности, была бы очень полезна возможность определять типы, которые можно эффективнее размещать в памяти. Также заметим, что при работе со значениями `Point3D` наличие индивидуальности у конкретного объекта не принесло бы программисту особой пользы, потому что две точки равны в том и только том случае, если равны все их поля.

Этот пример демонстрирует две разные концепции программирования, которые становятся возможными, если отказаться от индивидуальности объектов:

- *Сглаживание кучи* — не использовать косвенные обращения по указателю для объектов, у которых нет индивидиуальности; это повышает плотность использования памяти.
- *Скаляризация* — объект, у которого нет индивидуальности, можно разбивать на поля и при необходимости восстанавливать в других местах.

Как выясняется, эти свойства сказываются на пользовательской модели объектов без индивидуальности.

> **ПРИМЕЧАНИЕ** Скаляризация, то есть способность виртуальной машины разбивать и воссоздавать объекты-значения так, как она считает нужным, оказывается неожиданно полезной. JIT-компилятор в JVM использует прием *escape-анализа*, который заметно выигрывает, если есть возможность свободно разбивать объекты-значения на отдельные поля и передавать их в коде по отдельности.

https://t.me/javalib

С учетом сказанного разговор о Valhalla также можно начать с вопроса: «Можно ли обойтись без заголовочного налога?» В общем и целом ответ будет положительным, но для этого должны выполняться следующие условия:

- У объектов нет индивидуальности.
- Класс является конечным, так что целевые объекты всех вызовов методов известны во время загрузки класса.

Первое свойство фактически отменяет необходимость в mark-слове заголовка, а второе значительно сокращает необходимость в klass-слове (за дополнительной информацией о klass-слове обращайтесь к главам 4 и 17).

Klass-слово все еще необходимо, пока объект находится в куче, если только он не будет деструктурирован как поле экземпляра в другом объекте или как элемент массива, потому что может понадобиться описание структуры полей объекта, например, чтобы сборщик мусора мог обойти граф объектов. Но когда объекты скаляризованы, заголовок можно удалить.

Таким образом, с точки зрения разработчика, один из главных результатов Valhalla состоит в том, что в экосистеме Java появляется новая разновидность значений — так называемые *объекты-значения*, которые являются экземплярами *классов-значений*. Предполагается, что эти новые классы представляют небольшие неизменяемые типы без индивидуальности.

> **ПРИМЕЧАНИЕ** На этапе разработки классы-значения обозначались несколькими разными терминами, в том числе «примитивные классы» и «встраиваемые типы». Трудно выбрать правильное название, особенно в зрелом языке, который уже задействовал многие распространенные термины для своих понятий.

Вот примеры практического применения классов-значений:

- Новые разновидности чисел: беззнаковые байты, 128-разрядные целые числа, числа с плавающей точкой половинной точности.
- Комплексные числа, цвета, векторы и другие многомерные числовые значения.
- Числа с единицами измерения: размер, температура, скорость, денежные суммы и т. д.
- Элементы отображений, строки баз данных и типы для множественных возвращаемых значений.
- Неизменяемые курсоры, подмассивы, промежуточные потоки данных и другие абстракции представления структур данных.

Также не исключено, что некоторые существующие типы могут дорабатываться и эволюционировать так, чтобы представляться в виде классов-значений. Например, `Optional` и многие типы из `java.time` — очевидные кандидаты на преобразование в классы-значения в будущих версиях, если это будет оправданно.

https://t.me/javalib

ПРИМЕЧАНИЕ Записи (Records) не имеют прямого отношения к классам-значениям, но весьма вероятно, что многие записи будут агрегатами, не требующими индивидуальности, так что концепция записи-значения может оказаться полезной.

Если эту новую форму значений получится реализовать на JVM, то для таких классов, как обсуждавшиеся выше точки в пространстве, сглаженная структура памяти вроде изображенной на рис. 18.4 будет намного эффективнее.

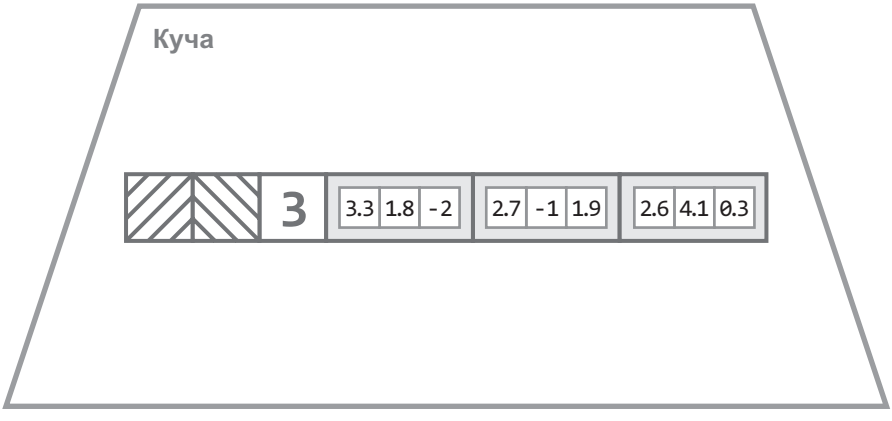


Рис. 18.4. Массив встроенных точек

Такая структура памяти близка к той, которая известна программистам C как массив из `struct`, но без рисков, связанных с низкоуровневым доступом к памяти. Сглаженная структура сокращает не только затраты памяти, но и нагрузку на сборщики мусора.

18.4.1. Изменение модели языка

Самое большое изменение, которое необходимо внести в язык, — пересмотр концепции `java.lang.Object` как всеобщего надкласса, потому что он содержит такие методы, как `wait()` и `notify()`, которые неотъемлемо связаны с индивидуальностью объекта. Без заголовка объекта не будет mark-слова, чтобы хранить монитор объекта, и никакого идентификатора, который можно использовать для ожидания. У объекта также нет четко определенного срока жизни, потому что его можно свободно копировать, а полученные копии неотличимы от оригинала. Вместо этого в `java.lang` определяются два новых интерфейса — `IdentityObject` и `ValueObject`. Объекты-значения подробно описываются в JEP 401 (https://openjdk.org/jeps/401), но по сути, все классы-значения неявно реализуют `ValueObject`.

Все классы, обладающие индивидуальностью, неявно реализуют `IdentityObject`, и все уже существующие конкретные классы отнесены к этой категории. Существующие интерфейсы и абстрактные классы (в большинстве своем) не расширяют ни одного из новых интерфейсов. Проектировщики API могут обновить

https://t.me/javalib

свои интерфейсы так, чтобы они явно расширяли `IdentityObject`, если их функциональность окажется несовместимой с новой семантикой.

Классы-значения объявляются с `final` и не могут объявляться с `abstract`. Они не могут реализовывать (прямо или косвенно) `IdentityObject`. Чтобы проверить, является ли объект объектом-значением, можно воспользоваться `instanceof`.

> **ПРИМЕЧАНИЕ** Кроме того что для объектов-значений нельзя вызвать `wait()` или `notify()`, они также не могут содержать синхронизированных методов или блоков, потому что у объектов-значений нет мониторов.

Сам класс `Object` требует незначительных модификаций, потому что он не будет реализовывать ни `IdentityObject`, ни `ValueObject` и будет больше напоминать абстрактный класс или интерфейс. Смысл следующего кода тоже изменится:

```
var o = new Object();
```

Ожидается, что `o` будет содержать экземпляр некоторого анонимного подкласса `Object`, который по соображениям обратной совместимости будет интерпретироваться как класс, обладающий индивидуальностью.

И хотя исходные цели классов-значений кажутся понятными, у них просматриваются далеко идущие последствия. Чтобы проект был успешным, необходимо изучить, к каким логическим заключениям приводит внедрение третьей разновидности значений.

18.4. ПОСЛЕДСТВИЯ ВНЕДРЕНИЯ ОБЪЕКТОВ-ЗНАЧЕНИЙ

Присваивание для объектов-значений имеет вполне очевидную семантику: происходит такое же копирование битов, как и в случае примитивов. (При присваивании ссылок тоже копируются биты, но при этом мы получаем две ссылки на одну и ту же область кучи.) Но что произойдет, если нужно сконструировать объект-значение, не полностью идентичный оригиналу, то есть измененную копию?

Вспомните, что объекты-значения являются неизменяемыми: они содержат только конечные поля. Это означает, что их состояние нельзя изменить операциями `putfield`. Необходим другой механизм, чтобы создавать объекты-значения, состояние которых отличается от состояния оригинала. Для этого потребуются новые инструкции:

- `aconst_init`
- `withfield`

Инструкция `withfield` фактически работает как эквивалент методов `with*()` (которые обсуждались в главе 15) на уровне байт-кода.

https://t.me/javalib

Другая новая инструкция, `aconst_init`, предоставляет значение по умолчанию для экземпляра класса-значения. Разберемся, для чего она нужна.

В современном Java и у примитивов и у ссылок на объекты есть значение по умолчанию, при котором все биты заполнены нулями, а `null` понимается как обнуленное значение для ссылок. Но если попытаться расширить эту семантику на работу с объектами-значениями, возникают две взаимосвязанные проблемы:

- У некоторых классов-значений нет хорошего варианта для значения по умолчанию.
- Возникает возможность *разрыва значений*.

Первая проблема на самом деле сводится к тому, что вы хотите как-то обозначить, что объект-значение еще не получил значения, но у Java уже есть вариант на такой случай: `null`. Более того, пользователи уже привыкли иметь дело с исключениями `NullPointerException` (NPE), когда встречается неинициализированное значение.

Второй пункт — разрыв — на самом деле старая проблема в новом обличии. В старых версиях Java, которые работали на 32-разрядном оборудовании, возникали неочевидные потенциальные проблемы с 64-разрядными значениями (например, `long`). В частности, чтобы записать значение `long`, выполнялись две раздельные операции записи (32-разрядные), и читающий поток мог увидеть состояние, в котором завершилась только одна из них. В результате он наблюдал «разорванное» значение `long` — ни в состоянии «до», ни в состоянии «после».

С типами-значениями эта проблема может возникнуть снова. Если значения можно скаляризовать, то как гарантировать атомарность записи?

Для этого нужно осознать, что классы-значения представляют не одну новую форму данных, а *две*. Если вы хотите избежать разрыва, нужно использовать ссылку. Вспомните хорошо известную идею: промежуточный уровень косвенности позволяет обновлять значения, не разрывая их.

Также стоит учесть, что у некоторых классов нет разумного значения по умолчанию, которое бы соответствовало нулевым битам. Например, каким должно быть значение по умолчанию `LocalDate` после миграции в класс-значение? Кто-то скажет, что нулевые биты можно интерпретировать как нулевое смещение от начала эпохи (то есть 1 января 1970 года), но такое решение кажется крайне ненадежным.

Так мы приходим к понятию *ссылок, лишенных индивидуальности*, — по сути, это объекты, у которых мы отняли индивидуальность. На низком уровне это все еще позволяет JIT оптимизировать соглашения вызова (например, передавая значения в скалярном виде) и выполнять скаляризацию, но пропадают улучшения, связанные с использованием памяти в куче. С такими объектами всегда приходится работать по ссылке, как с объектами, у которых есть индивидуальность, и по умолчанию они имеют тривиальное значение `null`. Если у объекта должно

https://t.me/javalib

быть нетривиальное значение по умолчанию, за это отвечают конструкторы или фабричные методы, как и должно быть.

Кроме того, для продвинутых сценариев использования также существуют *примитивные типы-значения*. Они ведут себя скорее как встроенные «полноценные примитивы», допуская и сглаживание в куче, и скаляризацию, как для объектов-значений. Однако дополнительные преимущества не даются даром: в качестве значения по умолчанию требуется передавать нулевые биты, и возникает риск разрыва при обновлениях, что может приводить к гонке при работе с данными.

> **ПРИМЕЧАНИЕ** Предполагается, что примитивные типы-значения предназначены только для малых значений (не более 64–128 бит на современном оборудовании), и программирование с ними требует особой осторожности.

Разрыв часто подвергает пользователя потенциальным угрозам безопасности, хотя появляется искушение заявить, что проблема существует только в «плохих» программах с гонкой по данным. В любом случае это новая разновидность потенциального дефекта параллельного программирования, для правильной защиты от которой понадобятся блокировки.

Другой аспект примитивных классов-значений заключается в том, что среда выполнения должна знать, как размещать их в памяти. Поэтому невозможно создать поле примитивного класса-значения, которое ссылается (прямо или косвенно) на объявляющий класс. Другими словами, экземпляры примитивных классов-значений не могут содержать циклические структуры данных примитивных классов-значений — они должны иметь структуру фиксированного размера, чтобы их можно было сглаживать в куче. В целом предполагается, что большинство пользователей будут применять объекты-значения без индивидуальности, а расширенные примитивы будут использоваться намного реже.

В завершение этого раздела рассмотрим особенности того, как классы-значения представляются в байт-коде. Вспомните, что в главе 4 встречалось понятие дескриптора типа. Ссылочные типы с индивидуальностью обозначаются в байт-коде с помощью *L-дескрипторов*, например `Ljava/lang/String;` для строки.

Чтобы описывать значения примитивных классов, добавляется новый базовый тип — *Q-дескриптор* — с такой же структурой, как L-дескриптор (например, `QPoint3D;` для примитивного класса `Point3D`). Операции как с Q-, так и с L-значениями выполняются одним и тем же набором байт-кодов, а именно кодами, которые начинаются с a (например, `aload` или `astore`).

Значения, для ссылок на которые используются Q-дескрипторы (иногда называемые «минимальными» объектами), отличаются от ссылок на объекты-значения несколькими важными качествами:

- Ссылки на объекты-значения, как и все ссылки на объекты, могут принимать значение `null`, а минимальные значения — нет.

https://t.me/javalib

- Операции загрузки и сохранения ссылок выполняются атомарно в отношении друг друга, тогда как загрузка и сохранение достаточно больших минимальных значений может приводить к разрыву, как в случае `long` и `double` для 32-разрядной реализации.
- Графы объектов не могут быть циклическими, если они связываются через цепочку Q-дескрипторов: класс `C` в своей структуре не может ссылаться на `QC;` прямо или косвенно.
- По техническим причинам JVM должна загружать классы, обозначенные Q-дескрипторами, гораздо раньше, чем классы, обозначенные L-дескрипторами.

Эти свойства фактически представляют собой кодировку на уровне байт-кода для некоторых свойств значений и примитивных объектов, которые уже встречались вам ранее.

Прежде чем расстаться с Valhalla, давайте добавим последний фрагмент головоломки, вернувшись к обобщенным типам. Необходимость рассмотреть эту тему вполне естественно следует из того, что у нас появились объекты-значения и примитивные объекты.

18.4.3. Снова об обобщениях

Если в языке Java появятся классы-значения, возникает естественный вопрос о том, можно ли будет использовать классы-значения в обобщенных типах, например в качестве значения параметра-типа. Если бы это было невозможно, это значительно ограничило бы пользу от этой функциональности. Поэтому проектирование на высоком уровне изначально опиралось на предположение, что классы-значения будут допустимыми значениями параметров-типов в расширенной форме обобщений.

К счастью, роль `Object` в Valhalla несколько изменилась: он был ретроспективно назначен надклассом как для значений, так и для объектов с индивидуальностью. Это позволяет отнести объекты-значения к области обобщений. Однако примитивные типы тоже было бы желательно интегрировать в эту модель.

В долгосрочной перспективе предполагается расширить обобщения, чтобы обеспечить абстракцию над всеми типами, включая классы-значения и существующие примитивы (а также `void`). Для успешного воплощения этого замысла нужно, чтобы существующие библиотеки (особенно библиотеки JDK) эволюционировали в сторону совместимости с новым подходом, чтобы полноценно использовать эту функциональность.

В частности, понадобится обновить базовые типы-значения (`int`, `boolean` и т. д.) так, чтобы они превратились в примитивные классы-значения и чтобы в результате базовые примитивные значения стали примитивными объектами. Также это означает, что придется переделать классы-обертки, чтобы они вписывались в модель примитивных классов.

https://t.me/javalib

Такое расширение обобщений создаст *специализацию обобщений* для примитивов, а значит, в Java появятся элементы системы программирования с обобщениями наподобие тех механизмов, которые встречаются в других языках, например шаблонов C++. На момент написания книги работа над обобщениями еще находилась на ранней стадии и все соответствующие документы JEP имели статус черновиков.

18.5. JAVA 18

Любой материал, в котором авторы пытаются заглянуть в будущее, неизбежно устаревает к тому моменту, когда его будут читать. На момент написания книги была выпущена версия Java 18 (в марте 2022 года). Следующие документы JEP были предназначены для этой версии и сформировали содержимое нового выпуска:

- JEP 400: UTF-8 по умолчанию.
- JEP 408: Простой веб-сервер.
- JEP 413: Фрагменты кода в документации Java API.
- JEP 416: Повторная реализация базовой рефлексии с дескрипторами методов.
- JEP 417: API векторов (третья инкубационная версия).
- JEP 418: Интерфейс провайдера служб для преобразования адресов интернета.
- JEP 419: API внешних функций и памяти (вторая инкубационная версия).
- JEP 420: Поиск по шаблону для switch (вторая предварительная версия).
- JEP 421: Упразднение финализации при удалении.

В этом списке изменения, которые касаются UTF-8, базовой рефлексии и финализации, — это внутренние изменения, которые приводят в порядок и упрощают внутренние механизмы языка и платформы, чтобы обеспечить основу для дальнейшего развития.

API векторов и внешних функций — очередной важный этап на пути внедрения проекта Panama, а новая итерация поиска по шаблону — нововведение проекта Amber. В Java 18 нет никаких JEP, которые интегрировали бы какую-то часть Loom или Valhalla. На момент написания книги ходили неподтвержденные слухи, что первая версия Loom будет опубликована в виде предварительной функциональности в версии Java 19 (ее выход запланирован на сентябрь 2022 года[1]).

[1] Так и произошло. — *Примеч. ред.*

https://t.me/javalib

ИТОГИ

Один из исходных принципов проектирования языка Java заключался в том, что язык должен развиваться осторожно: не следует внедрять новую функциональность, пока не будет полностью понятно, какие последствия она повлечет для языка в целом. Некоторые другие языки прогрессируют активнее, чем Java, из-за чего разработчики порой пеняют, что «Java нужно развиваться быстрее». Однако у скорости есть и обратная сторона: бывает, что другие языки спешат, а потом, в соответствии с пословицей, смешат людей. Если интегрировать в язык неудачное проектировочное решение, оно фактически останется там навечно.

Java придерживается стратегии консервативного развития: прежде чем закрепить функциональность в языке, необходимо сначала убедиться, что мы правильно понимаем ее саму и ее возможные последствия. Пусть другие языки прокладывают новые пути (или, как скажет циник, «принимают на себя удар») — Java сделает для себя выводы из их экспериментов.

Собственно, заимствование концепций из одних языков программирования в другие можно считать традиционной практикой проектирования языков. В этом наглядно воплощается идея, которую иногда формулируют так: «Великие художники крадут». Эту цитату часто приписывают Стиву Джобсу, но он не придумал ее, а просто позаимствовал (или украл) у других мыслителей.

На самом деле этот принцип, похоже, открывали неоднократно, но одна из исходных форм прослеживается в следующей цитате:

> Один из самых верных признаков — то, как поэт заимствует. Незрелые поэты подражают; зрелые поэты крадут; плохие поэты обезображивают то, что берут, а хорошие поэты улучшают это, или, по крайней мере, делают из этого нечто иное.
>
> *Т. С. Элиот*

Замечание Элиота применимо к создателям языков программирования в той же степени, что и к поэтам. Действительно выдающиеся языки (и их проектировщики) свободно заимствуют (или крадут) друг у друга. Хорошие идеи, которые изначально выражаются на каком-то одном языке, не замыкаются в нем — собственно, это один из признаков, по которым мы узнаем, что идея была хорошей.

В этой последней главе мы рассмотрели четыре текущих проекта в составе OpenJDK. Собранные воедино, они стремятся представить принципиально новую будущую версию Java.

Некоторые из этих проектов чрезвычайно амбициозны, другие более скромны. Все они будут реализованы в рамках регулярного цикла выпуска новых версий Java. Язык Java, о котором мы будем писать через год или через три года, будет заметно отличаться от того, о котором мы пишем сегодня.

https://t.me/javalib

Вот основные аспекты Java, которые, на наш взгляд, будут переработаны в первую очередь:

- *Слияние объектно-ориентированного и функционального программирования*: проект Amber вводит новые языковые возможности, которые помогают сблизить эти модели.
- *Многопоточность*: проект Loom представляет новую модель потоков, которые участвуют во вводе/выводе.
- *Структура памяти*: проект Valhalla решает сразу несколько проблем, улучшая заполняемость памяти и расширяя возможности обобщений.
- *Улучшенная совместимость с низкоуровневым кодом*: проект Panama помогает устранить некоторые проблемы взаимодействия с JNI и рядом других низкоуровневых технологий.
- *Текущая доработка внутренней реализации*: серия документов JEP посвящена тому, чтобы неспешно удалять с платформы аспекты, которые стали ненужными.

Окончательная форма будущего языка Java пока что продолжает вырисовываться и на данный момент еще не полностью прояснилась. Однако можно уверенно утверждать, что и через 25 лет Java все еще останется силой, с которой следует считаться. Язык пережил несколько крупных «тектонических сдвигов» в мире программного обеспечения — таким послужным списком можно гордиться, и он обеспечивает хороший задел на будущее.

https://t.me/javalib

Приложения

https://t.me/javalib

A Выбор версии Java

С изменениями в распространении и поддержке Oracle JDK появилась некоторая неопределенность относительно того, чем различаются Oracle JDK, сборки Oracle OpenJDK и сборки OpenJDK от других поставщиков в отношении прав на использование. Существуют различные способы получения бесплатных обновлений (включая обновления безопасности), а также различные модели платной поддержки от разных разработчиков. За полным изложением этой темы обращайтесь к авторитетному руководству «Java остается бесплатным» по адресу http://mng.bz/Qvdw от сообщества Java Champions (https://dev.java/community/jcs/) — независимой организации отраслевых лидеров Java.

A.1. JAVA ОСТАЕТСЯ БЕСПЛАТНЫМ

Сборки OpenJDK со свободной лицензией GPLv2+CE по-прежнему можно получить от разных поставщиков, включая Oracle (см. https://openjdk.java.net/legal/gplv2+ce.html). Oracle JDK остается бесплатным в некоторых обстоятельствах, как описано далее в этом разделе.

A.1.1. Java SE / OpenJDK / Сборки Oracle OpenJDK / Oracle JDK

Сообщество OpenJDK создает и сопровождает эталонную реализацию (RI, Reference Implementation) спецификации Java SE с открытым кодом на условиях лицензии GPLv2+CE, как того требует процесс JCP (Java Community Process). Каждый выпуск новой функциональности определяется обобщающим документом JSR (Java Specification Request). Реализации Java SE (в основном на базе OpenJDK) доступны от разных разработчиков, в том числе от Alibaba, Amazon, Azul, BellSoft, Eclipse Adoptium (преемник AdoptOpenJDK), IBM, Microsoft, Red Hat, Oracle, SAP и др.

https://t.me/javalib

Версия Oracle JDK 8 прошла через процесс завершения общедоступных обновлений: это означает, что начиная с апреля 2019 года обновления требуют контракта на поддержку для промышленного использования. Как упоминалось ранее, OpenJDK 8, 11 и 17 с полностью свободной лицензией можно получить у других поставщиков. Oracle также предоставляет бесплатные двоичные файлы для Oracle JDK 17.

В этом приложении основное внимание будет сосредоточено на Java SE 8, 11 и 17.

A.2. ДЛЯ ТЕХ, КТО ОСТАЕТСЯ НА JAVA SE 8

Некоторые пользователи желают остаться на Java SE 8 по разным причинам:

1. После выхода обновления за апрель 2019 года на Oracle JDK 8 распространяются ограничения по коммерческому использованию. Чтобы получить обновленный двоичный дистрибутив Java SE 8, нужно либо оплатить план платной поддержки для Oracle JDK 8, либо использовать двоичный дистрибутив Java SE 8 или OpenJDK 8 от другого поставщика.
2. Если вы не используете Oracle JDK 8, ваш текущий поставщик Java SE 8 или OpenJDK 8 может предложить платные планы обновления или поддержки.

A.2.1. Бесплатное использование Java SE 8

Если вам нужны бесплатные обновления (в том числе обновления безопасности) для Java SE 8, используйте дистрибутив OpenJDK, который прошел тесты на соответствие спецификации (TCK), например решения от Amazon, Azul, BellSoft, Eclipse Adoptium, IBM, Microsoft, Red Hat или SAP.

A.3. КАК ПОЛУЧИТЬ JAVA SE 11

Чтобы получить Java SE 11, можно воспользоваться вариантами, перечисленными ниже. Пожалуйста, внимательно прочитайте условия, особенно о том, как Oracle JDK управляет выпусками и обновлениями Java SE 11:

1. Для Java SE 11 Oracle предоставляет свои JDK (на базе OpenJDK) на следующих условиях:
   1) сборки Oracle OpenJDK 11 — на условиях лицензии GPLv2+CE;
   2) Oracle JDK — на условиях платной коммерческой лицензии (но бесплатно для личного пользования, разработки, тестирования, прототипирования и демонстрации, а также для некоторых видов приложений) — для тех, кому не подходит GPLv2+CE или кто использует Oracle JDK с продуктами или службами Oracle.
2. Также доступны двоичные дистрибутивы Java SE и OpenJDK от многих других поставщиков. Они предоставляют обновления (в том числе обновления

https://t.me/javalib

безопасности) на разные периоды, но обычно более продолжительные — для LTS-версий.

А.3.1. Бесплатное использование Java SE 11

Если вам нужны бесплатные обновления (в том числе обновления безопасности) для Java SE 11, используйте дистрибутив OpenJDK, который прошел тесты на соответствие спецификации (TCK), например решения от Amazon, Azul, BellSoft, Eclipse Adoptium, IBM, Microsoft, Red Hat или SAP.

А.4. КАК ПОЛУЧИТЬ JAVA SE 17 (LTS)

Чтобы получить Java SE 11, в вашем распоряжении есть варианты, перечисленные ниже. Пожалуйста, внимательно прочитайте условия, особенно о том, как Oracle JDK управляет выпусками и обновлениями Java SE 17:

1. Начиная с Java SE 17, Oracle предоставляет свои JDK (на базе OpenJDK) на следующих условиях:
   1) сборки Oracle OpenJDK — по лицензии GPLv2+CE;
   2) Oracle JDK — по лицензии NFTC (No-Fee Terms and Conditions) на три года, с обычной коммерческой лицензией по истечении этого периода.
2. Также доступны двоичные дистрибутивы Java SE и OpenJDK от многих других поставщиков. Они предоставляют обновления (в том числе обновления безопасности) на разные периоды, но обычно более продолжительные — для LTS-версий.

ПРИМЕЧАНИЕ Лицензия NTFC устанавливает ряд ограничений на свободное вторичное распространение Oracle JDK 17. Обязательно ознакомьтесь с подробностями в тексте лицензии.

А.4.1. Бесплатное использование Java SE 17

Если вам нужны бесплатные обновления (в том числе обновления безопасности) для Java SE 17, используйте дистрибутив OpenJDK, который прошел тесты на соответствие спецификации (TCK), например решения от Amazon, Azul, BellSoft, Eclipse Adoptium, IBM, Microsoft, Red Hat или SAP.

А.5. ПЛАТНАЯ ПОДДЕРЖКА

Существует широкий диапазон вариантов платной поддержки для двоичных дистрибутивов Java SE и OpenJDK 8, 11 и 17 от Azul, BellSoft, IBM, Oracle, Red Hat и т. д. Azul также предоставляет версии со среднесрочной поддержкой.

https://t.me/javalib

Б Потоки данных в Java 8

Это приложение кратко напомнит вам о потоках данных в Java 8 и связанных с ними аспектах базового функционального программирования. Если вы не знакомы с синтаксисом лямбда-выражений в Java 8 или с философией, на которую опирается их архитектура, вам стоит начать с вводной книги, например: Рауль-Габриэль Урма, Марио Фуско, Алан Майкрофт (Raoul-Gabriel Urma, Mario Fusco, and Alan Mycroft) «Modern Java in Action: Lambdas, Streams, Functional and Reactive Programming»[1]. В Java 8 лямбда-выражения появились как часть проекта Lambda, общие цели которого можно охарактеризовать так:

- Позволить разработчикам писать более ясный и компактный код.
- Обновить библиотеки коллекций Java в духе современности.
- Ввести абстракцию, которая позволяла бы удобно использовать базовые идиомы функционального программирования.

В этом приложении рассматриваются обновления в библиотеках коллекций, методы по умолчанию и абстракция потока данных (`Stream`) как тип функционального контейнера для элементов данных.

Б.1. ОБРАТНАЯ СОВМЕСТИМОСТЬ

Одно из самых важных качеств платформы Java — обратная совместимость. Создатели Java всегда руководствовались принципом, что код, который был написан или откомпилирован для более ранней версии платформы, должен

[1] Урма Р., Фуско М., Майкрофт А. «Современный язык Java. Лямбда-выражения, потоки и функциональное программирование». СПб., издательство «Питер».

https://t.me/javalib

продолжать работать на более поздних ее выпусках. Это позволяет разработчикам сохранять уверенность в том, что обновление их платформы Java не повредит приложениям, которые работают в настоящее время.

К сожалению, обратная совместимость ограничивает возможные направления эволюции платформы, и эти ограничения влияют на разработчиков.

Например, из соображений обратной совместимости платформа Java не может добавлять новые методы в существующие интерфейсы JDK. Чтобы понять, почему это так, рассмотрим следующую ситуацию: если новая версия некоторого интерфейса `IFoo` добавит новый метод `newWithPlatformReleaseN()` в выпуске *N* платформы Java, то этого метода не будет во всех предшествующих реализациях `IFoo`, которые были откомпилированы на платформе версии $N-1$ (и ниже), из-за чего старые реализации `IFoo` не будут компоноваться с платформой Java версии *N*.

Это ограничение создавало серьезную проблему для реализации лямбда-выражений в JDK 8, потому что главная цель проектирования заключалась в том, чтобы стандартные структуры данных JDK можно было обновить для реализации идиом из функциональной школы программирования. Разработчики намеревались внедрить новые методы, которые с помощью лямбда-выражений выражают функциональные идеи (такие, как `map()` и `filter()`), в библиотеки коллекций Java.

Б.2. МЕТОДЫ ПО УМОЛЧАНИЮ

Чтобы решить эту проблему, появился совершенно новый механизм *методов по умолчанию*, который позволяет обновлять интерфейсы в новых выпусках платформы Java.

> **ПРИМЕЧАНИЕ** Начиная с Java 8, метод по умолчанию (он же необязательный метод) можно добавлять в любой интерфейс. В него должна входить реализация, которая называется *реализацией по умолчанию* и записывается во встроенном виде в определении интерфейса. Такое изменение определения интерфейса носит эволюционный характер и не нарушает обратную совместимость.

Методы по умолчанию должны подчиняться основным правилам:

- Любая реализация интерфейса может (но не обязана) реализовать метод по умолчанию.
- Если реализующий класс реализует метод по умолчанию, используется реализация из этого класса.
- Если реализующий класс не реализует метод по умолчанию, используется реализация по умолчанию (из определения интерфейса).

Рассмотрим небольшой пример. Одним из методов по умолчанию, которые были добавлены в интерфейс `List` в JDK 8, был метод `sort()`. Его определение выглядит так:

https://t.me/javalib

```
public default void sort(Comparator<? super E> c) {
    Collections.<E>sort(this, c);
}
```

Это означает, что у любого объекта `List` есть метод экземпляра `sort()`, с помощью которого список можно сортировать на месте, используя подходящий объект `Comparator`. Любая реализация `List` может предоставить собственное переопределение поведения `sort()`, но если она этого не сделает, то будет доступна реализация по умолчанию, предоставленная во вспомогательном классе `Collections`.

Механизм методов по умолчанию основан на загрузке классов. Когда загружается реализация интерфейса, JVM проверяет, есть ли в файле класса все необязательные методы. Если они есть, то загрузка класса продолжается как обычно. Если нет, то JVM модифицирует байт-код реализации, добавляя в него реализации по умолчанию для отсутствующих методов.

> **ПРИМЕЧАНИЕ** Методы по умолчанию отражают фундаментальное изменение в подходе Java к объектно-ориентированному программированию. Начиная с Java 8, интерфейсы могут содержать код реализации. Многие разработчики воспринимают это как ослабление жестких правил одиночного наследования Java.

Разработчики должны понимать один важный фактор: если использовать методы по умолчанию, то возможны конфликты реализаций по умолчанию, причем эти конфликты могут проявляться двумя способами. Во-первых, если в реализующем классе уже есть метод с таким же именем и сигнатурой, как у нового метода по умолчанию, то вместо реализации по умолчанию всегда будет использоваться реализация, которая существовала ранее.

Во-вторых, если класс реализует два интерфейса, каждый из которых содержит метод по умолчанию с одинаковым именем и сигнатурой, то класс должен реализовать этот метод. (При этом можно выбрать, делегировать ли управление одной из реализаций по умолчанию или сделать что-то совершенно иное.) В этом случае возникает вероятность того, что добавление метода по умолчанию в интерфейс нарушит работоспособность клиентского кода, потому что если клиентский код уже реализует другой интерфейс с методом по умолчанию, появляется риск конфликта реализаций. Однако на практике такие ситуации встречаются очень редко, и считается, что этот риск — невысокая цена за многие преимущества, присущие методам по умолчанию.

Б.3. ПОТОКИ ДАННЫХ

Вспомните, что одной из целей проекта Lambda было сделать так, чтобы в языке Java было удобно выражать приемы функционального программирования. Например, благодаря этому в Java появились простые способы записи идиом `map()` и `filter()`.

https://t.me/javalib

Чтобы реализовать эти идиомы, в исходном эскизном проекте Java 8 эти методы добавлялись непосредственно в классические интерфейсы коллекций Java как дополнительные методы по умолчанию. Однако такой подход оказался неудовлетворительным по нескольким причинам.

Прежде всего, поскольку имена `map()` и `filter()` встречаются достаточно часто, был слишком высок риск конфликтов с существующими реализациями: во многих пользовательских реализациях коллекций есть методы с такими же именами, которые не соблюдают предполагаемую семантику новых методов.

Вместо этого появилась новая абстракция потока данных — `Stream`. Этот интерфейс представляет собой тип контейнера, который в каком-то смысле близок к итератору и позволяет работать с коллекциями и агрегированием данных в более функциональном стиле.

В `Stream` были включены все новые функционально-ориентированные методы, такие как `map()`, `filter()`, `reduce()`, `forEach()` и `flatMap()`. В методах `Stream` широко применяются функциональные интерфейсы, например, лямбда-выражения в качестве аргументов.

`Stream` лучше всего рассматривать как последовательность потребляемых элементов. Это означает, что после того, как элемент извлечен из `Stream`, он становится недоступным — практически так же, как в случае с `Iterator`.

> **ПРИМЕЧАНИЕ** Поскольку объекты Stream потребляются, их не следует использовать повторно или сохранять во временных переменных. Присваивание значения Stream локальной переменной — это почти всегда верный признак «кода с душком».

В традиционные классы коллекций, такие как `List` и `Set`, добавился новый метод по умолчанию `stream()`. Он возвращает объект `Stream` для коллекции по аналогии с тем, как метод `iterator()` использовался в коде с классическими коллекциями.

Б.3.1. Пример

Следующий фрагмент показывает, как с помощью объекта `Stream` и лямбда-выражения можно реализовать идиому фильтра:

```
List<String> myStrings = getSomeStrings();
String search = getSearchString();

System.out.println(myStrings.stream()
                            .filter(s -> s.equals(search))
                            .collect(Collectors.toList()));
```

Обратите внимание, что нам приходится также вызвать `collect()`: так происходит потому, что `filter()` возвращает другой объект `Stream`. Чтобы после фильтрации получился снова тип коллекции, нужно каким-то образом специально преобразовать `Stream` в `Collection`.

https://t.me/javalib

Общая схема выглядит примерно так:

```
       stream()  filter()   map()  collect()
Collection → Stream → Stream → Stream → Collection
```

Основная идея — построить «конвейер» операций, которые должны применяться к потоку данных. Фактическое содержимое каждой операции будет задаваться лямбда-выражением. В конце конвейера результаты нужно материализовать обратно в коллекцию, поэтому используется метод `collect()`.

Рассмотрим часть определения интерфейса `Stream`, в которой определяются методы `map()` и `filter()`):

```
public interface Stream<T> extends BaseStream<T, Stream<T>> {
    Stream<T> filter(Predicate<? super T> predicate);

    <R> Stream<R> map(Function<? super T, ? extends R> mapper);

    // ...
}
```

> **ПРИМЕЧАНИЕ** Не пугайтесь зловещих обобщений в этих определениях. Все части «? super» и «? extends» означают следующее: «Выполнить правильную операцию, если у объектов в потоке данных есть подклассы».

В этих определениях фигурируют два новых интерфейса: `Predicate` и `Function`, которые находятся в пакете `java.util.function`. Оба интерфейса содержат только один метод, у которого нет реализации по умолчанию. Поэтому для них можно написать лямбда-выражение, которое автоматически преобразуется в экземпляр нужного типа.

> **ПРИМЕЧАНИЕ** Вспомните, что когда платформа Java обнаруживает лямбда-выражение, она всегда выполняет преобразование в нужный тип функционального интерфейса (через автоматическое выведение типа).

Рассмотрим пример кода. Допустим, вы моделируете популяции выдр, одни из которых дикие, а другие живут в зоопарках. Вы хотите узнать, сколько выдр, содержащихся в неволе, находятся под присмотром смотрителей-стажеров. С лямбда-выражениями и потоками данных это делается очень легко:

```
Set<Otter> ots = getOtters();
System.out.println(ots.stream()
    .filter(o -> !o.isWild())
    .map(o -> o.getKeeper())
    .filter(k -> k.isTrainee())
    .collect(Collectors.toList())
    .size());
```

Сначала мы фильтруем поток данных, чтобы обрабатывались только выдры, живущие в зоопарках. Затем мы выполняем операцию `map()`, чтобы получить поток

https://t.me/javalib

данных смотрителей вместо потока данных выдр (обратите внимание, что тип потока меняется со `Stream<Otter>` на `Stream<Keeper>`). Затем мы снова фильтруем поток, чтобы выбрать только смотрителей-стажеров, после чего материализуем результат в конкретный экземпляр коллекции с помощью метода `Collectors.toList()`. Наконец, чтобы вернуть количество элементов полученного списка, мы используем уже знакомый метод `size()`.

В этом примере выдры преобразуются в смотрителей, которые отвечают за их содержание. При этом состояние ни одной выдры не изменяется, то есть отсутствуют побочные эффекты.

> **ПРИМЕЧАНИЕ** По соглашениям языка Java код в выражениях `map()` и `filter()` не должен иметь побочных эффектов. Тем не менее среда выполнения Java не контролирует, выполняется ли это правило, так что будьте осторожны. Вам всегда следует соблюдать эти соглашения в своем коде.

Если в нашем коде нужно изменить какое-нибудь внешнее состояние, можно использовать один из двух подходов в зависимости от того, чего вы хотите добиться. Если вы хотите сформировать агрегированное состояние (например, промежуточную сумму возрастов выдр), можно вызвать `reduce()`. А если вы хотите выполнить более общее преобразование состояния (например, передать выдр новому смотрителю, когда старый увольняется), лучше подойдет цикл `forEach()`.

Посмотрим, как можно было бы вычислить средний возраст выдр с помощью метода `reduce()`:

```
var kate = new Keeper();
var bob = new Keeper();
var splash = new Otter();
splash.incAge();
splash.setKeeper(kate);
Set<Otter> ots = Set.of(splash);

double aveAge = ((double) ots.stream()
    .map(o -> o.getAge())
    .reduce(0, (x, y) -> {return x + y;} )) / ots.size();
System.out.println("Средний возраст: "+ aveAge);
```

Прежде всего мы отображаем выдр на их возраст, а затем используем метод `reduce()`. Он получает два аргумента: исходное значение (часто называемое *нулем*) и функцию, которая должна применяться пошагово. В нашем примере это простое сложение, потому что мы хотим просуммировать возрасты всех выдр. Наконец, общий возраст делится на количество выдр.

Обратите внимание, что второй аргумент `reduce()` — лямбда-выражение с двумя аргументами. Проще всего понимать это так, что первый из двух аргументов содержит промежуточную сумму операции агрегирования, а второй — переменную цикла, который перебирает элементы коллекции.

https://t.me/javalib

Наконец, обратимся к случаю, когда нужно изменить состояние. Для этого мы применим операцию `forEach()`. В нашем примере смотритель Кейт уходит в отпуск, поэтому всех ее выдр необходимо передать Бобу. Задача легко решается таким кодом:

```
ots.stream()
.filter(o -> !o.isWild())
.filter(o -> o.getKeeper().equals(kate))
.forEach(o -> o.setKeeper(bob));
```

Следует заметить, что ни `reduce()`, ни `forEach()` не используют `collect()`. `reduce()` накапливает состояние при переборе потока данных, а `forEach()` просто применяет действие к каждому элементу потока, так что в обоих случаях нет необходимости материализовать поток заново.

Б.4. ОГРАНИЧЕНИЯ КОЛЛЕКЦИЙ

Коллекции Java отлично послужили языку. Однако они опираются на идею о том, что все элементы коллекции существуют и представляются где-то в памяти. Это означает, что они не позволяют представлять более общие данные, например бесконечные множества.

Для примера рассмотрим множество всех простых чисел. Его нельзя смоделировать в виде `Set<Integer>`, потому что мы не знаем всех простых чисел, да и памяти в куче, безусловно, не хватит для того, чтобы разместить их все. В ранних версиях Java эту задачу было бы очень трудно решить с помощью стандартных коллекций.

Можно сконструировать представление данных, которое работает преимущественно с итераторами и переводит нижележащие коллекции на вспомогательную роль. Однако эта практика требует высокой дисциплины и не относится к очевидным сценариям работы с коллекциями Java. В прошлом, если разработчик хотел использовать такой подход, ему обычно приходилось полагаться на внешнюю библиотеку, которая поддерживала эту функциональность более качественно.

К счастью, потоки данных Java решают эту проблему благодаря тому, что интерфейс `Stream` позволяет эффективнее работать с более общими структурами данных, чем базовые конечные коллекции. А значит, `Stream` можно рассматривать как более общую структуру, чем `Iterator` или `Collection`.

> **ПРИМЕЧАНИЕ** `Stream` не управляет хранением элементов в памяти и не дает возможности обращаться к отдельным элементам прямо из потока.

Тем не менее `Stream` — это на самом деле не структура данных, а абстракция для работы с данными, хотя различия между этими двумя понятиями достаточно тонкие.

https://t.me/javalib

Б.5. БЕСКОНЕЧНЫЕ ПОТОКИ ДАННЫХ

Давайте чуть глубже разберемся в том, как моделировать бесконечную последовательность чисел. Из ее определения вытекает ряд следствий:

- Весь поток данных невозможно материализовать в коллекцию, что не позволяет применять такие методы, как `collect()`.
- Чтобы работать с потоком данных, нам понадобится извлекать из него элементы.
- Нам понадобится код, который возвращает следующий элемент в тот момент, когда он будет нужен.

Такой подход также означает, что значения выражений не будут вычисляться, пока в них не возникнет необходимость.

До выхода Java 8 значение выражения всегда вычислялось в тот момент, когда оно связывалось с переменной или передавалось функции. Это поведение называется *неотложным вычислением* и используется по умолчанию в большинстве массовых языков программирования.

В Java 8 появилась новая парадигма программирования: там, где это возможно, Stream использует *отложенные вычисления*. Эта новая функциональность — чрезвычайно мощная, и к ней привыкаешь не сразу. Отложенные вычисления подробнее рассматривались в главе 15. Лямбда-выражения были включены в Java, чтобы упростить жизнь обычных программистов, хотя это потребовало дополнительных усложнений в устройстве платформы.

Б.6. РАБОТА С ПРИМИТИВАМИ

С API потоков данных связан один важный вопрос, на котором мы до сих пор не останавливались: как работать с примитивными типами? Обобщения Java не позволяют использовать примитивный тип в качестве параметра-типа, поэтому нельзя написать `Stream<int>`. К счастью, в пакете `Stream` есть средства, которые помогают обойти эту проблему. Рассмотрим пример:

```
double totalAge = ((double) ots.stream()
                            .map(o -> o.getAge())
                            .reduce(0, (x, y) -> {return x + y;} ));

double aveAge = totalAge / ots.size();
System.out.println("Средний возраст: "+ aveAge);
```

Здесь на большинстве стадий конвейера на самом деле используются примитивные типы. Проанализируем код более подробно, чтобы разобраться, как это делается.

Прежде всего, пусть вас не смущает приведение к `double`. Оно нужно только для того, чтобы операция деления правильно вычисляла среднее значение вместо целочисленного деления.

https://t.me/javalib

В аргументе `map()` передается лямбда-выражение, которое принимает `Otter` и возвращает `int`. Если бы можно было использовать обобщения Java, то лямбда-выражение было бы преобразовано в объект, который реализует `Function<Otter, int>`. Но поскольку обобщения Java этого не позволяют, нужен другой способ закодировать тот факт, что возвращаемым типом является `int`. Мы сделаем так, чтобы этот тип был включен в имя типа, — тогда фактически выведенным типом станет `ToIntFunction<Otter>`. Этот тип называется *примитивной специализацией* типа функции и используется, чтобы предотвратить упаковку и распаковку между `int` и `Integer`, что позволяет не генерировать лишние объекты и использовать типы функций, специфические для текущего примитивного типа.

Разобьем вычисление среднего на меньшие части. Возраст каждой выдры мы получаем с помощью такого выражения:

```
ots.stream().map(o -> o.getAge())
```

А теперь взгляните на определение вызываемого метода `map()`:

```
IntStream map(ToIntFunction<? super T> mapper);
```

Отсюда видно, что мы используем специальный тип функции `ToIntFunction`, а чтобы представить поток данных `int`, также применяется специализированная форма `Stream`.

Результат этой операции передается методу `reduce()`, который определяется так:

```
int reduce(int identity, IntBinaryOperator op);
```

Это тоже специализированная форма, которая работает исключительно с `int` и принимает лямбда-выражение с двумя аргументами (оба аргумента имеют тип `int`), над которыми выполняется свертка.

`reduce()` является накапливающей операцией (а следовательно, она вычисляется в неотложном режиме), поэтому конвейер вычисляется в этой точке и возвращает одно значение, которое затем приводится к `double` и преобразуется в общее среднее.

Если вы упустили все эти подробности о примитивах, не огорчайтесь: одно из достоинств автоматического выведения типов заключается в том, что многие отличия почти постоянно остаются скрытыми от разработчика.

Напоследок скажем несколько слов на тему, которую многие разработчики понимают неправильно: параллельные операции с потоками данных.

Б.7. ПАРАЛЛЕЛЬНЫЕ ОПЕРАЦИИ?

В старых версиях Java (7 и ранее) все операции с коллекциями выполнялись последовательно. Независимо от размера коллекции, для операции использовалось только одно ядро процессора. Когда наборы данных растут, такой подход становится слишком расточительным, и одной из потенциальных целей проекта

https://t.me/javalib

Lambda было обновить поддержку коллекций, чтобы эффективно использовать многоядерные процессоры.

> **ПРИМЕЧАНИЕ** Отложенные вычисления для потоков данных позволяют фреймворку лямбда-выражений предоставить поддержку параллельных операций.

Основное предположение в Stream API заключается в том, что создание объекта потока (на основе коллекции или другими способами) должно быть низкозатратным, однако некоторые операции в конвейере могут обходиться дорого. С этим предположением параллельный конвейер можно описать так:

```
s.stream()
    .parallel()
    // последовательность операций с потоком данных
    .collect( ... );
```

Метод `parallel()` преобразует последовательный поток данных в параллельные операции. Это позволяет обычным разработчикам полагаться на то, что `parallel()` предоставит точку входа в прозрачный параллелизм, а поддержка этого параллелизма — забота авторов библиотеки, а не конечных пользователей.

Теоретически это звучит превосходно, но на практике реализация и другие нюансы делают механизм `parallel()` гораздо менее полезным. В главе 16 эта тема рассматривалась подробнее.

Из-за этих ограничений настоятельно рекомендуется избегать параллельных потоков данных, если только вы не можете доказать (методами из главы 7), что ваше приложение выиграет, если их задействовать. На практике авторам встретилось не более 4–5 случаев, когда параллельные потоки данных оказывались действительно эффективными.

https://t.me/javalib

Бенджамин Эванс, Джейсон Кларк, Мартин Фербург

Java для опытных разработчиков

2-е издание

https://t.me/javalib

https://t.me/javalib